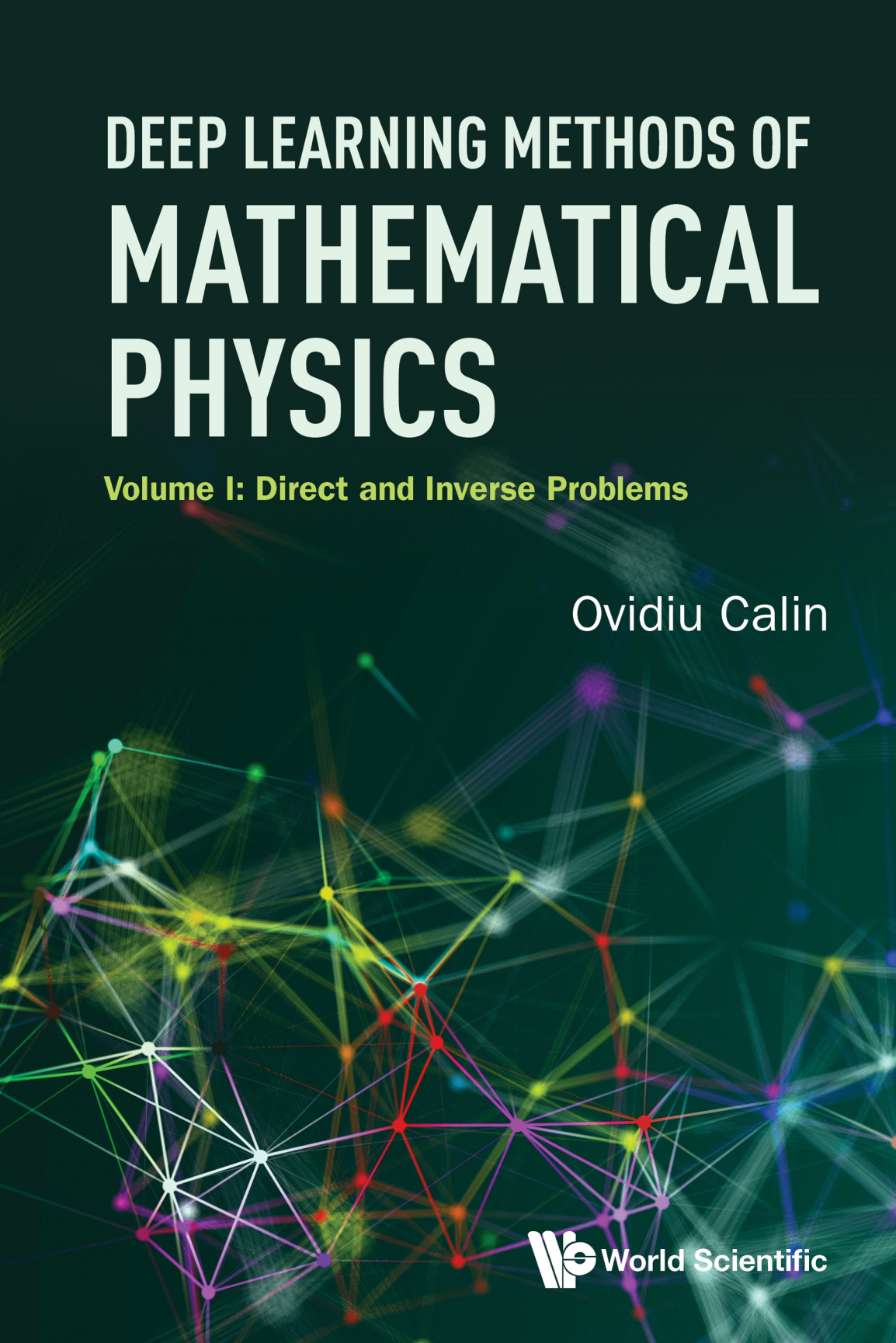
DEEP LEARNING METHODS OF MATHEMATICAL PHYSICS

Volume I: Direct and Inverse Problems

Ovidiu Calin



World Scientific

DEEP LEARNING METHODS OF MATHEMATICAL PHYSICS

Volume I: Direct and Inverse Problems

Other World Scientific Titles by the Author

An Informal Introduction to Stochastic Calculus with Applications
ISBN: 978-981-4678-93-3
ISBN: 978-981-4689-91-5 (pbk)

An Informal Introduction to Stochastic Calculus with Applications
Second Edition
ISBN: 978-981-12-4709-5
ISBN: 978-981-12-4756-9 (pbk)

Deterministic and Stochastic Topics in Computational Finance
ISBN: 978-981-320-307-5
ISBN: 978-981-320-308-2 (pbk)

Stochastic Geometric Analysis with Applications
ISBN: 978-981-12-8327-7

Computational Formalisms in Euclidean Geometry
Volume I: A Trigonometric, Vectorial and Complex Numbers Approach
ISBN: 978-981-98-0616-4
ISBN: 978-981-98-0713-0 (pbk)

DEEP LEARNING METHODS OF MATHEMATICAL PHYSICS

Volume I: Direct and Inverse Problems

Ovidiu Calin
Eastern Michigan University, USA

World Scientific

NEW JERSEY • LONDON • SINGAPORE • BEIJING • SHANGHAI • HONG KONG • TAIPEI • CHENNAI • TOKYO

Published by

World Scientific Publishing Co. Pte. Ltd.

5 Toh Tuck Link, Singapore 596224

USA office: 27 Warren Street, Suite 401-402, Hackensack, NJ 07601

UK office: 57 Shelton Street, Covent Garden, London WC2H 9HE

British Library Cataloguing-in-Publication Data
A catalogue record for this book is available from the British Library.

DEEP LEARNING METHODS OF MATHEMATICAL PHYSICS
Volume I: Direct and Inverse Problems

Copyright © 2026 by World Scientific Publishing Co. Pte. Ltd.

All rights reserved. This book, or parts thereof, may not be reproduced in any form or by any means, electronic or mechanical, including photocopying, recording or any information storage and retrieval system now known or to be invented, without written permission from the publisher.

For photocopying of material in this volume, please pay a copying fee through the Copyright Clearance Center, Inc., 222 Rosewood Drive, Danvers, MA 01923, USA. In this case permission to photocopy is not required from the publisher.

ISBN 978-981-98-2723-7 (hardcover)
ISBN 978-981-98-2792-3 (paperback)
ISBN 978-981-98-2724-4 (ebook for institutions)
ISBN 978-981-98-2725-1 (ebook for individuals)

For any available supplementary material, please visit
https://www.worldscientific.com/worldscibooks/10.1142/14702#t=suppl

Desk Editor: Tan Rok Ting

Printed in Singapore

Foreword

The rapid progress of Artificial Intelligence (AI) and Deep Learning (DL) is reshaping many scientific disciplines, including *Mathematical Physics*–a field fundamentally governed by differential equations. Traditionally, problems in Mathematical Physics have been addressed through analytical techniques and numerical solvers that integrate the governing equations. As systems become more complex, nonlinear, or chaotic, however, classical methods can struggle to capture full dynamics or to infer governing laws directly from data. Deep Learning complements these approaches by providing flexible, data-driven surrogates and physics-informed formulations that scale to high complexity while respecting the underlying differential structure.

The year 2024 is particularly significant because it coincides with the awarding of the Nobel Prizes in Physics [90] and Chemistry [89] to research that has made pioneering use of Deep Learning methods to tackle problems that had resisted solution for decades. These major achievements show that Deep Learning is no longer just a computer tool. It has become a key part of modern science, helping researchers discover things that were once impossible to uncover.

This book explores the emerging role of Deep Learning as a computational tool in Mathematical Physics. Neural architectures such as Neural Ordinary Differential Equations (Neural ODEs), Physics-Informed Neural Networks (PINNs), Hamiltonian Neural Networks (HNNs), and Lagrangian Neural Networks (LNNs) allow the direct encoding of physical principles into learning models.

Deep Learning methods not only address direct problems–traditionally solved using analytical techniques or numerical simulations–but also open new avenues for solving inverse problems. They enable the inference of system parameters, conservation laws, and even the underlying governing equations directly from experimental or simulated data.

Deep Learning offers unique advantages for classical mechanics problems. Architectures designed with physical insights can enforce conservation laws, handle complex boundary conditions, and generalize to unseen scenarios. This integration of Physics and machine learning not only improves predictive capabilities but also opens avenues for discovering new physical models directly from data.

This book emphasizes both the practical implementation and the theoretical underpinnings of these methods. Through numerous examples and comparisons, the text illustrates how Deep Learning models compare to, complement or surpass traditional solvers in certain regimes.

Many examples in this book include Keras implementations, either embedded in the text or linked to online Google Colab notebooks. These examples make strong class projects: readers can modify inputs and hyperparameters to observe how the outputs change. In some cases, the exposition is intentionally kept at a theoretical level, inviting the reader to implement the corresponding code. Keras is used throughout because it is widely adopted, well documented, and well supported within the TensorFlow ecosystem. Each chapter concludes with end-of-chapter exercises, several of which are suitable as course projects and reinforce the methods introduced in that chapter.

This book is neither a primer on machine learning nor a physics textbook. It is intended for readers already comfortable with Deep Learning who want to see how modern architectures can be applied to problems in Mathematical Physics. The material is organized as a collection of worked examples—ranging from simple to challenging—that illustrate when Deep Learning helps, how to set it up, and what its practical limits are.

The book targets senior undergraduates but is also suitable for graduate students, researchers, and practitioners in Physics, Applied Mathematics, Engineering, and Computer Science. The ideal reader is familiar with basic Classical Mechanics, differential equations, and introductory machine learning concepts. Prior experience with Python, TensorFlow, or Keras is helpful but not required; the book provides ample examples and guidance to support implementation.

Overview

The book is organized into three main parts: *The first part* is the most elementary and it is an introduction to neural networks and its applications to mathematics problems. *The second part* focuses on using Deep Learning models to solve forward problems, including classical benchmarks such as the harmonic oscillator, planetary motion, and the simple pendulum. Performance is compared with classical numerical methods. *The third part* deals with inverse problems, exploring data-driven discovery of system parameters, physical laws, and conservation laws.

Part I

This part contains two chapters that lay the groundwork for the material developed in the chapters that follow.

Chapter 1 surveys feed-forward neural networks, with an emphasis on their use in Physics. It introduces the mathematical structure of neural networks and explains why these structures are relevant for physical modeling. The chapter serves as an introduction, helping the reader become familiar with the notation and ideas developed in the subsequent chapters.

Chapter 2 applies Deep Learning to a broad spectrum of equations—algebraic, functional, integral, ordinary differential, and partial differential. It also presents more advanced topics, including multiphase resolution strategies, eigenvalue problems, and the method of characteristics implemented with neural networks. The central message

is that neural networks can unify and often replace a wide "zoo" of traditional solvers across diverse equation types, highlighting their broad applicability.

Part II

This part focuses on solving *direct (forward)* problems with Deep Learning–specifically, predicting the motion of dynamical systems using neural networks.

Chapter 3 applies neural networks to classical mechanics. It covers learning trajectories of dynamical systems, reconstructing velocity fields, and several practical case studies: the mass-spring system, Newton's equation of motion, free fall from high altitude, and the two-body problem.

Chapter 4 treats Lagrangian systems as dynamical systems described by the Lagrangian formalism. Case studies include the harmonic oscillator, the catenary, the brachistochrone, the simple pendulum, and Euler's elastica.

Chapter 5 focuses on Hamiltonian systems as dynamical systems described by the Hamiltonian formalism. Case studies include the harmonic oscillator, the simple pendulum, and charged particle motion in a magnetic field, along with special topics such as generating functions and canonical transformations.

Chapter 6 is dedicated to conservation laws, emphasizing their role as regularizers for machine-learning models applied to dynamical systems. We present three implementation strategies: additive soft constraints, hard architectural embeddings, and integral constraints. The chapter also discusses how to identify conservation laws via functional invariance and Noether's theorem, deriving them from both Lagrangian and Hamiltonian formulations.

Chapter 7 treats the Hamilton–Jacobi equation from a neural-network perspective. In addition to the HJ equation, we solve the eiconal equation using neural networks and demonstrate applications to computing Riemannian distances.

Chapter 8 demonstrates how neural networks can be used to solve boundary value problems, including several classical examples such as the Stefan problem, the Muskat problem, and the punctured membrane problem. In many of these cases, analytic formulas for the free boundary are unavailable, making neural-network approaches especially valuable for practical applications.

Chapter 9 employs neural networks to solve classical PDEs in Mathematical Physics: elliptic problems (Poisson and Dirichlet boundary-value problems), parabolic problems (the Cauchy problem for the heat equation, the Schrödinger equation, and mixed initial–boundary value problems), and hyperbolic problems (the Cauchy problem for the wave equation in one, two, and three dimensions, as well as mixed boundary-value formulations, with applications to determining electromagnetic fields). These PDEs admit classical solutions from the standard theory, which we use as benchmarks for validation.

Part III

This part focuses on *inverse problems*, which are often challenging for classical methods because they can be *ill-posed*—lacking a solution, admitting multiple solutions, or being unstable with respect to data perturbations. In such settings, neural networks act as powerful regularizers, effectively selecting a *pseudosolution* that best fits the data and imposed physics, and that lies closest—under the chosen criteria—to the true solution.

Chapter 10 introduces inverse problems in mechanics. It covers learning a velocity field from data and recovering a force field from position measurements. Implementations are provided.

Chapter 11 develops conservation laws directly from data by identifying consistent patterns and invariants without relying on explicit dynamical models. It serves as a complement to Chapter 6, where conservation laws were derived from the underlying dynamical structure.

Chapter 12 addresses the inverse problem of parameter estimation given data and a specified mathematical model. Examples include frequency estimation, pendulum length identification, thermal diffusivity recovery, and parameter estimation for the Lotka–Volterra predator–prey system.

Chapter 13 takes a more ambitious step toward equation discovery. It recovers time-dependent coefficients for known ODE models and, from data alone, identifies governing laws such as ODE model structures, Euler–Lagrange equations, Hamilton's equations, and heat kernels.

Inverse problems for classical PDEs–such as learning boundary or initial conditions for the Laplace, heat, or wave equations are addressed in **Chapter 14**.

Chapter 15 develops more sophisticated deep-learning architectures beyond simple feed-forward networks. Topics include residual networks, neural ODEs, Hamiltonian neural networks, Lagrangian neural networks, and Hamilton–Jacobi neural networks, as well as attention mechanisms applied to these models. We also analyze trajectory "hot spots" using attention, identifying high-attention regions as maximum-entropy points of the attention distribution.

Chapter 16 presents several notable applications of Deep Learning, including Abel's problem, the inverse Sturm–Liouville problem, computed tomography, and GPS–INS sensor fusion.

Code Repository and Colab Access

Interactive **Google Colab** notebooks are provided so readers can run and modify the code without local setup, fostering exploration and experimentation. Colab links are included in each implementation section. Readers open a view-only notebook, create their own copy, and then modify it.

To support hands-on learning, all computational experiments, examples, and full implementations are available in a dedicated **GitHub repository** at
`https://github.com/ocalin911/Deep-Learning-Models-of-Mathematical-Physics`.

Acknowledgments

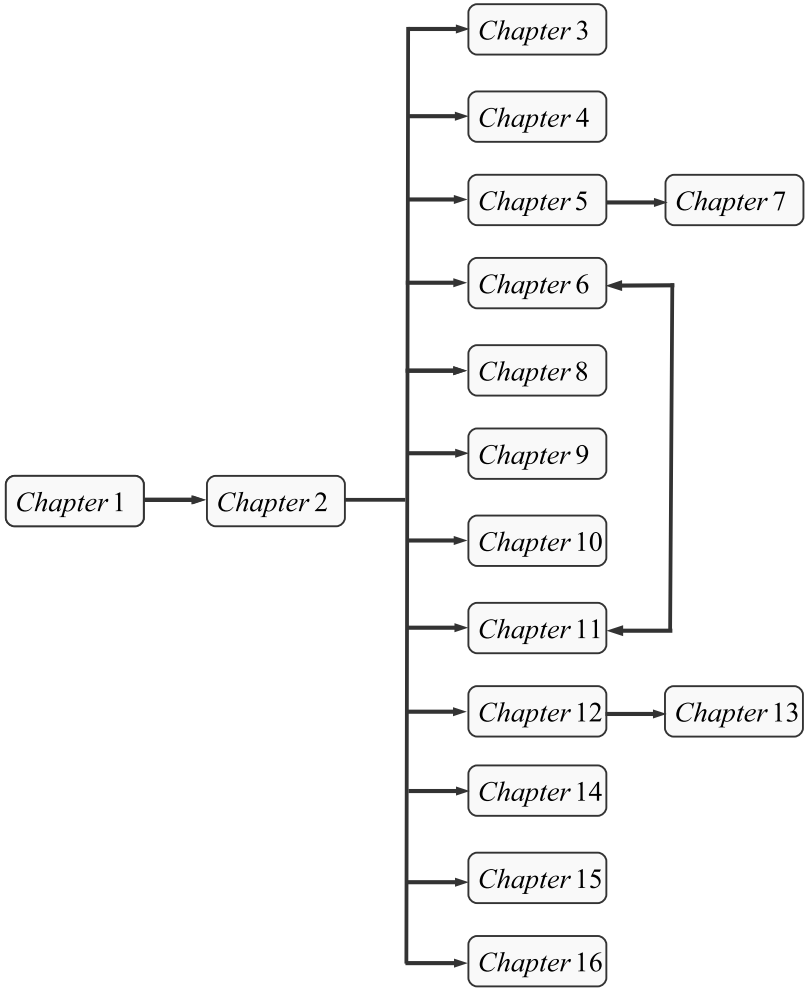
The material in this book originated in a graduate course, *Artificial Intelligence in Physics*, taught in the Physics Department at the University of Bucharest during the 2024–2025 academic year, while the author was a **Fulbright Fellow** in Romania. The course provided valuable feedback and insights that shaped the content and structure of this book.

In addition to Fulbright support, the author gratefully acknowledges the **sabbatical leave** granted by Eastern Michigan University for 2025–2026, which provided the dedicated time needed to expand the lectures and computational experiments into the present book.

Sincere thanks are also due to the editors at World Scientific for their careful and constructive work.

O. Calin

Ypsilanti, Michigan
November 2025


Chapter 1
Chapter 2
Chapter 3
Chapter 4
Chapter 5
Chapter 7
Chapter 6
Chapter 8
Chapter 9
Chapter 10
Chapter 11
Chapter 12
Chapter 13
Chapter 14
Chapter 15
Chapter 16

List of Notations and Symbols

$f_\theta(x)$	Output of a neural network with parameters θ and input x.
$NN(\theta; x)$	Output of a neural network with parameters θ and input x.
θ, w, b	Trainable parameters (weights and biases) of the neural network.
x	Input vector or data point.
$\mathcal{L}(\theta)$	Loss (or cost) function used for training the neural network.
$\nabla_\theta \mathcal{L}$	Gradient of the loss function with respect to parameters θ.
t	Time variable in time-dependent problems.
$\dot{x}, x'$	Time derivative of x, i.e., $\frac{dx}{dt}$.
$H(q, p)$	Hamiltonian function of a system.
$L(x, \dot{x})$	Lagrangian function of a dynamical system.
$u(t)$	Elongation (displacement) of a string at time t.
$\mathcal{D}$	Dataset containing input–output pairs used for training or testing.
$\overline{\mathcal{D}}$	The closure of $\mathcal{D}$.
Δf	Laplacian of a function f.
$\Box f$	d'Alembertian of a function f.
$\phi(\cdot)$	Activation function applied within neural network layers.
$a_{ij}(x_0)$	Attention matrix with anchor x_0.

This page intentionally left blank

Contents

Foreword v

List of Notations and Symbols xi

I Introduction to Neural Networks and Applications 1

1 Review of Feed-forward Neural Networks 3
- 1.0.1 Overview of FNNs . . . 3
- 1.0.2 Mathematical Structure of a Feed-forward Network . . . 4
- 1.0.3 Neuron Model and Activation Functions . . . 5
- 1.0.4 Assessing Network Quality . . . 7
- 1.0.5 Training Framework: Loss Minimization . . . 8
- 1.0.6 Backpropagation Algorithm and Gradient Flow . . . 10
- 1.0.7 Approximation Theory and Expressive Power . . . 12
- 1.0.8 Training versus Testing Data . . . 13
- 1.0.9 Generalization, Overfitting, and Regularization . . . 14
- 1.0.10 Why Deep Learning for Physics? . . . 16

2 Deep Learning for Solving Equations 19
- 2.1 Solving Polynomial Equations with NN . . . 19
  - 2.1.1 The Direct Method . . . 19
  - 2.1.2 The Inverse Function Method . . . 22
  - 2.1.3 Practical Implementations . . . 23
  - 2.1.4 Exercises . . . 30
- 2.2 Solving Functional Equations . . . 31
  - 2.2.1 The Cauchy Functional Equation . . . 32
  - 2.2.2 A Simple Multiplicative Functional Equation . . . 33
  - 2.2.3 Another Simple Functional Equation . . . 35
- 2.3 Practical Implementations . . . 36
  - 2.3.1 Solving Cauchy Functional Equation with NN . . . 36
  - 2.3.2 Solving $f(x+y) = f(x)\,f(y)$ with Neural Nets . . . 38
  - 2.3.3 Exercises . . . 41
- 2.4 Solving Integral Equations . . . 42
  - 2.4.1 Integral Equations of the First Kind . . . 42
  - 2.4.2 Integral Equations of the Second Kind . . . 44

2.4.3 Implementation of a Linear Integral Equation 45
2.4.4 Learning the Kernel via Neural Networks 47
2.4.5 Regularization Techniques for Kernel Identification 48
2.4.6 Exercises 50
2.5 Solving ODEs 52
2.5.1 First Order ODEs 52
2.5.2 Second Order ODEs 55
2.5.3 Practical Implementations 58
2.5.4 Newton's Cooling Law 65
2.5.5 Exercises 75
2.6 A Hybrid Method 76
2.6.1 Implementation 77
2.7 Equation Discovery from Data 79
2.8 Inverting Integrals 80
2.8.1 Practical Implementations 81
2.8.2 Exercises 82
2.9 Two-Phase Resolution Neural Network Solution 83
2.9.1 Three-Phase Resolution Neural Network Method 87
2.9.2 Exercises 89
2.10 Dirichlet Eigenvalue Problem 91
2.10.1 Practical Implementation 94
2.10.2 Exercises 103
2.11 The Method of Characteristics 106
2.12 Exercises 113

II Direct Problems: Predicting Motion with Deep Learning 117

3 Introduction to Classical Mechanics with Neural Nets 119
3.1 Learning Trajectories of Dynamical Systems 119
3.1.1 Neural Network Approximation of the Curve 119
3.1.2 Loss Functions 119
3.1.3 Total Loss and Optimization 121
3.1.4 Finding Integral Curves of a Vector Field 122
3.1.5 Particle Trajectory Reconstruction from Position and Velocity . 124
3.2 Reconstructing a Vector Field 125
3.3 Reconstructing the Flow 126
3.4 Practical Implementation 127
3.5 Damped Spring-Mass System 130
3.6 Practical Implementation 133
3.7 Newton's Equation 135
3.7.1 Coordinates Change 139
3.7.2 Galilean Transforms 140
3.7.3 Free-falling from High Altitude 142
3.7.4 Damped Spring-Mass System 147

3.7.5 The Two-Body Problem . . . 147
3.8 Exercises . . . 151

4 Lagrangian Systems **153**
4.1 The Harmonic Oscillator . . . 154
4.2 The Catenary Curve . . . 156
4.3 The Brachistochrone Problem . . . 159
4.4 The Simple Pendulum . . . 171
4.5 Elastica Curves . . . 175
4.6 Exercises . . . 183

5 Hamiltonian Systems **187**
5.1 Hamiltonian Functions . . . 187
5.2 Neural-network Approximation . . . 189
5.3 The Harmonic Oscillator . . . 190
5.4 The Simple Pendulum . . . 191
5.5 Charged Particles in a Constant Magnetic Field . . . 193
5.6 Canonical Transformations . . . 196
5.7 Generating Functions . . . 200
5.8 Exercises . . . 203

6 Conservation Laws **207**
6.1 Conservation Laws as Regularizers . . . 207
6.1.1 Soft-constraint Regulariser . . . 207
6.1.2 Hard Embedding . . . 213
6.1.3 Integral Constraint . . . 216
6.2 Finding Conservation Laws . . . 221
6.2.1 Functional Invariance . . . 221
6.2.2 Noether's Theorem . . . 224
6.3 Conservation in Phase Space . . . 229
6.4 Exercises . . . 231

7 Hamilton–Jacobi Theory **237**
7.1 Hamilton–Jacobi Equation . . . 237
7.2 Using Neural Networks . . . 242
7.2.1 Solving the Hamilton–Jacobi Equation with NN . . . 242
7.2.2 Solving for the Dynamics . . . 244
7.2.3 Practical Implementation . . . 245
7.3 The Eiconal Equation . . . 246
7.3.1 Practical Implementation . . . 249
7.4 Finding Riemannian Distance . . . 250
7.4.1 NN Implementation . . . 255
7.4.2 Finding the Euclidean Distance . . . 257
7.5 Exercises . . . 258

8 Free-boundary Value Problems **263**
8.1 The One-phase Stefan Problem . . . 263
8.2 The 1D Muskat Problem . . . 268
8.3 The Punctured Membrane Problem . . . 274
8.3.1 A Radial Symmetric Case . . . 282
8.4 Exercises . . . 286

9 Classical PDEs of Mathematical Physics **289**
9.1 Elliptic Equations . . . 290
9.1.1 Solving Poisson Equation with Dirichlet Condition . . . 290
9.1.2 Solving a Nonlinear Dirichlet Problem . . . 295
9.2 Exercises . . . 297
9.3 Parabolic Equations . . . 298
9.3.1 The Mixed Problem . . . 299
9.3.2 The Cauchy Problem . . . 302
9.3.3 Schrödinger's Equation . . . 304
9.4 Exercises . . . 307
9.5 Hyperbolic Equations . . . 310
9.5.1 The Cauchy Problem . . . 310
9.5.2 Absorbing Boundary Condition . . . 318
9.5.3 The Mixed Problem . . . 319
9.5.4 Determining an Electromagnetic Field . . . 322
9.6 Exercises . . . 324

III Inverse Problems: Inferring Physics from Data **329**

10 Introduction to Inverse Problems in Mechanics **331**
10.1 Why Deep Learning in Inverse Problems? . . . 331
10.2 Learning a Velocity Field from Position Data . . . 331
10.2.1 Implementation . . . 332
10.3 Learning a Force Field from Position Data . . . 336
10.3.1 Implementation . . . 338
10.3.2 Exercises . . . 340

11 Identifying Conservation Laws **343**
11.1 Mechanical Systems . . . 343
11.1.1 Implementations . . . 347
11.2 Fluid Dynamics . . . 351
11.2.1 Conservation of mass . . . 352
11.2.2 Conservation of Momentum . . . 356
11.2.3 The State Equation . . . 363
11.2.4 One-Dimensional Adiabatic Process . . . 364
11.3 Exercises . . . 369

12 Parameter Estimation **371**
12.1 Newton's Cooling Law . . . 371
12.2 Frequency Estimation . . . 378
12.3 Pendulum Length Estimation . . . 379
12.4 Estimating Thermal Diffusivity . . . 380
12.5 Parameter Estimation for Lotka–Volterra System . . . 386
12.6 Exercises . . . 389

13 Equations Discovery **397**
13.1 Estimation of a Time-dependent Coefficient . . . 397
13.2 Finding all Time-dependent Coefficients . . . 404
13.3 First Order ODE Discovery . . . 405
13.4 Second Order ODE Discovery . . . 407
13.5 Euler-Lagrange Equation Discovery . . . 409
13.6 Hamiltonian System Discovery . . . 413
13.7 PDE Discovery . . . 415
13.8 Recovering Heat Kernels . . . 419
13.9 The Heat Kernel on $\mathbb{S}^1$ . . . 425
13.10Exercises . . . 427

14 Inverse Problems for Classical PDEs **431**
14.1 Learning a Dirichlet Boundary Condition for the Laplace Equation . . . 431
14.2 Learning the Initial Condition for the Heat Equation . . . 433
14.3 Learning the Initial Conditions for Wave Equation . . . 434
14.4 Exercises . . . 436

15 Special Structures **441**
15.1 Residual and Neural ODEs . . . 441
15.2 Hamiltonian Neural Networks (HNN) . . . 445
15.3 Reconstruction of Phase-Space Trajectories . . . 450
15.4 Exercises . . . 452
15.5 Lagrangian Neural Networks (LNN) . . . 455
15.6 Exercises . . . 460
15.7 Hamilton–Jacobi Neural Networks . . . 464
15.8 Generating-Function Networks . . . 467
15.9 Attention Mechanism along Trajectories . . . 470
15.10Neural ODEs with Attention . . . 480
15.11Attention and Hamiltonian Neural Networks . . . 482
15.12Attention and Lagrangians . . . 485
15.13Learning the Constraint Manifold . . . 487
15.14Exercises . . . 488

16 Further Applications **493**
16.1 Abel's Mechanical Problem . . . 493
16.2 Inverse Sturm–Liouville Problem . . . 497
16.3 Computer Tomography . . . 502

16.4 Deep Learning for GPS–INS Sensor Fusion 507

17 Miscellaneous Exercises **513**

Bibliography **521**

Index **529**

Part I

Introduction to Neural Networks and Applications

This page intentionally left blank

Chapter 1

Review of Feed-forward Neural Networks

Feed-forward neural networks (FNNs) form the foundational architecture for learning nonlinear mappings from data. Their mathematical representation is straightforward, their training relies on efficient gradient-based optimization, and their expressive power is theoretically well established through universal approximation results. As such, they provide a natural baseline model for data-driven approaches in Physics.

However, conventional FNNs possess no inherent understanding of the underlying structure of physical systems. They do not enforce dynamical laws, respect conservation principles, or preserve geometric properties. Consequently, while they can accurately interpolate training data, their extrapolation behavior and long-term stability are often unreliable in dynamical applications.

These observations highlight a central theme of this book: *incorporating physical knowledge into neural models improves generalization, stability, and interpretability.* The methods introduced, ranging from neural differential equation frameworks to architectures that preserve energy and geometry, build upon the strengths of feed-forward network.

Together, these developments transition from general-purpose function approximation to physics-aware learning, aligning neural computation with the structure of dynamical systems.

This chapter provides only a concise overview of feed-forward neural networks, which serve as the driving engine for the methods developed in the subsequent chapters. Readers interested in a more detailed exposition are encouraged to consult, for an accessible introduction, the books of Goodfellow et al. [40] and Ye [102]. A more mathematically rigorous treatment can be found in Calin [15].

1.0.1 Overview of FNNs

Feed-forward neural networks, also known as multilayer perceptrons, represent one of the fundamental computational architectures in Deep Learning. They define a parametric family of functions capable of approximating complex nonlinear input–output relationships.

For instance, in the context of Classical Mechanics, one frequently encounters problems in which we aim to approximate deterministic mappings between physical quantities. Examples include learning force fields as functions of configuration, approximating Hamiltonian or Lagrangian functions from observational data, or constructing surrogate models for expensive numerical solvers.

From a supervised learning perspective, we are given a dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N} \subset \mathbb{R}^n \times \mathbb{R}^m, \tag{1.0.1}$$

where the inputs x_i may represent initial conditions, generalized coordinates, or full system states, and outputs y_i represent corresponding measurable quantities such as forces, velocities, or potential energies. The goal is to learn a function

$$f_\theta : \mathbb{R}^n \to \mathbb{R}^m, \tag{1.0.2}$$

parameterized by θ, that approximates an unknown physical relationship $y = g(x)$. When the underlying mapping is smooth and well-defined—as is typically the case in deterministic mechanics—FNNs provide a powerful and flexible framework for approximation.

A conceptual advantage of feed-forward networks lies in their *universality*: with sufficient width or depth, they can approximate any continuous function defined on a compact domain to arbitrary accuracy. This makes them ideal baseline models when investigating the expressive power of machine learning methods for physical systems. Before introducing architectures that explicitly encode physical structures—such as Physics-Informed Neural Networks (PINNs)—it is essential to establish feed-forward networks as the canonical reference model for nonlinear function approximation.

However, while FNNs are expressive, they do not inherently incorporate physical constraints such as time evolution structure, symplectic geometry, or conservation laws. Their behavior outside the observed data domain *is determined purely by learned statistical correlations, not by the underlying Physics.* Consequently, they may struggle with long-term prediction stability or global generalization in dynamical systems. These limitations motivate the development of *physics-aware architectures* found in later chapters of this book.

Despite these challenges, feed-forward neural networks remain the first tool of choice when constructing data-driven models of mechanical systems. They provide a clean mathematical framework, a well-understood optimization landscape, and a strong theoretical foundation that informs the design of more specialized models that follow.

1.0.2 Mathematical Structure of a Feed-forward Network

The bare anatomy of a FNN A feed-forward neural network is a parameterized function constructed as a composition of affine transformations followed by nonlinear activation functions. The architecture is organized in layers: *an input layer, one or more hidden layers, and an output layer.* Information flows strictly in the forward direction, without recurrent or feedback connections.

Let the input be denoted by

$$a^{(0)} = x \in \mathbb{R}^n, \tag{1.0.3}$$

and let the network contain L layers of learnable parameters, indexed by $l = 1, \ldots, L$. Each layer performs an affine transformation followed by a nonlinear activation. Specifically, for layer l we define

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \tag{1.0.4}$$

where $W^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ is the weight matrix, $b^{(l)} \in \mathbb{R}^{d_l}$ is the bias vector, and d_l denotes the width of layer l. The activation is then given by

$$a^{(l)} = \sigma^{(l)}\left(z^{(l)}\right), \tag{1.0.5}$$

where $\sigma^{(l)} : \mathbb{R}^{d_l} \to \mathbb{R}^{d_l}$ is a nonlinear function applied component-wise.

The complete feed-forward mapping can therefore be expressed as a nested composition:

$$f_\theta(x) = a^{(L)} = \sigma^{(L)} \circ \mathcal{A}^{(L-1)} \circ \cdots \circ \sigma^{(2)} \circ \mathcal{A}^{(1)}(x), \tag{1.0.6}$$

where each $\mathcal{A}^{(l)}$ denotes the affine map in (1.0.4).

The set of all network parameters is

$$\theta = \left\{ W^{(1)}, b^{(1)}, \ldots, W^{(L)}, b^{(L)} \right\}. \tag{1.0.7}$$

Descriptors of a FNN: width, depth, capacity The *depth* of the network corresponds to the number of learnable layers L, while the *width* refers to the number of units d_l in each hidden layer. These architectural choices influence both the expressive power and generalization behavior of the model. In general:

- Increasing width expands the dimensionality of the function space the network can represent.
- Increasing depth enables hierarchical feature extraction and can lead to more efficient representations of complex nonlinear functions.

For classical mechanics applications, the choice of architecture often reflects the intrinsic dimensional structure of the underlying physical model. For instance, when learning mappings between generalized coordinates and forces, the input and output dimensions are dictated by the system's degrees of freedom. Hidden layers provide the flexibility needed to approximate nonlinear functional dependencies.

We note that, unlike function approximation methods tailored specifically for physical systems (e.g., Hamiltonian neural networks or Lagrangian neural networks), the feed-forward architecture does not impose any constraints related to energy preservation, phase space geometry, or temporal evolution. This lack of inductive bias introduces challenges when modeling dynamical systems, motivating the specialized architectures discussed in subsequent chapters.

1.0.3 Neuron Model and Activation Functions

At the core of a feed-forward network lies the concept of *neuron*, a computational unit that processes information through three fundamental components:

1. a set of real-valued inputs,

2. corresponding learnable weights and a bias term,

3. a nonlinear activation function.

Let $x \in \mathbb{R}^{d_{l-1}}$ be the input to a neuron in layer l. The neuron computes a weighted aggregation of the inputs through the scalar product

$$z = \langle w, x \rangle + b = \sum_{j=1}^{d_{l-1}} w_j x_j + b, \tag{1.0.8}$$

where $w \in \mathbb{R}^{d_{l-1}}$ is the weight vector and $b \in \mathbb{R}$ is the bias. Geometrically, the scalar product $\langle w, x \rangle$ measures the projection of the input vector x along the direction of w. The neuron thus learns which directions in input space are most relevant for producing an informative output.

To introduce nonlinearity into the model, the aggregated signal z is passed through an activation function

$$a = \sigma(z), \tag{1.0.9}$$

where $\sigma : \mathbb{R} \to \mathbb{R}$ is applied elementwise across neurons in a layer. The activation function is essential: without it, the network would collapse to a single affine transformation regardless of the number of layers. Also, the activation function's nonlinearity is crucial: without it, the network reduces to a linear model regardless of its depth.

Activation Functions In modeling physical systems, it is desirable that the learned mappings possess smoothness properties consistent with the underlying mechanics. For this reason, we use *differentiable activation functions* throughout this book. Among the most commonly used are:

Hyperbolic tangent:

$$\sigma(x) = \tanh(x), \tag{1.0.10}$$

a smooth, bounded, odd function widely used in dynamical system modeling due to its analytic properties and ability to represent both positive and negative outputs.

Softplus:

$$\sigma(x) = \log\left(1 + e^{x}\right), \tag{1.0.11}$$

a smooth approximation of the rectified linear unit (ReLU), avoiding nondifferentiability at the origin and producing strictly positive outputs.

Swish:

$$\sigma(x) = x \cdot \mathrm{sigm}(x) = \frac{x}{1 + e^{-x}}, \tag{1.0.12}$$

which maintains smoothness and often improves optimization dynamics due to nonmonotonic curvature.

By contrast, the widely used ReLU activation

$$\sigma(x) = \max\{0, x\}$$

is *not* adopted in this text, as its nondifferentiability at $x = 0$ conflicts with the continuous differentiability expected in physical systems and obstructs the computation of higher-order derivatives required in many mechanics-based learning tasks.

There is no single best activation function. The optimal choice is task-dependent, and in many cases *Swish* has been found to outperform *tanh* due to its favorable gradient properties.

Nonlinearity The nonlinear activation function enables the network to approximate complex maps that cannot be represented by linear models. In particular, the repeated composition of affine transformations with nonlinear activations produces highly expressive function families capable of capturing intricate relationships among physical variables.

Later sections connect these mathematical choices to the structural and geometric properties of mechanical systems, highlighting both the strengths and limitations of standard activation-based neuron models.

1.0.4 Assessing Network Quality

When evaluating the *quality* of a neural network for Physics, the following three criteria are particularly useful:

1. Expressivity. What class of functions can the network represent? In our setting, many target maps (e.g., forces, Hamiltonians, constitutive laws) are *real-analytic* and therefore very smooth. Although universal approximation theorems guarantee that networks with suitable (non-polynomial) activations can uniformly approximate any continuous function on a compact set, Physics problems often demand more: stable higher-order derivatives and smooth vector fields. For this reason, throughout this book we adopt *real-analytic* activations–such as tanh, Swish, and Softplus–so that the learned models inherit differentiability properties compatible with the governing equations. In short, while continuity suffices for universality, smooth/analytic activations are preferable for most problems in Physics.

2. Efficiency. How many resources (neurons, weights, layers, total parameters) are required to approximate a given function to a target accuracy? The answer depends on the function's intrinsic complexity (e.g., regularity, compositional structure, effective dimension). Depth can dramatically reduce parameter counts for hierarchically structured mappings, whereas purely shallow approximators may require exponentially many units in high dimensions. Efficiency considerations guide architectural choices (width/depth) and regularization.

3. Learnability. How rapidly and reliably can the network discover good parameters from data? Learnability reflects optimization landscape properties and gradient flow. Smooth activations (e.g., Swish, tanh, Softplus) generally yield better-behaved derivatives and facilitate backpropagation, whereas saturating sigmoids can induce vanishing

gradients in deep stacks. Practical learnability is enhanced by well-scaled initializations, normalization, and physics-aware losses or constraints that reduce the search space to physically admissible solutions.

1.0.5 Training Framework: Loss Minimization

Supervised training of feed-forward networks proceeds by minimizing an *empirical loss* (also called a *cost function*) that quantifies misfit between predictions and targets. Given data

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N} \subset \mathbb{R}^n \times \mathbb{R}^m, \tag{1.0.13}$$

and a parameterized model $f_\theta : \mathbb{R}^n \to \mathbb{R}^m$, the empirical loss is

$$\mathcal{L}_N(\theta) \;=\; \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\big(f_\theta(x_i),\, y_i\big), \tag{1.0.14}$$

where $\mathcal{L} : \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}_+$ is a sample-wise loss, often being described by a distance, or–in the case when the points are probability distributions–the loss is described by a Kullback–Leibler divergenge, or cross-entropy.

Losses for Regression For real-valued regression targets (typical in mechanics), the mean squared error (MSE) is standard:

$$\mathcal{L}_{\mathrm{MSE}}\big(f_\theta(x),\, y\big) \;=\; \frac{1}{m}\, \|f_\theta(x) - y\|_2^2, \tag{1.0.15}$$

where $\|\cdot\|_2$ denotes the Euclidean norm. When appropriate, one augments (1.0.14) with a regularization penalty to control capacity (e.g., weight decay),

$$\mathcal{J}_\lambda(\theta) \;=\; \mathcal{L}_N(\theta) \;+\; \lambda\, \|\theta\|_2^2, \tag{1.0.16}$$

where $\lambda \geq 0$ tunes the strength of the penalty. This L^2-regularization has the role of keeping the weight parameters θ small, so overfitting is not likely to occur.

Mini-batch gradient estimate The symbol $\widehat{\nabla}\mathcal{L}_N(\theta)$ denotes a *stochastic (mini-batch) estimate* of the full empirical gradient

$$\nabla \mathcal{L}_N(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta\, \mathcal{L}\big(f_\theta(x_i), y_i\big).$$

Given a mini-batch $\mathcal{B} \subset \{1, \ldots, N\}$ of size $|\mathcal{B}| = B$, sampled (typically uniformly without replacement) from the training set, we form

$$\widehat{\nabla}\mathcal{L}_N(\theta) \;=\; \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\theta\, \mathcal{L}\big(f_\theta(x_i), y_i\big). \tag{1.0.17}$$

Under uniform sampling, $\widehat{\nabla}\mathcal{L}_N(\theta)$ is an *unbiased* estimator, i.e., $\mathbb{E}[\widehat{\nabla}\mathcal{L}_N(\theta)] = \nabla\mathcal{L}_N(\theta)$, with variance decreasing as B increases. Stochastic gradient descent (SGD) updates

$$\theta_{t+1} = \theta_t - \eta_t\, \widehat{\nabla}\mathcal{L}_N(\theta_t)$$

use (1.0.17) to trade exactness for computational efficiency, enabling frequent parameter updates and better utilization of large datasets. In practice, variants (momentum, Adam, etc.) replace or augment $\widehat{\nabla}\mathcal{L}_N(\theta)$ with smoothed/normalized forms but retain the same estimator structure.

Learning-rate schedule It was found efficient to decrease the learning rate η_t over training iterations to allow rapid progress early on and finer adjustments later. An example of a simple step schedule is

$$\eta_t = \begin{cases} 10^{-2}, & 0 \le t < 500, \\ 10^{-3}, & 500 \le t < 1000, \\ 5 \times 10^{-4}, & t \ge 1000, \end{cases} \tag{1.0.18}$$

where t indexes epochs (or, analogously, iteration blocks). Such decay reduces oscillations near minima and typically improves convergence efficiency of the loss optimization. The optimal schedule is problem-dependent but the principle–*large steps early, small steps late*–is broadly effective.

Chain Rule via Computation Graphs Let the network be composed of layers

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = \sigma^{(l)}\big(z^{(l)}\big), \quad l = 1, \dots, L, \tag{1.0.19}$$

with $a^{(0)} = x$ and output $f_\theta(x) = a^{(L)}$. For a single sample (x, y), define the sample loss $\ell(\theta) = \mathcal{L}(a^{(L)}, y)$. Gradients with respect to parameters follow from the chain rule along the computation graph:

$$\frac{\partial \ell}{\partial W^{(l)}} \;=\; \delta^{(l)}\big(a^{(l-1)}\big)^\top, \qquad \frac{\partial \ell}{\partial b^{(l)}} \;=\; \delta^{(l)}, \tag{1.0.20}$$

where the backpropagated sensitivities $\delta^{(l)}$ satisfy

$$\delta^{(L)} \;=\; \frac{\partial \mathcal{L}}{\partial a^{(L)}} \;\odot\; \sigma^{(L)\prime}\big(z^{(L)}\big), \tag{1.0.21}$$

$$\delta^{(l)} \;=\; \big(W^{(l+1)}\big)^\top \delta^{(l+1)} \;\odot\; \sigma^{(l)\prime}\big(z^{(l)}\big), \quad l = L-1, \dots, 1, \tag{1.0.22}$$

and $\odot$ denotes the Hadamard (componentwise) product. The differentiability of the activations $\sigma^{(l)}$ ensures that all Jacobian factors in (1.0.29) are well-defined, which is essential in mechanics-oriented models that require stable higher-order derivatives.

Physics-Based Losses and Hybrid Training In modeling physical systems, the loss function may incorporate not only data fidelity but also physical laws. Instead of (or in addition to) minimizing discrepancies between predictions and observations, one penalizes violations of differential equations, conservation principles, or geometric invariants.

If the governing relation is given by a differential equation

$$\mathcal{F}\big(x, f_\theta(x), \nabla f_\theta(x)\big) = 0, \tag{1.0.23}$$

a Physics residual loss can be added:

$$\mathcal{L}_{\text{phys}}(\theta) = \frac{1}{N_r} \sum_{j=1}^{N_r} \left\| \mathcal{F}\big(x_j, f_\theta(x_j), \nabla f_\theta(x_j)\big) \right\|_2^2, \tag{1.0.24}$$

evaluated at collocation points $\{x_j\}$. This enables *unsupervised* training driven solely by the physical law when labeled data are absent.

In many applications, one employs a hybrid objective that blends data and Physics:

$$\mathcal{L}_{\text{hybrid}}(\theta) = \mathcal{L}_N(\theta) + \lambda_{\text{phys}}\, \mathcal{L}_{\text{phys}}(\theta), \tag{1.0.25}$$

where $\lambda_{\text{phys}} > 0$ balances empirical accuracy with physical consistency.

Physical constraints may also encode conservation laws–e.g., conservation of energy, momentum, or mass/flux–introduced as *soft constraints* (penalties) within the loss. Initial and boundary conditions can likewise be enforced softly via penalty terms; however, when possible, it is often more efficient to *embed* such constraints *hard* into the network architecture so they are satisfied identically by construction, thereby reducing tuning burden and improving numerical stability.

In summary, training proceeds by minimizing losses (cost functions) defined over data, Physics, or their combination. This perspective provides a flexible mechanism for incorporating domain knowledge into learning while remaining compatible with standard, scalable first-order optimization methods.

1.0.6 Backpropagation Algorithm and Gradient Flow

Training a neural network consists of minimizing a loss (cost) function. The standard approach is *gradient descent*, which iteratively updates the parameters in the direction of steepest decrease of the loss:

$$\theta_{t+1} = \theta_t - \eta\, \nabla_\theta \mathcal{L}(\theta_t),$$

where $\eta > 0$ is the learning rate. This requires computing the gradient of the loss with respect to all network parameters. In feed-forward networks composed of many layers, this computation must be carried out efficiently, and this is precisely the role of the *backpropagation algorithm*.

Although alternative learning mechanisms may exist in biological neural systems, and the brain very likely does not optimize using backpropagation, at present no more efficient algorithm for training deep parametric models is known. Backpropagation therefore remains the core optimization tool in modern Deep Learning.

Backpropagation Mechanics Backpropagation applies the chain rule layer by layer from the output back to the input. Let the network be composed of layers

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = \sigma^{(l)}\big(z^{(l)}\big), \quad l = 1, \ldots, L, \tag{1.0.26}$$

with $a^{(0)} = x$ and output $f_\theta(x) = a^{(L)}$. For a single sample (x, y), define the sample loss $\ell(\theta) = \mathcal{L}(a^{(L)}, y)$. Gradients with respect to parameters follow from the chain rule along the computation graph:

$$\frac{\partial \ell}{\partial W^{(l)}} \;=\; \delta^{(l)} \big(a^{(l-1)}\big)^\top, \qquad \frac{\partial \ell}{\partial b^{(l)}} \;=\; \delta^{(l)}, \tag{1.0.27}$$

where the backpropagated sensitivities $\delta^{(l)}$ satisfy

$$\delta^{(L)} \;=\; \frac{\partial \mathcal{L}}{\partial a^{(L)}} \;\odot\; \sigma^{(L)\prime}\big(z^{(L)}\big), \tag{1.0.28}$$

$$\delta^{(l)} \;=\; \big(W^{(l+1)}\big)^\top \delta^{(l+1)} \;\odot\; \sigma^{(l)\prime}\big(z^{(l)}\big), \;\; l = L-1, \dots, 1, \tag{1.0.29}$$

and $\odot$ denotes the Hadamard (componentwise) product. The differentiability of the activations $\sigma^{(l)}$ ensures that all Jacobian factors in (1.0.29) are well-defined, which is essential in mechanics-oriented models that require stable higher-order derivatives.

Vanishing and Exploding Gradients The recursive structure in backpropagation rules reveals a major issue in deep networks: repeated multiplication by activation derivatives can cause one of the following problems:

$$(i)\;\; |\delta^{(l)}| \to 0 \quad \text{(vanishing gradient)}, \qquad (ii)\;\; |\delta^{(l)}| \to \infty \quad \text{(exploding gradient)}.$$

Vanishing gradients slow or halt learning in early layers. Exploding gradients lead to numerical instability. Controlling gradient flow is therefore crucial in deep architectures.

In the present book we avoided the use of *Sigmoid* activations and preferred Swish activation instead.

Why Sigmoid Activations Fail in Deep Models The logistic sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

saturates rapidly, yielding derivatives

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \le 0.25,$$

which are close to zero over most of the activation range once training begins. As depth increases, the vanishing gradient problem becomes severe, making sigmoid activations unsuitable for deep models used in nonlinear physical systems.

Swish Improves Gradient Propagation A preferred activation in this text is the smooth and non-monotonic *Swish* function:

$$\mathrm{Swish}(x) = x \cdot \mathrm{sigm}(x) = \frac{x}{1 + e^{-x}}, \tag{1.0.30}$$

which offers improved gradient flow in deep networks, non-saturating behavior for positive activations, and smoothness compatible with physical differential structure.

Empirical results show that Swish reduces optimization difficulties associated with saturating activations and accelerates convergence in regression-based Physics modeling.

In summary, backpropagation enables efficient gradient computation for gradient descent training, but its success depends critically on maintaining healthy gradient flow. Activation function choice plays a central role in alleviating vanishing gradients, making smooth functions such as Swish preferable for Deep Learning architectures in mechanics.

1.0.7 Approximation Theory and Expressive Power

Feed-forward neural networks possess remarkable expressive capabilities as *universal function approximators*. Their mathematical power arises from composing affine maps with nonlinear activations, yielding rich families of parameterized functions. In the context of mechanics, this expressive power is critical: many maps of interest, such as force fields, Hamiltonian flows, and constitutive models, are nonlinear and often defined on high-dimensional state spaces.

Universal Approximation Theorem One of the foundational results justifying neural network modeling is the Universal Approximation Theorem (UAT). In a common formulation this states:

Theorem *Let $\sigma : \mathbb{R} \to \mathbb{R}$ be a continuous, bounded, non-polynomial activation function. Then, for any continuous function $f : K \to \mathbb{R}$ defined on a compact set $K \subset \mathbb{R}^n$ and any $\varepsilon > 0$, there exists a feed-forward network with a single hidden layer such that*

$$\|f - f_\theta\|_\infty < \varepsilon.$$

Thus, even shallow networks are capable of approximating any continuous function to arbitrary accuracy. However, this result is existence-based: it does not quantify the number of neurons required, nor does it address optimization or generalization.

The foundational proof of the Universal Approximation Theorem in the continuous case was given by Cybenko [30]. This landmark paper established the theoretical basis for neural network expressivity.

A broader version of the theorem was later developed by Hornik, Stinchcombe, and White [48], who proved that multilayer feed-forward networks with a wide class of activation functions can approximate any Borel measurable function, thereby generalizing Cybenko's result.

Classical universal approximation theorems were further extended from finite dimensional continuous functions to *functionals*–that is, mappings from infinite-dimensional function spaces into $\mathbb{R}$–in the work of Chen and Chen [22]. They proved that, under mild continuity assumptions, neural networks can approximate any continuous functional on a compact subset of a Banach space. This result is particularly relevant to dynamical systems and operator learning, since many problems in Physics and mechanics involve mapping entire trajectories or functions to scalar quantities such as norms, energies, or PDE-derived observables.

Depth Versus Width While a single wide hidden layer has universal approximation capability, the number of required neurons may grow exponentially in dimension. Introducing *depth* (multiple hidden layers) can dramatically improve efficiency: hierarchical representation allows the reuse and composition of learned features.

In particular, it has been noticed that:

- Some functions can be represented with polynomial width when depth grows, but require exponential width if depth is fixed.
- Deeper networks can approximate functions with fewer parameters by exploiting multiscale or hierarchical structure.

This principle aligns well with physical system modeling, where complex interactions arise through compositions of simpler transformations.

Smoothness and Physical Systems In many physical applications, the target functions are not arbitrary: they are smooth, structured, and often governed by differential constraints. Consequently, smooth activations (e.g., tanh, Swish, Softplus) allow FNNs to approximate such functions with controlled regularity, enabling stable computation of derivatives, and better alignment with the smooth nature of physical laws. This is essential when networks approximate vector fields or Hamiltonians that must be differentiated repeatedly to evaluate dynamics.

Global Versus Local Behavior Although the UAT guarantees approximation accuracy on compact domains, physical systems often require reliable extrapolation beyond the data range. Standard FNNs do not encode priors such as: energy conservation, symplectic invariance, controlled long-term stability. Therefore, while FNNs serve as powerful baselines, additional physical structure must be enforced to avoid unphysical predictions as well as meaningful extrapolations.

In summary, feed-forward networks are theoretically capable of approximating the nonlinear mappings encountered in mechanics, but efficiency and physical reliability depend strongly on architectural depth, smooth activation functions, and the inclusion of structural constraints. These considerations motivate the physics-aware neural architectures developed in later chapters.

1.0.8 Training versus Testing Data

In traditional supervised learning, data are often split into three disjoint sets: *training* (e.g., 70%), *validation* (10%), and *testing* (20%). The training set is used to fit the model by minimizing a loss function (typically via gradient descent). The validation set is used to tune hyperparameters and to decide when to stop training (e.g., early stopping). The testing set is kept strictly held out and is used only for final, unbiased performance evaluation.

A few standard pathologies are worth recalling:

- *Overfitting:* training loss is very small while test loss is large; mitigation includes regularization, data augmentation, or architectural constraints.

- *Overtraining (no early stopping):* both training and test losses begin to rise after some epochs due to excessive optimization steps; training should be halted at or near the epoch where validation (or test, if no validation is used) loss attains its minimum.

The physics-informed setting In many chapters of this book we employ Physics-informed approaches, where, in addition to data, we incorporate *known physical structure* (differential operators, boundary/initial conditions, conservation laws) directly into the loss or architecture.

For *forward problems*, we frequently possess high-quality reference solutions (either analytic, when available, or accurate numerical solutions such as high-order Runge–Kutta). In such cases, we often use essentially *all* available data together with the physics-based terms to train the model, and then *validate* the result by comparing against these trusted reference solutions rather than relying on a conventional held-out test split.

In *inverse problems*–where unknown parameters or relationships must be inferred from data–learning efficiency is assessed on held-out test data, especially when no *a priori* knowledge of the governing law or parameters is available. In most cases we generate *synthetic data* from models with known ground truth; validation then consists of comparing the learned parameters and predictions against these known values.

1.0.9 Generalization, Overfitting, and Regularization

The goal of training is not only to match the training data, but to learn a mapping that performs well on unseen data and remains consistent with underlying physical laws. In machine learning terminology, this property is known as *generalization*.

However, a neural network with sufficiently many parameters can approximate the training data extremely well, even capturing noise, discretization artifacts, or sampling errors. This phenomenon is called *overfitting*. While synthetic Physics data may seem noise-free, numerical integration, finite precision, and discretization introduce high-frequency components that an overly flexible network may memorize rather than generalize beyond.

Regularization To mitigate overfitting, we introduce *regularization*, as a mechanism that encourages simpler, smoother, or more physically plausible models. Typical approaches include the following.

(i) *Weight regularization* (e.g., L^2 penalties) discourages overly large parameter values:

$$\mathcal{L}_\lambda(\theta) = \mathcal{L}_N(\theta) + \lambda\|\theta\|_2^2.$$

(ii) *Early stopping* halts training when performance on a validation set degrades.

(iii) *Dropout or noise injection* reduces reliance on particular neurons or samples (less common for Physics models due to smoothness disruption).

(iv) *Dataset augmentation* introduces variations consistent with known invariances (e.g., symmetries in rigid-body mechanics).

Besides the aforementioned classical regularizations, when dealing with Physics problems we may employ further methods of regularization which contain Physics built in, encoding known geometric and physical structures, which tend to generalize better because they reduce the space of admissible solutions. Such regularization structures may include:

(i) Conservation laws (such as energy, momentum, etc.),

(ii) Symmetries and invariances (to rotations, translations, etc.),

(iii) Stability and boundedness in time (such as positivity),

(iv) Geometric constraints (such as symplectic form preservation, manifold constraints, etc.).

These constraints can be implemented in the following forms:

1. *Soft constraints* by additively penalizing violations in the loss,

2. *Hard constraints* by embedding the structure into the neural network architecture so that violations are impossible.

Embedding known Physics directly into the model often yields superior generalization, especially for long-term trajectory prediction.

While fully unconstrained feed-forward networks are universal approximators, they may generalize poorly in physical systems unless regularization or physical inductive biases are incorporated. The next chapters introduce architectures specifically designed to encode mechanical laws and geometric constraints, improving both accuracy and physical credibility.

Limitations of Classical FNN While feed-forward neural networks possess strong theoretical and practical approximation capabilities, their use in modeling mechanical systems encounters fundamental limitations when physical structure is not explicitly incorporated. These limitations arise from the fact that standard FNNs operate purely as static function approximators without embedding the dynamical laws, geometric invariants, or stability properties intrinsic to mechanics.

Mechanical systems obey differential equations that define their time evolution. A standard FNN does not encode this dynamical structure; thus, naive FNN-based prediction of dynamics often leads to divergence or qualitatively incorrect behavior after only a short integration interval.

The phase space of mechanical systems usually carries a geometric structure. For example, Hamiltonian systems evolve under a symplectic map, preserving phase-space volume and energy. Since classical FNNs do not enforce geometry, conservation laws or constraints, they may learn mappings that fit the training data locally but violate global laws of mechanics, leading to unphysical qualitative behavior.

Even if the universal approximation theorem holds accurately only on compact sets, the learned model may extrapolate poorly outside the sampled region. This leads to drift in energy levels, loss of periodic or quasiperiodic structure, or chaotic amplification of modeling inaccuracies. This behavior compromises reliability in applications such as trajectory prediction.

1.0.10 Why Deep Learning for Physics?

For decades, classical numerical methods (finite differences/elements, spectral schemes, variational integrators) have been the workhorse for solving forward Physics problems. What, then, is distinctive about Deep Learning?

1. *Amortized computation (train once, reuse many times).* Classical solvers must be rerun from scratch for each new parameter, initial condition, or boundary configuration, which can be expensive. A neural surrogate is trained once and then provides fast evaluations across a wide range of queries, dramatically reducing time-to-solution at inference.

2. *Data–Physics fusion.* Traditional solvers faithfully enforce *known* governing equations but do not learn *unknown* closures, constitutive relations, or empirical corrections. Neural models can incorporate Physics via constraints (e.g., PDE residuals, conservation laws) *and* absorb information from data, closing gaps where the equations are incomplete or parameters are uncertain.

3. *Inverse problems and ill-posedness.* Many inverse problems are ill-posed under classical formulations. Deep learning offers powerful regularization through architectural priors, smooth activations, and Physics-informed losses, and enables end-to-end differentiation (via autodiff) for efficient gradient-based parameter and field inference. In practice, such models often outperform purely numerical approaches on challenging inverse tasks.

In short, Deep Learning complements–not replaces–numerical simulation: it amortizes computation, learns from data where Physics is incomplete, and provides effective tools for ill-posed inverse problems, all while allowing physical constraints to be enforced through the loss or the architecture.

In summary, classical feed-forward neural networks provide a powerful starting point for nonlinear function approximation but lack the inductive biases required for physically meaningful dynamical modeling. These limitations motivate the development of specialized architectures that incorporate conservation laws, symplectic structure, and differential constraints directly into the network design. The following chapters explore such physics-aware neural representations in detail.

Further reading Over the past decade, Deep Learning has become a transformative tool in Physics, bridging data-driven modeling with the structure of physical laws. Lin, Tegmark, and Rolnick [62] provided early theoretical insight into why deep networks efficiently represent physical functions, emphasizing locality, compositionality, and symmetry as key ingredients. Carleo et al. [19] offered a broad review of how machine learning, particularly neural networks, has permeated nearly every branch of Physics, from quantum many-body systems to high-energy experiments. The rise of Physics-informed approaches–such as those used in this book–initiated by Raissi, Perdikaris, and Karniadakis [79] and consolidated in the comprehensive review by Karniadakis et al. [53], established a framework for embedding differential equations and conservation laws directly into neural architectures.

Subsequent advances–such as the Neural ODEs [23], and Hamiltonian or Lagrangian neural networks [42, 29]–have brought learning closer to physical interpretability by

respecting variational and geometric principles. More recent developments like DeepONet [64] and Fourier Neural Operators [61] extend Deep Learning beyond finite-dimensional mappings to operator learning, enabling the direct approximation of solution operators for PDEs. Together, these works demonstrate that Deep Learning has evolved from a computational tool to a genuine modeling framework for mathematical physics-capable of solving, discovering, and generalizing physical laws.

Finally, we highly recommend that readers consult the excellent online book by Thuerey et al. [91], which provides hands-on code examples, insightful discussions, and a variety of nontrivial simulations illustrating the practical use of Deep Learning techniques in Physics.

This page intentionally left blank

Chapter 2

Deep Learning for Solving Equations

In this chapter, we demonstrate how Deep Learning provides a unified framework for solving a wide range of equations, including algebraic equations, functional equations, integral equations, and ordinary differential equations (ODEs) with boundary or initial conditions.

Traditionally, each of these types of equations belongs to distinct domains of mathematics and has been addressed by specialized numerical methods. For example, algebraic equations are typically solved using methods such as bisection or Newton's method, while ordinary differential equations are handled by techniques like Euler's method or Runge–Kutta schemes. These diverse approaches highlight the fragmented nature of conventional numerical analysis.

In contrast, Deep Learning offers a *single, unitary method based on neural networks that can be applied across all these classes of problems.* By training a neural network to approximate the solution of an equation, we can replace the kaleidoscope of traditional methods with only one flexible framework.

Moreover, the Deep Learning methods introduced in this chapter serve as a foundation for later applications in Classical Mechanics, where a variety of mathematical tools — including those for solving differential and integral equations — play a crucial role. In subsequent chapters, we will build on these methods to address complex dynamical systems and other problems in Physics.

2.1 Solving Polynomial Equations with NN

Consider a polynomial equation $P(x) = 0$, where $P(x) = a_n x^n + \cdots + a_1 x + a_0$ is a polynomial in the variable x with real coefficients. Our objective is to find the roots of this equation. There are several strategies for achieving this task using neural networks.

2.1.1 The Direct Method

For the sake of simplicity we shall assume in the beginning that the equation $P(x) = 0$ has only one real root, x^*. Later, we shall consider the case of multiple real roots and complex roots.

To obtain the root x^* we shall employ a feed-forward neural network, $f_\theta(t)$, which is used to learn the root. The elements of the network are as follows:

• The input to the network, denoted by t can be any real number, preferable not too far from the root x^*. This can be accomplished using the bounds provided by Exercise 2.1.1, by which the input is chosen to be any number satisfying

$$|t| \leq \max\Big\{1, \frac{1}{|a_n|}(|a_{n-1}| + \cdots + |a_0|)\Big\}.$$

This input is only one number, preferably a random initial guess within the aforementioned bounds.

• The output of the network is one-dimensional, given by $f_\theta(t)$. Because polynomial functions do not oscillate very drastically—having only a fixed number of x-axis intercepts—it is expected that a neural network architecture with a single hidden layer of neurons will be sufficient to achieve satisfactory results. In other words, we can employ neural networks of the type

$$f_\theta(t) = \sum_{i=1}^{N} \alpha_i \, \sigma(w_i t + b_i) + \beta,$$

where σ is the activation function, α_i and β are the weights from the hidden layer to the output neuron and the bias of the output neuron (which has a linear activation), and w_i and b_i are the weights and biases of the hidden neurons. However, if the function $P(x)$ is something more complex than a polynomial, then more depth (more hidden layers) might be needed for the network.

• The loss function measures how far the network output $f_\theta(t)$ is from the root and it is given by

$$\mathcal{L}(\theta) = |P(f_\theta(t))|^2.$$

By minimizing this loss, the neural network learns values $f_\theta(t)$ that $P(f_\theta(t)) \to 0$ as $\theta \to \theta^*$, effectively solving the equation.

The training is done using backpropagation. Thus, the loss is used to compute the gradient with respect to network parameters θ and update them via gradient descent,

$$\theta_{new} = \theta_{old} - \lambda \nabla_\theta L(\theta_{old}).$$

The learning rate, $\lambda > 0$, is chosen small enough such that it enables a stable convergence. The training process stops when the loss converges to a sufficiently small value, indicating that $P(f_\theta^*(t)) \approx 0$. Finally, the root is then given by $x^* = f_{\theta^*}(t)$, where $\theta^* = \arg\min \mathcal{L}(\theta)$. Besides the gradient descent method, any other variant of it (such as Adam, Adagrad, Nesterov, etc.) can also be applied.

We make the remark that since this neural network does not use a target, it does not use a supervised learning.

Extension to multiple real roots Once a real root x^* of $P(x)$ has been found, synthetic division can be used to factor the polynomial. Specifically, one obtains a polynomial $G(x)$ of degree $n-1$ such that

$$P(x) = (x - x^*)\, G(x).$$

The same procedure that was applied to $P(x)$ can now be applied to $G(x)$ to find additional roots. This process is repeated recursively until one of the following occurs:

1. The degree of $G(x)$ becomes zero, in which case all the roots of $P(x)$ are real.
2. The degree of $G(x)$ remains nonzero (and is even), indicating that the remaining roots must occur as complex conjugate pairs.

Extension to complex roots In the case when $P(x)$ has complex roots, we modify our approach by using a neural network with a real input t and a two-dimensional output:

$$f_\theta(t) = \big(\rho_\theta(t),\, \varphi_\theta(t)\big),$$

where $\rho_\theta(t)$ represents the modulus and $\varphi_\theta(t)$ the phase. We then associate the output to a complex number in polar form:

$$x_\theta = \rho_\theta(t)\, e^{i\varphi_\theta(t)}.$$

Our objective is to tune the parameters θ so that x_θ becomes a root of the polynomial $P(x)$. To achieve this, we define the loss function as the squared modulus of $P(x_\theta)$:

$$\mathcal{L}(\theta) = \big|P(x_\theta)\big|^2 = P(x_\theta)\,\overline{P(x_\theta)}.$$

If we express the polynomial in the form

$$P(x) = \sum_{k=0}^{n} a_k x^k,$$

then substituting $x_\theta = \rho\, e^{i\varphi}$ (with $\rho = \rho_\theta(t)$ and $\varphi = \varphi_\theta(t)$) yields

$$P(x_\theta) = \sum_{k=0}^{n} a_k\, \rho^k\, e^{ik\varphi}.$$

Thus, the loss function becomes

$$\mathcal{L}(\theta) = \left|\sum_{k=0}^{n} a_k\, \rho^k\, e^{ik\varphi}\right|^2 = \left(\sum_{k=0}^{n} a_k\, \rho^k\, e^{ik\varphi}\right)\left(\sum_{j=0}^{n} a_j\, \rho^j\, e^{-ij\varphi}\right).$$

Expanding the product, we have

$$\mathcal{L}(\theta) = \sum_{j,k=0}^{n} a_j\, a_k\, \rho^{j+k}\, e^{i(k-j)\varphi}.$$

Since $e^{i(k-j)\varphi} = \cos((k-j)\varphi) + i\sin((k-j)\varphi)$ and $\mathcal{L}(\theta)$ is a real number, we focus on its real part:

$$\mathcal{L}(\theta) = \sum_{j,k=0}^{n} a_j\, a_k\, \rho^{j+k} \cos((k-j)\varphi).$$

By minimizing $\mathcal{L}(\theta)$ with respect to the network parameters θ, the neural network is trained to produce a complex number $x_{\theta^*} = \rho_{\theta^*}(t)\, e^{i\varphi_{\theta^*}(t)}$ that approximates a root of $P(x)$. We also note that the complex conjugate $\bar{x}_{\theta^*} = \rho_{\theta^*}(t)\, e^{-i\varphi_{\theta^*}(t)}$ is also a root of $P(x)$.

2.1.2 The Inverse Function Method

This method can be applied to the case of solving the equation $F(x) = 0$, where $F : D \to \mathbb{R}$ is a continuous and invertible function, with $D \subset \mathbb{R}$. By applying the inverse function F^{-1} to both sides of the equation, we obtain $F^{-1}(F(x)) = F^{-1}(0)$, which simplifies to $x = F^{-1}(0)$. Therefore, finding the inverse of F suffices to solve a large class of equations. This will be done using neural networks in two ways: *supervised* and *unsupervised*.

Supervised inverse construction In this approach we first generate a dataset of input-output pairs $\{(y_i, x_i)\}$ by sampling values x_i from the domain D and computing the corresponding values $y_i = F(x_i)$. This dataset serves to train a neural network to learn the inverse mapping. The network receives inputs $\{y_i\}$ and produces outputs $f_\theta(y_i)$ via a feed-forward architecture composed of dense layers. The corresponding labels are given by $\{x_i\}$, and the goal is to train the network so that $f_\theta(y_i)$ approximates x_i for each i.

The training objective is to minimize a loss function that measures the discrepancy between the network's prediction and the given labels

$$\mathcal{L}_{\text{sup}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} |f_\theta(y_i) - x_i|^2 .$$

Once the network is trained and the optimal parameters $\theta^* = \arg\min \mathcal{L}_{\text{sup}}(\theta)$ are obtained, the inverse function is given by $F^{-1}(y) = f_{\theta^*}(y)$. Consequently, the approximate solution to $F(x) = 0$ is given by $x^* = f_{\theta^*}(0)$.

Unsupervised inverse construction In the *unsupervised* approach, no explicit inverse dataset is required. Instead of providing explicit input-output pairs (as in supervised learning), in this case we rely on the inverse conditions $F(F^{-1}(y)) = y$ and $F^{-1}(F(x)) = x$. Therefore, it suffices to find a neural network f_θ satisfying $F(f_\theta(y)) = y$ for $y \in F(D)$ and $f_\theta(F(x)) = x$ for $x \in D$. In fact, only one of these relations is needed to construct the loss function. Thus, we choose a loss function which penalizes the difference between $f_\theta(y)$ and y as

$$\mathcal{L}_{\text{unsup}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} (F(f_\theta(y_i)) - y_i)^2,$$

where $\{y_i\}$ are inputs to the network, which are chosen within the function's range $F(D)$. The desired inverse is given by $F^{-1}(y) = f_{\theta^*}(y)$, where $\theta^* = \arg\min_\theta \mathcal{L}_{\text{unsup}}(\theta)$, and the solution of $F(x) = 0$ is given by $x^* = f_{\theta^*}(0)$.

A hybrid approach A hybrid strategy can combine both supervised and unsupervised methods to take advantage of their respective strengths. In this case, we assume some data $\{y_i = F(x_i), x_i\}$ are known and the loss function is a weighted sum

$$\mathcal{L}(\theta) = \alpha \, \mathcal{L}_{\text{sup}}(\theta) + \beta \, \mathcal{L}_{\text{unsup}}(\theta),$$

where $\alpha, \beta > 0$ are hyperparameters that balance the contributions of the supervised and unsupervised components. This hybrid approach often yields better accuracy and improved stability during training.

2.1.3 Practical Implementations

The implementations have been carried out in Google Colab using the Keras libraries. Below, we present a few examples, and the reader is encouraged to explore the accompanying Google Colab notebook at:

`https://tinyurl.com/ya78jb7x`

The aforementioned page illustrates both supervised and unsupervised constructions of the inverse mapping for solving polynomial equations.

The direct method implementation We shall exemplify this method for the particular case of the polynomial function $P(x) = x^3 - 2x^2 + 4x - 8$, whose graph is captured by Fig. 2.1. We note that $P(x)$ has only one root, $x^* = 2$.

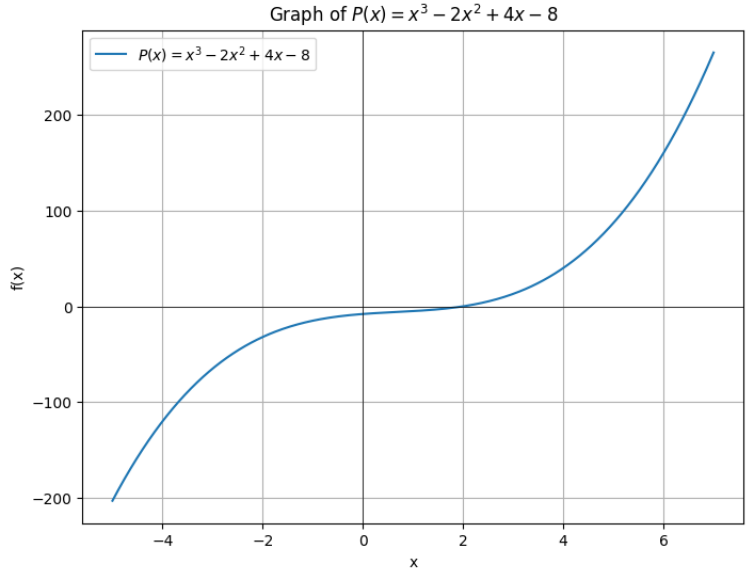


Figure 2.1: Graph of $P(x) = x^3 - 2x^2 + 4x - 8$ generated using Python.

After importing TensorFlow for building the model, NumPy for numerical operations, and Matplotlib for plotting

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
```

a simple feed-forward network with one hidden layer (with 20 neurons and tanh activation function) and a single output neuron is built:

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(20, activation='tanh', input_shape=(1,)),
    tf.keras.layers.Dense(1)
])
```

The loss function uses the dummy target y_{true}, while measuring the square of the network output

```
def custom_loss(y_true, y_pred):
    return tf.reduce_mean(tf.square(f(y_pred)))
```

The model is compiled with the Adam optimizer and trained for 300 epochs on a single dummy input, $t = 1$, since the loss is computed solely from the network's output:

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.02)
model.compile(optimizer=optimizer, loss=custom_loss)
t = 1.0
dummy_input = np.array([[t]])
dummy_target = np.array([[0.0]])
model.fit(dummy_input, dummy_target, epochs=300, verbose=2)
```

The trained network is queried with the dummy input, and its output is taken as the approximate solution of the equation

```
x_solution = model(tf.constant([[t]])).numpy()[0, 0]
print(f"Approximate solution: x = {x_solution}")
```

After 300 epochs the loss becomes practically a zero machine number, obtaining the root $x^* = 2$:

```
Epoch 300/300
1/1 - 0s - 61ms/step - loss: 0.0000e+00
Approximate solution: x = 2.0
```

Once a real root was found, $x = 2$, we can obtain the quotient polynomial $G(x)$ such that $P(x) = (x - 2)G(x)$ by implementing a synthetic division:

```
def synthetic_division(coeffs, c):
    quotient = [coeffs[0]]
    for coef in coeffs[1:]:
        quotient.append(coef + c * quotient[-1])
    remainder = quotient.pop()  # Last element is the remainder
    return quotient, remainder

# Given polynomial P(x) = x^3 - 2*x^2 + 4*x - 8
coeffs = [1, -2, 4, -8]
# Dividing by (x - 2), so c = 2
```

```
c = 2

quotient, remainder = synthetic_division(coeffs, c)
print("Quotient coefficients:", quotient)
print("Remainder:", remainder)
```

Running the code yields the output

```
Quotient coefficients: [1, 0, 4]
Remainder: 0
```

which shows that the quotient is $G(x) = x^2 + 4$.

We next aim to solve the equation $x^2 + 4 = 0$, by representing a complex number in polar form. We use a neural network that accepts a fixed input t (dummy input) and produces a two-dimensional output $f_\theta(t) = (\rho_\theta(t), \varphi_\theta(t))$. We then interpret the network's output as the complex number $x_\theta = \rho_\theta\, e^{i\varphi_\theta}$.

For the polynomial $G(x) = x^2 + 4$, the coefficients are $a_2 = 1$, $a_1 = 0$, and $a_0 = 4$. In this case the loss function is given by

$$\mathcal{L}(\theta) = |G(x)|^2 = G(x)\overline{G(x)} = a_0^2 + 2\,a_0\,a_2\,\rho_\theta^2\cos(2\varphi_\theta) + \rho_\theta^4,$$

which, with $a_0 = 4$ and $a_2 = 1$, becomes

$$\mathcal{L}(\theta) = 16 + 8\,\rho_\theta^2\cos(2\varphi_\theta) + \rho_\theta^4.$$

The goal is to minimize this loss with respect to the network parameters θ. The code is given below:

```
# Polynomial coefficients for P(x) = x^2 + 4 = 0
# Here, a2 = 1 and a0 = 4.
a2 = 1.0
a0 = 4.0
def custom_loss(y_true, y_pred):
    # Extract the two outputs: first is rho, second is varphi.
    rho = y_pred[:, 0]
    varphi = y_pred[:, 1]
    # Compute the loss as described.
    loss = a0**2 + 2 * a0 * a2 * tf.square(rho) * tf.cos(2 * varphi)\\
            + tf.pow(rho, 4)
    return tf.reduce_mean(loss)

# Build the neural network model.
# The model takes a dummy input (a scalar) and outputs
#a two-dimensional vector [rho, varphi].
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1,)),
    tf.keras.layers.Dense(20, activation='tanh'),
```

```
    tf.keras.layers.Dense(2)  # Outputs: [rho, varphi]
])

# Compile the model with the custom loss and an Adam optimizer.
optimizer = tf.keras.optimizers.Adam(learning_rate=0.01)
model.compile(optimizer=optimizer, loss=custom_loss)

# Since the loss depends solely on the network's output,
#we use a dummy input.
t = 1.0  # The input can be any fixed number.
dummy_input = np.array([[t]])
dummy_target = np.array([[0.0, 0.0]])
# Dummy target (not used in the loss)

# Train the model.
model.fit(dummy_input, dummy_target, epochs=500, verbose=2)

# Extract the outputs from the trained model.
output = model.predict(dummy_input)
rho = output[0, 0]
varphi = output[0, 1]
# Construct the approximate complex solution in polar form.
solution1 = rho * np.exp(1j * varphi)
print("Approximate complex solution:", solution1)
solution2 = rho * np.exp(-1j * varphi)
print("Approximate complex solution:", solution2)
```

Running the code for 500 epochs, using the dummy input $t = 1$, and a one-hidden layer neural network with 20 neurons, we obtain

```
Epoch 500/500
1/1 - 0s - 59ms/step - loss: 0.0000e+00
1/1  0s 90ms/step
Approximate complex solution: (8.742278e-08+2j)
Approximate complex solution: (8.742278e-08-2j)
```

Neglecting the zero machine number in the real part, we obtained two complex conjugate roots, $\pm 2i$.

Inverse function approximation by supervised learning In this section, we present a Keras implementation that approximates the inverse function of a polynomial by supervised learning. The goal is to learn the inverse mapping $f^{-1}(y)$ for the function

$$f(x) = x^3 - 2x^2 + 4x - 8.$$

First we show that f is invertible by proving that it is increasing. To show that f is an increasing function, we first compute its derivative, $f'(x) = 3x^2 - 4x + 4$. The

quadratic $f'(x)$ is positive for all x if its discriminant is negative. Since $\Delta = -32 < 0$, the quadratic $3x^2 - 4x + 4$ has no real roots and, because its leading coefficient is positive, we conclude that $f'(x) > 0$, for all $x \in \mathbb{R}$. Thus, $f(x)$ is strictly increasing over the entire real line, and hence there is an inverse $f^{-1} : \mathbb{R} \to \mathbb{R}$.

The following code snippets outline the process of finding the inverse in a supervised way.

We start by defining the function and then we generate training data:

```
# Define the function f(x)
def f(x):
    return x**3 - 2*x**2 + 4*x - 8

# Generate training data
x_train = np.linspace(-4, 5, 1000).reshape(-1, 1)  # Sample x values
y_train = f(x_train)  # Compute corresponding y values
```

Here, we define the polynomial $f(x) = x^3 - 2x^2 + 4x - 8$ and generate 1000 training examples with x values in the interval $[-4, 5]$. The corresponding y values are computed using $f(x)$.

We build the following neural network model:

```
# Build the neural network
model = Sequential([
    Dense(80, activation='tanh', input_dim=1),
    # Hidden layer with 80 neurons
    Dense(1)  # Output layer
])
```

A feed-forward neural network is constructed using Keras. It consists of:

- A hidden layer with 80 neurons and `tanh` activation (the input dimension is 1).
- An output layer with a single neuron that yields the predicted value.

To compile and train the model we use the code:

```
# Compile the model with mean squared error loss
model.compile(optimizer=Adam(learning_rate=0.01), loss='mse')

# Train the model
model.fit(y_train, x_train, epochs=100, batch_size=32, verbose=1)
```

In order to learn the inverse function f^{-1}, we swap the roles of inputs and targets: the network is given y as input and trained to output x. The model is compiled with the Adam optimizer (learning rate 0.01) and the mean squared error (MSE) loss function, then trained for 100 epochs with a batch size of 32.

The next snippet is testing and plotting the inverse function:

```
# Generate a range of y values for testing
y_vals = np.linspace(-100, 100, 1000).reshape(-1, 1)

# Use the trained model to predict the inverse
#function values (f^{-1}(y))
x_preds = model.predict(y_vals)

# Plot the inverse function
plt.figure(figsize=(8, 6))
plt.plot(0, 2, "ro", markersize=8)  # Plot the point (0,2) in red
plt.plot(y_vals, x_preds, label="$f^{-1}(y)$", linewidth=2)
plt.title("Inverse Function $f^{-1}(y)$", fontsize=16)
plt.xlabel("$y$", fontsize=14)
plt.ylabel("$x = f^{-1}(y)$", fontsize=14)
plt.grid(alpha=0.3)
plt.legend(fontsize=12)
plt.show()
```

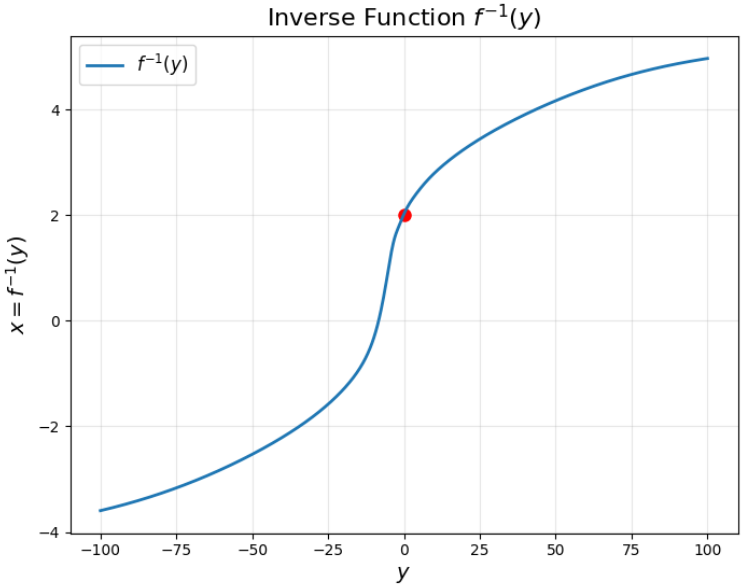


Figure 2.2: Graph of the inverse of the function $f(x) = x^3 - 2x^2 + 4x - 8$ obtained by supervised learning.

A range of 1000 y values from -100 to 100 is generated. The trained model is used to predict the corresponding x values, thereby approximating the inverse function $f^{-1}(y)$. The graph of the inverse function is plotted, see Fig. 2.2, and the point $(0, 2)$ is highlighted in red. Following the red dot yields $f^{-1}(0) = 2$, so the solution of $f(x) = 0$ is $x = 2$.

Inverse function approximation by unsupervised learning We consider again the polynomial $f(x) = x^3 - 2x^2 + 4x - 8$. Our goal is to learn its inverse function, $f^{-1}(y)$, using a neural network and an unsupervised learning method, whose loss is based on the functional equation $f(f^{-1}(y)) = y$.

We start by generating the y-values needed for training. Specifically, we create y-values in the range $[-50, 50]$ with 1000 evenly spaced points. These values act as inputs to our network because we want the network to learn how to produce x such that $f(x) = y$:

```
y_train = np.linspace(-50, 50, 1000).reshape(-1, 1)
# Sample y values
```

A feed-forward network is constructed with two hidden layers (50 neurons in the first layer, 20 in the second), each using the hyperbolic tangent activation function. The network outputs a single value, which is interpreted as $x \approx f^{-1}(y)$:

```
model = Sequential([
    Dense(50, activation='tanh', input_dim=1),
     # Hidden layer with 50 neurons
    Dense(20, activation='tanh'),
     # Another hidden layer
    Dense(1)
     # Output layer
])
```

Next we define the custom loss function. This loss is "unsupervised" because we do not have direct (y, x) pairs. Instead, for each input y, the model produces a predicted x. Then $f(x)$ should equal y. Consequently, the loss measures $\big(f(\text{model}(y)) - y\big)^2$ and drives the network to learn the inverse mapping:

```
def unsupervised_loss(y_true, y_pred):
    # Compute f(NN(y)) - y
    return tf.reduce_mean(tf.square(f(y_pred) - y_true))
```

The model is compiled with the Adam optimizer (learning rate of 0.01) and the custom unsupervised loss function:

```
model.compile(optimizer=Adam(learning_rate=0.01),
      loss=unsupervised_loss)
```

Then we feed the same array `y_train` as both inputs and targets because the loss function already handles the relationship $f(\text{model}(y)) \approx y$. Training is performed for 100 epochs with a batch size of 32:

```
model.fit(y_train, y_train, epochs=100, batch_size=32, verbose=1)
```

We create a new range of y-values for testing. The model predicts $x \approx f^{-1}(y)$ for these values:

```
y_vals = np.linspace(-50, 50, 1000).reshape(-1, 1)
x_preds = model.predict(y_vals)
```

Finally, we visualize $f^{-1}(y)$ by plotting the predicted x-values against the y-values. This provides a graphical representation of the learned inverse function, see Fig. 2.3.

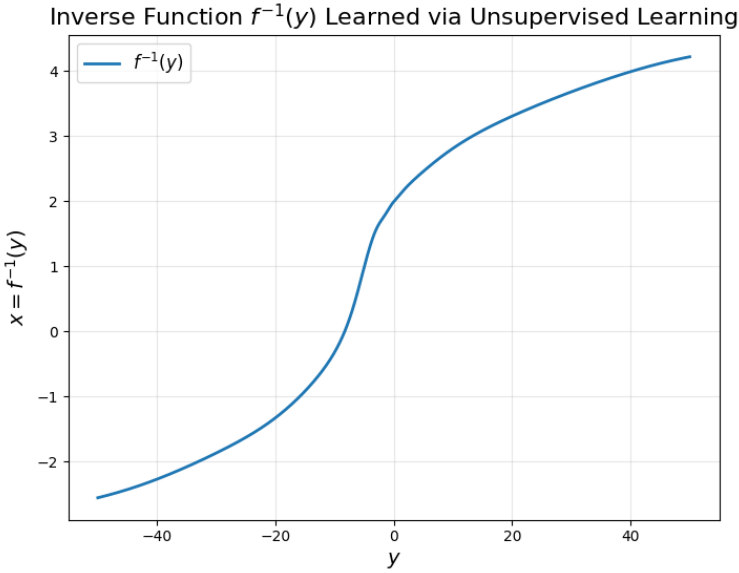


Figure 2.3: Graph of the inverse of the function $f(x) = x^3 - 2x^2 + 4x - 8$ constructed by unsupervised learning.

2.1.4 Exercises

Exercise 2.1.1 Let

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_0$$

be a polynomial with real coefficients. Show that any real root x of the polynomial $p(x)$ satisfies

$$|x| \leq \max\left\{1, \frac{|a_{n-1}| + |a_{n-2}| + \cdots + |a_0|}{|a_n|}\right\}.$$

Note that this inequality also holds for complex roots if $|x|$ denotes the modulus of the root.

Exercise 2.1.2 Suppose that

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 = 0$$

is a polynomial equation. Prove that any root x of this equation satisfies the bound $|x| \leq M$, where

$$M = 2 \max \left\{ \left(\frac{|a_i|}{|a_n|} \right)^{\frac{1}{n-i}} \; ; \; 0 \leq i < n \right\}.$$

Exercise 2.1.3 Let $f(z) = a_n z^n + \cdots + a_0$ be a polynomial with complex coefficients. Suppose that for some $r > 0$ and for some $k \in \{0, 1, \ldots, n\}$ the inequality

$$|a_k| \, r^k > \sum_{j \neq k} |a_j| \, r^j$$

holds. Prove that $f(z)$ has exactly k roots (counted with multiplicity) inside the open disk $B(0, r) = \{z \in C : |z| < r\}$.

Exercise 2.1.4 Let $P(x)$ be a polynomial of degree n with k real roots, denoted $x_1, x_2, \ldots, x_k$.

(a) Explain how to extend a neural network method for finding a *single* real root to the case where the polynomial has *multiple* real roots.

(b) Describe how one might construct a neural network $f_\theta : \mathbb{R} \to \mathbb{R}^k$ so that it outputs a vector containing approximations of the k real roots. Include a discussion of how the loss function could be formulated when the ordering of the roots is not predetermined.

(c) Alternatively, propose a scheme in which the real axis is divided into k intervals, each containing exactly one real root. How could separate neural networks (or a single network with k output branches) be employed to approximate each root independently?

(d) Discuss why a *permutation-invariant* loss function

$$\mathcal{L}(\theta) \; = \; \min_{\pi \in S_k} \sum_{i=1}^{k} \left| \hat{x}_{\pi(i)} - x_i \right|^2$$

(where S_k is the group of permutations with k elements, $\hat{x}_i$ are predicted roots and x_i are true roots) may be necessary. Illustrate how this loss ensures that the network does not rely on a fixed order of the roots.

Exercise 2.1.5 The Lambert function $W(x)$ is the inverse of the function $f(x) = xe^x$, namely it satisfies the equation $W(x)\, e^{W(x)} = x$, for all x real.

(a) Use an unsupervised learning method to find and draw the inverse of the function $W(x)$.

(b) Using $W(0) = 0$ and $W(-1/e) = -1$, use a hybrid approach to find the inverse of W.

2.2 Solving Functional Equations

A *functional equation* is a relationship that specifies how a function's values at different points relate to each other. This imposes an *algebraic constraint* on all possible functions in a chosen class (here, continuous functions). In such problems, the constraint

typically narrows down the immense space of continuous functions to a small *family* of solutions, which often depends on a few parameters.

In the following sections, we will examine a few simple, well-known functional equations and explore how neural networks can be employed to solve them. The provided examples illustrate how a neural network can be trained to learn the solution of a simple functional equation by enforcing an algebraic constraint and initial conditions through loss functions.

2.2.1 The Cauchy Functional Equation

A classical example of a functional equation is

$$f(x+y) = f(x) + f(y), \tag{2.2.1}$$

with $f : \mathbb{R} \to \mathbb{R}$, which is commonly known as the *Cauchy functional equation*. When f is assumed to be continuous (or monotonic), it can be shown that the only solutions are linear functions of the form

$$f(x) = cx,$$

where c is a constant. However, if no regularity condition (such as continuity or monotonicity) is imposed, there exist highly pathological solutions that cannot be expressed in a simple closed form, and hence will not be treated here.

We aim to solve the classical Cauchy functional equation (2.2.1) subject to the initial condition

$$f(1) = c,$$

where c is a given real number. Our goal is to recover the linear behavior of the solution using a neural network approach. We treat the problem as a *constraint-informed learning task*, where the constraint is given by equation (2.2.1).

Neural Network Model We define a neural network $f_\theta : \mathbb{R} \to \mathbb{R}$, with trainable parameters θ, intended to approximate the unknown function f. For example, f_θ can be implemented as a simple feed-forward network with a few dense layers. Once trained, the network output should satisfy the functional identity

$$f_\theta(x+y) \approx f_\theta(x) + f_\theta(y), \quad \text{and} \quad f_\theta(1) \approx c.$$

Generating Data Points Instead of using standard supervised learning with input-output pairs, we construct data that enforce the constraint structure:

- *Sampling domain:* Generate values $\{x_i\}_{i=1}^N$ and $\{y_j\}_{j=1}^N$ from an interval such as $[-1, 1]$ or $[0, 2]$.
- *Constraint condition:* For each pair (x_i, y_j), enforce that $f(x_i+y_j) = f(x_i)+f(y_j)$.
- *Initial condition:* Include the condition $f(1) = c$ into the loss function.

Defining the Loss Function We define a total loss that penalizes violations of both the functional identity and the initial condition:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{eq}}(\theta) + \lambda_{\text{IC}}\, \mathcal{L}_{\text{IC}}(\theta),$$

where the equation loss is

$$\mathcal{L}_{\text{eq}}(\theta) = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} \left[f_\theta(x_i + y_j) - (f_\theta(x_i) + f_\theta(y_j)) \right]^2,$$

and the initial condition loss is

$$\mathcal{L}_{\text{IC}}(\theta) = \left(f_\theta(1) - c \right)^2.$$

Here, $\lambda_{\text{IC}} > 0$ is a hyperparameter that controls how strongly the initial condition is enforced relative to the constraint equation.

Training The parameters θ are updated via gradient descent using an optimizer such as Adam or stochastic gradient descent (SGD), to minimize the total loss:

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta).$$

The network gradually adjusts to match the Cauchy functional constraint on sampled points while also approximating the initial value $f(1) = c$.

Reconstructing the solution Once training is complete, the resulting network $f_{\theta^*}(x)$ approximates the solution to the functional equation. Under the assumption of continuity, we expect this solution to closely match the linear function $f(x) = cx$. One can verify this by evaluating the network at several values of x and checking whether $f_{\theta^*}(x) \approx cx$, or just plotting the network output and the linear function on the same graph and observe that the graphs are (almost) identical.

2.2.2 A Simple Multiplicative Functional Equation

Another classical functional equation, closely tied to the logarithmic function, is

$$f(xy) = f(x) + f(y), \tag{2.2.2}$$

where $f : (0, \infty) \to \mathbb{R}$ is a continuous function. It is well known that the only continuous solutions to this equation are of the form

$$f(x) = c \ln(x),$$

where c is a real constant. This identity captures the defining property of the natural logarithm, and any other continuous function satisfying (2.2.2) differs from it only by a constant scaling factor.

In this section, we demonstrate how to approximate the solution of equation (2.2.2) using a neural network. The approach is similar to that used for the Cauchy equation and relies on enforcing the functional identity via a loss function defined on a set of sampled input values.

Neural network model We define a neural network with positive inputs,

$$f_\theta : (0, \infty) \to \mathbb{R},$$

parameterized by weights θ, designed to approximate the function $f(x)$. The network is built using dense layers with smooth activation functions (e.g., `tanh`) to ensure continuity and differentiability.

Constructing the dataset To enforce the identity (2.2.2), we proceed as follows:

- *Sampling:* Generate values x_i, y_j uniformly from a fixed interval, ensuring all values remain positive.
- *Constraint generation:* For each pair (x_i, y_j), the identity imposes

  $$f(x_i y_j) = f(x_i) + f(y_j).$$

  These values are used to construct the loss.
- *Initial condition:* We consider the initial condition $f(e) = c$.

Loss function We define a loss that penalizes the deviation from the multiplicative identity:

$$\mathcal{L}_{\text{eq}}(\theta) = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} \left[f_\theta(x_i y_j) - (f_\theta(x_i) + f_\theta(y_j)) \right]^2 .$$

If we choose to impose the initial condition $f(e) = c$, we can add:

$$\mathcal{L}_{\text{IC}}(\theta) = \left[f_\theta(e) - c \right]^2 ,$$

where e denotes the base of the natural logarithm. Then the total loss becomes:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{eq}}(\theta) + \lambda \mathcal{L}_{\text{IC}}(\theta),$$

with $\lambda > 0$ a weighting hyperparameter.

Training the network We use a gradient-based optimizer such as Adam to minimize $\mathcal{L}(\theta)$, adjusting the weights of the network so that the output increasingly satisfies the identity:

$$f_\theta(xy) \approx f_\theta(x) + f_\theta(y), \quad \text{and} \quad f_\theta(e) \approx c.$$

Interpreting the result Once training converges, the neural network approximates a function $f_\theta(x)$ that closely satisfies the identity (2.2.2). Because of the uniqueness result (under continuity), we expect:

$$f_\theta(x) \approx c \ln(x),$$

for some constant c depending on the initialization. To verify this, one may compare $f_\theta(x)$ to $c \ln(x)$ on a grid of values and compute the error, or just plot on the same graph both the output of the network and the logarithmic function and note that they have the same graphs.

2.2.3 Another Simple Functional Equation

Another classic functional equation is

$$f(x+y) = f(x)\, f(y),$$

which describes the multiplicative behavior of the function with respect to addition. When $f : \mathbb{R} \to \mathbb{R}$ is assumed to be continuous and nonzero, it is well known that the only solutions are exponential functions of the form

$$f(x) = a^x,$$

where a is a fixed nonzero constant. This functional equation thus characterizes the exponential function up to base a, and captures a fundamental identity of exponentiation.

In this section, we use a neural network approach to approximate a solution to this functional equation. Specifically, we will recover the function $f(x) = a^x$ by enforcing the identity $f(x+y) = f(x)f(y)$ through a suitable loss function, and incorporate the initial condition $f(1) = a$ into training.

Neural Network Model Let $f_\theta : \mathbb{R} \to \mathbb{R}$ be a feed-forward neural network with trainable parameters θ, designed to approximate the function $f(x)$. The network should satisfy:

$$f_\theta(x+y) \approx f_\theta(x)\, f_\theta(y), \quad \text{and} \quad f_\theta(1) \approx a.$$

Sampling and Data Preparation To train the model, we construct synthetic data that encode the identity:

- *Domain sampling:* Choose N values $\{x_i\}$, $\{y_j\}$ from an interval such as $[-2, 2]$, forming N^2 pairs (x_i, y_j).
- *Functional constraint:* For each pair, enforce the condition:

$$f(x_i + y_j) = f(x_i) \cdot f(y_j).$$

- *Initial condition:* Include the requirement $f(1) = a$ as a constraint in the loss.

Loss Function We define a loss that penalizes deviation from the multiplicative property:

$$\mathcal{L}_{\text{eq}}(\theta) = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} \left[f_\theta(x_i + y_j) - f_\theta(x_i) \cdot f_\theta(y_j) \right]^2 .$$

To enforce the initial condition $f(1) = a$, we define:

$$\mathcal{L}_{\text{IC}}(\theta) = \left[f_\theta(1) - a \right]^2 .$$

The total loss becomes:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{eq}}(\theta) + \lambda\, \mathcal{L}_{\text{IC}}(\theta),$$

where $\lambda > 0$ balances the equation constraint and the initial condition.

Training Procedure Using a standard optimizer such as Adam, we minimize $\mathcal{L}(\theta)$ to obtain

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta).$$

After training, the network $f_{\theta^*}(x)$ should approximate a function satisfying both the multiplicative property and the initial condition.

Result Interpretation Under the assumption of continuity, we know that the solution must be of the form

$$f(x) = a^x.$$

Hence, if the network is well trained, the learned function $f_{\theta^*}(x)$ should behave like a^x over the training domain. One can verify this numerically or visually by plotting $f_{\theta^*}(x)$ versus the true function a^x and computing the approximation error.

The aforementioned examples demonstrates how neural networks can approximate the continuous solution to a functional equation by learning the algebraic structure encoded in the function identity. By encoding this structure in a loss function and using neural optimization, we can recover the solution through data-driven learning. The reader can try other examples of functional equations proposed in the Exercises section.

2.3 Practical Implementations

The next few sections are based on Keras implementations done in Google Colab and can be accessed at

`https://tinyurl.com/2jw9dnmb`

They contain two of the aforementioned functional equations. Based on the information presented here, the reader should be able to extend the code to other examples.

2.3.1 Solving Cauchy Functional Equation with NN

In this section, we shall use a shallow feed-forward neural network to approximate the solution of the Cauchy functional equation

$$f(x + y) = f(x) + f(y),$$

subject to the initial condition $f(1) = 3$. As established earlier, the continuous solutions to this equation are linear functions of the form $f(x) = cx$. Therefore, the expected solution for this case is $f(x) = 3x$.

Data Generation To approximate the solution using a neural network, we first generate 1000 random values for each $\{x_i\}$ and $\{y_i\}$ in the interval $[-2, 2]$:

```
x_samples = np.random.uniform(-2, 2, 1000).reshape(-1, 1)
y_samples = np.random.uniform(-2, 2, 1000).reshape(-1, 1)
xy_samples = x_samples + y_samples
```

These samples represent the inputs used to enforce the functional equation.

Neural Network Architecture We define a simple feed-forward neural network with one hidden layer, having 10 neurons and ReLU activations:

```
model = Sequential([
    Dense(10, activation='relu', input_dim=1),
    Dense(1)
])
```

The simplicity of this network is suggested by the linearity of the solution. This network maps a scalar input x to a scalar output $f_\theta(x)$, where θ represents the trainable parameters.

Loss Function Design The total loss combines two components:

- A *functional equation loss* that enforces $f(x+y) \approx f(x) + f(y)$,
- A *boundary condition loss* that ensures $f(1) \approx 3$.

The loss function is implemented as:

```
def custom_loss(y_true, y_pred):
    f_x = model(tf.constant(x_samples, dtype=tf.float32))
    f_y = model(tf.constant(y_samples, dtype=tf.float32))
    f_xy = model(tf.constant(xy_samples, dtype=tf.float32))
    functional_loss = tf.reduce_mean(tf.square(f_xy - f_x - f_y))
    f_1 = model(tf.constant([[1.0]], dtype=tf.float32))
    boundary_loss = tf.square(f_1 - 3.0)
    return functional_loss + boundary_loss
```

Training the Model The model is compiled and trained for 30 epochs using the Adam optimizer with a batch size of 30. Dummy inputs are provided since the actual input is handled inside the loss function:

```
model.compile(optimizer=Adam(learning_rate=0.01), loss=custom_loss)

dummy_input = np.zeros_like(x_samples)
dummy_target = np.zeros_like(x_samples)

model.fit(dummy_input, dummy_target, epochs=30, batch_size=32,
 verbose=1)
```

Visualization After training, we compare the neural network approximation $f_\theta(x)$ with the true solution $f(x) = 3x$ by plotting both functions over the interval $[-2, 2]$:

```
x_plot = np.linspace(-2, 2, 1000).reshape(-1, 1)
f_x_plot = model.predict(x_plot)
true_f_x = 3 * x_plot
plt.figure(figsize=(8, 6))
```

```
plt.plot(x_plot, f_x_plot, label="$f(x)$ (Learned)", linewidth=2)
plt.plot(x_plot,true_f_x, label="$3x$(True Function)",linewidth=2,
         color='red')
plt.title("Comparison of Learned $f(x)$ and True $3x$", fontsize=16)
plt.xlabel("$x$", fontsize=14)
plt.ylabel("$f(x)$", fontsize=14)
plt.grid(alpha=0.3)
plt.legend(fontsize=12)
plt.show()
```

The resulting graph should confirm that the neural network has successfully learned the linear function $f(x) = 3x$, consistent with the theoretical solution. The graph, given in Fig. 2.4, shows a perfect overlap of the theoretical solution and the learned solution.

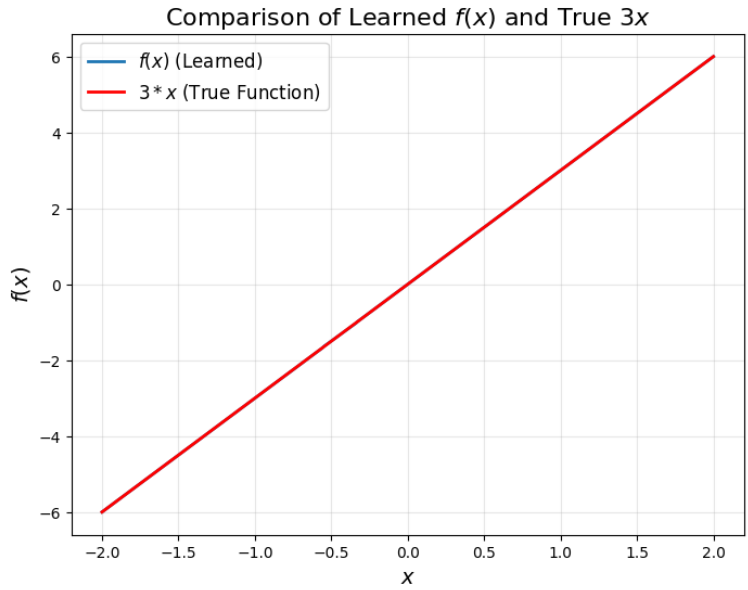


Figure 2.4: The learned solution $f_{\theta^*}(x) \approx 3x$ with a high degree of accuracy.

2.3.2 Solving $f(x+y) = f(x)\,f(y)$ with Neural Nets

We revisit the multiplicative functional equation

$$f(x+y) = f(x)f(y),$$

which, under the assumption of continuity and positivity, admits exponential solutions of the form

$$f(x) = a^x.$$

In this example, we use a neural network to learn the solution corresponding to the initial condition $f(1) = 2$, which implies $f(x) = 2^x$.

Sampling Input Pairs We generate random samples $x_i, y_i \in [-2, 2]$ and compute their sums $x_i + y_i$, which will serve as input data enforcing the functional equation:

```
x_samples = np.random.uniform(-2, 2, 1000).reshape(-1, 1)
y_samples = np.random.uniform(-2, 2, 1000).reshape(-1, 1)
xy_samples = x_samples + y_samples
```

Neural Network Architecture A feed-forward neural network with two hidden layers is constructed:

```
model = Sequential([
    Dense(100, activation='relu', input_dim=1),
    Dense(50, activation='relu'),
    Dense(1)
])
```

This model approximates the function $f(x)$ through its output layer.

Custom Loss Function We define a loss that penalizes deviation from the functional identity $f(x + y) = f(x)f(y)$ and enforces the initial condition $f(1) = 2$:

```
def custom_loss(y_true, y_pred):
    f_x = model(tf.constant(x_samples, dtype=tf.float32))
    f_y = model(tf.constant(y_samples, dtype=tf.float32))
    f_xy = model(tf.constant(xy_samples, dtype=tf.float32))
    functional_loss = tf.reduce_mean(tf.square(f_xy - f_x * f_y))

    f_1 = model(tf.constant([[1.0]], dtype=tf.float32))
    boundary_loss = tf.square(f_1 - 2.0)

    return functional_loss + boundary_loss
```

Training the Network The model is compiled with the Adam optimizer and trained with dummy data (unused inputs and targets, since the loss is internally computed):

```
model.compile(optimizer=Adam(learning_rate=0.01), loss=custom_loss)

dummy_input = np.zeros_like(x_samples)
dummy_target = np.zeros_like(x_samples)

model.fit(dummy_input, dummy_target, epochs=50, batch_size=32,verbose=1)
```

Visualization and Evaluation After training, we evaluate the learned function on a dense grid and compare it to the true exponential function 2^x:

```
x_plot = np.linspace(-2, 2, 1000).reshape(-1, 1)
f_x_plot = model.predict(x_plot)
true_f_x = 2 ** x_plot
```

The result is visualized below:

```
plt.figure(figsize=(8, 6))
plt.plot(x_plot, f_x_plot, label="$f(x)$ (Learned)", linewidth=2)
plt.plot(x_plot, true_f_x, label="$2^x$ (True Function)", linewidth=2,
color='red')
plt.title("Comparison of Learned $f(x)$ and True $2^x$", fontsize=16)
plt.xlabel("$x$")
plt.ylabel("$f(x)$")
plt.grid(alpha=0.3)
plt.legend(fontsize=12)
plt.show()
```

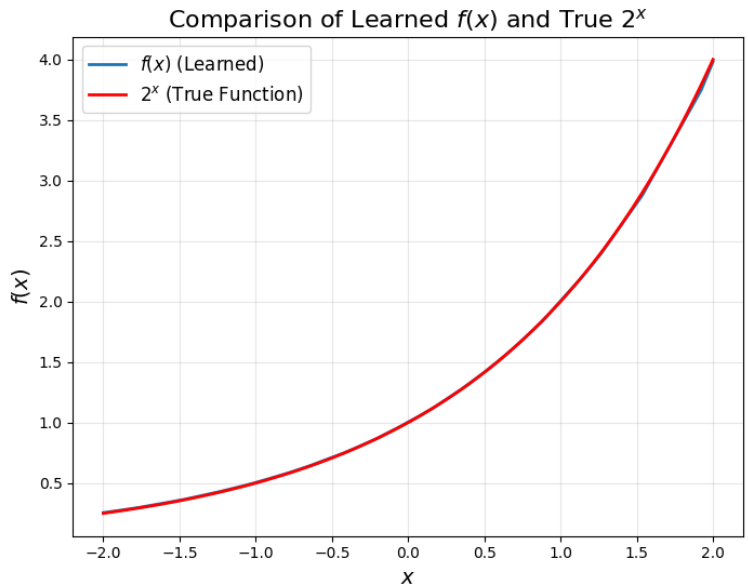


Figure 2.5: The learned solution $f_{\theta^*}(x) \approx 2^x$ with a high degree of accuracy.

This plot demonstrates the close match between the neural network output and the true exponential function, validating that the model has effectively learned the functional identity, see Fig. 2.5.

2.3.3 Exercises

Exercise 2.3.1 Let $f : (0, \infty) \to \mathbb{R}$ be a continuous function that satisfies the functional equation

$$f(xy) = f(x)\, f(y), \quad \forall x, y > 0.$$

(*a*) Use a feed-forward neural network to approximate the function $f(x)$ satisfying the functional identity above and the initial condition $f(2) = 4$.

(*b*) Show that the learned function approximates the quadratic solution $f(x) = x^2$. For this purpose, plot both the neural network approximation and the exact quadratic function on the same graph over an interval such as $x \in [0.5, 3]$.

Exercise 2.3.2 A related functional equation is

$$f\left(\frac{x}{y}\right) = \frac{f(x)}{f(y)},$$

studied for $x, y > 0$. Under suitable regularity conditions such as continuity, the only real-valued solutions are of the form $f(x) = x^c$, where c is a constant.

(*a*) Use a feed-forward neural network to approximate a continuous function $f(x)$ satisfying the identity above and the initial condition $f(2) = 4$.

(*b*) Show that the learned function approximates the quadratic solution $f(x) = x^2$. Plot both the neural network output and the exact solution on the same graph to compare their behaviors over a selected domain (e.g., $x \in [0.5, 3]$).

Exercise 2.3.3 Consider the functional equation $f\left(x^y\right) = [f(x)]^y$, defined for $x > 0$ and $y \in \mathbb{R}$. Under mild regularity assumptions such as continuity and positivity of $f(x)$, the only solutions are of the form $f(x) = x^c$, for some real constant c.

(*a*) Use a feed-forward neural network to approximate a continuous function $f(x)$ satisfying the identity above and the initial condition $f(2) = 4$.

(*b*) Demonstrate that the learned function approximates the quadratic solution $f(x) = x^2$. Visualize this by plotting both the neural network approximation and the exact solution on the same graph over a suitable domain.

Exercise 2.3.4 Let $f : \mathbb{R} \to \mathbb{R}$ be a continuous function satisfying the functional equation

$$f(x + y) = f(x) - f(y), \quad \text{for all } x, y \in \mathbb{R}.$$

(*a*) Show algebraically that the only continuous solution is the zero function, $f(x) = 0$, for all $x \in \mathbb{R}$.

(*b*) Construct a feed-forward neural network to approximate such a function f, enforcing the identity $f(x + y) = f(x) - f(y)$ as a loss constraint.

(*c*) Train the network using random samples $x_i, y_i \in [-2, 2]$, and observe whether the network converges to the zero function.

(*d*) Plot the output $f(x)$ of the trained model over an interval and verify that it approximates the constant zero function.

Exercise 2.3.5 Consider the functional equation

$$f(x+y) = \frac{1}{2}\left(f(x) + f(y)\right), \quad \text{for all } x, y \in \mathbb{R},$$

with the initial condition $f(1) = 1$. It can be shown that the only continuous solution to this equation is the constant function $f(x) = 1$ for all $x \in \mathbb{R}$.

(a) Use a feed-forward neural network to approximate a continuous function $f(x)$ satisfying the functional identity above.

(b) Enforce the initial condition $f(1) = 1$ as part of the loss function.

(c) Train the network using random pairs $x_i, y_i \in [-2, 2]$, and verify that the learned function approximates the constant solution $f(x) = 1$.

(d) Plot the learned function $f(x)$ and compare it to the true constant function over the interval $[-2, 2]$.

2.4 Solving Integral Equations

In this section we shall discuss a neural network approach of integral equations of the first and of the second kind. Similar considerations can be used to approach other types of integral equations.

2.4.1 Integral Equations of the First Kind

An integral equation of the first kind is a type of equation in which the unknown function appears only under the integral sign. A standard example is the *Fredholm equation* of the first kind:

$$f(x) = \int_a^b K(x,t)\,\varphi(t)\,dt, \tag{2.4.3}$$

where $f(x)$ is a known function, $\varphi(t)$ is the unknown function to be determined, and $K(x,t)$ is a given function called the *kernel* of the integral equation. The main task is to determine $\varphi(t)$ given the data $f(x)$ and $K(x,t)$.

Since the output of a neural network is continuous, we shall consider the particular case where the external function $f(x)$ and the kernel $K(x,t)$ are both continuous. In this setting, it is natural to seek a solution $\varphi : [a,b] \to \mathbb{R}$ that is also continuous.

It is worth noting that in general, integral equations of the first kind with continuous kernel $K(x,t)$ and continuous function $f(x)$ do not guarantee the existence of a unique solution for the unknown function $\varphi(t)$, even when φ is assumed to be continuous. This is because Fredholm integral equations of the first kind are typically *ill-posed* in the sense of Hadamard.[1]

[1]This means that a solution may not exist for all choices of f; if a solution exists, it may not be unique; and even when uniqueness holds, the solution may not depend continuously on the data f, meaning that small errors in f can lead to large deviations in the solution φ.

Despite the ill-posedness of first-kind integral equations, using neural networks represents an experimental tool for finding a continuous solution $\varphi(t)$, if exists, which otherwise might have been difficult to spot by direct methods.

We now describe how a feed-forward neural network can be used to approximate a solution to a Fredholm integral equation of the first kind, where $f(x)$ is a known continuous function, $K(x,t)$ is a known continuous kernel, and $\varphi(t)$ is the unknown continuous function to be recovered. We work under the assumption that the integral equation (2.4.3) is not ill-posed.

Neural network approximation of the solution We model the unknown function $\varphi(t)$ using a neural network $\varphi(t) \approx f_\theta(t)$, where f_θ is a feed-forward neural network with trainable parameters θ. The input to the network is $t \in [a,b]$. The goal is to determine the optimal θ such that $f_\theta(t)$ closely approximates the true solution.

Unsupervised learning framework This problem is addressed via an *unsupervised learning* framework, since we do not have access to input-output training pairs $(t_i, \varphi(t_i))$. Instead, we use the known function $f(x)$ and the structure of the integral equation to define a loss function.

First, we approximate the left-hand side $f(x)$ using numerical quadrature applied to the neural network output as

$$f_\theta^{\text{approx}}(x) := \int_a^b K(x,t)\, f_\theta(t)\, dt.$$

Then, we define the loss function as the mean squared error between the given data $f(x)$ and its approximation $f_\theta^{\text{approx}}(x)$:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(f_\theta^{\text{approx}}(x_i) - f(x_i) \right)^2,$$

where $\{x_i\}_{i=1}^N \subset [a,b]$ is a set of collocation points at which the integral equation is evaluated.

Numerical Implementation The integral $f_\theta^{\text{approx}}(x)$ is approximated using quadrature rules (e.g., the rectangles, the trapezoidal, or Simpson's rule) over a set of evaluation points $\{t_j\}$ in the interval $[a,b]$:

$$f_\theta^{\text{approx}}(x_i) \approx \sum_{j=1}^{M} K(x_i, t_j) f_\theta(t_j)\, \Delta t.$$

Once the loss function is defined, the neural network is trained using gradient-based optimization (such as Adam or SGD) to minimize $\mathcal{L}(\theta)$, thus forcing the network to satisfy the integral equation.

It is worth noting the advantages of this method over the classical methods. This approach is mesh-free, regularized by the neural network architecture, and capable of producing smooth approximations even in the presence of mild ill-posedness.

Remark 2.4.1 Equation (2.4.3) can be expressed in operator form as $T(\varphi) = f$, where T is an integral operator defined by $T(\varphi)(x) := \int_a^b K(x,t)\,\varphi(t)\,dt$. The task is to recover the function φ from the data f. In this framework, the neural network serves as a functional approximation to the inverse operator T^{-1}, producing a continuous function $\varphi \approx f_\theta \approx T^{-1}(f)$. This approach is applied even in situations where T is not classically invertible–highlighting the neural network's ability to approximate generalized or regularized solutions in ill-posed settings. In a way, the neural network f_θ can be seen as a pseudo-inverse of the operator T.

2.4.2 Integral Equations of the Second Kind

An integral equation of the *second kind* is one in which the unknown function appears both inside and outside the integral. A standard form of such an equation is

$$\varphi(x) = f(x) + \lambda \int_a^b K(x,t)\,\varphi(t)\,dt, \tag{2.4.4}$$

where $f(x)$ is a known function, $K(x,t)$ is the kernel, λ is a real parameter, and $\varphi(t)$ is the unknown function to be determined. Unlike the first kind, this type of equation typically has better analytical properties. Under relatively mild assumptions on K and f, such equations often admit a unique and stable solution. For this reason, integral equations of the second kind are more amenable to numerical approximation, including by neural network methods. For instance, we have the following existence and uniqueness result:

Theorem 2.4.2 (Contraction Mapping Principle) *Consider the Fredholm integral equation of the second kind:*

$$\varphi(x) = f(x) + \lambda \int_a^b K(x,t)\,\varphi(t)\,dt,$$

where $f \in C[a,b]$, and the kernel $K(x,t) \in C([a,b]\times[a,b])$ is continuous. Suppose that the real constant λ satisfies

$$|\lambda| \cdot \sup_{x\in[a,b]} \int_a^b |K(x,t)|\,dt < 1.$$

Then, there exists a unique continuous solution $\varphi \in C[a,b]$ to the integral equation. Moreover, the solution can be obtained as the unique fixed point of the integral operator

$$T(\varphi)(x) := f(x) + \lambda \int_a^b K(x,t)\,\varphi(t)\,dt,$$

using iterative methods such as successive approximations (Picard's iteration).

Once it is established that the integral equation admits a unique continuous solution, it follows that any approximation obtained using a neural network converges to that unique solution–assuming successful training. In this context, neural networks offer

an alternative to classical techniques such as Picard's iteration for solving integral equations.

Moreover, the use of neural networks extends beyond the contraction mapping regime; they can be employed even when the contraction condition is not satisfied. In such cases, neural networks serve as powerful investigative tools for exploring the existence and qualitative behavior of approximate solutions.

We now present how a feed-forward neural network can be employed to solve integral equations of the second kind. These equations take the general form (2.4.4) where $f(x)$ is a known continuous function, $K(x,t)$ is a continuous kernel, λ is a scalar parameter, and $\varphi(t)$ is the unknown function to be determined.

Neural Network Approximation We model the unknown function $\varphi(t)$ by a neural network

$$\varphi(t) \approx f_\theta(t),$$

where f_θ is a neural network with trainable parameters θ. The network input is one dimensional, $t \in [a,b]$. The goal is to determine θ such that the neural network approximates the solution of the integral equation (2.4.4).

Unsupervised Learning Framework The learning process is *unsupervised*, meaning we do not require labeled input-output pairs. Instead, the integral equation itself serves as the constraint that guides the training. Given a set of collocation points $\{x_i\}_{i=1}^N \subset [a,b]$, we define the following residual function:

$$\mathcal{R}_\theta(x_i) := f_\theta(x_i) - f(x_i) - \lambda \int_a^b K(x_i,t)\, f_\theta(t)\, dt.$$

We then define the loss function as the mean squared residual over the sampled points:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left|\mathcal{R}_\theta(x_i)\right|^2.$$

Numerical Implementation In practice, the integral in the residual is computed using a numerical quadrature rule on a set of discretization points $\{t_j\}_{j=1}^M$:

$$\int_a^b K(x_i,t)\, f_\theta(t)\, dt \approx \sum_{j=1}^{M} K(x_i,t_j)\, f_\theta(t_j)\, \Delta t.$$

Once the loss function $\mathcal{L}(\theta)$ is defined, we minimize it using standard gradient-based optimization algorithms (e.g., Adam, or Stochastic Gradient Descent). The neural network thus learns to approximate a function $f_\theta(x)$ that satisfies the integral equation across the sampled domain.

2.4.3 Implementation of a Linear Integral Equation

The next implementations can be found at the following Google Colab page:

`https://tinyurl.com/jjrvxuh2`

We consider the Fredholm integral equation of the second kind:

$$\varphi(x) = 1 + \tfrac{1}{2}\int_0^1 e^{-xt}\,\varphi(t)\,dt. \tag{2.4.5}$$

This equation has a unique continuous solution φ, as the Contraction Mapping Principle condition can be verified as follows. For $x > 0$ we can compute

$$\tfrac{1}{2}\int_0^1 e^{-xt}\,dt = \frac{1-e^{-x}}{2x}.$$

Since the function

$$g(x) = \frac{1-e^{-x}}{x}$$

has a continuous extension to $x = 0$ with $\lim_{x\to 0} g(x) = 1$ and is increasing on $(0, 1]$, we have

$$\tfrac{1}{2}\sup_{x\in[0,1]} \frac{1-e^{-x}}{x} = \frac{1-e^{-1}}{2} \approx 0.31 < 1,$$

which verifies that the contraction mapping hypothesis holds.

Neural Network Approximation In the example given by equation (2.4.5) we implement a neural network solution $f_\theta(x)$ using an unsupervised learning framework in Keras. In this case $f(x) = 1$ and the kernel is $K(x, t) = e^{-xt}$.

A feed-forward neural network is constructed with five hidden layers (with 64 neurons in each of the layers) with the `swish` activation function, and a final output layer producing a scalar value for $\varphi(x)$.

We noticed that deeper architectures provide more accurate results; this conclusion was obtained by validation against the solution obtained by applying the Contraction Mapping Principle.

Then we define the custom loss function. The custom loss function enforces the integral equation by:

- Defining integration points t over $[0, 1]$.
- Evaluating the network output $f_\theta(x)$ at these points.
- Approximating the integral $\int_0^1 K(x, t)\varphi(t)\,dt$ using a numerical quadrature (here, the mean over $4,000$ points).
- Computing the residual $f_\theta(x) - f(x) -$ integral and returning its mean squared error.

Compiling and Training the Model The model is compiled with the Adam optimizer using a learning rate of 0.01. Since the loss function derives its supervision from the integral equation, dummy target data is used. The model is then trained for $1,000$ epochs.

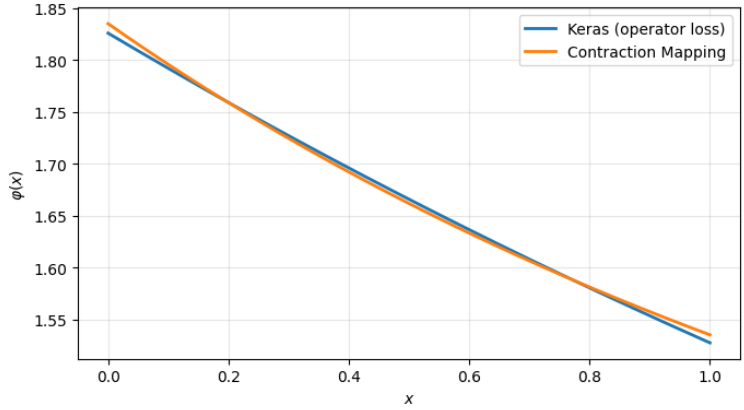


Figure 2.6: The learned solution to the equation (2.4.5) after training for $1,000$ epochs against the solution obtained by Contraction Mapping Principle.

Predicting and Plotting the Solution Finally, the model is evaluated on a dense grid over $[0,1]$ to obtain the learned approximation $f(x)$, which is then plotted, see Fig. 2.6. The validation is done by comparison with the solution obtained by applying the contraction mapping

$$\varphi_{k+1}(x) = T(\varphi_k(x)) = 1 + \frac{1}{2}\int_0^1 e^{-tx}\varphi_k(t)dt, \quad \phi_1(x) = 1$$

with $10,000$ iterations. We note a small mismatch at the end points

```
[NN ] phi(0) 1.8256377251,   phi(1)  1.5277598446
[CM ] phi(0) 1.8347584076,   phi(1)  1.5354450446
```

The maximum absolute error between the two solutions is 0.00909 and is subject to decrease even provided more hidden layer are considered.

2.4.4 Learning the Kernel via Neural Networks

Suppose we are given two continuous functions $\varphi : [a,b] \to \mathbb{R}$, $f : [c,d] \to \mathbb{R}$, and we wish to recover a continuous kernel $K(x,t) : [c,d] \times [a,b] \to \mathbb{R}$ satisfying the Fredholm equation

$$f(x) = \int_a^b K(x,t)\,\varphi(t)\,dt. \tag{2.4.6}$$

To do this, we introduce a feed-forward neural network $k_\theta : [c,d] \times [a,b] \to \mathbb{R}$, with parameters θ, which takes as input the pair $(x,t) \in [c,d] \times [a,b]$ and outputs $k_\theta(x,t) \approx K(x,t)$.

To this end, we choose sample points $\{x_i\}_{i=1}^N \subset [c,d]$ and $\{t_j\}_{j=1}^M \subset [a,b]$, and approximate the integral by a quadrature rule with weights $\{w_j\}$, for instance, $w_j =$

$(b-a)/M$. The predicted value of the integral at x_i is

$$\hat{f}_\theta(x_i) = \sum_{j=1}^{M} k_\theta(x_i, t_j)\, \varphi(t_j)\, w_j.$$

We define the loss as the mean squared residual between $\hat{f}_\theta(x_i)$ and the given $f(x_i)$:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\Big[f(x_i) \;-\; \hat{f}_\theta(x_i)\Big]^2 = \frac{1}{N}\sum_{i=1}^{N}\Big[f(x_i) - \sum_{j=1}^{M} k_\theta(x_i, t_j)\, \varphi(t_j)\, w_j\Big]^2.$$

Minimizing $\mathcal{L}(\theta)$ by gradient-based optimization forces the network output $k_\theta(x,t)$ to satisfy the integral equation at the collocation points, thereby yielding an approximation to the true kernel $K(x,t)$.

Remark 2.4.3 It is worthy to point out the following signal-processing interpretation. In signal-processing terms, the integral equation (2.4.6) can be viewed as a continuous-time filtering operation, where $\varphi(t)$ is the *raw* (input) signal, $K(x,t)$ is the filter's kernel, and $f(x)$ is the *filtered* (output) signal. Thus, recovering K from observations of φ and f corresponds to the classical *deconvolution* problem: given input-output pairs $\{\varphi(t), f(x)\}$, determine the filter $K(x,t)$ that maps φ into f.

2.4.5 Regularization Techniques for Kernel Identification

Since the problem of recovering $K(x,t)$ from the integral equation (2.4.6) is ill-posed, one must introduce *regularization* to obtain a stable, unique approximation of K. One effective approach of common regularization strategies is to restrict the search to a well-chosen subclass of kernels (for example, symmetric kernels, time-invariant kernels, or probability kernels), within which the integral equation admits a unique solution.

Symmetric kernels In this case we assume $[a,b] = [c,d]$ and we look for a kernel satisfying $K(x,t) = K(t,x)$. A suitable regularization term enforcing symmetry of the learned kernel $k_\theta(x,t)$ is

$$\mathcal{R}_{\text{sym}}(\theta) \;=\; \int_a^b \int_a^b \big|\, k_\theta(x,t) \;-\; k_\theta(t,x)\big|^2 \, dt\, dx.$$

Adding $\alpha\, \mathcal{R}_{\text{sym}}(\theta)$ to the loss $\mathcal{L}(\theta)$ penalizes any deviation from the relation $k_\theta(x,t) = k_\theta(t,x)$, thereby encouraging a symmetric solution. Here, α is a non-negative hyper-parameter such that the learned kernel both fits the integral equation (2.4.6) while it remains (approximately) symmetric.

Time-invariant kernels In many physical applications the system is assumed to be *time-invariant*, meaning that in this case the kernel takes the particular form $K(x,t) = h(x-t)$, and hence the integral operator reduces to the familiar convolution relation

$$f(x) \;=\; \int_a^b h(x-t)\, \varphi(t)\, dt \;=\; (h * \varphi)(x).$$

Identifying K in this setting is therefore equivalent to estimating the function $h(\tau)$ by a feed-forward neural network, $k_\theta(\tau)$. Regularizing under the time-invariant assumption reduces the dimensionality of the problem from recovering a two-variable function $K(x,t)$ to recovering a single-variable function $h(\tau)$.

Tikhonov Regularization This is a technique used for stabilizing ill-posed inverse problems by adding a penalty term that forces the learned kernel to remain small in a certain norm, with possibility of yielding a stable, unique approximation even when the unregularized problem admits infinitely many or unstable solutions.

More precisely, we replace the pure data-fitting loss

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\Big[f(x_i) - \sum_{j=1}^{M} k_\theta(x_i,t_j)\,\varphi(t_j)\,w_j\Big]^2$$

by a penalized loss

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{data}}(\theta) \;+\; \alpha\,\big\|\mathcal{P}\,k_\theta\big\|_2^2$$

where $\mathcal{P}$ is a differential operator. Typically, one may choose $\mathcal{P} = \text{Id}$, or $\mathcal{P} = \frac{\partial}{\partial x}$, or $\mathcal{P} = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial t^2}$ to enforce different types of smoothness. Also, $\alpha > 0$ is the *regularization parameter*. In the case when $K(x,t)$ is a heat kernel, then $\mathcal{P}$ is a hyperbolic operator.

Regularizing to a Probability Kernel If we wish our learned kernel $K(x,t)$ to act as a conditional probability density in t for each x, it must satisfy two properties:

$$K(x,t) \;\geq 0 \quad \text{and} \quad \int_a^b K(x,t)\,dt \;=\; 1 \quad \text{for all } x\,.$$

This can be accomplished using neural-networks by the following two common strategies:

(i) Penalty-based Regularization One way is to consider the network output $k_\theta(x,t)$ and add to the loss $\mathcal{L}(\theta)$ the following two penalty terms:

$$\mathcal{R}_{\text{neg}}(\theta) = \int_c^d \int_a^b \big[\min\{0,\, k_\theta(x,t)\}\big]^2\,dt\,dx,$$

$$\mathcal{R}_{\text{norm}}(\theta) = \int_c^d \left[\int_a^b k_\theta(x,t)\,dt - 1\right]^2 dx.$$

The first penalizes any negative values, while the second enforces unit mass. Thus, the total loss becomes

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}(\theta) + \alpha\,\mathcal{R}_{\text{neg}}(\theta) + \beta\,\mathcal{R}_{\text{norm}}(\theta),$$

with $\alpha, \beta > 0$ hyperparameters.

(ii) Softmax-style Parameterization In this case we consider the output of a feed-forward neural network given by $u_\theta(x,t)$, and then define the kernel by

$$K_\theta(x,t) \;=\; \frac{\exp\big(u_\theta(x,t)\big)}{\displaystyle\int_a^b \exp\big(u_\theta(x,s)\big)\,ds}\,.$$

This guarantees $K_\theta(x,t) \geq 0$ and automatically normalizes $\int_a^b K_\theta(x,t)\,dt = 1$.

Separable Kernel Approximation We may enforce an approximation of the kernel with the following simple structure

$$K(x,t) \;\approx\; \sum_{r=1}^{R} u_r(x)\, v_r(t).$$

This can be built using a feed-forward neural network $k_\theta(x,t) = (u_\theta(x), v_\theta(t))$ with two parallel branches:

$$u_\theta(x) = \big(u_1(x), \ldots, u_R(x)\big), \qquad v_\theta(t) = \big(v_1(t), \ldots, v_R(t)\big),$$

and then define

$$K_\theta(x,t) \;=\; \sum_{r=1}^{R} u_r(x)\, v_r(t).$$

The regularization term added in the continuous setting is

$$\mathcal{R}(\theta) \;=\; \|u_\theta\|_2^2 + \|v_\theta\|_2^2 = \sum_{r=1}^{R} \int_c^d |u_r(x)|^2\, dx \;+\; \sum_{r=1}^{R} \int_a^b |v_r(t)|^2\, dt.$$

Thus, the total loss becomes

$$\begin{aligned}\mathcal{L}_{\text{total}}(\theta) &= \frac{1}{N} \sum_{i=1}^{N} \Big[f(x_i) - \sum_{j=1}^{M} K_\theta(x_i, t_j)\, \varphi(t_j)\, w_j \Big]^2 + \alpha\, \mathcal{R}(\theta) \\ &= \frac{1}{N} \sum_{i=1}^{N} \Big[f(x_i) - \sum_{r=1}^{R} u_r(x_i) \sum_{j=1}^{M} v_r(t_j) \varphi(t_j) w_j \Big]^2 + \alpha\, \mathcal{R}(\theta).\end{aligned}$$

This regularization encourages the network to distribute most of its "energy" into the first few separable components (u_r, v_r), yielding a parsimonious kernel approximation.

2.4.6 Exercises

Exercise 2.4.4 Consider the integral equation

$$\varphi(x) = 1 + \int_0^1 (x + t)\, \varphi(t)\, dt.$$

Looking for an affine solution, $\varphi(x) = ax + b$, find the coefficients a and b determined by substituting into the equation and equating coefficients.

Exercise 2.4.5 (*a*) Find an affine solution to the integral equation

$$\varphi(x) = x + \int_0^1 (x + t)\, \varphi(t)\, dt.$$

(*b*) Construct a neural network to verify the solution obtained analytically in part (*a*).

Exercise 2.4.6 Consider the integral equation:

$$\varphi(x) = \cos(x) + \lambda \int_0^1 (x+t)\,\varphi(t)\,dt,$$

where $x \in [0,1]$ and $\lambda \in \mathbb{R}$ such that $|\lambda| < \frac{2}{3}$.

(a) Use the Contraction Principle to show that the equation admits a unique continuous solution.

(b) Use a neural network to find and plot the solution $\varphi(t)$ of the integral equation.

Exercise 2.4.7 Consider the Poisson kernel defined by

$$K(r,\theta) = \frac{1-r^2}{1-2r\cos\theta + r^2},$$

for $\theta \in [0, 2\pi]$ and $0 \leq r < 1$. We wish to find a function $\varphi : [0, 2\pi] \to \mathbb{R}$ that satisfies the integral equation

$$\int_0^{2\pi} K(r,\theta)\,\varphi(\theta)\,d\theta = r. \tag{2.4.7}$$

(a) Construct a neural network model that accepts θ as input and outputs an approximation $f_\theta(\theta)$ of $\varphi(\theta)$.

(b) Define a custom loss function that approximates the residual

$$R = \int_0^{2\pi} K(r,\theta)\,f_\theta(\theta)\,d\theta - r,$$

and minimizes the squared error $\mathcal{L} = R^2$.

(c) Train the network using a suitable set of collocation points over $[0, 2\pi]$.

(d) Finally, plot the learned function $\varphi(\theta) \approx f_\theta(\theta)$ over $\theta \in [0, 2\pi]$.

(e) Use the information from the Google Collab page

`https://tinyurl.com/jjrvxuh2`

to show that the integral equation (2.4.7) is ill-posed.

Exercise 2.4.8 (Gaussian kernel) Use a neural network approach to investigate the ill-posedness of the integral equation

$$x^2 e^{-x^2} = \int_{-\infty}^{\infty} e^{-\frac{(x-y)^2}{2}} \varphi(y)\,dy.$$

Exercise 2.4.9 (Exponential integral equation) Consider the integral equation

$$e^x = \int_0^1 K(x,t)\,e^t\,dt.$$

We wish to recover the continuous kernel $K(x,t)$ on $[0,1]\times[0,1]$ using a neural network following the following points:

(a) Let $k_\theta(x,t)$ be a feed-forward network with two inputs x and t and a single scalar output, intended to approximate $K(x,t)$.

(b) Choose N points $\{x_i\} \subset [0,1]$ and M points $\{t_j\} \subset [0,1]$.

(c) Use Riemannian weights w_j to approximate

$$\int_0^1 k_\theta(x_i,t)\, e^t\, dt \approx \sum_{j=1}^{M} k_\theta(x_i,t_j)\, e^{t_j}\, w_j.$$

(d) Define the mean-squared residual

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left[\sum_{j=1}^{M} k_\theta(x_i,t_j)\, e^{t_j}\, w_j \;-\; e^{x_i}\right]^2.$$

(e) Minimize $\mathcal{L}(\theta)$ (e.g. with Adam) so that $k_\theta(x,t)$ satisfies the integral equation at the collocation points.

(f) After training, evaluate $\hat{f}(x) = \sum_{j=1}^{M} k_\theta(x,t_j)\, e^{t_j}\, w_j$ on a fine grid of x-values and compare to e^x. Plot both curves and report the maximum absolute error.

(g) Evaluate $\|k_\theta(x,t) - e^{x-t}\|_2$ and provide its significance.

2.5 Solving ODEs

We shall we demonstrate how feed-forward neural networks can approximate solutions of first and second-order ODEs by embedding the differential equation and any initial or boundary conditions directly into the loss. This approach offers a unified alternative to traditional numerical schemes.

Since most equations of Physics are of the first and second order, this approach would suffice for most applications.

2.5.1 First Order ODEs

We consider the first-order ordinary differential equation

$$\frac{du}{dt} = F\big(t, u(t)\big), \qquad u(0) = u_0, \tag{2.5.8}$$

where: $u(t)$ is the unknown solution, $F(t,u)$ is a given function, and $u(0) = u_0$ is the prescribed initial condition.

Since we are aiming to find the solution using neural networks, we need first to be sure that the aforementioned problem is well-posed. It is known that the solution existence and uniqueness holds if the hypothesis of Picard-Lindelöf theorem are satisfied. These conditions are:

(i) the continuity of F on a neighborhood of $(0,u_0)$, such as

$$R = \big\{(t,u) : |t| \le \epsilon_1,\ |u-u_0| \le \epsilon_2\big\},\ \epsilon_1,\epsilon_2 > 0$$

and

(*ii*) the Lipschitz condition for F in the u-argument, namely, there exists $L > 0$ such that for all $(t, u_1), (t, u_2) \in R$,

$$\big|F(t, u_1) - F(t, u_2)\big| \;\le\; L\,\big|u_1 - u_2\big|.$$

Remark 2.5.1 We note that the Lipschitz condition (*ii*) is immediately satisfied if the partial derivative $\frac{\partial F}{\partial u}$ is bounded on R, since

$$\big|F(t, u_1) - F(t, u_2)\big| = \Big|\int_{u_2}^{u_1} \frac{\partial F}{\partial u}(t, s)\, ds\Big| \le \sup_{(t,u)\in R}\Big|\frac{\partial F}{\partial u}(t, u)\Big|\,\big|u_1 - u_2\big|.$$

Verifying boundedness of $\frac{\partial F}{\partial u}$ is often more straightforward in practice.

In the following we shall describe three variants for solving first-order ODEs with neural networks. Each uses a feed-forward network $f_\theta(t) \approx u(t)$; they differ only in the choice of loss function. We assume in the following that $\epsilon_1 > 1$, namely the ODE has a unique solution on the interval $[0, 1]$.

Method 1 To solve the initial-value problem (2.5.8), we introduce a feed-forward neural network $f_\theta(t) \approx u(t)$, with trainable parameter θ. We select a set of collocation times $\{t_i\}$ in the interval $[0, 1]$ and define two loss components:

(*i*) *Initial-Condition Loss:* $\mathcal{L}_{\rm bd}(\theta) = \big[f_\theta(0) \;-\; u_0\big]^2$, which enforces the boundary condition $f_\theta(0) \approx u_0$.

(*ii*) *ODE Residual Loss:*

$$\mathcal{L}_{\rm eq}(\theta) = \sum_i \Big[\,\tfrac{d}{dt}\, f_\theta(t_i) \;-\; F\big(t_i,\, f_\theta(t_i)\big)\Big]^2,$$

where $\frac{d}{dt} f_\theta(t_i)$ is obtained by automatic differentiation.[2] This term penalizes any violation of the ODE at the sample points t_i.

The *total loss* is obtained adding the previous two losses:

$$\mathcal{L}(\theta) = \mathcal{L}_{\rm bd}(\theta) + \mathcal{L}_{\rm eq}(\theta).$$

Minimizing $\mathcal{L}(\theta)$ yields a network $f_\theta(t)$ that satisfies both the initial condition and the differential equation, providing a smooth, mesh-free approximation to the true solution $u(t)$.

Remark 2.5.2 It is worth noting that the ODE residual loss in its continuous form over the interval $[0, 1]$ is given by

$$\mathcal{L}^{\rm cont}(\theta) = \big[f_\theta(0) - u_0\big]^2 \;+\; \int_0^1 \Big[f'_\theta(t) - F\big(t, f_\theta(t)\big)\Big]^2 dt,$$

[2] Keras supports automatic differentiation via TensorFlow's tf.GradientTape, which enables efficient computation of derivatives needed to enforce physical laws in Physics-Informed Neural Networks.

which, when minimized, enforces both the initial condition and the ODE in an L^2-sense over the entire domain.

Method 2 To solve the initial-value problem (2.5.8), we enforce the initial condition $u(0) = u_0$ exactly, by considering the hard-embedding

$$u(t) \;=\; u_\theta(\theta) = u_0 \;+\; t\, f_\theta(t),$$

where $f_\theta(t)$ is a feed-forward neural network. By construction, the initial condition is verified automatically

$$u(0) \;=\; u_0 + 0 \cdot f_\theta(0) \;=\; u_0.$$

Referring to the aforementioned construction, we customarily say that we had embedded the initial condition into the network architecture.
Differentiating, yields

$$\frac{du}{dt} = \frac{d}{dt}\big[u_0 + t\, f_\theta(t)\big] = f_\theta(t) + t\, \frac{d}{dt} f_\theta(t).$$

We define the residual of the ODE at a point t by

$$R(t;\theta) = \Big[f_\theta(t) + t\, f'_\theta(t)\Big] - F\big(t, u_0 + t\, f_\theta(t)\big).$$

The loss function is then the mean-squared residual over collocation points $\{t_i\}$:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\big[R(t_i;\theta)\big]^2 = \frac{1}{N}\sum_{i=1}^{N}\Big[f_\theta(t_i) + t_i\, f'_\theta(t_i) - F\big(t_i, u_0 + t_i\, f_\theta(t_i)\big)\Big]^2.$$

Minimizing $\mathcal{L}(\theta)$ enforces the ODE at each t_i while automatically satisfying $u(0) = u_0$.

Remark 2.5.3 (*i*) By using the trial solution $u(t) = u_0 + t\, f_\theta(t)$, the network needs only to learn the *correction* $f_\theta(t)$, rather than the full solution $u(t)$.
(*ii*) Usually, this construction often speeds up training. The idea of hard embedding initial conditions or boundary conditions into the neural network idea can be traced back to the pioneering work of Lagaris et al. [57].

Method 3 This method writes the ODE as an integral equation and then solves it. Thus, the initial-value problem (2.5.8) is equivalent to the following integral equation

$$u(t) = u_0 + \int_0^t F(s, u(s))\, ds, \quad 0 \le t \le 1. \tag{2.5.9}$$

We can use a feed-forward neural network to approximate the solution, $f_\theta(t) \approx u(t)$, employing the following L^2-type loss over the entire domain

$$\mathcal{L}_{\text{cont}}(\theta) = \int_0^1 \Big[u_\theta(t) - u_0 - \int_0^t F\big(s, u_\theta(s)\big)\, ds\Big]^2 dt. \tag{2.5.10}$$

In practice, however, the discrete-quadrature version of the loss described above is more commonly used.

To this end, we select N collocation points $\{t_i\}_{i=1}^N \subset [0,1]$. At each t_i, we apply a quadrature rule with nodes $\{s_{ij}\}_{j=1}^M$ and weights $\{w_j\}_{j=1}^M$ to approximate the continuous loss (2.5.10) by the discrete loss

$$\mathcal{L}_{\text{disc}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\Big[u_\theta(t_i) - u_0 - \sum_{j=1}^{M} F\big(s_{ij}, u_\theta(s_{ij})\big)\, w_j\Big]^2. \tag{2.5.11}$$

Considering $\theta^* = \arg\min \mathcal{L}_{\text{disc}}(\theta)$, we obtain the solution approximation $f_{\theta^*}(t) \approx u(t)$.

2.5.2 Second Order ODEs

We consider the second order ODE

$$u''(x) = F\big(t, u(t), u'(t)\big), \tag{2.5.12}$$

which can come either with initial conditions

$$u(0) = u_0,\ u'(0) = v_0, \tag{2.5.13}$$

or with boundary conditions

$$u(0) = u_0,\ u(1) = u_1. \tag{2.5.14}$$

The problem (2.5.12) with condition (2.5.13) admits usually a unique solution (at least for small t), whereas the boundary-value problem (2.5.12) with conditions (2.5.14), being a global problem, requires additional hypotheses to be well-posed.

As usual, there are several variants for approaching these problems. They all involve a feed-forward neural network $f_\theta(t)$ intended to approximate the solution $u(t)$, assuming it exists and is unique on the domain under consideration.

Method 1 For the problem (2.5.12) with initial-condition (2.5.13) we define

$$\mathcal{L}_{\text{ic}}(\theta) = \big[f_\theta(0) - u_0\big]^2 + \big[f'_\theta(0) - v_0\big]^2,$$
$$\mathcal{L}_{\text{eq}}(\theta) = \int_0^1 \Big[f''_\theta(t) - F\big(t,\, f_\theta(t),\, f'_\theta(t)\big)\Big]^2\, dt.$$

The total loss is given by the sum

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{ic}}(\theta) + \mathcal{L}_{\text{eq}}(\theta).$$

Using any variant of the gradient descent method we search for $\theta^* = \arg\min \mathcal{L}(\theta)$ and consider the solution approximation given by f_{θ^*}.

On the other side, for the problem (2.5.12) with boundary-condition (2.5.14) we consider

$$\mathcal{L}_{\text{bd}}(\theta) = \big[f_\theta(0) - u_0\big]^2 + \big[f_\theta(1) - u_1\big]^2,$$
$$\mathcal{L}_{\text{eq}}(\theta) = \int_0^1 \Big[f''_\theta(t) - F\big(t,\, f_\theta(t),\, f'_\theta(t)\big)\Big]^2\, dt.$$

The total loss is

$$\mathcal{L}(\theta) = \mathcal{L}_{\rm bd}(\theta) + \mathcal{L}_{\rm eq}(\theta),$$

which after minimization produces the solution approximation $f_{\theta^*}(t) \approx u(t)$.

In practice, the equation loss, $\mathcal{L}_{\rm eq}(\theta)$, is approximated by the Riemann sum

$$\mathcal{L}_{\rm eq}^{\rm disc}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\Big[f_\theta''(t_i) - F\big(t_i, f_\theta(t_i), f_\theta'(t_i)\big)\Big]^2,$$

where $\{t_i\}_{i=1}^N \subset [0,1]$ are given collocation points.

Method 2 Consider the second-order ODE (2.5.12) with initial conditions (2.5.13). This method employs an auxiliary trial solution of the form dictated by the initial-condition Taylor expansion. Any solution admits the expansion

$$u(t) = u_0 + t\,v_0 + O(t^2), \tag{2.5.15}$$

which automatically satisfies $u(0) = u_0$ and $u'(0) = v_0$. We then model the $O(t^2)$ remainder by a neural network and set

$$u(t) = u_\theta(t) = u_0 + t\,v_0 + \tfrac{1}{2}\,t^2\,f_\theta(t),$$

so that both initial conditions are built into the ansatz and only the network $f_\theta(t)$ must be learned.

Differentiating using product rule, yields

$$\begin{aligned} u'(t) &= v_0 + t\,f_\theta(t) + \tfrac{1}{2}\,t^2\,f_\theta'(t) \\ u''(t) &= f_\theta(t) + 2\,t\,f_\theta'(t) + \tfrac{1}{2}\,t^2\,f_\theta''(t). \end{aligned}$$

We define the residual at t be given by

$$\begin{aligned} R(t;\theta) &= u''(t) - F\big(t, u(t), u'(t)\big) \\ &= f_\theta(t) + 2\,t\,f_\theta'(t) + \tfrac{1}{2}\,t^2\,f_\theta''(t) \\ &\quad -F\Big(t, u_0 + t\,v_0 + \tfrac{1}{2}\,t^2\,f_\theta(t), f_\theta(t) + 2\,t\,f_\theta'(t) + \tfrac{1}{2}\,t^2\,f_\theta''(t)\Big), \end{aligned}$$

and consider the loss function be given by the mean-squared residual over collocation points $\{t_i\}$

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\big[R(t_i;\theta)\big]^2.$$

The minimization of $\mathcal{L}(\theta)$ enforces the ODE at each t_i while automatically satisfying the initial conditions $u(0) = u_0$ and $u'(0) = v_0$.

Method 3 This method applies to the ODE (2.5.12) with boundary conditions (2.5.14). We consider the trial solution

$$u(t) = u_\theta(t) = (1-t)u_0 + tu_1 + t(1-t)f_\theta(t), \quad 0 \le t \le 1,$$

which depends on a neural network $f_\theta(t)$. This solution obviously satisfies the boundary conditions, $u(0) = u_0$ and $u(1) = u_1$.

As an application of the product rule, we have

$$\begin{aligned} u'(t) &= (u_1 - u_0) + \big(1 - 2t\big)\, f_\theta(t) + t(1-t)\, f'_\theta(t) \\ u''(t) &= -2\, f_\theta(t) + 2\,(1-2t)\, f'_\theta(t) + t(1-t)\, f''_\theta(t). \end{aligned}$$

Then in this case the residual at t takes the form

$$\begin{aligned} R(t;\theta) &= u''(t) - F\big(t, u(t), u'(t)\big) \\ &= -\,2\, f_\theta(t) + 2\,(1-2t)\, f'_\theta(t) + t(1-t)\, f''_\theta(t) \\ &\quad - F\Big(t,\ (1-t)u_0 + tu_1 + t(1-t) f_\theta(t), \\ &\qquad\qquad (u_1 - u_0) + \big(1-2t\big)\, f_\theta(t) + t(1-t)\, f'_\theta(t)\Big). \end{aligned}$$

To approximate the solution, the neural network is considered at the optimal parameter given by

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta) = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \big[R(t_i;\theta)\big]^2.$$

Method 4 This method transforms the given second-order ODE into a first-order system of ODEs and then solves it by one of the previous methods. To this end, we rewrite the second-order equation

$$u''(t) \;=\; F\big(t, u(t), u'(t)\big)$$

as a first-order system as follows. We set $y_1(t) = u(t)$ and $y_2(t) = u'(t)$, so

$$y'_1(t) = u'(t) = y_2(t), \qquad y'_2(t) = u''(t) = F\big(t, y_1(t), y_2(t)\big).$$

Hence, the resulting first-order system is

$$\begin{cases} y'_1(t) = y_2(t), \\ y'_2(t) = F\big(t, y_1(t), y_2(t)\big), \end{cases}$$

with initial conditions

$$y_1(0) = u(0) = u_0, \qquad y_2(0) = u'(0) = v_0.$$

Let $y(t) = (y_1(t), y_2(t))^T$, $y_0 = (u_0, v_0)^T$ and $\tilde{F}(t,y) = \big(y_2, F\big(t, y_1(t), y_2(t)\big)\big)^T$. Then the aforementioned system becomes the initial-value first order ODE

$$y'(t) = \tilde{F}(t,y), \qquad y(0) = y_0,$$

which can be solved by one of the methods mentioned in Section 2.5.1.

Remark 2.5.4 (i) We note that this method needs only the first derivative of the network output. Neural networks compute $\frac{d}{dt}f_\theta(t)$ efficiently via automatic differentiation. By contrast, computing $\frac{d^2}{dt^2}f_\theta(t)$ requires two nested passes, which is both slower and numerically less stable.

(ii) This method trades differentiation order for system dimension. By rewriting a second-order ODE as a two-dimensional first-order system, we increase state-dimension by one and decrease the differentiation order. Neural nets handle higher-dimensional inputs with ease, whereas learning higher-order derivatives is costly. Hence, this method is more accessible and efficient in practice.

2.5.3 Practical Implementations

First-order ODEs For implementation details the reader is encouraged to explore the accompanying Google Colab notebook at:

`https://tinyurl.com/4wcfshpu`

We shall start by implementing the first-order ODE given by

$$u'(t) = 2u(t) + t \tag{2.5.16}$$
$$u(0) = 1 \tag{2.5.17}$$

via a neural network.

This ODE can be solved exactly as a linear equation by multiplying by the integrating factor $\mu(t) = e^{-2t}$. We obtain the exact solution

$$u_{\text{exact}}(t) = \frac{1}{4}(5e^{2t} - 2t - 1).$$

We will compare this closed-form expression with the network's output at the end to assess learning accuracy.

As usual, we start by loading dependencies:

- `numpy` for numerical arrays,
- `tensorflow.keras` for building and training the network,
- `matplotlib` for plotting results.

We encode the ODE's right-hand side function $F(u, t) = 2u + t$ as

```
def F(u, t):
    return 2 * u + t
```

Then we consider a feed-forward network consisting of

- Two hidden layers of 50 neurons each, with a tanh activation function.
- A final linear output layer returning the approximation $u(t)$.

as follows:

```
model = Sequential([
    Dense(50, activation='tanh', input_dim=1),
    Dense(50, activation='tanh'),
    Dense(1)
])
```

Following Method 1 of Section 2.5.1, we define a loss that enforces the initial condition $u(0) = 1$ via `boundary_loss` as well as the ODE residual $\big(u'(t_i) - (2u(t_i) + t_i)\big)^2$ averaged over all training points:

```
def custom_loss(y_true, y_pred):
    t_input = tf.constant(t_train, dtype=tf.float32)
    u_pred = model(t_input)

    with tf.GradientTape() as tape:
        tape.watch(t_input)
        u_t = model(t_input)
    du_dt = tape.gradient(u_t, t_input)

    boundary_loss = tf.square(model(tf.constant([[0.0]])) - 1.0)
    equation_loss =
    tf.reduce_mean(tf.square(du_dt - F(u_pred, t_input)))
    return 10*boundary_loss + equation_loss
```

The boundary loss is weighted with $w = 10$ to enforce the neural network solution to start from $u_0 = 1$. For the compilation part, we configure the network to use our custom loss and the Adam optimizer at learning rate 0.01:

```
model.compile(optimizer=Adam(learning_rate=0.01), loss=custom_loss)
```

The snippet

```
t_train = np.linspace(0, 1, 100).reshape(-1, 1)
dummy_target = np.zeros_like(t_train)
```

creates points $\{t_i\}$ evenly spaced in $[0, 1]$ as input as well as a dummy target array, which is unused by the custom loss.

We then run 200 epochs of gradient-based optimization, using a batch of 32:

```
model.fit(t_train, dummy_target, epochs=200, batch_size=32)
```

The next snippet generates a fine grid for $\{0 \le t \le 1\}$, predicts the neural output, $u_{\mathrm{NN}}(t)$, computes the exact solution, $u_{\mathrm{exact}}(t)$, and then it plots both curves to verify accuracy:

```
t_plot  = np.linspace(0, 1, 100).reshape(-1, 1)
u_nn    = model.predict(t_plot)
u_exact = exact_solution(t_plot)
```

```
plt.figure(figsize=(8,6))
plt.plot(t_plot, u_nn,    label="NN Solution", linewidth=2)
plt.plot(t_plot, u_exact, label="Exact Solution", linestyle='--')
plt.title("Solution of du/dt = 2u + t, u(0)=1")
plt.xlabel("t");  plt.ylabel("u(t)")
plt.grid(alpha=0.3);  plt.legend();  plt.show()
```

The graphs of the exact solution $u_{\text{exact}}(t)$ and the neural network's output $u_{\text{nn}}(t) = f_\theta(t)$ are shown in Fig. 2.7. The approximation is satisfactory, though not perfect; however, increasing the number of training epochs further improves accuracy.

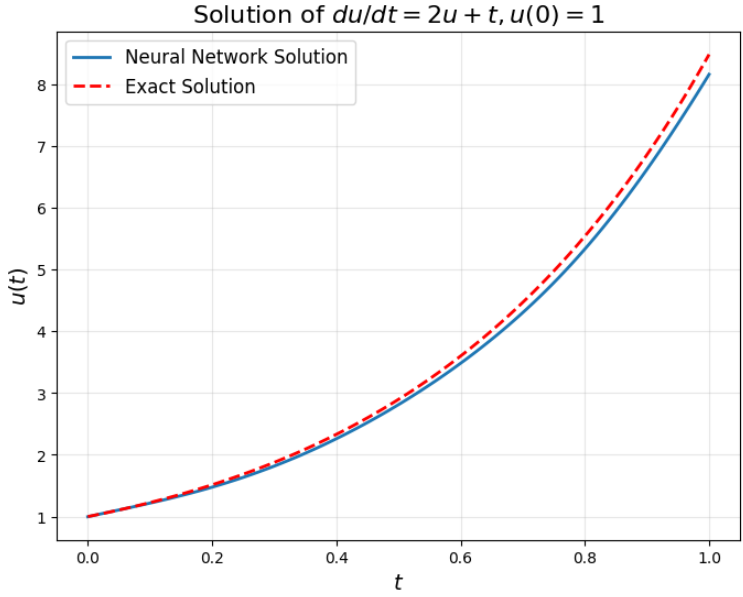


Figure 2.7: The graphs of the exact solution $u_{\text{exact}}(t) = \frac{1}{4}(5e^{2t} - 2t - 1)$ versus the neural network's output trained for 200 epochs in the case of Method 1.

In the following we shall solve the same first-order ODE by employing Method 2 of Section 2.5.1. In this case the custom loss involves the use of a trial solution $u_t = u_0 + t_{\text{input}} * nn_t$ as follows:

```
# Custom loss function
def custom_loss(y_true, y_pred):
    t_input = tf.constant(t_train, dtype=tf.float32) #Training points
    nn_t = model(t_input)  # NN(theta, t)

    # Compute u(t) = u_0 + t * NN(theta, t)
    u_t = u_0 + t_input * nn_t
```

```
    # Compute du/dt = NN(theta, t) + t * d(NN(theta, t))/dt
    with tf.GradientTape() as tape:
        tape.watch(t_input)
        nn_t_gradient = model(t_input)
    d_nn_dt = tape.gradient(nn_t_gradient, t_input)
    du_dt = nn_t + t_input * d_nn_dt

    # Residual of the ODE: du/dt - F(u, t)
    residual = du_dt - F(u_t, t_input)
    return tf.reduce_mean(tf.square(residual))
```

Training the model with only 50 epochs,

```
model.fit(t_train, dummy_target, epochs=50, batch_size=32)
```

produces a perfect match between the exact solution $u_{\text{exact}}(t)$ and the neural network's output $u_{\text{nn}}(t) = f_\theta(t)$, see Fig. 2.8. This shows the superiority of Method 2 over Method 1.

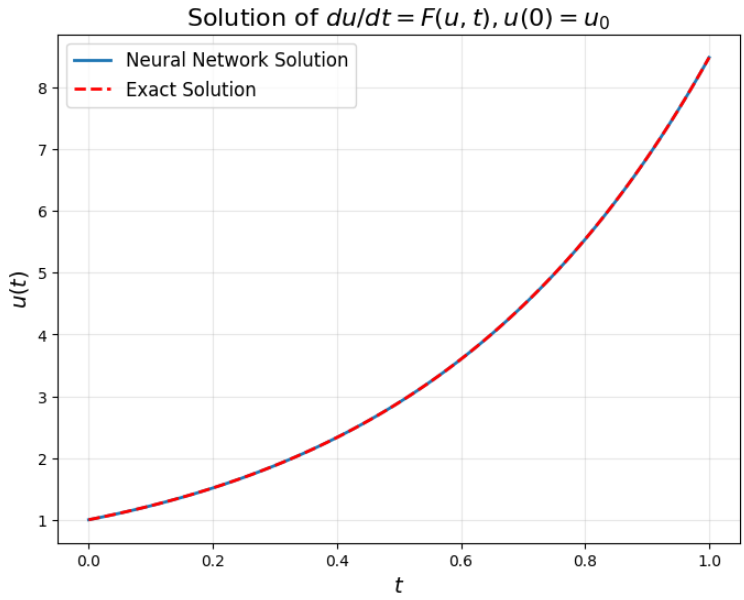


Figure 2.8: The graphs of the exact solution $u_{\text{exact}}(t) = \frac{1}{4}(5e^{2t} - 2t - 1)$ versus the neural network's output trained for 50 epochs in the case of Method 2.

Second-order ODEs We have in our attention the boundary-value problem

$$\begin{aligned} \frac{d^2u}{dt^2} &= -u, \qquad t \in [0,1] \\ u(0) &= 1, \qquad u(1) = \cos(1), \end{aligned}$$

which corresponds to a simple harmonic motion with the exact solution $u_{exact}(t) = \cos t$. To approximate this solution, we employ a shallow neural network with one hidden layer given by the architecture

```
model = Sequential([
    Dense(50, activation='tanh', input_dim=1),
    Dense(1)
])
```

The custom loss is implemented as in Method 1 of Section 2.5.2

```
def custom_loss(y_true, y_pred):
    t_input = tf.constant(t_train, dtype=tf.float32) #Training points

    # Compute NN(theta, t) and its derivatives
    with tf.GradientTape() as tape1:
        tape1.watch(t_input)
        with tf.GradientTape() as tape2:
            tape2.watch(t_input)
            nn_t = model(t_input)
        d_nn_dt = tape2.gradient(nn_t, t_input) #First derivative
    d2_nn_dt2 = tape1.gradient(d_nn_dt, t_input) #Second derivative

    # Equation loss: d^2u/dt^2 - F(u, t)
    equation_loss =
    tf.reduce_mean(tf.square(d2_nn_dt2 - F(nn_t, t_input)))

    # Boundary condition loss: (NN(theta, 0) - u_0)^2
     + (NN(theta, 1) - u_1)^2
    boundary_loss = tf.square(model(tf.constant([[0.0]])) - u_0)
    + tf.square(model(tf.constant([[1.0]])) - u_1)

    # Total loss
    return equation_loss + 10*boundary_loss
```

We enforced the boundary conditions by using a weight $w = 10$. Training for 250 epochs produced the satisfactory solution shown in Fig. 2.9.

The implementation details can be found in the accompanying Google Colab notebook at:

`https://tinyurl.com/4cvrf9mv`

In the following we apply Method 3 for solving the boundary value problem (2.5.18)-(2.5.18). The only difference from Method 1 consists of a modification of the custom loss, which now involves a trial solution:

```
def custom_loss(y_true, y_pred):
    t = tf.constant(t_train)
```

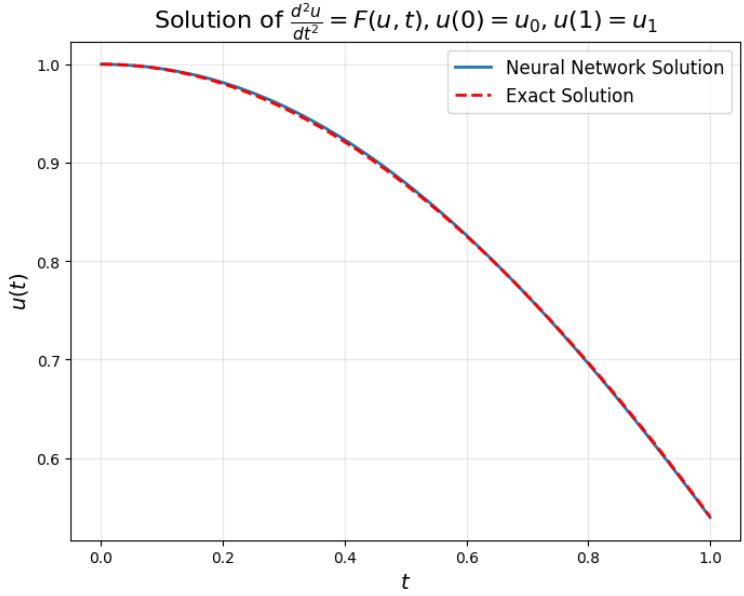


Figure 2.9: The graphs of the exact solution $u_{\text{exact}}(t) = \cos t$ versus the neural network's output trained for 250 epochs in the case of Method 1.

```
with tf.GradientTape(persistent=True) as tape:
    tape.watch(t)
    nn = model(t)                              # NN(,t)
    # Trial solution: u(t) = (1t)u0 + t u1 + t(1t)*NN(,t)
    u = (1 - t)*u0 + t*u1 + t*(1 - t)*nn
    du_dt  = tape.gradient(u, t)               #1st derivative
    d2u_dt2 = tape.gradient(du_dt, t)          #2nd derivative
# residual of the ODE
eq_loss = tf.reduce_mean((d2u_dt2 - F(u, t))**2)
return eq_loss
```

The idea is to superimpose on the straightline solution

$$u_{\text{lin}}(t) = (1-t)\,u_0 + t\,u_1$$

a perturbation term, $t(1-t)\,f_\theta(t)$. Since $t(1-t) = 0$ at $t = 0$ and $t = 1$, the boundary conditions $u(0) = u_0$ and $u(1) = u_1$ hold automatically, leaving the neural network $f_\theta(t)$ free to learn the interior correction.

The graphs of the exact solution and the neural network approximations are almost identical to those in Fig. 2.9, the only difference being that the error is now several orders of magnitude smaller. More precisely, using Method 1 we obtain:

```
Epoch 250/250
```

```
4/4  0s 7ms/step - loss: 0.0018
```

whereas using Method 3 we obtain a much smaller error, which is given by

```
Epoch 250/250
4/4  0s 7ms/step - loss: 2.8449e-06
```

This clearly shows the superiority of Method 3 over Method 1 of Section 2.5.2.

However, Method 3 has its own limitations. If the solution oscillates, then the neural network has a hard time to fit the exact solution. For instance, this occurs in the case of the boundary-value problem

$$\begin{aligned} \frac{d^2u}{dt^2} &= -u, \qquad t \in [0,1] \\ u(0) &= 1, \qquad u(15) = \cos(15), \end{aligned}$$

as it can be seen in Fig. 2.10.

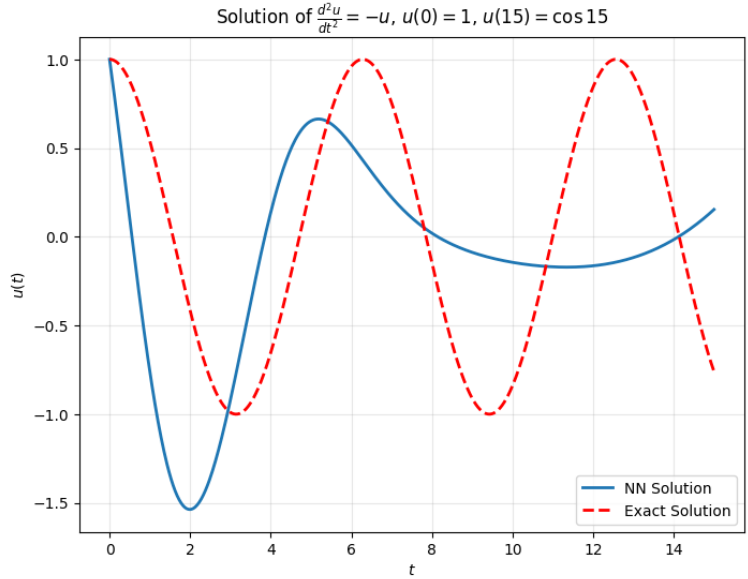


Figure 2.10: The graphs of the exact solution $u_{\text{exact}}(t) = \cos t$ versus the neural network's output trained for 250 epochs in the case of Method 3 for $t \in [0, 15]$.

However, Method 3 has its own limitations. When the solution oscillates, the neural network struggles to fit the exact solution. For example, consider the boundary-value problem

$$\begin{aligned} \frac{d^2u}{dt^2} &= -u, \qquad t \in [0,15], \\ u(0) &= 1, \qquad u(15) = \cos(15), \end{aligned}$$

as illustrated in Fig. 2.10. There are two ways to address this issue:

1. Split the interval $[0, 15]$ into subintervals shorter than one period, solve on each subinterval separately, and then concatenate the local solutions into a global one.

2. Use the two-phase method: in Phase I learn a coarse approximation of the solution, then in Phase II train a higher-capacity network on the residual (see Section 2.9).

2.5.4 Newton's Cooling Law

In this section, we aim to determine the temperature curve of a cooling body. We begin by reconstructing the temperature curve from measured data using plain polynomial interpolation.

Next, we employ a neural network to approximate the temperature based solely on the data. We then incorporate physical knowledge by using the governing differential equation to define a physics-informed loss, allowing for unsupervised learning from only the initial condition.

Finally, we introduce a hybrid learning approach–combining both supervised learning from data and unsupervised learning from the equation–and demonstrate that it yields superior performance compared to the other methods.

The entire code details regarding Newton's cooling law can be found at the following notebook:

`https://tinyurl.com/443htwmv`

Using interpolation of data We shall reconstruct the temperature from the data measurements given in Table 2.1. The temperature $u(t)$ is measured in degrees Celsius and the time t is in minutes. Thus, the initial temperature is $u(0) = 90°\text{C}$, and after 10 minutes it has dropped to $u(10) = 30.40°\text{C}$. These are not experimental measurements

t	$u(t)$
0.00	90.00
1.11	64.42
2.22	49.75
3.33	41.33
4.44	36.50
5.56	33.73
6.67	32.14
7.78	31.22
8.89	30.70
10.00	30.40

Table 2.1: Data points from Newton's Law of Cooling: $u(t) = 30 + (90 - 30)e^{-0.5t}$.

but synthetic data generated from the exact cooling law

$$u(t) = 30 + (90 - 30)e^{-0.5t},$$

as shown by the following code snippet:

```
# Constants for Newton's Law of Cooling
T = 30       # Ambient temperature
u0 = 90      # Initial temperature
k = 0.5      # Decay constant

# Exact solution of Newton's Law of Cooling
def exact_solution(t):
    return T + (u0 - T) * np.exp(-k * t)

# Generate 10 data points from the exact solution
t_data = np.linspace(0, 10, 10).reshape(-1, 1)
# 10 points in the interval [0, 10]
u_data = exact_solution(t_data)
```

Plotting the data,

```
# Plot the 10 data points (without the exact solution line)
plt.figure(figsize=(10, 6))
plt.scatter(t_data, u_data, color='b', label="Data Points", zorder=5)
plt.xlabel('Time (t)')
plt.ylabel('Temperature (u(t))')
plt.title("10 Data Points from Newton's Cooling Law")
plt.legend()
plt.grid(True)
plt.show()
```

we obtain the plot as in Fig. 2.11.

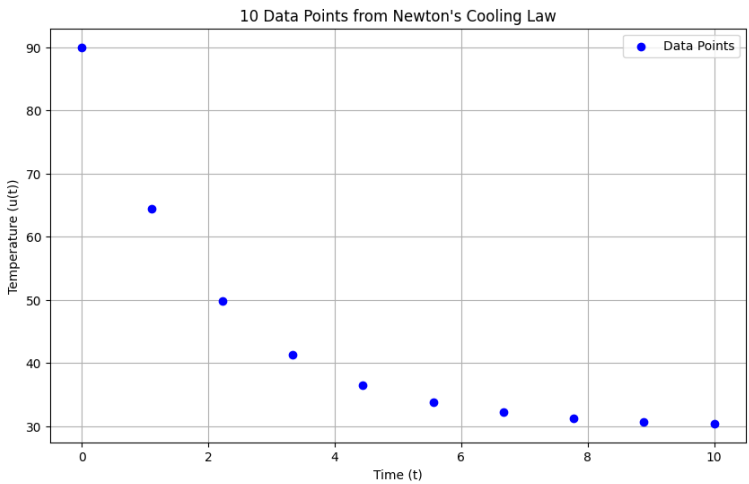


Figure 2.11: The plot graph of 10 temperature measurements of a hot body.

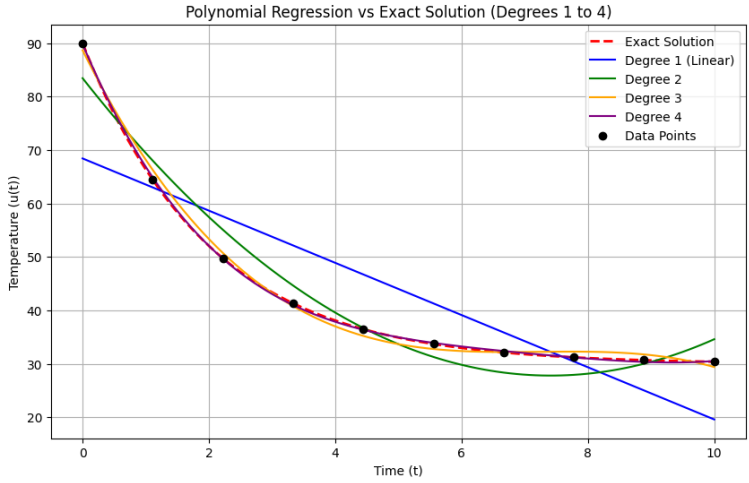


Figure 2.12: The regression polynomials versus the exact solution.

We fit polynomial regression models with varying degrees, from 1 to 4, then plot the exact solution, and the data points:

```
# Exact solution of Newton's Law of Cooling
def exact_solution(t):
    return T + (u0 - T) * np.exp(-k * t)

# Function to perform polynomial regression and predict values
def polynomial_regression(t_data, u_data, degree, t_test):
    poly = PolynomialFeatures(degree)
    t_poly = poly.fit_transform(t_data)

    # Train the model
    model = LinearRegression()
    model.fit(t_poly, u_data)

    # Predict values using the fitted model
    t_test_poly = poly.transform(t_test)
    u_pred = model.predict(t_test_poly)

    return u_pred

# Generate 10 data points from the exact solution
t_data = np.linspace(0, 10, 10).reshape(-1, 1)
# 10 points in the interval [0, 10]
u_data = exact_solution(t_data)
```

```
# Generate a range of time values for plotting
t_test = np.linspace(0, 10, 100).reshape(-1, 1)

# Perform polynomial regression for degrees 1, 2, 3, and 4
u_pred_deg1 = polynomial_regression(t_data, u_data, 1, t_test)
u_pred_deg2 = polynomial_regression(t_data, u_data, 2, t_test)
u_pred_deg3 = polynomial_regression(t_data, u_data, 3, t_test)
u_pred_deg4 = polynomial_regression(t_data, u_data, 4, t_test)

# Exact solution for comparison
u_exact = exact_solution(t_test)
```

We observe from Fig. 2.12 that the degree-4 polynomial provides the best fit over the interval $t \in [0, 10]$. Note, however, that outside this interval the degree-4 polynomial will diverge to infinity and no longer be reliable. This is a serious reason of using another interpolation method, such as using neural networks.

Using supervised learning and neural nets We shall use a neural network and supervised learning to learn the temperature from the data in Table 2.1. The following code constructs a feed-forward neural network with two hidden layers of 40 neurons each:

```
model = Sequential([
    Input(shape=(1,)),
    Dense(40, activation='relu'),
    Dense(40, activation='relu'),
    Dense(1,  activation='linear')
])
```

The network is trained for 2,000 epochs on the 10 observed pairs (t_i, u_i) from Newton's cooling experiment. Being a supervised learning, we use the mean squared error:

```
model.compile(optimizer='adam', loss='mse')
```

Unsurprisingly, the reconstructed temperature curve does not perfectly match the exact solution, as there simply isn't enough information between those sparse measurements to pin down the shape of the curve exactly, see Fig. 2.13.

It is worth noting that simply increasing the number of training epochs does not eliminate this mismatch, since the loss has already saturated near zero after the first 1,000 epochs, see Fig. 2.14.

Therefore, increasing the network capacity, such as the width and the depth would be the solution of learning faster the data. To show this, in the following we used three architectures of increasing capacities, as follows:

(i) The first network has one hidden layer with 20 neurons;
(ii) The second network has two hidden layers with $40 : 20$ neurons;
(iii) The third network has three hidden layers with $80 : 40 : 20$ neurons.

These model architectures are implemented as follows:

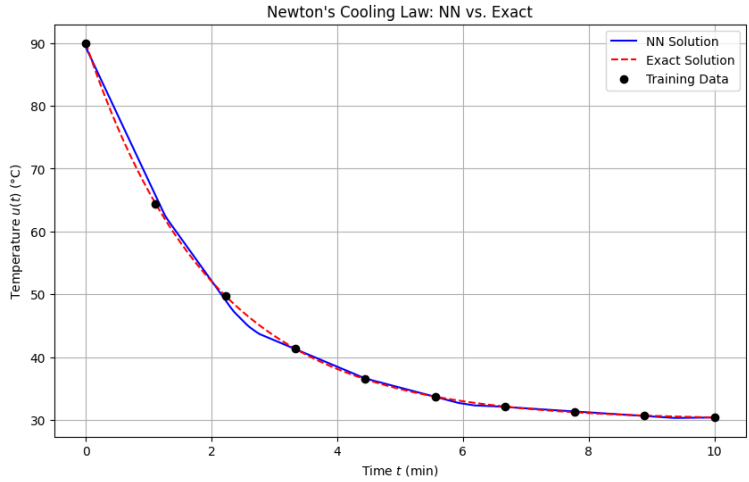


Figure 2.13: Learning the temperature curve by supervised learning from 10 measurements.

Figure 2.14: The loss versus number of epochs in the case of supervised learning.

```
def make_model(arch_id):
    m = Sequential([ Input(shape=(1,)) ])
    if arch_id == 1:
        m.add(Dense(20, activation='relu'))
    elif arch_id == 2:
        m.add(Dense(40, activation='relu'))
        m.add(Dense(20, activation='relu'))
    else:  # arch_id == 3
        m.add(Dense(80, activation='relu'))
        m.add(Dense(40, activation='relu'))
        m.add(Dense(20, activation='relu'))
    m.add(Dense(1, activation='linear'))
    m.compile(optimizer='adam', loss='mse')
    return m
```

The evolution of the loss functions for these three architectures over the number of epochs is shown in Fig. 2.15. We observe that the larger-capacity network converges in fewer epochs.

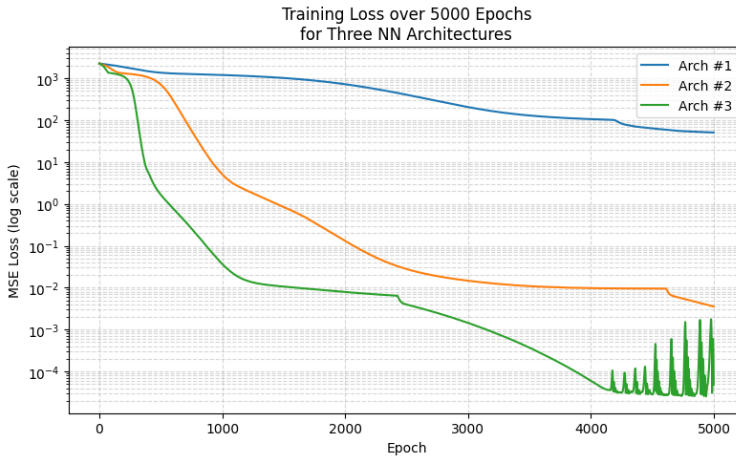


Figure 2.15: The loss versus number of epochs in the case of supervised learning for three architectures of increasing capacities.

We note in Fig. 2.15 an oscillation of the loss for the third architecture. This is due to the fact that the learning rate was not reduced properly in that regime, and each update overshoots the true minimum and bounces back, producing small oscillations around the optimal point.

In what follows, we will demonstrate how to achieve an exact match between the network's approximation and the true cooling curve using two strategies: the *unsupervised approach*, which relies solely on the underlying physical law (in this case–Newton's cooling equation), and the *hybrid approach* that combines a physics-informed loss with the available measurement data.

Using unsupervised learning We shall use a Physics-Informed Neural Network (PINN) to solve Newton's law of cooling

$$\frac{du}{dt} = -k\big(u(t) - T_{\text{env}}\big), \quad u(0) = u_0,$$

without relying on intermediate labeled data. The key point is that we know the exact law of temperature evolution. We also know the environment temperature, $T_{\text{env}} = 30.0$, the initial temperature of the body, $u_0 = 90.0$, and the decay constant, $k = 0.5$.

Let $f_\theta : [0, 10] \longrightarrow \mathbb{R}$ be a feed-forward network with parameters θ. Its output $f_\theta(t)$ is intended to approximate the true temperature $u(t)$, which is the solution of the aforementioned initial condition problem.

We define the residual of the ODE at a collocation point t_j as

$$R(t_j; \theta) = \frac{d}{dt} f_\theta(t_j) + k\left(f_\theta(t_j) - T_{\text{env}}\right).$$

The physics-informed loss is then

$$\mathcal{L}_{\text{ODE}}(\theta) = \frac{1}{N} \sum_{j=1}^{N} \big[R(t_j; \theta)\big]^2.$$

We enforce the initial condition $f_\theta(0) = u_0$ via

$$\mathcal{L}_{\text{IC}}(\theta) = \big[f_\theta(0) - u_0\big]^2.$$

Then we combine the two contributions as

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{ODE}}(\theta) \; + \; \lambda\, \mathcal{L}_{\text{IC}}(\theta),$$

where $\lambda > 0$ balances the strength of the initial-condition constraint.

We minimize $\mathcal{L}(\theta)$ with respect to θ using a gradient-based optimizer. Because no measured $u(t)$ values are required (besides the initial condition $u(0)$), this method is fully *unsupervised*.

The trained network $f_{\theta^*}(t)$ yields a smooth approximate solution over the entire interval $[0, 10]$, recovering the exponential cooling behavior from first principles alone. If we train for about 10,000 epochs we obtain a very good match to the exact solution, see Fig. 2.16.

However, Fig. 2.17 shows that the loss saturates near zero after 6,000 epochs. Although this unsupervised learning recovers the exact solution, it takes far too long to train. Can we accelerate convergence by incorporating a few measured data points of the body temperature? The answer is yes! In the next paragraph, we introduce a hybrid approach which does just that.

Using hybrid learning In the *hybrid* approach, we combine both measurement data and the physics-informed loss to train the network $f_\theta(t)$. Specifically, we define three loss terms:

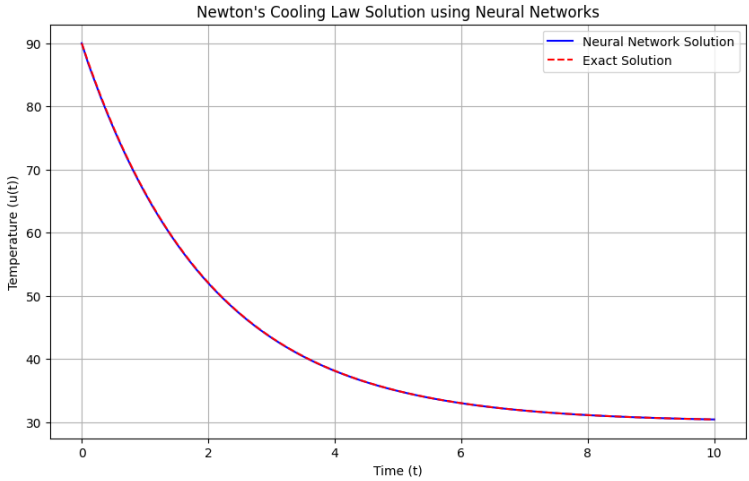


Figure 2.16: The unsupervised learning after 10,000 epochs of training produces a perfect match to the exact solution.

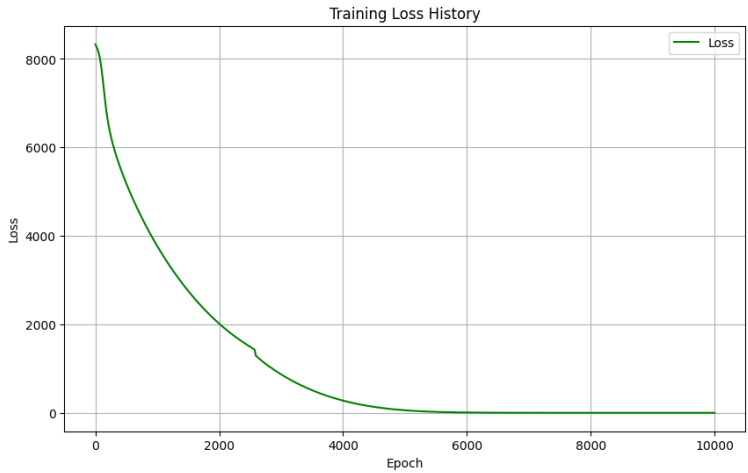


Figure 2.17: The loss evolution for unsupervised training saturates after 6,000 epochs.

(*i*) The Data Loss:

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} \big[f_\theta(t_i) - u_i\big]^2,$$

which enforces the network to match the measured temperatures $\{(t_i, u_i)\}$.
(*ii*) The ODE Loss:

$$\mathcal{L}_{\text{ODE}}(\theta) = \frac{1}{N_{\text{ODE}}} \sum_{j=1}^{N_{\text{ODE}}} \Big[\tfrac{d}{dt} f_\theta(t_j) + k\big(f_\theta(t_j) - T_{\text{env}}\big)\Big]^2,$$

which enforces Newton's cooling law at collocation points $\{t_j\}$, with derivatives computed by automatic differentiation.
(*iii*) The Initial-Condition Loss:

$$\mathcal{L}_{\text{IC}}(\theta) = \big[f_\theta(0) - u_0\big]^2.$$

The *total loss* is a weighted sum:

$$\mathcal{L}(\theta) = w_{\text{data}}\,\mathcal{L}_{\text{data}} \;+\; w_{\text{ODE}}\,\mathcal{L}_{\text{ODE}} \;+\; w_{\text{IC}}\,\mathcal{L}_{\text{IC}},$$

where the weights w_{data}, w_{ODE}, $w_{\text{IC}} > 0$ balance data fidelity against physical conformity and the initial condition.

By anchoring the solution at measured points and filling in the gaps via the ODE, the hybrid PINN typically converges much faster than a purely physics-informed model, while still respecting both data and underlying Physics, see Fig 2.18. That a hybrid learning approach converges in far fewer epochs is evident from the loss curve, which drops steeply and then saturates near zero after only about 500 epochs, see Fig. 2.19.

Remark 2.5.5 In order to improve the learning process we can adopt a *trial-solution* ansatz

$$u(t) \;=\; u_\theta(t) \;=\; u_0 \;+\; t\, f_\theta(t),$$

which is a method introduced at page 54. This enforces the initial condition $u(0) = u_0$ by construction, thereby eliminating any separate initial-condition term in the loss. Substituting into Newton's cooling law,

$$\frac{du}{dt} = f_\theta(t) + t\,\frac{d}{dt} f_\theta(t) \;=\; -\,k\big(u(t) - T_{\text{env}}\big),$$

we define the single physics-informed loss

$$\mathcal{L}(\theta) = \int_0^T \Big[f_\theta(t) + t\, f'_\theta(t) + k\big(u_0 + t\, f_\theta(t) - T_{\text{env}}\big)\Big]^2\, dt.$$

This ansatz both accelerates training, since the network only needs to learn the correction $f_\theta(t)$, and avoids the need for an explicit boundary-condition loss. The details of this implementation can be found in the Colab notebook accompanying this section.

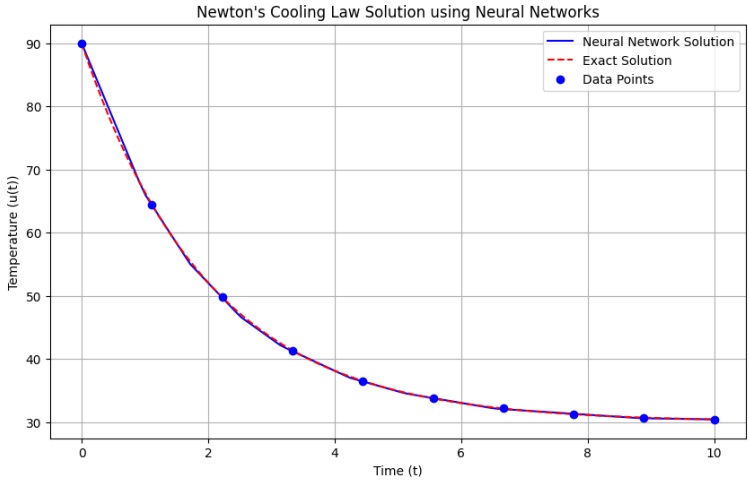


Figure 2.18: The hybrid learning after 6,000 epochs.

Figure 2.19: The loss evolution for hybrid training saturates after just 500 epochs.

2.5.5 Exercises

Exercise 2.5.6 Consider the initial-value problem

$$\frac{du}{dt} = -u + \sin t, \qquad u(0) = 2,$$

on the interval $t \in [0, 2]$.

(a) Show that the given problem has a unique solution and find it.

(b) Use each of the following neural-network approaches to approximate $u(t)$, and compare their accuracy:

1. Take a feed-forward network $f_\theta(t) \approx u(t)$ and minimize the loss

$$\mathcal{L}(\theta) = \big[f_\theta(0) - 2\big]^2 + \sum_i \Big[f'_\theta(t_i) + f_\theta(t_i) - \sin t_i\Big]^2,$$

   sampled at collocation points $\{t_i\} \subset [0, 2]$.

2. Use the form $u(t) = 2 + t\,NN_\theta(t)$, which enforces $u(0) = 2$ exactly, and minimize

$$\sum_i \Big[NN_\theta(t_i) + t_i\,NN'_\theta(t_i) + t_i\,NN_\theta(t_i) - \sin t_i\Big]^2.$$

3. Write the ODE in the integral form $u(t) = 2 + \int_0^t (-s + \sin s)\,ds$ and solve it using neural networks.

(c) For each method:

- Train over a suitable grid of t_i.
- Plot the learned $u(t)$ against the exact solution.
- Report the maximum absolute error on $[0, 2]$ in each case and state which is the best method for this ODE.

Exercise 2.5.7 Consider the second-order initial-value problem

$$\frac{d^2u}{dt^2} + u = \cos(2t), \qquad u(0) = 1, \quad u'(0) = 0,$$

for $t \in [0, \pi]$.

(a) Find its exact solution.

(b) Apply each of the following neural-network approaches to approximate $u(t)$:

1. Let $f_\theta(t) \approx u(t)$. Sample collocation points $\{t_i\} \subset [0, \pi]$ and minimize

$$\mathcal{L}(\theta) = \big[f_\theta(0) - 1\big]^2 + \big[f'_\theta(0)\big]^2 + \frac{1}{N}\sum_i \Big[f''_\theta(t_i) + f_\theta(t_i) - \cos(2t_i)\Big]^2.$$

2. Use the built-in ansatz

$$u(t) = 1 + 0 \cdot t + \tfrac{1}{2}\, t^2\, NN_\theta(t),$$

and minimize

$$\frac{1}{N} \sum_i \Big[t_i^2 NN_\theta''(t_i) + 2t_i NN_\theta'(t_i) + NN_\theta(t_i) - \cos(2t_i) \Big]^2.$$

3. Set $y_1 = u,\ y_2 = u'$. Then

$$\begin{cases} y_1' = y_2, \\ y_2' = \cos(2t) - y_1, \end{cases} \qquad y_1(0) = 1,\ y_2(0) = 0.$$

Use a network $(f_\theta, g_\theta) \approx (y_1, y_2)$ and minimize

$$\frac{1}{N} \sum_i \big[f_\theta'(t_i) - g_\theta(t_i) \big]^2 + \big[g_\theta'(t_i) + f_\theta(t_i) - \cos(2t_i) \big]^2 + \big[f_\theta(0) - 1 \big]^2 + \big[g_\theta(0) \big]^2.$$

(*c*) For each method:

- Train on a uniform grid of N collocation points in $[0, \pi]$.
- Plot the learned solution versus $u_{\text{exact}}(t)$.
- Report the maximum absolute error $\max_t |u_{\text{NN}}(t) - u_{\text{exact}}(t)|$ and state which neural net solution is closer to the exact solution.

2.6 A Hybrid Method

Numerical methods and Deep Learning can often complement each other when solving differential equations. A common strategy is to first approximate the solution with Euler's method and then refine it using a neural network.

Concretely, we consider the initial-value problem

$$\frac{dy}{dt} = f(t, y), \quad y(0) = y_0, \tag{2.6.18}$$

together with n measurements $\{(t_j, y_j^*)\}_{j=1}^n$. We seek a function $y(t)$ that not only satisfies the differential equation (2.6.18) but also interpolates the data as

$$y(t_j) = y_j^*, \quad j = 1, \ldots, n.$$

One way to solve equation (2.6.18) is using Euler's method, which converts the ODE into a discrete update scheme. To this end, we choose a small enough step size Δt and define

$$t_n = n\, \Delta t, \quad t_0 = 0.$$

Then for $n = 0, 1, 2, \ldots$, iterate

$$y_{n+1} = y_n + \Delta t\, f\big(t_n, y_n\big).$$

This formula follows from the finite-difference approximation

$$\frac{y(t_{n+1}) - y(t_n)}{\Delta t} \;\approx\; \frac{dy}{dt}\Big|_{t_n} = f\big(t_n, y(t_n)\big).$$

Euler's method gives a quick-and-dirty way to get a rough solution

$$y_{\rm Eu}(t) \;=\; y_k \;+\; f\big(t_k, y_k\big)\,(t - t_k), \qquad t_k \le t < t_{k+1},$$

which we shall refine by adding a neural-network-based correction. Since in general, the Euler estimates

$$y_k \;=\; y_{\rm Eu}(t_k)$$

do not match the observations y_k^*, giving errors

$$y_k^* - y_k, \quad k = 1, \ldots, n,$$

we shall introduce a feed-forward correction network $h_\theta(t)$ to offset these discrepancies. Fixing the data $\{(t_k, y_k^*)\}$, we define the loss

$$\mathcal{L}(\theta) \;=\; \sum_{k=1}^{n} \big(y_k^* - y_k - h_\theta(t_k)\big)^2 \;+\; \big(h_\theta(0)\big)^2,$$

and find $\theta^* = \arg\min_\theta \mathcal{L}(\theta)$ by gradient descent. The final corrected solution can be written as

$$y(t) \;=\; y_{\rm Eu}(t) \;+\; h_{\theta^*}(t).$$

2.6.1 Implementation

We shall implement this method in the case of the simple ODE

$$\frac{dy}{dt} = y, \quad y(0) = 1,$$

for which the exact solution is known, $y_{\rm exact}(t) = e^t$.

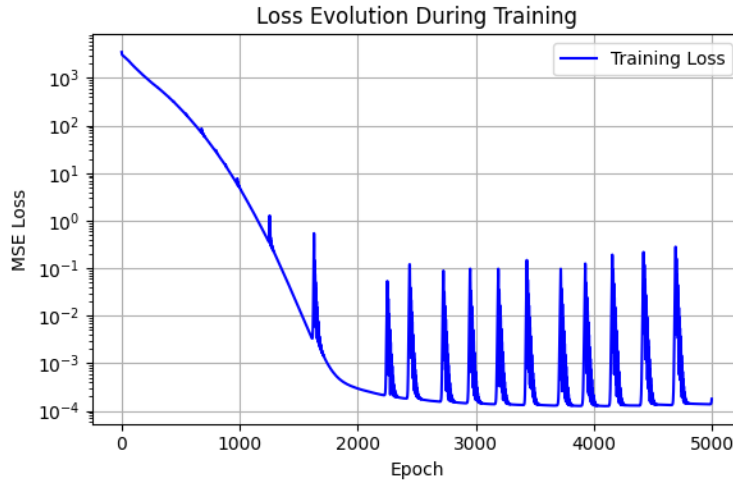
We implemented the procedure on the interval $[0, 6]$, by taking the step $\Delta t = 0.2$ and using a correction network $h_\theta(t)$ given by

```
inp = layers.Input(shape=(1,))
x = layers.Dense(64, activation='tanh')(inp)
x = layers.Dense(32, activation='tanh')(x)
out = layers.Dense(1)(x)
model_h = Model(inp, out)
```

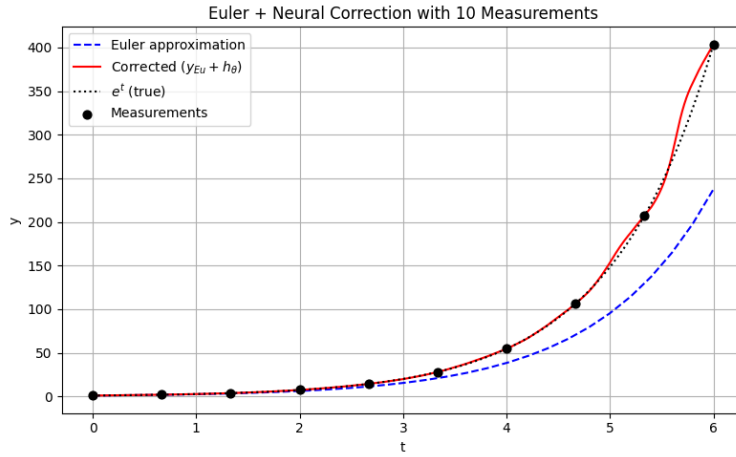
The training was performed for $5,000$ epochs and the loss and the graphs of the Euler's solution, hybrid solution and exact solution have been plotted in Fig. 2.20.

We note that Euler's solution diverges from the exact solution, $y_{\rm exact}(t) = e^t$, while the hybrid solution $y(t) = y_{Eu}(t) + h_\theta(t)$ fixes this inconvenient to a certain extent. The code details are available at

`https://tinyurl.com/mrxx9xjc`



(a)



(b)

Figure 2.20: (a) The loss function. (b) The graphs of solutions $y_{\text{Eu}}(t)$, $y(t)$ and $y_{\text{exact}}(t) = e^t$.

2.7 Equation Discovery from Data

Sometimes we need to infer an unknown differential equation directly from data. This is an inverse problem that identifies the ODE model that best reproduces the observations. Such tasks are ideally suited to deep-learning methods. Concretely, we consider the following one-dimensional ODE problem:

Problem *Find a function F so that the solution of the initial-value problem*

$$\frac{dx(t)}{dt} = F\big(x(t)\big), \qquad x(0) = x_0,$$

passes through the points $\{(t_j, x_j^*)\}_{j=1}^n$, i.e. $x(t_j) = x_j^*$ for each j.

We can tackle this problem in two ways:

1. *Pure Deep Learning*, where a neural network learns both the function F and the solution $x(t)$.
2. A *hybrid approach*, in which we first construct a rough solution via Euler's method and then refine it with a neural-network correction.

1. Pure Deep Learning. We employ a neural network

$$\mathrm{NN}(\theta; t, z) \;=\; \big(x_\theta(t),\, F_\theta(z)\big),$$

which takes $(t, z) \in \mathbb{R}^2$ as input. We then select N collocation points $\{t_j\}_{j=1}^N$ and train the network by minimizing the composite loss

$$\mathcal{L}(\theta) \;=\; L_{\mathrm{data}}(\theta) \;+\; \lambda\, L_{\mathrm{ODE}}(\theta),$$

where

$$L_{\mathrm{data}}(\theta) = \frac{1}{n}\sum_{k=1}^{n}\big(x_\theta(t_k) - x_k^*\big)^2, \qquad L_{\mathrm{ODE}}(\theta) = \frac{1}{N}\sum_{j=1}^{N}\big(\dot{x}_\theta(t_j) - F_\theta\big(x_\theta(t_j)\big)\big)^2,$$

and $\lambda > 0$ is a hyperparameter, roughly chosen as $\lambda \approx n/N$. If $\theta^* = \arg\min \mathcal{L}(\theta)$, then the desired function is F_{θ^*}.

2. A hybrid approach. We use a feed-forward neural network

$$\mathrm{NN}(\theta; z) = F_\theta(z)$$

to approximate the dynamics, and then generate a discrete solution via Euler's method:

$$\begin{aligned}
x_0 &= x(0),\\
x_1 &= x(t_1) = x_0 + F_\theta(x_0)\,(t_1 - 0),\\
&\;\;\vdots\\
x_n &= x(t_n) = x_{n-1} + F_\theta(x_{n-1})\,(t_n - t_{n-1}).
\end{aligned}$$

We then train F_θ by minimizing the data-fitting loss

$$\mathcal{L}(\theta) \;=\; \sum_{j=1}^{n}\big(x_j(\theta) - x_j^*\big)^2.$$

The learned dynamics is $F(z) = F_{\theta^*}(z)$, where $\theta^* = \arg\min_\theta \mathcal{L}(\theta)$.

2.8 Inverting Integrals

Let $g : [a, \infty) \to (0, \infty)$ be a continuous and positive function, and define

$$F(u) = \int_a^u g(x)\, dx.$$

Since $F'(u) = g(u) > 0$, the function F is strictly increasing on (a, ∞), and hence invertible on its range, with inverse $\varphi(x) = F^{-1}(x)$. Then the composition identity

$$F(\varphi(u)) = u$$

yields the equivalent integral form

$$\int_a^{\varphi(u)} g(x)\, dx = u. \tag{2.8.19}$$

Solving certain second-order differential equations sometimes reduces to an equation of the type (2.8.19); that is, to the problem of inverting an integral. If an explicit antiderivative F of g is known and its inverse F^{-1} can be computed analytically, then the solution $\varphi(u) = F^{-1}(u)$ is readily obtained. However, in most cases such an inversion is not analytically feasible. To address this, we propose using neural networks to approximate the inverse function φ and thus solve the problem numerically.

To this end, we employ a feed-forward neural network with a one-dimensional input u and one-dimensional output $f_\theta(u)$, which is trained to approximate the upper limit of integration function, i.e., $f_\theta(u) \approx \varphi(u)$. To achieve this, we define the following loss function:

$$\mathcal{L}(\theta) = \sum_{i=1}^{n} \Big(F\big(f_\theta(u_i)\big) - u_i \Big)^2 + \lambda \Big(f_\theta(0) - a \Big)^2, \tag{2.8.20}$$

where the first term enforces the functional identity $F(f_\theta(u)) = u$ for $u > a$, and the second term enforces the initial condition $f_\theta(0) = a$. The validity of this condition stems from the fact that $F(a) = 0$, which implies $\varphi(0) = a$. The collocation points $\{u_i\}_{i=1}^n$ are sampled from a bounded interval $[0, M]$, which lies within the range of F, so the network is trained to approximate the function $\varphi : [0, M] \to [a, \varphi(M)]$.

Minimizing this loss using a gradient-based optimizer yields an optimal parameter $\theta^* = \arg\min_\theta \mathcal{L}(\theta)$, and the resulting network $f_{\theta^*}(u)$ provides the desired approximation to $\varphi(u)$. The Lagrange multiplier $\lambda > 0$ is chosen to sufficiently enforce the initial condition.

Example 2.8.1 We illustrate a case of physical significance where solving a problem reduces to inverting an integral. Consider a force derived from a potential, given by $F(x) = -U'(x)$, and Newton's second law in the form

$$\ddot{x}(t) = F(x(t)).$$

Define the total energy as the sum of kinetic and potential energy:

$$E(x, \dot{x}) = \frac{1}{2}\dot{x}^2 + U(x).$$

This quantity is conserved along the trajectory, as shown by:

$$\frac{d}{dt}E(x,\dot{x}) = \dot{x}\big(\ddot{x} + U'(x)\big) = 0,$$

where we used the equation of motion. We note that we used notations $\dot{x} = \frac{dx}{dt}$ and $U' = \frac{dU}{dx}$. Hence, the energy remains constant:

$$E\big(x(t), \dot{x}(t)\big) = E.$$

Rewriting this as

$$\frac{1}{2}\dot{x}^2 + U(x) = E,$$

and solving for $\dot{x}$, we obtain the separable form

$$\frac{dx}{\sqrt{2E - U(x)}} = dt.$$

Integrating both sides yields

$$\int_{x_0}^{x(t)} \frac{1}{\sqrt{2E - U(s)}}\, ds = t,$$

where $x_0 = x(0)$ is the initial position. Determining the trajectory $x(t)$ now amounts to inverting this integral.

2.8.1 Practical Implementations

We consider a particular example where $g(x) = \frac{1}{x}$, with $x \geq 1$. The associated function can be computed explicitly as

$$F(u) = \int_1^u \frac{1}{x}\, dx = \ln u.$$

The inverse function is $\varphi(x) = F^{-1}(x) = e^x$. Our goal is to approximate this inverse using a neural network, that is, to learn $f_\theta(x) \approx e^x$. In this case the associated loss has the following form

$$\mathcal{L}(\theta) = \sum_{i=1}^{n} \Big(\int_1^{f_\theta(u_i)} \frac{1}{x}\, dx - u_i \Big)^2 + \lambda \Big(f_\theta(0) - 1 \Big)^2.$$

Using an architecture with only two hidden layers

```
model = Sequential([
    Dense(50, activation='tanh', input_shape=(1,)),
    Dense(50, activation='tanh'),
    Dense(1)
])
```

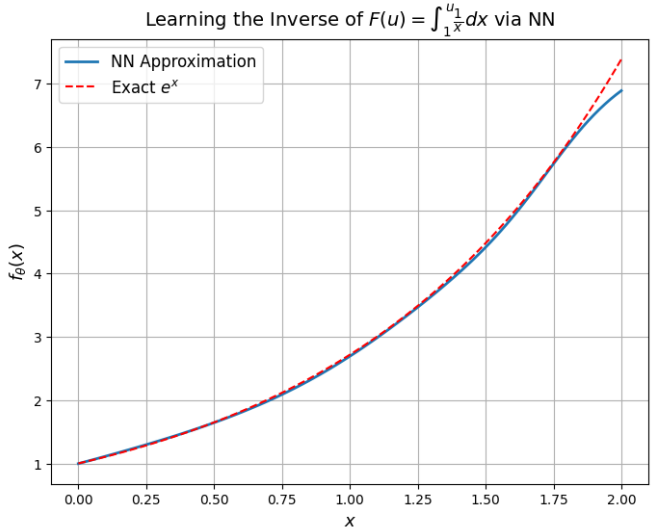


Figure 2.21: The graphs of the exact solution $y = e^x$ versus the neural network's output $y = f_\theta(x)$ trained for 500 epochs.

and training for 500 epochs, we obtained an almost a perfect match of the exponential function, see Fig. 2.21. The goodness approximation can be seen from the error obtained:

```
Epoch 500/500
4/4  0s 11ms/step - loss: 0.0020
```

The entire code details can be found at the notebook:

`https://tinyurl.com/2dw8jtz5`

2.8.2 Exercises

Exercise 2.8.1 Consider the integral equation

$$\int_0^{u(t)} \frac{dx}{1+x^2} = t, \qquad t \in [0, T].$$

Use a feed-forward neural network to approximate the solution $u(t)$ following the steps:
(a) Construct a neural network $f_\theta(t) \approx u(t)$, where f_θ is a function of one variable.
(b) Define the residual at each collocation point t_i as

$$R_i(\theta) = \int_0^{f_\theta(t_i)} \frac{dx}{1+x^2} - t_i.$$

(c) Use numerical quadrature (e.g., midpoint or Riemann sum) to approximate the integral in the residual.

(d) Define the loss function as

$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} R_i(\theta)^2,$$

where $\{t_i\}_{i=1}^n \subset [0,T]$ are the collocation points.

(e) Train the network by minimizing $\mathcal{L}(\theta)$, and plot the approximation $f_\theta(t)$ of $u(t)$.

(f) Using that the exact solution is $u(t) = \tan(t)$, validate the neural network's output.

Exercise 2.8.2 Consider the nonlinear integral equation

$$\int_0^{\cos\alpha(t)} \frac{dx}{\sqrt{x(1-x^2)}} = t, \qquad t \in [0,T],$$

where $\alpha(t)$ is an unknown function to be determined.

Use a neural network to approximate the solution $\alpha(t)$ satisfying the above equation and following the steps:

(a) Construct a feed-forward neural network $f_\theta(t) \approx \alpha(t)$.

(b) For each t_i in a set of collocation points $\{t_1, t_2, \ldots, t_n\} \subset [0,T]$, compute the residual

$$R_i(\theta) = \int_0^{\cos f_\theta(t_i)} \frac{dx}{\sqrt{x(1-x^2)}} - t_i.$$

(c) Define the loss function as

$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} R_i(\theta)^2.$$

(d) Train the network to minimize $\mathcal{L}(\theta)$ and plot the learned approximation $f_\theta(t)$ over $[0,T]$.

2.9 Two-Phase Resolution Neural Network Solution

Consider the linear operator equation $P(\varphi) = g$, where P is a known linear mapping on a space of continuous functions, g is the given right-hand side, and φ is the unknown solution. We describe next a two-phase resolution neural-network strategy that divides the task into a coarse approximation followed by a refinement as follows. This accuracy control technique can be traced back to the work of Briggs [12].

Phase I: Low-Capacity Approximation

1. Choose a *low-capacity* feed-forward network f_θ (with relatively few parameters) and train it to minimize the data-fidelity loss

$$\mathcal{L}_1(\theta) = \big\|P(f_\theta) \;-\; g\big\|^2.$$

2. After training, f_θ provides a rough approximation of the solution, $\varphi \approx f_\theta$. Then we define the residual

$$r \;=\; g \;-\; P(f_\theta).$$

Phase II: High-Capacity Residual Correction

1. Introduce the correction function $\psi = \varphi - f_\theta$. By linearity of P,

$$P(\psi) \;=\; P(\varphi) - P(f_\theta) \;=\; g - P(f_\theta) \;=\; r.$$

Hence, ψ satisfies the subsidiary equation

$$P(\psi) = r.$$

2. Approximate ψ with a *higher-capacity* network h_w, trained to minimize the new loss

$$\mathcal{L}_2(w) = \big\|P(h_w) \;-\; r\big\|^2,$$

so that we have the approximation $\psi \approx h_w$.

In order to reconstruct the solution φ of the initial problem, $P(\varphi) = g$, we consider the sum of the two networks,

$$\hat{\varphi}(x) \;=\; f_\theta(x) \;+\; h_w(x),$$

and note that

$$P\big(\hat{\varphi}\big) = P(f_\theta) + P(h_w) \approx (g - r) + r = g,$$

which provides an accurate approximation to the true solution φ, namely, $\varphi \approx \hat{\varphi}$.

This two-phase approach leverages a cheap coarse fit, f_θ, to capture the main solution behavior and a richer corrective network, h_w, to resolve fine details, improving training stability.

Remark 2.9.1 Note that the above two-phase strategy relies critically on the linearity of the operator $\mathcal{P}$. If $\mathcal{P}$ is nonlinear, one can first linearize it, by computing its Fréchet derivative $\mathcal{P}'(f_\theta)$ around the coarse approximation f_θ, and then apply the same residual-correction method to the resulting linearized equation $\mathcal{P}'(f_\theta)\,\psi = r$.

Example: ODE with Oscillatory Source

As a nontrivial test of the two-phase strategy, consider the first order differential equation with final condition on the interval $[-5, 0]$:

$$\varphi'(x) + \varphi(x) \;=\; e^{-x}\,\sin\!\big(\tfrac{x}{k}\big), \qquad \varphi(0) = 0,$$

where $k > 0$ is a given constant.

To find the analytical solution, we use an integrating-factor calculation with $\mu(x) = e^x$ to obtain

$$\varphi(x) = ke^{-x}\Big(1 - \cos\frac{x}{k}\Big).$$

Its graph oscillates between the graphs $y = 0$ and $y = 2ke^{-x}$, see Fig. 2.22.

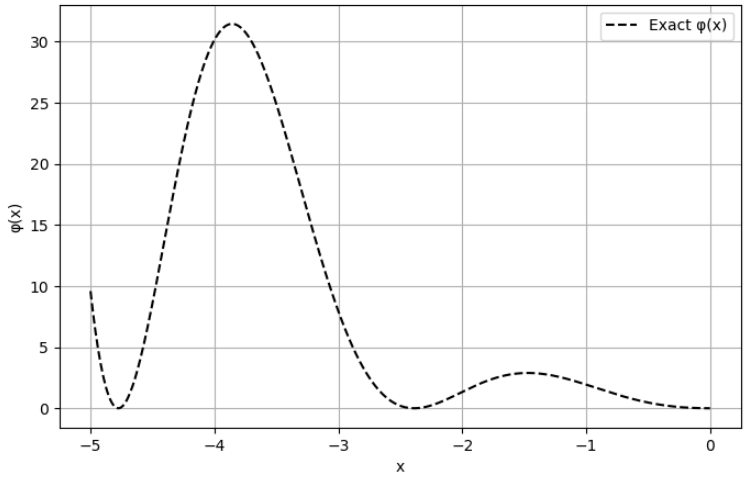


Figure 2.22: Graph of the solution $\varphi(x) = ke^{-x}\left(1 - \cos\frac{x}{k}\right)$, with $k = 0.38$.

Phase I: Low-Capacity Approximation

1. Define a small network $f_\theta(x)$ (e.g. one hidden layer of 15 neurons with tanh activations):

```
f_model = Sequential([
  Input(shape=(1,)),
  Dense(15, activation='tanh'),
  Dense(1)
])
```

2. Train f_θ on 500 collocation points $\{x_i\} \subset [-5, 0]$ by minimizing

$$\mathcal{L}_1(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left[f'_\theta(x_i) + f_\theta(x_i) - e^{-x_i}\sin(x_i/k)\right]^2,$$

using automatic differentiation to compute f'_θ.

```
x_train =
    np.linspace(-5.0, 0.0, 500).reshape(-1,1).astype(np.float32)
def train_step_phase1(x):
  with tf.GradientTape(persistent=True) as tape:
      tape.watch(x)
      f_pred = f_model(x)                  # f(x)
      f_x = tape.gradient(f_pred, x)       # f'(x)
      res = f_x + f_pred - rhs_tf(x)       # residual
      loss = tf.reduce_mean(tf.square(res))
```

```
    # compute gradients w.r.t. model weights
    grads = tape.gradient(loss, f_model.trainable_weights)
  optimizer1.apply_gradients(zip(grads, f_model.trainable_weights))
    del tape
    return loss

# Train Phase I
for epoch in range(epochs_phase1):
    loss1 = train_step_phase1(tf.constant(x_train))
    if epoch % 100 == 0:
        print(f"Phase I Epoch {epoch}, Loss: {loss1.numpy():.6f}")
```

3. Compute the residual

$$r(x_i) = e^{-x_i}\sin\left(\tfrac{x_i}{k}\right) - \left[f'_\theta(x_i) + f_\theta(x_i)\right]$$

as in the code

```
  x_tensor = tf.constant(x_train)
with tf.GradientTape() as tape:
    tape.watch(x_tensor)
    f_pred_tf = f_model(x_tensor)
f_x_tf = tape.gradient(f_pred_tf, x_tensor)
r_train = rhs_tf(x_tensor) - (f_x_tf + f_pred_tf)  # shape (500,1)
r_tf = tf.stop_gradient(r_train)
```

Phase II: High-Capacity Residual Correction

1. Introduce the correction $\psi(x) = \varphi(x) - f_\theta(x)$. By linearity,

$$\psi'(x) + \psi(x) = \left[\varphi'(x) + \varphi(x)\right] - \left[f'_\theta(x) + f_\theta(x)\right] = r(x).$$

2. Approximate ψ with a larger network $h_w(x)$, trained to minimize

$$\mathcal{L}_2(w) = \frac{1}{N}\sum_{i=1}^{N}\left[h'_w(x_i) + h_w(x_i) - r(x_i)\right]^2.$$

The larger network has two layers:

```
  h_model = Sequential([
    Input(shape=(1,)),
    Dense(50, activation='tanh'),
    Dense(50, activation='tanh'),
    Dense(1)
])
```

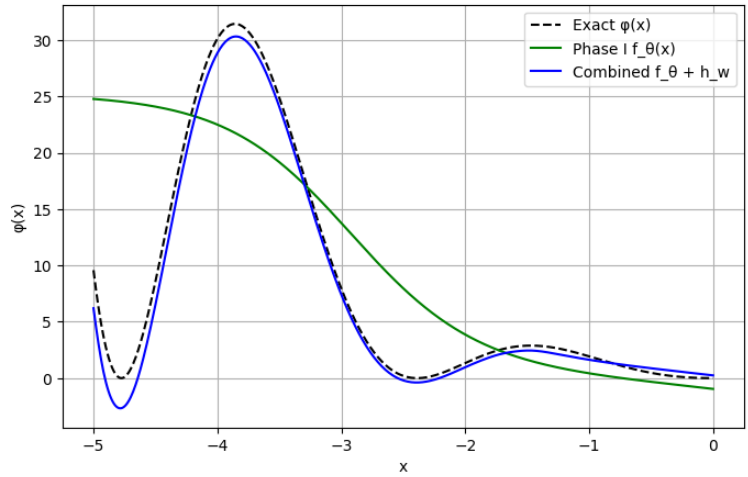


Figure 2.23: Graphs of the exact solution, Phase I and Phase II solutions, with $k = 0.38$.

Reconstruction The composite solution

$$\hat{\varphi}(x) = f_\theta(x) + h_w(x)$$

then satisfies the original ODE and boundary condition to high accuracy.

Training in Phase I with $2,000$ epochs and in Phase II with $3,000$ epochs, we obtain the results given in Fig. 2.23. We note that the first phase solution is a very rough approximation, while after the second phase, the composite solutions is a relatively good approximation of the true solution.

The reader can find the full Keras implementation of this example on Google Colab at:

`https://tinyurl.com/3ahv4aws`

2.9.1 Three-Phase Resolution Neural Network Method

Building on the previous two-phase residual-correction strategy, one can further decompose the approximation into three successive stages of resolution: coarse, medium, and fine, each handled by a neural network of increasing capacity.

Suppose we wish to solve the equation $P(\varphi) = g$ for a linear operator P. We introduce three networks, f_θ, m_η, h_w, with parameter vectors θ, η, w, of low, medium, and high capacity respectively. The composite approximation is given by their sum

$$\hat{\varphi}(x) = f_\theta(x) \; + \; m_\eta(x) \; + \; h_w(x).$$

Phase I: Coarse Approximation We train the network f_θ to minimize the loss

$$\mathcal{L}_1(\theta) = \big\| P(f_\theta) - g \big\|^2,$$

so that f_θ captures the dominant, low-frequency components of the solution. Then we define the first residual

$$r_1 = g - P(f_\theta).$$

Phase II: Medium-Capacity Correction Next, we introduce the intermediate correction

$$\psi_2 = \varphi - f_\theta,$$

which satisfies $P(\psi_2) = r_1$. We train the medium-capacity network m_η by minimizing

$$\mathcal{L}_2(\eta) = \big\|P(m_\eta) - r_1\big\|^2,$$

so that m_η refines the approximation by recovering the mid-scale features. We compute the second residual

$$r_2 = r_1 - P(m_\eta) = g - P\big(f_\theta + m_\eta\big).$$

Phase III: Fine-Scale Correction Finally, we set $\psi_3 = \varphi - f_\theta - m_\eta$, which satisfies $P(\psi_3) = r_2$. The high-capacity network h_w is trained to minimize

$$\mathcal{L}_3(w) = \big\|P(h_w) - r_2\big\|^2,$$

capturing the finest-scale details of the solution.

Reconstruction The sum

$$\hat{\varphi}(x) = f_\theta(x) + m_\eta(x) + h_w(x)$$

satisfies

$$P(\hat{\varphi}) = P(f_\theta) + P(m_\eta) + P(h_w) \approx g - r_1 + r_1 - r_2 + r_2 = g,$$

recovering the true solution in three successive refinement steps. This three-phase approach allows one to control model capacity at each scale, speed up convergence by first capturing large-scale structure, and then progressively correct medium and fine details.

The need for the three-phase method is emphasised in the following example.

Example 2.9.2 Consider the boundary-value ODE on $[0, 1]$:

$$\varphi'(x) + \varphi(x) \;=\; \underbrace{\sin(\pi x)}_{\text{low-freq}} \;+\; \underbrace{0.5\,\sin(20\pi x)}_{\text{mid-freq}} \;+\; \underbrace{0.1\,\sin(200\pi x)}_{\text{high-freq}}, \qquad \varphi(0) = 0.$$

Its exact solution is

$$\varphi(x) = e^{-x} \int_0^x e^s \Big[\sin(\pi s) + 0.5\,\sin(20\pi s) + 0.1\,\sin(200\pi s)\Big]\, ds,$$

which exhibits three clearly separated scales.

A single neural network (or even a two-phase residual correction) often captures only the dominant low-frequency component and misses one of the smaller-scale oscillations. In contrast, a three-phase strategy proceeds as follows:

1. Phase I (Coarse Network): f_θ learns the low-frequency term $\sin(\pi x)$.
2. Phase II (Medium Network): m_η is trained on the residual to capture the mid-frequency term $0.5\sin(20\pi x)$.
3. Phase III (Fine Network): h_w is then trained on the remaining residual to resolve the high-frequency term $0.1\sin(200\pi x)$.

The composite approximation $\hat{\varphi}(x) = f_\theta(x) + m_\eta(x) + h_w(x)$ thus recovers all three scales accurately while controlling model capacity and training stability at each phase.

Remark 2.9.3 The same residual-correction idea generalizes to an arbitrary number of phases, each targeting a distinct frequency band of the solution. Suppose the true solution admits a Fourier expansion

$$\varphi(x) = \sum_{m=0}^{M} a_m \cos(\omega_m x) + \sum_{m=1}^{N} b_m \sin(\nu_m x)\,.$$

One can then equip each phase $k = 0, 1, \ldots, K$ with its own network $u^{(k)}$ of appropriate capacity to learn the spectral content in a specified frequency window. Concretely:

Phase 0: Train $u^{(0)}$ to approximate the low-frequency (or mean) component.

Phase 1: Compute the first residual $r_1 = g - P(u^{(0)})$, then train $u^{(1)}$ on r_1 so that $P(u^{(1)})$ captures the next band of Fourier modes.

$\vdots$

Phase k: Compute $r_k = r_{k-1} - P\big(u^{(k-1)}\big)$ and train $u^{(k)}$ to fit r_k, thereby learning the modes in the k-th frequency interval $[\omega_k, \omega_{k+1}]$.

After K phases, the composite

$$\hat{\varphi} = \sum_{k=0}^{K} u^{(k)}$$

recovers the full Fourier-series solution, with each network specializing in a particular frequency band. This *divide-and-conquer* approach mimics classical multiresolution analysis and can dramatically improve convergence and stability when the solution spans a wide spectrum of frequencies.

2.9.2 Exercises

Exercise 2.9.4 Consider the initial-value problem on $[0, 1]$:

$$\varphi'(x) + \varphi(x) = e^x, \qquad \varphi(0) = 0.$$

(a) Show that its exact solution is

$$\varphi(x) = \tfrac{1}{2}\big(e^{x} - e^{-x}\big) = \sinh(x).$$

(b) Use the two-phase neural-network strategy to approximate φ.

(c) Plot on the same graph the exact solution, the coarse solution and the composite solution.

Exercise 2.9.5 (Two-phase NN solution) Consider the Fredholm integral equation of the second kind on $[0, 1]$:

$$\varphi(x) \; - \; \int_0^1 x\, t\, \varphi(t)\, dt \; = \; 1,$$

(a) Show that the unique continuous solution is

$$\varphi(x) \; = \; 1 \; + \; \frac{3}{4}\, x.$$

(b) Use the two-phase neural-network strategy to approximate φ following the next road map:

1. Phase I (Coarse Approximation):
   - Define a *low-capacity* network $f_\theta(x)$.
   - Train f_θ on collocation points $x_i \in [0, 1]$ by minimizing the loss

     $$\mathcal{L}_1(\theta) = \frac{1}{N} \sum_{i=1}^{N} \Big[f_\theta(x_i) - \int_0^1 x_i\, t\, f_\theta(t)\, dt - 1 \Big]^2,$$

     where the integral is approximated by a suitable quadrature.
   - Compute the residual

     $$r(x_i) = 1 - \Big[f_\theta(x_i) - \int_0^1 x_i\, t\, f_\theta(t)\, dt \Big].$$

2. Phase II (Residual Correction):
   - Set $\psi(x) = \varphi(x) - f_\theta(x)$. By linearity of the operator $P[\phi] = \phi(x) - \int_0^1 x\, t\, \phi(t)\, dt$, ψ satisfies

     $$\psi(x) \; - \; \int_0^1 x\, t\, \psi(t)\, dt = r(x).$$

   - Define a *higher-capacity* network $h_w(x)$.
   - Train h_w by minimizing

     $$\mathcal{L}_2(w) = \frac{1}{N} \sum_{i=1}^{N} \Big[h_w(x_i) - \int_0^1 x_i\, t\, h_w(t)\, dt - r(x_i) \Big]^2.$$

3. Reconstruction and Validation:

   - Form the composite approximation $\hat{\varphi}(x) = f_\theta(x) + h_w(x)$.
   - Plot $\hat{\varphi}(x)$ together with the exact solution $1 + \frac{3}{4}x$ on $[0, 1]$.
   - Compute the maximum absolute error $\max_i \big|\hat{\varphi}(x_i) - [1 + \frac{3}{4}x_i]\big|$.

Exercise 2.9.6 (*a*) Solve analytically the following ODE on $[0, 1]$:

$$\varphi'(x) + \varphi(x) = \cos(\pi x) + 0.5\cos(30\pi x) + 0.1\cos(100\pi x), \quad \varphi(0) = 0.$$

(*b*) Use three neural networks of increasing capacity, $\hat{\varphi} = f_\theta + m_\eta + h_w$, trained in succession by fitting

$$\begin{aligned} \text{Phase I:} \quad & f'_\theta + f_\theta \approx \cos(\pi x), \\ \text{Phase II:} \quad & m'_\eta + m_\eta \approx 0.5\cos(30\pi x) + \big[\cos(\pi x) - (f'_\theta + f_\theta)\big], \\ \text{Phase III:} \quad & h'_w + h_w \approx 0.1\cos(100\pi x) + \big[0.5\cos(30\pi x) - (m'_\eta + m_\eta)\big]. \end{aligned}$$

(*c*) Plot $\hat{\varphi}$ against the exact solution and report the maximum absolute error.

2.10 Dirichlet Eigenvalue Problem

We consider the boundary-value eigenproblem

$$f''(t) = -\lambda\, f(t), \quad f(0) = 0,\ f(\pi) = 0,$$

on $t \in [0, \pi]$. We can verify that the exact eigenpairs are

$$\lambda_n = n^2, \quad f_n(t) = \sin(nt), \quad n = 1, 2, 3, \ldots$$

We now describe how to train a neural network to approximate the first few eigenvalues λ and eigenfunctions $f(t)$, with the normalization $\int_0^\pi f(t)^2\, dt = 1$.

Finding the first eigenpair To approximate the first eigenpair $(\lambda_1, f_1(t))$ we use a single neural network

$$\mathrm{NN}(\theta; t) \;=\; \big(G(\theta; t),\, F(\theta; t)\big)$$

where:

- $G(\theta; t)$ is intended to be (nearly) constant in t and approximate the first eigenvalue λ_1.
- $F(\theta; t)$ approximates the first eigenfunction $f_1(t)$, subject to normalization condition $\displaystyle\int_0^\pi F(\theta; t)^2\, dt = 1$.

We shall start by defining the residual and boundary constraints as follows.

(*i*) *The ODE residual.* Let

$$R(\theta;t) \;=\; F''(\theta;t) \;+\; G(\theta;t)\,F(\theta;t),$$

which, ideally should vanish for all $t \in (0,\pi)$.

(*ii*) *The boundary conditions.* We enforce

$$F(\theta;0) = 0, \quad F(\theta;\pi) = 0.$$

Then we select N_r intermediate points $\{\,t_1,\, t_2, \ldots, t_{N_r}\,\} \subset (0,\pi)$. We will evaluate residuals and enforce "almost-constant" behavior of $G(\theta;t)$ at these points.

We define the individual loss terms as follows:

(*i*) *ODE loss:*

$$\mathcal{L}_{\mathrm{ODE}}(\theta) \;=\; \frac{1}{N_r}\sum_{i=1}^{N_r}\big[R(\theta;t_i)\big]^2.$$

(*ii*) *Boundary-condition loss:*

$$\mathcal{L}_{\mathrm{BC}}(\theta) \;=\; \big[F(\theta;0)\big]^2 \;+\; \big[F(\theta;\pi)\big]^2.$$

(*iii*) *Constant-λ loss:* Because $G(t;\theta)$ should approximate a constant eigenvalue, define the mean over collocation points:

$$\overline{G}(\theta) \;=\; \frac{1}{N_r}\sum_{i=1}^{N_r} G(\theta;t_i).$$

Then

$$L_{\mathrm{const}}(\theta) \;=\; \frac{1}{N_r}\sum_{i=1}^{N_r}\big[G(\theta;t_i) \;-\; \overline{G}(\theta)\big]^2.$$

(*iv*) *Normalization loss:* Enforce $\displaystyle\int_0^\pi F(\theta;t)^2\,dt = 1$. Approximate via a simple Riemann sum on collocation points:

$$L_{\mathrm{unit}}(\theta) \;=\; \Big(\sum_{i=1}^{N_r} F(\theta;t_i)^2\,\Delta t \;-\; 1\Big)^2, \quad \Delta t = \frac{\pi}{N_r}.$$

The *total loss* for the first eigenpair is obtained by combining the aforementioned losses with weights $\alpha, \beta, \gamma > 0$:

$$\mathcal{L}_{\mathrm{tot}}(\theta) \;=\; \mathcal{L}_{\mathrm{ODE}}(\theta) \;+\; \alpha\,\mathcal{L}_{\mathrm{BC}}(\theta) \;+\; \beta\,\mathcal{L}_{\mathrm{const}}(\theta) \;+\; \gamma\,\mathcal{L}_{\mathrm{unit}}(\theta).$$

We minimize $\mathcal{L}_{\mathrm{tot}}(\theta)$ (e.g., via gradient descent) to find optimal θ^*. Then:

$$G(\theta^*;t) \;\approx\; \lambda_1, \quad F(\theta^*;t) \;\approx\; f_1(t).$$

Finding the second eigenpair Once we have $\theta^* = \theta^{(1)*}$ for the first eigenpair, we want $F(\theta;t)$ to be orthogonal[3] to $f_1(t)$. Thus we add an orthogonality penalty given by the following *orthogonality loss:*

$$\mathcal{L}^{(2)}_{\text{orth}}(\theta) \;=\; \Big(\int_0^\pi F(\theta;t)\, f_1(t)\, dt\Big)^2.$$

This can be approximated by

$$\mathcal{L}^{(2)}_{\text{orth}}(\theta) \;=\; \Big(\sum_{i=1}^{N_r} F(\theta;t_i)\, F(\theta^{(1)};t_i)\, \Delta t\Big)^2.$$

The total loss for the second eigenpair becomes

$$\mathcal{L}^{(2)}_{\text{tot}}(\theta) = \mathcal{L}_{\text{ODE}}(\theta) + \alpha\, \mathcal{L}_{\text{BC}}(\theta) + \beta\, \mathcal{L}_{\text{const}}(\theta) + \gamma\, \mathcal{L}_{\text{unit}}(\theta) + \delta\, \mathcal{L}^{(2)}_{\text{orth}}(\theta),$$

with a new weight $\delta > 0$. Minimizing $\mathcal{L}^{(2)}_{\text{tot}}(\theta)$, we find $\theta^{(2)*}$. Then

$$G(\theta^{(2)*};t) \approx \lambda_2, \quad F(\theta^{(2)*};t) \approx f_2(t),$$

with $\displaystyle\int_0^\pi f_2(t)\, f_1(t)\, dt \approx 0$.

In order to obtain higher eigenpairs we need to introduce new orthogonal conditions and repeat a similar procedure.

Remark 2.10.1 (*i*) Sometimes, even if the first eigenpair works fine, the second one might not be as accurate. In that case, we readjust the hyperparameter δ by multiplying it by a significant factor to give the orthogonality condition more weight in the loss expression. This way, $\mathcal{L}_{\text{orth}}$ is no longer "too small" compared with the other losses.

(*ii*) If the first eigenfunction $f_1(t)$ is learned accurately but subsequent eigenfunctions are not, one can also use a Gram-Schmidt type procedure rather than relying solely on an orthogonality penalty. The idea is to force the new trial function to be exactly orthogonal to already-found eigenfunctions. More concretely, we suppose that $f_1(t)$ is known. We then seek the second eigenpair $(\lambda_2, f_2(t))$. Let the network output be $\widetilde{F}(\theta;t)$. We will build a function $F_\perp(\theta;t)$ that is orthogonal to $f_1(t)$ as follows. First, we compute the projection onto f_1. To this end, let

$$c_1(\theta) \;=\; \int_0^\pi \widetilde{F}(\theta;t)\, f_1(t)\, dt \;\approx\; \sum_{i=1}^{N_r} \widetilde{F}(\theta;t_i)\, f_1(t_i)\, \Delta t, \quad \Delta t \;=\; \frac{\pi}{N_r}.$$

Set

$$F_\perp(\theta;t) \;=\; \widetilde{F}(\theta;t) \;-\; c_1(\theta)\, f_1(t).$$

Then, by construction,

$$\int_0^\pi F_\perp(\theta;t)\, f_1(t)\, dt = 0.$$

[3] Any two eigenfunctions corresponding to distinct eigenvalues are orthogonal.

This relation enforces exact orthogonality to f_1.

We next enforce the PDE and boundary conditions on $F_\perp$ as follows:

- Define the residual

$$R_\perp(\theta;t) \;=\; F_\perp''(\theta;t) \;+\; G(\theta;t)\,F_\perp(\theta;t).$$

  We want $R_\perp(\theta;t) \approx 0$ for $t \in (0,\pi)$.

- The boundary conditions become $F_\perp(\theta;0) = 0, \quad F_\perp(\theta;\pi) = 0.$
- Normalization now reads $\int_0^\pi \big[F_\perp(\theta;t)\big]^2\,dt = 1$, again approximated by a discrete sum over collocation points.

Combining the losses

$$\mathcal{L}^\perp_{\mathrm{ODE}}(\theta) = \frac{1}{N_r}\sum_{i=1}^{N_r}\big[R_\perp(\theta;t_i)\big]^2,$$

$$\mathcal{L}^\perp_{\mathrm{BC}}(\theta) = \big[F_\perp(\theta;0)\big]^2 + \big[F_\perp(\theta;\pi)\big]^2,$$

$$\mathcal{L}^\perp_{\mathrm{unit}}(\theta) = \Big(\sum_{i=1}^{N_r}\big[F_\perp(\theta;t_i)\big]^2\,\Delta t \;-\; 1\Big)^2,$$

$$\mathcal{L}_{\mathrm{const}}(\theta) = \frac{1}{N_r}\sum_{i=1}^{N_r}\big[G(\theta;t_i) - \overline{G}(\theta)\big]^2,$$

we obtain the following total loss:

$$\mathcal{L}^{(2)}_{\mathrm{tot}}(\theta) \;=\; \mathcal{L}^\perp_{\mathrm{ODE}}(\theta) \;+\; \alpha\,\mathcal{L}^\perp_{\mathrm{BC}}(\theta) \;+\; \beta\,\mathcal{L}_{\mathrm{const}}(\theta) \;+\; \gamma\,\mathcal{L}^\perp_{\mathrm{unit}}(\theta).$$

Minimizing $\mathcal{L}^{(2)}_{\mathrm{tot}}(\theta)$ we find $\theta^{(2)*}$. Then $\lambda_2 \approx G(\theta^{(2)*};t)$ and $f_2(t) \approx F_\perp(\theta^{(2)*};t)$.

Higher eigenpairs can be treated similarly. For instance, given f_1 and f_2, we can obtain f_3 by constructing

$$F_\perp(t;\theta) \;=\; \widetilde{F}(t;\theta) \;-\; \big\langle \widetilde{F},\, f_1\big\rangle f_1(t) \;-\; \big\langle \widetilde{F},\, f_2\big\rangle f_2(t),$$

and then minimize the analogous loss.

2.10.1 Practical Implementation

We shall provide next a detailed explanation of the neural-network implementation for finding the first eigenpair $(\lambda_1, f_1(t))$ of

$$f''(t) \;=\; -\,\lambda\,f(t), \quad f(0) = f(\pi) = 0,$$

with normalization $\int_0^\pi [f_1(t)]^2\,dt = 1$. The complete code details of this section can be found at the following Google Collab page:

`https://tinyurl.com/mrcxpz6v`

Finding the first eigenpair $(\lambda_1, f_1(t))$ After importing the necessary libraries (numpy, tensorflow, keras), we set the domain and training parameters as follows:

```
# Domain boundaries
T_MIN, T_MAX = 0.0, np.pi

# Training parameters
N_RESIDUAL = 100   # number of interior points for ODE residual
N_BC = 2           # boundary points (0 and pi)
N_NORM = 100       # number of points to approximate norm. int.
N_EPOCHS = 2000
LEARNING_RATE = 1e-3
```

In the previous snippet,

- `T_MIN` and `T_MAX` define the interval $[0, \pi]$.
- `N_RESIDUAL = 100` indicates that we sample 100 interior points to enforce the ODE.
- `N_BC = 2` corresponds to the two boundary conditions $f(0) = 0,\ f(\pi) = 0$.
- `N_NORM = 100` points will be used to approximate the normalization condition $\int_0^\pi f(t)^2\, dt = 1$. This condition is needed to set a length for the eigenfunction, thus determining it up to a sign.
- `N_EPOCHS` and `LEARNING_RATE` control the training duration and optimizer step size.

We then generate arrays of points in the interval $(0, \pi)$, at the boundaries $\{0, \pi\}$, and for the normalization constraint as follows:

```
# Generate points
t_res = np.linspace(T_MIN, T_MAX, N_RESIDUAL + 2)[1:-1].reshape(-1, 1)
t_bc = np.array([[T_MIN], [T_MAX]])  # boundary points
t_norm = np.linspace(T_MIN, T_MAX, N_NORM).reshape(-1, 1)

# Convert to TensorFlow constants
t_res_tf = tf.constant(t_res, dtype=tf.float32)
t_bc_tf = tf.constant(t_bc, dtype=tf.float32)
t_norm_tf = tf.constant(t_norm, dtype=tf.float32)
```

Here, `t_res` is a column vector of shape $(100, 1)$ containing interior points in $(0, \pi)$. `t_bc` is the vector $\left[\begin{smallmatrix} 0 \\ \pi \end{smallmatrix}\right]$. `t_norm` is an array holding 100 points for approximating the integral $\int_0^\pi f^2\, dt$. We convert each array into a TensorFlow constant so that they can be used in automatic differentiation.

Next we build the neural network. To this end, we define a Keras model that takes input t and outputs two values: an approximation to $\lambda(t)$ (which should be constant) and a prospective solution $f(t)$:

```
def build_eigen_net(hidden_units=50, n_hidden_layers=3):
    t_input = keras.Input(shape=(1,), name="t")
    x = t_input
    for _ in range(n_hidden_layers):
        x = layers.Dense(hidden_units, activation="tanh")(x)
    f_out = layers.Dense(1, activation=None, name="f_out")(x)
    lambda_out=layers.Dense(1,activation=None,name="lambda_out")(x)
    return keras.Model(inputs=t_input, outputs=[lambda_out, f_out])
```

The architecture of the network consists of:

- An input layer taking a single real number t.
- Three hidden layers (by default) of 50 units each, with `tanh` activation.
- A final pair of `Dense` layers producing:
  1. $\text{lambda}_{\text{out}}(\text{t})$, intended to approximate λ_1.
  2. $\text{f}_{\text{out}}(\text{t})$, intended to approximate $f_1(t)$.

The derivatives of the network output can be done using automatic differentiation. To evaluate $f''(t)$, we use two nested `tf.GradientTape` contexts. The inner tape computes $\frac{d}{dt}f(t)$, and the outer tape computes the derivative of that result. The snippet is as follows:

```
def compute_second_derivative(f_model, t):
    with tf.GradientTape() as tape2:
        tape2.watch(t)
        with tf.GradientTape() as tape1:
            tape1.watch(t)
            _, f_val = f_model(t)
        f_t = tape1.gradient(f_val, t)
    f_tt = tape2.gradient(f_t, t)
    return f_tt
```

Here, `tape1` records operations to compute $f(t)$ and then $\frac{df}{dt}$. Next, `tape2` differentiates $\frac{df}{dt}$ to get $\frac{d^2f}{dt^2}$.

The next ingredient is to define the loss function. This will be obtained by combining the following four terms defined as:

```
def first_eigen_loss(model, t_res, t_bc, t_norm,
                     alpha_bc=10.0,alpha_const=1.0,alpha_norm=10.0):
    # 1) Predict lambda and f at residual points
    lambda_res, f_res = model(t_res) # shape (N_RESIDUAL, 1)
    f_tt_res = compute_second_derivative(model, t_res)
    # shape (N_RESIDUAL, 1)
```

```
# 2) ODE residual: f'' + lambda * f  0
ode_residual = f_tt_res + lambda_res * f_res
loss_ode = tf.reduce_mean(tf.square(ode_residual))

# 3) Boundary conditions: f(0)=0, f(pi)=0
_, f_bc = model(t_bc)  # shape (N_BC, 1)
loss_bc = tf.reduce_mean(tf.square(f_bc))

# 4) lambda(t)  constant: minimize variance
lambda_mean = tf.reduce_mean(lambda_res)
loss_const = tf.reduce_mean(tf.square(lambda_res - lambda_mean))

# 5) Normalization: 0^ f(t)^2 dt = 1
_, f_norm = model(t_norm)  # shape (N_NORM, 1)
sq_f = tf.square(f_norm)
integral_approx = (np.pi / N_NORM) * tf.reduce_sum(sq_f)
loss_norm = tf.square(integral_approx - 1.0)

# Total loss
loss = (loss_ode
        + alpha_bc * loss_bc
        + alpha_const * loss_const
        + alpha_norm * loss_norm)
return loss, lambda_mean
```

To finalize the definition of the total loss we have to define a few more items:

- We set the ODE residual

$$R(t) \;=\; f''(t) + \lambda(t)\, f(t).$$

  On the `N_RESIDUAL` interior points `t_res`, we compute f'' via the function `compute_second_derivative` and form $\mathcal{L}_{\text{ODE}} = \frac{1}{N_r} \sum_{i=1}^{N_r} R^2(t_i)$.

- We enforce the following boundary conditions:

$$f(0) = 0, \quad f(\pi) = 0.$$

  We evaluate f_{out} at $\{0, \pi\}$ using model(t_{bc}), then we set the boundary condition loss $\mathcal{L}_{\text{BC}} = f(0)^2 + f(\pi)^2$.

- We try to make $\lambda(t)$ constant by minimizing its variance. To this end, we compute $\lambda_{\text{res}} = \lambda(t_i)$ for each interior point and set $\overline{\lambda} = \frac{1}{N_r} \sum \lambda_{\text{res}}$. Then let $\mathcal{L}_{\text{const}} = \frac{1}{N_r} \sum (\lambda_{\text{res}} - \overline{\lambda})^2$.

- We enforce an L^2-normalization condition. For this, we sample `N_NORM` points $\{t_i\}$ in $[0, \pi]$ to approximate $\int_0^\pi f(t)^2\, dt \approx \frac{\pi}{N_{\text{NORM}}} \sum f(t_i)^2$. We ask $\int_0^\pi f^2 = 1$ by taking $\mathcal{L}_{\text{norm}} = \left(\frac{\pi}{N_{\text{NORM}}} \sum f^2(t_i) - 1\right)^2$.
- Finally, we wet the total loss as the following weighted sum:

$$\mathcal{L} = \mathcal{L}_{\text{ODE}} + \alpha_{\text{BC}}\, \mathcal{L}_{\text{BC}} + \alpha_{\text{const}}\, \mathcal{L}_{\text{const}} + \alpha_{\text{norm}}\, \mathcal{L}_{\text{norm}}.$$

We then instantiate the model and optimizer, and run gradient descent for `N_EPOCHS` epochs (which was apriori defined to be $2,000$):

```
# Training the first eigenpair
model = build_eigen_net()
optimizer = keras.optimizers.Adam(learning_rate=LEARNING_RATE)

for epoch in range(N_EPOCHS):
    with tf.GradientTape() as tape:
        loss_value, lambda_mean = first_eigen_loss(
            model, t_res_tf, t_bc_tf, t_norm_tf
        )
    grads = tape.gradient(loss_value, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))

    if epoch % 300 == 0:
        print(f"Epoch {epoch}, Loss = {loss_value:.6f},
        lambda = {lambda_mean.numpy():.6f}")
```

Inside each epoch, we open a `tf.GradientTape` to record operations, compute the loss and approximate λ, then compute gradients and apply them via the Adam optimizer. Every 300 epochs, we print out the current loss and estimated λ.

After training, we read out the eigenvalue as a scalar and define a callable to evaluate $f_1(t)$ via `model.predict`:

```
# Extract eigenvalue and eigenfunction
lambda_1 = lambda_mean.numpy()  # approximate constant eigenvalue

# Eigenfunction as a Python callable
def eigenfunction(t):
    t_arr = np.array(t).reshape(-1, 1)
    _, f_arr = model.predict(t_arr)
    return f_arr

# Display results
print(f"\nFirst eigenvalue:  {lambda_1:.6f}")
```

```
import matplotlib.pyplot as plt

t_plot = np.linspace(0, np.pi, 200).reshape(-1, 1)
f_plot = eigenfunction(t_plot)
# Normalize for plotting
f_plot /= np.max(np.abs(f_plot))

plt.figure(figsize=(6, 4))
plt.plot(t_plot, f_plot, label=f"f(t), {lambda_1:.3f}")
plt.xlabel("t")
plt.ylabel("f(t)")
plt.title("First eigenfunction (normalized)")
plt.legend()
plt.grid(True)
plt.show()
```

We print the approximate λ_1 versus the number of training epochs, obtaining the following output:

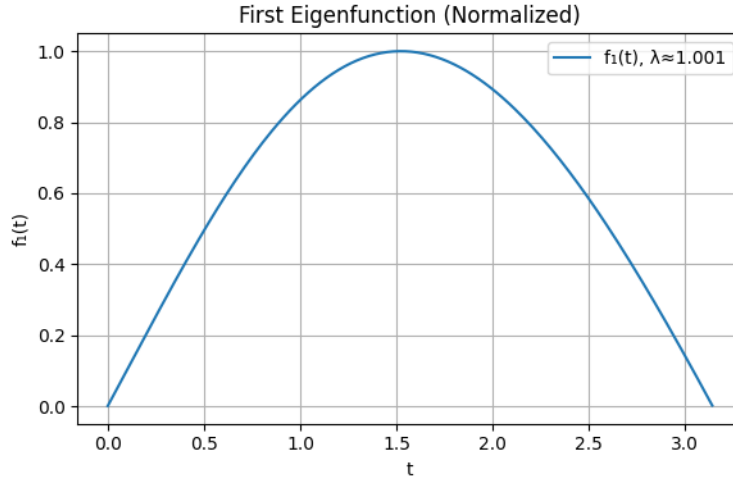
```
Epoch 0, Loss = 9.552553, lambda = 0.541182
Epoch 300, Loss = 0.063422, lambda = 0.916908
Epoch 600, Loss = 0.016858, lambda = 0.979487
Epoch 900, Loss = 0.008007, lambda = 0.994662
Epoch 1200, Loss = 0.005802, lambda = 0.997671
Epoch 1500, Loss = 0.004263, lambda = 1.000065
Epoch 1800, Loss = 0.003610, lambda = 1.000792
```

To visualize $f_1(t)$, we sample 200 points in $[0, \pi]$, evaluate the network, normalize its amplitude to unity, and plot using Matplotlib. The plot of the eigenfunction $f_1(t)$ and the associated loss versus training epochs is given in Fig. 2.24. The results recover accurately the theoretical eigenpair $(\lambda_1, f_1(t)) = (1, \frac{2}{\pi}\sin(t))$.

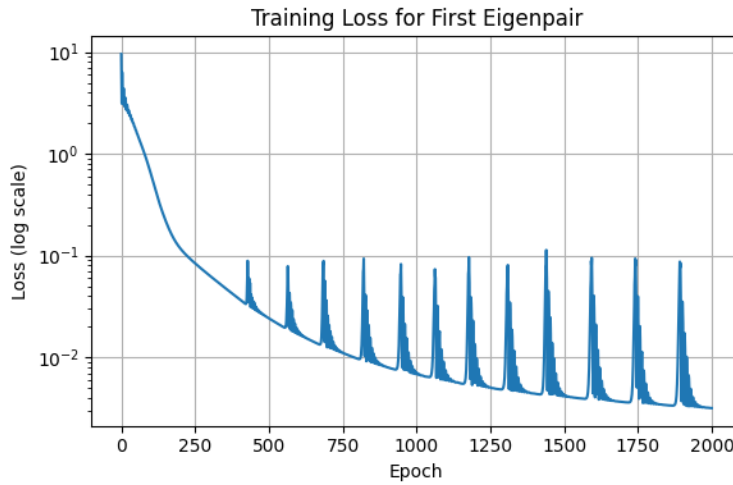
The periodic spikes in Fig. 2.24(b) occur because the total loss combines several competing objectives: minimizing the PDE residual $f'' + \lambda f$, enforcing the boundary conditions $f(0) = f(\pi) = 0$, keeping $\lambda(t)$ nearly constant, and satisfying the normalization $\int_0^\pi f(t)^2\, dt = 1$. When gradient updates reduce the PDE residual, the normalization and constant-λ terms temporarily worsen, so subsequent updates must correct them–causing brief increases in loss. Once those constraints are satisfied, the network again focuses on the PDE residual, and the cycle repeats. As a result, the loss decays overall but exhibits small oscillations each time one constraint pushes back against the others.

Finding the second eigenpair $(\lambda_2, f_2(t))$ We add to the previous loss an extra-term, consisting in an orthogonal loss, which ensures the orthogonality of the network output on the first eigenfunction, $f_1(t)$:

```
loss_orth = 0.0
if prev_funcs:
```



(a)



(b)

Figure 2.24: (a) The first eigenfunction $f_1(t)$ generated by the neural network. (b) The associated loss function is decreasing with sparse oscillations.

```
        for prev in prev_funcs:
            f_prev = prev(t_res_tf)  # shape (N_RESIDUAL, 1)
            inner_prod = tf.reduce_mean(f_res * f_prev) * np.pi
            loss_orth += tf.square(inner_prod)

    loss = (loss_ode
            + alpha_bc * loss_bc
            + alpha_const * loss_const
            + alpha_norm * loss_norm
            + alpha_orth * loss_orth)
    return loss, lambda_mean
```

The coefficient of the orthogonal loss was chosen (using trial and error) equal to 300, which is higher that the other weights in the loss:

```
def eigen_loss(model, t_res, t_bc, t_norm, prev_funcs=None,
               alpha_bc=10.0, alpha_const=1.0, alpha_norm=10.0,
               alpha_orth=300.0)
```

The plot of the first two eigenfunctions obtained by the neural network are given in Fig 2.25(a), which are very good approximations of the theoretical eigenfunctions of the Dirichlet problem. The eigenvalues obtained by the network are $\lambda_1 \approx 1.001391$ and $\lambda_2 \approx 3.910531$, which are approximations of the theoretical values $\lambda_1 = 1$ and $\lambda_2 = 4$. We notice the loss associated with the second eigenvalue does not converge to zero as fast as the loss associated with the first eigenvalue. This explains why the first eigenpair is a better approximation than the second one.

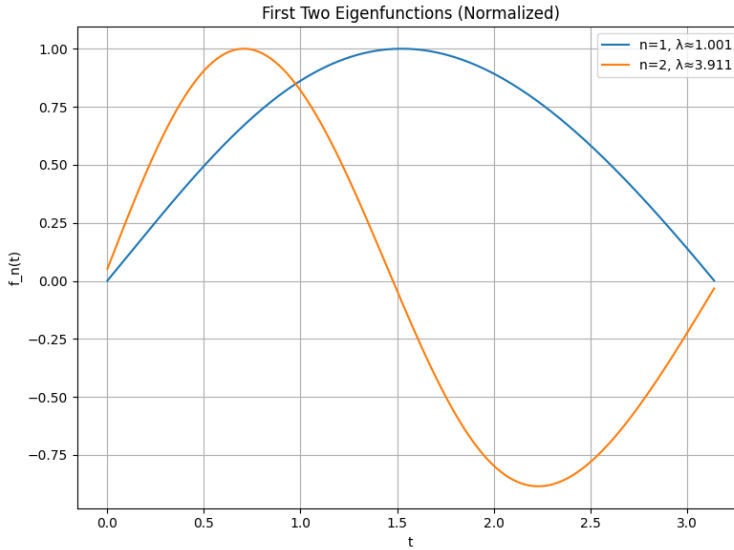
Remark 2.10.2 We note a peculiarity in the behavior of the total loss in Fig. 2.24(b) and Fig. 2.25(b), where quasiperiodic oscillations appear. This phenomenon arises whenever the total loss is defined as a combination of several component losses added as soft constraints,

$$\mathcal{L}_{\text{total}} = \alpha_1\mathcal{L}_1 + \alpha_2\mathcal{L}_2 + \cdots + \alpha_k\mathcal{L}_k.$$

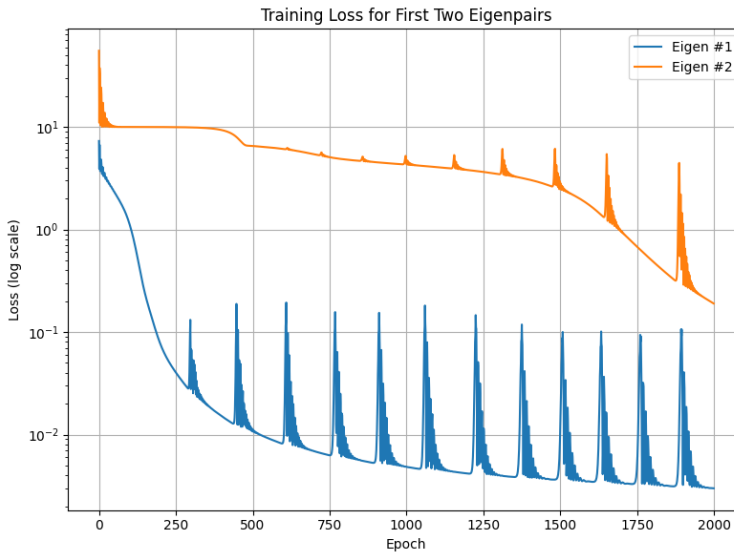
During backpropagation, each gradient step attempts to decrease all losses simultaneously, but each loss decreases at a different rate depending on its local geometry. As the optimizer progresses, one component may be reduced more rapidly than the others, after which another loss temporarily dominates the descent. This interplay produces oscillations in the total loss: progress in one direction partially undoes progress in another.

An intuitive analogy is that of a balance scale with multiple plates: at each step, a fixed-size cup of sand is transferred between plates. If the cup is too large (the learning rate too high), the scale overshoots equilibrium and oscillates around it. Only when the cup is sufficiently small does the system settle smoothly. Thus, the oscillations in the total loss can be effectively eliminated by reducing the learning rate, sometimes by half or even by an order of magnitude.

The conclusion is that the periodic oscillations in the total loss are caused by the competition between multiple gradient components pulling the parameters in different



(a)



(b)

Figure 2.25: (a) The first two eigenfunctions generated by the neural network. (b) The associated loss functions for each eigenfunction are decreasing with sparse oscillations.

directions. With a learning rate that is too high, the optimizer overshoots equilibrium in a cyclic way. To address the issue we can reduce the learning rate, or rebalance the loss terms, or use adaptive optimizers.

2.10.2 Exercises

Exercise 2.10.3 Consider the Hermite operator

$$P\,f(x) \;=\; \frac{d^2 f}{dx^2}(x) \;-\; x^2\,f(x)$$

on the real line. The first eigenvalue and eigenfunction satisfy

$$P\,f_1(x) \;=\; -\,\lambda_1\,f_1(x), \qquad f_1(x) \to 0 \text{ as } |x| \to \infty, \qquad \int_{-\infty}^{\infty} f_1(x)^2\,dx = 1.$$

Analytically, $\lambda_1 = 1$ and $f_1(x) \propto e^{-x^2/2}$. Your task is to use a neural network to approximate λ_1 and $f_1(x)$ on a truncated domain $[-L, L]$ (for some large $L > 0$). Specifically:

(a) Provide a description of your network architecture and loss function.

(b) Plot the learned eigenfunction and compare it with $e^{-x^2/2}$.

(c) Print the approximate eigenvalue λ_1 with numerical error.

Exercise 2.10.4 Consider the generalized Laguerre operator

$$\mathcal{L}_\alpha f(x) \;=\; x\,f''(x) \;+\; (\alpha + 1 - x)\,f'(x), \qquad x \in [0, \infty), \quad \alpha > -1.$$

The Laguerre functions $L_n^{(\alpha)}(x)$ satisfy

$$\mathcal{L}_\alpha\big[L_n^{(\alpha)}\big](x) \;=\; -\,n\,L_n^{(\alpha)}(x), \quad \int_0^\infty x^\alpha e^{-x}\left[L_n^{(\alpha)}(x)\right]^2 dx \;=\; \frac{\Gamma(n+\alpha+1)}{n!},$$

so that each eigenvalue $\lambda_n = n$ and eigenfunction $L_n^{(\alpha)}(x)$. Your task is to use a neural network to approximate $\lambda_1 = 1$ and the first Laguerre eigenfunction $L_1^{(\alpha)}(x)$ on a truncated domain $[0, R]$ for some large $R > 0$. You may choose $\alpha = 0$. More precisely,

(a) Describe your network architecture and loss function.

(b) Plot the learned eigenfunction and compare it with the exact Laguerre function.

(c) Provide the approximate eigenvalue λ_1 with numerical error.

Exercise 2.10.5 The eigenvalue problem for the Laplace operator on the unit circle S^1 can be written in angular coordinates $\varphi \in [0, 2\pi)$ as

$$\frac{1}{2}\,\frac{d^2 u(\varphi)}{d\varphi^2} \;=\; -\,\lambda\,u(\varphi), \quad u(\varphi + 2\pi) = u(\varphi).$$

Analytically, the eigenvalues and eigenfunctions are

$$\lambda_n = \frac{n^2}{2}, \qquad u_n(\varphi) = c_n\,e^{i\,n\,\varphi}, \quad c_n = \frac{1}{\sqrt{2\pi}}, \quad n = 0,\,1,\,2,\,\ldots$$

so that $\displaystyle\int_0^{2\pi} |u_n(\varphi)|^2\,d\varphi = 1.$

Your task is to use a neural network to approximate the first two nonzero eigenpairs $\big(\lambda_1, u_1(\varphi)\big)$ and $\big(\lambda_2, u_2(\varphi)\big)$ on $[0, 2\pi]$ by using a neural network

$$\mathrm{NN}(\theta\,;\varphi) \;=\; \Big(\Lambda(\theta\,;\varphi),\, F(\theta\,;\varphi)\Big),$$

where $\Lambda(\theta;\varphi)$ approximates the eigenvalue λ as a constant function of φ, and $F(\theta\,;\varphi)$ approximates the (possibly complex-valued) eigenfunction $u(\varphi)$, by enforcing periodic boundary conditions given by $F(0) = F(2\pi)$ and $F'(0) = F'(2\pi)$.

Exercise 2.10.6 Consider the radial Schrödinger equation for a hydrogen-like atom (atomic number $Z > 0$) with angular momentum quantum number ℓ:

$$-\frac{1}{2}\Big(\frac{d^2 f(r)}{dr^2} - \frac{\ell(\ell+1)}{r^2}\, f(r)\Big) - \frac{Z}{r}\, f(r) \;=\; E\, f(r), \quad r \in (0, \infty),$$

subject to the boundary conditions

$$f(0) = 0, \qquad f(r) \to 0 \text{ as } r \to \infty,$$

and the normalization

$$\int_0^\infty \big[f(r)\big]^2\, dr \;=\; 1.$$

Analytically, the bound-state energies are

$$E_n \;=\; -\frac{Z^2}{2n^2}, \quad n = \ell + 1,\ \ell + 2,\ \ell + 3,\ \ldots$$

and the corresponding radial functions $f_n(r)$ are proportional to

$$r^{\ell+1}\, e^{-Zr/n}\, L_{n-\ell-1}^{(2\ell+1)}\Big(\tfrac{2Zr}{n}\Big),$$

where $L_k^{(\alpha)}$ are associated Laguerre polynomials.

Your task is to use a neural network to approximate the first three energy levels E_1, E_2, E_3 and their associated normalized radial functions $f_1(r), f_2(r), f_3(r)$ in the case $Z = 1$ and $\ell = 0$. Proceed by following the next steps:

(*a*) *Domain Truncation and Collocation Points.* Since $r \in [0, \infty)$, truncate to $r \in [0, R]$ for a suitably large $R > 0$ (e.g., $R = 20/Z$), so that $f(r) \approx 0$ for $r \geq R$. Choose:

- N_r equally spaced residual points $\{r_i\}_{i=1}^{N_r} \subset (0, R)$.
- Boundary points $r = 0$ and $r = R$.

(*b*) *Neural Network Architecture.* For each eigenpair $(E_k, f_k(r))$, build a network

$$\mathrm{NN}(\theta; r) \;=\; \big(\mathcal{E}(\theta; r),\, F(\theta; r)\big),$$

where $\mathcal{E}(\theta; r)$ approximates the energy level E_k (enforced constant in r) and $F(\theta; r)$ approximates the radial function $f_k(r)$.

(*c*) *Automatic Differentiation for $F''(r)$.* Use nested `tf.GradientTape` (or an equivalent in your framework) to compute $\frac{d^2}{dr^2} F(\theta; r)$.

(*d*) *Loss Function.* At each collocation point r_i:

1. Compute

$$R(\theta; r_i) \;=\; -\tfrac{1}{2}\Big(F''(\theta; r_i) - \tfrac{\ell(\ell+1)}{r_i^2}\,F(\theta; r_i)\Big) - \tfrac{Z}{r_i}\,F(\theta; r_i) - \mathcal{E}(\theta; r_i)\,F(\theta; r_i),$$

   so that $R = 0$ enforces $-\frac{1}{2}(\,F'' - \frac{\ell(\ell+1)}{r^2}F) \;-\; \frac{Z}{r}F = E\,F$.

2. The PDE residual loss:

$$\mathcal{L}_{\mathrm{ODE}}(\theta) \;=\; \frac{1}{N_r}\sum_{i=1}^{N_r}\big[R(\theta; r_i)\big]^2.$$

3. Boundary-condition loss:

$$\mathcal{L}_{\mathrm{BC}}(\theta) \;=\; \big[F(\theta; 0)\big]^2 \;+\; \big[F(\theta; R)\big]^2.$$

4. Constant-energy loss:

$$\overline{\mathcal{E}}(\theta) \;=\; \frac{1}{N_r}\sum_{i=1}^{N_r}\mathcal{E}(\theta; r_i), \quad \mathcal{L}_{\mathrm{const}}(\theta) \;=\; \frac{1}{N_r}\sum_{i=1}^{N_r}\big[\mathcal{E}(\theta; r_i) - \overline{\mathcal{E}}(\theta)\big]^2.$$

5. Normalization loss (approximate $\int_0^R F(r)^2 dr = 1$):

$$\mathcal{L}_{\mathrm{norm}}(\theta) \;=\; \Big(\sum_{i=1}^{N_r} F(\theta; r_i)^2\,\Delta r \;-\; 1\Big)^2, \quad \Delta r = \frac{R}{N_r}.$$

6. Total loss for the k-th eigenpair:

$$\mathcal{L}_{\mathrm{tot}}^{(k)}(\theta) = \mathcal{L}_{\mathrm{ODE}}(\theta) + \alpha\,\mathcal{L}_{\mathrm{BC}}(\theta) + \beta\,\mathcal{L}_{\mathrm{const}}(\theta) + \gamma\,\mathcal{L}_{\mathrm{norm}}(\theta),$$

   where $\alpha, \beta, \gamma > 0$ are chosen weights.

(*e*) *Training Sequence.* Follow these steps:

1. Initialize the network parameters θ. Minimize $\mathcal{L}_{\mathrm{tot}}^{(1)}(\theta)$ to find the first eigenpair, producing $\theta^{(1)\,*}$. Then

$$E_1 \approx \overline{\mathcal{E}}\big(\theta^{(1)\,*}\big), \quad f_1(r) \approx F\big(\theta^{(1)\,*}; r\big).$$

2. To find the second eigenpair, impose orthogonality to $f_1(r)$ via a deflation step:

$$\widetilde{F}(\theta; r) \;\longrightarrow\; F_\perp(\theta; r) \;=\; \widetilde{F}(\theta; r) \;-\; \Big(\int_0^R \widetilde{F}(\theta; s)\,f_1(s)\,ds\Big)\,f_1(r).$$

   Use $F_\perp$ in place of F when forming $R(r; \theta)$, boundary-condition terms, and normalization for $\mathcal{L}_{\mathrm{tot}}^{(2)}(\theta)$. Minimize to find $\theta^{(2)\,*}$. Then

$$E_2 \approx \overline{\mathcal{E}}\big(\theta^{(2)\,*}\big), \quad f_2(r) \approx F_\perp\big(\theta^{(2)\,*}; r\big).$$

3. Similarly, to find the third eigenpair, enforce orthogonality to both $f_1(r)$ and $f_2(r)$ via

$$F_{\perp\perp}(r;\theta) \;=\; \widetilde{F}(r;\theta) - \Big(\int_0^R \widetilde{F}\, f_1\Big) f_1(r) - \Big(\int_0^R \widetilde{F}\, f_2\Big) f_2(r),$$

and minimize $L^{(3)}_{\text{tot}}$ using $F_{\perp\perp}$. Then

$$E_3 \approx \overline{\mathcal{E}}\big(\theta^{(3)\,*}\big), \quad f_3(r) \approx F_{\perp\perp}\big(\theta^{(3)\,*}; r\big).$$

(*f*) *Plotting Results.* On a single graph, plot the approximate eigenfunctions $\{f_1(r), f_2(r), f_3(r)\}$ versus $r \in [0, R]$. Label each curve by its corresponding energy E_k.

(*g*) *Comparison with Analytical Values.* Compare your computed energies E_1, E_2, E_3 with the exact values $\big(-Z^2/2, -Z^2/8, -Z^2/18\big)$ (for $\ell = 0$). Verify that the relative errors are small.

2.11 The Method of Characteristics

In this section, we present an application of finding the integral curves of a given vector field, using a PDE technique known as the *method of characteristics*. Because applying this method can be challenging—both in solving the associated equations and in inverting them—we propose using neural networks as an effective alternative.

Our goal is to solve the following initial value problem for a first-order PDE:

$$a(x,y)\,\partial_x u(x,y) + b(x,y)\,\partial_y u(x,y) \;=\; F\big(u(x,y)\big), \tag{2.11.21}$$

$$u(x,0) \;=\; \varphi(x), \tag{2.11.22}$$

where a, b, and u are smooth functions on $[a,b]\times[0,T]$, and φ is a smooth function on $[a,b]$. The function F is considered smooth and verifying Lipschitz condition.

In Physics, the variable x typically denotes position, while y denotes time. Hence, the initial condition specifies the value of u at the initial time $y = 0$. The PDE above can therefore be interpreted as a first-order evolution equation.

We will present two approaches: a straightforward "brute-force" method, and a more elegant one based on the method of characteristics; both methods involve neural networks.

The "brute-force" method This method bypasses the method of characteristics. We start by dividing the interval $[a,b]$ into K subintervals of equal length and $[0,T]$ into N equal time steps. The resulting collocation points are $(x_k, t_j) \in [a,b]\times[0,T]$. We approximate the solution $u(x,y)$ by a neural network $u_\theta(x,y) \approx u(x,y)$.

The initial condition loss and the PDE loss are defined respectively as

$$\mathcal{L}_{\text{IC}}(\theta) \;=\; \frac{1}{K}\sum_{k=1}^{K}\big(u_\theta(x_k,0) - \varphi(x_k)\big)^2,$$

$$\mathcal{L}_{\text{PDE}}(\theta) \;=\; \frac{1}{KN}\sum_{k=1}^{K}\sum_{j=1}^{N} R^2(x_k,t_j),$$

where the residual is

$$R(x_k, t_j) = a(x_k, t_j)\,\partial_x u_\theta(x_k, t_j) + b(x_k, t_j)\,\partial_y u_\theta(x_k, t_j) - F\big(u_\theta(x_k, t_j)\big).$$

The total loss is then

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{PDE}}(\theta) + \lambda\,\mathcal{L}_{\text{IC}}(\theta),$$

with $\lambda > 0$ a weighting parameter.

We note that this approach does not exploit the geometric interpretation of the problem, whereas the next method does.

The method of characteristics The operator

$$X = a(x, y)\,\partial_x + b(x, y)\,\partial_y$$

is a vector field on the domain $[a, b] \times [0, T]$. With this notation, equation (2.11.21) can be written as $X(u) = F(u)$.
If $c(t)$ denotes an integral curve of X, the left-hand side becomes a derivative with respect to the parameter t:

$$X(u)\big|_{c(t)} = \langle X, \nabla u\rangle\big|_{c(t)} = \frac{d}{dt}u(c(t)).$$

Therefore, along the integral curve $c(t)$, equation (2.11.21) takes the form

$$\frac{d}{dt}u(c(t)) = F\big(u(c(t))\big).$$

Letting $f(t) = u(c(t))$, this reduces to the first-order ODE

$$\frac{d}{dt}f(t) = F(f(t)),$$

which, in principle, can be solved by the method of separation of variables.

While this provides the values of u along a single integral curve $c(t)$, it does not yet incorporate the initial condition (2.11.22), nor does it provide values of u away from the trajectory $c(t)$.
To address this, we introduce a family of integral curves $c_s(t) = c(t, s)$, which fill the entire domain (or a subdomain of) $[a, b] \times [0, T]$ as the parameters s and t vary. When $t = 0$, we obtain the *initial curve*

$$c_s(0) = c(0, s) = (s, 0).$$

Under this assumption, the initial condition (2.11.22) specifies the values of u along this initial curve:

$$u(c_s(0)) = u(s, 0) = \varphi(s).$$

The mapping $t \mapsto c_s(t)$ describes the evolution of the initial point $c_s(0)$ along the characteristic curve. The idea is that the initial curve $c_s(0)$ should sweep through the domain as t varies. This property holds provided that $b \neq 0$.

The function $f(t) = u(c(t))$ now becomes

$$f_s(t) = u(c_s(t)),$$

which satisfies the ODE

$$\frac{d}{dt} f_s(t) = F(f_s(t)), \quad f_s(0) = u(s,0) = \varphi(s).$$

If F is sufficiently regular (e.g., satisfies a Lipschitz condition), there exists a unique solution $f_s(t)$ for t small enough.

By inverting the characteristic map $(s,t) \mapsto c_s(t) = (x(s,t),\, y(s,t))$, we recover the parameters as $s = s(x,y)$ and $t = t(x,y)$. Substituting these into the solution along characteristics yields

$$u(x,y) \;=\; f_{s(x,y)}\big(t(x,y)\big).$$

We now carry out these steps one at a time as follows:

1. Finding the family of integral curves. The integral curves $c_s(t) = \big(c_s^1(t),\, c_s^2(t)\big)$ satisfy the system

$$\begin{aligned} \frac{d}{dt} c_s^1(t) &= a\big(c_s(t)\big), \\ \frac{d}{dt} c_s^2(t) &= b\big(c_s(t)\big), \\ c_s^1(0) &= s, \quad c_s^2(0) = 0. \end{aligned}$$

We approximate these curves using a two-dimensional output neural network

$$\gamma_\theta(s,t) \;=\; (s,0) + t\,\mathrm{nn}(\theta; s, t),$$

trained by minimizing the loss function

$$\mathcal{L}_{\text{curves}}(\theta) = \frac{1}{KN} \sum_{k,j} \Big\{ \Big(\frac{\partial}{\partial t} \gamma^1_{\theta,\, s_k}(t_j) - a\big(\gamma_{\theta,\, s_k}(t_j)\big) \Big)^2 + \Big(\frac{\partial}{\partial t} \gamma^2_{\theta,\, s_k}(t_j) - b\big(\gamma_{\theta,\, s_k}(t_j)\big) \Big)^2 \Big\}.$$

The trained network $\gamma_{\theta^*}(s,t)$ then approximates $c_s(t)$, where

$$\theta^* \;=\; \arg\min_\theta \mathcal{L}_{\text{curves}}(\theta).$$

2. Finding $f_s(t) = u(c_s(t))$. To solve the initial value problem

$$\begin{aligned} \frac{d}{dt} f_s(t) &= F\big(f_s(t)\big), \\ f_s(0) &= \varphi(s), \end{aligned}$$

we employ a one-dimensional neural network

$$f_s(w;t) \;=\; \varphi(s) + t\,\mathrm{NN}(w; s, t),$$

which enforces the initial condition by construction. We train this network by minimizing the loss

$$\mathcal{L}_f(w) \;=\; \frac{1}{KN}\sum_{k,j}\Big(\frac{\partial}{\partial t}f_{s_k}(w;t_j) - F\big(f_{s_k}(w;t_j)\big)\Big)^2.$$

The resulting approximation is

$$f_s(w^*;t) \;\approx\; f_s(t),$$

where w^* denotes the optimal parameter vector.

3. Finding (x,y) in terms of (s,t). If the point (x,y) is represented as $c(t,s)$, then

$$x = c^1(t,s), \qquad y = c^2(t,s).$$

This system can be inverted to

$$s = s(x,y), \qquad t = t(x,y),$$

provided that the transformation Jacobian is nonsingular:

$$J(t,s) = \begin{vmatrix} \partial_s c_s^1(t) & \partial_t c_s^1(t) \\ \partial_s c_s^2(t) & \partial_t c_s^2(t) \end{vmatrix} \neq 0.$$

It suffices to require $J(0,s) \neq 0$, because by continuity it follows that $J(t,s) \neq 0$ for t sufficiently small. Since

$$J(0,s) = \begin{vmatrix} \partial_s c_s^1(0) & \partial_t c_s^1(0) \\ \partial_s c_s^2(0) & \partial_t c_s^2(0) \end{vmatrix} = \begin{vmatrix} 1 & a(s,0) \\ 0 & b(s,0) \end{vmatrix} = b(s,0),$$

the nonsingularity condition reduces to $b(s,0) \neq 0$, meaning that the coefficient b does not vanish along the initial curve.

Under this condition, the system can be locally inverted. To find the inverse, we employ two neural networks, $s_\eta(x,y)$ and $t_\eta(x,y)$, which satisfy

$$\begin{aligned} x &= c^1\big(t_\eta(x,y), s_\eta(x,y)\big), \\ y &= c^2\big(t_\eta(x,y), s_\eta(x,y)\big). \end{aligned}$$

The networks are trained jointly by minimizing the loss

$$\begin{aligned} \mathcal{L}_{\text{inverse}}(\eta) \;=\; & \frac{1}{KN}\sum_{k,j}\Big(x_k - c^1\big(t_\eta(x_k,y_j), s_\eta(x_k,y_j)\big)\Big)^2 \\ & + \frac{1}{KN}\sum_{k,j}\Big(y_j - c^2\big(t_\eta(x_k,y_j), s_\eta(x_k,y_j)\big)\Big)^2. \end{aligned}$$

4. Finding the solution. The solution is obtained by substituting x and y in terms of s and t into the expression for u:

$$u(x,y) = u\big(c(t,s)\big) = f_{w^*}\big(t_{\eta^*}(x,y), s_{\eta^*}(x,y)\big),$$

where f_{w^*} is obtained in Step 2, and t_{η^*}, s_{η^*} are obtained in Step 3.

Remark. Splitting the procedure into several smaller steps provides greater control over the solution-finding process. Thus, instead of training once the entire neural network, we train on modules and assemble the results at the end.

Implementation example We shall check the previous methods on the following initial condition PDE

$$\begin{aligned} x\partial_x u + \partial_y u &= u \\ u(x,0) &= x^2. \end{aligned}$$

For this problem there is a closed form solution. We can easily verify that $u(x,y) = x^2 e^{-y}$ is a global solution, with $(x,y) \in \mathbb{R} \times [0,+\infty)$. Thus, we can compare the neural network solution against this closed-form solution.

1. We used the "brute-force" setup on the domain $(a,b) \times [0,T] = [0,1] \times [0,1]$, discretised into $5{,}000 \times 512$ collocation points. The network had three hidden layers with 64 neurons each, and we compared `tanh` versus `swish` activations, training for 3,000 epochs.

First, we show slices of the solution at four representative times $y \in \{0.25,\ 0.50,\ 0.75,\ 1.00\}$. With `swish`, the slices coincide with the exact ones essentially perfectly, see Fig. 2.26; with `tanh`, the error grows as y increases, see Fig. 2.27.

The loss curves tell the same story: the `tanh` model stabilises around 10^{-5}, while `swish` reaches about 10^{-7}, roughly two orders of magnitude lower, see Fig. 2.28.

Is there an activation function better than `swish`? Possibly, but the "best" choice is problem dependent and remains an empirical question.

The complete code can be found at: `https://tinyurl.com/mx5x7uz7`

2. We now apply the method of characteristics. The initial curve is given by $c_s(0) = (s,0)$, and the family of integral curves starting at $c_s(0)$ is $c_s(t) = (se^t, t)$. The condition $b(s,0) = 1 \neq 0$ is clearly satisfied for all s. Consequently, the system

$$x = se^t, \qquad y = t$$

can be inverted to

$$s = xe^{-y}, \qquad t = y.$$

The associated equation

$$\frac{d}{dt} f_s(t) = f_s(t), \qquad f_s(0) = s^2$$

has the solution $f_s(t) = s^2 e^t$. Substituting $s = s(x,y)$ and $t = t(x,y)$ from the above expressions yields

$$u(x,y) = s^2 e^t \big|_{s=xe^{-y},\, t=y} = x^2 e^{-y}.$$

As an exercise, the reader can verify that these formulas can also be recovered using a neural network approach; see Exercise 2.12.1.

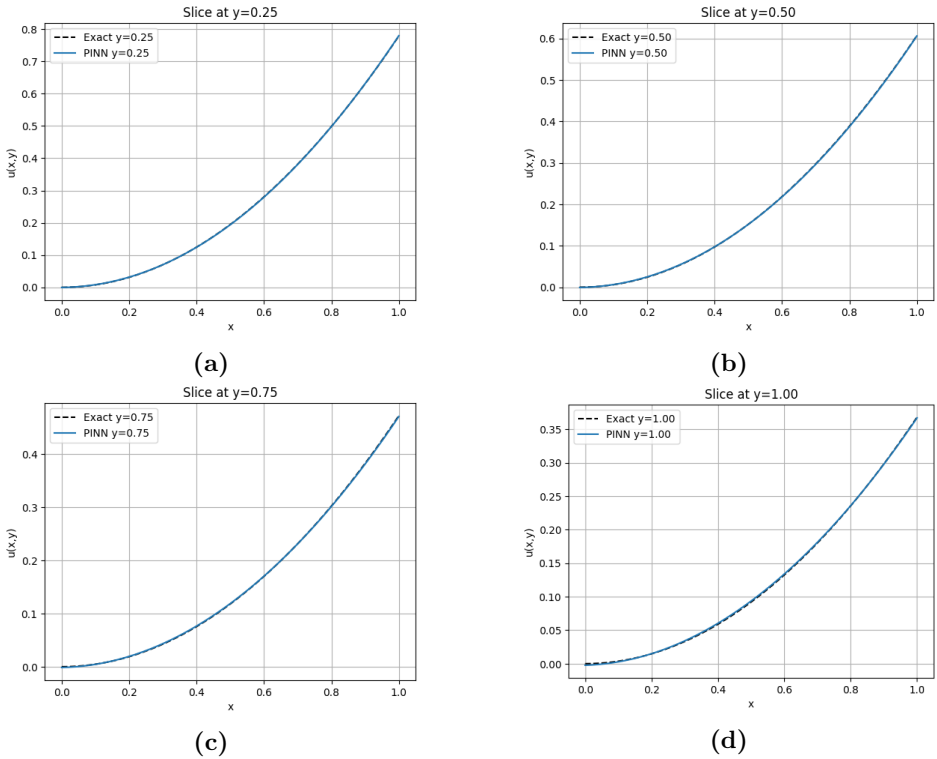


Figure 2.26: Slices through the neural network solution $u(x, y)$ in the case of `swish` activation function for: (a) $y = 0.25$; (b) $y = 0.50$; (c) $y = 0.75$; (d) $y = 1.00$.

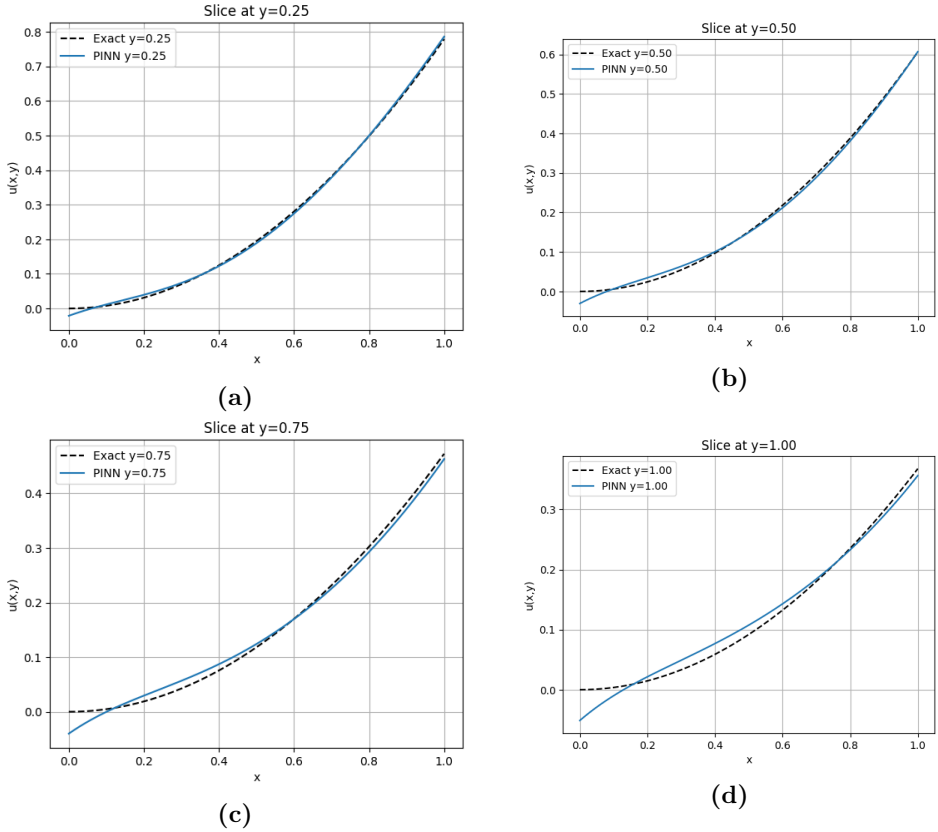


Figure 2.27: Slices through the neural network solution $u(x, y)$ in the case of `tanh` activation function for: (a) $y = 0.25$; (b) $y = 0.50$; (c) $y = 0.75$; (d) $y = 1.00$.

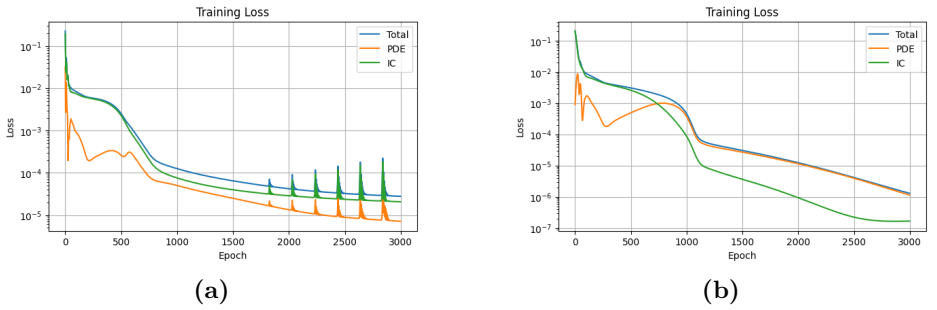


Figure 2.28: Loss functions in the cases of: (a) `tanh` activation; (b) `swish` activation.

Remark 2.11.1 If the problem's dimension is small, both the brute-force method and the method of characteristics yield the same solution. However, as the dimension increases, the brute-force approach loses its efficiency.

A general multi-dimensional problem takes the form

$$\sum_{i=1}^{n} a_i(x)\,\partial_{x_i} u = F(u), \qquad u\big|_{x_n=0} = \varphi(x_1, \ldots, x_{n-1}).$$

The characteristic curves (i.e., the integral curves of the vector field $X = \sum_{i=1}^{n} a_i(x)\,\partial_{x_i}$) are determined by

$$\frac{d}{dt} x_j(t,s) = a_j\big(x(t)\big),$$

with initial conditions on the hyperplane $\{x_n = 0\}$:

$$x(0,s) = (s_1, \ldots, s_{n-1}, 0).$$

Along these curves, the PDE reduces to the ODE

$$\frac{d}{dt} f_s(t) = F\big(f_s(t)\big), \qquad f_s(0) = \varphi(s_1, \ldots, s_{n-1}),$$

where $f_s(t) = u(c(t))$ and $c(t) = x(t,s)$.

The system

$$x_j = x_j(s_1, \ldots, s_{n-1}, t), \quad 1 \le j \le n,$$

can be inverted locally, provided that

$$\det\left(\frac{\partial(x_i)}{\partial(s_j,t)}\right)_{t=0} = a_n(s_1, \ldots, s_{n-1}, 0) \neq 0.$$

All these steps can be implemented using neural networks, whereas a brute-force approach in n variables quickly becomes inefficient in higher dimensions.

2.12 Exercises

Exercise 2.12.1 Consider the first-order linear PDE

$$x\,u_x(x,y) + u_y(x,y) = u(x,y), \qquad (x,y) \in \mathbb{R} \times [0,\infty),$$

with initial condition

$$u(x,0) = x^2.$$

(a) Derive the characteristic ODEs and their initial curve:

$$\dot{x} = x, \qquad \dot{y} = 1, \qquad (x(0), y(0)) = (s, 0).$$

Using a small neural network $\gamma_\theta(t;s) = (x_\theta(t;s), y_\theta(t;s))$, learn the family of integral curves starting at $(s,0)$ by minimising the residuals

$$\big\|\dot{x}_\theta - x_\theta\big\|^2 \;+\; \big\|\dot{y}_\theta - 1\big\|^2,$$

subject to the initial conditions $x_\theta(0;s) = s$, $y_\theta(0;s) = 0$. Report the learned parametric family $c_s(t)$.

(b) From the learned (or derived) characteristics $c_s(t) = (x(t), y(t))$, invert the mapping $(s,t) \mapsto (x,y)$ to obtain $s = s(x,y)$ and $t = t(x,y)$.

(c) Along each characteristic, solve the initial-value problem

$$\frac{d}{dt} f_s(t) = f_s(t), \qquad f_s(0) = s^2,$$

and express $f_s(t)$ explicitly.

(d) Compose your results from (b)–(c) to obtain the exact solution $u(x,y)$ in closed form.

(e) Train a PINN $u_\theta(x,y)$ for the PDE on the square $[0,1] \times [0,1]$ using a standard residual loss for $xu_x + u_y - u = 0$ and the boundary condition $u(x,0) = x^2$. Compare u_θ with the exact solution from (d) on the slices $y \in \{0.00, 0.25, 0.50, 0.75, 1.00\}$. Plot both the slice curves and the slice errors as functions of x.

Exercise 2.12.2 Consider the first-order linear PDE

$$y\,\partial_x u(x,y) \;+\; x\,\partial_y u(x,y) \;=\; u(x,y), \qquad (x,y) \in \mathbb{R}^2,$$

with initial condition

$$u(x,0) = 1 \quad \text{for all } x \in \mathbb{R}.$$

(a) Derive the characteristic ODEs and solve them starting from the initial curve $(x(0), y(0)) = (s, 0)$:

$$\dot{x} = y, \qquad \dot{y} = x, \qquad \dot{f} = f, \qquad f(0) = 1.$$

Show that

$$x(t) = s\cosh t, \qquad y(t) = s\sinh t, \qquad f_s(t) = e^t.$$

(b) Invert the map $(s,t) \mapsto (x,y)$ to obtain

$$s = \sqrt{x^2 - y^2}, \qquad t = \tanh^{-1}\!\Big(\frac{y}{x}\Big) = \frac{1}{2}\ln\!\Big(\frac{x+y}{x-y}\Big),$$

and conclude that on the domain $x^2 > y^2$ the solution is

$$u(x,y) = e^{\,\tanh^{-1}(y/x)} = \sqrt{\frac{x+y}{x-y}}.$$

Verify that $u(x,0) = 1$.

(c) Solve the PDE with a physics-informed neural network on a box that lies inside the domain of dependence $D = \{(x,y) : x^2 > y^2\}$. Use a residual loss for $yu_x + xu_y - u = 0$ and enforce $u(x,0) = 1$.

(d) Compare the learned solution $u_\theta(x,y)$ with the closed-form $u(x,y) = \sqrt{\frac{x+y}{x-y}}$ on several horizontal slices $y \in \{y_1, y_2, y_3, y_4\}$. Plot, for each chosen y, both curves as functions of x and the corresponding slice error.

Exercise 2.12.3 Consider the first-order PDE

$$x\,\partial_x u(x,y) + \left(1+y^2\right)\partial_y u(x,y) = u(x,y), \qquad u(x,0) = e^x.$$

(a) Use the method of characteristics to find the closed-form solution $u(x,y)$.

(b) Implement the "brute-force" PINN method to solve the PDE on a suitable rectangular domain $(x,y) \in [a,b] \times [0,T]$ and compare the numerical solution with the closed-form one by taking several y-slices.

(c) Plot the heat maps of both the exact and the neural network solutions, as well as their difference.

This page intentionally left blank

Part II

Direct Problems: Predicting Motion with Deep Learning

This page intentionally left blank

Chapter 3

Introduction to Classical Mechanics with Neural Nets

3.1 Learning Trajectories of Dynamical Systems

In this section, we describe how to use neural networks to learn and predict curves, specifically trajectories of dynamical systems. In mechanics, the evolution of a system is often represented by a curve $c(t)$ in the configurations space $\mathbb{R}^n$, where $t \in [0, T]$ is time. Typically, an explicit analytical expression for $c(t)$ is not available. However, partial information about the trajectory may be known from measurements, such as discrete points $P_i = c(t_i)$, $i = 1, \ldots, N$, sampled along the curve.

Additionally, boundary conditions, namely the initial and final points $c(0)$ and $c(T)$, may be known. In some cases, the trajectory is governed by a differential equation, which also might be available. Our goal is to approximate the target curve $c(t)$ using a neural network trained on this information.

3.1.1 Neural Network Approximation of the Curve

We define a feed-forward neural network $f_\theta(t) = NN(\theta; t)$, where:

- $t \in \mathbb{R}$ is the one-dimensional time input,
- $f_\theta(t) = \big(f_\theta^1(t), f_\theta^2(t), \ldots, f_\theta^n(t)\big) \in \mathbb{R}^n$ is the output vector approximating $c(t)$,
- θ represents the trainable network parameters (weights and biases).

The training objective is to adjust θ such that the network output $f_\theta(t)$ approximates the target trajectory $c(t) = \big(c^1(t), c^2(t), \ldots, c^n(t)\big)$.

3.1.2 Loss Functions

We distinguish several loss functions as follows.

Boundary Loss. We first enforce the boundary conditions by introducing the boundary loss

$$\mathcal{L}_{\text{bd}}(\theta) = \|f_\theta(0) - c(0)\|^2 + \|f_\theta(T) - c(T)\|^2,$$

where $\|\cdot\|$ denotes the Euclidean norm in $\mathbb{R}^n$. This term penalizes deviations from the known initial and final points of the trajectory.

Data Loss. If the trajectory is known at some intermediate points $\{t_j\}_{j=1}^k$, we define the data loss as

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{k}\sum_{j=1}^{k} \|f_\theta(t_j) - c(t_j)\|^2 = \sum_{j=1}^{k}\sum_{i=1}^{n} \left(f_\theta^i(t_j) - c^i(t_j)\right)^2.$$

This term ensures the network predictions align with the measured points along the curve.

Differential Equation Loss. If the trajectory is governed by a first-order differential equation:

$$\frac{dc^k}{dt} = v^k(c(t)), \quad k = 1, \ldots, n$$

where $v^k : \mathbb{R}^n \to \mathbb{R}$ are known smooth functions, we define the ODE loss:

$$\mathcal{L}_{\text{ODE}}(\theta) = \int_0^T \sum_{k=1}^{n} \left(\frac{d}{dt} f_\theta^k(t) - v^k(f_\theta(t))\right)^2 dt.$$

This loss measures how well the neural network output satisfies the underlying dynamics.

For practical implementation, the ODE loss is approximated by a discrete Riemann sum:

$$\tilde{\mathcal{L}}_{\text{ODE}}(\theta) = \frac{T}{N}\sum_{j=1}^{N}\sum_{k=1}^{n} \left(\frac{d}{dt} f_\theta^k(t_j) - v^k(f_\theta(t_j))\right)^2$$

where $\{t_j\}_{j=1}^N$ is a set of discretization points over the interval $[0, T]$.

Remark 3.1.1 In the discussion above, we considered dynamical systems governed by a first-order ordinary differential equation (ODE). However, similar arguments and methodologies apply when the system's dynamics are described by higher-order ODEs.

For example, let's consider the case where the target trajectory $c(t)$ is governed by a second-order ordinary differential equation:

$$\frac{d^2c^k(t)}{dt^2} = w^k\left(c(t), \frac{dc(t)}{dt}\right), \quad k = 1, \ldots, n$$

where $w^k : \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$ are known smooth functions defining the dynamics.

To incorporate this physical law into the learning framework, we define the ODE loss function as:

$$\mathcal{L}^{(2)}_{\text{ODE}}(\theta) = \int_0^T \sum_{k=1}^{n} \left(\frac{d^2}{dt^2} f_\theta^k(t) - w^k \left(f_\theta(t), \frac{d}{dt} f_\theta(t) \right) \right)^2 dt.$$

In practice, this continuous loss is approximated using a discrete set of points $\{t_j\}_{j=1}^N$, leading to the discrete ODE loss:

$$\tilde{\mathcal{L}}^{(2)}_{\text{ODE}}(\theta) = \frac{1}{N} \sum_{j=1}^{N} \sum_{k=1}^{n} \left(\frac{d^2}{dt^2} f_\theta^k(t_j) - w^k \left(f_\theta(t_j), \frac{d}{dt} f_\theta(t_j) \right) \right)^2.$$

Here:

- $\frac{d^2}{dt^2} f_\theta^k(t_j)$ is the second time derivative of the neural network output, computed via automatic differentiation.
- $\frac{d}{dt} f_\theta(t_j)$ is the first derivative (velocity), also computed automatically.

Minimizing $\tilde{\mathcal{L}}^{(2)}_{\text{ODE}}(\theta)$ ensures that the neural network output $f_\theta(t)$ approximates a solution to the second-order dynamics throughout the domain $[0, T]$.

In general, for any m-th order ODE, the neural network $f_\theta(t)$ can be trained to minimize a loss function that penalizes deviations from the ODE at the appropriate derivative order. Automatic differentiation tools available in modern deep learning frameworks make it feasible to compute higher-order derivatives efficiently during training.

Thus, the approach described here naturally generalizes to systems governed by higher-order differential equations, which frequently appear in Classical Mechanics, such as in Lagrangian or Newtonian formulations of motion.

3.1.3 Total Loss and Optimization

The total loss function combines the three components

$$\mathcal{L}_{\text{tot}}(\theta) = \lambda_{\text{bd}} \mathcal{L}_{\text{bd}}(\theta) + \lambda_{\text{data}} \mathcal{L}_{\text{data}}(\theta) + \lambda_{\text{ODE}} \tilde{\mathcal{L}}_{\text{ODE}}(\theta)$$

where λ_{bd}, λ_{data} and λ_{ODE} are positive Lagrange multipliers that can be increased to enforce some of the conditions more strictly if necessary. Otherwise, by default they are left equal to 1.

The optimization problem consists of minimizing the total loss:

$$\theta^* = \arg\min_\theta \mathcal{L}_{\text{tot}}(\theta).$$

At the optimum, the trained neural network $f_{\theta^*}(t)$ approximates the target curve $c(t)$, namely

$$f_{\theta^*}(t) \approx c(t), \quad \forall t \in [0, T].$$

Remark 3.1.2 (*i*) We make the remark that in practice, challenges may arise, such as the network failing to pass precisely through the boundary points $c(0)$ and $c(T)$. In such cases, increasing the weight λ_{bd} in the total loss function helps enforce these constraints more effectively.

(*ii*) Another important remark is that the set of intermediate points $\{t_j\}$ used for computing the data loss $\mathcal{L}_{\text{data}}$ and the set of points used for evaluating the ODE loss $\tilde{\mathcal{L}}_{\text{ODE}}$ are not necessarily the same.

(*iii*) In practice, obtaining measurements of the target curve $c(t)$ at many intermediate points can be costly, time-consuming, or even infeasible, especially in experimental setups where each measurement requires significant effort or resources. As a result, the number of available data points $\{t_j^{\text{data}}\}$ may be sparse, and there may be large intervals of time where no measurements are available.

(*iv*) On the other hand, the points used for computing the ODE loss $\tilde{\mathcal{L}}_{\text{ODE}}$, denoted $\{t_j^{\text{ODE}}\}$, are chosen freely for numerical integration and training purposes. These points can be sampled densely over the entire time domain $[0, T]$, including regions where no measurement data exists.

(*v*) This dense sampling of $\{t_j^{\text{ODE}}\}$ serves a crucial role: it allows the neural network to learn the dynamics of the system even in regions where measurements are sparse or missing. By enforcing the underlying differential equation throughout the domain, the model can generate physically consistent predictions and interpolate the trajectory $c(t)$ in unmeasured intervals.

In summary, while measurement data points are limited and may be widely spaced due to practical constraints, the ODE loss can be evaluated at a finer resolution. This helps the neural network generalize better and reconstruct the entire trajectory accurately, even in regions without direct observations.

This framework provides a flexible way to learn trajectories that incorporate measurement data, boundary conditions, and physical laws, making it well-suited for applications in mechanics and dynamical systems.

3.1.4 Finding Integral Curves of a Vector Field

Let $v = \big(v^1, v^2, \dots, v^n\big)$ be a vector field on $\mathbb{R}^n$. Given a point $p \in \mathbb{R}^n$ with coordinates $p = \big(x_0^1, x_0^2, \dots, x_0^n\big)$, our goal is to find the integral curve of the vector field v that passes through p. An *integral curve* is a function $c(t) = \big(c^1(t), c^2(t), \dots, c^n(t)\big)$ satisfying the initial condition: $c(0) = p$, and the differential equation

$$\frac{d}{dt} c^i(t) = v^i\big(c(t)\big), \quad i = 1, 2, \dots, n. \tag{3.1.1}$$

Under suitable conditions on v (for example, if v satisfies a Lipschitz condition), a unique local solution $c(t)$ exists for t in a small interval $[0, T]$. This system of ODEs can be equivalently written in integral form as:

$$c^i(t) = x_0^i + \int_0^t v^i\big(c(s)\big)\, ds, \quad i = 1, 2, \dots, n.$$

In many practical cases, an explicit solution to these equations is not available, which motivates the use of approximation methods.

Method I: Direct Neural Network Approximation

We approximate the integral curve $c(t)$ with a neural network. Consider a feed-forward neural network defined by $f_\theta(t) = \big(f_\theta^1(t), f_\theta^2(t), \ldots, f_\theta^n(t)\big)$, where:

- $t \in \mathbb{R}$ is the input (representing time),
- $f_\theta(t) \in \mathbb{R}^n$ is the network output approximating the integral curve,
- θ denotes the trainable parameters of the network.

The network is trained to satisfy two primary conditions:

1. ODE Loss. The network output should approximately satisfy the ODE

$$\frac{d}{dt} f_\theta^i(t) \approx v^i\big(f_\theta(t)\big), \quad i = 1, 2, \ldots, n.$$

We impose this condition by defining an ODE loss function:

$$\mathcal{L}_{\text{ODE}}(\theta) = \int_0^T \sum_{i=1}^n \left(\frac{d}{dt} f_\theta^i(t) - v^i\big(f_\theta(t)\big) \right)^2 dt.$$

In practice, this integral is approximated using a discrete set of time points.

2. Initial Condition Loss. To ensure that the network output satisfies the initial condition $f_\theta(0) = p$, we introduce an initial condition loss:

$$\mathcal{L}_{\text{IC}}(\theta) = \sum_{i=1}^n \big(f_\theta^i(0) - x_0^i\big)^2.$$

3. Total Loss. The overall objective is to minimize the total loss function:

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{ODE}}(\theta) + \mathcal{L}_{\text{IC}}(\theta).$$

Minimizing $\mathcal{L}_{\text{tot}}(\theta)$ with respect to the network parameters θ leads to an approximation $f_{\theta^*}(t)$ that closely follows the true integral curve $c(t)$.

In the following we shall discuss an alternative method.

Method II: Integral Formulation Approach

Instead of directly enforcing the differential form of the ODE, we can utilize its equivalent integral form. Recall that if $c(t)$ is the integral curve of the vector field $v = \big(v^1, v^2, \ldots, v^n\big)$ passing through $p = \big(x_0^1, x_0^2, \ldots, x_0^n\big)$, then for each component $c^i(t)$ we have satisfied formula (3.1.1).

In this method, we approximate $c(t)$ with a neural network

$$f_\theta(t) = \left(f_\theta^1(t), f_\theta^2(t), \ldots, f_\theta^n(t)\right).$$

The idea is to make the network output satisfy the integral equation by minimizing the discrepancy between $f_\theta(t)$ and the right-hand side of the aforementioned equation.

Specifically, for a discrete set of time points $\{t_j\}_{j=1}^N$, we define the loss function:

$$\mathcal{L}_2(\theta) = \sum_{i=1}^{n} \sum_{j=1}^{N} \left(f_\theta^i(t_j) - \left(x_0^i + \int_0^{t_j} v^i\big(f_\theta(s)\big)\, ds \right) \right)^2.$$

This loss function penalizes the squared error between the neural network's prediction $f_\theta^i(t_j)$ and the integrated dynamics $x_0^i + \int_0^{t_j} v^i(f_\theta(s))\, ds$. Minimizing $\mathcal{L}_2(\theta)$ with respect to the network parameters θ encourages the network to learn an approximation to the true integral curve of the vector field.

In practice, the integral $\int_0^{t_j} v^i\big(f_\theta(s)\big)\, ds$ is computed numerically (e.g., using a quadrature rule or a Riemann sum) for each time point t_j. This method is particularly useful when it is more natural or convenient to work with the integral formulation rather than differentiating the neural network output.

3.1.5 Particle Trajectory Reconstruction from Position and Velocity

In many real-world applications, continuous observations of a particle's motion are unavailable; instead, measurements are made at discrete time instances. Suppose a particle is observed at times

$$t_0 = 0 < t_1 < \cdots < t_N = T,$$

and at each time t_j both its position x_j and velocity v_j are recorded. The goal is to reconstruct the full trajectory of the particle from these data.

To tackle this problem, we employ a feed-forward neural network with input $t \in \mathbb{R}$ and n-dimensional output $f_\theta(t)$, which is intended to learn the particle trajectory $c(t)$. The network is trained to satisfy the conditions

$$c(t_j) = x_j \quad \text{and} \quad c'(t_j) = v_j, \quad \text{for } j = 1, \ldots, N.$$

These conditions are enforced by defining two loss functions. The *position loss* measures the discrepancy between the network output and the observed positions:

$$\mathcal{L}_{\text{pos}}(\theta) = \sum_{j=1}^{N} \left\| f_\theta(t_j) - x_j \right\|^2,$$

while the *velocity loss* quantifies the difference between the time derivative of the network output and the observed velocities:

$$\mathcal{L}_{\text{vel}}(\theta) = \sum_{j=1}^{N} \left\| \frac{d}{dt} f_\theta(t_j) - v_j \right\|^2.$$

In the above expressions, $\|\cdot\|^2$ denotes the square of the Euclidean norm in $\mathbb{R}^n$, the space where the curve $c(t)$ lies.

The overall objective is to minimize the combined loss, defined as a weighted sum of the position and velocity losses:

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{pos}}(\theta) + \lambda\, \mathcal{L}_{\text{vel}}(\theta),$$

where $\lambda > 0$ is a hyperparameter that controls the relative importance of the velocity constraint compared to the position constraint. By tuning λ, one can emphasize the accuracy of the velocity matching, thereby influencing the smoothness and dynamical consistency of the reconstructed trajectory.

The network parameters θ are optimized using a gradient-based optimizer (e.g., Adam or SGD) to minimize $\mathcal{L}_{\text{tot}}(\theta)$. Let

$$\theta^* = \arg\min_{\theta} \mathcal{L}_{\text{tot}}(\theta)$$

denote the optimal parameters. Minimizing $\mathcal{L}_{\text{tot}}(\theta)$ forces the neural network to produce a trajectory $f_{\theta^*}(t)$ that passes through the given positions $\{x_j\}$ and has the correct velocities $\{v_j\}$ at the measurement times.

During training, the neural network is adjusted such that both the position and velocity losses decrease. As a result, the network learns to produce a continuous function $f_{\theta^*}(t)$ that not only interpolates the measured positions x_j but also reproduces the corresponding velocity profiles v_j. Once training is complete, the network $f_{\theta^*}(t)$ provides a smooth approximation to the particle's trajectory over the entire time interval, and this function can be evaluated at any t to reconstruct the full trajectory.

3.2 Reconstructing a Vector Field

We have seen that given some points x_i at time instances t_i, a neural network can be trained to reconstruct a curve $c(t)$ that passes through these points. A similar idea can be employed to reconstruct an entire vector field from discrete measurements.

More precisely, consider the following problem: suppose that we have measurements of a vector field at a set of points, i.e., we are given pairs $\{(x_i, v_i)\}_{i=1}^N$, where each $x_i \in \mathbb{R}^n$ is a point in space and $v_i \in \mathbb{R}^n$ is the corresponding vector value of the field at that point. The goal is to construct a continuous vector field $v(x)$ such that for any point x in the domain, the network approximates the vector field value $v(x)$.

To achieve this, we first define a neural network $f_\theta(x)$ parameterized by θ that takes as input the spatial coordinate $x \in \mathbb{R}^n$ and outputs a vector $f_\theta(x) \in \mathbb{R}^n$. The network is designed to approximate the vector field, so that ideally,

$$f_\theta(x) \approx v(x).$$

In order to train this network, we introduce a loss function that measures the discrepancy between the network's output and the provided vector field measurements. A common choice is the mean squared error:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \|f_\theta(x_i) - v_i\|^2,$$

where $\|\cdot\|$ denotes the Euclidean norm in $\mathbb{R}^n$. This loss function encourages the network to produce vector field values that closely match the measured vectors at the corresponding positions.

The network parameters θ are then optimized using a gradient-based optimizer (such as Adam or SGD) to minimize $\mathcal{L}(\theta)$. During the training process, the network learns a continuous mapping $f_\theta(x)$ that approximates the underlying vector field from the discrete data. Once training is complete, the neural network $f_{\theta^*}(x)$ (with optimal parameters θ^*) provides an estimate of the vector field at any point x in the domain. In other words, given any x, one can evaluate $f_{\theta^*}(x)$ to obtain the corresponding vector $v(x)$.

In summary, by training a neural network on the discrete measurements $\{(x_i, v_i)\}$, we obtain a continuous representation of the vector field, enabling us to reconstruct the entire field and determine the vector v at any desired point x. An important consequence of this fact is to be able to construct the associated vector flow.

3.3 Reconstructing the Flow

We consider the following problem, which is equivalent to finding the flow from discrete measurements. Suppose we have measurements $\{(x_i, v_i)\}_{i=1}^N$ obtained from a system containing k particles. In practice, these measurements are unordered; that is, the N pairs (x_i, v_i) may come from any of the k particles. Our goal is to reconstruct the associated flow, namely, given a position x, to construct the trajectory (curve) of the particle that happens to pass through x at time $t = 0$.

The proposed approach proceeds in two steps.

Step 1: First, we use the discrete measurements to construct the vector field $v(x)$ on a region of $\mathbb{R}^n$, where x_i belong to, by training a neural network $f_\theta(x)$ as in Section 3.2. The network is trained to approximate the vector field, i.e.,

$$f_\theta(x) \approx v(x),$$

by minimizing the loss

$$\mathcal{L}_{\mathrm{VF}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \|f_\theta(x_i) - v_i\|^2,$$

where $\|\cdot\|$ denotes the Euclidean norm in $\mathbb{R}^n$. Once trained, we obtain an approximation $f_{\theta^*}(x)$ of the vector field.

Step 2: In the second step, we reconstruct the trajectory of a particle that passes through a given point x by computing the integral curve of the learned vector field. More precisely, we wish to solve the initial value problem

$$c'(t) = v(c(t)), \quad c(0) = x.$$

Since we only have an approximation $f_{\theta^*}(x)$ for $v(x)$, we instead solve

$$c'(t) = f_{\theta^*}(c(t)), \quad c(0) = x.$$

To do this, we introduce a second neural network $g_w(t)$ with weights w and input t, intended to approximate the trajectory $c(t)$. The network $g_w(t)$ is designed to satisfy the initial condition $g_w(0) = x$ and the differential equation

$$\frac{d}{dt} g_w(t) \approx f_{\theta^*}(g_w(t)).$$

We set up two loss functions to train $g_w(t)$. First, the *initial condition loss* is defined as

$$\mathcal{L}_{\text{IC}}(w) = \|g_w(0) - x\|^2 .$$

Second, the *ODE loss* enforces the differential equation constraint:

$$\mathcal{L}_{\text{ODE}}(w) = \int_0^T \left\| \frac{d}{dt} g_w(t) - f_{\theta^*}(g_w(t)) \right\|^2 dt.$$

In practice, the integral is approximated by a discrete sum over a set of time points. The overall loss for training the trajectory network is then given by

$$\mathcal{L}_{\text{tot}}(w) = \mathcal{L}_{\text{IC}}(w) + \lambda \mathcal{L}_{\text{ODE}}(w),$$

where $\lambda > 0$ is a hyperparameter that balances the relative importance of satisfying the ODE constraint versus the initial condition.

Once training is complete, the network $g_{w^*}(t)$ (with optimal weights w^*) provides a smooth approximation to the trajectory of the particle that was at position x at time $t = 0$.

In summary, this approach employs two neural networks in sequence: the first learns the vector field from the discrete measurements, and the second computes the integral curve of the learned field, thereby reconstructing the flow.

3.4 Practical Implementation

The following implementation code can be found at:

`https://tinyurl.com/2x8vyzju`

Consider the planar vector field

$$F(x, y) \;=\; (x + y)\,\frac{\partial}{\partial x} \;+\; (x - y)\,\frac{\partial}{\partial y},$$

which has a saddle point at the origin $(0, 0)$. The following code snippet constructs and visualizes this field on a uniform grid in $\mathbb{R}^2$, see Fig. 3.1.

```
import numpy as np
import matplotlib.pyplot as plt

# Create grid
x = np.linspace(-2, 2, 20)
y = np.linspace(-2, 2, 20)
```

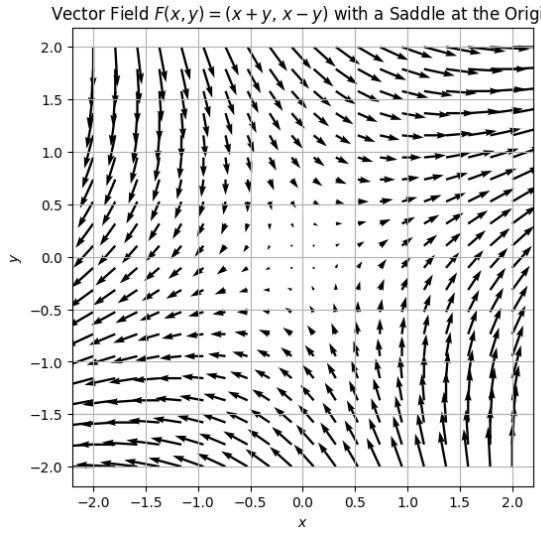


Figure 3.1: The vector field $F(x, y) = (x + y, x - y)$.

```
X, Y = np.meshgrid(x, y)
# Define the vector field components
U = X + Y      # component in the x-direction
V = X - Y      # component in the y-direction
# Plot with quiver
plt.figure(figsize=(6,6))
plt.quiver(X, Y, U, V, angles='xy', scale_units='xy', scale=10,
    width=0.005)
plt.title("Vector Field $F(x,y)=(x+y,\,x-y)$
    with a Saddle at the Origin")
plt.xlabel("$x$")
plt.ylabel("$y$")
plt.axis('equal')
plt.grid(True)
plt.show()
```

Then we randomly select $N = 100$ vectors, forming our training set for the vector field. Concretely:

```
# Stack all grid points and their vectors into one array
all_data = np.column_stack([
    X.ravel(),    # x-coordinate
    Y.ravel(),    # y-coordinate
    U.ravel(),    # vector's x-component
    V.ravel()     # vector's y-component
])
```

```
# Fix the random seed for reproducibility
np.random.seed(42)

# Choose 100 distinct indices without replacement
indices = np.random.choice(all_data.shape[0], size=100, replace=False)

# Extract the sampled rows
sample    = all_data[indices]
x_train   = sample[:, :2].astype(np.float32)
# (x_i, y_i) pairs, shape (100,2)
y_train   = sample[:, 2:].astype(np.float32)
# (u_i, v_i) vectors, shape (100,2)
```

Here, $\mathbf{x}_{\text{train}}$ is a (100×2) array of the sampled positions (x_i, y_i), and $\mathbf{y}_{\text{train}}$ is a (100×2) array of the corresponding vector values (u_i, v_i). These serve respectively as the inputs and targets for training a neural network

$$f_\theta : \mathbb{R}^2 \longrightarrow \mathbb{R}^2,$$

which learns to approximate the original vector field. The network has two hidden layers and a two-dimensional output:

```
model = Sequential([
    Input(shape=(2,)),
    Dense(32, activation='tanh'),
    Dense(32, activation='tanh'),
    Dense(2, activation='linear')
])
```

We compile the network using Adam optimizer and mean square error as a loss function:

```
model.compile(optimizer='adam', loss='mse')
```

Then we train on the sample data to obtain the optimal parameter θ

```
history = model.fit(x_train, y_train, epochs=500, batch_size=16)
```

Then we evaluate the learned field on the same grid

```
points = np.column_stack([X.ravel(), Y.ravel()]).astype(np.float32)
pred = model.predict(points)
U_pred = pred[:, 0].reshape(X.shape)
V_pred = pred[:, 1].reshape(Y.shape)
```

and plot both the learned (red) versus the original (blue) vector fields in Fig. 3.2.

The training converges very well, as the following loss indicates:

```
Epoch 500/500
7/7  0s 6ms/step - loss: 8.9969e-04
```

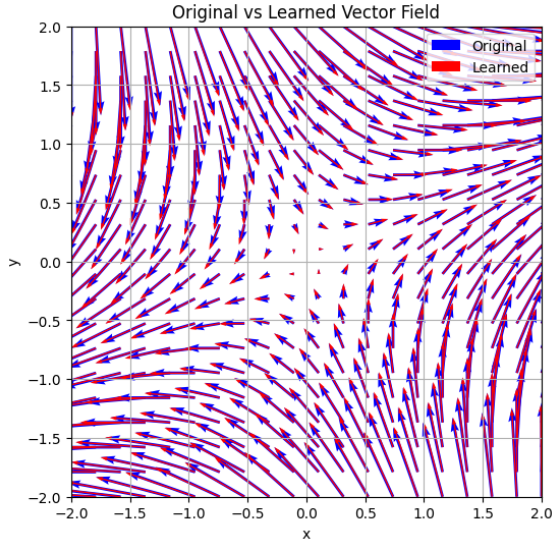


Figure 3.2: The original vector field $F(x, y) = (x + y, x - y)$, versus the learned vector field $f_\theta(x, y)$.

3.5 Damped Spring-Mass System

A mass m attached to an elastic spring with damping satisfies the ODE

$$m\,u''(t) + \mu\,u'(t) + k\,u(t) = 0, \tag{3.5.2}$$

where $u(t)$ is the displacement, k is the spring constant, and μ is the damping coefficient; there are no external forces. The characteristic equation

$$m\,r^2 + \mu\,r + k = 0$$

has discriminant $\Delta = \mu^2 - 4mk$. If $\Delta < 0$, the system exhibits underdamped, decaying oscillations. In that case, the general solution is

$$u(t) = A\,e^{-\frac{\mu}{2m}t}\,\cos\Big(\frac{\sqrt{-\Delta}}{2m}\,t + \phi_0\Big), \tag{3.5.3}$$

where the constants A and ϕ_0 are determined by the initial conditions $u(0) = u_0$ and $u'(0) = v_0$.

In the following we shall learn the solution of the equation (3.5.2) on $[0, T]$ using a neural network $f_\theta : [0, T] \to \mathbb{R}$, using the initial conditions $u(0) = u_0$, $u'(0) = v_0$ and some given data measurements $\{(t_i, u_i)\}_{i=1}^N$.

The network's loss is a weighted sum of three terms:

1. Initial-Condition Loss:

$$\mathcal{L}_{\mathrm{IC}}(\theta) = \big[f_\theta(0) - u_0\big]^2 + \big[f'_\theta(0) - v_0\big]^2.$$

2. Data Loss:

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\big[f_\theta(t_i) - u_i\big]^2.$$

3. Physics Loss: At collocation points $\{s_j\}_{j=1}^{M}$, we enforce the ODE residual

$$R(s_j;\theta) = m\, f_\theta''(s_j) + \mu\, f_\theta'(s_j) + k\, f_\theta(s_j),$$

then we define the physics-loss

$$\mathcal{L}_{\text{Phy}}(\theta) = \frac{1}{M}\sum_{j=1}^{M}\big[R(s_j;\theta)\big]^2.$$

Finally, the total loss combines the aforementioned losses

$$\mathcal{L}(\theta) = w_{\text{IC}}\,\mathcal{L}_{\text{IC}} \;+\; w_{\text{data}}\,\mathcal{L}_{\text{data}} \;+\; w_{\text{PDE}}\,\mathcal{L}_{\text{Phy}},$$

with positive weights $w_{\text{IC}}, w_{\text{data}}, w_{\text{PDE}} > 0$. Minimizing $\mathcal{L}(\theta)$ yields $f_{\theta^*}(t)$ that satisfies the initial conditions, matches the data, and enforces the governing Physics simultaneously.

Embedding the Initial Conditions via a Trial-Solution Instead of enforcing $u(0) = u_0$ and $u'(0) = v_0$ through explicit loss terms, we build them directly into our solution

$$u(t) = \; u_\theta(t) \;=\; u_0 \;+\; t\,v_0 \;+\; \tfrac{1}{2}\,t^2\,f_\theta(t),$$

where $f_\theta(t)$ is a neural network.

In this case, the *Data Loss* becomes

$$\mathcal{L}_{\text{data}}(\theta) = \; \frac{1}{N}\sum_{i=1}^{N}\big[u(t_i) - u_i\big]^2 \;=\; \frac{1}{N}\sum_{i=1}^{N}\Big[u_0 + t_i\,v_0 + \tfrac{1}{2}\,t_i^2\,f_\theta(t_i) - u_i\Big]^2.$$

To write the *Physics Loss* for the second-order ODE (3.5.2) we compute first

$$\begin{aligned} u'(t) &= v_0 + t f_\theta(t) + \frac{1}{2}t^2 f_\theta(t) \\ u''(t) &= f_\theta(t) + 2t\, f_\theta'(t) + \tfrac{1}{2}\,t^2\, f_\theta''(t). \end{aligned}$$

At collocation points $\{s_j\}$, the residual is defined by

$$\begin{aligned} R(s_j;\theta) = m\Big[f_\theta(s_j) + 2s_j f_\theta'(s_j) + \tfrac{1}{2}\,s_j^2 f_\theta''(s_j)\Big] + \mu\Big[v_0 + s_j f_\theta(s_j) + \tfrac{1}{2}\,s_j^2 f_\theta'(s_j)\Big] \\ + k\Big[u_0 + s_j v_0 + \tfrac{1}{2}\,s_j^2 f_\theta(s_j)\Big]. \end{aligned}$$

Hence, the physics-informed loss becomes

$$\mathcal{L}_{\text{Phy}}(\theta) = \; \frac{1}{M}\sum_{j=1}^{M}\big[R(s_j;\theta)\big]^2.$$

Since the initial conditions are built in, we only combine data and physics losses:

$$\mathcal{L}(\theta) = w_{\text{data}}\,\mathcal{L}_{\text{data}} \;+\; w_{\text{Phy}}\,\mathcal{L}_{\text{Phy}}.$$

Minimizing this objective trains $f_\theta(t)$ so that $u(t)$ both fits the measurements and satisfies the ODE, without any separate boundary-condition terms.

Embedding the Data Measurements via the Lagrange Polynomial We assume we are given N measurements $\{(t_i, u_i)\}_{i=1}^N$ but not the initial conditions. The Lagrange interpolating polynomial through the points $\{(t_i, u_i)\}_{i=1}^N$ can be written as

$$P(t) = \sum_{i=1}^{N} u_i \prod_{j\neq i} \frac{t - t_j}{t_i - t_j},$$

so that $P(t_i) = u_i$ exactly.

We then assume the full solution as

$$u(t) = P(t) + f_\theta(t),$$

where $f_\theta(t)$ is a neural network correction. This guarantees data interpolation provided $f_\theta(t_i) = 0$.

Enforcing $u(t_i) = u_i$, the *Data Loss* becomes

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\big[P(t_i) + f_\theta(t_i) - u_i\big]^2 = \frac{1}{N}\sum_{i=1}^{N}\big[f_\theta(t_i)\big]^2.$$

To write the *Physics Loss* we need

$$u'(t) = P'(t) + f'_\theta(t), \quad u''(t) = P''(t) + f''_\theta(t).$$

At collocation points $\{s_j\}$, define the residual

$$R(s_j;\theta) = m\big[P''(s_j) + f''_\theta(s_j)\big] + \mu\big[P'(s_j) + f'_\theta(s_j)\big] + k\big[P(s_j) + f_\theta(s_j)\big],$$

and form

$$\mathcal{L}_{\text{Phy}}(\theta) = \frac{1}{M}\sum_{j=1}^{M}\big[R(s_j;\theta)\big]^2.$$

To obtain the *Total Loss*, we combine the aforementioned losses

$$\mathcal{L}(\theta) = w_{\text{data}}\,\mathcal{L}_{\text{data}} \;+\; w_{\text{Phy}}\,\mathcal{L}_{\text{Phy}},$$

with positive weights $w_{\text{data}}, w_{\text{Phy}}$. Minimizing $\mathcal{L}(\theta)$ yields $f_{\theta^*}(t)$ such that $u(t) = P(t) + f_{\theta^*}(t)$ both interpolates the data and satisfies the ODE.

Remark 3.5.1 Even in this simple constant-coefficient example, where a closed-form solution is available, the neural-network approach remains valuable because it extends to far more complex scenarios, such as:

- Time-dependent parameters $m(t)$, $\mu(t)$, and $k(t)$.
- Non-constant external forcing terms.
- Higher-order or fully nonlinear differential equations.
- Irregular domains and complex boundary or initial conditions.

3.6 Practical Implementation

The following implementation code can be found at:

`https://tinyurl.com/2a73btnz`

In this section we shall consider the concrete case of the damped spring-mass system described by the ODE

$$u''(t) + 0.2u'(t) + 8u(t) = 0.$$

We generate sequentially $N \in \{6, 10, 40\}$ sample points in the interval $[0, 10]$ picked from the particular exact solution $u_p(t) = e^{-0.1t}\cos(\frac{\sqrt{31.96}}{2}t)$. The graph looks like in Fig. 3.3.

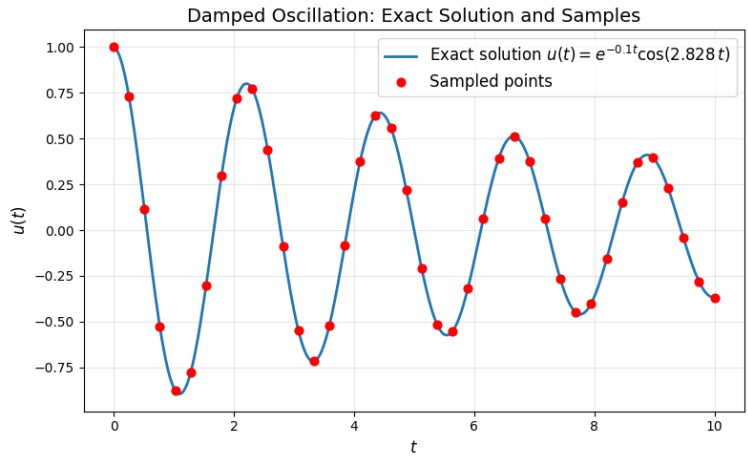


Figure 3.3: The graph of the particular solution $u_p(t) = e^{-0.1t}\cos(\frac{\sqrt{31.96}}{2}t)$ and 40 sample points.

We start by generating $N = 10$ equidistant data measurements $\{(t_i, u_i)\}_{i=1}^{10}$, from which we aim to recover the solution via a neural network. To this end, we employ a feed-forward network with three hidden layers:

```
model = Sequential([
    Input(shape=(1,)),
    Dense(32, activation='tanh'),
    Dense(32, activation='tanh'),
    Dense(32, activation='tanh'),
    Dense(1, activation='linear'),
])
```

and train it for 2,000 epochs. The training sample points are constructed by

```
def u_exact(t):
    alpha = 0.1                    # damping = 0.2/2
    omega = np.sqrt(31.96) / 2     # 2.828
    return np.exp(-alpha * t) * np.cos(omega * t)

t_samples = np.linspace(0, 10, 10).reshape(-1,1).astype(np.float32)
u_samples = u_exact(t_samples).astype(np.float32)
```

After training we do not obtain a satisfactory solution, as it does not pass through the given data points, see Fig. 3.4(a).

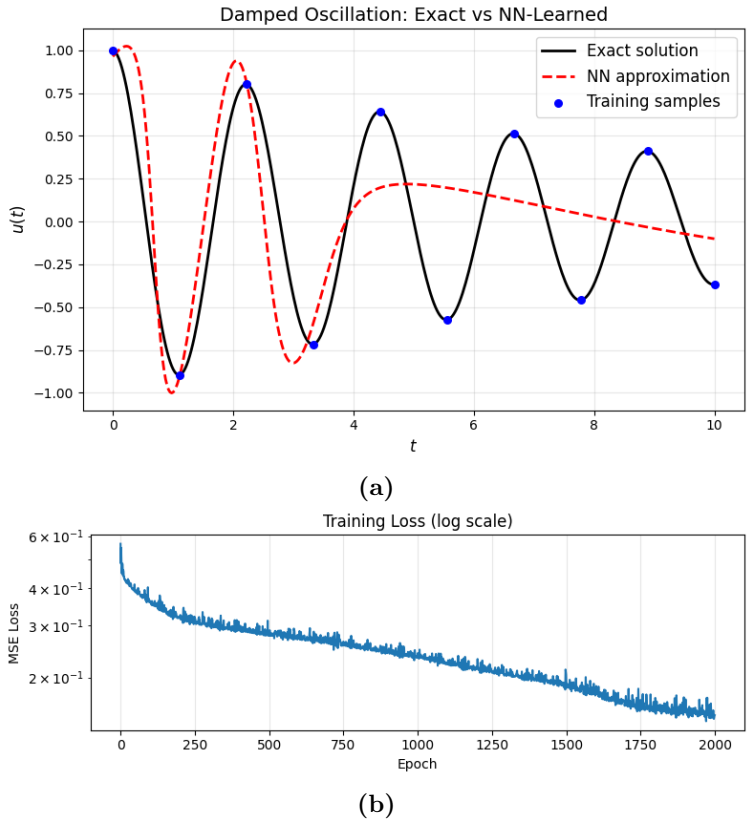


Figure 3.4: (a) The solution generated by a neural network on 10 sample points. (b) The associated loss function.

The failure reason is that the 10 sample points are picked exactly at the peaks and valleys of the wave. This choice does not provide enough information about the behavior of the wave between these points. To fix the problem, we shall increase the number of sample points to $N = 40$, expand the training to 6,000 epochs, and also add some L^2-regularization:

```
lambda_reg = 1e-4

model = Sequential([
    Input(shape=(1,)),
    Dense(32, activation='tanh',
          kernel_regularizer=l2(lambda_reg)),
    Dense(32, activation='tanh',
          kernel_regularizer=l2(lambda_reg)),
    Dense(32, activation='tanh',
          kernel_regularizer=l2(lambda_reg)),
    Dense(1, activation='linear',
          kernel_regularizer=l2(lambda_reg)),
])
```

Consequently, we obtain a very satisfactory solution, as shown in Fig. 3.5(a). The key to this success is the richer information provided by using a larger set of sample points.

In Fig. 3.5(b) we observe high-frequency oscillations in the loss, which is an indicative of instability. The choice of the L^2-regularization parameter λ_{reg} strongly affects this behavior as follows:

- A smaller λ_{reg} yields a more jagged loss curve with a lower asymptotic value, allowing the network to fit more closely but with increased risk of overfitting.
- A larger λ_{reg} produces a smoother loss curve that plateaus at a higher value, enforcing stronger weight decay and resulting in a more robust, but potentially under-fitting model.

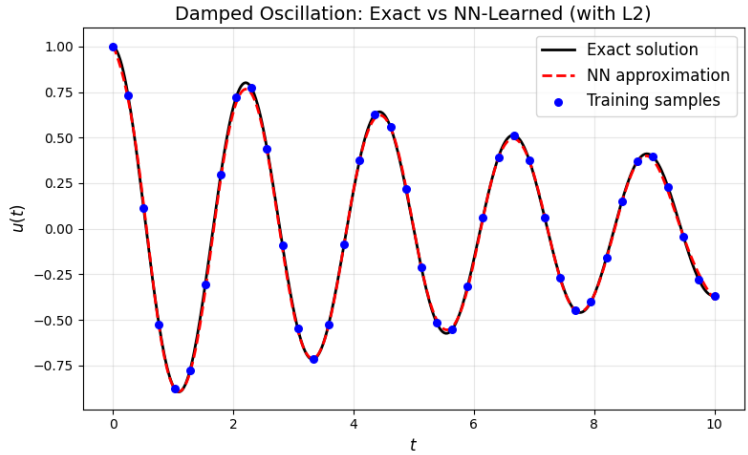
However, in practice, measurements may be sparse either because the system is poorly observed or because each measurement is costly. To show this, in the next example we use only $N = 6$ data points and train the same neural network architecture to interpolate them, as shown in Fig. 3.6(a).

While the network curve does pass exactly through these six points, it fails to follow the true solution between them. To remedy this, we add a physics-informed loss, enforcing the governing ODE at 500 collocation points, while keeping the network architecture fixed and training it for $6,000$ epochs. Although the resulting PINN does hit some of the data and begins to oscillate between points, however, it ultimately degrades, see Fig. 3.7(a). It is worth noting that we chose a smaller weight on the physics-loss than on the data loss to avoid overwhelming the network when only a few data points are available.

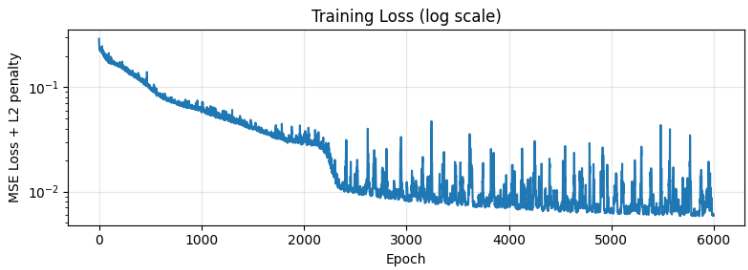
3.7 Newton's Equation

By the principle of Newtonian determinism, the trajectory $x(t)$ of a mechanical system is uniquely determined by its initial position $x(0) = x_0 \in \mathbb{R}^n$ and initial velocity $\dot{x}(0) = v_0 \in \mathbb{R}^n$. Consequently, there exists a force field

$$F : \mathbb{R}^n \times \mathbb{R}^n \times \mathbb{R} \longrightarrow \mathbb{R}^n$$

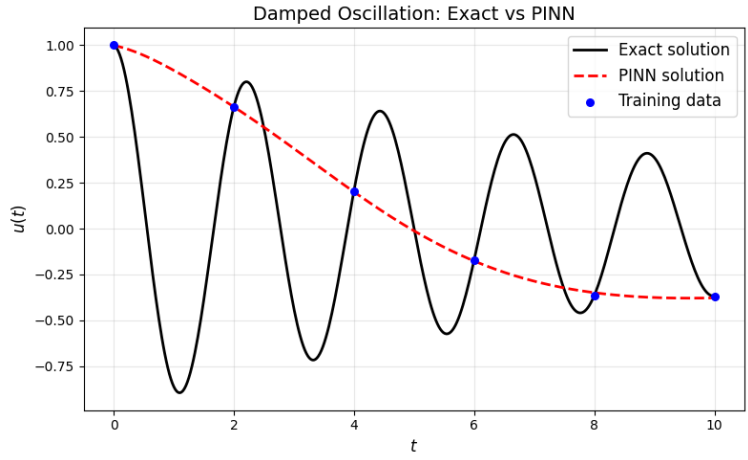


(a)

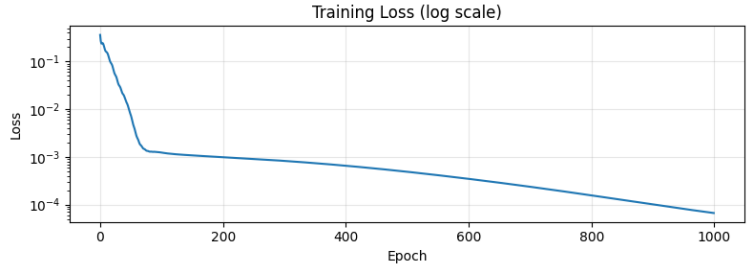


(b)

Figure 3.5: (a) The solution generated by a neural network on 40 sample points is satisfactory. (b) The associated loss function exhibits noise instability, which can be controlled by the L^2-regularization parameter.

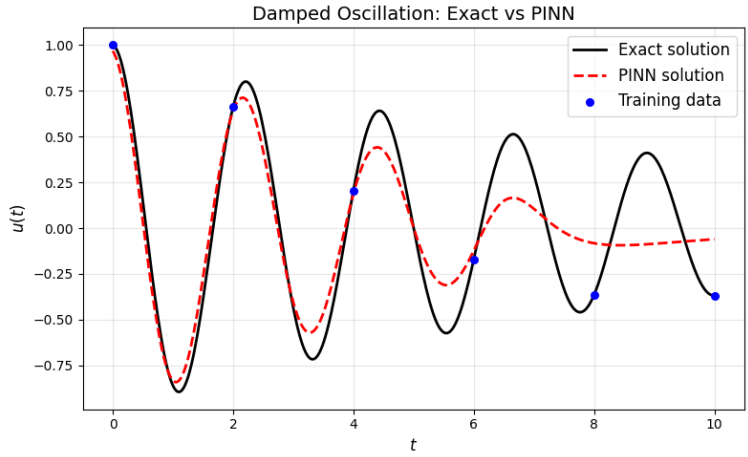


(a)

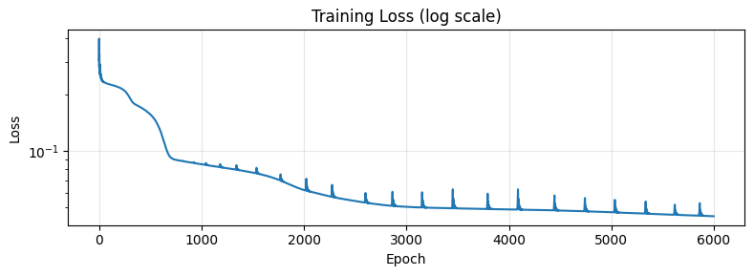


(b)

Figure 3.6: (a) The solution generated by a neural network on 6 sample points is not satisfactory. (b) The associated loss function is decreasing to a magnitude order less than 10^{-4}.



(a)



(b)

Figure 3.7: (a) The solution generated by a neural network on 6 sample points using a PINN architecture becomes satisfactory. (b) The associated loss function is decreasing with sparse oscillations.

such that

$$\ddot{x}(t) \;=\; F\big(x(t),\, \dot{x}(t),\, t\big). \tag{3.7.4}$$

This is Newton's equation, whose precise form depends on the system under study. By the classical existence and uniqueness theorem, (3.7.4) admits a locally unique solution, so the initial-value problem is well-posed (at least for a short time).

From a computational standpoint, this is a *direct problem*: given the force field F and the initial data (x_0, v_0), we seek the trajectory $x(t)$. The second-order nature of equation (3.7.4) makes it a perfect candidate for the neural-network ODE solvers described in Section 2.5.2. In the accompanying Keras implementation, both the first and second derivatives of the network's output were obtained via automatic differentiation.

3.7.1 Coordinates Change

Besides the classical methods presented in Section 2.5.2, we now introduce an alternative approach based on a change of variables. Let $y = G(x)$, with components

$$y^j = G^j(x_1, \ldots, x_n), \tag{3.7.5}$$

and inverse $x = H(y) = G^{-1}(y)$. Physically, this corresponds to moving to a new reference frame in which the force field simplifies. In the y-coordinates, Newton's equation

$$\ddot{x} \;=\; F\big(x, \dot{x}\big)$$

transforms into a new form

$$\ddot{y} \;=\; \Phi\big(y, \dot{y}\big).$$

If Φ is one of the classical solvable families listed below, we obtain an explicit solution $y(t)$. Finally, returning via $x(t) = H\big(y(t)\big)$ yields the desired trajectory in the original coordinates.

The force fields F in the x-coordinates and Φ in the y-coordinates are related via the chain rule. Since

$$\dot{x} = H'(y)\,\dot{y}, \quad \ddot{x} = H''(y)\,\dot{y}^2 \;+\; H'(y)\,\ddot{y},$$

substituting into $\ddot{x} = F\big(x, \dot{x}\big)$ and solving for $\ddot{y}$ gives

$$\ddot{y} \;=\; \Phi\big(y, \dot{y}\big) \;=\; \frac{F\big(H(y),\, H'(y)\,\dot{y}\big) \;-\; H''(y)\,\dot{y}^2}{H'(y)}. \tag{3.7.6}$$

Even though there is no single theorem characterizing all Φ for which the autonomous system $\ddot{y} = \Phi(y, \dot{y})$ admits a closed-form solution, one finds that several well-known families of Φ become reducible after an appropriate change of variables, as follows:

- *Linear equation:* $\Phi(y, \dot{y}) = a\dot{y} + by + c$
- *Clairaut-type equation:* $\Phi(y, \dot{y}) = \alpha y\dot{y} + \beta$
- *Homogeneous equation:* $\Phi(y, \dot{y}) = \Psi\Big(\frac{\dot{y}}{y}\Big)$

- *y-dependent:* $\Phi(y, \dot{y}) = \Phi(y)$
- *$\dot{y}$-dependent:* $\Phi(y, \dot{y}) = \Phi(\dot{y})$.

We shall denote by $\mathcal{A}$ the class of all functions Φ for which the second-order ODE

$$\ddot{y} = \Phi\big(y, \dot{y}\big)$$

admits a closed-form solution.

The task is then: *Find a change of variables $y = G(x)$ (as in (3.7.5)) such that the transformed force field Φ in (3.7.6) belongs to the solvable class $\mathcal{A}$.*

We propose to address this search for G using neural-networks optimization. To this end, consider $y = G_\theta(x)$ be the output of a feed-forward neural network, with equal input and output dimensions. Besides, we assume there is an inverse to this network, given by $x = H_\theta(y)$, fact that implies that the hidden layers have all the same width. We denote

$$\Phi_\theta\big(y, \dot{y}\big) \;=\; \frac{F\big(H_\theta(y),\, H'_\theta(y)\,\dot{y}\big) \;-\; H''_\theta(y)\,\dot{y}^2}{H'_\theta(y)}.$$

We define the loss function as the L^2-distance between the possible network outputs and the distinguished set $\mathcal{A}$ by

$$\mathcal{L}(\theta) = \inf_{\psi \in \mathcal{A}} \|\Phi_\theta - \psi\|_2.$$

And pick the network parameters for which this distance attains its minimum

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta).$$

By construction, the transformed equation $\ddot{y} = \Phi_{\theta^*}(y, \dot{y})$ belongs (or is closest) to the class $\mathcal{A}$, and hence can be solved explicitly by the known closed-form methods.

This rather theoretical approach nevertheless yields an interesting consequence when the original force field F coincides with its transformed version Φ. In that case, the change of variables is a *symmetry* of the ODE, meaning the equation $\ddot{x} = F(x, \dot{x})$ retains its form under the transformation $y = G(x)$. We shall expand this idea in the next section.

3.7.2 Galilean Transforms

We are interested in the invertible transforms $G : \mathbb{R}^n \to \mathbb{R}^n$ which leaves the equations of motion invariant, namely

$$\frac{d^2}{dt^2}(G(x(t)) = F\big(G(x(t)), \frac{d}{dt}G(x(t)\big).$$

Let $y = G(x)$, $H = G^{-1}$, and $\dot{y} = w$. Using relation (3.7.6), the condition $\Phi(y, \dot{y}) = F(y, \dot{y})$ becomes

$$\frac{F\big(H(y), H'(y)\,w\big) \;-\; H''(y)\,w^2}{H'(y)} \;=\; F(y, w).$$

To find the transformation $x = H(y)$ that satisfies this relation, we must solve the functional equation

$$F\big(H(y), H'(y)\,w\big) \;-\; H''(y)\,w^2 \;=\; H'(y)\,F(y,w). \tag{3.7.7}$$

We can approximate H by a neural network H_θ, which takes y as input and then minimize the loss

$$\mathcal{L}(\theta) \;=\; \iint \Big[F\big(H_\theta(y), H'_\theta(y)\,w\big) \;-\; H''_\theta(y)\,w^2 \;-\; H'_\theta(y)\,F(y,w)\Big]^2\,dy\,dw.$$

The previous integral holds on a bounded domain in the phase space $(y, \dot{y}) \in \mathbb{R}^n \times \mathbb{R}^n$.

In general, the functional equation (3.7.7) does not have a unique solution; moreover, its set of solutions forms a group of transformations. To single out one solution, additional constraints, such as an initial conditions $H(y_0) = H^0$ and $H'(y_0) = H^1$ must be imposed. This can be enforced in the loss via the extra-terms

$$\mathcal{L}_{\mathrm{IC}}(\theta) = \big(H_\theta(y_0) - H^0\big)^2 + \big(H'_\theta(y_0) - H^1\big)^2.$$

In many cases, the force derives from a potential, $F(x) = -\nabla U(x)$, so Newton's equation reads $\ddot{x} = F(x)$. In this situation, $F(x,w) = F(x)$, and equation (3.7.7) becomes, after separating variables,

$$F\big(H(y)\big) \;-\; H'(y)\,F(y) \;=\; H''(y)\,w^2.$$

Since the left-hand side is independent of w, we must have $H''(y) = 0$. Hence

$$H(y) = a\,y + b, \quad a, b \in \mathbb{R}.$$

Substituting back yields

$$F(a\,y + b) = a\,F(y), \quad \forall\, y \in \mathbb{R}.$$

There are two cases:
(*i*) *F is constant.* Let $F(y) = c$. Then $a = 1$ (while b remains arbitrary), and Newton's equation becomes $\ddot{x} = c$, with invariant transformations given by translations

$$x = H(y) = y + b, \qquad b \in \mathbb{R}.$$

(*ii*) *F is nonconstant.* In this case $b = 0$, and the relation reduces to

$$F(a\,y) = a\,F(y), \quad \forall\, a, y \in \mathbb{R}.$$

Setting $y = 1$ shows $F(a) = k\,a$, where $k = F(1)$. Thus $F(y) = k\,y$, and Newton's equation becomes $\ddot{x} = k\,x$. In this case the invariant transformations are homotheties

$$x = H(y) = a\,y, \qquad a \in \mathbb{R}.$$

We shall now consider several concrete physical problems governed by Newton's equation.

3.7.3 Free-falling from High Altitude

According to the experimentally established inverse-square law of Newtonian gravity, the radial acceleration of a test mass above the Earth's surface satisfies

$$x''(t) \;=\; -\,g\,\frac{r_0^2}{\big(r_0+x(t)\big)^2}, \tag{3.7.8}$$

where

$$r(t) = r_0 + x(t)$$

is the distance from Earth's center, r_0 is the Earth's radius, and g is the gravitational acceleration at the surface. We assume the initial conditions $x(0) = x_0$ and $x'(0) = v_0$. This corresponds to the Newtonian force

$$F(x) = -\frac{k}{(r_0+x)^2},$$

with $k = gr_0^2$. This force is derived from a potential, $F(x) = -\dfrac{\partial U(x)}{\partial x}$, with the potential function $U(x) = -\dfrac{k}{x+r_0}$. The total energy of the system is the sum between the kinetic and potential energies

$$E = \frac{1}{2}x'(t)^2 + U(x) = \frac{1}{2}x'(t)^2 - \frac{k}{x+r_0}, \tag{3.7.9}$$

and is conserved along the solution $x(t)$. Enforcing the initial data gives the value of the total energy constant

$$E \;=\; \tfrac{1}{2}\,v_0^2 \;-\; g\,\frac{r_0^2}{r_0+x_0}.$$

One may try to solve for the solution $x(t)$ in an explicit way. To this end, solving for $x'(t)$ yields the separable first-order equation

$$x'(t) = \pm\sqrt{v_0^2 - 2g\,\frac{r_0^2}{r_0+x_0} + 2g\,\frac{r_0^2}{r_0+x(t)}}.$$

Separating variables and integrating from x_0 to $x(t)$ gives the exact solution in implicit form:

$$\int_{x_0}^{x(t)} \frac{d\xi}{\sqrt{v_0^2 - 2g\,\dfrac{r_0^2}{r_0+x_0} + 2g\,\dfrac{r_0^2}{r_0+\xi}}} \;=\; t.$$

However, no simpler elementary closed-form for the trajectory $x(t)$ generally exists.[1]

[1]The reason is that the integral $\displaystyle\int \frac{dx}{\sqrt{A+\frac{B}{x}}}$ with A and B constants cannot be solved explicitly using elementary functions.

From this point, we shall use Deep Learning to find the solution. To this end, we shall look for a solution of the initial-condition equation (3.7.8) in the form

$$x_\theta(t) = x_0 + v_0 t + \frac{1}{2} t^2 f_\theta(t),$$

with $f_\theta(t)$ feed-forward neural network with input t. This idea follows the method introduced in Section 2.5.2 at page 56, so we skip the details. The reader can find the detailed code at the Colab page

```
https://tinyurl.com/2vkt65uz
```

Using the feed-forward network

```
model = Sequential([
    Input(shape=(1,)),
    Dense(62, activation='tanh'),
    Dense(32, activation='tanh'),
    Dense(16, activation='tanh'),
    Dense(1, activation='linear'),
])
```

and the optimizer

```
optimizer = Adam(learning_rate=1e-3)
```

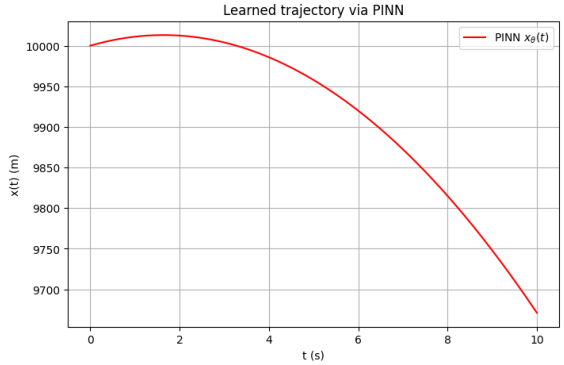
we obtain after $1,000$ epochs the trajectory depicted in Fig. 3.8(a). In the beginning this trajectory tends to point upward due to the effect of initial velocity, and then decreases.

It is interesting to note the shape of the network correction term $f_\theta(t)$ present in the $O(t^2)$-term of the solution $x_\theta(t)$, see Fig. 3.8(b). This correction term is negative, bounded, $-9.78 < f_\theta(t) < -9.64$, and decreases with a threshold for large t. This suggests that the solution is quadratic for large t, namely $x_\theta(t) \approx x_0 + v_0 t + \frac{1}{2} t^2 c$, with $c = \lim_{t\to\infty} f_\theta(t)$.

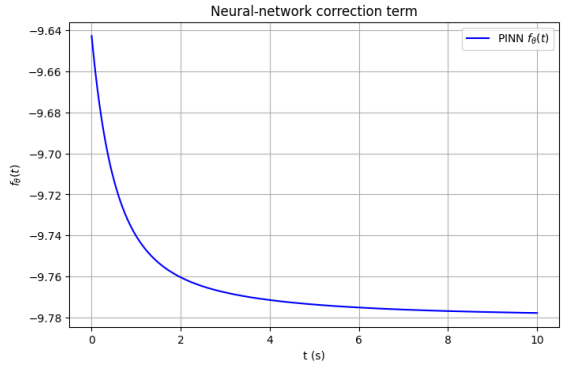
An even interesting remark can be applied to the loss evolution versus number of epochs, see Fig. 3.8(c). We note that the loss decreases in two regimes. The first is a steady pace during the first 750 epochs, and the later corresponds to a slower decrease regime. We note that even after $4,000$ epochs the loss has not yet reached its saturation threshold.

It is worth noting that the equation (3.7.8) contains more information than the conservation law (3.7.9). This is because the conservation law only fixes $|x'(t)|$, but not its sign. Therefore, the network can learn a solution that "oscillates" its velocity around zero while still satisfying the energy balance in a mean-square sense. Moreover, when you only minimize $[\frac{1}{2}x'(t)^2 + U(x(t)) - E]^2$, small errors in $x'(t)$ and $x(t)$ can trade off against each other and still keep the residual small, leading to "spurious" curves. Therefore, replacing the ODE loss

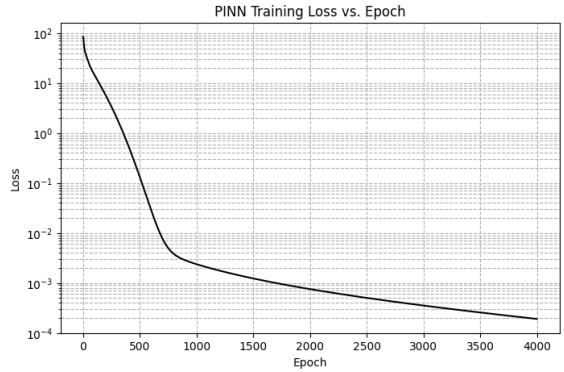
$$\mathcal{L}_{\text{ODE}}(\theta) = \sum_{k=1}^{N} [f''_\theta(t_k) + g\, \frac{r_0^2}{\big(r_0 + f_\theta(t_k)\big)^2}]^2,$$



(a)



(b)



(c)

Figure 3.8: (a) The solution $x(t)$ to the inverse-square law equation (3.7.8) subject to the initial conditions $x_0 = 10,000$ (meters) and $v_0 = 16$ (meters/second). (b) The neural network correction term $f_\theta(t)$ present in the ansatz solution $x_\theta(t) = x_0 + v_0 t + \frac{1}{2}t^2 f_\theta(t)$. (c) The corresponding total loss.

by the conservation loss

$$\mathcal{L}_{\text{cons}}(\theta) = \sum_{k=1}^{N} \Big(\frac{1}{2} f'_\theta(t_k) + U(f_\theta(t_k)) - E \Big)^2$$

does not lead in general to the correct trajectory $x(t)$, with exception of some very simple equations.

However, the conservation loss is still useful as it can be used as a regularization term. Thus, the total loss in this case becomes

$$\mathcal{L}_{\text{tot}} = \mathcal{L}_{\text{ODE}}(\theta) + \mathcal{L}_{\text{IC}}(\theta) + \lambda \mathcal{L}_{\text{cons}}(\theta),$$

where the initial condition loss is given by

$$\mathcal{L}_{\text{IC}}(\theta) = \|f_\theta(0) - x_0\|^2 + \|f'_\theta(0) - v_0\|^2.$$

The hyperparameter $\lambda > 0$ should be chosen small enough such that the conservation loss does not introduce any oscillations.

Comparison with traditional numerical methods Since there is no closed-form solution, in order to verify the correctness of the solution given by the neural network, we shall employ a numerical integration via Runge–Kutta method and then compare the results.

We attempt first a reduction to a first-order system by introducing the state vector

$$y(t) = \begin{pmatrix} x(t) \\ x'(t) \end{pmatrix},$$

so that

$$y'(t) = \begin{pmatrix} x'(t) \\ -g \, \dfrac{r_0^2}{\big(r_0 + x(t)\big)^2} \end{pmatrix}.$$

We then implement this system in Python as

```
def ode_system(t, y):
    x, xdot = y
    xdd    = -g * r0**2 / (r0 + x)**2
    return [xdot, xdd]
```

To perform the integration, we call SciPy's `solve_ivp` with the explicit Runge-Kutta method (RK45), specifying tight tolerances for accuracy:

```
sol = solve_ivp(
    ode_system,
    (0.0, 10.0),
    y0=[x0, v0],
    t_eval=np.linspace(0,10,500),
```

```
    method='RK45',
    atol=1e-8,
    rtol=1e-8
)
```

Here `t_eval` is an array of 500 equally-spaced times in $[0, 10]$. The solver returns `sol.t` and `sol.y[0]`, which contain the numerical approximations for t and $x(t)$, respectively.

Finally, we plot the numerical solution $x_{\text{num}}(t)$ against t:

```
plt.figure(figsize=(8,5))
plt.plot(sol.t, sol.y[0], 'b-', linewidth=2, label='RK45 solution')
plt.xlabel('t (s)')
plt.ylabel('x(t) (m)')
plt.title('Numerical Solution of $x\'\'=-g\,r_0^2/(r_0+x)^2$')
plt.legend()
plt.grid(True)
plt.show()
```

The resulting curve is smooth, monotonically decreasing and convex, as expected, see Fig. 3.9. We can now draw the following conclusions from these two approaches:

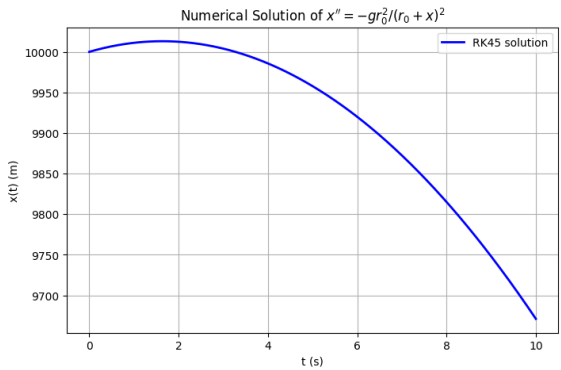


Figure 3.9: The solution $x(t)$ obtained via a Runge-Kutta method (RK45) to the inverse-square law equation.

Both methods recover the same trajectory $x(t)$. However, the Runge-Kutta solver is substantially faster than the neural-network-based method. This illustrates that, for direct-forward problems like this one, classical numerical integrators (e.g. Runge-Kutta) are far more efficient. Neural-network approaches, on the other hand, offer greater flexibility when tackling inverse or data-driven problems, which we will explore later in the book.

3.7.4 Damped Spring-Mass System

We consider the damped spring-mass system on $\mathbb{R}$ given by the ordinary differential equation

$$m u''(t) + \mu\, u'(t) + k\, u(t) \;=\; 0,$$

where $\mu > 0$ is the damping coefficient and $k > 0$ is the spring constant. The mass m can be taken equal to 1, since it can be absorbed into μ and k. In the underdamped case $\mu^2 < 4mk$, the homogeneous solution oscillates with exponential decay.

To enforce both initial conditions

$$u(0) = u_0, \quad u'(0) = v_0,$$

we introduce the *trial-solution ansatz*

$$u_\theta(t) \;=\; u_0 \;+\; v_0\, t \;+\; \tfrac{1}{2}\, t^2\, f_\theta(t),$$

where $f_\theta(t)$ is a feed-forward neural network with parameters θ and input $t \in \mathbb{R}$. By construction, the initial conditions are automatically satisfied

$$u_\theta(0) = u_0, \quad u_\theta'(0) = v_0.$$

Next, we select collocation points $\{t_i\}_{i=1}^N$ in the time interval of interest and define the *physics-informed loss*

$$\mathcal{L}_{\mathrm{ODE}}(\theta) \;=\; \frac{1}{N} \sum_{i=1}^{N} \Big[u_\theta''(t_i) + \mu\, u_\theta'(t_i) + k\, u_\theta(t_i) \Big]^2.$$

Here each residual

$$R(t_i; \theta) := u_\theta''(t_i) + \mu\, u_\theta'(t_i) + k\, u_\theta(t_i)$$

measures the violation of the ODE at t_i. We train θ by minimizing $\mathcal{L}_{\mathrm{ODE}}(\theta)$, using automatic differentiation to obtain the derivatives u_θ' and u_θ''. At convergence, the network-corrected solution $u_\theta(t)$ satisfies

$$u_\theta''(t) + \mu\, u_\theta'(t) + k\, u_\theta(t) \approx 0 \quad \text{for all sampled } t,$$

and thus approximates the true damped oscillatory response. Implementation details for this case can be found in Section 3.5 at page 130.

3.7.5 The Two-Body Problem

The Newtonian two-body problem describes the motion of two point masses m_1 and m_2, interacting only via their mutual gravitational attraction. Let their position vectors in an inertial frame in $\mathbb{R}^3$ be $\mathbf{r}_1(t) = (x_1(t), y_1(t)), z_1(t))$ and $\mathbf{r}_2(t) = (x_2(t), y_2(t)), z_2(t))$. Newton's law of gravitation together with the second law of dynamics provides the following ODE system

$$m_1\, \ddot{\mathbf{r}}_1 = -G\, \frac{m_1\, m_2}{\|\mathbf{r}_1 - \mathbf{r}_2\|^3}\, (\mathbf{r}_1 - \mathbf{r}_2)\,, \tag{3.7.10}$$

$$m_2\, \ddot{\mathbf{r}}_2 = -G\, \frac{m_1\, m_2}{\|\mathbf{r}_2 - \mathbf{r}_1\|^3}\, (\mathbf{r}_2 - \mathbf{r}_1)\,. \tag{3.7.11}$$

This system comes with some initial-value conditions given by $\mathbf{r}_1(0) = \mathbf{r}_1^0$, $\mathbf{r}_1'(0) = \mathbf{v}_1^0$, $\mathbf{r}_2(0) = \mathbf{r}_2^0$, and $\mathbf{r}_2'(0) = \mathbf{v}_2^0$.

To simplify the system integration it is customary to employ a center-of-mass reduction method as follows. Define the total mass of the system, $M = m_1 + m_2$, the center-of-mass coordinate

$$\mathbf{R}(t) \;=\; \frac{m_1\,\mathbf{r}_1 + m_2\,\mathbf{r}_2}{M}$$

and the relative separation vector, $\mathbf{r}(t) \;=\; \mathbf{r}_1 - \mathbf{r}_2$. Adding equations (3.7.10) and (3.7.11), it is straightforward to get

$$M\,\ddot{\mathbf{R}} = m_1\,\ddot{\mathbf{r}}_1 + m_2\,\ddot{\mathbf{r}}_2 = \mathbf{0},$$

so $\mathbf{R}(t)$ moves with constant velocity and may be set to zero by choice of origin. Meanwhile, subtraction of (3.7.11) from (3.7.10) yields the single reduced-mass equation

$$\mu\,\ddot{\mathbf{r}}(t) \;=\; -\,G\,\frac{m_1\,m_2}{r^3}\,\mathbf{r}(t), \tag{3.7.12}$$

where the reduced mass is $\mu = \dfrac{m_1\,m_2}{m_1+m_2}$ and $r = \|\mathbf{r}\| = \|\mathbf{r_1} - \mathbf{r_2}\|$ denotes the distance between the bodies.

There are two conservation laws for the above equation.

(i) Because the force is central (i.e. $\ddot{\mathbf{r}}$ points along $\mathbf{r}$) we have that $\mathbf{r}$ and $\ddot{\mathbf{r}}$ are collinear, so their vectorial product vanishes, $\mathbf{r} \times \ddot{\mathbf{r}} = 0$. Consequently, the rate of the angular momentum is

$$\frac{d}{dt}\mathbf{L} = \mu\,\frac{d}{dt}\big(\mathbf{r} \times \dot{\mathbf{r}}\big) = \mu\dot{\mathbf{r}} \times \dot{\mathbf{r}} + \mu\mathbf{r} \times \ddot{\mathbf{r}} = 0.$$

Therefore, the angular momentum $\mathbf{L} = \mu\,\mathbf{r} \times \dot{\mathbf{r}}$ is constant in time, so motion lies in a fixed plane.

(ii) The total energy:

$$E = \tfrac{1}{2}\,\mu\,\|\dot{\mathbf{r}}\|^2 \;-\; \frac{G\,m_1\,m_2}{r}$$

is constant. This follows from taking the derivative $\frac{d}{dt}E$ and using equation (3.7.12) to show that this vanishes.

In polar coordinates, $\mathbf{r} = (r, \phi)$ in the plane of motion, one obtains the classical Kepler-problem ODEs,

$$\mu\left(\ddot{r} - r\,\dot{\phi}^2\right) = -\,\frac{G\,m_1\,m_2}{r^2}, \qquad \mu\left(r^2\dot{\phi}\right) = L = \|\mathbf{L}\|. \tag{3.7.13}$$

It is known that these equations integrate to conic-section orbits

$$r(\theta) = \frac{p}{1 + e\cos(\phi - \phi_0)},$$

where e (eccentricity) and p are determined by E and L, see for instance Calin and Chang [17], pages 126–127.

We shall investigate next how a physics-informed neural network can learn the trajectory $\mathbf{r}(t)$. To this end, since the motion is plane, we work in polar coordinates (r, ϕ). First, we assumes the initial conditions:

$$r_\theta(0) = r_0, \quad \dot{r}_\theta(0) = v_{r,0}, \quad \phi_\eta(0) = \phi_0, \quad \dot{\phi}_\eta(0) = \frac{L}{\mu\, r_0^2}.$$

We introduce two neural networks $f_\theta(t)$ and $g_\eta(t)$ with trainable parameters θ and η to construct two trial-solutions to learn $r(t)$ and $\phi(t)$, respectively, as

$$r_\theta(t) = r_0 + t v_{r,0} + \frac{1}{2} t^2 f_\theta(t), \quad \phi_\eta(t) = \phi_0 + t \frac{L}{\mu\, r_0^2} + \frac{1}{2} t^2 g_\eta(t).$$

By automatic differentiation we compute $\dot{r}_\theta(t)$, $\ddot{r}_\theta(t)$, $\dot{\phi}_\eta(t)$. Then at collocation times $\{t_i\}$, we consider the following residuals

$$R_r(t_i) = \mu\big(\ddot{r}_\theta(t_i) - r_\theta(t_i)\, \dot{\phi}_\eta(t_i)^2\big) + \frac{G\, m_1 m_2}{r_\theta(t_i)^2},$$
$$R_\phi(t_i) = \mu\, r_\theta(t_i)^2\, \dot{\phi}_\eta(t_i) - L.$$

Then we form the total physics-informed loss

$$\mathcal{L}(\theta, \eta) = \frac{1}{N} \sum_{i=1}^{N} \Big[R_r(t_i)^2 + R_\phi(t_i)^2 \Big].$$

We minimize $\mathcal{L}(\theta, \eta)$ with a gradient-based optimizer. At convergence, the learned functions $r_\theta(t)$ and $\phi_\eta(t)$ satisfy both the radial and angular-momentum equations, reproducing the classical Keplerian orbits, without explicitly solving the conic-section formulas. The details of the code implementation can be found at:

`https://tinyurl.com/57un4rk5`

The brief idea of the code is as follows. We have considered the following physical constants

```
G = 1                          # gravitational constant
m1, m2 = 1.0, 2.0              # masses (units)
mu = m1 * m2 / (m1 + m2)       # reduced mass
L = 1.0                        # conserved angular momentum
r0 = 1.0                       # initial separation
v_r0 = 0.5                     # initial radial velocity
phi0 = 1.0                     # initial angle
```

The ODE was enforced on 100 collocation points in the interval $(0, 4)$. The network for radial correction $f_\theta(t)$ has two hidden layers as follows:

```
f_model = Sequential([
    Input(shape=(1,)),
    Dense(62, activation='tanh'),
    Dense(32, activation='tanh'),
    Dense(1, activation='linear'),
])
```

The same architecture was used for the angular correction network $g_\eta(t)$. The optimizer used is Adam with a learning rate of 0.001. After $5,000$ epochs of training we obtain the solution given by the arc of an ellipse as in Fig. 3.10(a). We note that the loss reaches its saturation level at about $4,000$ epochs, see Fig. 3.10(b).

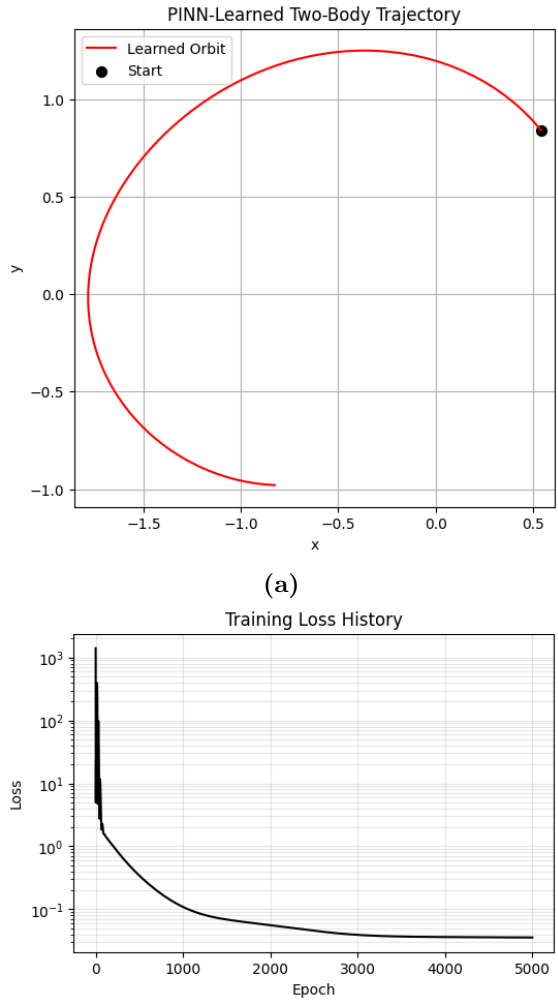


Figure 3.10: (a) The solution to the two-body problem exhibits an arc of ellipse. (b) The neural network loss decreases steadily, reaching its saturation level at about 4,000 epochs.

For the sake of comparison, the Colab file contains also a non-NN solver of exactly the same planar two-body problem, formulated in Cartesian coordinates and integrated with SciPy's `solve_ivp`. We note the difference in the speed of computing the trajectories, which is in advantage for the classical solver.

3.8 Exercises

Exercise 3.8.1 Consider the vector field in $\mathbb{R}^2$

$$F(x, y) = -y\,\partial_x + x\,\partial_y.$$

(a) Randomly sample $N = 100$ vectors from the field F to form a training dataset.

(b) Use a neural network to learn the vector field F through *hybrid learning*, combining both the training data and the corresponding differential equations.

(c) Plot, on the same coordinate system, the original vector field and the learned vector field. Also, plot the evolution of the loss function during training.

Exercise 3.8.2 Consider the planar vector field

$$F(x, y) = (x - y)\,\partial_x + (x + y)\,\partial_y.$$

(a) Find the integral curves of F by direct integration.

(b) Use a neural network to learn a family of integral curves of F, and compare the learned trajectories with the exact integral curves obtained in part (a).

Exercise 3.8.3 Consider the Newton equation for a damped spring-mass system,

$$y'' = ay' + by,$$

where a, b are constants, $y = y(t)$ represents the elongation, and the mass is assumed to be 1.

(a) Show by direct computation that the change of variable $z = e^{-\frac{a}{2}t}\,y$ transforms the equation into

$$z'' = kz,$$

where $k = k(a, b)$ is a constant to be determined.

(b) Use a neural network to verify that the coordinate transformation $z = e^{-\frac{a}{2}t}y$ simplifies the given equation to its reduced form.

Exercise 3.8.4 Consider the system

$$\ddot{x}_1 = -\frac{Mx_1}{(x_1^2 + x_2^2)^{1/2}},$$

$$\ddot{x}_2 = -\frac{Mx_2}{(x_1^2 + x_2^2)^{1/2}},$$

where $M > 0$.

(a) Construct a neural network

$$NN(\theta; x) = \big(r_\theta(x),\, \phi_\theta(x)\big)$$

that learns a change of coordinates transforming the above system into the polar form

$$\ddot{r} = r\dot{\phi}^2 - \frac{M}{r},$$
$$\ddot{\phi} = -\frac{2\dot{r}}{r}\dot{\phi}.$$

(b) Verify whether, for the trained parameters θ^*, the learned transformation satisfies

$$r^2_{\theta^*} = x_1^2 + x_2^2, \qquad \tan(\phi_{\theta^*}) = \frac{x_2}{x_1}.$$

Exercise 3.8.5 A coupled oscillation involving three springs with stiffness constants k_1, k_2, k_3 and two masses m_1, m_2 is described by Newton's equations

$$m_1\ddot{x}_1 = -(k_1 + k_2)\, x_1 + k_2\, x_2,$$
$$m_2\ddot{x}_2 = k_2\, x_1 - (k_2 + k_3)\, x_2.$$

(a) Find the exact solution in the symmetric case

$$k_1 = k_2 = k_3 = 1, \quad m_1 = m_2 = 1,$$

with initial conditions

$$x_1(0) = 1, \quad x_2(0) = 2, \quad \dot{x}_1(0) = 0, \quad \dot{x}_2(0) = 0.$$

(b) Construct a neural network

$$NN(\theta; t) = \big(x_1(\theta; t),\, x_2(\theta; t)\big)$$

that learns the system dynamics in an unsupervised manner. Incorporate the initial conditions directly into the network structure.

(c) Compare the exact solution obtained in part (a) with the learned solution from part (b).

Chapter 4

Lagrangian Systems

In Lagrangian mechanics, the evolution of a dynamical system is obtained by finding the curve $x\colon [0,1] \to \mathbb{R}^n$ that is a minimizer for the action functional

$$S[x] \;=\; \int_0^1 L\big(x(t), \dot{x}(t)\big)\, dt,$$

subject to the fixed endpoints $x(0) = x_0$, $x(1) = x_1$.

Here, $L\colon \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$ is the *Lagrangian*, a function of position x and velocity $\dot{x}$. A necessary condition for minimization is the stationarity condition given by the Euler–Lagrange equation,

$$\frac{d}{dt}\Big(\partial_{\dot{x}^j} L\big(x, \dot{x}\big)\Big) \;=\; \partial_{x^j} L\big(x, \dot{x}\big), \quad j = 1, \dots, n, \tag{4.0.1}$$

together with the boundary conditions $x(0) = x_0$, $x(1) = x_1$.

The Variational Method Rather than deriving and solving (4.0.1) directly, we can employ a single feed-forward neural network $f_\theta\colon [0,1] \to \mathbb{R}^n$ to represent the candidate trajectory and train it by minimizing a loss that approximates the action integral plus endpoint penalties. Concretely, we choose collocation times $0 = t_0 < t_1 < \cdots < t_N = 1$, and define

$$\mathcal{L}_{\text{lagr}}(\theta) \;=\; \frac{1}{N} \sum_{j=1}^{N} L\big(f_\theta(t_j),\, \dot{f}_\theta(t_j)\big),$$

$$\mathcal{L}_{\text{bd}}(\theta) \;=\; \big\| f_\theta(0) - x_0 \big\|^2 \;+\; \big\| f_\theta(1) - x_1 \big\|^2.$$

The total loss is

$$\mathcal{L}(\theta) \;=\; \mathcal{L}_{\text{lagr}}(\theta) \;+\; \mathcal{L}_{\text{bd}}(\theta).$$

By minimizing $\mathcal{L}(\theta)$ via gradient-based optimization, one obtains parameters θ^* for which $f_{\theta^*}(t)$ approximates the true solution curve of the Euler–Lagrange equations. As $N \to \infty$ and the quadrature becomes accurate, and the learned network converges to the exact minimizer of the action.

An advantage of the "action-minimization" approach over a PINN built directly from the Euler–Lagrange equations is that this works entirely at the level of the Lagrangian, without ever having to derive and implement the variational field equations themselves. On the other hand, the trade-off is the need to account for a sufficient number of collocation points $t_j \in [0,1]$ to accurately compute the action integral. Moreover, this method represents an unsupervised learning to find the solution of the Euler–Lagrange equation. The disadvantage is that in some cases, this method is outperformed by the PINN built directly from the Euler–Lagrange equations, and to fix the situation, extra measurement data is required.

We will now illustrate this action-minimization approach on several fundamental examples of Lagrangians.

4.1 The Harmonic Oscillator

The one-dimensional simple harmonic oscillator (with unit mass and spring constant $k > 0$) is governed by the Lagrangian

$$L\big(x, \dot{x}\big) = \frac{1}{2}\,\dot{x}^2 - \frac{k}{2}\,x^2,$$

in which the kinetic energy is $K = \frac{1}{2}\,\dot{x}^2$ and the potential energy is $U(x) = \frac{k}{2}\,x^2$.

We seek the trajectory $x\colon [0,1] \to \mathbb{R}$ satisfying the boundary conditions $x(0) = 1$, $x(1) = \cos(1)$, by approximating it with a neural network $f_\theta(t)$. We train f_θ by minimizing the loss

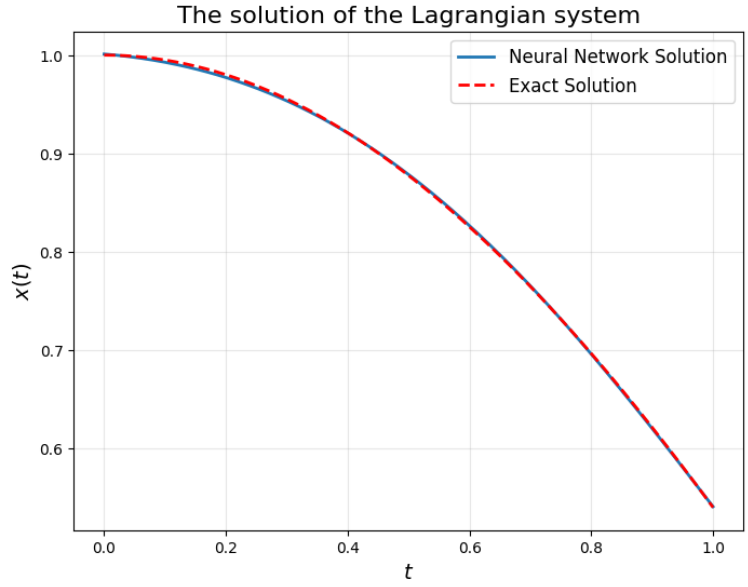
$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{j=1}^{N}\left[\frac{1}{2}\,\dot{f}_\theta(t_j)^2 - \frac{k}{2}\,f_\theta(t_j)^2\right] + \lambda_1\big(f_\theta(0) - 1\big)^2 + \lambda_2\big(f_\theta(1) - \cos(1)\big)^2.$$

Here the first term enforces the oscillator's Lagrangian density at the collocation points t_j, and the last two terms impose the endpoint constraints. In our implementation, we used $N = 10{,}000$ collocation points and chose penalty weights $\lambda_1 = 10$ and $\lambda_2 = 100$. The neural network architecture consisted of two hidden layers, each with 50 neurons and tanh activations. Training was carried out with the Adam optimizer for 100 epochs and a batch size of 32. Throughout all experiments we set the spring constant $k = 1$.

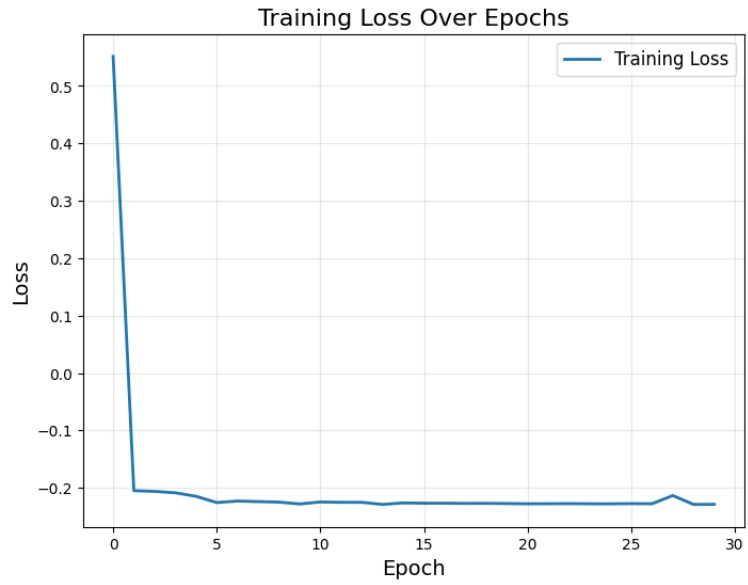
The comparison in Fig. 4.1(a) shows the exact solution $x_{\text{exact}}(t) = \cos t$ and the trained neural-network approximation $f_{\theta^*}(t)$ superimposed; the two curves coincide so closely that they are visually indistinguishable. Fig. 4.1(b) plots the loss versus training epoch, illustrating a rapid decrease followed by a plateau once the training process has effectively converged after about 5 epochs.

The detailed code can be found at

`https://tinyurl.com/457d626j`



(a)



(b)

Figure 4.1: (a) The comparison between the true solution and the neural network solution. (b) The corresponding total loss.

4.2 The Catenary Curve

The *catenary* is the curve assumed by a perfectly flexible, uniform chain or cable suspended by its ends under the influence of gravity. Its standard Cartesian equation is

$$y(x) \;=\; a\,\cosh\!\Big(\tfrac{x-x_0}{a}\Big),$$

where $a > 0$ is a constant ("the catenary parameter") and x_0 shifts the curve horizontally where

$$\cosh x = \frac{e^x + e^{-x}}{2}$$

denotes the hyperbolic cosine function.

Unlike a straight line (the shortest path), the catenary minimizes the total gravitational potential energy of the chain.

Before approaching the problem by neural networks, we shall formulate the catenary problem via Lagrangian formalism. To this end, we regard the chain profile as a function $y = y(x)$. Since the gravitational potential is proportional to the vertical coordinate, y, the total gravitational potential along the curve is obtained by integrating the gravitational potential along the curve, using the arc element $ds = \sqrt{\dot{x}(t)^2 + \dot{y}(t)^2}\,dt$, where t is a parameter along the curve. Thus, we are interested in minimizing the action integral

$$S[y] \;=\; \int y(t)\sqrt{\dot{x}(t)^2 + \dot{y}(t)^2}\,dt = \int_{x_1}^{x_2} y(x)\,\sqrt{1 + y'(x)^2}\,dx,$$

whose integrand plays the role of a Lagrangian

$$\mathcal{L}\big(y, y'\big) = y\,\sqrt{1 + (y')^2}\,.$$

Applying the Euler–Lagrange equation

$$\frac{\partial \mathcal{L}}{\partial y} \;-\; \frac{d}{dx}\Big(\frac{\partial \mathcal{L}}{\partial y'}\Big) \;=\; 0$$

yields the differential equation

$$y\,y'' \;=\; 1 + (y')^2.$$

Using standard integration techniques, we obtain its general solution given by

$$y(x) = a\,\cosh\!\Big(\tfrac{x-x_0}{a}\Big).$$

The constants a and x_0 are then fixed by imposing the two boundary conditions

$$y(x_1) = y_1 \quad \text{and} \quad y(x_2) = y_2\,,$$

so that the catenary passes exactly through $\big(x_1, y_1\big)$ and $\big(x_2, y_2\big)$. In general, solving these conditions for

$$a = a(y_1, y_2) \quad \text{and} \quad x_0 = x_0(y_1, y_2)$$

leads to transcendental equations, and therefore no simple closed-form expressions exist.

A Neural-Network Approach We aim to find a curve $(x(t), y(t))$ defined on the interval $t \in [0, 1]$, which connects the points $(x(0), y(0)) = (x_1, y_1)$ and $(x(1), y(1)) = (x_2, y_2)$, and minimizes the potential energy of a hanging chain. This energy is given by the functional:

$$S = \int_0^1 y(t)\sqrt{\dot{x}(t)^2 + \dot{y}(t)^2}\, dt.$$

To approximate the solution, we represent the curve as

$$(x_\theta(t), y_\theta(t)), \quad t \in [0, 1],$$

where the functions $x_\theta(t)$ and $y_\theta(t)$ are modeled using a feed-forward neural network $\mathrm{NN}(t; \theta) = (f_\theta(t), g_\theta(t))$, with $f_\theta, g_\theta : \mathbb{R} \to \mathbb{R}$. To enforce the boundary conditions exactly, we embed them into the parameterization:

$$x_\theta(t) = x_1 + (x_2 - x_1)t + t(1-t) \cdot f_\theta(t),$$
$$y_\theta(t) = y_1 + (y_2 - y_1)t + t(1-t) \cdot g_\theta(t).$$

This construction ensures that $x_\theta(0) = x_1$, $x_\theta(1) = x_2$, $y_\theta(0) = y_1$, and $y_\theta(1) = y_2$, regardless of the neural network outputs.

The neural networks are trained to minimize the aforementioned action integral $S(x, y)$. In practice, we approximate this integral using a Riemann sum over a uniform discretization of the interval $[0, 1]$. Given $N+1$ sample points $\{t_i\}_{i=0}^N$, the loss function becomes:

$$\mathcal{L}(\theta) \approx \frac{1}{N} \sum_{i=0}^{N} y_\theta(t_i)\sqrt{\dot{x}_\theta(t_i)^2 + \dot{y}_\theta(t_i)^2},$$

where the derivatives $\dot{x}_\theta(t)$ and $\dot{y}_\theta(t)$ are computed via automatic differentiation.

We minimize this loss function using a gradient-based optimizer, such as Adam, which updates the parameters θ of the neural networks to progressively reduce the total action.

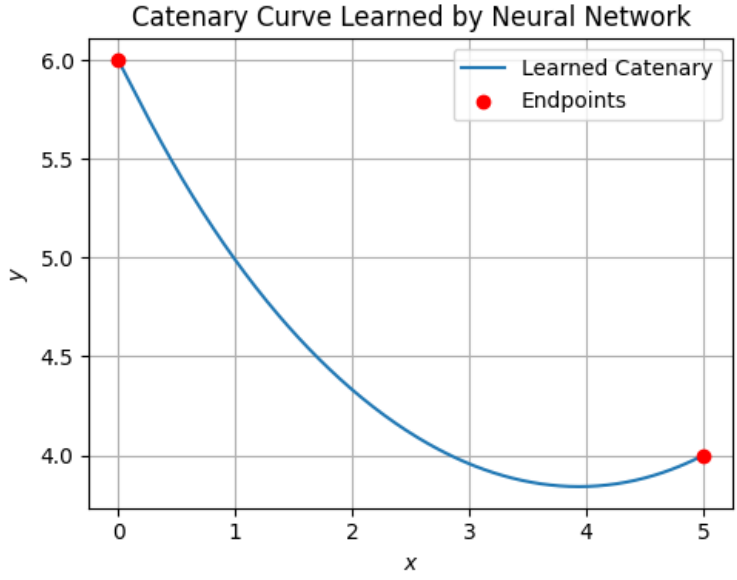
We shall implement and train the neural networks using `Keras`. At each training epoch we:

- Evaluate $x_\theta(t_i), y_\theta(t_i), \dot{x}_\theta(t_i), \dot{y}_\theta(t_i)$ for each t_i
- Compute the approximate loss $\mathcal{L}(\theta)$
- Update the neural network parameters θ via backpropagation

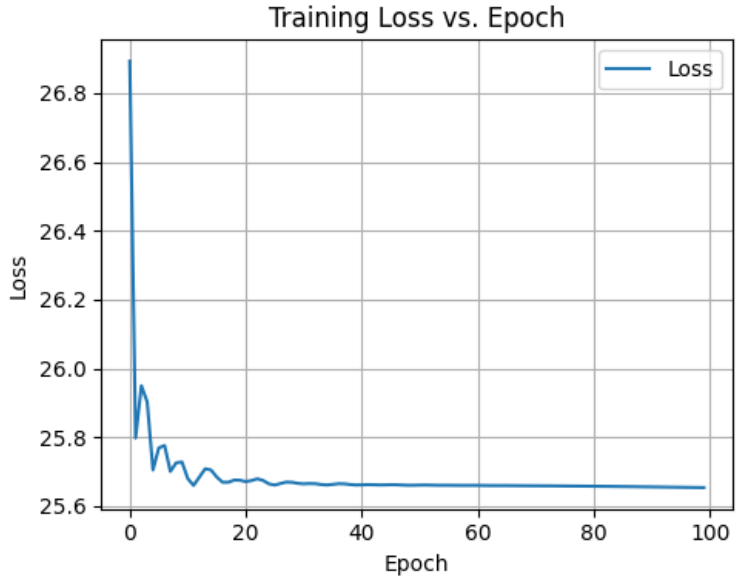
After training converges, the learned curve $(x_{\theta^*}(t), y_{\theta^*}(t))$ provides a numerical approximation of the true catenary that minimizes the action.

Keras Implementation We aim to find the catenary curve connecting the points $(x_1, y_1) = (0, 3)$ and $(x_2, y_2) = (5, 4)$. Following the previously outlined approach, we train a neural network to learn the parametric representation $(x_\theta(t), y_\theta(t))$, using a network with two dense layers with 32 neurons each and tanh activation. After 100 training epochs, the resulting curve and the corresponding loss function are shown in Figs. 4.2(a) and (b).

A natural question is whether the curve shown in Fig. 4.2(a) closely approximates a true catenary. To investigate this, we used `scipy` to fit the standard catenary model



(a)



(b)

Figure 4.2: (a) The approximation of the catenary curve after 100 epochs of training. (b) The corresponding total loss.

```
def catenary(x, a, x0):
    return a * np.cosh((x - x0) / a)
```

to the previously learned neural network curve. This fitting procedure yielded the estimated parameters:

$$a = 3.845133, \quad x_0 = 3.902177,$$

which come into the catenary's formula $y(x) = a \cosh \dfrac{x - x_0}{a}$.

When plotting the fitted catenary alongside the neural network-generated curve, the two are visually indistinguishable, as illustrated in Fig. 4.3. This shows that the neural network learned the catenary curve successfully.

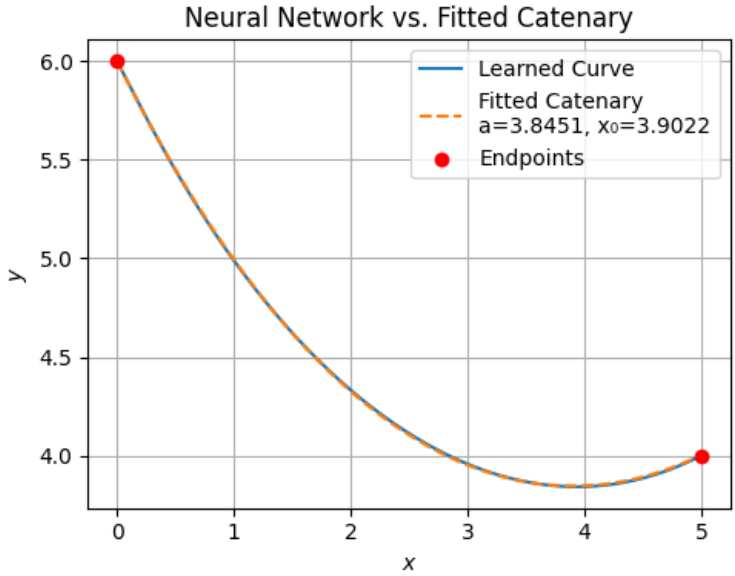


Figure 4.3: The fitted catenary and the NN-generated curve are indistinguishable.

The details of the code can be found at

`https://tinyurl.com/yvxab4a8`

4.3 The Brachistochrone Problem

The Brachistochrone problem, posed by Johann Bernoulli in 1696, asks for the curve along which a particle will descend from a higher point A to a lower point B in the shortest time under the influence of gravity, assuming no friction and no initial velocity. The problem drew the attention of the greatest mathematicians of the time, including Newton, Leibniz, and the Bernoulli brothers. Its solution marked the birth of the calculus of variations.

A naive assumption might suggest that the straight line between two points yields the shortest travel time, due to it being the shortest path. However, gravity accelerates a particle more rapidly along steeper slopes. Hence, a steeper initial descent followed by a flatter approach may reduce the overall time. This trade-off between path length and acceleration leads to a nontrivial optimal curve. The formal mathematical formulation is as follows.

Mathematical Formulation Let a particle of unit mass move under gravity along a smooth curve $y(x)$, connecting two points $A = (x_1, y_1)$ and $B = (x_2, y_2)$, with $x_1 < x_2$ and $y_1 > y_2$. Since the speed of a falling body[1] from height y is $v = \sqrt{2gy}$, the time taken to traverse this path is given by:

$$T[y] = \int \frac{1}{v}\, ds = \int_{x_1}^{x_2} \sqrt{\frac{1 + (y')^2}{2gy}}\, dx,$$

where $y' = \frac{dy}{dx}$ and g is the gravitational constant. The task is to find the function $y(x)$ minimizing this integral, subject to the given endpoints. Since there is unfortunately no closed-form solution $y = y(x)$ for this variational problem, we shall approach it parametrically, assuming that the curve is of the type

$$x = x(t), \qquad y = y(t), \qquad t \in [0, 1],$$

case in which the time becomes the following functional

$$T[x, y] = \int_0^1 \sqrt{\frac{x'(t)^2 + y'(t)^2}{2gy(t)}}\, dt,$$

subject to boundary conditions $(x(0), y(0)) = (x_1, y_1)$ and $(x(1), y(1)) = (x_2, y_2)$. The integrand provides the Lagrangian

$$L(x, y, x', y') = \sqrt{\frac{x'(t)^2 + y'(t)^2}{2gy(t)}}.$$

It can be shown that solving the associated Euler–Lagrange differential equations we obtain a flipped cycloid

$$\begin{aligned} x(t) &= a(ut - \sin(ut) + b, \\ y(t) &= -a(1 - \cos(ut)) + c, \end{aligned}$$

where $t \in [0, 1]$, $u > 0$ depends on the arc parameter, $a > 0$ is the shape parameter and (b, c) is a translation vector.

[1] At height y, due to conservation of energy, the kinetic energy equals the potential energy, $\frac{1}{2}mv^2 = mgy$. Solving for speed yields $v = \sqrt{2gy}$.

Remark 4.3.1 The parameters a, b, c, and u are determined by the specific endpoints conditions. Thus, taking $t = 0$ yields $(b, c) = (x_1, y_1)$, while choosing $t = 1$ leads, after eliminating parameter a, to the following equation in u

$$\frac{\cos u - 1}{u - \sin u} = \frac{y_2 - y_1}{x_2 - x_1}.$$

If $m = \dfrac{y_2 - y_1}{x_2 - x_1}$ denotes the slope of the line passing through the endpoints, considering the function

$$f(u) = \frac{\cos u - 1}{u - \sin u}, \quad u > 0$$

then the equation $f(u) = m$ has a finite number of positive solutions for $m < 0$, denoted by $u_1, \cdots, u_k$. For m small enough (actually, for $m < -0.2225$) the solution is unique, see the graph in Fig. 4.4.

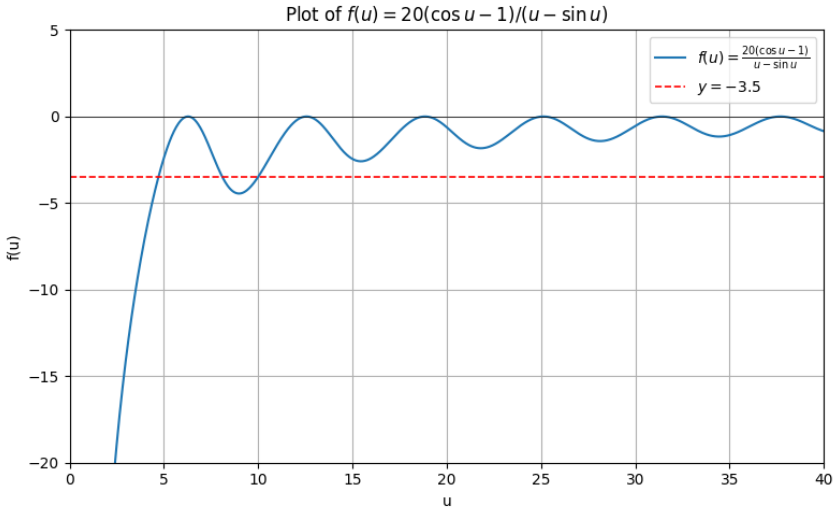


Figure 4.4: Any horizontal line of the form $y = m$, where $m < 0$, intersects the graph of $f(u)$ at only finitely many points. Each intersection corresponds to a solution of the equation $f(u) = m$, implying that the number of solutions is finite. The factor of 20 in the definition of $f(u)$ was introduced to amplify the oscillations for better visibility. In the plotted figure, we have chosen $m = -3.5$ as a representative example.

Each solution u_k corresponds to a brachistocrone solution curve passing through the endpoints. Only the smallest solution $u_1 > 0$ matters, since the others correspond to brachistocrone curves which introduce multiple cuspidal points.

Neural Network Approach We construct a Keras code that goes through the following things:
1) Boundary points. We set the two endpoints

$$A = (0, 1), \quad B = (1, 0),$$

which the learned curve must interpolate:

```
A = np.array([0.0, 1.0])   # (x0, y0)
B = np.array([1.0, 0.0])   # (x1, y1)
```

2) Parameter discretization. We introduce a uniform grid of t-values in $[0, 1]$,

$$t_i = \frac{i}{n_{\text{points}} - 1}, \quad i = 0, 1, \ldots, n_{\text{points}} - 1,$$

with $n_{\text{points}} = 1200$. These points serve as inputs to the network and for approximating integrals:

```
n_points = 1200
t_values = np.linspace(0, 1, n_points).reshape(-1,1).astype(np.float32)
```

3) Neural network architecture. We define a Keras `Model` taking t as input and producing two outputs $(x_\theta(t), y_\theta(t))$. The network has six hidden layers of 32 `tanh` neurons each. The final y-branch uses a `softplus` activation to enforce $y_\theta(t) > 0$. This is because the Lagrangian has a $\sqrt{y_\theta(t)}$ at the denominator and we need to be sure that the square root expression makes sense:

```
inp = keras.Input(shape=(1,))
h = layers.Dense(32, activation='tanh')(inp)
h = layers.Dense(32, activation='tanh')(h)
h = layers.Dense(32, activation='tanh')(h)
h = layers.Dense(32, activation='tanh')(h)
h = layers.Dense(32, activation='tanh')(h)
h = layers.Dense(32, activation='tanh')(h)
h = layers.Dense(32, activation='tanh')(h)
x_out = layers.Dense(1)(h)
y_out = layers.Dense(1, activation='softplus')(h)  # ensures y>0
model = keras.Model(inputs=inp, outputs=[x_out, y_out])
```

4) Boundary-condition loss. A helper function measures the squared error between the network's endpoints $\big(x_\theta(0), y_\theta(0)\big)$, $\big(x_\theta(1), y_\theta(1)\big)$ and the desired points A and B. These endpoint errors are heavily weighted to enforce the boundary conditions strongly:

```
def boundary_loss(x_pred, y_pred):
    x0, y0 = x_pred[0],   y_pred[0]
    x1, y1 = x_pred[-1],  y_pred[-1]
    return (
        tf.square(x0 - A[0]) +
        tf.square(y0 - A[1]) +
        30.0*tf.square(x1 - B[0]) +
        30.0*tf.square(y1 - B[1])
    )
```

5) Lagrangian (time-integral) loss. Using `tf.GradientTape`, we compute the derivatives $\dot{x}_\theta(t)$ and $\dot{y}_\theta(t)$. The integrand

$$\frac{\sqrt{\dot{x}_\theta^2 + \dot{y}_\theta^2}}{\sqrt{y_\theta(t)}}$$

represents the infinitesimal descent time under gravity. We approximate its integral over $t \in [0, 1]$ by a Riemann sum on the uniform grid:

```
def lagrangian_loss(t_tensor):
    with tf.GradientTape(persistent=True) as tape:
        tape.watch(t_tensor)
        x_pred, y_pred = model(t_tensor)
    dx_dt = tape.gradient(x_pred, t_tensor)
    dy_dt = tape.gradient(y_pred, t_tensor)
    y_safe = tf.maximum(y_pred, 1e-12)
    integrand = tf.sqrt(dx_dt**2 + dy_dt**2) / tf.sqrt(y_safe)
    dt = t_tensor[1] - t_tensor[0]
    return tf.reduce_sum(integrand * dt)
```

6) Total loss. The custom training loss is

$$\mathcal{L} = 100\,\mathcal{L}_{\text{boundary}} + \mathcal{L}_{\text{Lagrangian}},$$

where $\mathcal{L}_{\text{boundary}}$ and $\mathcal{L}_{\text{Lagrangian}}$ are defined previously.

7) Training. We compile the model with Adam (learning rate 10^{-2}) and train for 10 epochs, recording the loss history:

```
model.compile(optimizer=keras.optimizers.Adam(1e-2), loss=total_loss)
history = model.fit(
    t_values,
    [np.zeros_like(t_values), np.zeros_like(t_values)],
    epochs=10,
    verbose=0
```

8) Prediction. After training, we evaluate the network on the same t-grid to obtain $(x_\theta(t_i), y_\theta(t_i))$:

```
x_pred, y_pred = model.predict(t_values)
```

9) Exact cycloid computation. A standard cycloid parametrized by ϕ and a scale R is

$$x(\phi) = R\big(\phi - \sin\phi\big), \qquad y(\phi) = 1 \;-\; R\big(1 - \cos\phi\big).$$

At $\phi = 0$ this yields $(x, y) = (0, 1)$. To enforce that the curve ends at $(1, 0)$, we impose the two boundary conditions at $\phi = \phi_{\max}$:

$$R\big(\phi_{\max} - \sin\phi_{\max}\big) = 1, \qquad 1 - R\big(1 - \cos\phi_{\max}\big) = 0.$$

We eliminate R to obtain the single equation

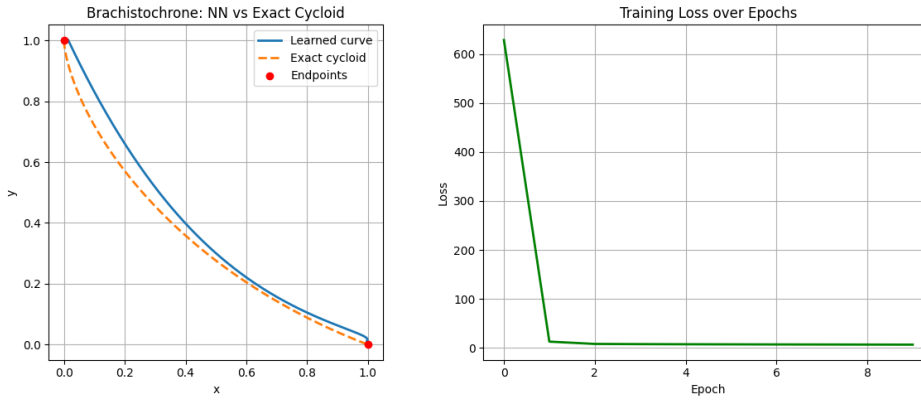
$$\phi_{\max} - \sin\phi_{\max} \;=\; 1 - \cos\phi_{\max},$$

which can be solved numerically for $\phi_{\max} \approx 2.41201$. Finally, substituting back gives

$$R \;=\; \frac{1}{\phi_{\max} - \sin\phi_{\max}} \approx 0.57291,$$

so that the parametrization above runs exactly from $(0, 1)$ to $(1, 0)$.

10) Visualization. We plot the learned NN-curve (solid blue), the exact cycloid (dashed orange), and the red endpoints, see Fig. 4.5(a). Then we plot training loss versus number of epochs, see Fig. 4.5(b).



(a) **(b)**

Figure 4.5: (a) The solution model and the exact cycloid between $(0, 1)$ and $(1, 0)$. (b) The training loss versus number of epochs.

We observe that the neural network fails to recover the exact cycloid shape, especially in the region where the derivative is large. Since the loss already reached the threshold stability level, further training will not improve this discrepancy, which actually stems from the safety clamp

```
y_safe = tf.maximum(y_pred, 1e-12),
```

which was introduced to prevent division by zero in the integrand.

A similar phenomenon arises in any variational problem whose Lagrangian contains factors at denominator that can vanish. Thus, the numerical safeguards necessary for stability inevitably introduce systematic errors that degrade the quality of the solution.

The complete code for this section can be found at:

`https://tinyurl.com/z6v5dcbr`

Optical Analogy According to Fermat's Principle, light travels along the path that minimizes its travel time. This principle, which lies at the heart of classical optics, can be used to reframe the problem of a particle falling under gravity into a light-ray tracing problem in an inhomogeneous medium.

Suppose we consider a hypothetical medium composed of infinitesimally-thick horizontal layers, each with a refractive index that depends on vertical position y. Specifically, we assume the refractive index at height y is given by

$$n(y) = \frac{c}{v(y)} = \frac{c}{\sqrt{2gy}},$$

where c is the speed of light in vacuum, and $v(y) = \sqrt{2gy}$ is the speed of light in the medium at height y. This choice of refractive index mimics the variation in speed a particle experiences as it descends under gravity. This assumption implies that light travels faster as it goes deeper into the medium. By this choice, the speed of light in such a medium is then

$$v_{\text{light}}(y) = \frac{c}{n(y)} = \sqrt{2gy},$$

identical to the speed of a particle falling freely from rest under gravity from height y.

By Fermat's principle, the time taken by light to travel along a path is given by

$$T_{\text{light}} = \int \frac{1}{v_{\text{light}}(y)}\, ds = \int \frac{n(y)}{c}\, ds,$$

where ds is the arc length element. Minimizing this time integral yields the trajectory of the light ray, which corresponds exactly to the brachistochrone curve. To show this we use Snell's law, which is a consequence of Fermat's principe of least time. Snell's law for refraction applied across each horizontal interface between layers at depth y with varying refractive indices is[2]

$$n(y)\sin\theta(y) = k, \text{ constant},$$

where θ is the angle between the light ray and the normal direction. As light travels downward through layers of increasing speed (i.e., decreasing $n(y)$), its angle increases, mirroring the steepening slope of the brachistochrone curve, see Fig. 4.6.

Next we shall express the sine factor in terms of vertical depth y as

$$\sin\theta(y) = \frac{k}{n(y)} = \frac{kv(y)}{c} = \frac{k}{c}\sqrt{2g}\sqrt{y}.$$

Taking the square and inverting yields

$$\frac{1}{\sin^2\theta(y)} = \frac{K^2}{y}, \tag{4.3.2}$$

[2]For the discrete case we have $n_1 \sin\theta_1 = n_2 \sin\theta_2$, where $n_i = n(y_i)$, $\theta_i = \theta(y_i)$, $i = 1, 2$.

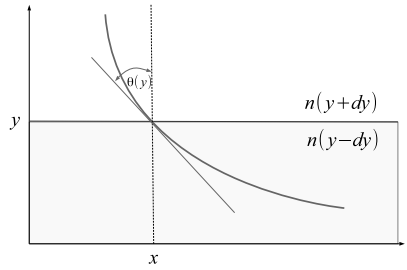


Figure 4.6: The angle between the tangent line and the normal, $\theta(y)$, satisfies Snell's law of refraction $\sin\theta(y) = \frac{k}{n(y)}$.

where $K = \dfrac{c}{k\sqrt{2g}}$. Since $\theta(y)$ is the angle made by the tangent to the graph $y = y(x)$ with the vertical direction, then the slope of the graph is given by its derivative as

$$y'(x) = \tan\left(\frac{\pi}{2} - \theta(y)\right) = \cot\theta(y). \tag{4.3.3}$$

Therefore, using some trigonometry and formula (4.3.3) we obtain

$$\frac{1}{\sin^2\theta(y)} = 1 + \cot^2\theta(y) = 1 + y'(x)^2.$$

Comparing with (4.3.2) yields the following differential equation in y

$$1 + y'(x)^2 = \frac{K^2}{y}. \tag{4.3.4}$$

Integrating, we obtain

$$\int \sqrt{\frac{y}{K^2 - y}}\, dy = x + C.$$

Since the left side integral cannot be expressed in terms of elementary functions, we lack an explicit elementary formula $y = y(x)$ for the light trajectory. This is the reason why the cycloid equations are always given in a parametric form and not in an explicit form. To this end, we shall choose the angle θ as a parameter and express both x and y coordinates in terms of θ. We can express from (4.3.2) the y coordinates in terms of θ as

$$y = K^2\sin^2\theta = \frac{K^2}{2}(1 - \cos(2\theta)). \tag{4.3.5}$$

We shall do the same for the x coordinate. We start from equation (4.3.3) using the chain rule formula $y'(x) = \dfrac{dy/d\theta}{dx/d\theta}$ to obtain

$$\frac{dx}{d\theta} = \tan\theta\,\frac{dy}{d\theta}. \tag{4.3.6}$$

The last factor can be computed by taking the derivative in (4.3.5)

$$\frac{dy}{d\theta} = 2K^2 \sin\theta\cos\theta = K^2 \sin(2\theta).$$

Substituting in equation (4.3.6) yields

$$\frac{dx}{d\theta} = 2K^2 \sin^2\theta = K^2(1-\cos(2\theta)).$$

Integrating with respect to θ we get

$$x(\theta) = \frac{K^2}{2}(2\theta - \sin(2\theta)) + C,$$

with $C \in \mathbb{R}$. Substituting $t = 2\theta$ and taking $C = 0$, and denoting $a = \dfrac{K^2}{2}$, we obtain the parametric equations of the standard cycloid

$$x(t) = a(t - \sin t) \tag{4.3.7}$$

$$y(t) = a(1 - \cos t). \tag{4.3.8}$$

Again, since relation (4.3.7) cannot be inverted explicitly as $t = t(x)$, there is no closed form expression for $y = y(x)$.

Neural network approach We now make use of the analytic conservation law (4.3.2)

$$\frac{y(x)}{\sin^2\theta(x)} \;=\; K^2,$$

which follows directly from the Snell's-law interpretation of the brachistochrone. Unlike the previous integral-minimization approach, which required numerical safeguards and introduced bias, this formulation permits perfect learning.

To this end, we define a small perturbation network $g_\theta(x)$ and reparameterize

$$f_\theta(x) \;=\; 1 \;-\; x \;+\; x(1-x)\, g_\theta(x),$$

so that the boundary conditions $f_\theta(0) = 1$ and $f_\theta(1) = 0$ are enforced exactly. Next, we set

$$q(x) \;=\; f_\theta(x)\,\big(1 + f_\theta'(x)^2\big),$$

which by the conservation law must be constant and equal to K^2. We then train the network by minimizing the sample variance of q:

$$\mathcal{L}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N}\big(q(x_i) - \overline{q}\big)^2, \quad \overline{q} = \frac{1}{N}\sum_{i=1}^{N} q(x_i).$$

Driving $\mathcal{L}(\theta) \to 0$ forces $q(x)$ to a perfect constant[3] and thus recovers the exact cycloid.

Implementation The overall procedure is:

[3]We use that a random variable X is constant if and only if $Var(X) = 0$.

- Discretize $x \in [0,1]$ on a uniform grid $\{x_i\}_{i=1}^{N}$, with $N = 5,000$:

```
  N = 5000
x_vals = np.linspace(0, 1, N + 1).reshape(-1, 1).astype(np.float32)
  x_tf   = tf.constant(x_vals)
```

- Build the perturbation network $g_\theta(x)$ as a feed-forward network with 4 hidden layers, 32 neurons each:

```
 def build_model():
    inp = keras.Input(shape=(1,))
    h = layers.Dense(32, activation='tanh')(inp)
    h = layers.Dense(32, activation='tanh')(h)
    h = layers.Dense(32, activation='tanh')(h)
    h = layers.Dense(32, activation='tanh')(h)
    out = layers.Dense(1)(h)
    return keras.Model(inputs=inp, outputs=out)
model = build_model()
```

- Reparameterize $f_\theta(x) = 1 - x + x(1-x)\, g_\theta(x)$ and compute its first derivative:

```
  def f_and_df(x):
    with tf.GradientTape() as tape:
        tape.watch(x)
        g = model(x)
        f = 1.0 - x + x*(1.0 - x)*g   # boundary conditions
    df = tape.gradient(f, x)
    return f, df
```

- Compute $q(x_i)$ and its mean $\overline{q}$ and use them to define the loss:

```
  def variance_loss():
    f, df = f_and_df(x_tf)
    q = f * (1.0 + df**2)
    q_mean = tf.reduce_mean(q)
    return tf.reduce_mean((q - q_mean)**2)
```

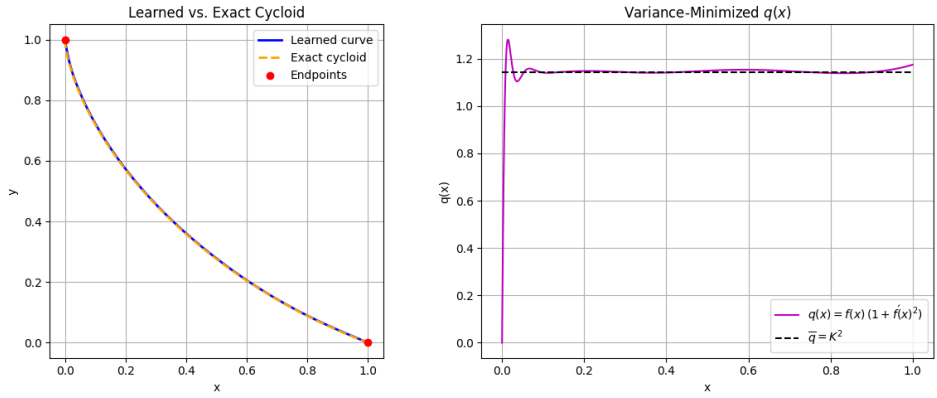
- Minimize the variance $\frac{1}{N}\sum(q(x_i) - \overline{q})^2$ by Adam optimizer

```
optimizer = keras.optimizers.Adam(learning_rate=1e-3)
```

  for $3,000$ epochs, using

- Upon convergence, $f_\theta(x)$ is the true cycloid arc, see Fig. 4.7(a) and $\overline{q} = K^2$ see Fig. 4.7(b). We notice that $f_\theta(x)$ provides a perfect learning of the cycloid arc. The code details are to be found at:

  https://tinyurl.com/z6v5dcbr



(a) **(b)**

Figure 4.7: (a) The solution model matches the exact cycloid between $(0, 1)$ and $(1, 0)$. (b) The mean $\bar{q} = K^2$.

Remark 4.3.2 We emphasize that the constant K was not known a priori but instead recovered from the network's output as the sample mean of

$$q(x) \;=\; f_\theta(x)\left(1 + f'_\theta(x)^2\right).$$

In practice, one observes small spurious oscillations in the estimated $q(x)$ near $x = 0$. These arise because the true cycloid has an unbounded derivative at the starting point, which our finite-precision computation cannot represent exactly. Despite this local artifact, the overall variance minimization remains accurate, and the same strategy will be applied in subsequent sections to enforce more general conservation laws.

Fermat's Principle and Variational Problems

Fermat's principle of least time can be extended to a broader class of variational problems. Specifically, given two points A and B, we seek a curve in the (x, y)-plane that connects these points and minimizes a potential-dependent functional of the form:

$$\int U(y)\, ds,$$

where $ds = \sqrt{\dot{x}(t)^2 + \dot{y}(t)^2}\, dt$ is the arc length element, and $U(y)$ is a given potential function. The associated Lagrangian for this problem is:

$$L = U(y)\sqrt{1 + (y')^2}.$$

Rather than applying the classical Euler–Lagrange formalism, we can solve this problem using Fermat's principle by interpreting the action as a *travel time* from point A to point B. Specifically, we define the travel time as:

$$T(A, B) = \int \frac{1}{v(y)}\, ds,$$

where $v(y)$ is the speed of a particle at height y. To make the two integrals equivalent, we set:

$$v(y) = \frac{1}{U(y)}.$$

This reformulation is analogous to the brachistochrone problem, where the path of least time corresponds to that of a light ray traveling through an inhomogeneous medium with spatially varying speed $v(y)$. In this optical context, *Snell's law* applies across infinitesimal layers of the medium and takes the form:

$$n(y)\sin\theta(y) = k,$$

where $\theta(y)$ is the angle between the trajectory and the vertical direction, $n(y)$ is the refractive index at height y, and k is a constant. Using the relation $n(y) = \frac{c}{v(y)}$, we find:

$$\sin\theta(y) = \frac{c}{kv(y)}, \quad \text{which implies} \quad \frac{1}{\sin^2\theta(y)} = \frac{c^2}{k^2 v(y)^2}.$$

From the geometric setup (as in the brachistochrone), we also have:

$$\frac{1}{\sin^2\theta(y)} = 1 + \big(y'(x)\big)^2.$$

Equating these expressions yields the differential equation:

$$1 + \big(y'(x)\big)^2 = \frac{K^2}{v(y)^2},$$

where $K = \frac{c}{k}$ is a constant. Substituting $v(y) = \frac{1}{U(y)}$, we arrive at the nonlinear ODE:

$$1 + \big(y'(x)\big)^2 = K^2 U(y)^2.$$

Solving for $y'(x)$, we can separate variables and integrate:

$$\int \frac{dy}{\sqrt{K^2U(y)^2 - 1}} = x - x_0. \tag{4.3.9}$$

The integral on the left-hand side can be computed in closed form for certain specific potential functions $U(y)$. Below we list several such cases.

- **Case $U(y) = y$:**

  $$\int \frac{dy}{\sqrt{K^2y^2 - 1}} = \frac{1}{K}\cosh^{-1}(Ky).$$

  Then (4.3.9) becomes

  $$y = \frac{1}{K}\cosh(K(x - x_0)),$$

  which recovers the catenary curve.

- **Case** $U(y) = \sqrt{y}$**:**

$$\int \frac{dy}{\sqrt{K^2 y - 1}} = \frac{2}{K}\sqrt{K^2 y - 1}.$$

Then (4.3.9) yields a parabola

$$y = \frac{1}{4}(x - x_0)^2 + \frac{1}{K^2}.$$

- **Case** $U(y) = 1/y$**:**

$$\int \frac{dy}{\sqrt{\frac{K^2}{y^2} - 1}} = -\sqrt{K^2 - y^2}.$$

Then (4.3.9) becomes

$$(x - x_0)^2 + y^2 = K^2,$$

which is the equation of a circle centered at $(x_0, 0)$ of radius K. This corresponds to the case of a minimal curve in an electrostatic potential.

- **Case** $U(y) = e^{ay}$**:**

$$\int \frac{dy}{\sqrt{K^2 e^{2ay} - 1}} = \frac{1}{aK}\cosh^{-1}(Ke^{ay}).$$

Then (4.3.9) implies

$$y = \frac{1}{a}\ln\cosh(aK(x - x_0)) - \frac{1}{a}\ln K,$$

which corresponds to a curve called logarithmic catenary.

4.4 The Simple Pendulum

The simple pendulum consists of a point mass m suspended from a massless rigid rod (or string) of length ℓ, swinging under gravity. Its configuration is described by the angle $\phi(t)$ between the rod and the vertical.
The kinetic energy is

$$K = \frac{1}{2}mv^2 = \frac{1}{2}m\Big(\frac{ds}{dt}\Big)^2 = \frac{1}{2}m\ell^2\dot{\phi}^2,$$

since the arc length satisfies $s = \ell\,\phi$. The potential energy, measured from the lowest point, is

$$U = mgh = mg\,\ell\,(1 - \cos\phi),$$

because the vertical rise is $\ell - \ell\cos\phi$.

Hence, the Lagrangian $L = K - U$ becomes

$$L(\phi, \dot{\phi}) = \frac{1}{2}\,m\ell^2\,\dot{\phi}^2 \;-\; mg\,\ell\,(1 - \cos\phi).$$

Up to an additive constant and an overall factor $m\ell^2$, one may equivalently work with the simplified Lagrangian

$$L(\phi, \dot{\phi}) = \frac{1}{2}\,\dot{\phi}^2 \;+\; \kappa\,\cos\phi, \qquad \kappa = \frac{g}{\ell} > 0. \tag{4.4.10}$$

The Euler–Lagrange equation then yields the nonlinear pendulum equation

$$\ddot{\phi} + \kappa\,\sin\phi = 0.$$

We will study its solutions subject to the initial conditions

$$\phi(0) = \phi_0, \qquad \dot{\phi}(0) = v_0.$$

To compute the pendulum trajectory $\phi(t)$, we parametrize ϕ by a neural network $\phi_\theta(t)$ and determine its weights θ in one of two equivalent ways:

- (i) **Action-minimization:** We minimize the discretized action

$$S[\phi_\theta] \;=\; \int_0^T L\big(\phi_\theta(t), \dot{\phi}_\theta(t)\big)\,dt,$$

  where $L = \frac{1}{2}\dot{\phi}^2 + \kappa\cos\phi$.

- (ii) **PINN (Euler–Lagrange) loss:** We enforce the ODE $\ddot{\phi} + \kappa\sin\phi = 0$ by minimizing

$$\mathcal{L}_{\text{PINN}}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N}\big(\ddot{\phi}_\theta(t_i) + \kappa\sin\phi_\theta(t_i)\big)^2$$

  over a grid $\{t_i\}$ and we choose $\kappa = 1$.

Either approach produces a $\phi_\theta(t)$ that satisfies the pendulum equation under the prescribed initial conditions, each with its own advantages and disadvantages. We will compare these neural-network solutions against a reference solution obtained by a classical numerical integrator (Runge–Kutta), since the pendulum equation admits no closed-form solution for direct comparison. The complete implementation is available at:

`https://tinyurl.com/3t6kek8x`

(i) **Action-minimization** We set the time horizon to $T = 1$, since for larger T the pure action-minimization tends to fail, so it is only reliable on short intervals. To approximate the action integral accurately, we use $N = 6,000$ uniformly spaced points on $[0, 1]$

```
N = 6000
t_vals = np.linspace(0, 1.0, N + 1).reshape(-1,1).astype(np.float32)
dt = t_vals[1,0] - t_vals[0,0]
t_tf = tf.constant(t_vals)
```

and compute the Lagrangian loss via the trapezoidal rule implemented in code as

```
def lagrangian_loss(x_t, dx_dt):
    integrand = 0.5*dx_dt**2 + tf.cos(x_t)
    f0, f1 = integrand[:-1], integrand[1:]
    return tf.reduce_sum(0.5*(f0+f1)) * dt
```

Our neural network $\phi_\theta(t)$ has four hidden layers of 32 neurons each, all with `tanh` activations:

```
def build_model():
    inp = keras.Input(shape=(1,))
    h = layers.Dense(32, activation='tanh')(inp)
    h = layers.Dense(32, activation='tanh')(h)
    h = layers.Dense(32, activation='tanh')(h)
    h = layers.Dense(32, activation='tanh')(h)
    out = layers.Dense(1)(h)
    return keras.Model(inputs=inp, outputs=out)
model = build_model()
```

The boundary-condition loss is enforcing $\phi(0) = \phi_0 = 1$ and $\dot{\phi}(0) = v_0 = 0$) via

```
def boundary_loss(x_t, dx_dt):
    x0, dx0 = x_t[0], dx_dt[0]
    return tf.square(x0 - 1.0) + tf.square(dx0 - 0.0)
```

and is scaled by a factor of $1,000$ via

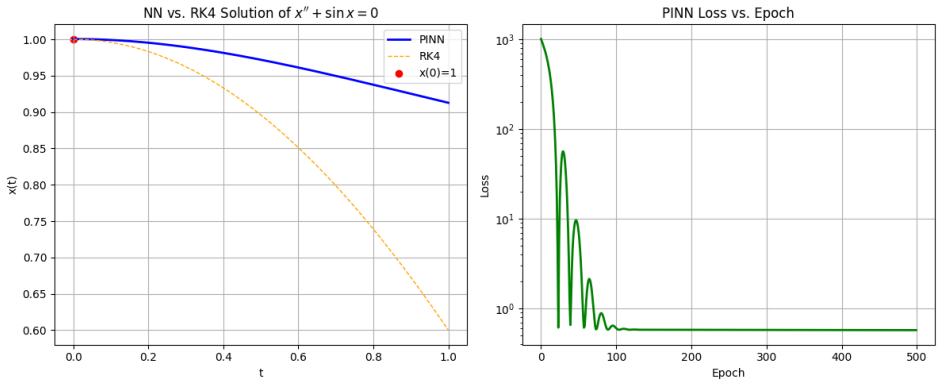
```
def compute_loss():
    x_t, dx_dt = x_and_dx(t_tf)
    return lagrangian_loss(x_t, dx_dt) + 1000*boundary_loss(x_t, dx_dt)
```

to balance against the Lagrangian loss evaluated over $6,000$ points. We train for 500 epochs using Adam optimizer with a learning rate 0.001.
Then we plot the learned solution alongside a reference solution obtained by the Runge–Kutta method, see Fig. 4.8.

We note that even at $T = 1$, the two curves diverge, demonstrating that action minimization alone is insufficient. To remedy this, we introduce a data loss at two interior anchor points at $t = 0.5$ and $t = 1.0$ (since $t = 0$ is fixed by the initial conditions), also weighted by $1,000$ via `fracs = [0.5, 1.0]`.
With these two enforced data points, the network's $\phi_\theta(t)$ almost perfectly overlaps the Runge–Kutta solution, see Fig. 4.9.

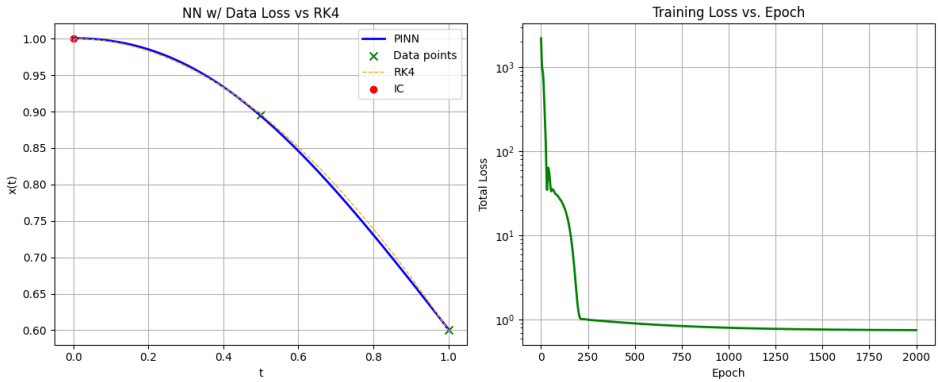
(*ii*) **PINN (Euler–Lagrange) loss** We select $N = 1,000$ equispaced collocation points $t_j \in [0, 1]$, and compute the residual

$$R(t_j) \;=\; \ddot{\phi}_\theta(t_j) \;+\; \kappa \sin\big(\phi_\theta(t_j)\big),$$



(a) **(b)**

Figure 4.8: (a) The solution model does not match the Runge–Kutta solution on $[0, 1]$. (b) The loss reaches the saturation trashed after 100 epochs.



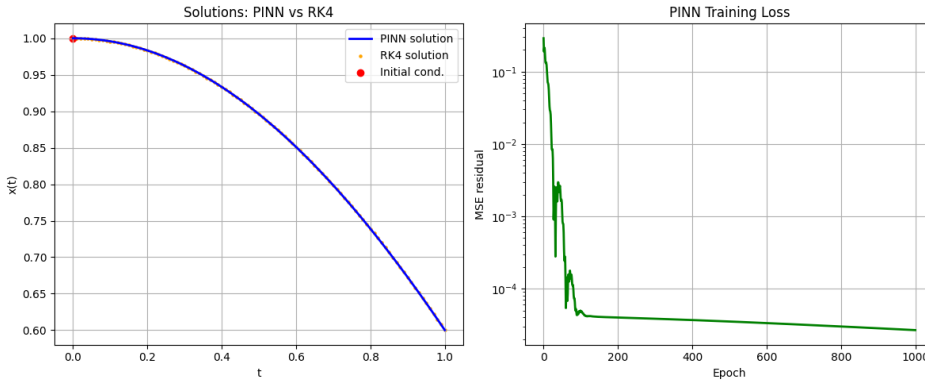
(a) **(b)**

Figure 4.9: (a) The solution model pinned down at $t = 0.5$ and $t = 1$ does match the Runge–Kutta solution on $[0, 1]$. (b) The loss reaches the saturation trashed after 1500 epochs.

where ϕ_θ is represented by a neural network with three hidden layers of 32 `tanh` neurons each. The PINN loss is the mean-squared residual,

$$\mathcal{L}_{\text{PINN}}(\theta) \;=\; \frac{1}{N}\sum_{j=1}^{N} R(t_j)^2,$$

and we retain the same boundary-condition penalty as before. After $1,000$ training epochs, the learned solution $\phi_\theta(t)$ is indistinguishably close to the reference solution computed via Runge–Kutta, see Fig. 4.11. This demonstrates that enforcing the Euler–Lagrange ODE directly is far more efficient than minimizing the action integral.



(a) **(b)**

Figure 4.10: (a) The solution model does match the Runge–Kutta solution on $[0, 1]$. (b) The loss reaches the saturation level after $1,000$ epochs.

However, on longer time intervals $T \gg 1$, the unconstrained PINN solution will eventually drift from the Runge–Kutta reference solution unless additional collocation or data points are provided. In Fig. 4.11 we used $T = 5$ and $N = 5,000$ collocation points.

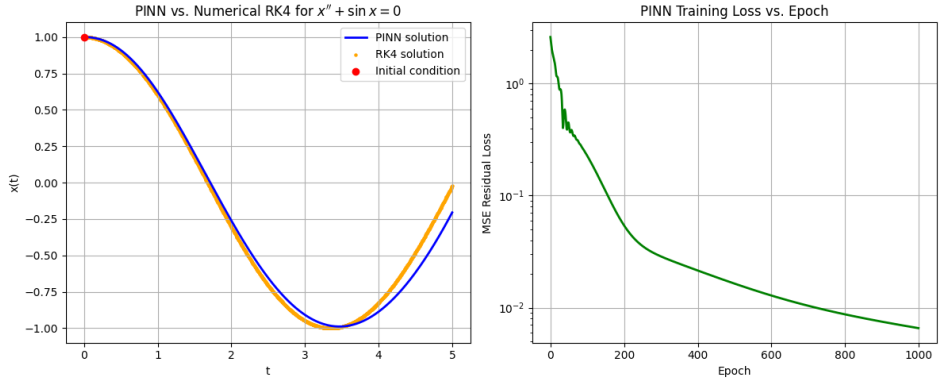
Conclusion. The action-minimization approach can succeed only when supplemented by sparse data constraints. The PINN model is robust on sufficiently short intervals. However, both methods improve if additional data points are provided.

4.5 Elastica Curves

An *Euler elastica curve* is a smooth plane curve that minimizes the bending energy, expressed by the Euler–Bernoulli functional

$$E(c) = \int_0^T \kappa(s)^2 \, ds,$$

where $c : [0, T] \to \mathbb{R}^2$ is parametrized by arc length s, so that $\|\dot{c}(s)\| = 1$, T is the curve length, and $\kappa(s) = \|\ddot{c}(s)\|$ is the curvature along $c(s)$.



(a) **(b)**

Figure 4.11: (a) The solution model diverges slowly from the Runge–Kutta solution on $[0, 5]$. (b) The associated loss.

The elastica curve is subject to boundary constraints on both positions and tangent directions

$$c(0) = A, \quad c(T) = B, \quad \dot{c}(0) = v_A, \quad \dot{c}(T) = v_B,$$

for prescribed points $A, B \in \mathbb{R}^2$ and unit vectors $v_A, v_B \in \mathbb{R}^2$.

The solutions are curves that distribute bending as uniformly as possible, subject to the given boundary conditions. Euler classified plane elastica curves into nine distinct classes and obtained their closed-form representations in terms of elliptic functions.

In the following, we approach this problem using neural networks, employing both soft constraints and hard embeddings.

Soft Constraints A straightforward way to address the elastica problem using neural networks is to incorporate both the boundary constraints and the unit-speed condition as soft penalties in the loss function.

We begin with a feedforward neural network that takes as input a parameter $s \in [0, T]$ and produces a two-dimensional output

$$c_\theta(s) = \big(c_\theta^1(s),\, c_\theta^2(s)\big).$$

We then compute the curvature proxy

$$\kappa_\theta(s) = \|\ddot{c}_\theta(s)\|^2 = \big(\ddot{c}_\theta^1(s)\big)^2 + \big(\ddot{c}_\theta^2(s)\big)^2,$$

under the unit-speed hypothesis $\|\dot{c}_\theta\| \approx 1$. Then we define the total loss

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{bend}}(\theta) + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}}(\theta) + \lambda_{\text{unit}}\mathcal{L}_{\text{unit}}(\theta),$$

where each component is given by

$$\mathcal{L}_{\text{bend}}(\theta) = \int_0^T \kappa_\theta(s)^2\, ds \;\approx\; \frac{T}{N}\sum_{j=1}^{N} \kappa_\theta(s_j)^2,$$

$$\mathcal{L}_{\mathrm{BC}}(\theta) = \|c_\theta(0) - A\|^2 + \|c_\theta(T) - B\|^2 + \|\dot{c}_\theta(0) - v_A\|^2 + \|\dot{c}_\theta(T) - v_B\|^2,$$

$$\mathcal{L}_{\mathrm{unit}}(\theta) = \int_0^T \left(\|\dot{c}_\theta(s)\|^2 - 1\right)^2 ds \;\approx\; \frac{T}{N}\sum_{j=1}^{N}\left(\left(\dot{c}_\theta^1(s_j)\right)^2 + \left(\dot{c}_\theta^2(s_j)\right)^2 - 1\right)^2,$$

where $\{s_j\}_{j=1}^N$ is a set of equidistant collocation points in the interval $[0, T]$. The parameters $\lambda_{\mathrm{BC}}, \lambda_{\mathrm{unit}}$ balance the relative contributions of the constraint penalties.

In the online implementation we aimed to construct an elastica satisfying the constraints

$$c(0) = (0, 0), \quad c(5) = (3, 3), \quad v_A = (1, 0), \quad v_B = (-1, 0).$$

The curve is supposed to be parametrized by arc length and have the length $T = 5$. The code can be found at `https://tinyurl.com/2krsedzx`

The network employed consists of three hidden layers with 64 neurons each, using the `tanh` activation function, and was trained for 4,000 epochs. The results are shown in Fig. 4.12(a). While the model captures the overall behavior, it fails to match the boundary velocities with sufficient accuracy. As indicated in Fig. 4.12(b), the loss has already reached a saturation level, so additional training does not improve the results. To address this limitation, a more suitable architecture is required, which we discuss in the next section.

Hard Embedding A more effective approach is to use a *hard embedding*, which enforces the boundary conditions exactly and avoids soft penalties for endpoint position and tangent constraints. This strategy has several advantages: for the same network architecture and number of training epochs it typically trains substantially faster (often by a factor of four) and achieves higher accuracy.

Let $A, B \in \mathbb{R}^2$, let $v_A, v_B \in \mathbb{R}^2$ be unit vectors, and let $T > 0$ be given, with $T > \mathrm{dist}(A, B)$. First we construct a *baseline* curve $c_0 : [0, T] \to \mathbb{R}^2$ that satisfies the boundary data

$$c_0(0) = A, \qquad c_0(T) = B, \qquad \dot{c}_0(0) = v_A, \qquad \dot{c}_0(T) = v_B.$$

Then define the trainable curve by

$$c_\theta(s) \;=\; c_0(s) \;+\; \phi(s)\,\mathrm{nn}_\theta(s), \qquad \phi(s) = s^2(T - s)^2,$$

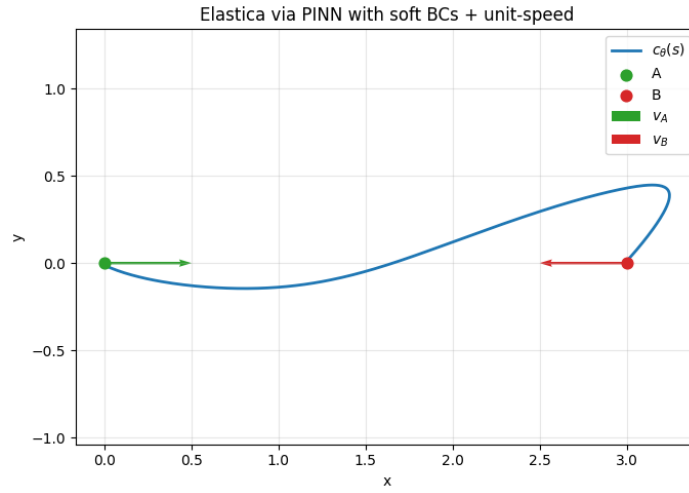
where $\mathrm{nn}_\theta : [0, T] \to \mathbb{R}^2$ is a unconstrained feed-forward neural network. Because $\phi(0) = \phi(T) = 0$ and $\phi'(0) = \phi'(T) = 0$, the embedding preserves both position and velocity at the endpoints:

$$c_\theta(0) = A, \quad c_\theta(T) = B, \quad \dot{c}_\theta(0) = v_A, \quad \dot{c}_\theta(T) = v_B.$$

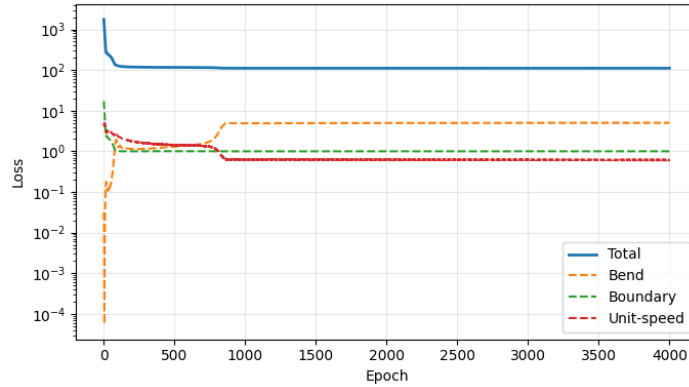
Thus, the endpoint constraints are enforced *exactly* for every value of the parameters θ.

The network is then trained to minimize only the interior objectives,

$$\mathcal{L}_{\mathrm{total}}(\theta) \;=\; \mathcal{L}_{\mathrm{bend}}(\theta) \;+\; \lambda_{\mathrm{unit}}\,\mathcal{L}_{\mathrm{unit}}(\theta),$$



(a)



(b)

Figure 4.12: (a) The elastica curve should satisfy $c(0) = (0, 0)$, $c(5) = (3, 3)$, $v_A = (1, 0)$, $v_B = (-1, 0)$; (b) The associated losses reach perfect saturation.

where $\mathcal{L}_{\text{bend}}(\theta) \approx \displaystyle\int_0^T \|\ddot{c}_\theta(s)\|^2\, ds$ is the bending-energy, and

$$\mathcal{L}_{\text{unit}}(\theta) \;\approx\; \int_0^T \big(\,\|\dot{c}_\theta(s)\|^2 - 1\,\big)^2\, ds$$

is a soft penalty promoting unit speed along the curve. No boundary penalties are needed, since the hard embedding already guarantees the endpoint conditions.

Constructing the baseline c_0. One convenient choice is the cubic Hermite interpolant. With $u = s/T \in [0,1]$, set

$$\begin{aligned} h_{00}(u) &= 2u^3 - 3u^2 + 1, & h_{10}(u) &= u^3 - 2u^2 + u, \\ h_{01}(u) &= -2u^3 + 3u^2, & h_{11}(u) &= u^3 - u^2. \end{aligned}$$

Define

$$c_0(s) \;=\; h_{00}(u)\,A \;+\; h_{10}(u)\,(Tv_A) \;+\; h_{01}(u)\,B \;+\; h_{11}(u)\,(Tv_B), \qquad u = \frac{s}{T}.$$

We can show by straightforward calculation that $c_0(0) = A$, $c_0(T) = B$, $\dot{c}_0(0) = v_A$, and $\dot{c}_0(T) = v_B$. To this end, we compute the basis values at the endpoints for h_{ij} and h'_{ij} as follows:

u	h_{00}	h_{10}	h_{01}	h_{11}
0	1	0	0	0
1	0	0	1	0

u	h'_{00}	h'_{10}	h'_{01}	h'_{11}
0	0	1	0	0
1	0	0	0	1

where $h'_{00}(u) = 6u^2 - 6u$, $h'_{10}(u) = 3u^2 - 4u + 1$, $h'_{01}(u) = -6u^2 + 6u$, $h'_{11}(u) = 3u^2 - 2u$. Evaluating c_0 at $s = 0$ (or $u = 0$) and $s = T$ (or $u = 1$) gives

$$c_0(0) = A, \qquad c_0(T) = B.$$

For the velocities, by the chain rule

$$\dot{c}_0(s) = \frac{dc_0}{ds} = \frac{1}{T}\,\frac{dc_0}{du} = \frac{1}{T}\Big(h'_{00}(u)\,A + h'_{10}(u)\,(Tv_A) + h'_{01}(u)\,B + h'_{11}(u)\,(Tv_B)\Big).$$

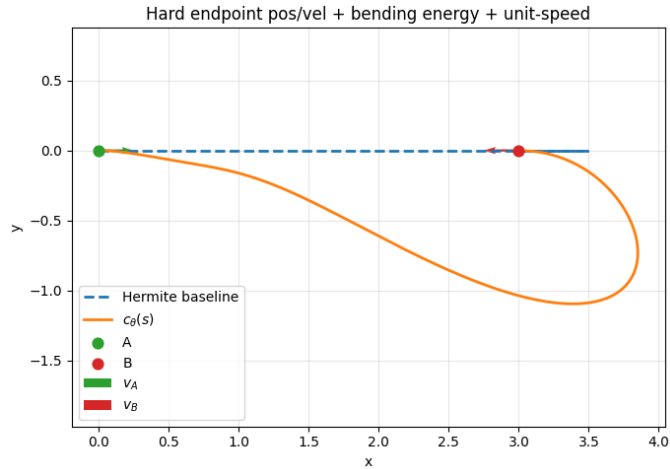
Using the derivative table at $u = 0$ and $u = 1$,

$$\dot{c}_0(0) = v_A, \qquad \dot{c}_0(T) = v_B.$$

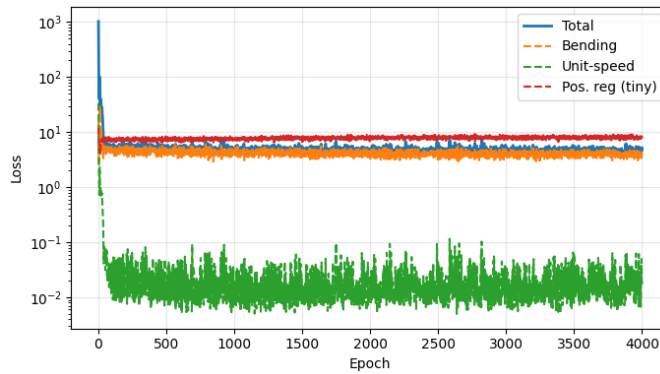
Hence the Hermite baseline enforces $c_0(0) = A$, $c_0(T) = B$, $\dot{c}_0(0) = v_A$, $\dot{c}_0(T) = v_B$ exactly. The Hermite baseline is represented in Fig. 4.13(a) by the blue dashed curve.

On the top of the baseline c_0 we add a correction term, $\phi(s)\,\text{nn}_\theta(s)$, which changes the curve inside, leaving its boundary unaffected, such as the bending energy is minimized.

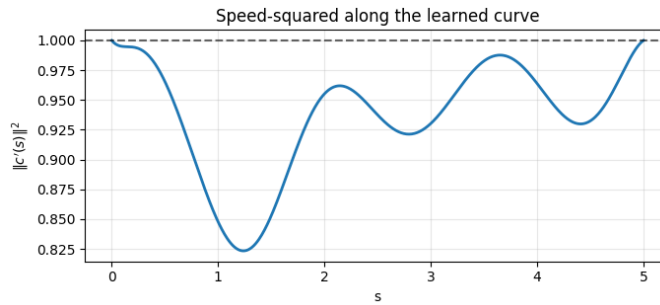
We used a feedforward architecture with three hidden layers of 64 neurons each, with `tanh` activation. We trained for $4,000$ epochs. The results are represented in Fig. 4.13. See the code at `https://tinyurl.com/2krsedzx`



(a)



(b)



(c)

Figure 4.13: (a) The elastica curve satisfying $c(0) = (0, 0)$, $c(5) = (3, 3)$, $v_A = (1, 0)$, $v_B = (-1, 0)$; (b) The associated losses — we note that the soft-constraint loss is two orders of magnitude smaller than the other losses; (c) The length of the velocity is not always equal to one.

More Hard Embedding The previous hard embedding does not incorporate the unit-speed property of the curve, which is therefore handled as a soft constraint. We note that the condition $\|\dot{c}(s)\| = 1$ is fundamentally different in nature from boundary conditions. Boundary constraints are *pointwise*, namely they only involve the values of $c(s)$ and $\dot{c}(s)$ at the endpoints, and can be enforced exactly by multiplying the free network output with shaping functions $\phi(s)$ that vanish at the boundaries.

In contrast, the unit-speed condition is a *differential* and a *global* constraint, requiring validity at every point s. This makes it of ODE-type, fundamentally harder to encode as a hard embedding. We now turn to the construction of such a curve. Let

$$A = (x_A, y_A), \quad B = (x_B, y_B), \quad v_A = (\cos\alpha, \sin\alpha), \quad v_B = (\cos\beta, \sin\beta).$$

Denote by $\varphi(s)$ the angle made by the velocity vector $\dot{c}(s)$ with the horizontal axis, so that

$$\dot{c}(s) = (\cos\varphi(s),\, \sin\varphi(s)).$$

This automatically enforces the unit-speed condition $\|\dot{c}(s)\| = 1$.

We seek a curve $c(s)$ that minimizes the bending energy

$$E[c] \;=\; \int_0^T \|\ddot{c}(s)\|^2\, ds \;=\; \int_0^T \big(\dot{\varphi}(s)\big)^2\, ds,$$

subject to the boundary conditions

$$c(0) = A, \quad c(T) = B, \quad \varphi(0) = \alpha, \quad \varphi(T) = \beta.$$

We continue our construction by introducing a one-dimensional free neural network $\rho_\theta(s)$ and define

$$\varphi_\theta(s) = \frac{T-s}{T}\,\alpha + \frac{s}{T}\,\beta + s(T-s)\,\rho_\theta(s).$$

Clearly, this parametrization ensures the boundary conditions $\varphi_\theta(0) = \alpha$ and $\varphi_\theta(T) = \beta$. To enforce the unit-speed condition, we set

$$\dot{c}_\theta(s) = \big(\cos\varphi_\theta(s),\, \sin\varphi_\theta(s)\big).$$

By integration, the curve takes the form

$$c_\theta(s) = \left(x_A + \int_0^s \cos\varphi_\theta(u)\, du,\; y_A + \int_0^s \sin\varphi_\theta(u)\, du\right).$$

This construction guarantees the conditions

$$c_\theta(0) = A, \quad \dot{c}_\theta(0) = v_A, \quad \dot{c}_\theta(T) = v_B.$$

However, the final boundary condition $c_\theta(T) = B$ must be enforced as

$$\begin{aligned} x_A + \int_0^T \cos\varphi_\theta(u)\, du &= x_B, \\ y_A + \int_0^T \sin\varphi_\theta(u)\, du &= y_B. \end{aligned}$$

Since these conditions are not automatically satisfied, they have to be added in the loss as soft constraints.

Next, we note that

$$\ddot{c}_\theta(s) = \big(-\sin\varphi_\theta(s)\,\dot{\varphi}_\theta(s),\ \cos\varphi_\theta(s)\,\dot{\varphi}_\theta(s)\big),$$

so that

$$\|\ddot{c}_\theta(s)\|^2 = \dot{\varphi}_\theta(s)^2.$$

Therefore, the bending loss becomes

$$\mathcal{L}_{\text{bend}}(\theta) = \int_0^T \|\ddot{c}_\theta(s)\|^2\, ds = \int_0^T \dot{\varphi}_\theta(s)^2\, ds.$$

The final condition is then enforced by minimizing the loss

$$\mathcal{L}_{\text{final}}(\theta) = \left(\int_0^T \cos\varphi_\theta(u)\, du - (x_B - x_A) \right)^2 + \left(\int_0^T \sin\varphi_\theta(u)\, du - (y_B - y_A) \right)^2.$$

The total loss is thus the combination of the previous two losses:

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{bend}}(\theta) + \lambda\, \mathcal{L}_{\text{final}}(\theta).$$

We conclude that three boundary conditions (initial position and the initial and final velocities) as well as the bending energy are successfully hard embedded into the model. The final position constraint, however, cannot be hard embedded and is therefore included as a soft constraint.

We used the same feedforward architecture with three hidden layers of 64 neurons each, employing `tanh` activation, and trained it for 4,000 epochs. The code is available at `https://tinyurl.com/2krsedzx`.

The resulting elastica curve is shown in Fig. 4.14(a) and appears quite accurate. Figure 4.14(b) displays the evolution of the argument angle $\varphi(s)$ as a function of s. Even with a hard embedding of the unit-speed condition, the speed is not identically equal to 1, as illustrated in Fig. 4.14(c). The losses saturate rather quickly, see Fig. 4.14(d). The final position loss, which was added as a soft constraint, is 2–3 orders of magnitude smaller than the other losses and exhibits greater noise (relative to its size).

Remark 4.5.1 There is a close analogy between elastica curves and the simple pendulum equation. Let $c : [0, T] \to \mathbb{R}^2$, $c(s) = (x(s), y(s))$, be a C^2-plane curve parametrized by arclength s, so that $\|\dot{c}(s)\| = 1$ for all $s \in [0, T]$. The unit tangent is

$$\dot{c}(s) = \big(\cos\varphi(s),\, \sin\varphi(s)\big),$$

where $\varphi(s)$ is the argument angle. The curvature is $\kappa(s) = \dot{\varphi}(s)$.

Scaling the Euler–Bernoulli bending energy by a factor of $1/2$ we get

$$E[c] \;=\; \frac{1}{2} \int_0^T \dot{\varphi}(s)^2\, ds,$$

which resembles a total kinetic energy. To enforce that the endpoints are fixed, we note that

$$c(L) - c(0) \;=\; \int_0^T \dot{c}(s)\, ds \;=\; \Big(\int_0^T \cos\varphi(s)\, ds,\; \int_0^T \sin\varphi(s)\, ds \Big).$$

Introducing Lagrange multipliers $\lambda_1, \lambda_2 \in \mathbb{R}$ for these two integral constraints, we consider the augmented functional

$$\mathcal{J}[\varphi] \;=\; \frac{1}{2}\int_0^T \dot{\varphi}(s)^2\, ds \;+\; \lambda_1 \int_0^T \cos\varphi(s)\, ds \;+\; \lambda_2 \int_0^T \sin\varphi(s)\, ds.$$

We can write $\mathcal{J}[\varphi] \;=\; \int_0^T L(\varphi(s), \dot{\varphi}(s))\, ds$, with the Lagrangian

$$\begin{aligned} L(\varphi(s), \dot{\varphi}(s)) \;&=\; \frac{1}{2}\dot{\varphi}(s)^2 \;+\; \lambda_1 \cos\varphi(s) \;+\; \lambda_2 \sin\varphi(s) \\ &=\; \frac{1}{2}\dot{\psi}(s)^2 + |\lambda| \sin\psi(s), \end{aligned}$$

where, $\psi(s) \;=\; \varphi(s) + \psi_0$ and we used notations $|\lambda| \;=\; \sqrt{\lambda_1^2 + \lambda_2^2}$ and choose $\psi_0 \;=\; \tan^{-1}(\lambda_1/\lambda_2)$. We have obtained the Lagrangian for a simple pendulum, where $\psi(s)$ is the angle and s is time. The constant $|\lambda|$ is the analog of the quotient g/ℓ. The corresponding Euler–Lagrange equation is

$$\ddot{\psi} + |\lambda| \sin\psi = 0$$

subject to the boundary conditions

$$\psi(0) = \alpha + \tan^{-1}\frac{\lambda_1}{\lambda_2}, \quad \psi(T) = \beta + \tan^{-1}\frac{\lambda_1}{\lambda_2}.$$

The reader may consult Singer's lecture notes [85] for a theoretical introduction to elastic curves and rods. Elastica in Euclidean space $\mathbb{R}^n$ are investigated in Dziuk et al. [33], while Jurdjevic [51] develops the theory of non-Euclidean elastica. The geometry of elastica in the complex projective plane is explored in Barros et al. [9].

4.6 Exercises

Exercise 4.6.1 Find the catenary curve connecting the points $(0, 6)$ and $(5, 4)$ using a one-dimensional neural network $f_\theta : [0, 5] \to \mathbb{R}$. Assume that the curve is of the form $(x, y_\theta(x))$, where $y_\theta(x)$ satisfies the fixed boundary conditions and minimizes the action associated with the Lagrangian

$$L(y, y') = y\sqrt{1 + (y')^2}.$$

Train the neural network to approximate the minimizing curve by discretizing the domain, computing the action integral using automatic differentiation, and optimizing the network parameters θ accordingly.

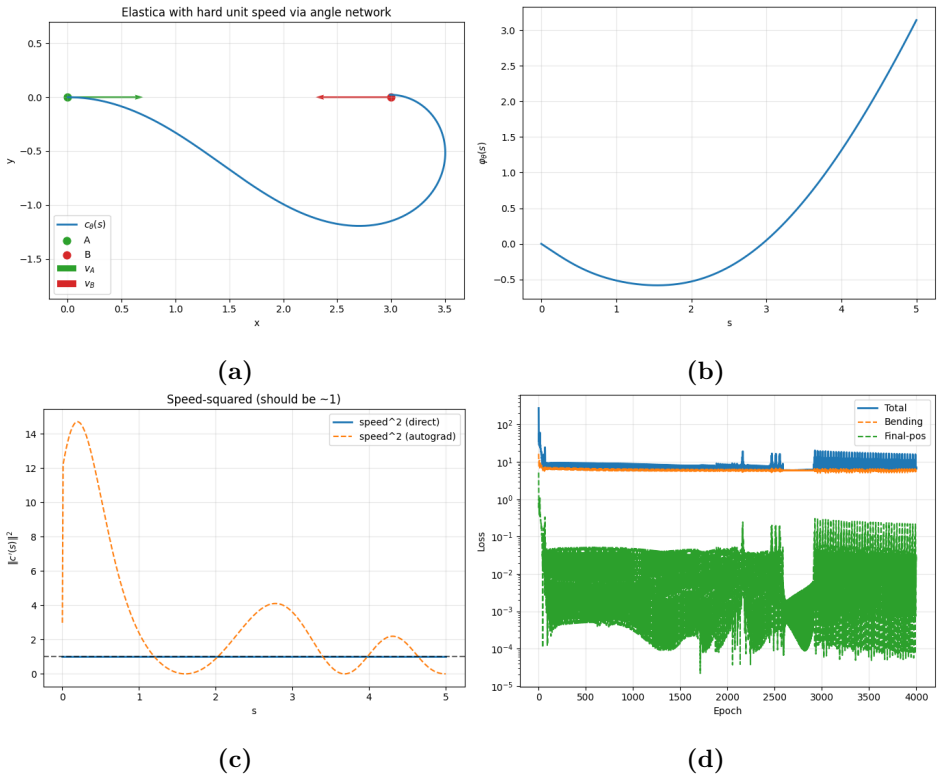


Figure 4.14: (a) Elastica curve satisfying the required boundary conditions; (b) The graph of the argument angle $\varphi(s)$ versus arc length s; (c) The graph of $\|\dot{c}(s)\|^2$ versus s; (d) The associated loss functions.

Exercise 4.6.2 (Electrostatic potential minimizing curve) (*i*) Find the curve connecting the points $(0, 6)$ and $(5, 4)$ that minimizes the electric potential

$$U(y) = \frac{1}{y},$$

using a two-dimensional neural network $(f_\theta(t), g_\theta(t))$ to represent the parametric curve $(x_\theta(t), y_\theta(t))$.

(*ii*) Then, show that the resulting curve is an arc of a circle centered on the x-axis.

Exercise 4.6.3 A cycloid satisfies the nonlinear differential equation

$$1 + y'(x)^2 = \frac{K^2}{y(x)}, \qquad y(0) = 1.$$

Use a neural network-based approach to approximate a solution of the form

$$y_\theta(x) = 1 + x \cdot \mathrm{NN}(x; \theta),$$

where $\mathrm{NN}(x; \theta)$ is a feed-forward neural network with parameters θ. Train the network so that $y_\theta(x)$ approximately satisfies the differential equation over a suitable interval.

Exercise 4.6.4 (*a*) Use a one-dimensional neural network to approximate the minimizer of the variational problem

$$T[y] \;=\; \int_0^1 \frac{\sqrt{1 + \big(y'(x)\big)^2}}{\sqrt{2\,y(x)}}\,\mathrm{d}x, \qquad y(0) = 1,\ y(1) = 0.$$

(*b*) Implement the boundary conditions $y(0) = 1$, $y(1) = 0$ by an appropriate reparameterization of the network output, discretize the integral via a Riemann sum, and train the network s to minimize $T[y]$.

(*c*) Compare your learned curve with the exact arc of the cycloid that passes through $(0, 1)$ and $(1, 0)$.

Exercise 4.6.5 (*a*) Use a one-dimensional neural network to approximate the minimizer of the variational problem

$$S[y] \;=\; \int_0^1 \sqrt{y(x)}\,ds = \int_0^1 \sqrt{y(1 + y'(x)^2)}\,dx \qquad y(0) = 1,\ y(1) = 2.$$

(*b*) Compare your learned curve with an arc of a parabola that passes through $(0, 1)$ and $(1, 0)$. Find the coefficients of this parabola.

Exercise 4.6.6 (*a*) Consider the variational problem

$$\min_y \int_{x_0}^{x_1} U\big(y(x)\big)\,\sqrt{1 + y'(x)^2}\,dx, \quad y(x_0) = y_0,\ y(x_1) = y_1.$$

Describe how to parameterize $y(x)$ with a feed-forward neural network $y_\theta(x)$ that exactly enforces the boundary conditions, and formulate the loss function whose minimization approximates the above integral via a Riemann or trapezoidal rule.

(*b*) Show that the associated Euler–Lagrange equation can be written in the form

$$y''(x) \;=\; \frac{U'\big(y(x)\big)}{U\big(y(x)\big)}\,\big(1 + y'(x)^2\big).$$

(*c*) Demonstrate that this ODE admits the conservation law

$$U\big(y(x)\big)^2\big(1 + y'(x)^2\big) = K^2,$$

and explain how one can recover both the constant K and the curve $y(x)$ by training the network to minimize the variance of $q(x) = U\big(y_\theta(x)\big)^2\big(1 + y_\theta'(x)^2\big)$.

Exercise 4.6.7 Implement the PINN (Euler–Lagrange) loss for the simple pendulum and train for $1,000$ epochs using each of the four learning rates

$$\alpha \in \{10^{-1},\, 10^{-2},\, 10^{-3},\, 10^{-4}\}.$$

For each α plot the total training loss versus epoch and compare the oscillatory behavior of the loss curves for the different α, and verify that smaller learning rates produce smaller variance (i.e. more stable convergence).

Chapter 5

Hamiltonian Systems

5.1 Hamiltonian Functions

In Hamiltonian mechanics, the evolution of a dynamical system with n degrees of freedom is described in the phase space by the curve $t \to (q(t), p(t)) \in \mathbb{R}^n \times \mathbb{R}^n$, where $q(t) = (q^1(t), \cdots, q^n(t))$ and $p(t) = (p^1(t), \cdots, p^n(t))$ denote the generalized positions and the conjugate momenta at time t. The trajectory $(q(t), p(t))$ can be derived from a *Hamiltonian function*

$$H \colon \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R},$$

as a solution of the *Hamiltonian system* of equations

$$\dot{q}(t) = \frac{\partial H}{\partial p}\big(q(t), p(t)\big) \tag{5.1.1}$$

$$\dot{p}(t) = -\frac{\partial H}{\partial q}\big(q(t), p(t)\big). \tag{5.1.2}$$

Because the phase space has dimension $2n$, one must prescribe $2n$ independent conditions, for example

$$q(0) = q_0, \qquad p(0) = p_0 \quad \text{(initial-value problem)}$$

or

$$q(0) = q_0, \qquad q(T) = q_T \quad \text{(two-point boundary problem)}$$

in order to determine a solution.

The classical Hamiltonian For a particle of unit mass moving in a smooth potential $U : \mathbb{R}^n \to \mathbb{R}$ the Hamiltonian is

$$H(q, p) = \tfrac{1}{2}\|p\|^2 + U(q),$$

where $\|p\| = (p_1^2 + \cdots + p_n^2)^{1/2}$. In this case, the Hamiltonian system of equations reduces to

$$\dot{q}(t) = p(t) \tag{5.1.3}$$

$$\dot{p}(t) = -\nabla U\big(q(t)\big), \tag{5.1.4}$$

which reproduce Newton's second law, $\ddot{q}(t) = -\nabla U\big(q(t)\big)$, in phase-space form.

Here, we adopted the notations that an overdot denotes differentiation with respect to time, and ∇U is the gradient of U with respect to q.

Solutions existence and uniqueness Before employing deep-learning methods to approximate solutions of Hamilton's equations, we must first ensure that classical solutions exist and are unique for the prescribed boundary conditions. For instance, it is well known that when the potential $U(q)$ is constant, linear, or quadratic, the associated Hamiltonian system possesses a unique global solution.

For the initial-value problem this claim follows from the Picard–Lindelöf theorem. In the one-dimensional case ($n = 1$) let

$$U(q) = aq^2 + bq + c, \qquad a, b, c \in \mathbb{R}.$$

Then the vector field of Hamilton's equations is globally Lipschitz because

$$\left\| \frac{\partial^2 H}{\partial q \, \partial p} \right\|_F \;=\; \left| U''(q) \right| \;=\; 2|a|, \qquad \forall q \in \mathbb{R}.$$

Hence, a unique solution exists for all time, and analogous arguments extend to two-point boundary problems.
Uniqueness can fail for higher order powers, such as in the case of the *quartic oscillator*, whose potential is $U(q) = q^4$.

The Lagrangian-induced Hamiltonian A Hamiltonian can also be obtained from a Lagrangian $L(q, \dot{q})$ that is quadratic in the velocity $\dot{q}$. To this end, define the momentum

$$p = \frac{\partial L}{\partial \dot{q}},$$

and treat this relation as an implicit equation for $\dot{q} = \dot{q}(p)$. Substituting that $\dot{q}$ into the Legendre transform gives

$$H(q, p) \;=\; \langle p, \dot{q} \rangle - L(q, \dot{q}). \tag{5.1.5}$$

Hamilton's equations can be checked directly as follows.

$$\frac{\partial H}{\partial p} \;=\; \dot{q}, \qquad \frac{\partial H}{\partial q} \;=\; -\frac{\partial L}{\partial q} \;=\; -\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}}\right) \;=\; -\dot{p},$$

where in the last identity we used the Euler–Lagrange equation for L.

For example, for the classical Lagrangian

$$L(q, \dot{q}) = \tfrac{1}{2}\|\dot{q}\|^2 - U(q),$$

we have $p = \dot{q}$. The corresponding Hamiltonian is

$$H(q, p) = \langle p, \dot{q} \rangle - L(q, \dot{q}) = \langle p, p \rangle - \left(\tfrac{1}{2}\|p\|^2 - U(q)\right) = \tfrac{1}{2}\|p\|^2 + U(q),$$

which is precisely the classical mechanical Hamiltonian.

Conversely, if you start with a quadratic Hamiltonian, you can recover the corresponding Lagrangian simply by applying the Legendre transform.

5.2 Neural-network Approximation

Assume the Hamiltonian system, together with its boundary conditions, has a unique solution $(q(t), p(t))$. We approximate this trajectory with a feed-forward neural network

$$f_\theta(t) \;=\; \big(q_\theta(t),\, p_\theta(t)\big),$$

whose input is the scalar time t and whose output is the two-dimensional phase-space vector.

Loss Functions Training the network $f_\theta(t)$ minimises a composite loss built from *boundary-matching* and *physics* terms.
We have two boundary-matching terms:

Boundary loss To enforce the two-point boundary conditions $q(0) = q_0$ and $q(T) = q_T$, we consider

$$\mathcal{L}_{\mathrm{bd}}(\theta) = \big\|q_\theta(0) - q_0\big\|^2 + \big\|q_\theta(T) - q_T\big\|^2.$$

The boundary loss can be eliminated by *embedding* the endpoint data directly into the network:

$$q_\theta(t) = \frac{T-t}{T}\, q_0 + \frac{t}{T}\, q_T + t(T-t)\,\mathrm{nn}^1(\theta; t),$$

$$p_\theta(t) = \frac{T-t}{T}\, p_0 + \frac{t}{T}\, p_T + t(T-t)\,\mathrm{nn}^2(\theta; t),$$

where $\mathrm{nn}(\theta; t) \;=\; \big(\mathrm{nn}^1(\theta; t), \mathrm{nn}^2(\theta; t)\big)$ is a feed-forward neural network with a two-dimensional output.

Initial-condition loss For initial-value problems with prescribed $q(0) = q_0$ and $p(0) = p_0$, we take

$$\mathcal{L}_{\mathrm{IC}}(\theta) = \big\|q_\theta(0) - q_0\big\|^2 + \big\|p_\theta(0) - p_0\big\|^2.$$

Similarly, this loss can be avoided if we consider the embedding

$$q_\theta(t) = q_0 + t\,\mathrm{nn}^1(\theta; t),$$

$$p_\theta(t) = p_0 + t\,\mathrm{nn}^2(\theta; t),$$

Remark By embedding the boundary conditions directly into the neural network and eliminating the boundary-loss term, the training time is reduced.

The Physics (ODE) loss This is obtained by taking a partition of the interval $[0, T]$ into m equal sub-intervals with collocation points $t_j = jT/m$ $(j = 1, \ldots, m)$. Using automatic differentiation to obtain the time derivatives $\dot{q}_\theta(t)$ and $\dot{p}_\theta(t)$, we penalise the residual of Hamilton's equations as

$$\mathcal{L}_{\mathrm{ODE}}(\theta) = \frac{1}{m}\sum_{j=1}^{m}\left\|\dot{q}_\theta(t_j) - \frac{\partial H}{\partial p}\big(q_\theta(t_j),\, p_\theta(t_j)\big)\right\|^2$$

$$+\, \frac{1}{m}\sum_{j=1}^{m}\left\|\dot{p}_\theta(t_j) + \frac{\partial H}{\partial q}\big(q_\theta(t_j),\, p_\theta(t_j)\big)\right\|^2.$$

To obtain the total loss, we combine the physics term with the appropriate boundary term, weighted by a hyper-parameter $\lambda > 0$ as

$$\mathcal{L}_{\text{total}}(\theta) = \begin{cases} \mathcal{L}_{\text{ODE}}(\theta) + \lambda\, \mathcal{L}_{\text{IC}}(\theta), & \text{(initial-value problems)}, \\ \mathcal{L}_{\text{ODE}}(\theta) + \lambda\, \mathcal{L}_{\text{bd}}(\theta), & \text{(boundary-value problems)}. \end{cases}$$

The coefficient λ is typically tuned so that the data and physics terms contribute comparably during training. For instance, if there are $m = 1,000$ collocation points and the boundary loss has just two terms, a good start is to consider $\lambda = 500$.

We will apply this method to several important cases.

5.3 The Harmonic Oscillator

For the one-dimensional harmonic oscillator the potential energy is $U(q) = \frac{1}{2}\, q^2$, so the Hamiltonian function takes the form

$$H(q,p) = \tfrac{1}{2}\, p^2 + \tfrac{1}{2}\, q^2.$$

With initial conditions $q(0) = q_0$ and $p(0) = p_0$, the Hamiltonian equations

$$\begin{aligned} \dot{q}(t) &= \frac{\partial H}{\partial p} = p(t) \\ \dot{p}(t) &= -\frac{\partial H}{\partial q} = -\,q(t), \end{aligned}$$

have an exact solution given by the circular curve

$$\begin{aligned} q(t) &= q_0 \cos t + p_0 \sin t \\ p(t) &= p_0 \cos t - q_0 \sin t. \end{aligned}$$

We shall show now that a neural network can learn the aforementioned solution. To this end, consider a feed-forward neural network $f_\theta(t) = \big(q_\theta(t),\, p_\theta(t)\big)$, whose output approximates $q(t)$ (position) and $p(t)$ (momentum). In this case the ODE loss in its continuous integral form and discrete form is given by

$$\begin{aligned} \mathcal{L}_{\text{ODE}}(\theta) \;&=\; \int_0^T \Big[\big(\dot{p}_\theta(t) + q_\theta(t)\big)^2 + \big(\dot{q}_\theta(t) - p_\theta(t)\big)^2 \Big]\, dt \\ &\approx \frac{1}{m} \sum_{j=1}^{m} \big(\, \dot{p}_\theta(t_j) + q_\theta(t_j)\big)^2 + \frac{1}{m} \sum_{j=1}^{m} \big(\, \dot{q}_\theta(t_j) - p_\theta(t_j)\big)^2. \end{aligned}$$

Using the initial-value conditions, we form the loss

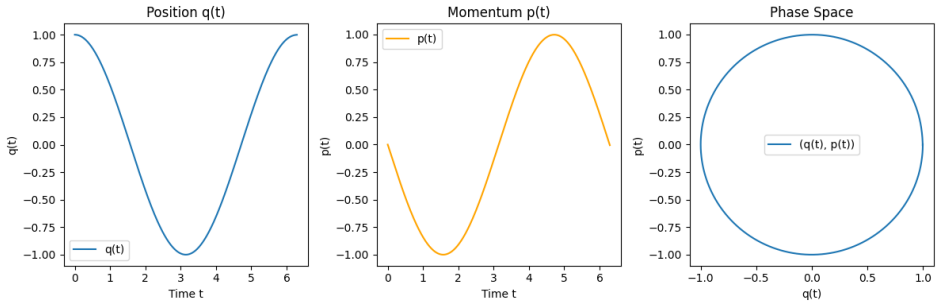
$$\mathcal{L}_{\text{IC}}(\theta) = \big\| q_\theta(0) - q_0 \big\|^2 + \big\| p_\theta(0) - p_0 \big\|^2,$$

and consider the total loss

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{ODE}}(\theta) + \lambda\, \mathcal{L}_{\text{IC}}(\theta), \qquad \lambda > 0.$$

Minimising $\mathcal{L}_{\text{total}}$ drives the network output $\big(q_\theta(t), p_\theta(t)\big)$ toward the exact solution of the harmonic oscillator while satisfying the prescribed initial or boundary conditions.

The experiment employs $m = 1,000$ collocation points on the interval $[0, 6.29]$ (approximately 2π). With initial conditions $q_0 = 1$ and $p_0 = 0$, we train a feed-forward network consisting of three hidden layers, each with 64 tanh neurons, followed by a two-component output layer. After $5,000$ training epochs the network reproduces the position curve $q(t)$, the momentum curve $p(t)$, and the phase-space trajectory $\big(q(t), p(t)\big)$, the last of which traces the expected circular orbit, see Fig. 5.1.



(a) **(b)** **(c)**

Figure 5.1: (a) The position curve, $q(t)$; (b) The momentum curve, $p(t)$; (c) The phase-space trajectory, $(q(t), p(t))$, which is a circle.

The detailed code is provided at:

`https://tinyurl.com/67bsucj5`

5.4 The Simple Pendulum

The dynamics of a simple planar pendulum of unit mass and unit length is governed by the Hamiltonian

$$H(q,p) = \frac{1}{2}\,p^2 - \kappa\cos q, \qquad \kappa > 0.$$

This expression follows from the Legendre transform of the Lagrangian (see Eq. (4.4.10))

$$L(q,\dot{q}) = \frac{1}{2}\,\dot{q}^2 + \kappa\cos q,$$

where q (often written φ) denotes the angular displacement.

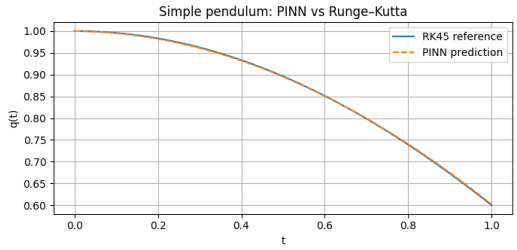
First compute the canonical momentum

$$p = \frac{\partial L}{\partial \dot{q}} = \dot{q}.$$

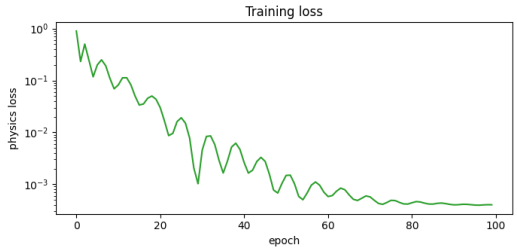
Substituting $\dot{q} = p$ into the Legendre transform $H = p\,\dot{q} - L$ gives

$$H(q,p) = p\dot{q} - L(q,\dot{q}) = \frac{1}{2}\,p^2 - \kappa\cos q,$$

which is the Hamiltonian previously stated.



(a)



(b)

Figure 5.2: (a) The comparison between the true solution given by Runge–Kutta method to the simple pendulum equation and the neural network solution for $q(t)$, with $t \in [0, 1]$. (b) The corresponding total loss.

The Hamiltonian system is

$$\dot{q} = p, \qquad \dot{p} = -\kappa \sin q,$$

together with the initial conditions $q(0) = q_0$ and $p(0) = p_0$.

While solving the system using neural networks, we embed the initial data directly into the network by setting

$$q_\theta(t) = q_0 + t\,\mathrm{nn}^1(\theta; t),$$
$$p_\theta(t) = p_0 + t\,\mathrm{nn}^2(\theta; t),$$

where $f_\theta(t) = \big(\mathrm{nn}^1(\theta; t), \mathrm{nn}^2(\theta; t)\big)$ is a feed-forward neural network with two outputs. The total (physics-informed) loss is

$$\mathcal{L}_{\text{total}}(\theta) = \frac{1}{m}\sum_{j=1}^{m}\Big(\dot{q}_\theta(t_j) - p_\theta(t_j)\Big)^2 + \frac{1}{m}\sum_{j=1}^{m}\Big(\dot{p}_\theta(t_j) + \kappa \sin q_\theta(t_j)\Big)^2,$$

and we minimise $\mathcal{L}_{\text{total}}$ with respect to the parameters θ.

With the initial values fixed at $q_0 = 1$ and $p_0 = 0$, the network was trained for only 100 epochs yet reached an excellent final loss of 4×10^{-4}, making the PINN output virtually indistinguishable from the Runge–Kutta reference solution, see Fig. 5.2(a).

The network architecture consists of three hidden layers, each with 64 neurons and tanh activation, followed by a two-component output layer. The network converges in under 100 epochs while preserving high accuracy, see Fig. 5.2(b).
The full code is available at:

`https://tinyurl.com/4h4tzw88`

5.5 Charged Particles in a Constant Magnetic Field

The following Hamiltonian describes the dynamics of a charged particle in a purely magnetic field

$$H(q,p) = \frac{1}{2}\left\|p - e\,A(q)\right\|^2,$$

where $A(q) = (A^1, A^2, A^3)$ is a vector gauge on $\mathbb{R}^3$ giving rise to the magnetic field $B = \operatorname{curl} A$. Choosing $m = e = 1$ and

$$A(q) = \Big(-\tfrac{b}{2}\,q_2,\ \tfrac{b}{2}\,q_1,\ 0\Big), \qquad b > 0,$$

we obtain the constant magnetic field $B = (0, 0, b)$. Then the Hamiltonian becomes the following sum of squares

$$H(q,p) = \frac{1}{2}\Big(p_1 + \tfrac{b}{2}q_2\Big)^2 + \frac{1}{2}\Big(p_2 - \tfrac{b}{2}q_1\Big)^2. \tag{5.5.6}$$

The Hamiltonian system is

$$\begin{aligned}
\dot{q}_1 &= \frac{\partial H}{\partial p_1} = p_1 + \frac{b}{2}q_2 \\
\dot{q}_2 &= \frac{\partial H}{\partial p_2} = p_2 - \frac{b}{2}q_1 \\
\dot{p}_1 &= -\frac{\partial H}{\partial q_1} = \frac{b}{2}(p_2 - \frac{b}{2}q_1) = \frac{b}{2}\dot{q}_2 \\
\dot{p}_2 &= -\frac{\partial H}{\partial q_2} = -\frac{b}{2}(p_1 + \frac{b}{2}q_2) = -\frac{b}{2}\dot{q}_1.
\end{aligned}$$

Differentiating the first two equations and using the last two we eliminate the momenta and obtain second order ODEs for q_1 and q_2

$$\begin{aligned}
\ddot{q}_1 &= \dot{p}_1 + \frac{b}{2}\dot{q}_2 = b\dot{q}_2 \\
\ddot{q}_2 &= \dot{p}_2 - \frac{b}{2}\dot{q}_1 = -b\dot{q}_1.
\end{aligned}$$

These equations can be written equivalently as $\ddot{q} = bJ\dot{q}$, where $J = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$. The equation is the analog of Lorentz's equation. The solution can be expressed in terms

of the initial conditions $q(0)$ and $\dot{q}(0)$ as follows[1]

$$(q_1(t), q_2(t)) = e^{bJt}K + C = R_{bt}\,K + C,$$

where

$$R_\phi = \begin{pmatrix} \cos\phi & \sin\phi \\ -\sin\phi & \cos\phi \end{pmatrix},$$

$$K = \frac{1}{b}\,J^{-1}\begin{pmatrix} \dot{q}_1(0) \\ \dot{q}_2(0) \end{pmatrix}, \qquad C = \begin{pmatrix} q_1(0) \\ q_2(0) \end{pmatrix} - K.$$

Hence, the trajectory $(q_1(t), q_2(t))$ is a circle centered at C with radius

$$\|K\| = \frac{\|\dot{q}(0)\|}{b}.$$

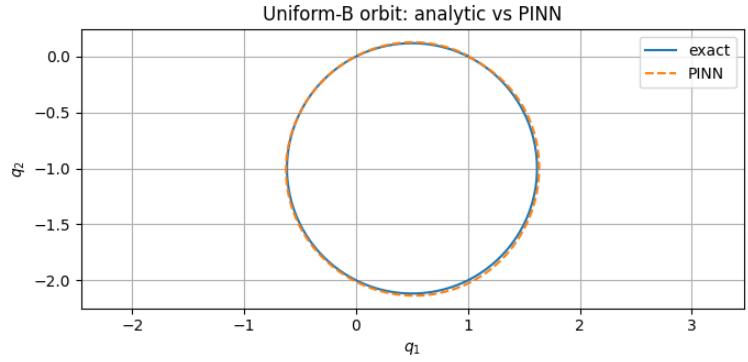
We shall attempt to solve the Hamiltonian system using a four dimensional output neural network and embed the initial conditions $(q(0), p(0))$ into the network using quadratic Taylor terms as

$$\begin{aligned}
q_1(\theta,t) &= q_1(0) + t\dot{q}_1(0) + \frac{1}{2}t^2\mathrm{nn}^1(\theta,t) \\
&= \Big(1 + \frac{b}{2}t\Big)q_1(0) + tp_1(0) + \frac{1}{2}t^2\mathrm{nn}^1(\theta,t) \\
q_2(\theta,t) &= q_2(0) + t\dot{q}_2(0) + \frac{1}{2}t^2\mathrm{nn}^2(\theta,t) \\
&= \Big(1 - \frac{b}{2}t\Big)q_2(0) + tp_2(0) + \frac{1}{2}t^2\mathrm{nn}^2(\theta,t) \\
p_1(\theta,t) &= p_1(0) + t\dot{p}_1(0) + \frac{1}{2}t^2\mathrm{nn}^3(\theta,t) \\
&= p_1(0) + \frac{b}{2}t\Big(p_2(0) - \frac{b}{2}q_1(0)\Big) + \frac{1}{2}t^2\mathrm{nn}^3(\theta,t) \\
p_2(\theta,t) &= p_2(0) + t\dot{p}_2(0) + \frac{1}{2}t^2\mathrm{nn}^4(\theta,t) \\
&= p_2(0) - \frac{b}{2}t\Big(p_1(0) + \frac{b}{2}q_2(0)\Big) + \frac{1}{2}t^2\mathrm{nn}^3(\theta,t).
\end{aligned}$$

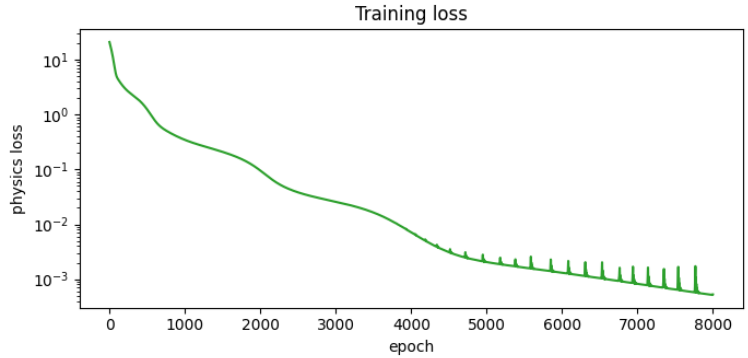
Consider the continuous integral version of the physics-informed loss built from the Hamiltonian equations

$$\begin{aligned}
\mathcal{L}(\theta) = &\int_0^1 \Big(\dot{q}_1(\theta,t) - p_1(\theta,t) - \frac{b}{2}q_2(\theta,t)\Big)^2 + \Big(\dot{q}_2(\theta,t) - p_2(\theta,t) + \frac{b}{2}q_1(\theta,t)\Big)^2 dt \\
&+ \int_0^1 \Big(\dot{p}_1(\theta,t) - \frac{b}{2}(p_2(\theta,t) - \frac{b}{2}q_1(\theta,t))\Big)^2 + \Big(\dot{p}_2(\theta,t) + \frac{b}{2}(p_1(\theta,t) + \frac{b}{2}q_2(\theta,t))\Big)^2 dt.
\end{aligned}$$

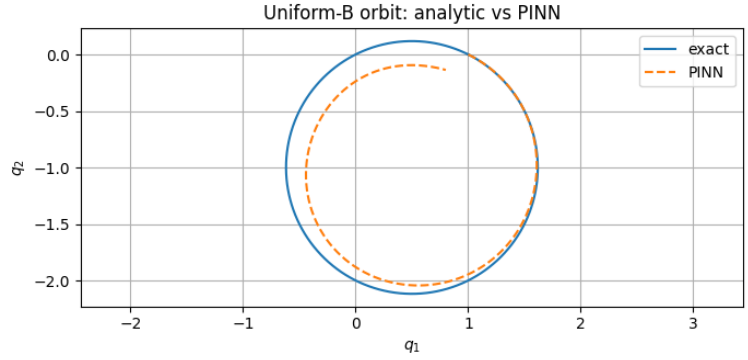
[1] See, for example, Calin-Chang-Greiner [18, p. 15].



(a)



(b)



(c)

Figure 5.3: (a) Comparison between the true circular trajectory and the PINN prediction $\big(q_1(t), q_2(t)\big)$ on $t \in [0, 2\pi]$ using Swish activation. (b) Corresponding total loss versus epoch. (c) Same comparison when Tanh activations are used.

NN implementation We approximate the two time-integrals with 200 collocation points on the interval $[0, 2\pi]$. The PINN employs six hidden layers of 64 units each and four linear outputs $\left(\mathrm{nn}^1, \ldots, \mathrm{nn}^4\right)$.

- *Swish activation.* With the Swish non-linearity the network reproduces the analytic orbit almost perfectly: the learned trajectory is indistinguishable from the reference circle, see Fig. 5.3(a).

- *Tanh activation.* Replacing Swish by tanh noticeably degrades long-time accuracy; the trajectory slowly drifts away from the circle, see Fig. 5.3(b).

These results underline the importance of choosing a suitable activation function in deep Hamiltonian PINNs.

To ensure smooth convergence we train for 8,000 epochs with a small learning rate of 5×10^{-5}, which suppresses large oscillations in the loss curve, see Fig. 5.3(b).

The detailed Keras code is available at

`https://tinyurl.com/wk674j3w`

5.6 Canonical Transformations

The Hamiltonian system of equations is a system of $2n$ ordinary differential equations on $\mathbb{R}^{2n}$, which in most cases cannot be solved in closed form. This is where the idea of *canonical transformations* comes into play. The idea is to consider a change of coordinates in the phase space

$$(q, p) \longrightarrow (Q, P) \tag{5.6.7}$$

around a given initial point (q_0, p_0), such that

$$\det \left. \frac{\partial(Q, P)}{\partial(q, p)} \right|_{(q_0, p_0)} \neq 0, \tag{5.6.8}$$

such that the Hamiltonian system

$$\dot{q}(t) = \frac{\partial H}{\partial p}(q(t), p(t)), \tag{5.6.9}$$

$$\dot{p}(t) = -\frac{\partial H}{\partial q}(q(t), p(t)), \tag{5.6.10}$$

with initial conditions $q(0) = q_0$ and $p(0) = p_0$, takes the transformed form

$$\dot{Q}(t) = \frac{\partial K}{\partial P}(Q(t), P(t)), \tag{5.6.11}$$

$$\dot{P}(t) = -\frac{\partial K}{\partial Q}(Q(t), P(t)), \tag{5.6.12}$$

with initial conditions $Q(0) = Q_0$, $P(0) = P_0$, and where the new Hamiltonian K is defined by

$$K(Q, P) = H(q(Q, P), p(Q, P)).$$

The hope—and the advantage of performing the change of coordinates (5.6.7)—is that the transformed system (5.6.11)–(5.6.12) can be integrated more easily to an exact solution $(Q(t), P(t))$. Then, by inverting the transformation (5.6.7), we obtain a solution to the original system (5.6.9)–(5.6.10):

$$q(t) = q(Q(t), P(t)), \quad p(t) = p(Q(t), P(t)).$$

Conditions. First, we investigate which transformations (5.6.7) convert the system (5.6.9)–(5.6.10) into the system (5.6.11)–(5.6.12).

Since (5.6.8) holds, then by continuity reasons it also holds on a neighborhood of (q_0, p_0). Then by the Inverse Function Theorem there exists a local inverse for (5.6.7) around (Q_0, P_0),

$$q = q(Q, P), \qquad p = p(Q, P),$$

where $Q_0 = Q(q_0, p_0)$ and $P_0 = P(q_0, p_0)$.

Assume that $(q(t), p(t))$ is a solution of the system (5.6.9)–(5.6.10) for small t. We now seek conditions under which the transformed system (5.6.11)–(5.6.12) holds. Using the chain rule, we compute

$$\begin{aligned}
\dot{Q}(t) &= \frac{d}{dt} Q(q(t), p(t)) = \frac{\partial Q}{\partial q}\dot{q}(t) + \frac{\partial Q}{\partial p}\dot{p}(t) \\
&= \frac{\partial Q}{\partial q}\frac{\partial H}{\partial p} - \frac{\partial Q}{\partial p}\frac{\partial H}{\partial q} = \frac{\partial H}{\partial p}\frac{\partial p}{\partial P} + \frac{\partial H}{\partial q}\frac{\partial q}{\partial P} \\
&= \frac{\partial H}{\partial P}(q(Q, P), p(Q, P)) = \frac{\partial K}{\partial P}(Q, P),
\end{aligned}$$

which gives equation (5.6.11), provided the identities

$$\frac{\partial Q}{\partial q} = \frac{\partial p}{\partial P}, \qquad \frac{\partial Q}{\partial p} = -\frac{\partial q}{\partial P} \tag{5.6.13}$$

hold.

A similar computation for $\dot{P}(t)$ provides

$$\begin{aligned}
\dot{P}(t) &= \frac{d}{dt} P(q(t), p(t)) = \frac{\partial P}{\partial q}\dot{q}(t) + \frac{\partial P}{\partial p}\dot{p}(t) \\
&= \frac{\partial P}{\partial q}\frac{\partial H}{\partial p} - \frac{\partial P}{\partial p}\frac{\partial H}{\partial q} = -\left(\frac{\partial H}{\partial q}\frac{\partial P}{\partial p} - \frac{\partial H}{\partial p}\frac{\partial P}{\partial q}\right) \\
&= -\left(\frac{\partial H}{\partial q}\frac{\partial q}{\partial Q} + \frac{\partial H}{\partial p}\frac{\partial p}{\partial Q}\right) = -\frac{\partial H}{\partial Q}(q(Q, P), p(Q, P)) \\
&= -\frac{\partial K}{\partial Q}(Q, P),
\end{aligned}$$

which yields equation (5.6.12), under the assumptions:

$$\frac{\partial P}{\partial p} = \frac{\partial q}{\partial Q}, \qquad \frac{\partial P}{\partial q} = -\frac{\partial p}{\partial Q}. \tag{5.6.14}$$

In conclusion, if the relations (5.6.13)–(5.6.14) hold, then the transformed Hamiltonian system (5.6.11)–(5.6.12) also holds.

These conditions can be expressed in matrix form as a condition on the Jacobian matrix of the transformation (5.6.7):

$$\begin{pmatrix} \frac{\partial Q}{\partial q} & \frac{\partial Q}{\partial p} \\ \frac{\partial P}{\partial q} & \frac{\partial P}{\partial p} \end{pmatrix} = \begin{pmatrix} \frac{\partial p}{\partial P} & -\frac{\partial q}{\partial P} \\ -\frac{\partial p}{\partial Q} & \frac{\partial q}{\partial Q} \end{pmatrix}. \tag{5.6.15}$$

Denoting by Δ the determinant of the matrix on the left side, we note that

$$\begin{pmatrix} \frac{\partial p}{\partial P} & -\frac{\partial q}{\partial P} \\ -\frac{\partial p}{\partial Q} & \frac{\partial q}{\partial Q} \end{pmatrix} = \Delta \begin{pmatrix} \frac{\partial q}{\partial Q} & \frac{\partial q}{\partial P} \\ \frac{\partial p}{\partial Q} & \frac{\partial p}{\partial P} \end{pmatrix}^{-1} = \Delta \begin{pmatrix} \frac{\partial Q}{\partial q} & \frac{\partial Q}{\partial p} \\ \frac{\partial P}{\partial q} & \frac{\partial P}{\partial p} \end{pmatrix}. \tag{5.6.16}$$

Comparing relations (5.6.15) and (5.6.16) yields $\Delta = 1$, which is equivalent to

$$\det \frac{\partial(Q,P)}{\partial(q,p)} = 1.$$

We would now like to construct a transformation of the form (5.6.7) — and this is where neural networks come into play — such that the new Hamiltonian has the following simple form:

$$K(Q,P) = QP.$$

In this case, the system (5.6.11)–(5.6.12) becomes

$$\begin{aligned} \dot{Q}(t) &= Q(t), \\ \dot{P}(t) &= -P(t). \end{aligned}$$

This system can be integrated exactly, yielding

$$Q(t) = Q_0 e^t, \qquad P(t) = P_0 e^{-t}.$$

We observe that along the trajectory, the product

$$Q(t)P(t) = Q_0 P_0$$

is constant; that is, the Hamiltonian $K(Q,P)$ is conserved along the flow.

Finding such a transformation is not always straightforward. We will first present an example where it can be constructed easily, and then use Deep Learning to handle the general case.

Example 5.6.1 Consider the Hamiltonian function $H(q,p) = \frac{1}{2}p^2 - \frac{1}{2}q^2$, and apply the transformation

$$Q = \frac{1}{\sqrt{2}}(p+q), \qquad P = \frac{1}{\sqrt{2}}(p-q),$$

which corresponds to a rotation by an angle $\pi/4$. Under this change of variables, the new Hamiltonian takes the simplified form $K(Q,P) = QP$. Therefore, the solutions of the transformed system are

$$Q(t) = Q_0 e^t, \qquad P(t) = P_0 e^{-t}.$$

Using the inverse transformation, we obtain the solutions in the original variables:

$$\begin{aligned} q(t) &= \frac{1}{\sqrt{2}}\left(Q(t) - P(t)\right) = \frac{1}{\sqrt{2}}\left(Q_0 e^{t} - P_0 e^{-t}\right), \\ p(t) &= \frac{1}{\sqrt{2}}\left(Q(t) + P(t)\right) = \frac{1}{\sqrt{2}}\left(Q_0 e^{t} + P_0 e^{-t}\right), \end{aligned}$$

where

$$Q_0 = \frac{1}{\sqrt{2}}(p_0 + q_0), \qquad P_0 = \frac{1}{\sqrt{2}}(p_0 - q_0).$$

These expressions represent the solution to the original Hamiltonian system defined by $H(q,p)$, with initial conditions $q(0) = q_0$ and $p(0) = p_0$.

The direct transformation. The desired transformation will be found using neural networks as follows. Consider a neural network

$$f_\theta(q,p) = \left(Q_\theta(q,p),\ P_\theta(q,p)\right)$$

with input (q,p) and two-dimensional output, such that

$$Q_\theta(q,p)\, P_\theta(q,p) \approx H(q,p).$$

We train this network by minimizing the loss

$$\mathcal{L}(\theta) = \frac{1}{m}\sum_{j=1}^{m} R(q_j,p_j)^2 + \lambda \sum_{j=1}^{m} \left(\Delta_\theta(q_j,p_j) - 1\right)^2,$$

where

$$\begin{aligned} R(q_j,p_j) &= Q_\theta(q_j,p_j)\, P_\theta(q_j,p_j) - H(q_j,p_j), \\ \Delta_\theta(q_j,p_j) &= \left(\frac{\partial Q_\theta}{\partial q}\frac{\partial P_\theta}{\partial p} - \frac{\partial Q_\theta}{\partial p}\frac{\partial P_\theta}{\partial q}\right)(q_j,p_j). \end{aligned}$$

Here, $q_j = q(t_j)$ and $p_j = p(t_j)$ are sampled at collocation points $\{t_j\}_{j=1}^m$. If the optimal weights are

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta),$$

then the desired transformation is given by

$$(q,p) \mapsto \left(Q_{\theta^*}(q,p),\ P_{\theta^*}(q,p)\right).$$

The inverse transformation. Since the determinant $\Delta_{\theta^*} \approx 1$, the neural network transformation $(q,p) \mapsto f_{\theta^*}(q,p)$ is locally invertible, with inverse

$$(Q,P) \mapsto f_{\theta^*}^{-1}(Q,P).$$

Then, the solution to the original Hamiltonian system (5.6.9)–(5.6.10) can be recovered by applying the inverse transformation:

$$\begin{aligned} (q(t),p(t)) &= f_{\theta^*}^{-1}(Q(t),P(t)) = f_{\theta^*}^{-1}(Q_0 e^{t}, P_0 e^{-t}) \\ &= f_{\theta^*}^{-1}\left(Q_{\theta^*}(q_0,p_0)\, e^{t},\ P_{\theta^*}(q_0,p_0)\, e^{-t}\right). \end{aligned}$$

The only remaining question is: *how do we compute* $f_{\theta^*}^{-1}$ *given* f_{θ^*}*?*

To address this, we introduce a new neural network $g_\eta(Q,P)$, with input (Q,P), output (q,p), and trainable parameters η, which approximates the inverse. We train g_η to satisfy the invertibility condition

$$g_\eta \circ f_{\theta^*}(q,p) = (q,p).$$

That is,

$$g_\eta\big(Q_{\theta^*}(q,p),\ P_{\theta^*}(q,p)\big) = (q,p).$$

To achieve this, we minimize the loss function

$$\tilde{\mathcal{L}}(\eta) = \frac{1}{m}\sum_{j=1}^{m} \big\|g_\eta\big(f_{\theta^*}(q_j,p_j)\big) - (q_j,p_j)\big\|_{\text{Eu}}^2 + \big\|g_\eta\big(f_{\theta^*}(q_0,p_0)\big) - (q_0,p_0)\big\|_{\text{Eu}}^2 .$$

After training, the solution of the original Hamiltonian system (5.6.9)–(5.6.10) is obtained as

$$(q(t),p(t)) = g_{\eta^*}\big(Q_{\theta^*}(t),\ P_{\theta^*}(t)\big) = g_{\eta^*}\big(Q_0 e^{t},\ P_0 e^{-t}\big),$$

where $\eta^* = \arg\min_\eta \tilde{\mathcal{L}}(\eta)$.

5.7 Generating Functions

In this section, we shall learn the solutions of the Hamiltonian system from its initial conditions by means of a *latent* function $G = G(q_0,p_0,t)$, such that

$$q(t) = \frac{\partial G}{\partial p_0}, \qquad p(t) = \frac{\partial G}{\partial q_0}. \tag{5.7.17}$$

The function $G(q_0,p_0,t)$ is called a *generating function.* We note that the mixed derivative $\dfrac{\partial^2 G}{\partial q_0 \partial p_0}$ captures how sensitive the solution $q(t)$ is to perturbations in the initial position q_0, or how sensitive $p(t)$ is to perturbations in the initial momentum p_0.

One way to construct such functions is directly by first solving the Hamiltonian system and then integrating the PDE system (5.7.17) to obtain the function G, which, in general, is not unique. We shall illustrate this method through the following examples.

Example 5.7.1 (The 1D Free Particle) Consider the Hamiltonian $H(q,p) = \frac{1}{2}p^2$. The Hamiltonian system can be integrated in closed form as

$$\begin{aligned} \dot{p} &= 0 \quad \Rightarrow \quad p(t) = p_0, \\ \dot{q} &= p \quad \Rightarrow \quad q(t) = p_0 t + q_0. \end{aligned}$$

Using these solutions, we seek to find G by solving the system (5.7.17):

$$\begin{aligned} \frac{\partial G}{\partial p_0} &= p_0 t + q_0, \\ \frac{\partial G}{\partial q_0} &= p_0. \end{aligned}$$

Integrating the second equation with respect to q_0, we obtain

$$G = q_0 p_0 + S(p_0, t),$$

where $S(p_0, t)$ is an unknown function of p_0 and t.

To determine S, differentiate G with respect to p_0 and use the first equation:

$$\frac{\partial G}{\partial p_0} = q_0 + \frac{\partial S}{\partial p_0} = p_0 t + q_0,$$

which gives

$$\frac{\partial S}{\partial p_0} = p_0 t.$$

Integrating with respect to p_0, we find

$$S(p_0, t) = \frac{1}{2} p_0^2 t + U(t),$$

where $U(t)$ is an arbitrary smooth function of t. Hence, the generating function is

$$G(q_0, p_0, t) = p_0 q_0 + \frac{1}{2} p_0^2 t + U(t).$$

Since in this case $\dfrac{\partial^2 G}{\partial q_0 \partial p_0} = 1$, the sensitivity of the solution with respect to initial data does not depend on time t.

Example 5.7.2 (The Harmonic Oscillator) Let $H(q, p) = \frac{1}{2}p^2 + \frac{1}{2}q^2$. Then the Hamiltonian system with initial conditions $q(0) = q_0$, $p(0) = p_0$ has the solution

$$\begin{aligned} q(t) &= q_0 \cos t + p_0 \sin t, \\ p(t) &= -q_0 \sin t + p_0 \cos t. \end{aligned}$$

Integrating the relation $q(t) = \frac{\partial G}{\partial p_0}$ yields

$$G = p_0 q_0 \cos t + \frac{1}{2} p_0^2 \sin t + S(q_0, t),$$

where S is a function to be determined. Differentiating with respect to q_0 and equating to $p(t)$ gives

$$\frac{\partial S}{\partial q_0} = p(t) - p_0 \cos t = -q_0 \sin t,$$

which implies $S(q_0, t) = -\frac{1}{2} q_0^2 \sin t + T(t)$, with $T(t)$ an arbitrary smooth function. Hence, we obtain the following generating function:

$$G(q_0, p_0, t) = p_0 q_0 \cos t + \frac{1}{2} \sin t \left(p_0^2 - q_0^2\right) + T(t).$$

Since $\dfrac{\partial^2 G}{\partial q_0 \partial p_0} = \cos t$, the sensitivity of the solution with respect to initial data does depend periodically on time t, with zero sensitivities at $t = \pi/2 + k\pi$.

NN Approach In the previous cases, we were able to find the generating function by integrating backward from the solution $(q(t), p(t))$. This method is of not much practical value, since the solution may not be explicitly known. In practice, what might be available are some observations of the trajectory's position and momentum, (q_j, p_j), at given times, $q_j = q(t_j)$ and $p_j = p(t_j)$. The advantage of this approach is most apparent in cases where an explicit solution to the Hamiltonian system is not available.

When solving the Hamiltonian system using PINNs, one customarily considers the neural network $t \mapsto (q_\theta(t), p_\theta(t))$ and employs the loss function

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{ODE}}(\theta) + \mathcal{L}_{\text{IC}}(\theta) + \mathcal{L}_{\text{data}}(\theta), \tag{5.7.18}$$

where

$$\begin{aligned}
\mathcal{L}_{\text{IC}}(\theta) &= \|q_\theta(0) - q_0\|^2 + \|p_\theta(0) - p_0\|^2, \\
\mathcal{L}_{\text{data}}(\theta) &= \frac{1}{m}\sum_j (q_\theta(t_j) - q_j)^2 + \frac{1}{m}\sum_j (p_\theta(t_j) - p_j)^2, \\
\mathcal{L}_{\text{ODE}}(\theta) &= \frac{1}{m}\sum_{j=1}^{m} \left(\dot{q}_\theta(t_j) - \partial_p H(q_j, p_j)\right)^2 + \frac{1}{m}\sum_{j=1}^{m} \left(\dot{p}_\theta(t_j) + \partial_q H(q_j, p_j)\right)^2.
\end{aligned}$$

A distinct approach is to use a neural network to learn a generating function and express the solutions in terms of it. More specifically, we consider a neural network

$$(q_0, p_0, t) \mapsto G_\theta(q_0, p_0, t),$$

and define the derived networks as

$$q_\theta(t) = \partial_{p_0} G_\theta(q_0, p_0, t), \tag{5.7.19}$$
$$p_\theta(t) = \partial_{q_0} G_\theta(q_0, p_0, t), \tag{5.7.20}$$

which can be considered a hard embedding of the solution. The loss function in this case takes the same form as in (5.7.18), with $q_\theta(t)$ and $p_\theta(t)$ defined via the embedding.

In the case where no data is provided, some modifications are necessary in the loss function. After removing the $\mathcal{L}_{\text{data}}(\theta)$ term, we use the following loss:

$$\tilde{\mathcal{L}}(\theta) = \tilde{\mathcal{L}}_{\text{ODE}}(\theta) + \mathcal{L}_{\text{IC}}(\theta), \tag{5.7.21}$$

where

$$\begin{aligned}
\tilde{\mathcal{L}}_{\text{ODE}}(\theta) &= \frac{1}{m}\sum_{j=1}^{m} \left(\dot{q}_\theta(t_j) - \partial_p H(q_\theta(t_j), p_\theta(t_j))\right)^2 \\
&\quad + \frac{1}{m}\sum_{j=1}^{m} \left(\dot{p}_\theta(t_j) + \partial_q H(q_\theta(t_j), p_\theta(t_j))\right)^2,
\end{aligned}$$

with q_θ and p_θ given by the embeddings (5.7.19)–(5.7.20).

Using the loss function $\tilde{\mathcal{L}}_{\mathrm{ODE}}(\theta)$ corresponds to an unsupervised learning of the solution.

Implementation example As an implementation example, we consider the case of the simple pendulum with the Hamiltonian function

$$H(q,p) = \frac{1}{2}p^2 - \cos q.$$

In this case, the Hamiltonian system of equations cannot be solved explicitly using elementary functions, so we employ the method of generating functions to approximate the solution. Specifically, we learn the solution of the pendulum system via a generating function $G(q_0, p_0, t)$, which is approximated a priori by a neural network. The network outputs the generating function G, and then uses automatic differentiation to extract the time-dependent solution components $q(t)$ and $p(t)$.

Training is conducted in an unsupervised manner, relying solely on the structure of the Hamiltonian system and the given initial conditions. The generating function G is modeled by the following neural architecture:

```
# Define the neural network G_theta(q0, p0, t)
def create_model():
    return tf.keras.Sequential([
        tf.keras.layers.InputLayer(input_shape=(3,)), #(q0,p0,t)
        tf.keras.layers.Dense(64, activation='tanh'),
        tf.keras.layers.Dense(64, activation='tanh'),
        tf.keras.layers.Dense(64, activation='tanh'),
        tf.keras.layers.Dense(1)  # scalar output G
    ])

model = create_model()
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
```

The training was performed over $1,000$ epochs. Figure 5.4(a) demonstrates that the learned solution $q(t)$ aligns almost perfectly with the numerical Runge–Kutta solution on the time interval $[0, 1]$, with the two curves appearing indistinguishable. The unsupervised loss is given in Fig. 5.4(b). The code details can be found at:

`https://tinyurl.com/4ya72xmk`

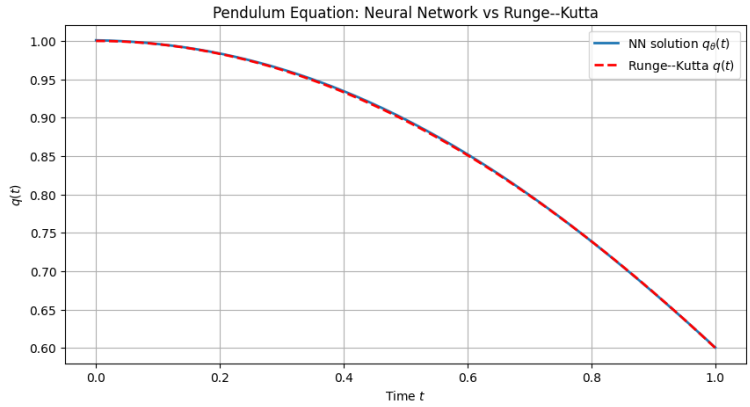
5.8 Exercises

Exercise 5.8.1 (Linear oscillator) Consider the Hamiltonian

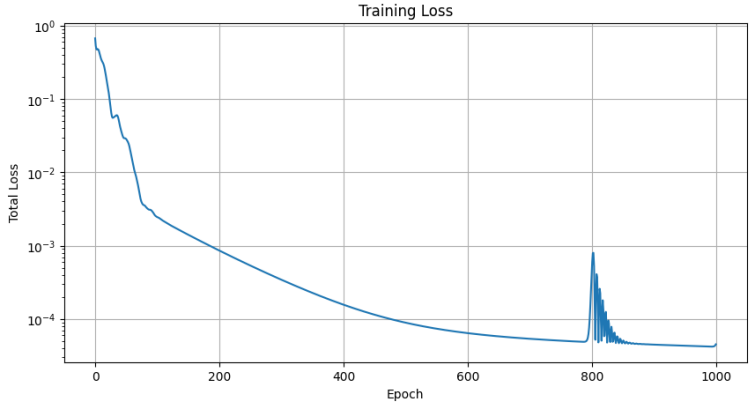
$$H(q,p) = \tfrac{1}{2}\,p^2 + k\,q, \qquad k > 0 \text{ (constant)}.$$

(*a*) Solve Hamilton's equations with the initial conditions $q(0) = q_0$ and $p(0) = p_0$.

(*b*) Train a neural network with $q_0 = 1$, $p_0 = 0$ to approximate this solution and compare the network output with the exact solution.



(a)



(b)

Figure 5.4: (a) The pendulum solution $q(t)$ and the Runge–Kutta solution match perfectly. (b) The loss function reaches saturation after $1,000$ epochs.

Exercise 5.8.2 (2-D isotropic harmonic oscillator) Consider a particle of unit mass ($m = 1$) moving in the plane under the isotropic harmonic potential with spring constant $k = 1$. The Hamiltonian is

$$H(x, y, p_x, p_y) = \frac{1}{2}\big(p_x^2 + p_y^2\big) + \frac{1}{2}\big(x^2 + y^2\big).$$

(a) Write down the Hamiltonian system and find the exact solution.

(b) Using a physics-informed neural network, learn the trajectory $\big(x(t), y(t)\big)$ on the interval $t \in [0,\, 2\pi]$. Use the embedding of the initial conditions by writing

$$\begin{aligned} x_\theta(t) &= x_0 + t\,\mathrm{nn}^1(t), \\ y_\theta(t) &= y_0 + t\,\mathrm{nn}^2(t), \\ p_{x,\theta}(t) &= p_{x0} + t\,\mathrm{nn}^3(t), \\ p_{y,\theta}(t) &= p_{y0} + t\,\mathrm{nn}^4(t), \end{aligned}$$

where the network $f_\theta(t) = \big(\mathrm{nn}^1, \mathrm{nn}^2, \mathrm{nn}^3, \mathrm{nn}^4\big)$ has two hidden layers of 64 tanh neurons each.

(c) Plot $x(t)$ and $y(t)$ obtained from the PINN together with the exact solutions from part (a) on the same graph, using different colours. Display the training loss versus epoch on a separate semilog plot, and comment on the agreement between the two solutions. Experiment with several initial solutions.

Exercise 5.8.3 (Particle in constant magnetic field and radial electric field) A non-relativistic particle of charge e and mass m moving in an electromagnetic field with scalar potential $\phi(q)$ and vector potential $A(q)$ is governed by the Hamiltonian

$$H(q, p) = \frac{1}{2m}\,\big\|p - e\,A(q)\big\|^2 + e\,\phi(q).$$

Let

$$A(q) = \Big(-\tfrac{b}{2}\,q_2,\ \tfrac{b}{2}\,q_1,\ 0\Big), \qquad b > 0, \qquad m = e = 1,$$

and $\phi(q) = -\frac{b^2}{4}\big(q_1^2 + q_2^2\big)$. Then

$$B = \mathrm{curl} A = (0, 0, b), \qquad E = -\nabla\phi = \tfrac{b^2}{2}\,(q_1, q_2).$$

(a) Derive Hamilton's equations for this $H(q, p)$.

(b) With initial data $q(0) = (1, 0)$ and $p(0) = (0, 1)$, use a physics-informed neural network to learn the trajectory $q(t)$.

(c) Show that $q(t)$ satisfies the matrix ODE $\ddot{q} = 2bJ\dot{q} + b^2 q$, where $J = \left(\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right)$. Solve it by setting $q(t) = e^{bJt} y(t)$ and reducing the problem to $\ddot{y}(t) = 0$.

(d) Compare the solutions obtained in parts (b) and (c) on $t \in [0, 2\pi]$ and discuss your findings.

Exercise 5.8.4 Hénon–Heiles Hamiltonian ([45]) is the polynomial

$$H(x, y, p_x, p_y) = \frac{1}{2}\big(p_x^2 + p_y^2\big) + \frac{1}{2}\big(x^2 + y^2\big) + x^2 y - \frac{1}{3}\, y^3.$$

(a) Derive Hamilton's equations.

(b) Choose the initial condition $(x_0, y_0, p_{x0}, p_{y0}) = (0,\, 0.25,\, 0,\, 0)$ and integrate the equations with a fourth-order Runge–Kutta scheme on $t \in [0, 50]$ for the energy $H = 0.10$.

(c) Use a PINN to learn the trajectory on $t \in [0, 50]$ for the energy $H = 0.10$. Compare the learned $(x(t), y(t))$ with the numerical solution from part (b) and report the final physics loss.

Exercise 5.8.5 (Generating function for the n-dimensional harmonic oscillator). Consider the Hamiltonian function

$$H(q, p) = \frac{1}{2}\sum_{k=1}^{n} p_k^2 - \frac{1}{2}\sum_{k=1}^{n} q_k^2$$

which describes an n-dimensional system of uncoupled harmonic oscillators.

(a) Solve the Hamiltonian system with initial conditions $q(0) = q_0$, $p(0) = p_0$, and show that the solution is given by

$$\begin{cases} q_k(t) = q_{0,k}\cos t + p_{0,k}\sin t, \\ p_k(t) = -q_{0,k}\sin t + p_{0,k}\cos t, \end{cases} \qquad \text{for } k = 1, \ldots, n.$$

(b) Use the relations

$$q_k(t) = \frac{\partial G}{\partial p_{0,k}}, \qquad p_k(t) = \frac{\partial G}{\partial q_{0,k}},$$

to construct a family of generating functions $G(q_0, p_0, t)$ that generate the above solutions by direct integration.

(c) Construct a family of generating functions G as a sum of functions of the form

$$G(q_0, p_0, t) = \sum_{k=1}^{n} \big(a(t) q_{0,k} p_{0,k} + b(t) p_{0,k}^2 + c(t) q_{0,k}^2\big)$$

and determine the functions $a(t), b(t), c(t)$ so that the relations above hold.

Chapter 6

Conservation Laws

A conservation law states that a measurable quantity of a physical system remains constant as the system evolves, provided no external influence adds or removes that quantity. Mathematically, it appears as a functional relation — linking positions, velocities, densities, masses, and so on — that is preserved along every trajectory of the dynamical system. Familiar examples include the conservation of energy, linear and angular momentum, mass, and electric charge.

6.1 Conservation Laws as Regularizers

Embedding such laws is an effective form of regularisation for physics-informed neural networks. Each conserved quantity confines the dynamics to a hypersurface of constant value; multiple conservation laws restrict the trajectory to the intersection of several hypersurfaces, which is a lower-dimensional manifold that sharply reduces the search space. Incorporating these constraints therefore speeds up convergence.

In practice, conservation laws can be incorporated into a PINN in several distinct ways:

(*i*) *Soft-constraint regulariser*: add a penalty term that measures the deviation of the conserved quantity from its initial value.

(*ii*) *Hard embedding*: parameterise the network so that the invariant is satisfied identically.

(*iii*) *Integral (weak) constraint*: impose the conserved quantity through an integral condition.

The optimal choice depends on the algebraic structure of the problem and on how strictly the invariant must be enforced during training.

6.1.1 Soft-constraint Regulariser

Assume a dynamical system possesses the conservation law

$$C\big(q(t), \dot{q}(t)\big) = C_0,$$

where the constant C_0 is determined from the initial data $q(0) = q_0$, $\dot{q}(0) = \dot{q}_0$ via $C_0 = C(q_0, \dot{q}_0)$.

Let $\mathcal{L}_{\text{ODE}}(\theta)$ denote the usual physics loss of the PINN. We augment it with a conservation penalty

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{ODE}}(\theta) + \lambda_{\text{cons}}\, \mathcal{L}_{\text{cons}}(\theta),$$

with

$$\mathcal{L}_{\text{cons}}(\theta) = \frac{1}{m} \sum_{j=1}^{m} \Big[C\big(q_\theta(t_j), \dot{q}_\theta(t_j)\big) - C_0 \Big]^2,$$

where $q_\theta(t)$ is a neural network which approximates the dynamical system trajectory $q(t)$.

The case when C_0 is unknown. If no initial values are prescribed we cannot compute C_0 directly. Instead, we minimize the *variance* of C over the collocation points

$$\mathcal{L}_{\text{cons}}(\theta) = \frac{1}{m} \sum_{j=1}^{m} \Big[C\big(q_\theta(t_j), \dot{q}_\theta(t_j)\big) - \bar{C}(\theta) \Big]^2,$$

where

$$\bar{C}(\theta) = \frac{1}{m} \sum_{j=1}^{m} C\big(q_\theta(t_j), \dot{q}_\theta(t_j)\big),$$

is the average of C at the collocation set.

A small (ideally zero) variance indicates that the network output preserves the invariant almost exactly. This formulation is useful when C_0 itself is unknown.

It is worth noting that the same ideas apply to conservation laws defined on the cotangent bundle, namely those of the form

$$C\big(q(t), p(t)\big) = C_0.$$

We illustrate both formulations in the examples that follow. We shall assume in all cases that the conservation law is known. We shall see in a future section how one can obtain conservation laws via neural networks from data measurements.

Example 6.1.1 (The harmonic oscillator). The Hamiltonian function for the one-dimensional harmonic oscillator introduced on page 190 is constant along every trajectory. Because H contains no explicit time dependence, its total derivative vanishes:

$$\frac{d}{dt} H\big(q(t), p(t)\big) = \frac{\partial H}{\partial q}\, \dot{q}(t) + \frac{\partial H}{\partial p}\, \dot{p}(t) = 0.$$

Consequently,

$$H\big(q(t), p(t)\big) = E_0 \qquad \text{for all } t,$$

i.e. the energy of the harmonic oscillator is conserved.

(i) Initial-value formulation. Given initial data $q(0) = q_0$ and $p(0) = p_0$, set $E_0 = H(q_0, p_0) = \frac{1}{2}(p_0^2 + q_0^2)$. The energy-constraint loss is

$$\mathcal{L}_{\text{energy}}(\theta) = \frac{1}{m} \sum_{j=1}^{m} \Big[\tfrac{1}{2}\big(p_\theta^2(t_j) + q_\theta^2(t_j)\big) - E_0 \Big]^2.$$

Recalling the ODE residual for the harmonic oscillator,

$$\mathcal{L}_{\mathrm{ODE}}(\theta) = \frac{1}{m}\sum_{j=1}^{m}\Big(\dot{p}_\theta(t_j) + q_\theta(t_j)\Big)^2 + \frac{1}{m}\sum_{j=1}^{m}\Big(\dot{q}_\theta(t_j) - p_\theta(t_j)\Big)^2,$$

and the initial-condition loss

$$\mathcal{L}_{\mathrm{IC}}(\theta) = \big\|q_\theta(0) - q_0\big\|^2 + \big\|p_\theta(0) - p_0\big\|^2,$$

we define the total loss

$$\mathcal{L}_{\mathrm{total}}(\theta) = \mathcal{L}_{\mathrm{ODE}}(\theta) + \lambda_{\mathrm{IC}}\,\mathcal{L}_{\mathrm{IC}}(\theta) + \lambda_{\mathrm{energy}}\,\mathcal{L}_{\mathrm{energy}}(\theta).$$

The positive weights λ_{IC} and $\lambda_{\mathrm{energy}}$ are chosen so that the three loss terms have comparable magnitudes during training.

(ii) Two-point boundary formulation. If only the positions at the endpoints are prescribed, $q(0) = q_0$ and $q(T) = q_T$, the constant energy E_0 cannot, in general, be expressed explicitly as a function $E_0(q_0, q_T)$. Instead we estimate it by the mean energy over the collocation set $\{t_j\}_{j=1}^m$:

$$\bar{E}(\theta) = \frac{1}{m}\sum_{j=1}^{m}\tfrac{1}{2}\big(p_\theta^2(t_j) + q_\theta^2(t_j)\big).$$

The corresponding constraint loss is

$$\mathcal{L}_{\mathrm{energy}}(\theta) = \frac{1}{m}\sum_{j=1}^{m}\Big[\tfrac{1}{2}\big(p_\theta^2(t_j) + q_\theta^2(t_j)\big) - \bar{E}(\theta)\Big]^2.$$

Considering the two-point boundary loss

$$\mathcal{L}_{\mathrm{bd}}(\theta) = \|q(0) - a_0\|^2 + \|q(T) - q_T\|^2,$$

the total loss in this case is

$$\mathcal{L}_{\mathrm{total}}(\theta) = \mathcal{L}_{\mathrm{ODE}}(\theta) + \lambda_{\mathrm{bd}}\,\mathcal{L}_{\mathrm{bd}}(\theta) + \lambda_{\mathrm{energy}}\,\mathcal{L}_{\mathrm{energy}}(\theta),$$

where the ODE residual is the same as in case *(i)*, while λ_{bd} and $\lambda_{\mathrm{energy}}$ are positive hyperparameters.

When the initial or boundary conditions are embedded directly into the network architecture, the corresponding loss terms $\mathcal{L}_{\mathrm{IC}}(\theta)$ or $\mathcal{L}_{\mathrm{bd}}(\theta)$ can be omitted from the total loss.

Implementations We shall use the harmonic-oscillator implementation from Section 5.3, available at:

`https://tinyurl.com/67bsucj5`

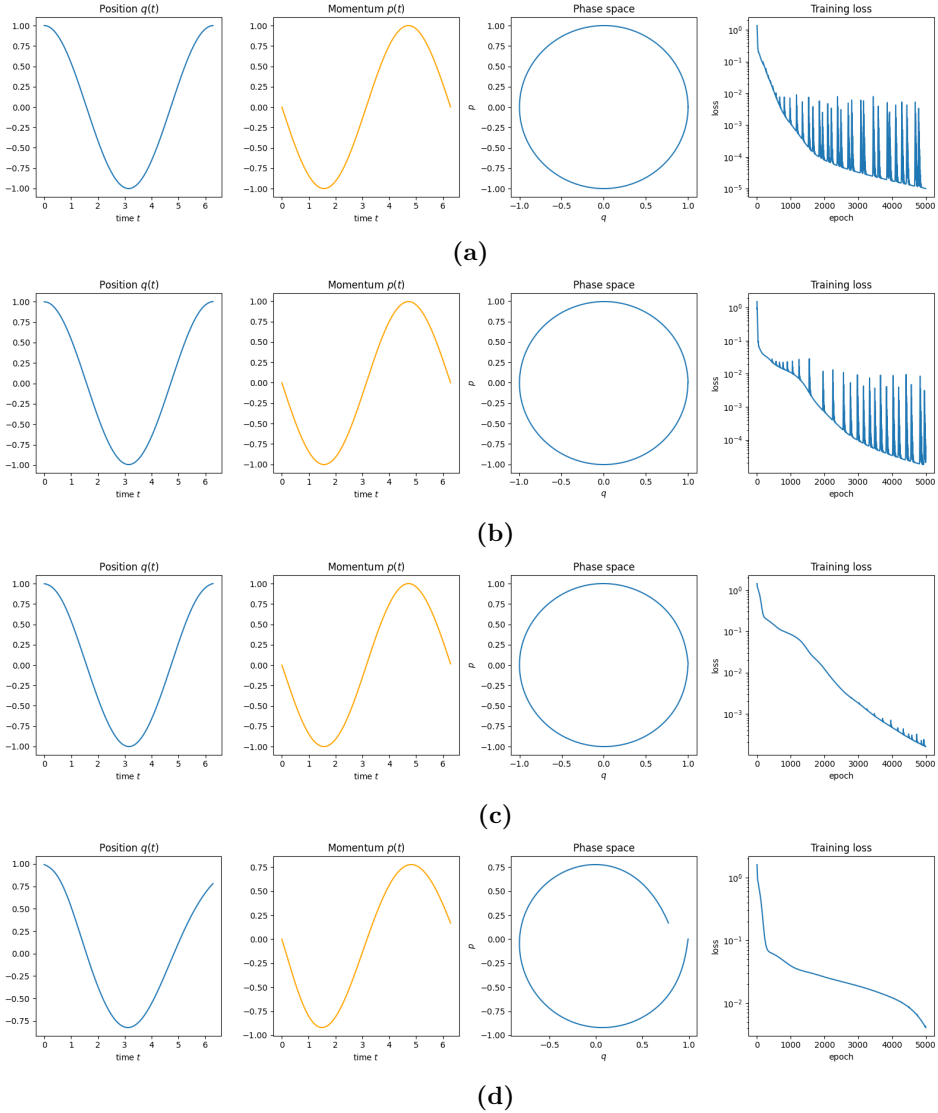


Figure 6.1: (a) Result obtained with the energy-conservation regulariser and learning rate $lr = 0.001$. (b) Result obtained *without* the energy-conservation regulariser, $lr = 0.001$. (c) Result obtained with the energy-regulariser and a reduced learning rate $lr = 0.0001$. (d) Result obtained *without* the energy-regulariser, $lr = 0.0001$.

Keeping the network architecture fixed (three hidden layers, each with 64 `tanh` units) and training for $5,000$ epochs in every run, we tested several learning rates with and without the additional energy-conservation term in the loss. The resulting observations are summarised below and are referred to Fig. 6.1.

When the learning rate is too high, the total loss oscillates sharply—regardless of whether the energy-conservation term is included. In our tests the loss swung over three orders of magnitude, from 10^{-5} to 10^{-2}; terminating training near a minimum therefore became a matter of chance.

The phase-space plots for both settings (Fig. 6.1(a) and Fig. 6.1(b)) are almost perfect circles, indicating that the qualitative dynamics is captured in either case. The only difference is that the lowest value reached during the oscillations is slightly smaller when the energy term is present. Given the wild fluctuations, however, we do not regard one configuration as decisively superior to the other.

Reducing the learning rate by an order of magnitude–from 0.001 to 0.0001–eliminates the wild oscillations in the loss. With this smaller step size, *including* the energy-conservation regulariser the network produces a complete, perfectly closed circle in phase space (Fig. 6.1(c)). *Without* the regulariser, the trajectory remains slightly open and closes only after many additional epochs (Fig. 6.1(d)).

Hence, adding the energy-conservation term accelerates convergence.

Example 6.1.2 (The simple pendulum). The Hamiltonian for the simple pendulum introduced on page 191 is constant along every trajectory, because H has no explicit time dependence. Energy conservation reads

$$H\big(q(t),p(t)\big) = \tfrac{1}{2}\,p(t)^2 - \kappa\cos q(t) = E, \qquad \forall t.$$

We impose two-point boundary conditions $q(0) = q_0$ and $q(1) = q_1$, embedding them directly in the network:

$$\begin{aligned} q_\theta(t) &= (1-t)\,q_0 + t\,q_1 + t(1-t)\;\mathrm{nn}^1(\theta;t),\\ p_\theta(t) &= \mathrm{nn}^2(\theta;t), \end{aligned}$$

where $\mathrm{nn}(\theta;t) = \big(\mathrm{nn}^1,\mathrm{nn}^2\big)$ is a feed-forward network with one input and two outputs. Let $\{t_j\}_{j=1}^m \subset [0,1]$ denote the collocation points. The network's mean energy is

$$\bar{E}(\theta) = \frac{1}{m}\sum_{j=1}^{m}\Big[\tfrac{1}{2}\,p_\theta(t_j)^2 - \kappa\cos q_\theta(t_j)\Big],$$

and its variance defines the energy penalty

$$\mathcal{L}_{\text{energy}}(\theta) = \frac{1}{m}\sum_{j=1}^{m}\Big[H\big(q_\theta(t_j),p_\theta(t_j)\big) - \bar{E}(\theta)\Big]^2.$$

The total loss is

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{ODE}}(\theta) + \lambda\,\mathcal{L}_{\text{energy}}(\theta),$$

with

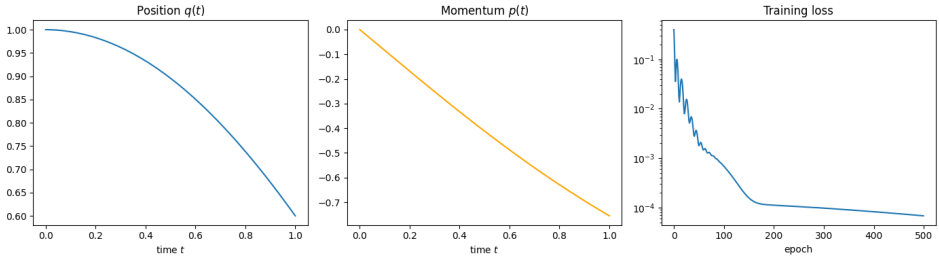
$$\mathcal{L}_{\text{ODE}}(\theta) = \frac{1}{m}\sum_{j=1}^{m}\Big[\dot{q}_\theta(t_j) - p_\theta(t_j)\Big]^2 + \frac{1}{m}\sum_{j=1}^{m}\Big[\dot{p}_\theta(t_j) + \kappa \sin q_\theta(t_j)\Big]^2,$$

and $\lambda > 0$ so that $\mathcal{L}_{\text{ODE}}$ and $\mathcal{L}_{\text{energy}}$ are of comparable sizes.

The implementation of this application is available at:

`https://tinyurl.com/4h4tzw88`

We used $1,000$ equally spaced collocation points on the interval $[0, 1]$, trained for 500 epochs, and kept the architecture at three dense layers with 64 `tanh` neurons each. The weight λ was set to 1. Boundary data were $q(0) = 1$ and $q(1) = 0.6$. The results are shown in Fig. 6.2. We note that Fig. 6.2(a) agrees with Fig. 5.2(a), which was validated by the Runge–Kutta numerical method.



(a) **(b)** **(c)**

Figure 6.2: (a) The position curve, $q(t)$; (b) The momentum curve, $p(t)$; (c) The associated loss.

Remark 6.1.3 (*i*) In general, a dynamical system might have more than one conservation law. If one knows a set of functionally independent conservation laws whose number equals the dimension of the phase space, the solution can, in principle, be characterised qualitatively. Each law defines a level surface, and functional independence ensures that these surfaces are not parallel but transversal, and hence they intersect.
(*ii*) For example, let's assume that a two-dimensional system admits two such conservation laws. When both are satisfied, their level surfaces intersect over a one-dimensional curve, which corresponds to the system's trajectory.
(*iii*) In theory, then, adding all independent invariants as soft-constraint terms in the loss could replace the ODE residual entirely. In practice, retaining the ODE term stabilises training and speeds up convergence, so a hybrid loss is usually preferred.

The next example provides an example of a dynamical system in which the conservation loss can replace the ODE loss.

Example 6.1.4 (Free particle in one spatial dimension). For an isolated particle of mass m subject to no external force, Newton's equation is $m\,\ddot{q}(t) = 0$. With initial data $q(0) = q_0$ and $\dot{q}(0) = v_0$ the solution is $q(t) = q_0 + v_0 t$.

A conservation law is the linear momentum

$$p(t) = m\,\dot{q}(t) = p_0 \qquad \text{(constant)}.$$

If the position is represented by a network $q_\theta(t)$, the ODE loss

$$\mathcal{L}_{\text{ODE}}(\theta) = \frac{1}{m}\sum_j \big[\ddot{q}_\theta(t_j)\big]^2$$

enforces $\ddot{q} = 0$ at the collocation points. The momentum loss

$$\mathcal{L}_{\text{mom}}(\theta) = \frac{1}{m}\sum_j \big[m\,\dot{q}_\theta(t_j) - p_0\big]^2$$

penalises deviations from the constant momentum p_0.
If we omit $\mathcal{L}_{\text{ODE}}$ and keep only $\mathcal{L}_{\text{mom}} + \mathcal{L}_{\text{IC}}$, the network still learns the exact solution, because $\mathcal{L}_{\text{mom}}$ alone enforces $m\,\dot{q} = p_0$, which upon differentiation yields Newton's equation.

6.1.2 Hard Embedding

Instead of penalising violations as extra loss terms, we can construct the output so that the conservation law is identically satisfied for every t. The conservation law is parameterized using some *latent functions*. Then we use a neural network to learn the latent functions. This parametrization depends on the problem at hand and might not be always possible. We shall provide a few examples.

Example 6.1.5 (One-dimensional harmonic oscillator). This example gives a *hard-embedding* alternative to the soft-constraint variant in Example 6.1.1. The Hamiltonian is $H(q,p) = \frac{1}{2}p^2 + \frac{1}{2}q^2$, and on every trajectory

$$\tfrac{1}{2}p^2 + \tfrac{1}{2}q^2 = 2E_0,$$

where the energy level E_0 is determined by the initial data $q(0)$ and $p(0)$.
Let $\phi_\theta\colon \mathbb{R}\to\mathbb{R}$ be a scalar neural network. Define

$$q_\theta(t) = \sqrt{2E_0}\cos\phi_\theta(t), \qquad p_\theta(t) = \sqrt{2E_0}\sin\phi_\theta(t).$$

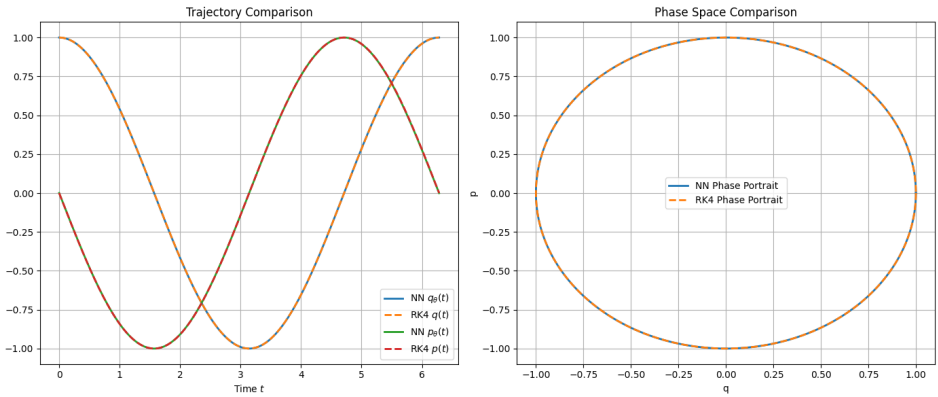
By construction, $\frac{1}{2}p_\theta^2(t) + \frac{1}{2}q_\theta^2(t) = 2E_0$ for all t. The ODE residual is given by

$$\begin{aligned}\mathcal{L}_{\text{ODE}}(\theta) &= \|\dot{q}_\theta - p_\theta\|_{L^2} + \|\dot{p}_\theta + q_\theta\|_{L^2} \\ &\approx \frac{1}{m}\sum_j \big[\dot{q}_\theta(t_j) - p_\theta(t_j)\big]^2 + \frac{1}{m}\sum_j \big[\dot{p}_\theta(t_j) + q_\theta(t_j)\big]^2.\end{aligned}$$

Let $\theta^* = \arg\min_\theta(\mathcal{L}_{\text{ODE}}(\theta) + \lambda\mathcal{L}_{IC}(\theta))$; then $\big(q_{\theta^*}(t), p_{\theta^*}(t)\big)$ approximates the exact solution $\big(q(t), p(t)\big)$. We make the remark that when using a hard-embedding alternative we cannot build the initial conditions into the network, so we cannot dispense the $\mathcal{L}_{IC}$ term. In the case of a non-quadratic potential function $U(q)$ the parametrization is not necessary possible.

Implementation The architecture uses a neural network with two hidden dense layers, each containing 64 neurons and using the `tanh` activation function. We train the network for 4,000 epochs using the Adam optimizer with a learning rate of 0.001. Thanks to the hard embedding, the network compiles faster than without it, and the results match the Runge–Kutta solution perfectly. See Fig. 6.3 for a comparison between the network output and the RK solution. They are virtually indistinguishable. The code can be found at

`https://tinyurl.com/vh5hpzs6`



(a) **(b)**

Figure 6.3: (a) The position and momentum curves for the one-dimensional harmonic oscillator and the exact solutions; (b) The phase space curve $(q(t), p(t))$, $t \in [0, 2\pi]$.

Example 6.1.6 (Two-dimensional harmonic oscillator). This example considers a dynamical system whose Hamiltonian function is given by

$$H(q_1, q_2, p_1, p_2) = \frac{1}{2}(p_1^2 + p_2^2) + \frac{k}{2}(q_1^2 + q_2^2), \qquad k > 0.$$

On every trajectory the energy is preserved

$$\frac{1}{2}(p_1^2(t) + p_2^2(t)) + \frac{k}{2}(q_1^2(t) + q_2^2(t)) = E_0, \qquad \forall t.$$

The parametrization of the conservation law of energy in this case uses a three-dimensional output neural network $NN(\theta; t) = \big(f_\theta(t), g_\theta(t), h_\theta(t)\big)$ and takes the following form

$$\begin{aligned} q_1(\theta; t) &= \sqrt{2E_0/k}\, \cos f_\theta(t)\, \cos g_\theta(t) \\ q_2(\theta; t) &= \sqrt{2E_0/k}\, \sin f_\theta(t)\, \cos g_\theta(t) \\ p_1(\theta; t) &= \sqrt{2E_0}\, \cos h_\theta(t)\, \sin g_\theta(t) \\ p_2(\theta; t) &= \sqrt{2E_0}\, \sin h_\theta(t)\, \sin g_\theta(t). \end{aligned}$$

Then we adjust θ so that the total loss

$$\begin{aligned}\mathcal{L}_{\text{total}}(\theta) =&\|\dot{q}_1(\theta;t) - p_1(\theta;t)\|_{L^2} + \|\dot{q}_2(\theta;t) - p_2(\theta;t)\|_{L^2}\\ &+ \|\dot{p}_1(\theta;t) + q_1(\theta;t)\|_{L^2} + \|\dot{p}_2(\theta;t) + q_2(\theta;t)\|_{L^2}\\ &+ \lambda_1\|q(\theta;0) - q_0\|^2 + \lambda_2\|p(\theta;0) - p_0\|^2\end{aligned}$$

is minimised.

Example 6.1.7 (Incompressibility). A vector field V is *incompressible* if it preserves volume along its flow; equivalently, $\operatorname{div} V = 0$.

(i) Two dimensions. We introduce a scalar *stream function* $\psi(x_1, x_2)$ and set

$$V = (V_1, V_2) = \big(\,\partial_2\psi,\ -\partial_1\psi\big).$$

Then

$$\operatorname{div} V = \partial_1 V_1 + \partial_2 V_2 = \partial_1\partial_2\psi - \partial_2\partial_1\psi = 0,$$

so incompressibility is satisfied identically. The latent function ψ can be learned with a scalar neural network $\psi_\theta(x)$. For instance, the two-dimensional Hamiltonian flow, $(\dot{q}(t), \dot{p}(t)) = (\partial_p H, -\partial_q H)$ is incompressible. We note that in this case the stream function is the Hamiltonian function, $\psi = H$.

(ii) Three dimensions. Introduce a *vector potential* $A = (A_1, A_2, A_3)$ and define $V = \operatorname{curl} A$, i.e.

$$V_1 = \partial_2 A_3 - \partial_3 A_1, \quad V_2 = \partial_3 A_1 - \partial_1 A_3, \quad V_3 = \partial_1 A_2 - \partial_2 A_1.$$

Because $\operatorname{div}\circ\operatorname{curl} \equiv 0$, the resulting field is divergence-free for any A. The latent vector potential A can be learned with a three-output neural network $A_\theta(x)$.
We shall provide next an application.

Helmholtz decomposition One application is the *Helmholtz decomposition* of a smooth vector field into the sum of an irrotational (curl-free) component and an incompressible (divergence-free) component. Given a vector field X on a bounded domain $D \subset \mathbb{R}^3$, there exist vector fields V and W such that

$$\operatorname{div} V = 0, \qquad \operatorname{curl} W = 0, \qquad X = V + W.$$

We note that this decomposition is in general not unique. Indeed, for any vector field H the pair

$$\widetilde{V} = V + H, \qquad \widetilde{W} = W - H$$

satisfies the same relation $X = \widetilde{V} + \widetilde{W}$. To retain the incompressible and irrotational properties we require

$$\operatorname{div} H = 0, \qquad \operatorname{curl} H = 0.$$

From $\operatorname{curl} H = 0$ we infer that $H = \nabla f$ for some smooth scalar function f. Substituting into $\operatorname{div} H = 0$ yields

$$\Delta f = 0,$$

so f is harmonic on D. Hence, the ambiguity in the decomposition consists precisely of adding or subtracting a *harmonic gradient field* H.

Well-posedness Before employing neural networks to compute the Helmholtz decomposition of a vector field X, we must ensure that the problem is *well-posed*, i.e. that the decomposition is unique. Uniqueness is equivalent to $H = 0$, or, equivalently, to the potential f being constant.

Consider a bounded domain $D \subset \mathbb{R}^3$ with outward unit normal N. Impose the boundary conditions

$$\langle V, N\rangle = 0, \qquad W \times N = 0 \quad \text{on } \partial D, \tag{6.1.1}$$

so that V is tangent to the boundary and W is normal to it. Any harmonic field H satisfying $\operatorname{div} H = \operatorname{curl} H = 0$ then obeys $H|_{\partial D} = 0$.

Since $H = \nabla f$ with $\Delta f = 0$, the boundary conditions reduce to the homogeneous Neumann problem

$$\Delta f = 0 \quad \text{in } D, \qquad \frac{\partial f}{\partial N} = 0 \quad \text{on } \partial D.$$

The only solution is a constant f; thus $H \equiv 0$ in D, and the decomposition $X = V + W$ is unique provided (6.1.1) holds.

The use of NN We shall start by parameterizing the vector fields, which will not be learned directly by a NN, but through the use of some latent functions.
We write V as a curl and W as a gradient,[1]

$$V = \operatorname{curl} A, \qquad W = \nabla \phi,$$

so that $X = \operatorname{curl} A + \nabla \phi$.
The latent potentials (A, ϕ) are approximated by a neural network $\big(A_\theta(x), \phi_\theta(x)\big)$, which minimizes the loss

$$\begin{aligned}\mathcal{L}(\theta) &= \int_D \big\|X - \operatorname{curl} A_\theta - \nabla\phi_\theta\big\|^2 dx + \lambda_1 \int_{\partial D} \langle \operatorname{curl} A_\theta, N\rangle^2 d\sigma + \lambda_2 \int_{\partial D} \|\nabla\phi_\theta \times N\|^2 d\sigma \\ &\approx \frac{1}{m_1} \sum_{x_i \in D} \|X(x_i) - \operatorname{curl} A_\theta(x_i) - \nabla\phi_\theta(x_i)\|^2 \\ &\quad + \frac{\lambda_1}{m_2} \sum_{y_j \in \partial D} \langle \operatorname{curl} A_\theta(y_j), N(y_j)\rangle^2 + \frac{\lambda_2}{m_2} \sum_{y_j \in \partial D} \|\nabla\phi_\theta(y_j) \times N(y_j)\|^2\end{aligned}$$

where $\lambda_i > 0$ are hyperparameters and $d\sigma$ represents the area measure along ∂D. The integrals were approximated using m_1 collocation points $\{x_i\}$ situated inside D and m_2 points $\{y_j\}$ on the boundary ∂D.

6.1.3 Integral Constraint

In many problems the conserved quantity is expressed as a *global* integral that is constant in time. We consider the conservation law as an integral functional

$$C(t) = \int_\Omega g\big(u(x,t), \nabla_{x,t} u(x,t)\big)\, dx = C(0),$$

[1] We used the operator relations $\operatorname{div} \circ \operatorname{curl} = 0$ and $\operatorname{curl} \circ \nabla = 0$, which hold on $\mathbb{R}^n$.

where g is a given smooth function of u and its space-time gradient, and $\Omega \subset \mathbb{R}^d$ is the spatial domain. Within a neural-network framework this global constraint can be enforced either as a soft constraint or as a hard embedding or both.

Several concrete examples are presented below.

Example 6.1.8 (Heat equation). Consider the one-dimensional heat equation on the interval (a, b)

$$\partial_t u(x,t) = \kappa\, \partial_x^2 u(x,t), \qquad x \in (a,b),\ t \in [0,T],$$

with initial data $u(x,0) = u_0(x)$, where $u(x,t)$ denotes the temperature and κ is the thermal diffusivity. Assume the *no-flux* Neumann boundary condition

$$\partial_x u(a,t) = \partial_x u(b,t), \qquad t \in [0,T].$$

Total heat conservation. Define the total heat at time t by

$$Q(t) = \int_a^b u(x,t)\, dx.$$

Differentiating and using the PDE and boundary condition,

$$\frac{dQ}{dt} = \int_a^b \partial_t u\, dx = \kappa \int_a^b \partial_x^2 u\, dx = \kappa\, \partial_x u\big|_{x=a}^{x=b} = 0,$$

so $Q(t) = Q(0)$ for all t, i.e. the total heat is conserved.

(*a*) *Hard embedding.* Let $f_\theta(x,t)$ be a scalar neural network that is *periodic* in x, namely $f_\theta(a,t) = f_\theta(b,t)$. Define

$$u_\theta(x,t) = u_0(x) + t\partial_x f_\theta(x,t).$$

Then the initial condition is satisfied, $u_\theta(x,0) = u_0(x)$ and

$$Q(t) = \int_a^b u_\theta(x,t)\, dx = \int_a^b u_0(x)\, dx + t\big(f_\theta(b,t) - f_\theta(a,t)\big) = Q(0),$$

so heat conservation is satisfied identically.

Training loss. Let $\{(t_j, x_k)\} \subset [0,T] \times (a,b)$ be collocation points. We compute the residual

$$R_\theta(t_j, x_k) = \partial_t u_\theta(x_k, t_j) - \kappa\, \partial_x^2 u_\theta(x_k, t_j),$$

and minimise the total

$$\mathcal{L}(\theta) = \sum_{j,k} R_\theta(t_j, x_k)^2 + \lambda \sum_j \big[f_\theta(a,t_j) - f_\theta(b,t_j)\big]^2,$$

where the second term enforces the periodic condition on f_θ.

(*b*) *Soft constraint.* The initial total heat is determined by the initial profile as

$$Q(0) \;=\; \int_a^b u_0(x)\, dx.$$

Let $u_\theta(x,t)$ be a scalar neural network with inputs x and t. Define the network's total heat

$$Q_\theta(t) \;=\; \int_a^b u_\theta(x,t)\,dx,$$

and enforce conservation through the soft penalty

$$\mathcal{L}_{\text{constr}}(\theta) = \frac{1}{m}\sum_{j=1}^{m}\Big[\,Q_\theta(t_j) - Q(0)\Big]^2.$$

The boundary and PDE residual losses are

$$\mathcal{L}_{\text{bd}}(\theta) = \frac{1}{m}\sum_{j=1}^{m}\Big[\partial_x u_\theta(a,t_j) - \partial_x u_\theta(b,t_j)\Big]^2, \qquad \mathcal{L}_{\text{PDE}}(\theta) = \sum_{j,k} R_\theta(t_j,x_k)^2,$$

where R_θ denotes the heat-equation residual. The initial condition $u_\theta(x,0) = u_0(x)$ is enforced by the loss

$$\mathcal{L}_{\text{IC}}(\theta) = \frac{1}{N}\sum_{k=1}^{N}\Big(u_\theta(x_k,0) - u_0(x_k)\Big)^2.$$

The total loss in this case is

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{PDE}}(\theta) + \lambda_1\,\mathcal{L}_{\text{bd}}(\theta) + \lambda_2\,\mathcal{L}_{\text{constr}}(\theta) + \lambda_3\,\mathcal{L}_{\text{IC}}(\theta),$$

with $\lambda_1, \lambda_2, \lambda_3 > 0$ chosen as hyper-parameters to balance the three terms.

Implementation We consider the case when the diffusion coefficient is $\kappa = 1/2$, and the space interval is $[a,b] = [0,1]$. We consider the initial temperature profile

$$u_0(x) = -2x^3 + 3x^2.$$

It is easy to verify the Neumann boundary condition for $u_0(x)$. The total heat can be computed from the initial temperature by integration

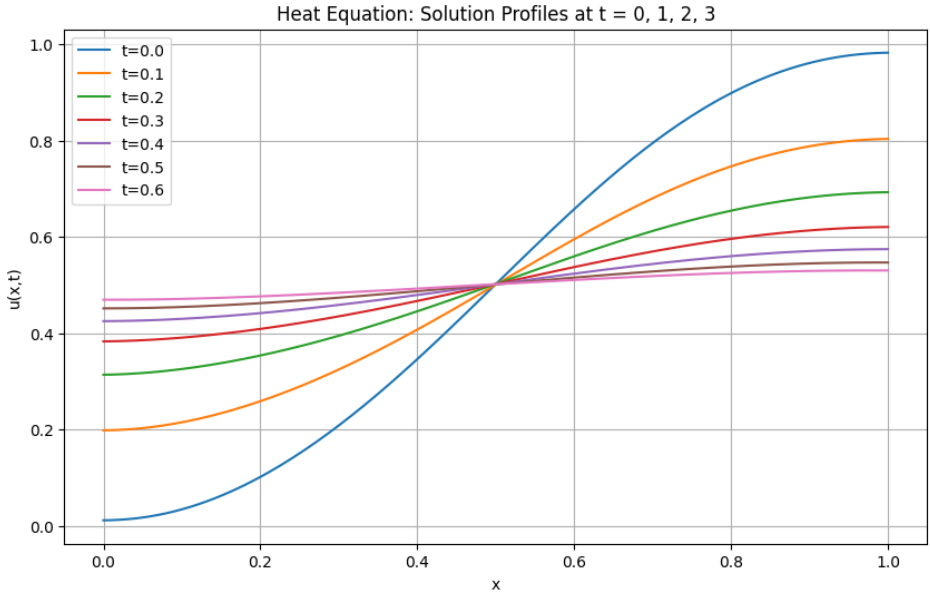
$$Q_0 = \int_0^1 u_0(x)\,dx = \frac{1}{2}.$$

The results obtained by running the code with soft constraints — trained for $1,000$ epochs using a learning rate of 0.0008, with the domain $[0,1]\times[0,1]$ divided into 50×50 collocation points, and using a neural network with 3 hidden layers of 64 neurons each with tanh activation — are shown in Fig. 6.4. The temperature profiles are plotted for different values of t.

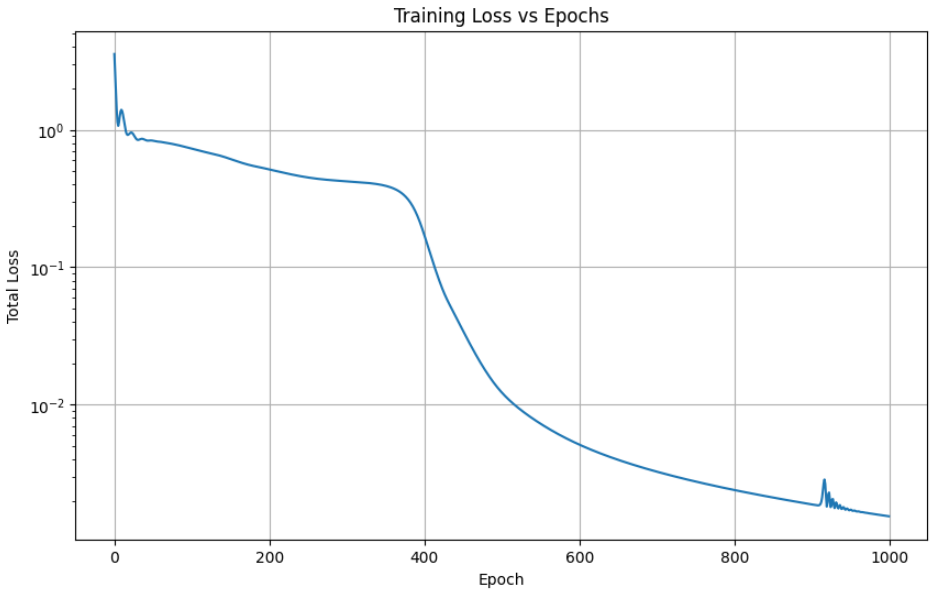
We note the existence of a fixed point for all temperature profiles: specifically, $u(0.5,t) = 0.5$ for all $t \geq 0$. This means that the temperature at the midpoint of the rod remains constant over time. This behavior is a consequence of the symmetry in the heat evolution along the rod. We also observe that $\lim_{t\to\infty} u(x,t) = 0.5$ for all $x \in [0,1]$, indicating that the system eventually reaches thermal equilibrium.

The detailed code is available at:

`https://tinyurl.com/yf9278zv`



(a) **(b)**



(a) **(b)**

Figure 6.4: (a) The temperature profiles $u(x,t)$ for $t \in \{0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6\}$; (b) The total loss versus epochs.

Example 6.1.9 (1-D Schrödinger equation). We consider the one-dimensional Schrödinger equation with a quadratic potential[2]

$$i\,\partial_t\psi(x,t) = -\frac{1}{2}\,\partial_x^2\psi(x,t) + \frac{1}{2}\,x^2\psi(x,t), \qquad x\in\mathbb{R},\ t\geq 0, \qquad \psi(x,0)=\psi_0(x).$$

Polar form. Write the solution in polar coordinates $\psi(x,t) = \rho(x,t)\,e^{i\varphi(x,t)}$, where $\rho = |\psi|$ and $\varphi = \arg\psi$. Denote $\psi_0(x) = \rho_0(x)e^{i\varphi_0(x)}$. The functions ρ and φ will be the latent variables learned by a neural network.

Probability conservation. Because $|\psi(x,t)|^2\,dx$ is the probability of finding the particle in dx, the total probability

$$P(t) = \int_{\mathbb{R}} |\psi(x,t)|^2\,dx$$

must satisfy $P(t)=1$ for all t.

Network architecture. Let the network output be the latent pair

$$\mathrm{NN}(\theta;x,t) = \big(\rho_\theta(x,t),\ \varphi_\theta(x,t)\big), \qquad \rho_\theta\geq 0,$$

and set $\psi_\theta(x,t) = \rho_\theta(x,t)\,e^{i\varphi_\theta(x,t)}$. Embed the initial data via

$$\rho_\theta(x,t) = \rho_0(x) + t\ \mathrm{nn}^1(\theta;x,t), \qquad \varphi_\theta(x,t) = \varphi_0(x) + t\ \mathrm{nn}^2(\theta;x,t),$$

where $(\mathrm{nn}^1,\mathrm{nn}^2)$ is an unconstrained two-output sub-network.

Collocation residual. On a mesh $\{(x_k,t_j)\}\subset[-A,A]\times[0,T]$ (with A,T chosen large enough) we define

$$R_\theta(x_k,t_j) = \Big|\, i\,\partial_t\psi_\theta + \tfrac{1}{2}\,\partial_x^2\psi_\theta - \tfrac{1}{2}\,x^2\psi_\theta\Big|(x_k,t_j),$$

and the PDE loss

$$\mathcal{L}_{\mathrm{PDE}}(\theta) = \frac{1}{MN}\sum_{j=1}^{N}\sum_{k=1}^{M} R_\theta(x_k,t_j)^2.$$

(*a*) *Soft probability constraint.* Impose $P(t)=1$ through

$$\mathcal{L}_{\mathrm{prob}}(\theta) = \frac{1}{N}\sum_{j=1}^{N}\Big(\int_{\mathbb{R}}\rho_\theta^2(x,t_j)\,dx - 1\Big)^2,$$

and train with

$$\mathcal{L}_{\mathrm{total}}(\theta) = \mathcal{L}_{\mathrm{PDE}}(\theta) + \lambda\,\mathcal{L}_{\mathrm{prob}}(\theta).$$

(*b*) *Hard probability embedding.* Alternatively, normalise the modulus at every time slice,

$$\widetilde{\rho}_\theta(x,t) = \frac{\rho_\theta(x,t)}{\|\rho_\theta(\cdot,t)\|_{L^2}}, \quad \psi_\theta = \widetilde{\rho}_\theta\,e^{i\varphi_\theta},$$

so that $P(t)\equiv 1$ automatically; in that case $\mathcal{L}_{\mathrm{prob}}$ can be omitted from the total loss.

[2]For simplicity we set the mass $m=1$ and Planck's constant $\hbar=1$.

6.2 Finding Conservation Laws

As shown in the previous section, *conservation laws* are useful because they can serve as regularisers for neural-network models. In this section we present several ways to identify such laws.

One approach is purely theoretical: by applying Noether-type results, one derives conserved quantities from the symmetries of the Lagrangian or Hamiltonian. A complementary approach is data-driven, using neural networks to discover conservation laws directly from trajectories and equations. This approach is postponed until Chapter 11.

6.2.1 Functional Invariance

In Physics, most Hamiltonians and Lagrangians are written as sums of squares or differences of squares. The invariance symmetries of these quadratic forms lead to conservation laws for the dynamical system.

For instance, the sum of squares $x_1^2 + x_2^2$ is invariant under the planar rotation

$$h(x_1, x_2) = (x_1 \cos\phi - x_2 \sin\phi,\ x_1 \sin\phi + x_2 \cos\phi)\,.$$

On the other hand, the difference of squares $x_1^2 - x_2^2$ is invariant under the hyperbolic rotation of the plane

$$h(x_1, x_2) = (x_1 \cosh\phi + x_2 \sinh\phi,\ x_1 \sinh\phi + x_2 \cosh\phi)\,,$$

where ϕ is a real parameter. These types of invariances give rise to conservation laws in Physics, which can be used in the regularization of Physics-Informed Neural Networks.

Invariance in the positions space. However, if the Lagrangian has a more elaborate form, it may not be straightforward to find an invariance transformation h. This is the problem we address in this section:

Problem *Let $F : \mathbb{R}^n \to \mathbb{R}$ be a differentiable function that decays rapidly at infinity, in the sense that*

$$\lim_{|x|\to\infty} e^{-\alpha|x|} F(x) = 0, \quad \textit{for some } \alpha > 0.$$

Find a diffeomorphism $h : \mathbb{R}^n \to \mathbb{R}^n$ such that

$$F(h(x)) = F(x), \quad \forall x \in \mathbb{R}^n.$$

To solve this problem, we employ a neural network $h_\theta : \mathbb{R}^n \to \mathbb{R}^n$,

$$h_\theta(x) = \left(h_\theta^1(x), \cdots, h_\theta^n(x)\right),$$

which is trained using the loss function

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{j=1}^{N} \left(F(x_j) - F(h_\theta(x_j))\right)^2,$$

where the collocation points $\{x_j\}$ are chosen near the origin, due to the rapidly decaying nature of F at infinity. Then h_{θ^*} is the desired diffeomorphism, where

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta).$$

Invertible Neural Network Construction Since the feed-forward neural network h_θ is intended to be a diffeomorphism from $\mathbb{R}^n$ to $\mathbb{R}^n$, we must impose certain assumptions and adopt specific architectural constructions.

Smoothness of activations. All activation functions are assumed to be sufficiently smooth so that the mapping

$$x \mapsto h_\theta(x)$$

is C^1-differentiable.

Equal input–output dimension per layer. Each layer of the network has width n, matching both the input and output dimensions of the overall map $h_\theta : \mathbb{R}^n \to \mathbb{R}^n$.

Invertible layer construction. To ensure that the Jacobian J_{h_θ} is nonsingular everywhere, we design each layer to be individually invertible. Since the composition of invertible smooth functions is again invertible and smooth, the entire network will then be a diffeomorphism.

To this end, we consider an arbitrary layer ℓ with input $x = (x_1, \dots, x_n)$ and output $y = (y_1, \dots, y_n)$. We randomly choose an integer k_ℓ with $1 \le k_\ell \le n$, and split the coordinates into two groups of sizes k_ℓ and $n - k_\ell$. For the sake of simplicity we assume that the first group corresponds to the first k_ℓ coordinates. Then we define the layer transformation by

$$\begin{aligned} y_j &= x_j, && 1 \le j \le k_\ell, \\ y_j &= x_j + \phi_{\theta_\ell}(x_1, \dots, x_{k_\ell}), && k_\ell < j \le n, \end{aligned}$$

where ϕ_{θ_ℓ} is a smooth activation function depending on parameters θ_ℓ and acting only on the first k_ℓ input coordinates (i.e. on the first group).

The Jacobian of this map is triangular with unit diagonal entries, hence $\det J = 1$, and the transformation is globally invertible.

A slightly more general *affine–triangular* version of the same idea is

$$\begin{aligned} y_j &= x_j, && 1 \le j \le k_\ell, \\ y_j &= x_j \, e^{\phi_j(x_1, \dots, x_{k_\ell})} + \psi_j(x_1, \dots, x_{k_\ell}), && k_\ell < j \le n, \end{aligned}$$

with smooth activation functions ϕ_j, ψ_j depending on some trainable parameters. The Jacobian of this transformation is also triangular, with determinant

$$\det J = \exp\Big(\sum_{k_\ell < j} \phi_j(x_1, \dots, x_{k_\ell}) \Big) \neq 0,$$

ensuring invertibility.

By composing several such triangular layers, each smooth and invertible, we obtain a globally invertible and continuously differentiable network h_θ. Therefore, h_θ is invertible, its inverse h_θ^{-1} is differentiable by the Inverse Function Theorem, and consequently h_θ is a *diffeomorphism* of $\mathbb{R}^n$.

Finding this invariance diffeomorphism is a static problem. From the Physics point of view, it is more appealing to deform it continuously into a one-parameter family of invariant diffeomorphisms h_s, such that

$$h_0(x) = x, \qquad h_s(h_u(x)) = h_{s+u}(x), \quad \forall x \in \mathbb{R}^n, \ s, u \ge 0.$$

The more interesting problem now is:

Problem *Find a one-parameter group of diffeomorphisms $h_s : \mathbb{R}^n \to \mathbb{R}^n$ such that*

$$F(h_s(x)) = F(x), \quad \forall x \in \mathbb{R}^n \text{ and } s \geq 0. \tag{6.2.2}$$

We start by assuming that such a family h_s exists. Then we associate a corresponding vector field V on $\mathbb{R}^n$ by

$$\frac{d}{ds} h_s(x)\Big|_{s=0} = V(x).$$

Since we also have

$$\frac{d}{ds} h_s(x) = V(h_s(x)),$$

it follows that $V(x)$ is the vector field tangent to the curves $s \mapsto h_s(x)$, which start at $h_0(x) = x$. In order to determine h_s, it suffices to find its associated vector field V.

In the following, we derive a functional relation for the vector field V. To this end, we differentiate the invariance relation $F(h_s(x)) = F(x)$ with respect to s:

$$\begin{aligned} 0 &= \frac{d}{ds} F(h_s(x)) = \frac{d}{ds} F(h_s^1(x), \cdots, h_s^n(x)) \\ &= \sum_{j=1}^n \frac{\partial F}{\partial x^j}\Big|_{h_s(x)} \frac{d}{ds} h_s^j(x) = \sum_{j=1}^n \left(\partial_j F \cdot V^j\right)\big|_{h_s(x)} \\ &= \langle \nabla F, V \rangle\big|_{h_s(x)}, \qquad \forall s \geq 0,\ \forall x \in \mathbb{R}^n. \end{aligned}$$

Since h_s is a diffeomorphism, for any $y \in \mathbb{R}^n$ there exist $x \in \mathbb{R}^n$ and $s \geq 0$ such that $y = h_s(x)$. Therefore, the above relation implies

$$\langle \nabla F(y), V(y) \rangle = 0, \quad \forall y \in \mathbb{R}^n.$$

Therefore, to find h_s, it suffices to find a vector field V that is perpendicular to the gradient ∇F. This will be done using an n-dimensional neural network

$$V_\theta(x) = \left(V_\theta^1(x), \cdots, V_\theta^n(x)\right),$$

which is trained to minimize the loss function

$$\mathcal{L}(\theta) = \int_{\mathbb{R}^n} \langle \nabla F(x), V_\theta(x) \rangle^2 \, dx \approx \frac{1}{N} \sum_{j=1}^N \left(\sum_{r=1}^n \partial_r F(x_j)\, V_\theta^r(x_j) \right)^2.$$

Denoting the optimal parameter by θ^*, we obtain the vector field V_{θ^*}, which is orthogonal to the gradient of F.

The final step is to retrieve the one-parameter family of diffeomorphisms from V by

$$h_s^\eta(x) = x + s V_{\theta^*}(x) + \frac{1}{2} s^2 \, \mathrm{nn}(\eta; s, x),$$

where $\mathrm{nn}(\eta; s, x)$ is a constraint-free neural network with parameters η and input $(s, x) \in [0, T] \times \mathbb{R}^n$.

To enforce the functional relation $F(h_s^\eta(x)) = F(x)$, we consider the loss function

$$\tilde{\mathcal{L}}(\eta) = \frac{1}{M}\sum_{k=1}^{M}\sum_{j=1}^{N}\Big(F(x_k) - F(h_{s_j}^\eta(x_k))\Big)^2,$$

for collocation sets $\{s_j\} \subset [0,T]$ and $\{x_k\} \subset \mathbb{R}^n$. By minimizing this loss, we obtain the desired candidate

$$h_s(x) = h_s^{\eta^*}(x).$$

Remark 6.2.1 (*i*) Given the function F, an invariant one-parameter family h_s of diffeomorphisms may not always exist. Furthermore, even if one exists, it is not unique, since any other vector field $\tilde{V}(x) = f(x)V(x)$, where f is a smooth function, is also perpendicular to ∇F and generates a different family $\tilde{h}_s$.
(*ii*) The group property $h_s \circ h_u = h_{s+u}$ follows from the fact that the map $s \mapsto h_s(x)$ is an integral curve of the vector field V.
(*iii*) The rapid decay of F at infinity ensures both stability and the integrability of the loss function.

Example 6.2.2 Let $F(x_1,x_2) = e^{-\frac{1}{2}(x_1^2+x_2^2)}$. Then $\nabla_x F(x) = -(x_1,x_2)\,e^{-\frac{1}{2}\rho^2(x)}$, where $\rho(x) = \sqrt{x_1^2+x_2^2}$. By inspection, we note that the vector field $V(x) = (-x_2, x_1)$ is perpendicular to ∇F. The equation $\dfrac{d}{ds}h_s(x) = V(h_s(x))$ becomes

$$\begin{aligned}\frac{dh_s^1(x)}{ds} &= -h_s^2(x),\\ \frac{dh_s^2(x)}{ds} &= \quad h_s^1(x),\end{aligned}$$

with initial conditions $h_0^1(x) = x_1$ and $h_0^2(x) = x_2$. Standard techniques for solving systems of ODEs yield the solution

$$h_s^1(x) = x_1\cos s - x_2 \sin s, \quad h_s^2(x) = x_1 \sin s + x_2 \cos s,$$

which represents a rotation by angle s of the initial point $x = (x_1, x_2)$.

6.2.2 Noether's Theorem

In the following, we consider two versions of Noether's theorem: one for the case in which the Lagrangian is invariant under spatial translations, and another in which it is invariant under time translations. The former results in the conservation of momentum, and the latter in the conservation of energy.

Spatial invariance. We consider a Lagrangian function $L : \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$, and we assume there exists a one-parameter group of diffeomorphisms $h_s : \mathbb{R}^n \to \mathbb{R}^n$ such that the following invariance property holds:

$$L(h_s(q), dh_s(\dot{q})) = L(q,\dot{q}), \qquad \forall s \geq 0,\ q,\dot{q} \in \mathbb{R}^n.$$

Here, $dh_s : \mathbb{R}^n \to \mathbb{R}^n$ denotes the differential of h_s, which is a linear operator whose matrix is the Jacobian of h_s:

$$dh_s(v) = \sum_{j,k=1}^{n} \frac{\partial h_s^j}{\partial v_k} v^k e_j,$$

where $v = (v_1, \ldots, v_n)^T$ and $e_j = (0, \ldots, \underbrace{1}_{j\text{th slot}}, \ldots, 0)^T$. We also denote by V the vector field generated by the flow $s \mapsto h_s(x)$.

Now, consider a curve $t \mapsto q(t) = (q^1(t), \cdots, q^n(t))$ in the configuration space of the dynamical system. Its velocity is $\dot{q}(t) = (\dot{q}^1(t), \cdots, \dot{q}^n(t))$. We differentiate with respect to the parameter s:

$$\begin{aligned}
0 &= \frac{d}{ds} L(h_s(q(t)), dh_s(\dot{q}(t))) \\
&= \partial_{q_k} L \cdot \frac{dh_s^k}{ds}(q(t)) + \partial_{\dot{q}_k} L \cdot \frac{d}{ds}\left(dh_s(\dot{q}(t))^k \right) \\
&= \partial_{q_k} L \cdot V^k_{h_s(q(t))} + \partial_{\dot{q}_k} L \cdot \frac{d}{ds}\left(\frac{\partial h_s^k}{\partial x_r} \dot{q}^r \right) \\
&= \partial_{q_k} L \cdot V^k_{h_s(q(t))} + \partial_{\dot{q}_k} L \cdot \frac{\partial}{\partial x_r}\left(\frac{dh_s^k}{ds} \right) \dot{q}^r(t) \\
&= \partial_{q_k} L \cdot V^k_{h_s(q(t))} + \partial_{\dot{q}_k} L \cdot \frac{\partial V^k}{\partial x_r}\Big|_{h_s(q)} \dot{q}^r(t) \\
&= \left(\partial_{q_k} L - \frac{d}{dt}(\partial_{\dot{q}_k} L) \right) V^k_{h_s(q(t))} + \frac{d}{dt}\left(\partial_{\dot{q}_k} L \cdot V^k_{h_s(q(t))} \right) \\
&= \left\langle \partial_q L - \frac{d}{dt}(\partial_{\dot{q}} L), V \right\rangle_{h_s(q(t))} + \frac{d}{dt} \langle \partial_{\dot{q}} L, V \rangle_{q(t)} .
\end{aligned}$$

Therefore, we obtain:

$$\frac{d}{dt} \langle \partial_{\dot{q}} L, V \rangle_{q(t)} = - \left\langle \partial_q L - \frac{d}{dt}(\partial_{\dot{q}} L), V \right\rangle_{h_s(q(t))}, \tag{6.2.3}$$

where $\langle \, , \, \rangle$ denotes the Euclidean scalar product. This formula yields the following version of Noether's theorem.

Theorem 6.2.3 *Assume the Lagrangian L is invariant under a one-parameter group of diffeomorphisms. Let $q(t)$ be a solution of the Euler–Lagrange equations. Then the quantity $\langle p(t), V_{q(t)} \rangle$ is conserved along the trajectory $q(t)$, where $p = \partial_{\dot{q}} L$ denotes the momentum.*

This theorem provides a conservation law of the form

$$\langle p(t), V_{q(t)} \rangle = \text{constant}$$

along any solution $q(t)$.

Example 6.2.4 The Lagrangian

$$L(q,\dot{q}) = \frac{1}{2}(\dot{q}_1^2 + \dot{q}_2^2) - \frac{1}{2}(q_1^2 + q_2^2)$$

is rotationally invariant:

$$L(q,\dot{q}) = L(R_s(q), dR_s(\dot{q})),$$

where R_s is a rotation by angle s. Since the map $x \mapsto R_s(x)$ describes a circle centered at the origin and passing through x, the associated vector field is $V(x) = (-x_2, x_1)$. Since the momenta are $p_j = \partial_{\dot{q}_j} L = \dot{q}_j$, the conservation law becomes

$$\langle p(t), V\rangle_{q(t)} = \dot{q}_2(t)q_1(t) - \dot{q}_1(t)q_2(t) = \text{constant}.$$

This represents the conservation of angular momentum.

Now, suppose the Lagrangian is invariant under n distinct one-parameter groups of diffeomorphisms $h_s^1, \ldots, h_s^n$, which are independent in the sense that their associated vector fields $V^1, \ldots, V^n$ are linearly independent at every point in $\mathbb{R}^n$. Then, by Noether's theorem, each of these symmetries generates a conservation law:

$$\langle p(t), V^j_{q(t)}\rangle = C^j, \qquad 1 \le j \le n.$$

Using equation (6.2.3) we obtain

$$\left\langle \partial_q L - \frac{d}{dt}(\partial_{\dot{q}} L), V^j \right\rangle_{h_s(q(t))} = 0,$$

and since the vector fields V^j are linearly independent, it follows that

$$\partial_q L = \frac{d}{dt}(\partial_{\dot{q}} L).$$

We arrive at the following converse statement:

If the Lagrangian admits n independent one-parameter symmetry groups, and the corresponding conservation laws hold, then $q(t)$ satisfies the Euler–Lagrange equations.

Hence, knowledge of n independent conservation laws is equivalent to the field equations.

We provide an example in this direction, following [18], page 26.

Example 6.2.5 Consider the Lagrangian with a nonholonomic constraint:

$$L(x,\dot{x}) = \frac{1}{2}(\dot{x}_1^2 + \dot{x}_2^2) + \lambda(\dot{x}_3 - 2x_2\dot{x}_1 + 2x_1\dot{x}_2),$$

where λ is a Lagrange multiplier. The momenta $p_i = \frac{\partial L}{\partial \dot{x}_i}$ are given by:

$$p_1 = \dot{x}_1 - 2\lambda x_2, \quad p_2 = \dot{x}_2 + 2\lambda x_1, \quad p_3 = \lambda.$$

There are three independent one-parameter groups of diffeomorphisms:

$$h_s(x) = (a_1(s) + x_1,\, a_2(s) + x_2,\, a_3(s) + x_3 - 2(a_1(s)x_2 - a_2(s)x_1)),$$

where $a(s) \in \{(0,0,s),(0,s,0),(s,0,0)\}$. The corresponding induced vector fields are:

$$\left.\frac{\partial h_s(x)}{\partial s}\right|_{s=0} = \begin{cases} (0,0,1), & \text{if } a(s) = (0,0,s), \\ (0,1,2x_1), & \text{if } a(s) = (0,s,0), \\ (1,0,-2x_2), & \text{if } a(s) = (s,0,0). \end{cases}$$

Since these three vector fields are linearly independent, Noether's theorem yields three functionally independent conservation laws:

$$\begin{aligned} \lambda &= \text{constant}, \\ \dot{x}_2 + 4\lambda x_1 &= \text{constant}, \\ \dot{x}_1 - 4\lambda x_2 &= \text{constant}. \end{aligned}$$

Differentiating these equations gives:

$$\begin{aligned} \ddot{x}_1 &= 4\lambda \dot{x}_2, \\ \ddot{x}_2 &= -4\lambda \dot{x}_1, \\ \lambda &= \text{constant}, \end{aligned}$$

which corresponds to the Euler–Lagrange system for the Lagrangian L.

There is an additional conservation law arising from Noether's theorem, due to the rotational invariance of the Lagrangian in the (x_1, x_2)-plane, corresponding to the one-parameter group of diffeomorphisms:

$$h_s(x_1,x_2,x_3) = (x_1\cos s - x_2 \sin s,\ x_1 \sin s + x_2 \cos s,\ x_3).$$

The associated vector field is $V = (x_2, -x_1, 0)$. The corresponding conservation law is:

$$\langle p(t), V(x(t))\rangle = p_1 x_2 - p_2 x_1 = \dot{x}_1 x_2 - \dot{x}_2 x_1 - 2\lambda(x_1^2 + x_2^2).$$

However, this conservation law is not independent. It depends on the previously identified ones.

Time invariance. We assume that the Lagrangian does not depend explicitly on time, namely $L = L(q, \dot{q})$, so

$$\frac{\partial}{\partial t} L(q(t), \dot{q}(t)) = 0.$$

Differentiating and using the Euler–Lagrange equations, we obtain:

$$\begin{aligned} \frac{d}{dt} L(q(t), \dot{q}(t)) &= \langle \partial_q L, \dot{q}\rangle + \langle \partial_{\dot{q}} L, \ddot{q}\rangle = \left\langle \frac{d}{dt}\partial_{\dot{q}} L, \dot{q} \right\rangle + \langle \partial_{\dot{q}} L, \ddot{q}\rangle \\ &= \frac{d}{dt}\left\langle \partial_{\dot{q}} L, \dot{q}\right\rangle. \end{aligned}$$

This implies

$$\frac{d}{dt}\left(\langle \partial_{\dot{q}} L, \dot{q}\rangle - L(q(t), \dot{q}(t))\right) = 0,$$

which means that the quantity

$$E = \langle \partial_{\dot{q}} L, \dot{q} \rangle - L(q(t), \dot{q}(t))$$

is constant along the trajectory $q(t)$. This is the *conserved energy* of the system. If initial conditions $q(0) = q_0$, $\dot{q}(0) = \dot{q}_0$ are given, then the energy is

$$E = E(q_0, \dot{q}_0).$$

If boundary conditions $q(0) = q_0$, $q(T) = q_T$ are given instead, then the energy takes the form

$$E = E(q_0, q_T).$$

In the first case, the energy is determined directly by substituting the initial conditions. In the latter case, an explicit expression for the energy may not always be easy to obtain.

Using the Legendre transform of the Lagrangian, we also obtain that if L does not depend explicitly on time, then the Hamiltonian is preserved along trajectories. This result follows from the Hamiltonian formalism. If L does not depend explicitly on time t, then the same is true for the Hamiltonian $H(q, p)$, as implied by the Legendre transform. Then, by Hamilton's equations:

$$\frac{d}{dt} H(q(t), p(t)) = \langle \partial_q H, \dot{q} \rangle + \langle \partial_p H, \dot{p} \rangle = \langle \partial_q H, \partial_p H \rangle + \langle \partial_p H, -\partial_q H \rangle = 0.$$

Therefore, the Hamiltonian is conserved along any trajectory of the system. Its value is the energy of the system.

The use of NN We have seen that symmetries of the Lagrangian lead to conservation laws for the underlying dynamical system. These symmetries usually are obtained by direct inspection of the Lagrangian expression. Now, the natural question that follows is: *how can we use neural networks to discover the symmetries of a given Lagrangian in the first place?*

This provides a theoretical framework for deriving conservation laws from the Lagrangian. In later chapters, we will explore how to identify conservation laws directly from data.

We begin by introducing a neural network $\mathrm{nn}(\theta; x, s)$ with an n-dimensional output, and define the transformation

$$h_s(\theta; x) = x + s\,\mathrm{nn}(\theta; x, s).$$

This construction automatically satisfies the initial condition $h_0(\theta; x) = x$. Moreover, it embeds the identity map at $s = 0$, since

$$\left.\frac{\partial h_s^j(\theta; x)}{\partial x_k}\right|_{s=0} = \delta_{jk} + s \left.\frac{\partial\, \mathrm{nn}^j(\theta; x, s)}{\partial x_k}\right|_{s=0} = \delta_{jk}.$$

Now, keeping x and v fixed, define the function

$$f_{x,v}(\theta; s) = L(h_s(\theta; x), dh_s(\theta; v)),$$

so that $f_{x,v}(\theta; 0) = L(x, v)$. The transformation h_s is said to preserve the Lagrangian if

$$f_{x,v}(\theta; s) = f_{x,v}(\theta; 0),$$

for all s. To enforce this approximately, we define the variance loss:

$$I(\theta; x, v) = \frac{1}{M} \sum_{j=1}^{M} \left(f_{x,v}(\theta; s_j) - L(x, v) \right)^2 .$$

To ensure that this invariance holds uniformly over a range of positions and velocities, we define the full loss function by averaging over a set of collocation points (x_k, v_i, s_j):

$$\mathcal{L}(\theta) = \frac{1}{M \cdot N \cdot K} \sum_{k=1}^{K} \sum_{i=1}^{N} \sum_{j=1}^{M} \left(f_{x_k,v_i}(\theta; s_j) - L(x_k, v_i) \right)^2 .$$

Minimizing this loss yields a parameter θ^* such that the learned transformation $h_s(\theta^*; x)$ is a candidate symmetry of the Lagrangian.

6.3 Conservation in Phase Space

Let $F : \mathbb{R}^n_q \times \mathbb{R}^n_p \to \mathbb{R}$ be a smooth function on phase space with canonical coordinates (q, p), and let $(q(t), p(t))$ evolve according to Hamilton's equations

$$\dot{q} = \frac{\partial H}{\partial p}, \qquad \dot{p} = -\frac{\partial H}{\partial q}.$$

We say that F is conserved along the flow if $\frac{d}{dt} F(q(t), p(t)) = 0$. By the chain rule,

$$\begin{aligned} \frac{dF}{dt} &= \frac{\partial F}{\partial q} \cdot \dot{q} + \frac{\partial F}{\partial p} \cdot \dot{p} \\ &= \frac{\partial F}{\partial q} \cdot \frac{\partial H}{\partial p} \; - \; \frac{\partial F}{\partial p} \cdot \frac{\partial H}{\partial q} \\ &= \sum_{i=1}^{n} \frac{\partial F}{\partial q_i} \cdot \frac{\partial H}{\partial p_i} \; - \; \sum_{i=1}^{n} \frac{\partial F}{\partial p_i} \cdot \frac{\partial H}{\partial q_i}. \end{aligned}$$

In $n = 1$ this equals the 2×2 determinant

$$\frac{dF}{dt} = \begin{vmatrix} \frac{\partial F}{\partial q} & \frac{\partial F}{\partial p} \\ \frac{\partial H}{\partial q} & \frac{\partial H}{\partial p} \end{vmatrix} = \operatorname{area}(\nabla F, \nabla H),$$

while in general it is the canonical *Poisson bracket*

$$\{F, H\} = \sum_{i=1}^{n} \left(\frac{\partial F}{\partial q_i} \frac{\partial H}{\partial p_i} - \frac{\partial F}{\partial p_i} \frac{\partial H}{\partial q_i} \right).$$

Hence

$$\frac{dF}{dt} = \{F, H\} \tag{6.3.4}$$

and F is conserved if and only if $\{F, H\} = 0$.

Geometric interpretation. It is useful to express this geometrically. We write $z = (q, p) \in \mathbb{R}^{2n}$ and let

$$J = \begin{pmatrix} 0 & I \\ -I & 0 \end{pmatrix}, \qquad X_H = J\nabla H = (\dot{q}, \dot{p})$$

be the symplectic matrix and the Hamiltonian vector field. Then[3]

$$\{F, H\} = \nabla F^\top J \nabla H.$$

The condition $\{F, H\} = 0$ is equivalent to $\nabla F^\top J \nabla H = 0$, i.e., $\nabla^J F := J\nabla F$ is collinear with X_H, so the flow is tangent to the level sets of F. Consequently, trajectories remain on the surface $\{F = c\}$ with $c = F(q(0), p(0))$, in agreement with the conservation of F. In particular, taking $F = H$ yields $\{H, H\} = 0$, which recovers conservation of energy.

Neural network approach We approximate the trajectory with a neural model $(q_\theta(t), p_\theta(t)) \approx (q(t), p(t))$ and train with a total loss

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \lambda\, \mathcal{L}_{\text{cons}}(\theta),$$

where $\mathcal{L}_{\text{data}}$ fits observations and $\mathcal{L}_{\text{cons}}$ enforces conservation as a soft constraint as follows:

(i) If F and H are both known. We enforce the conservation law at sampled times t_i along the predicted path:

$$\mathcal{L}_{\text{cons}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \Big(\{F, H\}\big(q_\theta(t_i), p_\theta(t_i)\big) \Big)^2.$$

(ii) If the vector field f_θ (instantaneous drift) is known or learned. If the Hamiltonian flow is given or modeled as $f_\theta(q, p) = (\dot{q}_\theta, \dot{p}_\theta)$, we use collocation points (q_i, p_i) to penalize the instantaneous time derivative of F:

$$\mathcal{L}_{\text{cons}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \Big(\nabla F(q_i, p_i)^\top\, f_\theta(q_i, p_i) \Big)^2.$$

Finding F. In concrete applications, the conserved function F on phase space is determined from empirical data (q_i, p_i) together with the known Hamiltonian function $H(q, p)$. We have a few variants as follows:

[3] In Hamiltonian mechanics we write this as $\{F, H\} = \omega(\nabla F, \nabla H)$, where ω is the symplectic form.

(i) Assuming the Hamiltonian function $H(q,p)$ is known, we need to solve the partial differential equations system

$$\sum_{i=1}^{n}\left(\frac{\partial F}{\partial q_i}\frac{\partial H}{\partial p_i}-\frac{\partial F}{\partial p_i}\frac{\partial H}{\partial q_i}\right)=0$$

or equivalently $\nabla F^{\top}\cdot\nabla^{J}H=0$, or $X_H(F)=0$, where X_H is the Hamiltonian flow. To solve this equation we can employ the method of characteristics, see Section 2.11.

(ii) In the case when the Hamiltonian function is of polynomial type, as it is in most elementary Physics problems, it makes sense to look for a conservation law under the following ansatz:

$$F(q,p)=\sum_{|\alpha|+|\beta|\leq m}c_{\alpha,\beta}q^{\alpha}p^{\beta}$$

with $\alpha=(\alpha_1,...,\alpha_n)$ and $\beta=(\beta_1,...,\beta_n)$ multiindices and $|\alpha|=\sum_i\alpha_i$.

To this end, we build a feature vector $\phi(q,p)\in\mathbb{R}^D$ containing all monomials

$$\phi_{\alpha,\beta}(z)=q^{\alpha}p^{\beta},\qquad\text{with } |\alpha|+|\beta|\leq m.$$

We parameterize by the network

$$NN(c;q,p)=F_c(q,p)=c^{\top}\phi(q,p),\qquad c\in\mathbb{R}^D,$$

where the coefficients $c_{\alpha\beta}$ are the trainable parameters. The loss is

$$\mathcal{L}(c)=\frac{1}{N}\sum_{i=1}^{N}(\{F_c,H\}(q_i,p_i))^2$$

at a set of collocation points (q_i,p_i), $1\leq i\leq N$.

(iii) If the Hamiltonian has a general form, then we learn the conservation law by a neural network

$$NN(\theta;q,p)=F_\theta(q,p)$$

such that the loss

$$\mathcal{L}(\theta)=\frac{1}{N}\sum_{i=1}^{N}(\{F_\theta,H\}(q_i,p_i))^2$$

is minimized.

6.4 Exercises

Exercise 6.4.1 (2D Harmonic Oscillator with Hard Embedding). This refers to the Exercise 6.1.6. Define:

$$q_1(t)=\sqrt{2E_0}\cos(\phi_1(t)),\qquad p_1(t)=\sqrt{2E_0}\sin(\phi_1(t)),$$
$$q_2(t)=\sqrt{2E_0}\cos(\phi_2(t)),\qquad p_2(t)=\sqrt{2E_0}\sin(\phi_2(t)),$$

where $\phi_1(t)$ and $\phi_2(t)$ are scalar-valued neural networks.

(a) Implement this model in Keras using two independent scalar neural networks for $\phi_1(t)$ and $\phi_2(t)$.

(b) Use hard embedding to enforce energy conservation automatically.

(c) Define the residual loss as:

$$\mathcal{L}_{\text{ODE}} = \|\dot{q}_1 - p_1\|^2 + \|\dot{p}_1 + q_1\|^2 + \|\dot{q}_2 - p_2\|^2 + \|\dot{p}_2 + q_2\|^2,$$

and add a loss term $\mathcal{L}_{\text{IC}}$ to encode initial conditions.

(d) Train the model and compare the neural network trajectory with the numerical solution obtained using the Runge–Kutta method.

(e) Plot $q_1(t)$ vs. t, $q_2(t)$ vs. t, and the phase portraits $(q_1(t), p_1(t))$, $(q_2(t), p_2(t))$ using scatter plots for both the learned and RK-based solutions.

Exercise 6.4.2 This exercise refers to the heat equation from Example 6.1.8. It was observed that $u(0.5, t) = 0.5$ for all values of time t.

(*a*) Use the heat equation to show that $\partial_x^2 u(x,t)\big|_{x=0.5} = 0$.

(*b*) For a fixed t, use a Taylor expansion about $x = 0.5$ to derive an approximate formula for the temperature:

$$u(x,t) \approx 0.5 + (x - 0.5)\,\eta(t) + O(x^3),$$

where $\eta(t) = \partial_x u(0.5, t)$.

(*c*) Plot the function $\eta(t)$ for $t \in [0, 5]$, and perform a parametric regression to find an elementary function that approximates $\eta(t)$.

Exercise 6.4.3 Consider the Lagrangian

$$L(q, \dot{q}) = \frac{1}{2}(\dot{q}_1^2 - \dot{q}_2^2) + \frac{1}{2}(q_1^2 - q_2^2).$$

(a) Show that L is invariant under the one-parameter group of hyperbolic rotations h_s.

(b) Find the vector field V associated with the transformations h_s.

(c) Apply Noether's theorem to derive a conservation law.

(d) Find the Euler–Lagrange equations and solve them.

(e) Use the Euler–Lagrange equations to show directly that $\dot{q}_1 q_2 - \dot{q}_2 q_1$ is a conserved quantity.

Exercise 6.4.4 Consider a Lagrangian that can be written as the sum of two Lagrangians with separable variables:

$$L(q_1, q_2, \dot{q}_1, \dot{q}_2) = L_1(q_1, \dot{q}_1) + L_2(q_2, \dot{q}_2).$$

Assume that L_1 and L_2 are invariant under the following symmetry transformations:

$$L_1(h_s(q_1),\, dh_s(\dot{q}_1)) = L_1(q_1, \dot{q}_1),$$
$$L_2(\rho_s(q_2),\, d\rho_s(\dot{q}_2)) = L_2(q_2, \dot{q}_2).$$

Let $V_{q_1} = \frac{d}{ds} h_s(q_1)\big|_{s=0}$ and $W_{q_2} = \frac{d}{ds}\rho_s(q_2)\big|_{s=0}$, and define the momenta by

$$p_1 = \frac{\partial L}{\partial \dot{q}_1}, \qquad p_2 = \frac{\partial L}{\partial \dot{q}_2}.$$

Show that the quantity

$$\langle p_1, V\rangle + \langle p_2, W\rangle$$

is a conservation law for the full Lagrangian L.

Exercise 6.4.5 A Lagrangian that does not depend explicitly on time and is invariant under a one-parameter group of diffeomorphisms possesses two conservation laws: one for energy and one for momentum.

(a) Find these two conservation laws for the Lagrangian

$$L(x, y, \dot{x}, \dot{y}) = \frac{1}{2}(\dot{x} + \dot{y}) + x\dot{y} - \dot{x}y - \frac{3}{2}(x^2 + y^2).$$

(b) Show that the solution of the Euler–Lagrange equations with initial conditions $x(0) = 4$, $\dot{x}(0) = 0$, $y(0) = 0$, and $\dot{y}(0) = 0$ traces a portion of an astroid, that is, a curve satisfying the implicit equation $x^{2/3} + y^{2/3} = 1$.

Exercise 6.4.6 (Catenary curve). Consider the Lagrangian

$$L(x, y, \dot{x}, \dot{y}) = y\sqrt{\dot{x}^2 + \dot{y}^2}.$$

(a) Show that L is invariant under x-translations, i.e., under the one-parameter group of diffeomorphisms $h_s(x, y) = (x + s, y)$.

(b) Use Noether's theorem to show that $\frac{y\dot{x}}{\sqrt{\dot{x}^2+\dot{y}^2}}$ is a conserved quantity.

(c) Show that the conservation law in (b) can be written geometrically as $y\cos\alpha = y_{\min}$, where α is the angle between the tangent line and the horizontal direction.

(d) Express $\cos\alpha$ in terms of the slope of the curve, $\tan\alpha$, and rewrite the conservation law as a differential equation.

(e) Let $(x_\theta(t), y_\theta(t))$ be a neural network which learns the catenary curve with end points $(x(0), y(0)) = (x_0, y_0)$ and $(x(1), y(1)) = (x_1, y_1)$. Write the associated loss function including the conservation law as a soft constraint.

Exercise 6.4.7 Find a diffeomorphism $h : \mathbb{R} \to \mathbb{R}$ such that $h(x)^2 = x^2$ and $h(1) = -1$. Construct such a function using a neural network model. Compare the resulting mapping with the symmetry $h(x) = -x$.

Exercise 6.4.8 Find a diffeomorphism $h : \mathbb{R}^2 \to \mathbb{R}^2$ such that

$$\|h(x)\|^2 = \|x\|^2$$

and $h(1, 0) = (0, 1)$. Construct such a mapping using a neural network, and compare the resulting transformation with the rotation about the origin by an angle of $\pi/2$.

Exercise 6.4.9 Let $F : \mathbb{R}^4 \to \mathbb{R}$ be the Lorentz norm defined by

$$F(x_1, x_2, x_3, x_4) = x_4^2 - (x_1^2 + x_2^2 + x_3^2).$$

(a) Consider the transformation $h : \mathbb{R}^4 \to \mathbb{R}^4$ given by

$$x' = h(x), \qquad \begin{cases} x_1' = \dfrac{x_1 - vx_4}{\sqrt{1-v^2}}, \\ x_2' = x_2, \\ x_3' = x_3, \\ x_4' = \dfrac{x_4 - vx_1}{\sqrt{1-v^2}}, \end{cases}$$

where $v \in (-1, 1)$ is a fixed parameter. Show by algebraic computation that h preserves the Lorentz norm, i.e.

$$F(h(x)) = F(x), \quad \text{for all } x \in \mathbb{R}^4.$$

(b) Use neural networks to construct diffeomorphisms $h : \mathbb{R}^4 \to \mathbb{R}^4$ that are invariant under F, but are distinct from the classical Lorentz transformations of $\mathbb{R}^4$ given in part (a).

Exercise 6.4.10 (Invariance in the tangent space). Let $F : \mathbb{R}^n \to \mathbb{R}$ be a smooth function. Consider a one-parameter group of diffeomorphisms $h_s : \mathbb{R}^n \to \mathbb{R}^n$ and denote by $dh_s|_x : T_x\mathbb{R}^n \to T_{h_s(x)}\mathbb{R}^n$ its differential at a point $x \in \mathbb{R}^n$, which is a linear map represented by the Jacobian matrix of h_s at x as

$$dh_s|_x(v) = \sum_{j,k=1}^{n} \frac{\partial h_s^j(x)}{\partial x_k} v^k e_j,$$

where e_j is the standard basis of $\mathbb{R}^n$.

Assume that for all vectors $v \in T_x\mathbb{R}^n$ and for all $s \geq 0$, the following invariance condition holds:

$$F(dh_s(v)) = F(v).$$

(a) Define $A_k^j(s) = \frac{\partial h_s^j}{\partial x_k}$ for notational convenience. Differentiate the invariance condition with respect to s, and show that

$$0 = \sum_{r=1}^{n} \partial_{x_r} F|_{dh_s(v)} \cdot \frac{d}{ds} A_k^r(s) v^k.$$

(b) Let V be the vector field associated with the one-parameter group h_s, defined by

$$V(x) = \frac{d}{ds} h_s(x)\Big|_{s=0}.$$

Use the commutation of derivatives to show that

$$\frac{d}{ds}A_k^r(s) = \left.\frac{\partial V^r}{\partial x_k}\right|_{h_s(x)}.$$

(c) Conclude that at $s = 0$, the following orthogonality condition holds:

$$\langle \nabla F(v), W(x) \rangle = 0, \qquad \forall x \in \mathbb{R}^n,\ v \in T_x\mathbb{R}^n,$$

where the vector field W is given by the directional derivative of V in the direction v:

$$W^r(x) = \sum_k \frac{\partial V^r}{\partial x_k} v^k = \left.\frac{d}{dt}V^r(x + tv)\right|_{t=0}.$$

This page intentionally left blank

Chapter 7

Hamilton–Jacobi Theory

7.1 Hamilton–Jacobi Equation

For a given Hamiltonian function $H(q, p, t)$, the nonlinear partial differential equation

$$\partial_t S + H(q, \partial_q S, t) = 0 \tag{7.1.1}$$

with $S = S(q, t)$, is called the *Hamilton-Jacobi equation*.

In general, a partial differential equation is more difficult to solve than a system of ordinary differential equations, such as the Hamiltonian system. However, when the Hamiltonian function does not depend explicitly on time t, the Hamilton-Jacobi equation can be approached by the method of separation of variables. This involves seeking a solution of the form

$$S(q_0, q, t) = T(t) + \Phi(q_0, q),$$

for which there exists a separation constant $C > 0$ such that

$$\partial_t T(t) = -C, \qquad H(q, \partial_q \Phi) = C.$$

Therefore, the solution takes the form

$$S(q_0, q, t) = -Ct + \Phi(q_0, q) + T_0, \tag{7.1.2}$$

where T_0 is an integration constant and $H(q, \partial_q \Phi) = C$.

There are two main challenges here:

(i) Solving the stationary equation $H(q, \partial_q \Phi) = C$, and

(ii) Expressing the constant C, which is a conserved quantity along the Hamiltonian solution, in terms of the boundary conditions, i.e., finding $C = C(q_0, q)$.

Since these problems often cannot be solved explicitly in closed form, the hope is that this difficulty can be mitigated by employing neural networks.

Knowing the solution S is valuable in two respects, which will be addressed in this chapter:

1. It can be used to integrate the dynamical system defined by the Hamiltonian H, and

2. It can be used to compute the Riemannian distance on curved spaces.

Since the Hamilton–Jacobi equation is nonlinear, it admits multiple solutions, which do not form a linear space, unlike linear ordinary differential equations whose solutions always form a linear space. Beyond seeking a solution in the form (7.1.2), one can identify a distinguished solution of geometric significance by employing a powerful approach based on the action integral along a trajectory of the dynamical system. The result can be stated as follows:

Proposition 7.1.1 *The action integral*

$$S = S(q_0, q, t) = \int_0^t L(q(s), \dot{q}(s))\, ds$$

satisfies the Hamilton–Jacobi equation with the initial condition $\lim_{t\to 0} S = 0$*, where* L *is the Lagrangian of the system and* $q(s)$ *satisfies the Euler–Lagrange equation*

$$\frac{\partial L}{\partial q} = \frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}}\right),$$

subject to the boundary conditions $q(0) = q_0$*,* $q(t) = q$*.*

The time t *is assumed to be sufficiently small so that there are no conjugate points along the path* $q(s)$ *with respect to the initial point* q_0*.*

Proof: Let q_0 be a fixed point in the configuration space. The action $S = S(q_0, q, t)$ depends on q and t. We shall compute the differential of S in two ways. Using the definition of the differential,

$$dS = \frac{\partial S}{\partial t}\, dt + \frac{\partial S}{\partial q}\, dq.$$

Taking the differential inside the integral and using the Legendre transform, we have

$$\begin{aligned} dS &= \frac{dS}{dt}\, dt = \frac{d}{dt}\int_0^t L\, ds\, dt = L(q(t), \dot{q}(t))\, dt \\ &= \big(p(t)\dot{q}(t) - H(q(t), p(t))\big)\, dt = p\, dq - H\, dt, \end{aligned}$$

which is also called the Poincaré one-form. Therefore,

$$dS = p\, dq - H(q, p)\, dt.$$

Using now that the one-forms $\{dt, dq\}$ are linearly independent, equating their coefficients in the two expressions for dS yields

$$\frac{\partial S}{\partial q} = p, \qquad \frac{\partial S}{\partial t} = -H(q, p).$$

Substituting the first equation into the latter gives

$$\frac{\partial S}{\partial t} = -H\left(q, \frac{\partial S}{\partial q}\right),$$

which is the Hamilton–Jacobi equation. ∎

A useful fact used later is the following consequence:

Corollary 7.1.2 *The integral action satisfies the equation*

$$\frac{\partial S}{\partial q} = p,$$

where $p = \dfrac{\partial L}{\partial \dot{q}}$.

Remark 7.1.3 Finding the solution S as an action integral involves solving the Euler–Lagrange equations and then integrating along the solution $q(s)$, which depends on its endpoints q_0 and $q = q(t)$. This method requires the explicit expression of the solution $q(s)$, which in turn we aim to determine. Thus, the inverse approach is usually adopted in the literature. This means that S is found by another method (such as separation of variables), after which we compute the momentum $p = \frac{\partial S}{\partial q}$. Since p and L are then known, solving the equation $\frac{\partial L}{\partial \dot{q}} = p$ for $q = q(t)$ produces the dynamical trajectory.

This approach can be carried out in closed form only for certain particular Lagrangians; the general case must be handled by neural networks.

A few explicitly worked examples will clarify the discussion.

Example 7.1.1 (1D Free Particle). The dynamics of a free particle of unit mass is described by the Lagrangian $L(q, \dot{q}) = \dfrac{1}{2}\dot{q}^2$, or equivalently, by the Hamiltonian function $H(q, p) = \dfrac{1}{2}p^2$. The Hamilton–Jacobi equation becomes

$$\frac{\partial S}{\partial t} + \frac{1}{2}\left(\frac{\partial S}{\partial q}\right)^2 = 0.$$

Using the method of separation of variables, we have $S = -Ct + \Phi(q_0, q)$, where

$$\left(\frac{d\Phi}{dq}\right)^2 = 2C > 0.$$

We choose the solution[1]

$$\Phi(q) = \sqrt{2C}\, q + \rho_0,$$

[1]We could also choose the conjugate solution $\Phi(q) = -\sqrt{2C}\, q + \rho_0$. A similar computation leads to the same expression for the solution $q(s)$.

where the constant ρ_0 is determined from the initial condition $\Phi(q_0) = 0$, yielding

$$\Phi(q) = \sqrt{2C}\,(q - q_0).$$

Therefore, the action is

$$S = -Ct + \sqrt{2C}\,(q - q_0).$$

Then the momentum becomes

$$p = \frac{\partial S}{\partial q} = \sqrt{2C},$$

i.e., it is constant along solutions. Coupling this with $p = \dfrac{\partial L}{\partial \dot{q}}$ gives

$$\dot{q}(s) = \sqrt{2C},$$

which can be integrated in closed form as

$$q(s) = \sqrt{2C}\,s + q_0.$$

The constant C can be determined from the boundary condition $q = q(t)$ as

$$\sqrt{2C} = \frac{q - q_0}{t}.$$

We have obtained the trajectory

$$q(s) = (q - q_0)\frac{s}{t} + q_0,$$

which is just a straight line passing through q_0 and q.

The solution S can be further computed in terms of the boundary values q_0 and q by substituting C back into its expression:

$$\begin{aligned} S &= -Ct + \sqrt{2C}\,(q - q_0) \\ &= -\frac{(q - q_0)^2}{2t^2}t + \frac{q - q_0}{t}(q - q_0) \\ &= \frac{(q - q_0)^2}{2t}. \end{aligned}$$

We note that the distance between q_0 and q can be expressed in terms of the action as

$$|q - q_0| = \sqrt{2tS}.$$

A similar relation holds in the general case of quadratic Hamiltonians, where the distance corresponds to a Riemannian distance along a curved configuration space.

Example 7.1.2 (1D Particle in a Potential). The dynamics of a unit mass particle in a potential $U(q)$ is described by the Lagrangian $L(q, \dot{q}) = \dfrac{1}{2}\dot{q}^2 - U(q)$, or equivalently,

by the Hamiltonian function $H(q,p) = \frac{1}{2}p^2 + U(q)$. The Hamilton–Jacobi equation becomes

$$\frac{\partial S}{\partial t} + \frac{1}{2}\left(\frac{\partial S}{\partial q}\right)^2 + U(q) = 0.$$

If try to use the method of separation of variables, we have

$$S = -Ct + \Phi(q),$$

where

$$\frac{1}{2}\left(\frac{\partial \Phi}{\partial q}\right)^2 = C - U(q).$$

This equation can be solved by separation of variable method, obtaining

$$\int_{\Phi_0}^{\Phi} \frac{dw}{\sqrt{2(C-U(w))}} = q - q_0. \tag{7.1.3}$$

The integral cannot be computed by quadratures for any potential function. The next example deals with the quadratic potential case.

Example 7.1.3 (Hyperbolic Oscillator). We consider the dynamics of a particle of unit mass in a negative quadratic potential

$$U(q) = -\frac{a^2}{2}q^2,$$

where $a > 0$ is a constant. The associated Lagrangian is

$$L(q,\dot{q}) = \frac{1}{2}\dot{q}^2 + \frac{a^2}{2}q^2,$$

and the Hamiltonian function is

$$H(q,p) = \frac{1}{2}p^2 - \frac{a^2}{2}q^2.$$

The Hamilton–Jacobi equation becomes

$$\frac{\partial S}{\partial t} + \frac{1}{2}\left(\frac{\partial S}{\partial q}\right)^2 = \frac{a^2}{2}q^2.$$

It can be shown, either by integrating the integral (7.1.3) or by direct verification, that in this case the solution takes the form

$$S(q_0,q,t) = \frac{a}{2\sinh(at)}\left[(q_0^2+q^2)\cosh(at) - 2qq_0\right], \qquad t > 0.$$

The momentum p can be computed in two ways: from the Lagrangian we have

$$p = \frac{\partial L}{\partial \dot{q}} = \dot{q},$$

and from the solution S we obtain

$$p = \frac{\partial S}{\partial q} = \frac{a}{\sinh(at)} \left(q \cosh(at) - q_0\right).$$

Equating these expressions yields the following equation for the dynamics:

$$\dot{q}(t) = \frac{a}{\sinh(at)} \left(q \cosh(at) - q_0\right), \qquad q(0) = q_0.$$

This can be integrated in closed form by multiplying by the integrating factor $\mu(t) = \frac{1}{\sinh(at)}$. We obtain

$$\frac{d}{dt}\left(\frac{q(t)}{\sinh(at)}\right) = -\frac{aq_0}{\sinh^2(at)},$$

which, upon integration, yields

$$q(t) = q_0 \cosh(at) + \kappa \sinh(at),$$

where κ is the integration constant, given by $\kappa = \dot{q}(0)$. If $q = q(t)$ is given, then

$$q(s) = q_0 \cosh(as) + \frac{q - q_0 \cosh(at)}{\sinh(at)} \sinh(as), \quad 0 \le s \le t.$$

7.2 Using Neural Networks

In general, one may use a neural network to find the dynamics via the solution of the Hamilton–Jacobi equation as follows:

1. Use a first neural network to find a solution S of the Hamilton–Jacobi equation.
2. Use another neural network to solve for $q(t)$ such that

$$\frac{\partial S}{\partial q} = \frac{\partial L}{\partial \dot{q}}.$$

We assume that none of the above equations can be solved explicitly.

7.2.1 Solving the Hamilton–Jacobi Equation with NN

Neural Network Representation We introduce a neural network $S_\theta(q, t)$, parameterized by weights and biases θ, to approximate the true solution $S(q, t)$. The network takes (q, t) as input and produces a scalar output:

$$(q, t) \mapsto S_\theta(q, t) \in \mathbb{R}.$$

Smooth activation functions such as tanh or softplus are preferred to ensure differentiability, since the Hamilton–Jacobi equation requires both spatial and temporal derivatives of S_θ.

Physics-Informed Loss Function The key idea is to construct a *physics-informed* loss function that enforces the PDE (7.1.1) at a set of collocation points.

HJ Residual Loss. We compute the momentum field

$$p_\theta(q,t) = \partial_q S_\theta(q,t)$$

via automatic differentiation. The HJ residual at (q,t) is

$$\mathcal{R}_\theta(q,t) := \frac{\partial S_\theta}{\partial t}(q,t) + H\big(q, p_\theta(q,t), t\big).$$

The residual loss is then

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_{\text{coll}}} \sum_{(q,t)\in\mathcal{C}} |\mathcal{R}_\theta(q,t)|^2 ,$$

where $\mathcal{C}$ is the set of collocation points and $N_{\text{coll}} = \text{card}(\mathcal{C})$.

Initial Condition Loss. The initial condition $S(q_0,t) = 0$ is enforced for any t by

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_{\text{IC}}} \sum_{t\in\mathcal{I}} |S_\theta(q_0,t)|^2 ,$$

where $\mathcal{I}$ is a set of collocation time instances.

Total Loss. The total loss function is

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{PDE}}(\theta) + \lambda_{\text{IC}}\, \mathcal{L}_{\text{IC}}(\theta),$$

where $\lambda_{\text{IC}} > 0$ balances the PDE and initial condition terms.

Training Procedure The training algorithm proceeds as follows:

1. *Sampling.* Generate collocation points (q,t) in the interior of the domain and initial condition points $(q,0)$.

2. *Automatic Differentiation.* Use an automatic differentiation framework to compute $\partial_t S_\theta$ and $\partial_q S_\theta$ exactly within the computational graph.

3. *Optimization.* Minimize $\mathcal{L}_{\text{total}}$ using a first-order optimizer (e.g., Adam) for initial convergence, followed by a quasi-Newton optimizer (e.g., L-BFGS) for fine-tuning.

4. *Validation.* Validate S_θ against known analytic solutions (if available) or check physical invariants, such as conservation of energy or momentum, implied by the Hamiltonian.

7.2.2 Solving for the Dynamics

Once the neural network $S_\theta(q,t)$ has been trained to minimize the total loss

$$\theta^* = \arg\min_\theta \mathcal{L}_{\text{total}}(\theta),$$

we obtain an approximation $S_{\theta^*}(q,t)$ to the solution function $S(q,t)$. The next step is to recover the actual dynamics $q(t)$ of the system from this solution.
Recall that, in Hamilton–Jacobi theory, the momentum is given by

$$p(q,\dot{q}) = \frac{\partial L(q,\dot{q})}{\partial \dot{q}},$$

where $L(q,\dot{q})$ is the system's Lagrangian. Since the Lagrangian is assumed known, this derivative can be computed *classically*, without resorting to autodifferentiation.
From the definition of S, we also have

$$p(q,t) = \frac{\partial S_{\theta^*}}{\partial q}(q,t).$$

Equating the two momentum expressions gives the functional equation for the trajectory $q(t)$:

$$\frac{\partial S_{\theta^*}}{\partial q}\big(q(t),t\big) = p\big(q(t),\dot{q}(t)\big). \tag{7.2.4}$$

Neural Network Representation of the Trajectory We now approximate the trajectory $q(t)$ itself using a second neural network

$$q_w : t \mapsto q_w(t),$$

where w denotes the trainable parameters (weights and biases) of the network. The input is time t, and the output is the predicted generalized coordinate $q_w(t)$.

Given $q_w(t)$, we can compute its time derivative $\dot{q}_w(t)$ via automatic differentiation with respect to t. This allows us to evaluate the right-hand side of (7.2.4) through the known expression for $p(q,\dot{q})$.

Loss Function for the Dynamics To find the solution $q(t)$, we construct a *dynamics loss* by enforcing (7.2.4) at a discrete set of time points $\{t_j\}_{j=1}^N$:

$$\mathcal{L}_{\text{dyn}}(w) = \frac{1}{N}\sum_{j=1}^{N}\left\| \frac{\partial S_{\theta^*}}{\partial q}\big(q_w(t_j),t_j\big) - p\big(q_w(t_j),\dot{q}_w(t_j)\big)\right\|^2.$$

Initial conditions are enforced by an additional term

$$\mathcal{L}_{\text{IC}}^{(q)}(w) = \left\|q_w(0) - q_0\right\|^2.$$

The total loss for this stage becomes

$$\mathcal{L}_{\text{total}}^{(q)}(w) = \mathcal{L}_{\text{dyn}}(w) + \lambda_{\text{IC}}^{(q)}\mathcal{L}_{\text{IC}}^{(q)}(w),$$

where $\lambda_{\text{IC}}^{(q)} > 0$ balances the dynamics constraint and the initial condition.

We make the remark that for some Lagrangians, such as $L(q,\dot{q}) = \frac{1}{2}\dot{q}^2 + U(q)$, the loss $\mathcal{L}_{\text{dyn}}$ takes the simplified form

$$\mathcal{L}_{\text{dyn}}(w) = \frac{1}{N}\sum_{j=1}^{N}\left\|\frac{\partial S_{\theta^*}}{\partial q}\big(q_w(t_j),t_j\big) - \dot{q}_w(t_j)\right\|^2.$$

Training the Trajectory Network The training of the trajectory network $q_w(t)$ begins by sampling time points within the domain of interest and computing both $q_w(t_j)$ and its time derivative $\dot{q}_w(t_j)$ using automatic differentiation. At the same points, the gradient of the previously trained action function S_{θ^*} with respect to q is evaluated, also via automatic differentiation, giving $\partial_q S_{\theta^*}(q_w(t_j),t_j)$. These quantities are then compared through the momentum–velocity relation $p(q,\dot{q})$ derived from the known Lagrangian, producing the dynamics loss $\mathcal{L}_{\text{dyn}}(w)$.

An additional term $\mathcal{L}_{\text{IC}}^{(q)}(w)$ enforces the initial condition $q_w(0) = q_0$, and the total loss $\mathcal{L}_{\text{total}}^{(q)}(w)$ combines both contributions with a weighting parameter. The network parameters w are optimized to minimize this total loss, typically using a gradient-based method such as Adam, ensuring that the learned trajectory satisfies both the Hamilton–Jacobi-derived momentum condition and the prescribed initial data.

Conclusion After optimization, the network $q_{w^*}(t)$ approximates the physical trajectory consistent with both the Hamilton–Jacobi solution S_{θ^*} and the known momentum–velocity relation from the Lagrangian. This is a two-stage approach:

1. First learns $S(q,t)$ via a Physics-Informed Neural Network satisfying the Hamilton–Jacobi equation.

2. Then reconstructs $q(t)$ by solving the consistency equation (7.2.4) with a second neural network.

This separation of concerns makes the method flexible and modular: the S-network captures the underlying dynamics in phase space, while the q-network extracts specific trajectories corresponding to chosen initial conditions.

7.2.3 Practical Implementation

We shall consider an implementation example which refers to the free particle of Example 7.1.2. In this example the Lagrangian is $L = \frac{1}{2}\dot{q}^2$ and the momentum is given by $p = \dot{q}$. We look for the solution which satisfies the boundary conditions $q(0) = 1$ and $q(1) = 3$, by means of the action S by solving the equation $\dot{q} = \dfrac{\partial S}{\partial q}$.

In the implementation, two separate neural networks were employed, each tailored to a different stage of the problem. The first network, $S_\theta(q,t)$, was designed to approximate the solution of the Hamilton–Jacobi equation for the action $S(q,t)$ corresponding to the free particle Lagrangian $L(q,\dot{q}) = \frac{1}{2}\dot{q}^2$. Its architecture consisted of a fully connected feed-forward neural network with three hidden layers of 64 neurons each,

equipped with `tanh` activation functions to capture the smooth nonlinear structure of the solution.

The second network, $q_w(s)$, was trained to recover the trajectory $q(s)$, guided by the relation $\dot{q} = \partial_q S$ and the imposed boundary conditions $q(0) = 1$ and $q(1) = 3$. This trajectory network had a similar architecture with two hidden layers of 32 neurons.

Both models were trained using the Adam optimizer with a learning rate of 10^{-3}. The action network required approximately $1,000$ epochs for convergence, ensuring stability of the Hamilton–Jacobi solution, while the trajectory network converged more rapidly, within 500 epochs, due to the direct enforcement of the dynamical constraint and boundary conditions.

At the end we compared the neural solution $q_{w^*}(s)$ with the exact trajectory

$$q_{\text{exact}}(s) = 2s + 1,$$

which follows directly from solving the Euler-Lagrange equation with the imposed boundary data. The comparison demonstrates that the neural network is able to recover the linear interpolation dictated by the principle of stationary action, see Fig. 7.1. The full detailed code can be found at

`https://tinyurl.com/yhbew2w9`

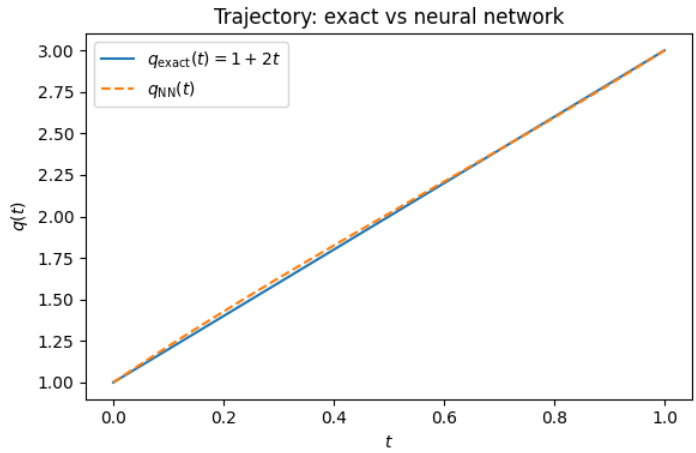


Figure 7.1: The network solution $q_{w^*}(t)$ and the exact solution $q_{\text{exact}}(t) = 2t + 1$ look pretty closed to each other.

7.3 The Eiconal Equation

In the case where the Hamiltonian function does not depend explicitly on time, the method of separation of variables can be applied to solve the Hamilton–Jacobi equation.

We have seen that its solution may then be written in the form

$$S(q,t) = -Ct + \Phi(q),$$

where Φ satisfies

$$H(q, \partial_q \Phi) = C, \qquad \Phi(q_0) = 0,$$

with $C > 0$ a constant. We shall refer to the relation

$$H(q, \partial_q \Phi) = C \tag{7.3.5}$$

as the *generalized eiconal equation*. In particular, when the Hamiltonian is quadratic in the momenta, this reduces to the standard eiconal equation

$$\|\partial_q \Phi\|^2 = 2C.$$

As an alternative way to recover the dynamics of the trajectory $q(s)$, one may employ the eiconal equation in place of the Hamilton–Jacobi equation. The main advantage of this approach is that it eliminates the explicit dependence on time t, thereby reducing the problem by one variable.

Before proceeding further, we shall revisit Example 7.1.3 using the eiconal equation.

Example 7.3.1 (Hyperbolic Oscillator). We recall that in this case the Lagrangian and the Hamiltonian are given respectively by

$$L(q,\dot{q}) = \tfrac{1}{2}\dot{q}^2 + \tfrac{a^2}{2}q^2, \qquad H(q,p) = \tfrac{1}{2}p^2 - \tfrac{a^2}{2}q^2,$$

with $a > 0$ constant. The eiconal equation (7.3.5) becomes

$$\tfrac{1}{2}\left(\tfrac{d\Phi}{dq}\right)^2 - \tfrac{a^2}{2}q^2 = C.$$

Thus the solution function Φ satisfies

$$\begin{aligned} \left(\tfrac{d\Phi}{dq}\right)^2 &= 2C + a^2 q^2, \\ \Phi(q_0) &= 0. \end{aligned}$$

On the other hand, the relation $p = \dfrac{\partial S}{\partial q}$ is equivalent to $\dot{q} = \dfrac{d\Phi}{dq}$. This can be written as an ODE for $q(s)$:

$$\begin{aligned} \frac{dq(s)}{ds} &= \sqrt{2C + a^2 q(s)^2}, \\ q(0) &= q_0, \quad q(t) = q, \end{aligned}$$

where we assumed $q > q_0$, so the positive square root is appropriate.

The above first-order ODE has two boundary conditions, so it appears overdetermined. However, since C is a parameter (the system's energy), its value is fixed by the

final condition $q(t) = q$. Using separation of variables and integrating between 0 and s gives

$$\int_{q_0}^{q(s)} \frac{dx}{\sqrt{2C + a^2x^2}} = s \quad \Leftrightarrow \quad \frac{1}{a}\sinh^{-1}\Big(\tfrac{ax}{\sqrt{2C}}\Big)\Big|_{x=q_0}^{x=q(s)} = s.$$

Solving for $q(s)$ and applying the identities

$$\sinh(x+y) = \sinh x\cosh y + \cosh x \sinh y, \qquad \cosh(\sinh^{-1} x) = \sqrt{1+x^2},$$

we obtain

$$q(s) = q_0\cosh(as) + \frac{1}{a}\sqrt{2C + a^2q_0^2}\,\sinh(as), \qquad 0 \le s \le t.$$

Setting $s = t$ and using $q(t) = q$ yields

$$\frac{1}{a}\sqrt{2C + a^2q_0^2} = \frac{q - q_0\cosh(at)}{\sinh(at)}.$$

Substituting back into the previous expression gives the trajectory

$$q(s) = q_0\cosh(as) + \frac{q - q_0\cosh(at)}{\sinh(at)}\sinh(as), \qquad 0 \le s \le t.$$

There are a few remarks to make now:

1. We do not need to solve the eiconal equation explicitly for $\Phi(q)$ in order to determine the dynamical trajectory $q(s)$.

2. The constant of energy C depends on the boundary conditions q_0, q, the time t, and the parameter a.

Using neural networks Using the insights gained from the preceding example, we are now in a position to propose a neural network–based formulation of this problem. We consider that the boundary conditions q_0, q are given. The key idea is to design a network that simultaneously learns the constant C, the solution Φ, and the trajectory $q(s)$.

There are two possible variants:

1. The constant C is considered as a trainable parameter of the network. In this case, we introduce a neural network with parameters $w = (\theta, C)$ and two outputs, defined as

$$NN(w; s) \;=\; \big(\Phi_w(q),\; q_w(s)\big).$$

The training objective is to minimize the total loss

$$\begin{aligned}\mathcal{L}_{\text{tot}}(w) \;&=\; \|H(q_w, \partial_q\Phi_w) - C\|^2 + \Big\|\frac{\partial L}{\partial \dot q}(q_w, \dot q_w) - \partial_q\Phi_w\Big\|^2 \\ &\quad + \|\Phi_w(q_0)\|^2 + \|q_w(0) - q_0\|^2 + \|q_w(t) - q\|^2.\end{aligned}$$

The first term enforces the generalized eiconal equation, the second aligns the momentum derived from the Lagrangian with the gradient of the phase function Φ, the next term anchors $\Phi(q_0)$ to zero, and the last two terms ensure that the learned trajectory satisfies the prescribed boundary conditions.

2. The constant C is learned by the network as an output. More precisely, we consider a neural network of parameter θ with three outputs and one-dimensional input $s \in [0, t]$:

$$NN(\theta; s) \;=\; \big(C_\theta(s), \Phi_\theta(q),\; q_\theta(s)\big).$$

The loss encodes the fact that C is constant by considering the extra soft-constraint $|\partial_s C_\theta(s)|^2$:

$$\begin{aligned}\mathcal{L}_{\text{tot}}(\theta) \;&=\; \|H(q_\theta, \partial_q \Phi_\theta) - C_\theta\|^2 + \Big\| \frac{\partial L}{\partial \dot{q}}(q_\theta, \dot{q}_\theta) - \partial_q \Phi_\theta \Big\|^2 \\ &\quad + \|\partial_s C_\theta\|^2 + \|\Phi_\theta(q_0)\|^2 + \|q_\theta(0) - q_0\|^2 + \|q_\theta(t) - q\|^2.\end{aligned}$$

For the purpose of practical implementation, the above norms are evaluated as sums of squares over a set of collocation points. For example, the first term is approximated by

$$\|H(q_\theta, \partial_q \Phi_\theta) - C_\theta\|^2 \;\approx\; \frac{1}{N} \sum_{j=1}^{N} \Big(H\big(q_j, \partial_q \Phi(q_j)\big) - C_\theta(s_j) \Big)^2,$$

where $s_j \in [0, t]$ are the collocation points and $q_j = q_\theta(s_j)$ denotes the corresponding trajectory samples.

Also for the sake of simplicity, instead of minimizing $\|\partial_s C_\theta\|^2$ we prefer to minimize the variance

$$Var(C_\theta) = \frac{1}{N} \sum_j (C_\theta(s_j) - \overline{C}_\theta)^2,$$

with $\overline{C}_\theta = \frac{1}{N} \sum_j C_\theta(s_j)$, and to add it as an additive term to the total loss.

We also make the note that to reduce the training time we can remove the boundary soft constraints and replace them by the hard embedding

$$q_\theta(s) = q_0 + (q - q_0)\frac{s}{t} + s(t - s)\mathrm{nn}_\theta(s), \tag{7.3.6}$$

where $\mathrm{nn}_\theta(s)$ appears as the third component of the network as

$$NN(\theta; s) \;=\; \big(C_\theta(s), \Phi_\theta(q),\; \mathrm{nn}_\theta(s)\big).$$

7.3.1 Practical Implementation

We consider the case of the hyperbolic oscillator from Example 7.3.1, with boundary conditions $q_0 = 1$, $q(3) = 6$, and parameter $a = 1$. Instead of fixing the integration constant, we introduce it as a trainable scalar

```
C_var = tf.Variable(0.1,dtype=tf.float64,trainable=True, name="C").
```

The trajectory network $q_\theta(s)$ is a fully connected feed-forward neural network with three hidden layers of 64 neurons each, using tanh activation. Boundary conditions are enforced through the parametrization in (7.3.6). The phase network $\Phi_\theta(q)$ has as similar architecture as $q_\theta(s)$.

The total loss function is a combination of several loss functions

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{eic}}(\theta) + \mathcal{L}_{\text{dyn}}(\theta) + \mathcal{L}_{\phi_0}(\theta),$$

where

$$\mathcal{L}_{\text{eic}}(\theta) = \left\| \tfrac{1}{2}\big(\partial_q \Phi_\theta\big)^2 - \tfrac{a^2}{2}\, q_\theta^2 - C \right\|^2,$$

enforces the eiconal equation,

$$\mathcal{L}_{\text{dyn}}(\theta) = \left\| \dot{q}_\theta - \partial_q \Phi_\theta \right\|^2,$$

captures the dynamics, and

$$\mathcal{L}_{\phi_0}(\theta) = \|\Phi_\theta(q_0)\|^2,$$

enforces the initial condition for the phase function $\Phi(q)$.

The model was trained with the Adam optimizer using a learning rate of 5×10^{-5}. After 10,000 training epochs, we compare the exact and learned solutions in Fig. 7.2(a). The evolution of the individual loss terms and the total loss is shown in Fig. 7.2(b).

We note that for small values of s there remains a slight discrepancy between the exact solution and the neural network approximation. To reduce this mismatch, instead of uniformly sampling collocation points we introduced a distribution biased towards $s = 0$, given by

```
u ~ Uniform(0,1),
s = tT * u**k_bias + eps
```

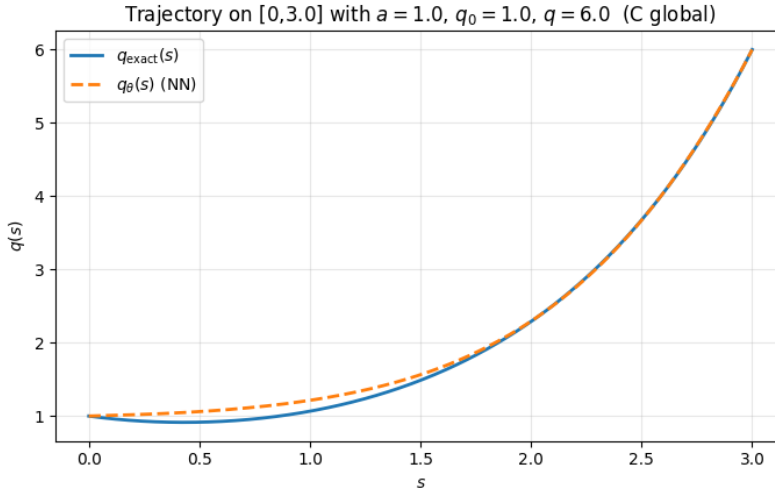
with `k_bias = 2.9` This refinement reduces the gap, although a complete closure requires incorporating an initial velocity condition, $\dot{q}(0) = p_0$. The full detailed code can be found at

`https://tinyurl.com/yhbew2w9`

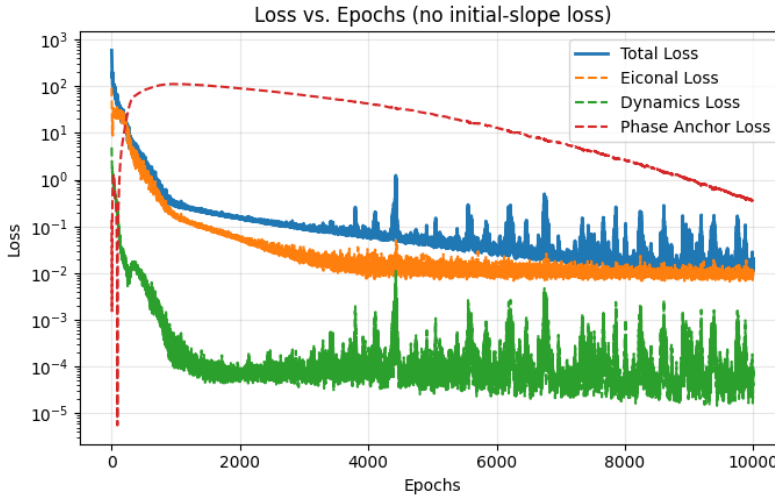
7.4 Finding Riemannian Distance

In differential geometry, finding the Riemannian distance between two points is a central problem of study. We consider a Riemannian manifold $(\mathbb{R}^n, g_{ij})$, where $g_{ij}(q)$ are the components of the metric tensor. At each point q, the metric tensor is a symmetric, nondegenerate, and positive-definite matrix. The length of any vector $v \in T_q\mathbb{R}^n$ is given by

$$\|v\|_g = \sqrt{\sum_{i,j=1}^{n} g_{ij}(q) v^i v^j}.$$



(a)



(b)

Figure 7.2: (a) The network solution and the exact solution for the hyperbolic oscillator; (b) The associated loss functions.

The length of a smooth curve $q(s)$ from $q_0 = q(0)$ to $q = q(t)$ is then

$$\ell = \int_0^t \|\dot{q}(s)\|_g \, ds.$$

A curve of shortest length between two points q_0 and q is called a *geodesic*. It is well known that, if the points q_0 and q are sufficiently close, there exists a unique geodesic joining them. In fact, there is an open geodesic ball centered at q_0 such that every point q in this ball is connected to q_0 by a unique geodesic.

In differential geometry, geodesics are curves with zero acceleration with respect to a linear connection called the Levi–Civita connection. In this section, we take a different approach, using the Lagrangian and Hamiltonian formulations. We consider the Lagrangian on $\mathbb{R}^n \times \mathbb{R}^n$ given by the kinetic energy of a unit-mass particle moving on the aforementioned manifold:

$$L(q, \dot{q}) = \frac{1}{2}\|\dot{q}\|_g^2 = \frac{1}{2} \sum_{i,j=1}^{n} g_{ij}(q)\, \dot{q}^i \dot{q}^j.$$

It can be shown that the solutions of the Euler–Lagrange equations are, locally, geodesics. Furthermore, applying the Legendre transform, the associated Hamiltonian function is

$$H(q, p) = \frac{1}{2} \sum_{i,j=1}^{n} g^{ij}(q)\, p_i p_j,$$

where $g^{ij}(q)$ are the components of the inverse matrix of $g_{ij}(q)$.

The k-th component of the momentum is obtained from the Lagrangian as

$$p_k = \frac{\partial L}{\partial \dot{q}^k} = \sum_{i=1}^{n} g_{ik}(q)\, \dot{q}^i.$$

This relation can be inverted to express the velocities in terms of the momenta:

$$\dot{q}^i = \sum_{k=1}^{n} g^{ik}(q)\, p_k,$$

where $g^{ik}(q)$ are the components of the inverse matrix of $g_{ij}(q)$.

The Hamilton–Jacobi equation takes the form

$$\frac{\partial S}{\partial t} + \frac{1}{2} \sum_{i,j=1}^{n} g^{ij}(q)\, \frac{\partial S}{\partial q^i}\, \frac{\partial S}{\partial q^j} = 0,$$

or equivalently,

$$\frac{\partial S}{\partial t} + \frac{1}{2}\|\nabla_q S\|_g^2 = 0, \tag{7.4.7}$$

where

$$\nabla_q S = \sum_{i,k=1}^{n} g^{ik}(q)\, \frac{\partial S}{\partial q^k}\, \frac{\partial S}{\partial q^i}$$

is the gradient of S on the Riemannian manifold $(\mathbb{R}^n, g_{ij})$.

If one attempts to solve the Hamilton–Jacobi equation by separation of variables, one obtains

$$S(q,t) = -Ct + \Phi(q), \tag{7.4.8}$$

where Φ satisfies the *eiconal equation*

$$\|\nabla_q \Phi\|_g^2 = 2C > 0. \tag{7.4.9}$$

In this case, both the Hamilton–Jacobi equation and the eiconal equation admit closed-form solutions, as stated in the following result.

Proposition 7.4.1 *The action on a Riemannian manifold can be expressed as*

$$S(q_0, q, t) = \frac{d^2(q_0, q)}{2t}, \tag{7.4.10}$$

where $d(q_0, q)$ is the Riemannian distance between q_0 and q, provided the points are sufficiently close.

Proof: Let $q(s)$ be the geodesic connecting $q_0 = q(0)$ to $q = q(t)$. The proximity of q_0 and q guarantees the existence and uniqueness of this geodesic. The Riemannian distance between the two points is the length of this geodesic:

$$d(q_0, q) = \int_0^t \|\dot{q}(s)\|_g \, ds.$$

By Proposition 7.1.1, the action is given by

$$S(q_0, q, t) = \int_0^t L(q(s), \dot{q}(s)) \, ds = \frac{1}{2} \int_0^t \|\dot{q}(s)\|_g^2 \, ds.$$

Applying Cauchy's inequality, we obtain

$$d(q_0, q) = \int_0^t \|\dot{q}(s)\|_g \, ds \leq \left(\int_0^t \|\dot{q}(s)\|_g^2 \, ds \right)^{1/2} \left(\int_0^t 1 \, ds \right)^{1/2} = \sqrt{2tS}.$$

Since the Hamiltonian $H(q,p)$ does not depend explicitly on t, it is conserved along the geodesic. Equivalently, the kinetic energy is constant along the trajectory:

$$E = \frac{1}{2} \|\dot{q}(s)\|_g^2 \quad \text{is independent of } s.$$

Consequently, $\|\dot{q}(s)\|_g$ is constant, meaning the speed of the geodesic is uniform. Under this condition, the Cauchy inequality above becomes an equality, yielding

$$d(q_0, q) = \sqrt{2tS},$$

which is equivalent to the desired relation. ∎

Corollary 7.4.2 *The function $\Phi(q) = \sqrt{2C}\, d(q_0, q)$ is a solution of the eiconal equation* (7.4.9) *with initial condition $\Phi(q_0) = 0$.*

Proof: Substituting $S(q_0, q, t) = \dfrac{d^2(q_0, q)}{2t}$ into the Hamilton–Jacobi equation (7.4.7) and multiplying through by $2t^2$ yields

$$-d^2(q_0, q) + \frac{1}{4}\left\|\nabla_g\big(d^2(q_0, q)\big)\right\|_g^2 = 0.$$

This simplifies to

$$\|\nabla_q d(q_0, q)\|_g^2 = 1.$$

Multiplying both sides by $2C > 0$ gives

$$\|\nabla_q \Phi(q)\|_g^2 = 2C,$$

which shows that $\Phi(q) = \sqrt{2C}\, d(q_0, q)$ satisfies the eiconal equation with the stated initial condition. ∎

Therefore, relation (7.4.8) becomes

$$S = -Ct + \sqrt{2C}\, d(q_0, q). \tag{7.4.11}$$

The constant C can now be expressed in terms of the boundary values q_0, q, and t as follows. Substituting S from equation (7.4.10) in (7.4.11), after some algebra we obtain the perfect square

$$t\left(\sqrt{C} - \frac{d(q_0, q)}{\sqrt{2}\, t}\right)^2 = 0.$$

Hence,

$$C = \frac{d^2(q_0, q)}{2t^2}.$$

We note that this is precisely the constant energy of the system, since

$$E = \frac{1}{2}\|\dot{q}(s)\|_g^2 = \frac{1}{t}\int_0^t \frac{1}{2}\|\dot{q}(s)\|_g^2\, ds = \frac{S}{t} = \frac{d^2(q_0, q)}{2t^2} = C, \quad \forall s \in [0, t].$$

We can now determine the Riemannian distance between q_0 and q on the manifold $(\mathbb{R}^n, g_{ij})$ in two ways:

1. If the *correct* solution of the Hamilton–Jacobi equation with initial condition $S(q_0, 0) = 0$ is known, then

$$d(q_0, q) = \sqrt{2t\, S(q, t)}.$$

2. If the *correct* solution of the eiconal equation

$$\|\nabla_q \Phi\|_g^2 = 1,$$

with initial condition $\Phi(q_0) = 0$ is known, then

$$d(q_0, q) = \Phi(q).$$

Both equations can generally be solved numerically using neural networks. However, computing the Riemannian distance cannot be done in a purely unsupervised manner. As we shall see later, a small supervised component is required to ensure the correct selection of the solution to the Hamilton–Jacobi equation.

Remark 7.4.3 If one attempts to find the Riemannian distance directly using neural networks, the task becomes very complex due to the extremely large search space. To illustrate this, let

$$\mathcal{P}(q_0, q) = \{c : [0, t] \to \mathbb{R}^n \mid c \text{ is smooth}, \, c(0) = q_0, \; c(t) = q\}$$

be the space of smooth curves joining the points q_0 and q. To compute the distance between q_0 and q, we consider the length of the shortest such curve:

$$d(q_0, q) = \min_{c \in \mathcal{P}(q_0, q)} \int_0^t \|\dot{c}(s)\|_g \, ds.$$

A customary approach would be to introduce an auxiliary, free n-dimensional neural network $\mathrm{nn}(\theta; s)$ and parameterize the curve as

$$c_\theta(s) = \frac{t-s}{t} q_0 + \frac{s}{t} q + s(t-s)\,\mathrm{nn}(\theta; s).$$

The corresponding loss function could be defined as

$$\mathcal{L}_{\text{length}}(\theta) = \frac{1}{N} \sum_{j=1}^{N} \|c_\theta(s_j) - c_\theta(s_{j-1})\|_{\text{Eu}},$$

where $\{s_j\}$ are collocation points in $[0, t]$.

The main challenge with this direct method is that the search space of curves is vast, making optimization difficult. To address this issue, one would need to incorporate a "hard embedding" of geometric constraints to reduce the complexity. This is precisely why we prefer to use the Hamilton–Jacobi or the eiconal equation as an intermediate step to compute the Riemannian distance more efficiently.

7.4.1 NN Implementation

The eiconal equation associated with the Hamiltonian function

$$H(q, p) = \tfrac{1}{2} \sum_{i,j=1}^{n} g^{ij}(q)\, p_i p_j$$

is $\|\nabla_q \Phi(q)\|_g^2 = 1$. Therefore, $\Phi(q)$ satisfies the initial condition problem

$$\begin{aligned} \sum_{i,j=1}^{n} g^{ij}(q)\, \partial_{q_i} \Phi(q)\, \partial_{q_j} \Phi(q) &= 1, \\ \Phi(q_0) &= 0. \end{aligned}$$

We present two variants of solving this problem.

1. Using the boundary condition as a soft constraint. Consider a feed-forward neural network with parameters θ and input q:

$$\mathrm{NN}(\theta; q) = \Phi_\theta(q).$$

This is trained to minimize the loss

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{eic}}(\theta) + \lambda_{IC}\mathcal{L}_{\text{IC}}(\theta) + \lambda_{\text{data}}\mathcal{L}_{\text{data}}(\theta),$$

where

$$\mathcal{L}_{\text{eic}}(\theta) = \frac{1}{N_Q}\sum_{q\in Q}\left(\sum_{i,j=1}^{n} g^{ij}(q)\,\partial_{q_i}\Phi_\theta(q)\,\partial_{q_j}\Phi_\theta(q) - 1\right)^2,$$

with Q a set of collocation points, and

$$\mathcal{L}_{\text{IC}}(\theta) = \big(\Phi_\theta(q_0)\big)^2.$$

The data loss assumes that the distances $d(q_0, q_i)$ are known for a given set of m points $\{q_i\}_{i=1}^m$ and is defined by

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{m}\sum_{i=1}^{m}\big(\Phi_\theta(q_i) - d(q_0, q_i)\big)^2.$$

Because the eiconal equation is nonlinear, its solution is generally non-unique. The data term selects the physically relevant distance solution among admissible ones, and therefore cannot be neglected in practice.

2. Using the boundary condition as a hard embedding. If the neural network is chosen in the form

$$\Phi_\theta(q) = (q - q_0)\cdot \mathrm{nn}(\theta; q) = \sum_{i=1}^{n}(q^i - q_0^i)\,\mathrm{nn}^i(\theta; q),$$

where $\mathrm{nn}(\theta; q) = \big(\mathrm{nn}^1(\theta; q), \ldots, \mathrm{nn}^n(\theta; q)\big)$ is an unconstrained network, then the boundary condition is automatically satisfied. In this case, the total loss reduces to

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{eic}}(\theta) + \lambda_{\text{data}}\mathcal{L}_{\text{data}}(\theta).$$

Let

$$\theta^* = \arg\min_\theta \mathcal{L}_{\text{total}}(\theta).$$

Then the Riemannian distance between q_0 and q is approximated by

$$d(q_0, q) = \Phi_{\theta^*}(q).$$

This becomes more transparent in the next implementation example.

7.4.2 Finding the Euclidean Distance

We consider the Hamiltonian $H(q,p) = \frac{1}{2}(p_1^2 + p_2^2)$ with the corresponding eiconal equation

$$(\partial_{q_1}\Phi(q))^2 + (\partial_{q_2}\Phi(q))^2 = 1, \qquad \Phi(q_0) = 0.$$

Besides the distance function

$$\Phi(q) = \|q\| = \sqrt{q_1^2 + q_2^2},$$

this equation admits many other solutions. For example,

$$\Phi(q) = q_1 \cos\alpha + q_2 \sin\alpha,$$

where α is a real parameter.

All these solutions lead to the same dynamics, though not to the same distance function. From the solution of the eiconal equation we obtain the momenta

$$p_1 = \partial_{q_1}\Phi(q) = \cos\alpha, \qquad p_2 = \partial_{q_2}\Phi(q) = \sin\alpha,$$

while from the associated Lagrangian

$$L(q,\dot{q}) = \tfrac{1}{2}(\dot{q}_1^2 + \dot{q}_2^2)$$

we have

$$p_1 = \dot{q}_1, \qquad p_2 = \dot{q}_2.$$

Hence the equations of motion are

$$\dot{q}_1 = \cos\alpha, \qquad \dot{q}_2 = \sin\alpha,$$

which integrate to

$$q(t) = q_0 + (\cos\alpha, \sin\alpha)\, t.$$

Thus, the trajectory is a straight line through q_0 in the direction of the unit vector $(\cos\alpha, \sin\alpha)$.

However, when computing distances, the choice of solution is crucial: only one corresponds to the true distance. However, the neural network tends to select the simplest solution–much in the spirit of Occam's razor–which, in this case, does not yield the correct distance. To address this, we enforce additional constraints by specifying $m \in \{0,2,3,4\}$ reference points where the true distance is known.

The feed-forward neural network $\Phi_\theta(q)$ is trained to minimize the loss function

$$\begin{aligned}\mathcal{L}_{\text{total}}(\theta) &= \frac{1}{N_Q}\sum_{q\in Q}\big((\partial_{q_1}\Phi_\theta(q))^2 + (\partial_{q_2}\Phi_\theta(q))^2 - 1\big)^2 \\ &\quad + \lambda\big(\Phi_\theta(q_0)\big)^2 + \frac{1}{m}\sum_{i=1}^{m}\big(\Phi_\theta(q_i) - \|q_i\|_{\text{Eu}}\big)^2.\end{aligned}$$

We implement this model in Keras, and then use Python code we plot the solution of the implicit equation $\Phi_{\theta^*}(q) = 1$, which corresponds to the unit circle. Pinning Φ to 1 at a few well-spaced points on the circle breaks the degeneracy and steers the learned level set $\Phi = 1$ toward the true circle.

We have considered several cases:

• 0 points or 2 points: the solution degenerates into an almost linear function, see Figs. 7.3(a)–(b).

• 3 points: the solution bends to capture part of a circle, see Fig. 7.3(c).

• 4 points (chosen as the cardinal directions): the entire unit circle is recovered, see Fig. 7.3(d).

The complete details of the code can be found at

`https://tinyurl.com/33z2svdb`

Remark 7.4.4 To obtain a specific solution of a nonlinear equation, one generally needs to adopt a hybrid strategy that combines supervised and unsupervised learning. In contrast, for linear differential equations with prescribed boundary or initial conditions, a purely unsupervised learning approach can, at least in principle, yield the solution.

7.5 Exercises

Exercise 7.5.1 (1D Linear Oscillator). Consider the Lagrangian $L : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$ given by

$$L(q, \dot{q}) = \frac{1}{2}\dot{q}^2 - aq,$$

where $a > 0$ is a constant.

(a) Find the closed-form expression for the action $S(q_0, q, t)$.

(b) Using the action found in part (a) and the formula $\dfrac{\partial S}{\partial q} = \dot{q}$, solve for the trajectory $q(s)$ subject to the conditions $q(0) = q_0$ and $q(t) = q$.

(c) Solve part (b) again, but this time using a neural network approximation $\mathrm{nn}_\theta(s)$ for $q(s)$. Formulate a suitable loss function that enforces:

- The momentum condition $\dfrac{\partial S}{\partial q} = \dot{q}(s)$.
- The boundary conditions $q(0) = q_0$ and $q(t) = q$.

Train the network on a set of collocation points in $[0, t]$, with $t = 1$, and compare the neural network solution with the analytical one obtained in part (b).

Exercise 7.5.2 (nD Free Particle). Consider the Lagrangian

$$L(q, \dot{q}) = \frac{1}{2}\sum_{j=1}^{n} \dot{q}_j^2.$$

(a) Find the closed-form expression for the action $S(q_0, q, t)$ in two ways:

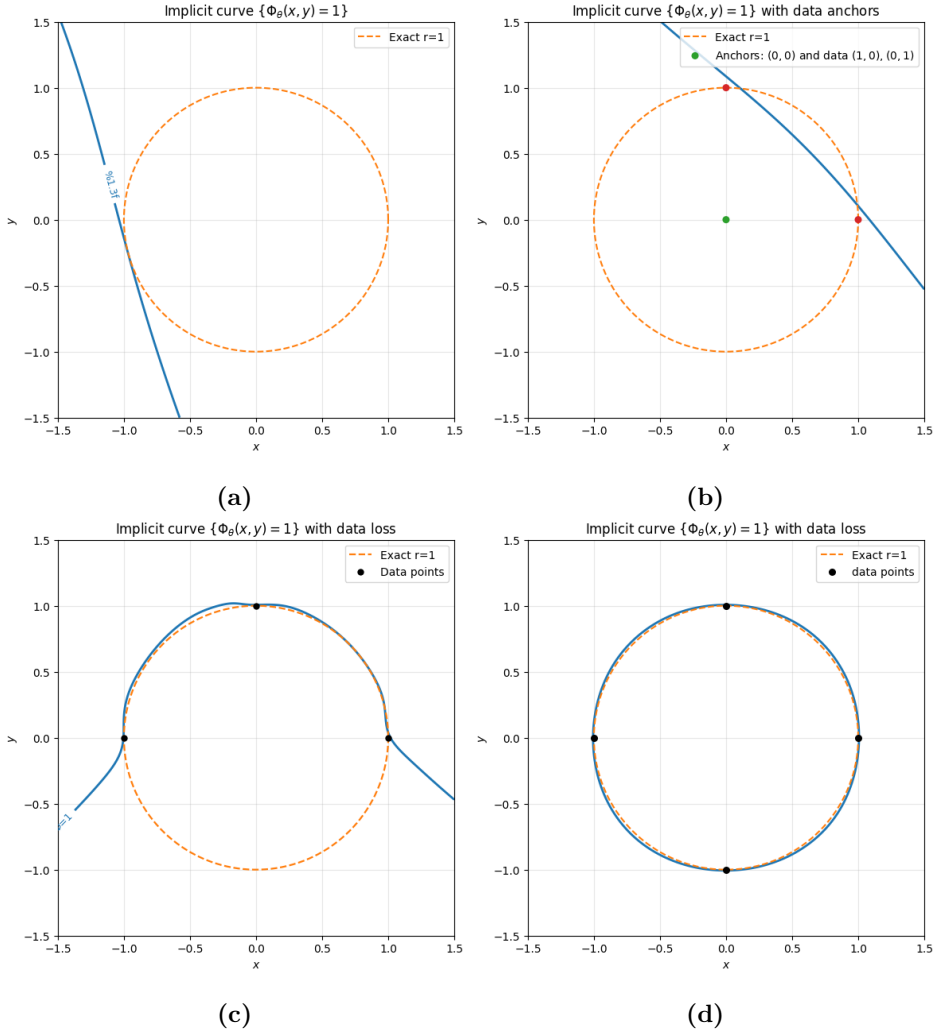


Figure 7.3: (a) The case of purely unsupervised learning, with no data points; (b) The case of two data points $\{(0,1),(1,0)\}$; (c) The case of three data points $\{(0,1),(1,0),(-1,0)\}$; (d) The case of four data points $\{(0,\pm 1),(\pm 1,0)\}$.

1. by solving the Hamilton–Jacobi equation, and
2. by first solving for the dynamics and then integrating the Lagrangian.

(b) Using the action found in part (a) and the relation

$$\frac{\partial S}{\partial q_j} = \dot{q}_j,$$

determine the trajectory $q(s)$ subject to the conditions $q(0) = q_0$ and $q(t) = q$.

Exercise 7.5.3 Let $q(t)$ be a solution of the Euler–Lagrange equation for the Lagrangian $L(q, \dot{q})$. Consider the second Lagrangian

$$\widetilde{L}(q, \dot{q}) = \frac{1}{2} L(q, \dot{q})^2.$$

(a) Show that

$$\frac{d}{dt}\frac{\partial \widetilde{L}}{\partial \dot{q}} - \frac{\partial \widetilde{L}}{\partial q} = p\,\frac{d}{dt}\big(p\,\dot{q}\big),$$

where $p = \dfrac{\partial L}{\partial \dot{q}}$.

(b) Show that if $p\,\dot{q}$ is a conserved quantity for the Lagrangian L, then $q(t)$ also satisfies the Euler–Lagrange equations for $\widetilde{L}$.

(c) Show that quadratic Lagrangians

$$L(q, \dot{q}) = \sum_{i,j} a_{ij}(q)\, \dot{q}^i \dot{q}^j,$$

with a_{ij} nondegenerate, have the property described in part (b).

(d) Repeat the computations for $\widetilde{L} = e^L$.

Exercise 7.5.4 Let $q(t)$ be a solution of the Euler–Lagrange equation for the Lagrangian $L(q, \dot{q})$, and consider the action

$$S(q_0, q, t) = \int_0^t L\big(q(s), \dot{q}(s)\big)\, ds.$$

Define the modified action

$$\widetilde{S}(q_0, q, t) = \int_0^t \frac{1}{2}\, L\big(q(s), \dot{q}(s)\big)^2\, ds.$$

(a) Show that $\widetilde{S}$ satisfies the equation

$$\partial_t \widetilde{S} = \frac{1}{2}\big(p^2 \dot{q}^2 + H^2(q, p)\big),$$

where $p = p\big(q, \partial_q \widetilde{S}\big)$ is an implicit solution of

$$p\, H(q, p) = -\partial_q \widetilde{S}.$$

(b) Consider the particular case $L(q,\dot{q}) = \frac{1}{2}\dot{q}^2$. Show that in this case $\widetilde{S}$ satisfies

$$\partial_t \widetilde{S} = \kappa \left(\partial_q \widetilde{S}\right)^{4/3},$$

with $\kappa = \frac{5\sqrt[3]{2}}{4}$.

Exercise 7.5.5 (Uniform circular motion). A unit-mass particle in uniform circular motion is described by the Lagrangian

$$L(q_1, q_2, \dot{q}_1, \dot{q}_2) = \tfrac{1}{2}(\dot{q}_1^2 + \dot{q}_2^2) + (q_1\dot{q}_2 - q_2\dot{q}_1).$$

(a) Show that the corresponding Hamiltonian is

$$H(q_1, q_2, p_1, p_2) = \tfrac{1}{2}(p_1^2 + p_2^2) + \tfrac{1}{2}(p_1 q_2 - p_2 q_1) + \tfrac{1}{8}(q_1^2 + q_2^2).$$

(b) Verify that

$$S(q_1, q_2, t) = \tfrac{1}{2}(q_1^2 + q_2^2)\cot t$$

is a solution of the Hamilton–Jacobi equation

$$\partial_t S + \tfrac{1}{2}\big((\partial_{q_1} S)^2 + (\partial_{q_2} S)^2\big) + \tfrac{1}{2}(\partial_{q_1} S\, q_2 - \partial_{q_2} S\, q_1) + \tfrac{1}{8}(q_1^2 + q_2^2) = 0,$$

with initial condition $S(0,0,t) = 0$.

(c) Using part (b) and the fact that $\dot{q} = \nabla_q S$ with initial condition $q(0) = (0,0)$, determine the trajectory of the particle starting from the origin.

(d) Solve for the trajectories using a neural network, and compare the results with the closed-form solution obtained in part (c).

Exercise 7.5.6 (Inverse Quadratic Potential). Consider a unit-mass particle moving in a plane under the influence of an inverse quadratic potential. Its dynamics are governed by the Lagrangian

$$L(q,\dot{q}) = \tfrac{1}{2}(\dot{q}_1^2 + \dot{q}_2^2) + \tfrac{1}{2}\frac{k^2}{q_1^2 + q_2^2}, \qquad k > 0 \text{ constant}.$$

(a) Derive the associated Hamiltonian function and formulate the Hamilton–Jacobi equation.

(b) Using neural networks, determine the dynamics given by $\dot{q} = \nabla_q S$, subject to the initial condition $q(0) = (1,0)$ and $k = 1$.

Exercise 7.5.7 (Kepler's Problem). Consider a unit-mass particle moving in a plane under the influence of a gravitational potential. Its dynamics are described by the Lagrangian

$$L(q,\dot{q}) = \tfrac{1}{2}(\dot{q}_1^2 + \dot{q}_2^2) + \frac{k^2}{\sqrt{q_1^2 + q_2^2}}.$$

(a) Derive the corresponding Hamiltonian function and formulate the Hamilton–Jacobi equation.

(b) Write the associated eiconal equation.

(c) Using neural networks, determine the dynamics given by $\dot{q} = \nabla_q \Phi$, subject to the initial condition $q(0) = (0,1)$.

Exercise 7.5.8 (Upper Half-Plane). Consider the hyperbolic plane

$$\mathbf{H}^2 = \{(x, y) \in \mathbb{R}^2 : y > 0\}$$

endowed with the Riemannian metric $g_{ij} = y^{-2}\delta_{ij}$.

(a) Write down the Lagrangian and Hamiltonian of a unit-mass particle moving freely in $(\mathbf{H}^2, g_{ij})$.

(b) Formulate the associated eiconal equation with the initial condition at $q_0 = (0, 1)$.

(c) Using the fact that the hyperbolic distance from $(0, 1)$ in $(\mathbf{H}^2, g_{ij})$ is given by the closed formula

$$d_{\mathbb{H}}\big((0,1),(x,y)\big) = \operatorname{arcosh}\left(1 + \frac{x^2 + (y-1)^2}{2y}\right),$$

compute the distances to a few points $(x_i, y_i) \in \mathbf{H}^2$ and use them to construct the data loss $\mathcal{L}_{\text{data}}(\theta)$.

(d) Use a neural network to approximate the hyperbolic distance from $q(0) = (0, 1)$ by minimizing a loss function that combines the eiconal loss, the data loss, and the initial condition loss. Then plot the unit circle centered at $(0, 1)$ in the hyperbolic metric.

(e) Show analytically that the unit hyperbolic circle centered at $(0, 1)$ is in fact the Euclidean circle

$$x^2 + (y - \cosh(1))^2 = \sinh^2(1).$$

Compare this result graphically with the neural network approximation from part (d).

Chapter 8

Free-boundary Value Problems

Free-boundary problems arise when the shape of the domain itself is part of the unknown, and they appear in many areas of Physics and Engineering. Classical examples include the Stefan problem, which models phase transitions such as melting and solidification; the Muskat problem, which describes the motion of interfaces between fluids in porous media; and the punctured membrane problem, where elastic deformation generates a dynamically evolving hole. These problems are notoriously challenging because the governing PDEs must be solved simultaneously with moving boundary conditions.

Recently, deep learning methods have shown promise in this context: by representing both the field variables and the free boundary through neural networks, one can enforce PDEs and interface conditions via physics-informed losses, opening new perspectives for approximating solutions in cases where analytical formulas are unavailable.

8.1 The One-phase Stefan Problem

The *one-phase Stefan problem* is a classical free-boundary problem that describes the evolution of the interface between two phases of a material, such as the boundary between ice and liquid water during melting. In this setting, the liquid phase (water) is governed by the heat equation, since heat conduction controls the temperature distribution in the melted region. The solid phase (ice) is assumed to remain at a constant temperature (typically the melting point), so its internal dynamics are not modeled explicitly.

The problem was first posed in 1889 in a paper by Josef Stefan [87], where the melting of ice in polar seas was analyzed. Theory and applications of phase change can be found in Rubinstein [82], while numerical techniques for solving moving boundary problems can be found in Crank [28]. A rigorous mathematical theory is developed in Friedman [35] and Meirmanov [71], while the relation to Thermodynamics is done in Visintin [96].

Mathematical setup Let $u(x,t)$ denote the temperature of water at position x along a pipe of length $L = 2$ at time t. Initially, the pipe is filled with water in the region $[0,1]$, while the section $[1,2]$ is frozen. Thus, the initial ice-water interface is located

at the midpoint, $x = 1$. The initial temperature distribution in the liquid region is prescribed by $u(x, 0) = f(x)$ for $0 < x < 1$, where $f(x) > 0$ and $f(1) = 0$, while in the frozen part $u(x, 0) = 0$ for $1 < x < 2$. At the left end of the pipe, heat is applied so that $u(0, t) = \phi(t)$ with $\phi(t) > 0$ and $\phi(0) = f(0)$, ensuring consistency at the initial point.

The moving boundary between water and ice is denoted by $s(t)$, with $s(0) = 1$. On the liquid side of this free boundary, the heat equation holds:

$$\partial_t u = \partial_x^2 u, \quad 0 < x < s(t),\ t > 0,$$

while in the solid region the temperature remains zero, $u(x, t) = 0$ for $s(t) < x < 2$, see Fig. 8.1.

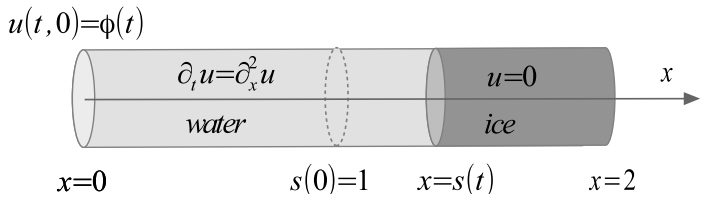


Figure 8.1: One-phase Stefan problem in a long thin tube.

At the interface, the temperature is also zero: $u(s(t), t) = 0$. Conservation of energy, combined with Fourier's law of heat conduction, implies that the velocity of the moving boundary is proportional to the heat flux at the interface. This yields the *Stefan condition*:

$$s'(t) = -\partial_x u(s(t), t), \quad t > 0.$$

Putting everything together, the problem can be formulated as finding $u(x, t)$ and $s(t)$ that satisfy

$$\begin{aligned}
\partial_t u &= \partial_x^2 u, && 0 < x < s(t),\ t > 0,\\
u(x, t) &= 0, && s(t) < x < 2,\ t > 0,\\
u(x, 0) &= f(x), && 0 \le x \le 1,\\
u(0, t) &= \phi(t), && t > 0,\\
u(s(t), t) &= 0, && t > 0,\\
s'(t) &= -\partial_x u(s(t), t), && t > 0,\\
s(0) &= 1.
\end{aligned}$$

It can be shown that whenever a solution exists, the free boundary moves to the right, i.e., $s'(t) \ge 0$. It is also physically plausible to believe that the free boundary moves to the right, because the warmer water continuously transfers heat to the colder ice at the interface, supplying the latent heat needed for melting and thereby causing the ice region to recede and the liquid region to expand.

The one-phase Stefan problem described above is known to be *well-posed*; see, for example, A. Friedman [35]. Specifically, we have:

Theorem 8.1.1 *Suppose that the initial and boundary data satisfy*

$$f \in C^1([0,1]), \quad \phi \in C^1([0,T]), \quad f(x) \geq 0,\ \phi(t) \geq 0,\ f(0) = \phi(0),\ f(1) = 0.$$

Then, for any finite time interval $0 \leq t \leq T$, there exists a unique solution to the free-boundary problem.

This result of global existence and uniqueness provides a rigorous foundation for employing neural networks to approximate both the solution $u(x,t)$ and the moving interface $s(t)$. We shall accomplish this in a few ways.

Using NN with soft constraints In this approach, the boundary and initial conditions are imposed as *soft constraints* in the loss function. However, the initial condition on the free boundary, $s(0) = 1$, is *hard embedded* into the network architecture, since we found that treating it as a soft constraint alone led to unsatisfactory results.

We therefore consider a neural network of the form

$$NN(\theta; t, x) = \big(u_\theta(x,t),\ s_\theta(t)\big),$$

where u_θ is represented by a standard feed-forward network, while s_θ is modeled using a special architecture. More specifically, the branch that learns the free boundary is defined as

$$s_\theta(t) \ = \ 1 + \int_0^t R_\theta(\tau)\, d\tau,$$

with

$$R_\theta(\tau) = \mathrm{sp}\big(r_\theta(\tau)\big),$$

which guarantees $R_\theta(\tau) \geq 0$, since $\mathrm{sp}(x) = \ln(1 + e^x)$ is the softplus function. Here, $r_\theta(\tau)$ is itself a neural network subject to training. This embedding ensures both $s_\theta(0) = 1$ and strict monotonicity, $s'_\theta(t) > 0$.

The total loss is the weighted combination of several losses as

$$\begin{aligned} \mathcal{L}_{\mathrm{tot}}(\theta) \ &= \ w_{pde}\mathcal{L}_{\mathrm{PDE}}(\theta) + w_{ic}\mathcal{L}_{\mathrm{IC}}(\theta) + w_{bc}\mathcal{L}_{\mathrm{BC}}(\theta) \\ &\quad + w_{bm}\mathcal{L}_{\mathrm{BM}}(\theta) + w_{stf}\mathcal{L}_{\mathrm{STF}}(\theta) + w_{pos}\mathcal{L}_{\mathrm{pos}}(\theta), \end{aligned}$$

with each weighted factor balancing the relative importance of each term during training. Here we have

$$\mathcal{L}_{\mathrm{PDE}}(\theta) = \int_0^T \int_0^{s_\theta(t)} \Big(\partial_t u_\theta(x,t) - \partial_{xx} u_\theta(x,t)\Big)^2 dx\, dt,$$

enforces the heat equation $\partial_t u = \partial_{xx} u$ inside the moving liquid domain,

$$\mathcal{L}_{\mathrm{IC}}(\theta) = \int_0^1 \Big(u_\theta(x,0) - f(x)\Big)^2 dx,$$

enforces the initial condition $u(x,0) = f(x)$ for $0 \le x \le 1$,

$$\mathcal{L}_{\rm BC}(\theta) = \int_0^T \Big(u_\theta(0,t) - \phi(t)\Big)^2\, dt,$$

enforces the boundary condition $u(0,t) = \phi(t)$ at the heated end of the pipe,

$$\mathcal{L}_{\rm BM}(\theta) = \int_0^T \Big(u_\theta(s_\theta(t),t)\Big)^2\, dt,$$

enforces the vanishing of temperature at the moving interface, $u(s(t),t) = 0$,

$$\mathcal{L}_{\rm STF}(\theta) = \int_0^T \Big(s'_\theta(t) + \partial_x u_\theta(s_\theta(t),t)\Big)^2\, dt,$$

enforces the Stefan condition, $s'(t) = -\partial_x u(s(t),t)$, which expresses the energy balance at the interface,

$$\mathcal{L}_{\rm pos}(\theta) = \int_0^T \int_0^{s_\theta(t)} \Big(\min\{0,\, u_\theta(x,t)\}\Big)^2\, dx\, dt,$$

penalizes negative temperature values in the liquid region, ensuring the physically plausible constraint $u(x,t) \ge 0$.

Implementation example We shall consider the case when $f(x) = 1 - x$ and $\phi(t) = 1/(1+t)$, with $(x,t) \in [0,L] \times [0,T]$, with $L = 2$ and $T = 10$. The u_θ-branch has a feed-forward architecture with 3 hidden layers, each having 64 tanh-activated neurons, while the r_θ-branch has 2 hidden layers with 32 tanh-activated neurons. Taking the weights $w_{pde} = 1.0$, $w_{ic} = 20.0$, $w_{bc} = 20.0$, $w_{bm} = 20.0$, $w_{stf} = 20.0$, and $w_{pos} = 1.0$ and training for $2,000$ epochs we obtained the free boundary given by Fig. 8.2(a).

Since the total loss is composed of several soft constraint terms, each oscillating at different frequencies, we employed a small learning rate in order to stabilize training. In particular, we used the Adam optimizer with a learning rate of 10^{-4}, which reduces oscillations in the optimization process. This choice explains the relatively mild fluctuations of the loss curve even over a large number of epochs, see Fig. 8.2(b).

The details of the code can be found at `https://tinyurl.com/2p9j4w4a`.

Using NN with hard embedding We consider a neural network of the form

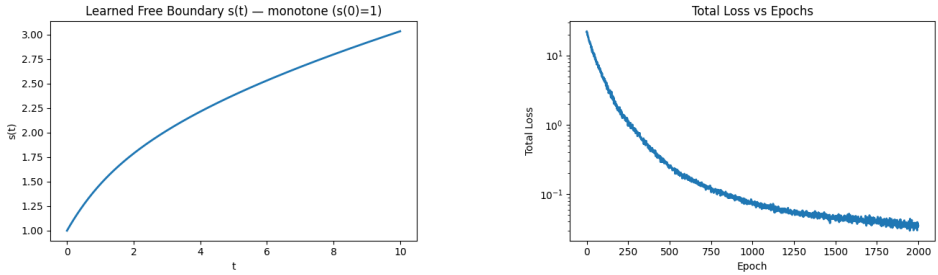
$$NN(\theta;t,x) = \big(u_\theta(x,t),\, s_\theta(t)\big),$$

where the solution component u_θ is represented with a hard embedding:

$$u_\theta(x,t) = f(x) + \phi(t) - f(0) + t\, x\, \mathrm{nn}(\theta;t,x),$$

with $\mathrm{nn}(\theta;t,x)$ a feed-forward network subject to training. With this construction, the initial and boundary conditions are automatically satisfied:

$$u_\theta(x,0) = f(x), \quad \text{since } \phi(0) = f(0), \qquad u_\theta(0,t) = \phi(t).$$



(a) **(b)**

Figure 8.2: The case of soft constraints: (a) The time evolution of the free boundary $s(t)$; (b) The associated total loss.

The initial condition on the free boundary, $s(0) = 1$, is also *hard embedded* as before:

$$s_\theta(t) \;=\; 1 + \int_0^t R_\theta(\tau)\, d\tau,$$

where

$$R_\theta(\tau) = \mathrm{sp}\big(r_\theta(\tau)\big),$$

and $\mathrm{sp}(\cdot)$ denotes the softplus activation, ensuring $R_\theta(\tau) \geq 0$ and thus $s'_\theta(t) > 0$.

With the hard embedding of the initial and boundary conditions, the total loss simplifies to

$$\mathcal{L}_{\mathrm{tot}}(\theta) \;=\; w_{\mathrm{PDE}}\,\mathcal{L}_{\mathrm{PDE}}(\theta) + w_{\mathrm{BM}}\,\mathcal{L}_{\mathrm{BM}}(\theta) + w_{\mathrm{STF}}\,\mathcal{L}_{\mathrm{STF}}(\theta) + w_{\mathrm{pos}}\,\mathcal{L}_{\mathrm{pos}}(\theta),$$

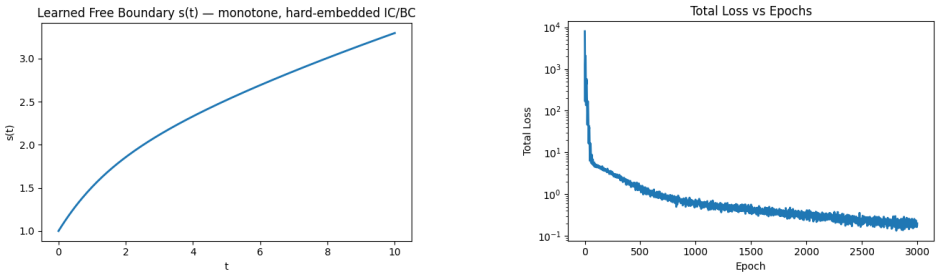
where each of these terms have been defined previously.

Implementation example We shall consider again the case when $f(x) = 1 - x$ and $\phi(t) = 1/(1+t)$, with $(x,t) \in [0,L] \times [0,T]$, with $L = 2$ and $T = 10$. We keep the same number of hidden layers and activations for u_θ and s_θ as in the case of soft constraint implementation. The only modification is the increase of the training epochs to $3,000$ and modifying the learning rate to 5×10^{-4}.

The evolution of the free boundary is shown in Fig. 8.3(a), and closely resembles the profile obtained earlier. The corresponding loss function, displayed in Fig. 8.3(b), shows only mild oscillations.

The details of the code can be found at `https://tinyurl.com/2p9j4w4a`.

Remark 8.1.2 (i) The hard-constraint formulation trains faster than the soft-constraint one. This is because hard embeddings automatically enforce initial and boundary conditions by construction, which removes the corresponding loss terms from the optimization problem. As a result, the optimizer only needs to minimize the PDE residual, the Stefan condition, and the free-boundary condition, leading to faster convergence and greater training stability.



(a) **(b)**

Figure 8.3: The case of hard encoding: (a) The time evolution of the free boundary $s(t)$; (b) The associated total loss.

(*ii*) In contrast, with soft constraints the optimizer must balance multiple competing loss terms of different magnitudes, which typically slows convergence and requires careful tuning of the loss weights.

(*iii*) Hard constraints were used to enforce the boundary and initial conditions directly in the network architecture. In contrast, more complex requirements, such as the Stefan condition and the PDE residuals, cannot be embedded in this way and are therefore imposed as soft constraints through the loss function.

8.2 The 1D Muskat Problem

The one-dimensional Muskat problem is a free-boundary problem that models the displacement of one incompressible fluid by another in a tube filled with a porous medium, under Darcy's law. Two fluids of different densities and viscosities (water and oil) occupy disjoint regions in the tube, separated by a moving interface. Because the fluids are immiscible (no mixing), the interface remains sharp, and its evolution is determined by the flow.

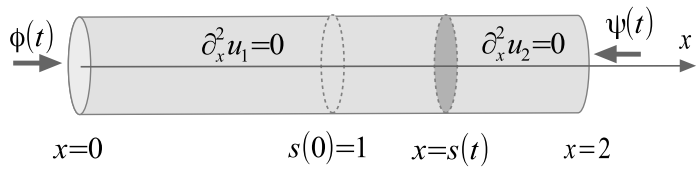


Figure 8.4: Muskat's problem in a long thin tube.

Setting up the equation We denote by $u_1(x,t)$ and $u_2(x,t)$ the pressures in the two fluids. These pressures are controlled by some boundary condition pressures applied at

the endpoints of the tube as

$$u_1(0,t) = \phi(t), \qquad u_2(0,t) = \psi(t),$$

see Fig. 8.4. By Darcy's law, the velocities of the two flows are proportional to the negative gradient of their pressures

$$v_1(x,t) = -\alpha_1 \partial_x u_1(x,t), \qquad v_2(x,t) = -\alpha_2 \partial_x u_2(x,t), \tag{8.2.1}$$

where the positive constants α_i are proportional to the inverse of the viscosity of each fluid.

Since the fluids are incompressible, the velocity fields must be divergence-free. In one spatial dimension, the divergence reduces to a single derivative, so the conditions become

$$\partial_x v_1(x,t) = 0, \qquad 0 < x < s(t),\ 0 < t \le T, \tag{8.2.2}$$

$$\partial_x v_2(x,t) = 0, \qquad s(t) < x < L,\ 0 < t \le T, \tag{8.2.3}$$

where $s(t)$ denotes the position of the free boundary between the two fluids. Substituting these relations into (8.2.1) yields

$$\partial_x^2 u_1(x,t) = 0, \qquad 0 < x < s(t),\ 0 < t \le T, \tag{8.2.4}$$

$$\partial_x^2 u_2(x,t) = 0, \qquad s(t) < x < L,\ 0 < t \le T, \tag{8.2.5}$$

which shows that the pressures are harmonic functions in their respective domains.

We also have two conditions at the interface between the fluids. One is the continuity of pressure, which writes as

$$u_1(s(t),t) = u_2(s(t),t), \qquad 0 < t \le T,$$

and the other is a kinematic condition stating that the interface moves at a rate equal to the velocity of the fluids at the interface

$$s'(t) = v_1(s(t),t) = v_2(s(t),t),$$

which can be written in terms of pressures as

$$s'(t) = -\alpha_1 u_1(s(t),t) = -\alpha_2 u_2(s(t),t).$$

Furthermore, we assume that the initial position of the interface is $s(0) = 1$. Consequently, the domain length must satisfy $L > 1$, and for simplicity we typically set $L = 2$.

To conclude, the one-dimensional Muskat problem can be formulated as finding $u_i(x,t)$ and the moving interface $s(t)$ such that

$$\begin{aligned}
\partial_x^2 u_1(x,t) &= 0, \qquad 0 < x < s(t),\ 0 < t \le T,\\
\partial_x^2 u_2(x,t) &= 0, \qquad s(t) < x < L,\ 0 < t \le T,\\
u_1(0,t) &= \phi(t), \quad u_2(L,t) = \psi(t),\\
s'(t) &= -\alpha_1\, \partial_x u_1(s(t),t) \;=\; -\alpha_2\, \partial_x u_2(s(t),t),\\
u_1(s(t),t) &= u_2(s(t),t), \qquad 0 < t \le T,\\
s(0) &= 1,
\end{aligned}$$

with α_i given positive constants.

Explicit solution Since the problem is one-dimensional, it admits an explicit solution. We then employ a neural network to approximate the solution and compare it with the analytical result, in order to assess the accuracy and effectiveness of the neural network approach.

For each fixed t, the harmonicity conditions $\partial_x^2 u_i = 0$ imply that

$$\begin{aligned} u_1(x,t) &= a_1(t)x + b_1(t), \quad 0 < x < s(t) \\ u_2(x,t) &= a_2(t)(x-L) + b_2(t), \quad s(t) < x < L, \end{aligned}$$

for some smooth functions a_i and b_i.

Using the boundary conditions at $x = 0$ and $x = L$ yields $b_1(t) = \phi(t)$ and $b_2(t) = \psi(t)$. The rate of change of the interface becomes

$$s'(t) = -\alpha_1 a_1(t) = -\alpha_2 a_2(t),$$

which implies

$$a_1(t) = -\frac{1}{\alpha_1} s'(t), \qquad a_2(t) = -\frac{1}{\alpha_2} s'(t).$$

Therefore the pressure solutions become

$$u_1(x,t) = -\frac{1}{\alpha_1} s'(t)x + \phi(t), \quad 0 < x < s(t) \tag{8.2.6}$$

$$u_2(x,t) = -\frac{1}{\alpha_2} s'(t)(x-L) + \psi(t), \quad s(t) < x < L. \tag{8.2.7}$$

Hence, it suffices to find $s'(t)$.

To this end, we substitute $a_i(t)$ in terms of $s'(t)$ in the pressure continuity condition $u_1(s(t),t) = u_2(s(t),t)$ and obtain

$$A\, s(t)s'(t) + Bs'(t) = \phi(t) - \psi(t),$$

with $A = \frac{1}{\alpha_1} - \frac{1}{\alpha_2}$ and $B = \frac{L}{\alpha_2}$. Writing it as an exact equation

$$\frac{d}{dt}\Big(\frac{1}{2} A\, s(t)^2 + Bs(t)\Big) = \phi(t) - \psi(t)$$

and integrating between 0 and t, using the initial condition $s(0) = 1$, yields the implicit solution for the boundary

$$\frac{1}{2} A\, s(t)^2 + Bs(t) + C = \int_0^t (\phi(z) - \psi(z))\, dz, \tag{8.2.8}$$

with $C = -(\frac{1}{2}A + B)$. There are two cases:

(i) If $\alpha_1 = \alpha_2 = \alpha$, then $A = 0$ and (8.2.8) becomes a linear equation with the solution

$$s(t) = 1 + \frac{1}{B} \int_0^t (\phi(z) - \psi(z))\, dz.$$

Its rate of change is given by

$$s'(t) = \frac{1}{B}(\phi(t) - \psi(t)),$$

which shows that whenever the pressure applied at the left end of the tube exceeds the pressure at the right end, i.e. $\phi(t) > \psi(t)$, the free boundary moves to the right (increases), and conversely it moves to the left when $\phi(t) < \psi(t)$. This is consistent to the physical fact that the interface advances to the region with lower pressure.

(*ii*) If $\alpha_1 \neq \alpha_2$, then $A \neq 0$. Finding $s(t)$ from the quadratic equation (8.2.8) makes sense as long as its discriminant is non-negative, namely

$$D(t) = (A+B)^2 + 2A \int_0^t (\phi(z) - \psi(z))\, dz \geq 0.$$

To fix ideas, we assume that the left fluid (oil) is more viscous than the right one (water), that is, $\frac{1}{\alpha_1} > \frac{1}{\alpha_2}$, so that $A > 0$. In this case, the positive solution for the interface is obtained via the quadratic formula,

$$s(t) = \frac{-B + \sqrt{D(t)}}{A}. \tag{8.2.9}$$

We observe that the position of the moving boundary at time t depends only on the difference of cumulative pressures up to t, namely

$$\int_0^t \phi(z)\, dz \; - \; \int_0^t \psi(z)\, dz.$$

If there is an instance $t_0 > 0$ such that this difference vanishes, the quadratic expression reduces to

$$s(t_0) = \frac{-B + (A+B)}{A} = 1 = s(0), \tag{8.2.10}$$

which means the interface passes through its initial position.

In order to determine the solutions $u_i(x,t)$, we first differentiate (8.2.10):

$$s'(t) \; = \; \frac{D'(t)}{2A\sqrt{D(t)}} \; = \; \frac{\phi(t) - \psi(t)}{\sqrt{D(t)}},$$

and then substitute this expression into relations (8.2.6)–(8.2.7) to obtain

$$u_1(x,t) \;=\; -\frac{1}{\alpha_1}\frac{\phi(t) - \psi(t)}{\sqrt{D(t)}}x + \phi(t), \quad 0 < x < s(t) \tag{8.2.11}$$

$$u_2(x,t) \;=\; -\frac{1}{\alpha_2}\frac{\phi(t) - \psi(t)}{\sqrt{D(t)}}(x - L) + \psi(t), \quad s(t) < x < L. \tag{8.2.12}$$

Neural network solution We consider a neural network

$$NN(\theta; x, t) = \big(u_1(\theta; x, t),\, u_2(\theta; x, t),\, s_\theta(t)\big),$$

with two inputs (x, t) and three outputs, constructed with the following embedding architecture:

$$\begin{aligned} u_1(\theta; x, t) &= \phi(t) + x\,\mathrm{nn}^1(\theta; x, t), \\ u_2(\theta; x, t) &= \psi(t) + (x - L)\,\mathrm{nn}^2(\theta; x, t), \\ s_\theta(t) &= 1 + \int_0^t r(\theta; \tau)\, d\tau, \end{aligned}$$

where $\mathrm{nn}^j(\theta; x, t)$ and $r(\theta; \tau)$ are trainable neural networks.

The training process minimizes the total loss

$$\mathcal{L}_{\mathrm{tot}}(\theta) = w_{\mathrm{pde}}\mathcal{L}_{\mathrm{PDE}}(\theta) + w_{\mathrm{bm}}\mathcal{L}_{\mathrm{BM}}(\theta) + w_{\mathrm{m}}\mathcal{L}_M(\theta),$$

where

$$\mathcal{L}_{\mathrm{PDE}}(\theta) = \int_0^T \int_0^{s_\theta(t)} \big(\partial_x^2 u_1(\theta; x, t)\big)^2\, dx\, dt + \int_0^T \int_{s_\theta(t)}^1 \big(\partial_x^2 u_2(\theta; x, t)\big)^2\, dx\, dt,$$

enforces the PDE constraints in both fluid regions,

$$\mathcal{L}_{\mathrm{BM}}(\theta) = \int_0^T \Big(u_1(\theta; s_\theta(t), t) - u_2(\theta; s_\theta(t), t)\Big)^2\, dt,$$

enforces continuity of pressure across the moving boundary, and

$$\begin{aligned} \mathcal{L}_M(\theta) = &\int_0^T \int_0^{s_\theta(t)} \Big(s'_\theta(t) + \alpha_1\, \partial_x u_1(\theta; x, t)\Big)^2\, dx\, dt \\ &+ \int_0^T \int_{s_\theta(t)}^1 \Big(s'_\theta(t) + \alpha_2\, \partial_x u_2(\theta; x, t)\Big)^2\, dx\, dt, \end{aligned}$$

enforces the Muskat equilibrium boundary condition.

We note that in the PDE loss implementation we use the formula

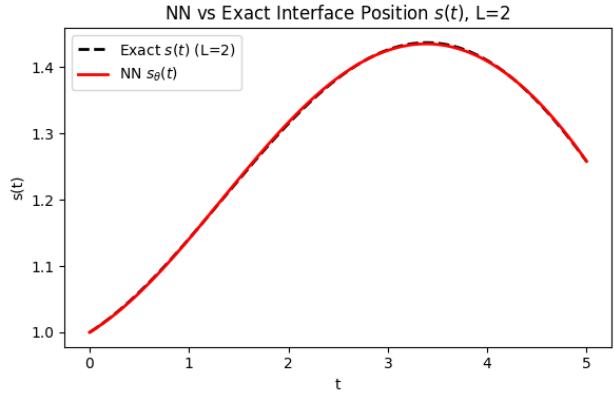
$$\int_0^{s_\theta(t)} \big(\partial_x^2 u_1(\theta; x, t)\big)^2\, dx = \int_0^L H(s(t) - x)\big(\partial_x^2 u_1(\theta; x, t)\big)^2\, dx,$$

where $H(y)$ is the Heaviside function[1] which is approximated smoothly by

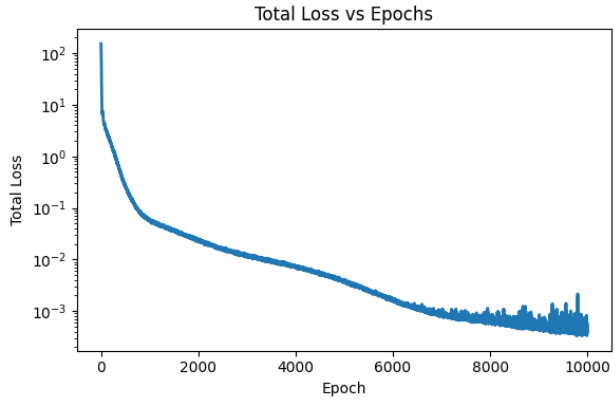
$$H(y) \approx \frac{1}{2}(1 + \tanh(ky)), \qquad k \text{ large}.$$

All integrals are approximated using a set of collocation points. The learned free boundary is then given by $s_{\theta^*}(t)$, where θ^* denotes the trained network parameters.

[1] This is a discontinuous function given by $H(y) = \begin{cases} 1, & \text{if } y \geq 0 \\ 0, & \text{otherwise.} \end{cases}$



(a)



(b)

Figure 8.5: (a) The free boundaries $s_{\theta^*}(t)$ and $s_{\text{exact}}(t)$ are indistinguishable; (b) The associated total loss.

Implementation example We consider the following parameter values: $L = 2$, $\alpha_1 = 1$, $\alpha_2 = 2$, and $T = 5$. The coefficient functions are chosen as

$$\phi(t) = 1 + 0.25\sin(0.8t), \qquad \psi(t) = 1 - 0.15\cos(0.6t + 0.3).$$

The neural network architectures are defined as follows: $\text{nn}^1(\theta; x, t)$ and $\text{nn}^2(\theta; x, t)$ each contain three hidden layers with 64 neurons per layer, using tanh activation. In contrast, $r_\theta(t)$ is modeled with two hidden layers of 32 tanh-activated neurons.

For the integral term $\int_0^t r_\theta(y)\,dy$, we use 100 discretization points and approximate it with the midpoint Riemann sum. The loss function is weighted so that the moving boundary and Muskat losses have weight 20, while the PDE loss has weight 1. Training is performed with the Adam optimizer using a small learning rate of 10^{-4} over $10,000$ epochs. Due to the low learning rate, additional training could be beneficial. Loss oscillations begin to appear around epoch $7,000$, as shown in Fig. 8.5(b).

For validation, we compute the exact free-boundary solution $s_{\text{exact}}(t)$ using the closed-form expression (8.2.9). Figure 8.5(a) presents a comparison of the neural network solution and the exact solution, showing the two curves are visually indistinguishable. This demonstrates both the accuracy and efficiency of the neural network approximation.

Finally, we note that the free boundary $s(t)$ increases on the interval $(0, 3.3)$, after which it begins to decrease. This behavior indicates that during the first 3.3 seconds, the interface moves to the right, and subsequently reverses direction, moving to the left.

The details of the code can be found at `https://tinyurl.com/55jjjyas`.

Historical Perspective The Muskat problem has its origins in the seminal 1935 paper by Morris Muskat [74], which laid the foundation for modeling fluid displacement in porous media using potential theory. This classical formulation has since evolved through rigorous mathematical development.

A modern, comprehensive analysis of the Muskat flow, employing abstract parabolic evolution equations to study well-posedness and geometric properties such as volume conservation, is presented by Prüss and Simonett [77]. Global classical solutions using fixed-point arguments are established in the work of Yi [103], providing foundational results for the existence of smooth interfaces over time.

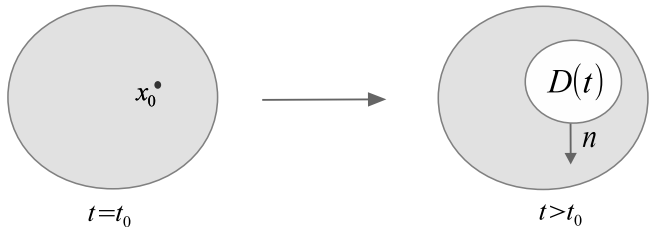
An early extension of the problem beyond its classical setting is offered by Fulks and Guenther [36], who incorporate nonlinearities and complex boundary interactions into the model. A deeper connection between microscopic structure and macroscopic behavior, specifically for viscoelastic filtration, is developed in Meirmanov's analysis [70], extending the classical Darcy flow framework.

In the two-dimensional case, Matioc [69] provides a detailed study of the Muskat problem, demonstrating local well-posedness for large initial data and examining the parabolic nature of the governing equations. Finally, important advances in global solvability and regularity, particularly for both 2D and 3D formulations, are presented by Constantin et al. [26], marking a significant contribution to the understanding of long-time behavior in the Muskat problem.

8.3 The Punctured Membrane Problem

We consider an elastic membrane with fixed boundary, occupying a bounded planar domain $\Omega \subset \mathbb{R}^2$. At time t_0, the membrane is punctured at a point $x_0 \in \Omega$. Under the action of elastic forces, this puncture evolves for $t > t_0$ into a finite hole, denoted by $D(t)$. Physically, one may view this as an elastic drum with its rim clamped along $\partial\Omega$, where a single puncture develops into an opening inside the drumhead, see Fig. 8.6.

The mathematical objective is to describe this situation as a *free-boundary problem*, in which both the membrane deformation and the evolving hole boundary $\partial D(t)$ are unknown. A natural formulation is variational: the deformation minimizes the elastic energy of the membrane subject to fixed outer boundary conditions and traction-free conditions along the hole boundary.



s

Figure 8.6: A single puncture x_0 at time t_0 develops into an opening $D(t)$ at time t inside the drumhead.

Closed-form solutions exist only in very special symmetric cases (for example, a circular drum punctured at its center, leading to radial cavitation solutions). In general, however, the problem must be tackled numerically. In this section we explore a deep learning approach to approximate both the deformation field and the evolving free boundary $D(t)$.

Notations We begin by setting the notation. Let $x_0 \in \Omega$ be the puncture point in the membrane domain Ω, and denote by $D(t)$ the hole that develops from x_0 at time $t > t_0$.

The stretch of the initial elastic membrane is described by a deformation mapping $\varphi : \Omega \to \mathbb{R}^2$, which is clamped on the boundary by a prescribed Dirichlet condition $\varphi|_{\partial\Omega} = \phi$, where ϕ is a given continuous function on $\partial\Omega$. If φ is chosen to be the identity mapping (the inclusion), then the membrane is initially unstressed and no hole develops from the puncture point x_0. Thus, the initial deformation mapping φ directly influences the eventual formation and shape of the hole $D(t)$.

For $t > t_0$, the current elastic domain is $\Omega_t = \Omega \setminus D(t)$, and the membrane deformation is described by a time-dependent mapping $\varphi_t : \Omega_t \to \mathbb{R}^2$, which satisfies the same boundary condition $\varphi_t|_{\partial\Omega} = \phi$. The problem is therefore to determine both the evolving hole $D(t)$ and the deformation mapping φ_t for all times $t > t_0$.

At equilibrium, as $t \to \infty$, the deformation map converges to $\varphi_\infty : \Omega \setminus D_\infty \to \mathbb{R}^2$, where D_∞ denotes the equilibrium position of the hole. Thus, in addition to studying the time evolution of the deformation and the hole, we must also determine their equilibrium configurations.

The stress tensor To model how the membrane Ω_t is stretched, we introduce a stress tensor induced by the deformation. This is the Riemannian metric tensor obtained from the mapping φ_t, which is assumed to be an isometric immersion. In other words, the metric on Ω_t is the pullback of the Euclidean metric from $\mathbb{R}^2$ via φ_t:

$$g^t_{ij}(x) = \varphi_t^*(\delta_{ij}) = \left\langle \partial_{x_i}\varphi_t(x),\, \partial_{x_j}\varphi_t(x) \right\rangle_{\mathbb{R}^2} = \partial_{x_i}\varphi_t^1\, \partial_{x_j}\varphi_t^1 + \partial_{x_i}\varphi_t^2\, \partial_{x_j}\varphi_t^2.$$

In matrix form, this can be expressed as

$$g^t(x) = F_t(x)^\top F_t(x),$$

where

$$F_t(x) = \nabla\varphi_t(x) = \left(\partial_{x_i}\varphi_t^j(x)\right)_{i,j} \in \mathbb{R}^{2\times 2}$$

is the deformation gradient matrix, and $F_t^\top$ denotes its transpose. The tensor $g^t(x)$ is the *right Cauchy–Green deformation tensor*, see [68] and [47].

The stretch in the direction of a vector $v = (v_1, v_2)$ at a point $x \in \Omega_t$ is then computed by the quadratic form

$$g^t(v, v) = \sum_{i,j} g_{ij}^t(x)\, v_i v_j.$$

By construction, $g^t(x)$ is symmetric, positive definite, and nondegenerate at each point $x \in \Omega_t$. This means that the membrane exhibits strictly positive stretch in every direction, and it is precisely this accumulation of elastic stretch that drives the transition of a puncture point into a finite convex hole.

Geometrically, the family $\{\Omega_t\}_{t>0}$ forms a nested sequence of shrinking subdomains of Ω, namely

$$\partial\Omega \subseteq \Omega_{t'} \subseteq \Omega_t \subseteq \Omega, \quad 0 < t < t'.$$

We remark that the Riemannian manifold (Ω_t, g^t) is isometrically immersed in the Euclidean plane $(\mathbb{R}^2, \delta_{ij})$, and therefore it may be regarded as a submanifold of $\mathbb{R}^2$ at any time t.

Matrix invariants At each point $x \in \Omega_t$ we associate two invariants of the metric tensor $g^t(x)$, namely its trace and determinant. These quantities have direct physical interpretations and will serve as the basis for constructing an energy functional.

Let $\lambda_1(x)$ and $\lambda_2(x)$ denote the eigenvalues of $g^t(x)$, which are the solutions of the characteristic equation

$$\lambda^2 - \operatorname{Tr}(g^t(x))\,\lambda + \det g^t(x) = 0.$$

We have $0 < \lambda_1(x) \le \lambda_2(x)$. Geometrically, $\lambda_1(x)$ represents the minimal stretch in a unit vector direction at x,

$$\lambda_1(x) = \min_{\|v\|=1} g^t(x)(v, v),$$

while $\lambda_2(x)$ represents the maximal stretch,

$$\lambda_2(x) = \max_{\|v\|=1} g^t(x)(v, v).$$

If the unit vector is parametrized as $v = (\cos\theta, \sin\theta)$, then the average stretch at x is the angular average

$$\frac{1}{2\pi}\int_0^{2\pi} g^t(x)(v, v)\, d\theta = \frac{1}{2\pi}\int_0^{2\pi} \left(g_{11}^t(x)\cos^2\theta + g_{22}^t(x)\sin^2\theta + 2g_{12}^t(x)\cos\theta\sin\theta\right) d\theta,$$

which simplifies to

$$\frac{1}{2}\left(g_{11}^t(x) + g_{22}^t(x)\right) = \frac{1}{2}\operatorname{Tr}(g^t(x)).$$

The membrane is called *incompressible* if the area element is preserved under the deformation. In the Euclidean plane, the standard volume form is $dv = dx_1 \wedge dx_2$. The manifold (Ω_t, g^t) carries the Riemannian volume form

$$dv^t(x) = \sqrt{\det g^t(x)}\, dx_1 \wedge dx_2.$$

One verifies that this is precisely the pullback of dv by φ_t:

$$\begin{aligned}
\varphi_t^*(dv) &= \varphi_t^*(dx_1 \wedge dx_2) = \varphi_t^*(dx_1) \wedge \varphi_t^*(dx_2) \\
&= d\varphi_t^1 \wedge d\varphi_t^2 \\
&= \big(\partial_{x_1}\varphi_t^1 dx_1 + \partial_{x_2}\varphi_t^1 dx_2\big) \wedge \big(\partial_{x_1}\varphi_t^2 dx_1 + \partial_{x_2}\varphi_t^2 dx_2\big) \\
&= \big(\partial_{x_1}\varphi_t^1\, \partial_{x_2}\varphi_t^2 - \partial_{x_2}\varphi_t^1\, \partial_{x_1}\varphi_t^2\big)\, dx_1 \wedge dx_2 \\
&= \det(\nabla\varphi_t)\, dx_1 \wedge dx_2 = \sqrt{\det(\nabla\varphi_t^\top \nabla\varphi_t)}\, dx_1 \wedge dx_2 \\
&= \sqrt{\det g^t(x)}\, dx_1 \wedge dx_2 = dv^t(x).
\end{aligned}$$

Hence, the membrane is incompressible if φ_t is volume-preserving, i.e.

$$dv^t(x) = dv(x),$$

which is equivalent to $\det g^t(x) = 1$.

Energy density The incompressibility condition $\det g^t(x) = 1$ implies $\lambda_1(x)\lambda_2(x) = 1$. Consequently,

$$\operatorname{Tr} g^t(x) = \lambda_1(x) + \lambda_2(x) = \lambda_1(x) + \frac{1}{\lambda_1(x)} \geq 2.$$

We may therefore define the nonnegative *energy density*

$$w_t(x) = \tfrac{1}{2}\operatorname{Tr}(g^t(x)) - 1 \geq 0,$$

which vanishes precisely when $\lambda_1(x) = \lambda_2(x)$, i.e. when the stretch is isotropic at point x.

Furthermore, the energy density $w_t(x)$ can be expressed in terms of the deformation mapping gradient $\nabla\varphi_t$ as

$$\begin{aligned}
w_t(x) &= \tfrac{1}{2}\operatorname{Tr}\big(g^t(x)\big) - 1 = \tfrac{1}{2}\operatorname{Tr}\big(\nabla\varphi_t(x)^\top \nabla\varphi_t(x)\big) - 1 \\
&= \tfrac{1}{2}\sum_{i,j}\big(\partial_{x_i}\varphi_t^j(x)\big)^2 - 1 \\
&= \tfrac{1}{2}\,\|\nabla\varphi_t(x)\|_F^2 - 1,
\end{aligned}$$

where $\|\cdot\|_F$ denotes the Frobenius norm of a matrix.

At any time $t > t_0$ the membrane *total stretch energy* is defined by

$$E(t) = \int_{\Omega_t} w_t(x)\, dx,$$

provided the deformation is clamped at the boundary, $\varphi_t(x) = \phi(x)$, $x \in \partial\Omega$. This energy depends both on the integrand w_t and on the evolving integration domain Ω_t.

Mathematical formulation Using the calculus of variations, we arrive at two coupled free-boundary value problems: one governing the *evolution* $(\varphi_t, D(t))$, and the other describing the *equilibrium configuration* $(\varphi_\infty, D_\infty)$.

1. *Evolution problem.* The unknowns are φ_t and $D(t)$, which satisfy the system

$$\begin{cases} \partial_t \varphi_t(x) - \Delta \varphi_t(x) = 0, & x \in \Omega_t, \\ \varphi_t(x) = \phi(x), & x \in \partial\Omega, \\ \partial_n \varphi_t(x) = 0, & x \in \partial D(t). \end{cases}$$

Here n denotes the unit normal to the moving boundary $\partial D(t)$, and $\partial_n \varphi_t(x) = (\nabla \varphi_t(x))^T n$ is the normal derivative of φ_t along $\partial D(t)$. The first equation indicates that the deformation evolves by the heat equation in the punctured domain $\Omega_t = \Omega \setminus D(t)$. The second imposes clamped boundary data on $\partial\Omega$, while the third expresses the traction-free condition along the boundary of the puncture.

2. *Equilibrium problem.* The unknowns are φ_∞ and D_∞, determined by

$$\begin{cases} \Delta \varphi_\infty(x) = 0, & x \in \Omega \setminus D_\infty, \\ \varphi_\infty(x) = \phi(x), & x \in \partial\Omega, \\ \partial_n \varphi_\infty(x) = 0, & x \in \partial D_\infty, \\ \frac{1}{2}\|\nabla \varphi_\infty(x)\|^2 = 1, & x \in \partial D_\infty. \end{cases}$$

At equilibrium, the heat equation has relaxed to yield a harmonic function on $\Omega \setminus D_\infty$. The final condition ensures that along the free boundary ∂D_∞ the energy density reaches its critical value, i.e. $w_\infty(x) = 0$ for $x \in \partial D_\infty$.

Neural network solution In general, the two free-boundary problems described above cannot be solved in closed form. We now illustrate how the *equilibrium problem* can be approximated using neural networks. The *evolution problem* can be treated in a similar fashion, with the additional parameter of time t.

Since both the deformation φ_∞ and the free boundary D_∞ must be determined, we employ two neural networks, one for each unknown object.

The deformation map is approximated by a two-dimensional network

$$\varphi_\theta : \Omega \setminus D_\infty \longrightarrow \mathbb{R}^2,$$

parametrized by weights θ. To enforce the Dirichlet boundary condition on $\partial\Omega$, we embed it directly into the network architecture via

$$\varphi_\theta(x) \;=\; \phi(x) \;+\; b(x)\,\mathrm{nn}(\theta, x),$$

where $\mathrm{nn}(\theta, x)$ is a standard feedforward network and $b(x)$ is a smooth function which vanishes on $\partial\Omega$. The choice of $b(x)$ is not unique, but its role is to guarantee that the boundary condition $\varphi_\theta(x) = \phi(x)$ is satisfied exactly.

On the other side, the hole D_∞ can be described using a level-set representation formula. To this end, we introduce a second neural network

$$F_\eta : \Omega \longrightarrow \mathbb{R},$$

such that

$$D_\infty \approx \{x \in \Omega : F_\eta(x) < 0\}, \qquad \partial D_\infty \approx \{x \in \Omega : F_\eta(x) = 0\},$$

where the parameters η control the shape of the hole.

Thus, the combined neural network

$$NN(\theta, \eta; x) = \big(\varphi_\theta(x),\, F_\eta(x)\big)$$

is trained to minimize a total loss functional that uses soft constraints to penalize violations of the PDE and free-boundary conditions:

$$\mathcal{L}(\theta, \eta) = \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}} + \lambda_{\text{free}}\mathcal{L}_{\text{free}}.$$

The components of the loss are given as follows:

$$\mathcal{L}_{\text{PDE}} = \int_{\Omega \setminus D_\infty} \big\|\Delta\varphi_\theta(x)\big\|^2 \, dx,$$

$$\mathcal{L}_{\text{free}} = \mathcal{L}_{\text{Neumann}} + \mathcal{L}_{\text{energy}},$$

where

$$\mathcal{L}_{\text{Neumann}} = \int_{\partial D_\infty} \big\|\partial_n\varphi_\theta(x)\big\|^2 \, ds,$$

$$\mathcal{L}_{\text{energy}} = \int_{\partial D_\infty} \big(\tfrac{1}{2}\|\nabla\varphi_\theta(x)\|^2 - 1\big)^2 \, ds.$$

The training task is therefore formulated as the minimization problem

$$(\theta^*, \eta^*) = \arg\min_{\theta,\eta} \mathcal{L}(\theta, \eta),$$

which yields both the deformation map φ_{θ^*} and the equilibrium free boundary D_∞ defined implicitly by F_{η^*}.

The integrals appearing in the loss are approximated by discrete sums over collocation points. In this setting, however, the integration domain depends on the unknown free boundary, so a fixed uniform grid cannot be precomputed in advance. Instead, the procedure is carried out adaptively as follows.

Collocation on the punctured domain To choose collocation points in $\Omega \setminus D_\infty$ we sample N random candidate points x_j in Ω. Then we retain only those with $F_\eta(x_j) > 0$ and use them to compute the integral as

$$\int_{\Omega \setminus D_\infty} \big\|\Delta\varphi_\theta(x)\big\|^2 \, dx \approx \frac{|\Omega|}{N} \sum_{j=1}^{N} \big\|\Delta\varphi_\theta(x_j)\big\|^2 \mathbf{1}_{F_\eta(x_j)>0}$$

where $\mathbf{1}$ is the indicator function.

Collocation on the free boundary Choosing collocation points on D_∞ is more delicate, since a candidate sample point $x \in \Omega$ will not in general satisfy $F_\eta(x) = 0$ exactly. For clarity, we drop the subscript η in the following discussion, as the parameters remain fixed during this procedure.

The idea is to begin with candidate points x for which $F(x)$ is close to zero, say $|F(x)| < \varepsilon$, and then project them onto the zero level set. To this end, we define the auxiliary loss function

$$\mathcal{L}(y) = \tfrac{1}{2} F(y)^2,$$

and initialize $y_0 = x$. We then construct a sequence (y_n) by gradient descent:

$$y_{n+1} = y_n - \lambda_n \nabla \mathcal{L}(y_n).$$

Since $\nabla \mathcal{L}(y) = F(y) \nabla F(y)$, we can approximate

$$\begin{aligned} F(y_{n+1}) &= F(y_n - \lambda_n \nabla \mathcal{L}(y_n)) + \text{error} \\ &= F(y_n) - \lambda_n \nabla \mathcal{L}(y_n) \cdot \nabla F(y_n) + \text{error} \\ &= F(y_n) - \lambda_n F(y_n) \, \|\nabla F(y_n)\|^2 + \text{error} \\ &= F(y_n)\big(1 - \lambda_n \|\nabla F(y_n)\|^2\big) + \text{error}. \end{aligned}$$

Choosing

$$\lambda_n = \frac{1}{\|\nabla F(y_n)\|^2}$$

cancels the bracket, leading to the update rule

$$y_{n+1} = y_n - \frac{F(y_n)}{\|\nabla F(y_n)\|^2} \nabla F(y_n).$$

Thus, after a sufficient number of iterations, the sequence (y_n) converges to a point y satisfying $F(y) = 0$, i.e. a collocation point lying on the free boundary ∂D_∞. If the initial candidate point x lies sufficiently close to the free boundary, then only a few projection iterations are required. Hence, as a preprocessing step, we first collect candidate points satisfying the proximity condition

$$|F_\eta(x)| < \varepsilon,$$

which ensures that the selected samples are already near ∂D_∞.

Energy variation The evolution and equilibrium problems formulated earlier are both derived from computations of the energy variation. Differentiating $E(t)$ with respect to time gives

$$\begin{aligned} \frac{d}{dt} E(t) &= \frac{d}{dt} \int_{\Omega_t} w_t(x) \, dx \\ &= \int_{\Omega_t} \partial_t w_t(x) \, dx + \int_{\partial D(t)} w_t(y) \, v_n(y) \, d\sigma_y, \end{aligned}$$

where $v_n(y)$ denotes the normal velocity of the free boundary $\partial D(t)$ at point y.

The first integral can be simplified by parts using Green's formula. Since

$$w_t(x) = \tfrac{1}{2}\|\nabla\varphi_t(x)\|^2 - 1,$$

we obtain

$$\begin{aligned}\int_{\Omega_t} \partial_t w_t(x)\,dx &= \int_{\Omega_t} \nabla\varphi_t \cdot \nabla(\partial_t\varphi_t)\,dx \\ &= -\int_{\Omega_t} \Delta\varphi_t \cdot \partial_t\varphi_t\,dx + \int_{\partial\Omega_t} \partial_t\varphi_t(y)\,\partial_n\varphi_t(y)\,d\sigma_y,\end{aligned}$$

where Δ denotes the 2-dimensional Euclidean Laplacian.

The boundary integral vanishes because

$$\int_{\partial\Omega_t} = \int_{\partial\Omega} - \int_{\partial D(t)},$$

and by the boundary conditions we have $\partial_t\varphi_t|_{\partial\Omega} = 0$ (clamped outer boundary), $\partial_n\varphi_t|_{\partial D(t)} = 0$ (traction-free hole boundary). Thus,

$$\int_{\Omega_t} \partial_t w_t(x)\,dx = -\int_{\Omega_t} \Delta\varphi_t \cdot \partial_t\varphi_t\,dx.$$

Energy-dissipative dynamics. To ensure that the total energy decreases monotonically in time, we prescribe evolution laws for both the deformation φ_t and the free boundary $D(t)$ that guarantee dissipation. A natural choice is given by the gradient flow of the energy:

- Inside domain Ω_t, the deformation evolves according to the heat equation

$$\partial_t\varphi_t = \Delta\varphi_t \quad \text{in } \Omega_t.$$

- On the free boundary $\partial D(t)$, the normal velocity is chosen proportional to the negative of the energy density:

$$v_n = -w_t \quad \text{on } \partial D(t),$$

  where $w_t(x) = \frac{1}{2}\|\nabla\varphi_t(x)\|^2 - 1$.

With these dynamics, the time derivative of the energy satisfies

$$\frac{d}{dt}E(t) = -\int_{\Omega_t} \|\Delta\varphi_t(x)\|^2\,dx - \int_{\partial D(t)} w_t(y)^2\,d\sigma_y \quad \leq 0,$$

showing that $E(t)$ is a non-increasing function of time.

In the limit $t \to \infty$, the energy reaches a minimal threshold and $\frac{d}{dt}E(t) \to 0$. This implies that both integrals vanish, which in turn forces the corresponding integrands to vanish:

$$\Delta\varphi_\infty = 0 \quad \text{in } \Omega \setminus D_\infty, \qquad w_\infty(y) = 0 \quad \text{on } \partial D_\infty.$$

Equivalently, the limiting equilibrium configuration $(\varphi_\infty, D_\infty)$ satisfies

$$\Delta\varphi_\infty = 0 \quad \text{in } \Omega \setminus D_\infty, \qquad \tfrac{1}{2}\|\nabla\varphi_\infty(y)\|^2 = 1 \quad \text{on } \partial D_\infty.$$

8.3.1 A Radial Symmetric Case

We now consider a particular case in which the punctured membrane problem admits a closed-form solution, and we shall compare this explicit solution with the approximation obtained using a neural network.

Specifically, we take the case of the unit disk

$$\Omega = \{x \in \mathbb{R}^2 : |x| < 1\},$$

punctured at the center $x_0 = (0, 0)$. At equilibrium, the hole is a disk of radius ρ, given by

$$D = \{x \in \mathbb{R}^2 : |x| < \rho\}.$$

Thus, the punctured domain is the annulus

$$\Omega \setminus D = \{x \in \mathbb{R}^2 : \rho < |x| < 1\}.$$

In general, any deformation $\varphi : \Omega \setminus D \to \mathbb{R}^2$ can be written in polar coordinates $x = (r\cos\alpha, r\sin\alpha)$ as

$$\varphi(x) = u(r, \alpha)e_r + v(r, \alpha)e_\alpha,$$

where

$$e_r = \frac{x}{|x|} = (\cos\alpha, \sin\alpha), \qquad e_\alpha = (-\sin\alpha, \cos\alpha),$$

form an orthonormal basis at each point.

For simplicity, we assume the deformation is radially symmetric. This assumption reduces the free boundary to a single parameter. In this case,

$$\varphi(x) = u(r)e_r = \big(u(r)\cos\alpha,\ u(r)\sin\alpha\big).$$

The gradient in polar coordinates is expressed as

$$\nabla\varphi = \frac{\partial\varphi}{\partial r}\, e_r + \frac{1}{r}\,\frac{\partial\varphi}{\partial\alpha}\, e_\alpha.$$

A direct computation yields

$$\|\nabla\varphi\|^2 = \|\nabla\varphi^1\|^2 + \|\nabla\varphi^2\|^2 = u'(r)^2 + \frac{1}{r^2}u(r)^2.$$

The vector Laplacian of the radially symmetric deformation φ is

$$\Delta\varphi = (\Delta\varphi^1, \Delta\varphi^2) = \left(u''(r) + \frac{1}{r}u'(r) - \frac{1}{r^2}u(r)\right) e_r.$$

Therefore, the equilibrium free-boundary problem with outer Dirichlet boundary condition $\varphi(x) = 1$ on $\partial\Omega$ reduces to the following system:

$$\begin{cases} u''(r) + \dfrac{1}{r}u'(r) - \dfrac{1}{r^2}u(r) = 0, \qquad \rho < r < 1, \\ u(1) = 1, \\ u'(\rho) = 0, \\ \frac{1}{2}\left((u'(\rho))^2 + \dfrac{u(\rho)^2}{\rho^2}\right) = 1. \end{cases}$$

Since the second-order ODE for $u(r)$ is an Euler equation, we look for solutions of the form $u(r) = r^m$ and obtain the characteristic exponents $m \in \{-1, 1\}$. Thus the general solution is

$$u(r) = Ar + \frac{B}{r}, \qquad A, B \in \mathbb{R}.$$

Imposing the boundary conditions yields

$$A = \frac{1}{1+\rho^2}, \qquad B = \frac{\rho^2}{1+\rho^2}.$$

Using the traction-free condition $u'(\rho) = 0$, the energy constraint reduces to

$$\tfrac{1}{2}\,\frac{u(\rho)^2}{\rho^2} = 1.$$

Since

$$u(\rho) = \frac{2\rho}{1+\rho^2},$$

this condition becomes an equation for ρ:

$$\frac{2}{(1+\rho^2)^2} = 1,$$

which admits the positive solution

$$\rho = \sqrt{\sqrt{2}-1} \;\approx\; 0.6436.$$

Substituting this into the equilibrium deformation formula gives

$$\varphi(x) = \frac{|x|}{\sqrt{2}} \;+\; \frac{\sqrt{2}-1}{\sqrt{2}\,|x|}, \qquad \rho < |x| < 1.$$

Remark 8.3.1 If the boundary condition is prescribed as $\varphi_{|\partial\Omega} = c > 0$ (constant), then the equilibrium hole radius depends on c, say $\rho = \rho(c)$. As c increases, the hole radius $\rho(c)$ also increases. In particular, one finds that for $c = \sqrt{2}$ the hole radius reaches $\rho = 1$, meaning that the puncture expands to fill the entire unit disk. Hence, for any $c \geq \sqrt{2}$ the equilibrium hole coincides with the whole disk.

Neural network formulation To approximate the radially symmetric punctured membrane problem we employ a hybrid architecture consisting of:

1. A feed-forward neural network $NN_\theta(r)$, parametrized by weights θ, which approximates the radial deformation profile $u(r)$.

2. A single trainable scalar parameter $\rho \in (0, 1)$, representing the radius of the free boundary.

Boundary condition embedding. To enforce the outer Dirichlet condition $u(1) = 1$, we define

$$u_\theta(r) = 1 + (1-r)\,NN_\theta(r),$$

so that $u_\theta(1) = 1$ holds exactly.

Loss function. The training loss consists of three terms:

$$\mathcal{L}(\theta,\rho) \;=\; \lambda_{\mathrm{PDE}}\mathcal{L}_{\mathrm{PDE}}(\theta,\rho)+\lambda_{\mathrm{Neu}}\mathcal{L}_{\mathrm{Neu}}(\theta,\rho)+\lambda_{\mathrm{energy}}\mathcal{L}_{\mathrm{energy}}(\theta,\rho)$$

with

$$\begin{aligned}\mathcal{L}_{\mathrm{PDE}}(\theta,\rho) &= \int_{\rho}^{1}\left(u_\theta''(r)+\frac{1}{r}u_\theta'(r)-\frac{1}{r^2}u_\theta(r)\right)^2 dr\\ &= \int_{0}^{1} H_k(r-\rho)\left(u_\theta''(r)+\frac{1}{r}u_\theta'(r)-\frac{1}{r^2}u_\theta(r)\right)^2 dr,\end{aligned}$$

with

$$H_k(x)=\frac{1}{2}(1+\tanh(kx)),\qquad k \text{ large},$$

$$\mathcal{L}_{\mathrm{Neu}}(\theta,\rho)=\big(u_\theta'(\rho)\big)^2,\qquad \mathcal{L}_{\mathrm{energy}}(\theta,\rho)=\left(\tfrac{1}{2}\left((u_\theta'(\rho))^2+\tfrac{u_\theta(\rho)^2}{\rho^2}\right)-1\right)^2.$$

Training. The pair (θ,ρ) is optimized simultaneously by minimizing $\mathcal{L}(\theta,\rho)$. At convergence, the network $u_\theta(r)$ provides the deformation profile and the scalar parameter ρ gives the equilibrium radius of the punctured hole.

Numerical results We note that the use of the differentiable mask function

$$H_k(x-\rho) \;=\; \tfrac{1}{2}\big(1+\tanh(k(x-\rho))\big),$$

with $k=50$, effectively resolves the collocation point selection problem: H_k acts as a smooth approximation of the indicator of the set $\{x\ge\rho\}$, taking values close to 1 for $x\ge\rho$ and close to 0 otherwise.

The neural network employed has three hidden layers with 32 neurons per layer, using tanh activation. Training was performed over 7,000 epochs with a learning rate of 5×10^{-4} and a batch size of 200.

Figure 13.1(a) shows that the free-boundary radius ρ stabilizes near the theoretical threshold $\rho\approx0.643$ after about 6,500 epochs:

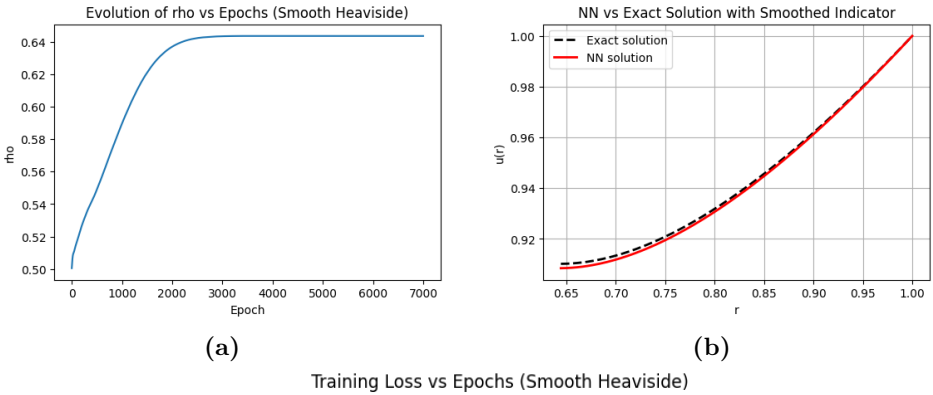
```
Epoch 6500: Loss=0.000011, rho=0.6436
```

Figure 13.1(b) compares the learned solution $u_{\theta^*}(r)$ with the exact solution

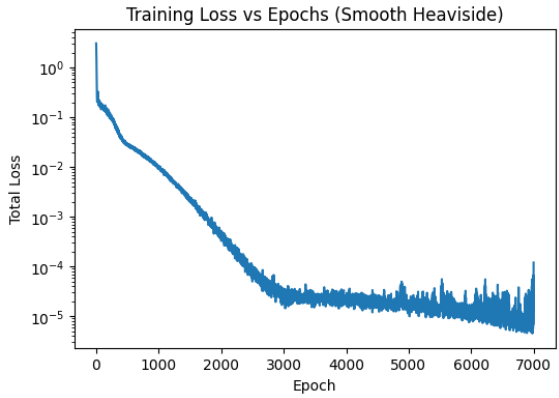
$$u_{\mathrm{exact}}(r) \;=\; \frac{r}{\sqrt{2}} \;+\; \frac{\sqrt{2}-1}{\sqrt{2}\,r},\qquad \rho<r<1,$$

demonstrating an excellent approximation quality. Finally, Fig. 13.1(c) illustrates the training loss: while it decreases steadily at first, oscillations appear after a large number of epochs, indicating that the network has essentially converged.

The code detail is available at `https://tinyurl.com/7e3j3msm`.



(a) **(b)**



(c)

Figure 8.7: An unsupervised training for the radial symmetric deformation: (a) The free-boundary radius ρ stabilizes near the theoretical threshold $\rho \approx 0.643$; (b) Comparison between the neural network solution $u_{\theta^*}(r)$ and the exact solution $u_{\text{exact}}(r)$; (c) The associated loss function.

References and further reading The dynamics and equilibrium of punctured membranes have been investigated in several works. An asymptotic PDE analysis of perforated plates and membranes, highlighting the relationship between hole geometry and stress fields, is provided by Andrianov and Danishevskyy [6]. A PDE-based formulation of punctured and indented membranes, including cavitation phenomena, is developed by Nadler and Steigmann [75]. Sellier [84] introduces an inverse elasticity problem using PDE methods, with punctured membranes serving as benchmark cases.

A rigorous mathematical treatment of the deformation gradient $\nabla\varphi_t$ and the right Cauchy–Green tensor $(\nabla\varphi_t)^T\nabla\varphi_t$ within the framework of nonlinear elasticity is given by Marsden and Hughes [68]. An engineering-oriented perspective is provided by Holzapfel [47], which emphasizes the role of invariants of the right Cauchy–Green tensor in constitutive modeling.

For broader theoretical background, the monograph of Alt and Caffarelli [5] offers

a foundational treatment of free-boundary problems in PDEs and variational settings. In addition, the classical work of Kinderlehrer and Stampacchia [54] provides a comprehensive introduction to variational inequalities, which play a central role in the PDE formulation of free boundary problems.

8.4 Exercises

Exercise 8.4.1 (Free-boundary for sublimation). A flat block of dry ice (solid CO_2) sublimates directly into gas. Sublimation is modeled as a diffusion-driven process. Let g denote the saturation vapor density of CO_2 at the sublimating interface, and let ρ denote the density of solid CO_2. At time $t = 0$, the solid surface is located at $x = 0$. Denote by $u(x,t)$ the CO_2 vapor density in air. At infinity, the vapor density vanishes, i.e. $u(x,t) \to 0$ as $x \to \infty$. The corresponding free-boundary problem is:

$$\begin{aligned}
\partial_t u &= k\partial_x^2 u, \qquad x > s(t), \quad t > 0 \\
u(x,0) &= f(x), \quad x \geq 0 \\
u(s(t),t) &= g, \quad t \geq 0 \\
(\rho - g)s'(t) &= k\partial_x u(s(t),t) \\
s(0) &= 0 \\
\lim_{x\to\infty} u(x,t) &= 0,
\end{aligned}$$

where $k > 0$ is a diffusion constant and the free-boundary $s(t)$ represents the level of the dry ice bloc at time t.

Assuming $k = f = g = 1$, $\rho = 2$, solve the problem using neural networks. Draw the graph of the free-boundary $s(t)$.

Exercise 8.4.2 (Two-phase Stefan problem). The next model describes the process of ice melting in a tube of length $L = 2$. The location of water-ice interface is denoted by $s(t)$ and the temperatures of water and ice are denoted respectively by u_w and u_i. The problem asks to find the functions $u_w(x,t)$, $u_i(x,t)$ and $s(t)$ such that

$$\begin{aligned}
\partial_t u_w &= a_w\partial_x^2 u_w, \qquad 0 < x < s(t), \quad t > 0 \\
\partial_t u_i &= a_i\partial_x^2 u_i, \qquad s(t) < x < 2, \quad t > 0 \\
u_w(x,0) &= f_w(x), \qquad 0 \leq x \leq 1 \\
u_i(x,0) &= f_i(x), \qquad 1 \leq x \leq 2 \\
\partial_x u_w(0,t) &= -\phi(t), \qquad \phi(t) \geq 0, \quad t > 0 \\
\partial_x u_i(2,t) &= 0, \quad t > 0 \\
u_w(s(t),t) &= u_i(s(t),t) = 0, \quad t > 0 \\
s'(t) &= -\alpha_w\partial_x u_w(s(t),t) + \alpha_i\partial_x u_i(s(t),t), \quad t > 0 \\
s(0) &= 1,
\end{aligned}$$

where a_w, a_i α_w and α_i are positive constants and f_i, f_w and ϕ are given functions.

(a) Explain how the problem can be solved using neural networks using both soft constraints and hard embeddings.

(b) Assume $a_w = a_i = \alpha_w = \alpha_i = 1$ and $f_w(x) = f_i(x) = 1 - x$. Use neural networks to find the free boundary $s(t)$ and draw its graph.

Exercise 8.4.3 (Curve of smallest stretch). Let $\varphi : \Omega \to \mathbb{R}^2$ be a membrane deformation, satisfying $\Delta\varphi = 0$. The stretch in the direction of a vector $v \in \mathbb{R}^2$ is defined by

$$S(v) = (\nabla\varphi)^T v.$$

Given two points $x_0, x_1 \in \Omega$ and a smooth curve $c : [0, T] \to \Omega$ with $c(0) = x_0$ and $c(T) = x_1$, the stretch energy along c is defined as

$$E[c] \;=\; \frac{1}{2}\int_0^T \|S(\dot{c}(t))\|^2\, dt.$$

(a) Derive the Euler–Lagrange equation associated with the functional $c \mapsto E[c]$.

(b) Explain how a neural network can be employed to approximate the curve of minimal stretch connecting x_0 and x_1.

Exercise 8.4.4 Let $\varphi : \Omega\backslash D \to \mathbb{R}^2$ be an equilibrium membrane deformation with puncture D. Let $x_0, x_1 \in \partial D$. Show that the curve c which joins x_0 and x_1 and has the smallest stretch is part of the boundary ∂D.

This page intentionally left blank

Chapter 9

Classical PDEs of Mathematical Physics

Most equations of Mathematical Physics involve second-order differential operators. One reason is that many of these equations arise as Euler–Lagrange equations associated with variational principles, which naturally lead to second-order derivatives.

A general second-order differential operator has the form

$$\mathcal{P}(x,\partial) = \sum_{i,j=1}^{n} a_{ij}(x)\,\partial_{x_i}\partial_{x_j} + \sum_{k=1}^{n} b_k(x)\,\partial_{x_k} + c(x),$$

where the coefficient functions $a_{ij}(x), b_k(x), c(x)$ are assumed to be smooth.

The classification of such operators depends on the properties of the quadratic form associated with the leading coefficients $a_{ij}(x)$. Let

$$A(x) = \big(a_{ij}(x)\big)_{1\le i,j\le n}$$

be the principal coefficient matrix at point x. Then:

- The operator is called *elliptic* at x if the quadratic form

  $$\sum_{i,j=1}^{n} a_{ij}(x)\,\xi_i\xi_j$$

  is positive definite (or negative definite) for all $\xi \neq 0$.

- It is called *hyperbolic* at x (with respect to some distinguished direction, typically time) if the quadratic form has exactly one negative eigenvalue and the rest positive (or vice versa). This ensures a well-posed Cauchy problem resembling wave propagation.

- It is called *parabolic* at x if the quadratic form is positive semidefinite but degenerate, meaning that some eigenvalues vanish. This is typical for diffusion-type equations.

The classical prototypes of these categories are:

- The *Laplace operator* $\Delta = \sum_i \partial_{x_i}^2$, which is elliptic and models static or equilibrium processes.
- The *heat operator* $\partial_t - \Delta$, which is parabolic and models diffusion and dissipative evolution.
- The *d'Alembert operator* $\Box = \partial_t^2 - \Delta$, which is hyperbolic and models wave propagation.

In this chapter, we study the direct problem of finding solutions to boundary value and initial value problems associated with these classes of operators via neural networks.

9.1 Elliptic Equations

9.1.1 Solving Poisson Equation with Dirichlet Condition

Consider a bounded, connected domain $\Omega \subset \mathbb{R}^n$, together with continuous functions $f \in C(\Omega)$ and $\varphi \in C(\partial\Omega)$. The *Dirichlet problem* for the Poisson equation is given by

$$\Delta u = f \quad \text{in } \Omega, \tag{9.1.1}$$

$$u = \varphi \quad \text{on } \partial\Omega. \tag{9.1.2}$$

This problem is well-posed: it admits a unique solution $u \in C^2(\Omega) \cap C(\overline{\Omega})$. Moreover, if the Green's function[1] $G(x, y)$ of the domain Ω is known, the solution admits the explicit representation

$$u(x) = -\int_\Omega G(x,y) f(y)\, dy - \int_{\partial\Omega} \varphi(y)\, \frac{\partial G}{\partial n_y}(x,y)\, d\sigma_y, \quad x \in \Omega,$$

where $\frac{\partial}{\partial n_y}$ denotes the outward normal derivative at $y \in \partial\Omega$ and $d\sigma_y$ is the surface measure.

The special case $f = 0$ corresponds to the *Laplace equation*, in which case the solution u is a harmonic function with prescribed boundary data. In particular, when $\Omega = B(0, R)$ is the ball of radius R centered at the origin, the formula reduces to *Poisson's formula*:

$$u(x) = \frac{R^2 - |x|^2}{R\,\omega_n} \int_{\partial B(0,R)} \frac{\varphi(y)}{\|x - y\|^n}\, d\sigma_y, \quad x \in B(0, R),$$

where ω_n is the surface area of the unit sphere in $\mathbb{R}^n$.

[1] A function $G(x, y)$ is called the *Green's function* for the Laplacian in Ω with Dirichlet boundary conditions if it satisfies

$$\Delta_x G(x,y) = -\delta(x - y), \qquad x \in \Omega,$$
$$G(x,y) = 0, \qquad x \in \partial\Omega,$$

and $G(\cdot, y)$ has suitable regularity in $\Omega \setminus \{y\}$.

Here δ is the Dirac delta in the variable x. Intuitively, $G(x, y)$ is the potential at the point x due to a unit "point source" placed at y, with the boundary held at zero potential.

For general domains Ω without symmetries, the Green's function cannot usually be expressed in closed form. In such cases, *neural networks* can provide a flexible and powerful tool for approximating the solution u directly, bypassing the need for an explicit Green's function.

Neural network approach To proceed, we decompose the Dirichlet problem (9.1.1)–(9.1.2) into the sum of two simpler problems by exploiting linearity:

1. A Poisson problem with homogeneous boundary condition:

$$\begin{aligned} \Delta v &= f \quad \text{in } \Omega, && (9.1.3)\\ v &= 0 \quad \text{on } \partial\Omega. && (9.1.4) \end{aligned}$$

2. A Laplace problem with nonhomogeneous Dirichlet boundary condition:

$$\begin{aligned} \Delta w &= 0 \quad \text{in } \Omega, && (9.1.5)\\ w &= \varphi \quad \text{on } \partial\Omega. && (9.1.6) \end{aligned}$$

If $v(x)$ and $w(x)$ denote the solutions of the above problems, then their sum

$$u(x) = v(x) + w(x)$$

provides the unique solution to the original problem (9.1.1)–(9.1.2). In what follows, we shall employ neural networks to approximate the solutions of each subproblem separately, using two methods.

The PDE enforcement method This neural network approach enforces the PDE constraints (Poisson and Laplace equations) directly through the loss functions, while boundary conditions are imposed either explicitly (via hard embedding) or softly (via penalization). The method is effective provided a sufficiently large and well-distributed set of collocation points is used.

1. *Solving the Poisson problem.* To solve the first problem, we construct a neural network approximation in which the vanishing boundary condition is hard-embedded:

$$v_\theta(x) \;=\; b(x)\,\mathrm{nn}(\theta; x),$$

where $b \in C^2(\Omega)\cap C(\overline{\Omega})$ is a smooth function satisfying $b|_{\partial\Omega} = 0$. We then choose N collocation points $x_j \in \Omega$ and define the loss function

$$\mathcal{L}_1(\theta) \;=\; \frac{1}{N}\sum_{j=1}^{N} \big(\Delta v_\theta(x_j) - f(x_j)\big)^2.$$

Minimization of this loss yields the approximate solution

$$v(x) \;\approx\; v_{\theta^*}(x), \quad \text{where } \theta^* = \arg\min_\theta \mathcal{L}_1(\theta).$$

2. *Solving the Laplace problem.* For the second problem, we use a neural network $w_\theta(x)$, trained to minimize the composite loss

$$\mathcal{L}_2(\theta) = \frac{1}{N}\sum_{j=1}^{N} \left|\Delta w_\theta(x_j)\right|^2 + \frac{\lambda}{M}\sum_{k=1}^{M} \big(w_\theta(y_k) - \varphi(y_k)\big)^2,$$

where $x_j \in \Omega$ are interior collocation points and $y_k \in \partial\Omega$ are boundary points. The resulting approximation is

$$w(x) \;\approx\; w_{\theta^*}(x).$$

The variational method In this section, we present an alternative and equivalent framework based on the *variational principle*, which provides additional insight and numerical stability.

1. *Solving the Poisson problem.* To solve the first problem, we recall that the Poisson equation

$$\Delta v = f \quad \text{in } \Omega$$

is the Euler–Lagrange equation associated with the energy functional

$$\mathcal{E}[v] \;=\; \int_\Omega \big(\tfrac{1}{2}\|\nabla v(x)\|^2 - f(x)\, v(x)\big)\, dx,$$

where the gradient is given by $\nabla v = (\partial_{x_1} v(x), \ldots, \partial_{x_n} v(x))$.

The integrand represents the difference between the *kinetic energy density* $\frac{1}{2}\|\nabla v\|^2$ and the *potential energy density* $U(v) = f\, v$.

To enforce the homogeneous boundary condition, we use the hard embedding

$$v_\theta(x) \;=\; b(x)\,\mathrm{nn}(\theta; x),$$

where $b \in C^2(\Omega) \cap C(\overline{\Omega})$ is a smooth function satisfying $b|_{\partial\Omega} = 0$.

The variational loss based on the discretized energy functional is then defined as

$$\widetilde{\mathcal{L}}_1(\theta) \;\approx\; \frac{1}{N}\sum_{j=1}^{N} \left(\tfrac{1}{2}\|\nabla v_\theta(x_j)\|^2 - f(x_j)\, v_\theta(x_j)\right),$$

where $x_j \in \Omega$ are collocation points. Minimizing this loss gives the approximation

$$v(x) \;\approx\; v_{\theta^*}(x), \quad \text{where } \theta^* = \arg\min_\theta \widetilde{\mathcal{L}}_1(\theta).$$

2. *Solving the Laplace problem.* For the second problem, we use that the Laplace equation

$$\Delta w = 0 \quad \text{in } \Omega$$

is the Euler–Lagrange equation associated with the *Dirichlet energy functional*

$$\mathcal{D}[w] \;=\; \int_\Omega \tfrac{1}{2}\|\nabla w(x)\|^2 dx.$$

We approximate the solution by a neural network $w_\theta(x)$. To incorporate the boundary condition $w = \varphi$ on $\partial\Omega$, we add it as a soft penalty term to the variational loss:

$$\widetilde{\mathcal{L}}_2(\theta) \;\approx\; \frac{1}{N}\sum_{j=1}^{N} \tfrac{1}{2}\|\nabla w_\theta(x_j)\|^2 + \frac{\lambda}{M}\sum_{k=1}^{M} \big(w_\theta(y_k) - \varphi(y_k)\big)^2,$$

where $x_j \in \Omega$ are interior collocation points and $y_k \in \partial\Omega$ are boundary points. Minimizing this loss yields the approximation

$$w(x) \;\approx\; w_{\theta^*}(x).$$

Radial Reduction of the Poisson Equation Exploiting the rotational invariance of the Laplacian allows us to reduce the dimension of certain problems. For instance, consider the Poisson equation with homogeneous boundary conditions on the ball $B(0,R) \subset \mathbb{R}^n$, when the source function $f(x)$ is radially symmetric. By symmetry, the solution $v(x)$ must also be radially symmetric, which reduces the problem to one spatial variable.

More precisely, if

$$f(x) = \rho(|x|), \qquad v(x) = \psi(|x|),$$

then, in polar coordinates, the Poisson equation $\Delta v = f$ reduces to the *radial Poisson equation*

$$\psi''(r) + \frac{n-1}{r}\psi'(r) = \rho(r), \quad r = \|x\|.$$

To enforce the homogeneous boundary condition $\psi(R) = 0$, we embed it directly into the neural network ansatz by setting

$$\psi_\theta(r) = (R - r)\,\phi_\theta(r),$$

where ϕ_θ is represented by a neural network.

The loss function is then defined by sampling N collocation points $\{r_j\}$ in $(0, R]$:

$$\mathcal{L}_{\text{rad}}(\theta) = \frac{1}{N}\sum_{j=1}^{N}\left(\psi_\theta''(r_j) + \frac{n-1}{r_j}\,\psi_\theta'(r_j) - \rho(r_j)\right)^2.$$

This radial reduction significantly simplifies the learning problem, since the neural network now depends only on the one-dimensional input r.

Remark 9.1.1 Since the solution is radially symmetric, $v(0)$ must be a local extremum, which implies

$$\psi'(0) = 0.$$

We can encode this as a soft constraint by adding the penalty term

$$|\psi_\theta'(0)|^2$$

to the previous loss.

Example Consider the Poisson equation with homogeneous boundary conditions

$$\begin{cases} \Delta v(x) = 12|x| + 6, & x \in B(0,1) \subset \mathbb{R}^2, \\ v(x) = 0, & x \in \partial B(0,1). \end{cases}$$

A direct computation shows that the solution is given by

$$v(x) = |x|^3 + |x|^2 - 2, \qquad x \in B(0,1).$$

Thus, the solution is a radial cubic polynomial in $r = |x|$, which vanishes at the boundary $r = 1$.

We now verify this conclusion using a neural network approach. To this end, we embed the boundary condition by choosing

$$\psi_\theta(r) = (1 - r)\,\phi_\theta(r), \quad 0 \le r \le 1,$$

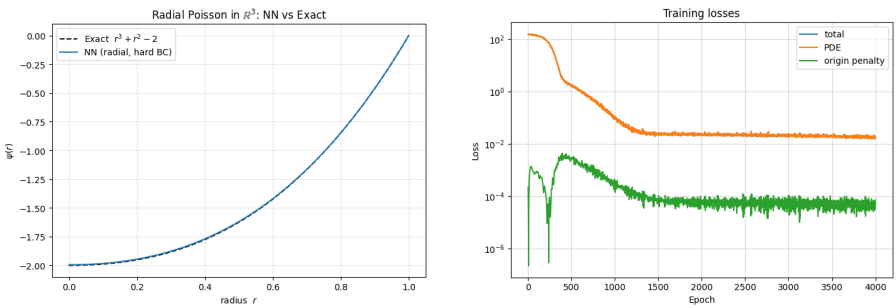
where ϕ_θ is a neural network. The corresponding loss function, obtained from the radial Poisson equation in $\mathbb{R}^3$ and the soft constraint at the origin, is defined as

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{j=1}^{N}\left(\psi_\theta''(r_j) + \frac{2}{r_j}\,\psi_\theta'(r_j) - 12r_j - 6\right)^2 + \lambda|\psi_\theta'(0)|^2.$$

We trained the neural network for $4,000$ epochs using the Adam optimizer with a learning rate of 10^{-4}. The architecture consisted of a feedforward network with three hidden layers, each containing 64 neurons with Swish activation.

The resulting approximation is visually indistinguishable from the exact solution, see Fig. 9.1(a). The loss curve, shown in Fig. 9.1(b), decreases steadily during training and saturates, indicating convergence of the method.

The code details can be found at `https://tinyurl.com/ysjs97x8`.



(a) **(b)**

Figure 9.1: (a) The neural network solution $\psi_\theta(r)$ matches the exact solution $\psi(r) = r^3 + r^2 - 2$; (b) The associated losses.

Poisson Equation in Physics The Poisson equation models static phenomena such as potentials generated by sources: mass (gravity), charge (electrostatics), heat (thermal conduction), stress (elasticity), and pressure (fluids).

1. In *electrostatics*, Poisson's equation governs the relation between the electrostatic potential $V(x)$ and the charge density $\rho_{\text{charge}}(x)$:

$$\Delta V(x) \;=\; -\frac{\rho_{\text{charge}}(x)}{\epsilon_0},$$

where $V(x)$ is the electrostatic potential at $x \in \mathbb{R}^3$, $\rho_{\text{charge}}(x)$ is the electric charge density, and ϵ_0 is the permittivity of free space.

If some measurements of the electric charge density are given, $\rho_i = \rho_{charge}(x_i)$ at N space collocation points $\{x_i\}_{i=1}^N \subset \Omega$ and if the potential is grounded on the boundary of Ω, namely $V_{|\partial\Omega} = 0$, then the potential can be learned by a neural network $V_\theta(x)$ which minimizes the energy loss with boundary constraints

$$\begin{aligned}\mathcal{L}(\theta) \;&=\; \int_\Omega \frac{1}{2}\|\nabla V_\theta(x)\|^2 + \frac{1}{\epsilon_0}\rho_{\text{scharge}}(x)V_\theta(x)dx + \lambda\int_\partial V(y)^2 d\sigma_y \\ &\approx\; \frac{|\Omega|}{N}\sum_{i=1}^N\left[\frac{1}{2}\left\|\nabla V_\theta(x_i)\right\|^2 \;+\; \frac{1}{\epsilon_0}\,\rho_i\,V_\theta(x_i)\right] \;+\; \lambda\,\frac{|\partial\Omega|}{M}\sum_{k=1}^M\big(V_\theta(y_k)\big)^2,\end{aligned}$$

with boundary collocation points $\{y_k\}_{k=1}^M \subset \partial\Omega$.

2. In *Newtonian gravity*, Poisson's equation describes how the gravitational potential $u(x)$ is generated by the mass density $\rho(x)$:

$$\Delta u(x) \;=\; 4\pi G\,\rho(x),$$

where $u(x)$ is the gravitational potential at $x \in \mathbb{R}^3$, $\rho(x)$ is the mass density, and G is the universal gravitational constant.

3. In the steady state of heat the temperature distribution $u(x)$ at equilibrium with sources or sinks $q(x)$ satisfies $\Delta u(x) = -\frac{q(x)}{k}$ where k is a thermal conductivity constant.

4. In elasticity the displacement potential $\Phi(x)$ satisfies $\Delta\Phi(x) = -f(x)$, where $f(x)$ encodes body forces.

9.1.2 Solving a Nonlinear Dirichlet Problem

Let $\Omega \subset \mathbb{R}^n$ be a bounded domain with sufficiently smooth boundary $\partial\Omega$. We consider the nonlinear Dirichlet boundary value problem

$$\begin{cases}\Delta u(x) = f(x, u(x)), & x \in \Omega, \\ u(x) = \varphi(x), & x \in \partial\Omega,\end{cases}$$

where $f : \overline{\Omega} \times \mathbb{R} \to \mathbb{R}$ is a nonlinear source term and $\varphi : \partial\Omega \to \mathbb{R}$ is the prescribed boundary condition.

Unlike the linear Poisson problem, closed-form solutions for such nonlinear equations rarely exist. However, the following result guarantees that the problem is well-posed, which justifies searching for approximate solutions:

Theorem 9.1.2 *Suppose that $\varphi \in C(\partial\Omega)$, $f \in C^2(\overline{\Omega} \times \mathbb{R})$, and that f is monotone increasing in the variable u. Then the Dirichlet problem admits a unique classical solution*

$$u \in C^2(\Omega) \cap C(\overline{\Omega}).$$

This theorem ensures both *existence* and *uniqueness* of the solution, meaning that neural network methods can be reliably employed to approximate the solution numerically.

Neural network approach The neural network approach employs a parametric function $u_\theta(x)$, represented by a neural network, to approximate the solution of the nonlinear Dirichlet problem. As in the linear case, the training can be carried out in two different ways.

1. For the *PDE enforcement method*, we define the loss function as

$$\mathcal{L}(\theta) \;=\; \frac{1}{N}\sum_{j=1}^{N}\Big(\Delta u_\theta(x_j) - f(x_j, u_\theta(x_j))\Big)^2 \;+\; \frac{\lambda}{M}\sum_{k=1}^{M}\big(u_\theta(y_k) - \varphi(y_k)\big)^2,$$

where $\{x_j\}_{j=1}^N \subset \Omega$ are collocation points in the interior of the domain, and $\{y_k\}_{k=1}^M \subset \partial\Omega$ are boundary points used to enforce the Dirichlet condition.

2. For the *variational method*, we define

$$F(x,u) \;=\; \int_0^u f(x,y)\,dy,$$

so that the PDE $\Delta u(x) = f(x,u)$ on Ω arises as the Euler–Lagrange equation of the energy functional

$$E[u] \;=\; \int_\Omega \left(\tfrac{1}{2}\|\nabla u(x)\|^2 - F(x,u(x))\right)\,dx.$$

In this framework, the neural network $u_\theta(x)$ is trained to minimize the regularized energy

$$\mathcal{L}(\theta) \;=\; E[u_\theta] + \frac{\lambda}{M}\sum_{k=1}^{M}\big(u_\theta(y_k) - \varphi(y_k)\big)^2,$$

where the second term enforces the Dirichlet boundary condition at boundary collocation points $\{y_k\}_{k=1}^M \subset \partial\Omega$.

When choosing collocation points on the boundary $\partial\Omega$, a natural strategy is to distribute them uniformly in arc-length (or surface area in higher dimensions).

However, in regions of higher curvature, the solution may vary more rapidly, and thus a denser sampling is needed. A practical approach is to distribute boundary points with density proportional to the local curvature $\kappa(y)$ of $\partial\Omega$, while keeping a uniform baseline distribution elsewhere. This ensures both global coverage and finer resolution in geometrically complex regions.

9.2 Exercises

Exercise 9.2.1 Consider the Laplace equation with Dirichlet boundary conditions

$$\Delta u = 0 \quad \text{in } \Omega = [0,1] \times [0,1],$$

together with the boundary data

$$\varphi(x,1) = \sin(\pi x), \qquad \varphi(0,y) = 0, \qquad \varphi(1,y) = 0, \qquad \varphi(x,0) = 0.$$

(a) Use a neural network to approximate the solution $u(x,y)$. Implement both the PDE enforcement method and the variational method.

(b) Show that the exact solution of this problem is

$$u_{\text{exact}}(x,y) \;=\; \frac{\sinh(\pi y)}{\sinh(\pi)} \sin(\pi x).$$

(c) Compare the neural network solution with the exact solution. Adjust the network architecture and training parameters until the two solutions match closely.

Exercise 9.2.2 Consider the nonlinear Dirichlet problem

$$\begin{cases} \Delta f(x) = f(x)^3, & x \in B(0,1) \subset \mathbb{R}^3, \\ f(x) = 1, & x \in \partial B(0,1). \end{cases}$$

(a) Construct a neural network $f_\theta(x)$ to approximate the solution of this problem. Use either the PDE enforcement method or the variational method described previously.

(b) Train the network using collocation points inside the ball $B(0,1)$ and on the boundary $\partial B(0,1)$.

(c) Analyze the performance of the NN solution. In particular, check whether the boundary condition is satisfied and whether the PDE residual $\Delta f_\theta(x) - f_\theta(x)^3$ is small across Ω.

Exercise 9.2.3 Consider the nonlinear Dirichlet problem

$$\begin{cases} \Delta f = f^3, & \text{in } B(0,1) \subset \mathbb{R}^3, \\ f = 1, & \text{on } \partial B(0,1). \end{cases}$$

We seek a radially symmetric solution $f(x) = \psi(r)$ with $r = \|x\|$. Recall the radial Laplacian in $\mathbb{R}^3$:

$$\Delta f = \psi''(r) + \frac{2}{r}\,\psi'(r).$$

(a) Show that ψ must satisfy the boundary value problem

$$\psi''(r) + \frac{2}{r}\,\psi'(r) = \psi(r)^3, \qquad 0 < r < 1,$$

with $\psi'(0) = 0$ (regularity at the origin) and $\psi(1) = 1$ (Dirichlet data).

(b) Prove that any smooth radial solution is monotone increasing on $(0,1)$, hence $\psi(0) < 1$.

(c) Reformulate the problem as a shooting problem: for $a = \psi(0)$ solve $\psi'' + \frac{2}{r}\psi' = \psi^3$ with $\psi(0) = a$, $\psi'(0) = 0$, and find a such that $\psi(1) = 1$.

(d) Solve the ODE using the neural network $\psi_\theta(r)$ and minimizing the loss

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{j=1}^{N}\left(\psi_\theta''(r_j) + \tfrac{2}{r_j}\psi_\theta'(r_j) - \psi_\theta(r_j)^3\right)^2 + \lambda_1\,\psi_\theta'(0)^2 + \lambda_2\,(\psi_\theta(1)-1)^2,$$

where ψ_θ is a NN in $r \in [0,1]$ and (r_j) are interior collocation points.

Hint. Use $(r^2\psi')' = r^2\psi^3$ to deduce $\psi'(r) > 0$ for $r > 0$ when $\psi \geq 0$, and continuity of the shooting map $a \mapsto \psi_a(1)$ to justify existence of a unique $a \in (0,1)$ with $\psi_a(1) = 1$.

Exercise 9.2.4 (Exterior Dirichlet problem). Consider the Laplace equation in the exterior of the unit disk in $\mathbb{R}^2$:

$$\begin{cases} \Delta f = 0, & \text{in } \Omega := \mathbb{R}^2 \setminus \overline{B(0,1)}, \\ f = 1, & \text{on } \partial B(0,1), \\ f(x) \to 0, & \text{as } |x| \to \infty. \end{cases}$$

Assume a radially symmetric solution $f(x) = \psi(r)$ with $r = \|x\|$.

(a) Show that the radial Laplacian in $\mathbb{R}^3$ gives

$$\psi''(r) + \frac{2}{r}\,\psi'(r) \;=\; 0, \qquad r > 1,$$

with boundary/decay conditions $\psi(1) = 1$ and $\psi(r) \to 0$ as $r \to \infty$.

(b) Solve this ODE and prove that the unique solution is

$$\psi(r) = \frac{1}{r}, \qquad r \geq 1.$$

(*Hint:* Integrate $(r^2\psi')' = 0$.)

(c) Using a 1D neural network $\psi_\theta(r)$ on $[1,\infty)$ (truncated numerically to $[1, R_{\max}]$), minimize

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{j=1}^{N}\left(\psi_\theta''(r_j) + \tfrac{2}{r_j}\psi_\theta'(r_j)\right)^2 + \lambda_1\big(\psi_\theta(1)-1\big)^2 + \lambda_2\,\psi_\theta(R_{\max})^2,$$

and compare the learned $\psi_\theta(r)$ with the exact profile $1/r$.

9.3 Parabolic Equations

In this section, we study the *heat operator* $\mathcal{P}(\partial) = \partial_t - \Delta$ in two settings:

1. The *mixed problem*, where both initial and boundary conditions are prescribed, and

2. The *Cauchy problem*, where the solution is specified only through initial conditions in the whole space.

The heat operator governs diffusion-type phenomena and typically admits smooth, classical solutions when the data (initial and boundary conditions) are sufficiently regular.

In the framework of *neural networks*, the goal is to approximate solutions of these problems directly from the PDE and the associated constraints. When a classical solution exists, the neural network can recover it to high accuracy.

When no classical solution exists (for instance, due to rough, irregular or incompatible data), the network converges to the function that best approximates the *generalized solution*.[2] This provides a practical computational approach even in ill-posed or non-classical cases.

9.3.1 The Mixed Problem

We consider an open bounded domain $\Omega \subset \mathbb{R}^n$ with smooth boundary $\partial\Omega$, and a time interval $(0, T)$ with $T > 0$. For convenience, we introduce the following notations:

- the *space–time cylinder*: $Q_T = \Omega \times (0, T)$,
- the *lateral boundary*: $S_T = \partial\Omega \times (0, T)$.

We study the following mixed initial–boundary value problem for the heat equation:

$$\begin{cases} \partial_t u - \Delta u = f, & \text{in } Q_T, \\ u(0, x) = u_0(x), & x \in \Omega, \\ u(t, y) = \psi(t, y), & (t, y) \in S_T, \end{cases}$$

where $u(t, x)$ represents the temperature at point x and time t. The data consist of:

- a source term $f : Q_T \to \mathbb{R}$,
- an initial condition $u_0 : \Omega \to \mathbb{R}$,
- and boundary data $\psi : S_T \to \mathbb{R}$,

which satisfy the natural compatibility condition

$$u_0(y) = \psi(0, y), \quad \text{for all } y \in \partial\Omega.$$

[2]These generalized solutions are also called *solutions in the sense of distributions*, a concept introduced by Laurent Schwartz in the mid-20th century. In this framework, functions are extended to act on smooth test functions via duality, allowing one to rigorously define derivatives of objects that are not classically differentiable (e.g., the Dirac delta distribution).

Under suitable smoothness assumptions on f, u_0, and ψ, this mixed problem admits a unique classical solution $u \in C^{1,2}(Q_T) \cap C(\overline{Q_T})$, where $C^{1,2}$ denotes functions with one continuous time derivative and two continuous spatial derivatives, see Friedman [34]. If the data are less regular, the problem is still well-posed in the sense of weak solutions in appropriate Sobolev spaces.

Neural Network Approximation

Neural networks can be used to approximate solutions of this problem. To this end, a neural network with the initial condition embedded

$$u_\theta(t,x) = u_0(x) + t\,\mathrm{nn}_\theta(t,x)$$

is trained to satisfy the PDE and boundary condition by minimizing the loss

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{PDE}}(\theta) + \mathcal{L}_{\text{bd}}(\theta),$$

where

$$\mathcal{L}_{\text{PDE}}(\theta) \;=\; \frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M}\Big(\partial_t u_\theta(t_i,x_j) - \Delta u_\theta(t_i,x_j) - f(t_i,x_j)\Big)^2,$$

and

$$\mathcal{L}_{\text{bd}}(\theta) \;=\; \frac{1}{NK}\sum_{i=1}^{N}\sum_{k=1}^{K}\big(u_\theta(t_i,y_k) - \psi(t_i,y_k)\big)^2,$$

over collocation points $(t_i,x_j) \in Q_T$ and boundary points $y_k \in \partial\Omega$.

The neural network u_θ can approximate classical solutions whenever they exist; otherwise, it converges to the weak solution that best fits the data in the chosen loss framework.

Example 9.3.1 We consider the particular case of a finite rod of length L, namely the case when $\Omega = (0,L)$, with given initial temperature distribution $u_0(x)$. If the boundary temperatures are modeled by $\psi_i(t)$, we obtain the following mixed problem:

$$\begin{cases} \partial_t u - \partial_x^2 u = f, & \text{in } Q_T = (0,L)\times(0,T),\\ u(0,x) = u_0(x), & x \in (0,L),\\ u(t,0) = \psi_1(t),\\ u(t,L) = \psi_2(t), \quad t \ge 0, \end{cases}$$

with compatibility conditions $u_0(0) = \psi_1(0)$ and $u_0(L) = \psi_2(t)$. Then $u(t,x)$ represents the evolution of the temperature in the rod at point x at time t. For this setup we can embed the boundary conditions in the neural network as

$$u_\theta(t,x) = \frac{x}{L}\psi_2(t) + \frac{L-x}{L}\psi_1(t) + x(L-x)\mathrm{nn}(\theta;t,x).$$

This formula satisfies the boundary conditions but not the initial condition. To accommodate also for the initial condition, we set $u_\theta(0,x) = u_0(x)$, solve for $\mathrm{nn}(\theta;0,x)$ and write

$$\mathrm{nn}(\theta;t,x) = \mathrm{nn}(\theta;0,x) + t\mathrm{NN}(\theta;t,x).$$

With this modification, we obtain a hard embedding of both the initial and boundary conditions into the network as

$$u_\theta(t,x) = u_0(x) + \frac{x}{L}(\psi_2(t) - \psi_2(0)) + \frac{L-x}{L}(\psi_2(t) - \psi_1(0)) + tx(L-x)NN(\theta;t,x),$$

where $NN(\theta;t,x)$ is a free feed-forward neural network.

With the aforementioned embedding it suffices to minimize just the PDE loss

$$\mathcal{L}_{\text{PDE}}(\theta) \;=\; \frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M}\Big(\partial_t u_\theta(t_i,x_j) - \partial_x^2 u_\theta(t_i,x_j) - f(t_i,x_j)\Big)^2.$$

Implementation To implement the previous example, we fix the parameters as $L = \pi$, $f = 0$, $\psi_1 = 0$, $\psi_2 = 0$, and $u_0(x) = \sin x$. The corresponding mixed problem is

$$\begin{cases} \partial_t u - \partial_x^2 u = 0, & \text{in } (0,\pi)\times(0,T), \\ u(0,x) = \sin x, & x \in (0,\pi), \\ u(t,0) = 0, \quad u(t,\pi) = 0, & t \geq 0. \end{cases}$$

As validation, we use the exact solution

$$u_{\text{exact}}(t,x) = e^{-t}\,\sin x.$$

The neural network is constructed with a hard embedding that enforces both the Dirichlet boundary conditions and the initial condition:

$$u_\theta(t,x) \;=\; \sin x + t\,x(\pi - x)\,\text{NN}(\theta;t,x).$$

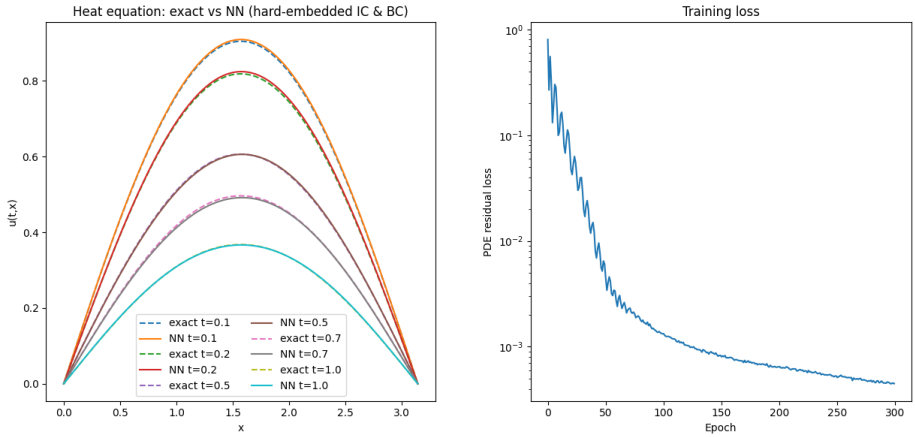
The training minimizes the PDE residual loss

$$\mathcal{L}_{\text{PDE}}(\theta) \;=\; \frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M}\Big(\partial_t u_\theta(t_i,x_j) - \partial_x^2 u_\theta(t_i,x_j)\Big)^2,$$

evaluated at collocation points (t_i, x_j). For training, we sampled $N = 128$ time points and $M = 256$ spatial points.

We used a feed-forward neural network with 4 hidden layers of 64 `tanh`-activated neurons, trained for 300 epochs with the Adam optimizer at learning rate 10^{-3}. The neural network solution matches the exact solution profiles for $t \in \{0.1, 0.2, 0.5, 0.7, 1.0\}$, see Fig. 9.2(a). After 300 epochs the loss is still steadily decreasing, see Fig. 9.2(b), and with further training the NN solution becomes indistinguishable from the exact solution.

The code details can be found at `https://tinyurl.com/bdef9zsd`.



(a) **(b)**

Figure 9.2: (a) The neural network solution $u_\theta(t,x)$ matches the exact solution $u(t,x) = e^{-t}\sin x$ at instances $t \in \{0.1, 0.2, 0.5, 0.7, 1.0\}$; (b) The associated loss.

9.3.2 The Cauchy Problem

When $\Omega = \mathbb{R}^n$, no boundary conditions are required, and the problem reduces to an initial value problem, known as the *Cauchy problem*:

$$\partial_t u(t,x) - \Delta u(t,x) = f(t,x), \quad (t,x) \in (0,\infty) \times \mathbb{R}^n, \tag{9.3.7}$$

$$u(0,x) = u_0(x), \quad x \in \mathbb{R}^n, \tag{9.3.8}$$

where u_0 and f are prescribed functions on $\mathbb{R}^n \times [0,\infty)$. The function f represents the source density in $\mathbb{R}^n$, while u_0 describes the initial heat distribution.

To set a framework in which this problem is well-posed, we introduce the following notation:

$$C^{1,2}((0,\infty) \times \mathbb{R}^n) = \left\{u \,;\, \partial_t u,\, \partial_{x_i}\partial_{x_j} u \in C((0,\infty) \times \mathbb{R}^n)\right\},$$

and let $\mathcal{M}$ denote the set of functions that are continuous on $[0,\infty) \times \mathbb{R}^n$ and bounded on sets of the form $[0,T] \times \mathbb{R}^n$, with $T > 0$.

We also denote by

$$E(x,t) = H(t)\,\frac{1}{(4\pi t)^{n/2}} \exp\left(-\frac{\|x\|^2}{4t}\right), \quad (x,t) \in (0,\infty) \times \mathbb{R}^n,$$

the heat kernel, where $H(t)$ is the Heaviside step function.

The well-posedness of the problem, together with the existence of a classical solution, is given by the following result (see, e.g., Guenther and Lee [43]):

Theorem 9.3.2 *Let u_0 be a continuous and bounded function on $\mathbb{R}^n$, and assume f, $\partial_t f$, $\partial_{x_i} f \in \mathcal{M}$. Then the function*

$$u(t,x) = \int_{\mathbb{R}^n} E(x-y,t)\,u_0(y)\,dy \;+\; \int_0^t \int_{\mathbb{R}^n} E(x-y,t-s)\,f(y,s)\,dy\,ds$$

is the unique solution of the Cauchy problem (9.3.7)–(9.3.8) *satisfying*

$$u \in \mathcal{M} \cap C^{1,2}((0,\infty) \times \mathbb{R}^n).$$

To solve the problem by neural networks, we embed the initial conditions into the network by defining

$$u_\theta(t,x) = u_0(x) + t\,\mathrm{nn}(\theta; t, x),$$

where $\mathrm{nn}(\theta; t, x)$ denotes the neural network output. The network is then trained to satisfy the PDE by minimizing the loss

$$\mathcal{L}_{\mathrm{PDE}}(\theta) \;=\; \frac{1}{NM} \sum_{i=1}^{N} \sum_{j=1}^{M} \Big(\partial_t u_\theta(t_i, x_j) - \Delta u_\theta(t_i, x_j) - f(t_i, x_j) \Big)^2.$$

Implementation example Consider the Cauchy problem

$$\partial_t u(t,x) - \partial_x^2 u(t,x) \;=\; \sin t, \tag{9.3.9}$$

$$u(0,x) \;=\; e^{-x^2}, \quad x \in \mathbb{R}. \tag{9.3.10}$$

A straightforward computation shows that a classical solution is

$$u(t,x) = 1 - \cos t \;+\; \frac{1}{\sqrt{1+4t}} \exp\!\Big(-\tfrac{x^2}{1+4t} \Big).$$

To approximate this solution with a neural network, we embed the initial condition into the ansatz

$$u_\theta(t,x) = e^{-x^2} + t\,\mathrm{nn}(\theta; t, x),$$

where $\mathrm{nn}(\theta; t, x)$ denotes the neural network output. The network is trained by minimizing the PDE residual

$$\mathcal{L}_{\mathrm{PDE}}(\theta) \;=\; \frac{1}{NM} \sum_{i=1}^{N} \sum_{j=1}^{M} \Big(\partial_t u_\theta(t_i, x_j) - \partial_x^2 u_\theta(t_i, x_j) - \sin(t_i) \Big)^2.$$

Since the spatial domain is $\mathbb{R}$, we approximate it by the truncated interval $[-4, 4]$, using 128 uniformly spaced collocation points. In time, we take 128 collocation points in the interval $(0, 2)$. The network is trained for $2,000$ epochs with the Adam optimizer at a learning rate of 10^{-3}. The architecture is a fully connected feed-forward network with three hidden layers, each using tanh activations. Validation is performed against the exact solution at the time instances $t = 0.1,\ 0.5,\ 1.0,\ 1.5,\ 2.0$, see Fig. 9.3. Overall, the neural network approximation matches the exact solution well, although the agreement tends to deteriorate as t increases.

The code details can be found at `https://tinyurl.com/2fv7m9k5`.

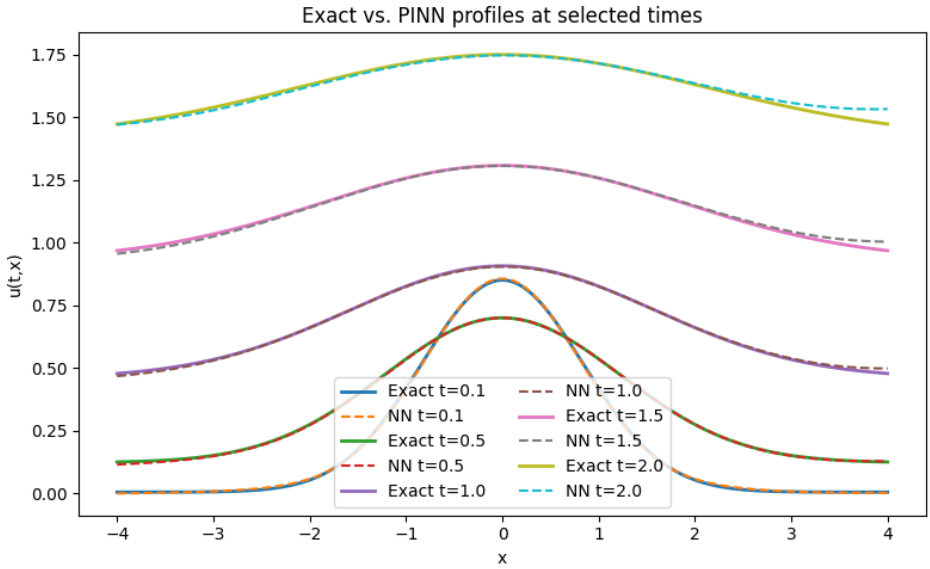


Figure 9.3: The exact solution versus the NN-solution at $t = 0.1, 0.5, 1.0, 1.5, 2.0$.

9.3.3 Schrödinger's Equation

Another fundamental linear PDE of parabolic type is the time-dependent Schrödinger equation for a quantum particle of mass m moving in a potential $U : \mathbb{R}^3 \to \mathbb{R}$. The complex wave function $\psi : \mathbb{R} \times \mathbb{R}^3 \to \mathbb{C}$ encodes probability densities via an integral as

$$\int_D |\psi(t,x)|^2 \, dx \quad = \text{probability that the particle is in } D \subset \mathbb{R}^3 \text{ at time } t,$$

and satisfies

$$i\hbar\, \partial_t \psi(t,x) + \frac{\hbar^2}{2m}\, \Delta_x \psi(t,x) = U(x)\, \psi(t,x), \qquad t \in \mathbb{R},\ x \in \mathbb{R}^3, \tag{9.3.11}$$

with the mass conservation relation $\int_{\mathbb{R}^3} |\psi(t,x)|^2 \, dx = 1$ for all t, and initial data

$$\psi(0,x) = \psi_0(x) \in L^2(\mathbb{R}^3), \qquad \|\psi_0\|_{L^2} = 1.$$

We shall describe a few methods of solving the Schrödinger equation.

Method 1: To solve (9.3.11) with neural networks by a brute force method, we write $\psi = u + iv$ with $u = Re\, \psi$, $v = Im\, \psi$. Substituting into (9.3.11) and equating real and imaginary parts gives the coupled system

$$\hbar\, \partial_t u + \frac{\hbar^2}{2m}\, \Delta v = U(x)\, v, \tag{9.3.12}$$

$$-\hbar\, \partial_t v + \frac{\hbar^2}{2m}\, \Delta u = U(x)\, u, \tag{9.3.13}$$

with $u(0,x) = u_0(x) = Re\, \psi_0(x)$ and $v(0,x) = v_0(x) = Im\, \psi_0(x)$.

We then parameterize u and v by real-valued networks $u_\theta(t,x)$ and $v_\theta(t,x)$, or equivalently, we use a complex network $\psi_\theta = u_\theta + iv_\theta$. The total loss combines PDE residuals, initial conditions, and L^2-mass conservation:

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{PDE},u}(\theta) + \mathcal{L}_{\text{PDE},v}(\theta) + \mathcal{L}_{\text{IC},u}(\theta) + \mathcal{L}_{\text{IC},v}(\theta) + \lambda_{\text{cons}}\,\mathcal{L}_{\text{cons}}(\theta) + \lambda_{\text{bd}}\,\mathcal{L}_{\text{bd}}(\theta),$$

where for interior collocation points $\{(t_i, x_j)\}$ the PDE residuals enforce equations (9.3.12)–(9.3.13)

$$\mathcal{L}_{\text{PDE},u}(\theta) = \frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M}\Big(\hbar\,\partial_t u_\theta(t_i,x_j) + \tfrac{\hbar^2}{2m}\,\Delta v_\theta(t_i,x_j) - U(x_j)\,v_\theta(t_i,x_j)\Big)^2,$$

$$\mathcal{L}_{\text{PDE},v}(\theta) = \frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M}\Big(-\hbar\,\partial_t v_\theta(t_i,x_j) + \tfrac{\hbar^2}{2m}\,\Delta u_\theta(t_i,x_j) - U(x_j)\,u_\theta(t_i,x_j)\Big)^2.$$

In implementation both Laplacians Δu_θ and Δv_θ are considered in the x variable and are obtained by automatic differentiation.

Initial conditions are enforced along initial-space samples $\{x_k^{(0)}\}$ by

$$\begin{aligned}\mathcal{L}_{\text{IC},u}(\theta) &= \frac{1}{K}\sum_{k=1}^{K}\big(u_\theta(0,x_k^{(0)}) - u_0(x_k^{(0)})\big)^2,\\ \mathcal{L}_{\text{IC},v}(\theta) &= \frac{1}{K}\sum_{k=1}^{K}\big(v_\theta(0,x_k^{(0)}) - v_0(x_k^{(0)})\big)^2.\end{aligned}$$

The mass conservation is imposed on a large enough truncated box $\Omega = [-L,L]^3$ with quadrature nodes $\{x_q\}_{q=1}^{Q}$ and normalized weights $\{w_q\}$,

$$\mathcal{L}_{\text{cons}}(\theta) = \frac{1}{N_t}\sum_{i=1}^{N_t}\left(\sum_{q=1}^{Q}\big(u_\theta(t_i,x_q)^2 + v_\theta(t_i,x_q)^2\big)\,w_q \;-\; 1\right)^2.$$

To emulate the unbounded domain, we add a homogeneous boundary penalty:

$$\mathcal{L}_{\text{bd}}(\theta) = \frac{1}{N_t N_b}\sum_{i=1}^{N_t}\sum_{b=1}^{N_b}\Big(u_\theta(t_i,x_b)^2 + v_\theta(t_i,x_b)^2\Big),$$

for boundary points $x_b \in \partial\Omega$.

We can accommodate for optional data term as follows. If observations $\psi(t_i,x_j)$ (or densities $|\psi|^2$) are available, include

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_{\text{obs}}}\sum_{(i,j)\in\mathcal{I}}\big|u_\theta(t_i,x_j) + iv_\theta(t_i,x_j) - \psi_{ij}\big|^2,$$

or, for density-only data, a term in $\big(|\psi_\theta|^2 - d_{ij}\big)^2$.

Method 2: This method uses a *propagator layer* by which the solution can be represented using an integral transform with kernel $K : \mathbb{R} \times \mathbb{R}^3 \times \mathbb{R}^2 \to \mathbb{C}$ as

$$\psi(t,x) = \int_{\mathbb{R}^3} K(t;x,y)\psi_0(y)dy, \qquad t > 0.$$

The propagator $K(t;x,y)$ is similar to a heat kernel, in the sense that:

(i) It satisfies Schrödinger's equation (9.3.11) in variable x, and

(ii) For $t \searrow 0$ it tends to the Dirac function $\delta(x-y)$.

In fact Schrödinger's equation can be reduced to a heat equation with potential as follows. Consider the change of coordinates

$$\tau = \frac{t}{i\hbar}, \qquad \xi = \frac{\sqrt{2m}}{\hbar}x.$$

Then Schrödinger's equation (9.3.11) becomes the heat equation

$$\partial_\tau \phi - \Delta_\xi \phi = V(\xi)\phi, \tag{9.3.14}$$

with initial condition $\phi(0,\xi) = \phi_0(\xi)$, where

$$\phi(\tau,\xi) = \psi\Big(-i\hbar\tau, \frac{\hbar}{\sqrt{2m}}\xi\Big), \quad \phi_0(\xi) = \psi_0\Big(\frac{\hbar}{\sqrt{2m}}\xi\Big), \quad V(\xi) = U\Big(\frac{\hbar}{\sqrt{2m}}\xi\Big).$$

Let $H(\tau;\xi,\zeta)$ be the heat kernel of the equation (9.3.14). Closed form expressions are known for a variety of potentials $V(\xi)$ (such as zero, constant, linear, quadratic, see for instance Calin et al. [16]). The Schrödinger kernel can be obtained from the heat kernel via relation

$$K(t;x,y) = H\Big(\frac{t}{i\hbar}; \frac{\sqrt{2m}}{\hbar}x, \frac{\sqrt{2m}}{\hbar}y\Big).$$

When a closed-form expression for the propagator K is unavailable, we can *hard-embed* the representation formula

$$\psi_\theta(t,x) \;=\; \int_{\mathbb{R}^3} K_\theta(t;x,y)\,\psi_0(y)\,dy,$$

where the neural kernel K_θ is parameterized so as to respect the Schrödinger structure. Since the Schrödinger kernel relates to the heat kernel, the kernel-learning techniques developed for the heat equation in Section 13.8 can be adapted here by analytic continuation. The network is trained to minimize the loss

$$\begin{aligned}\mathcal{L}(\theta) =& \frac{1}{NM}\sum_{i,j}|\psi_\theta(t_i,x_j)-\psi_{ij}|^2 + \frac{\lambda}{N}\Big(\int_{\mathbb{R}^3}|\psi_\theta(t_i,x)|^2dx - 1\Big)^2 \\ &+ \frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M}\Big(\hbar\,\partial_t\psi_\theta(t_i,x_j) + \tfrac{\hbar^2}{2m}\,\Delta\psi_\theta(t_i,x_j) - U(x_j)\,\psi_\theta(t_i,x_j)\Big)^2.\end{aligned}$$

Remark 9.3.3 The time-dependent Schrödinger equation plays in Quantum Mechanics a role analogous to Hamilton–Jacobi in Classical Mechanics. There is also a relation with the elasticity theory as follows.

If we consider $U = 0$ in the system (9.3.12)–(9.3.13) we obtain the coupled real system

$$\partial_t u \; + \; \frac{\hbar}{2m}\,\Delta v = 0, \tag{9.3.15}$$

$$-\partial_t v \; + \; \frac{\hbar}{2m}\,\Delta u = 0. \tag{9.3.16}$$

Differentiating the first relation in time and using the second equation yields

$$\partial_t^2 u + \frac{\hbar^2}{4m^2}\,\Delta^2 u = 0,$$

and symmetrically

$$\partial_t^2 v + \frac{\hbar^2}{4m^2}\,\Delta^2 v = 0.$$

These equations describe the vibration of a vibrating plate, see Courant and Hilbert [27], page 307.

Thus, in the free case the Schrödinger evolution is equivalent to *two identical vibrating plate equations* (for u and v), with biharmonic operator Δ^2 and plate-wave speed $\hbar/(2m)$. The two infinite "plates" are phase-locked as the real and imaginary parts of ψ and obey the probability conservation law

$$\int_{\mathbb{R}^3} (u^2(t,x) + v^2(t,x))dx = 1.$$

9.4 Exercises

Exercise 9.4.1 Consider the mixed initial–boundary value problem

$$\begin{cases} \partial_t u - \partial_x^2 u \;=\; t\,x(x-\pi), & (t,x) \in (0,1] \times (0,\pi), \\ u(0,x) \;=\; \sin(3x), & x \in (0,\pi), \\ u(t,0) = 0, \quad u(t,\pi) = 0, & t \in (0,1]. \end{cases}$$

(a) Solve this problem numerically using a physics-informed neural network with a *hard embedding* that enforces the initial and boundary conditions:

$$u_\theta(t,x) \;=\; \sin(3x) \;+\; t\,x(\pi - x)\,N_\theta(t,x),$$

where N_θ is a feed-forward network.

(b) Train u_θ by minimizing the interior residual

$$\mathcal{L}_{\mathrm{PDE}}(\theta) = \frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M}\Big(\partial_t u_\theta(t_i,x_j) - \partial_x^2 u_\theta(t_i,x_j) - t_i\,x_j(x_j-\pi)\Big)^2,$$

with collocation points $(t_i, x_j) \in (0,1] \times (0,\pi)$.

(c) Use automatic differentiation to compute $\partial_t u_\theta$ and $\partial_x^2 u_\theta$. A practical choice is $N = 128$ time samples and $M = 256$ spatial samples per epoch, a network with 3–4 hidden layers and 64 tanh (or swish) neurons per layer, and the Adam optimizer with learning rate 10^{-3}.

(d) After training, plot on the same axes the learned profile $x \mapsto u_\theta(t, x)$ and the reference profile at times $t \in \{0.1,\ 0.2,\ 0.5,\ 0.7,\ 1.0\}$.

Exercise 9.4.2 (Radially symmetric heat equation in the unit disk). Consider the mixed problem

$$\begin{cases} \partial_t u - \Delta u = 0, & (t, x) \in (0, T) \times B(0, 1), \\ u(0, x) = 1 - |x|^2, & x \in B(0, 1) \subset \mathbb{R}^2, \\ u(t, x) = 0, & (t, x) \in (0, T) \times \partial B(0, 1). \end{cases}$$

(a) Using the radial symmetry of the data, assume

$$u(t, x) = \phi(t, r), \qquad r = |x|,$$

and rewrite the PDE in the variables (t, r).

Hint: Recall that the radial Laplacian in $\mathbb{R}^2$ is given by

$$\Delta u = \phi_{rr}(t, r) + \frac{1}{r}\,\phi_r(t, r).$$

(b) Implement a neural network approach to approximate $\phi(t, r)$ on $(t, r) \in (0, T) \times (0, 1)$, embedding the boundary condition $\phi(t, 1) = 0$ and the initial condition $\phi(0, r) = 1 - r^2$.

(c) Plot the approximate solution $\phi(t, r)$ for the time slices $t = 0.1,\ 0.2,\ 0.5,\ 0.7$, and compare the profiles.

Exercise 9.4.3 (Heat equation on the unit circle). Consider the heat equation on the circle parameterized by $s \in (0, 2\pi)$:

$$\begin{cases} \partial_t u - \partial_s^2 u = 0, & (t, s) \in (0, T) \times (0, 2\pi), \\ u(0, s) = \sin(s), & s \in (0, 2\pi), \\ u(t, 0) = u(t, 2\pi), & t \in (0, T), \\ \partial_s u(t, 0) = \partial_s u(t, 2\pi), & t \in (0, T), \\ \partial_s^2 u(t, 0) = \partial_s^2 u(t, 2\pi), & t \in (0, T). \end{cases}$$

(a) Explain why the above boundary conditions enforce periodicity of the solution $u(t, s)$ on the unit circle.

(b) Construct a neural network approximation $u_\theta(t, s)$ to the solution, embedding the periodicity condition.

(c) Train the network on collocation points (t_i, s_j) and compare the approximate solution with the exact solution

$$u_{\text{exact}}(t, s) = e^{-t} \sin(s).$$

(d) Plot solution profiles for $t = 0.1,\ 0.2,\ 0.5,\ 1.0$ and comment on the agreement.
Hint. To enforce periodicity in s, one can represent the angular variable using the Fourier embedding

$$z(s) = (\cos s, \sin s),$$

and then feed $(t, z(s))$ into the neural network as $u_\theta(t, s) = NN(\theta; t, z(s))$. This ensures that the learned solution $u_\theta(t, s)$ is 2π-periodic in s.

Exercise 9.4.4 Consider the Cauchy problem

$$\begin{aligned} \partial_t u(t, x) - \Delta u(t, x) &= \cos t, \quad (t, x) \in (0, \infty) \times \mathbb{R}^2, \\ u(0, x) &= e^{-\|x\|^2}, \quad x \in \mathbb{R}^2. \end{aligned}$$

(a) Verify by direct computation that

$$u_{\text{exact}}(t, x) \;=\; \sin t \;+\; \frac{1}{1 + 4t} \exp\Big(-\tfrac{\|x\|^2}{1+4t}\Big)$$

is a classical solution of the problem.

(b) Using the radial symmetry of the data, show that the problem can be rewritten as a one-dimensional Cauchy problem for

$$u(t, x) = \rho(t, r), \qquad r = \|x\|.$$

(c) Approximate the solution of the reduced problem from part (b) using a neural network with the ansatz

$$\rho_\theta(t, r) = e^{-r^2} + t\, \mathrm{nn}(\theta; t, r),$$

where the initial condition is embedded into the architecture. Train the network by minimizing the PDE residual and validate the approximation against the exact solution from part (a) at the time instances

$$t = 0.1,\ 0.5,\ 0.75,\ 1.0,\ 1.5.$$

Exercise 9.4.5 Consider the Cauchy problem

$$\partial_t u(t, x) - \Delta u(t, x) = t, \quad (t, x) \in (0, \infty) \times \mathbb{R}^2,$$

with initial condition

$$u(0, x) = x_1 + 2x_2, \quad x = (x_1, x_2) \in \mathbb{R}^2.$$

(a) Show that the exact solution is

$$u(t,x) = \tfrac{1}{2}t^2 + x_1 + 2x_2.$$

(b) Construct a neural network approximation $u_\theta(t,x)$ to solve the PDE, embedding the initial condition into the ansatz. Train the network by minimizing the PDE residual

$$\mathcal{L}_{\text{PDE}}(\theta) = \frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M}\Big(\partial_t u_\theta(t_i,x_j) - \Delta u_\theta(t_i,x_j) - t_i\Big)^2,$$

and validate your numerical solution against the exact solution given in part (a).

9.5 Hyperbolic Equations

This section deals with the wave equation

$$\partial_t^2 u(t,x) \;-\; c^2\,\Delta u(t,x) \;=\; f(t,x),$$

where Δ is the spatial Laplacian and t denotes time. The function $f(t,x)$ is a space-time dependent source term.

This prototype of hyperbolic PDE models vibrations of elastic media under a force density $f(t,x)$. Typical instances include:

- $n = 1$: transverse vibrations of a string, or longitudinal vibrations of an elastic bar;
- $n = 2$: vibrations of an elastic membrane, or surface waves on a lake;
- $n = 3$: propagation of acoustic waves in air, and electromagnetic waves such as light.

Under standard isotropy assumptions, the constant $c > 0$ represents the wave propagation speed. For the sake of simplicity we shall assume $c = 1$.

Our goal is to explore neural-network-based methods for solving both the Cauchy (initial value) problem and mixed (initial-boundary value) problems for the wave equation, comparing data-driven and physics-informed approaches.

9.5.1 The Cauchy Problem

For any spatial dimension $n \geq 1$, the initial value problem for the wave equation is

$$\begin{aligned}
\partial_t^2 u(t,x) - \Delta u(t,x) &= f(t,x), \qquad (t,x) \in (0,\infty)\times\mathbb{R}^n,\\
u(0,x) &= u_0(x), \qquad x \in \mathbb{R}^n,\\
\partial_t u(0,x) &= u_1(x), \qquad x \in \mathbb{R}^n,
\end{aligned}$$

where $f : (0,\infty)\times\mathbb{R}^n \to \mathbb{R}$ and $u_0, u_1 : \mathbb{R}^n \to \mathbb{R}$ are prescribed functions. The initial conditions specify the initial displacement u_0 and initial velocity u_1 of $u(t,x)$. Under

standard regularity assumptions on f, u_0, and u_1, this Cauchy problem is well-posed and admits a unique classical solution $u \in C^2\big((0,\infty)\times\mathbb{R}^n\big)$.

Our approach depends on the spatial dimension. For $n = 1$ we use d'Alembert's formula; for $n = 2$, Poisson's solution formula; and for $n = 3$, Kirchhoff's formula—each providing a classical representation of the solution. In all cases, we solve the problem with neural networks and validate the results against the corresponding analytical formulas.

The case $n = 1$. The initial value problem is well-posed in this case and we have:

Proposition 9.5.1 (d'Alembert's formula). Let $f \in C^1([0,\infty)\times\mathbb{R})$, $u_0 \in C^2(\mathbb{R})$ and $u_1 \in C^1(\mathbb{R})$. Then the Cauchy problem

$$\begin{aligned} \partial_t^2 u(t,x) - \partial_x^2 u(t,x) &= f(t,x), \qquad (t,x)\in(0,\infty)\times\mathbb{R}, \\ u(0,x) &= u_0(x), \qquad x\in\mathbb{R}, \\ \partial_t u(0,x) &= u_1(x), \qquad x\in\mathbb{R}, \end{aligned}$$

has a unique classical solution $u \in C^2([0,\infty)\times\mathbb{R})$, given by d'Alembert's formula:

$$\begin{aligned} u(t,x) &= \frac{1}{2}(u_0(x+t)+u_0(x-t)) \\ &\quad + \frac{1}{2}\int_{x-t}^{x+t} u_1(y)\,dy \\ &\quad + \frac{1}{2}\int_0^t \int_{x-(t-s)}^{x+(t-s)} f(y,s)\,dy\,ds, \ \forall x\in\mathbb{R}, t\geq 0. \end{aligned}$$

By linearity, the solution decomposes as the sum of three parts, each solving a Cauchy problem with one nonzero datum:

$$(f,u_0,u_1) \;=\; (0,u_0,0) \;+\; (0,0,u_1) \;+\; (f,0,0).$$

Accordingly, each term in d'Alembert's formula corresponds to one of these three problems.

Consequently, when using neural networks to recover the solution $u(t,x)$, we may either (i) train a single network for the full data, or (ii) split the problem into three subproblems and sum their outputs.

Single-network approach. Embed the initial data as

$$u_\theta(t,x) \;=\; u_0(x) \;+\; t\,u_1(x) \;+\; t^2\,\mathrm{nn}(\theta;t,x),$$

and train by minimizing the PDE residual

$$\mathcal{L}(\theta) \;=\; \frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M}\big(\partial_t^2 u_\theta(t_i,x_j) \;-\; \partial_x^2 u_\theta(t_i,x_j) \;-\; f(t_i,x_j)\big)^2,$$

for $N\times M$ collocation points (t_i,x_j), with $0\leq t_i\leq T$ and $-L\leq x_j\leq L$ for a sufficiently large L.

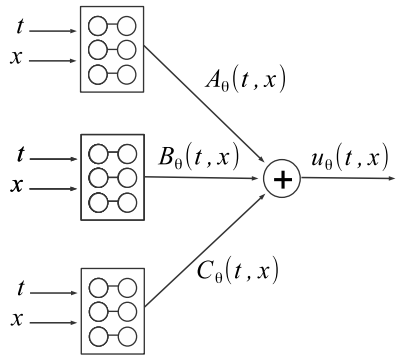


Figure 9.4: The three-network approach to solve the Cauchy problem for the wave equation.

Decomposed (three-network) approach. Use three networks, each tailored to one datum:

$$\begin{aligned} A_\theta(t,x) &= u_0(x) \;+\; t^2\,\mathrm{nn}^{(1)}(\theta;t,x),\\ B_\theta(t,x) &= t\,u_1(x) \;+\; t^2\,\mathrm{nn}^{(2)}(\theta;t,x),\\ C_\theta(t,x) &= t^2\,\mathrm{nn}^{(3)}(\theta;t,x), \end{aligned}$$

each trained with the same PDE residual loss, that enforces the wave equation. The final approximation is the sum

$$u_\theta(t,x) \;=\; A_\theta(t,x) \;+\; B_\theta(t,x) \;+\; C_\theta(t,x),$$

which recovers the solution for general data, see Fig. 9.4.

Implementation example. Consider the Cauchy problem

$$\begin{aligned} \partial_t^2 u(t,x) - \partial_x^2 u(t,x) &= t\,x, && (t,x)\in(0,\infty)\times\mathbb{R},\\ u(0,x) &= \ln\bigl(1+x^2\bigr), && x\in\mathbb{R},\\ \partial_t u(0,x) &= 2e^x, && x\in\mathbb{R}. \end{aligned}$$

By d'Alembert's formula, the classical solution is

$$u_{\mathrm{exact}}(t,x) = \tfrac{1}{2}\bigl[\ln\bigl(1+(x+t)^2\bigr) + \ln\bigl(1+(x-t)^2\bigr)\bigr] + 2e^x\sinh t \;+\; \tfrac{1}{6}\,t^3 x.$$

Using the single-network approach, we approximate u with

$$u_\theta(t,x) = \ln\bigl(1+x^2\bigr) \;+\; 2t\,e^x \;+\; t^2\,\mathrm{nn}(\theta;t,x),$$

and train by minimizing the PDE residual loss.

The collocation points are sampled on the domain $[0, 2] \times [-4, 4]$, with $N = 138$ time points and $M = 256$ spatial points. The neural network takes (t, x) as inputs and has three hidden layers of 63 neurons with tanh activations. Training is performed for 2,000 epochs using the Adam optimizer with learning rate 10^{-3}. Solution profiles at $t = 0.1, 0.5, 1.0, 1.5, 2.0$ are plotted and validated against the exact solution, see Fig. 9.5(a). We note that for larger values of t the neural network solution tends to not match exactly the exact solution. The loss function did not get to saturation, see Fig. 9.5(b). With more training more overlaps could be achieved.

Solution profiles at $t = 0.1, 0.5, 1.0, 1.5, 2.0$ are plotted and compared with the exact solution, see Fig. 9.5(a). For larger t, the neural network approximation deviates more noticeably from the exact solution. The loss has not yet plateaued, see Fig. 9.5(b), suggesting that additional training would further improve the agreement.

The code details can be found at `https://tinyurl.com/2rm7jhdj`.

The case $n = 2$. The initial value problem is well-posed in two spatial dimensions:

Proposition 9.5.2 (Poisson's formula). Let $f \in C^2\big([0,\infty) \times \mathbb{R}^2\big)$, $u_0 \in C^3(\mathbb{R}^2)$, and $u_1 \in C^2(\mathbb{R}^2)$. Then the Cauchy problem

$$\begin{aligned} \partial_t^2 u(t,x) - \Delta u(t,x) &= f(t,x), && (t,x) \in (0,\infty) \times \mathbb{R}^2, \\ u(0,x) &= u_0(x), && x \in \mathbb{R}^2, \\ \partial_t u(0,x) &= u_1(x), && x \in \mathbb{R}^2, \end{aligned}$$

admits a unique classical solution $u \in C^2\big([0,\infty) \times \mathbb{R}^2\big)$, given by Poisson's formula:

$$\begin{aligned} u(t,x) = \frac{1}{2\pi} \frac{\partial}{\partial t} \int_{B_t(x)} \frac{u_0(y)}{\sqrt{t^2 - \|x-y\|^2}}\, dy \;+\; \frac{1}{2\pi} \int_{B_t(x)} \frac{u_1(y)}{\sqrt{t^2 - \|x-y\|^2}}\, dy \\ +\; \frac{1}{2\pi} \int_0^t \int_{B_{t-s}(x)} \frac{f(s,y)}{\sqrt{(t-s)^2 - \|x-y\|^2}}\, dy\, ds, \end{aligned}$$

where $B_t(x) := \{\, y \in \mathbb{R}^2 : \ \|y - x\| < t \,\}$ denotes the open disk of radius t centered at x.

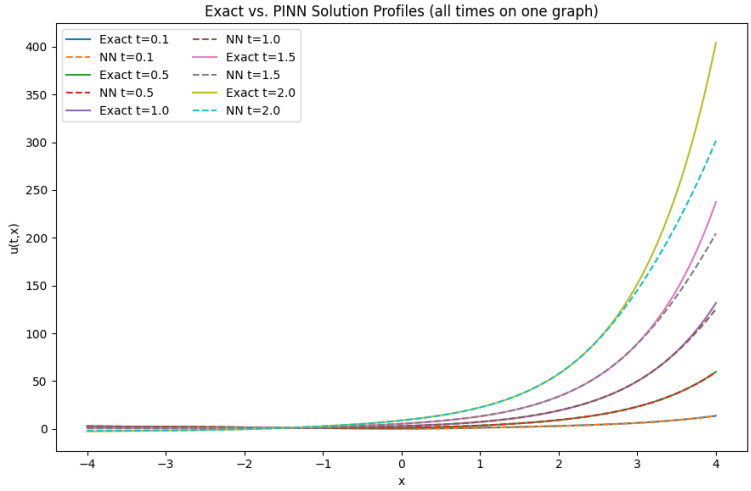
When using neural networks to approximate the solution, we take

$$u_\theta(t,x) = u_0(x) + t\, u_1(x) + t^2\, \mathrm{nn}(\theta; t, x),$$

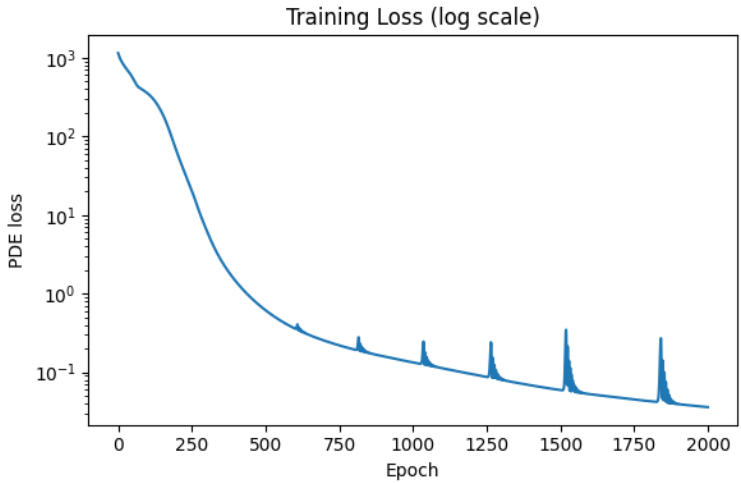
and train by minimizing the associated PDE residual loss.

Radial reduction. In the presence of rotational symmetry the problem reduces from three variables to two (time and radius). Let $r = \|x\|$ and assume $f(t,x) = F(t,r)$, $u_0(x) = w_0(r)$, $u_1(x) = w_1(r)$. Seeking a radial solution $w(t,r) = u(t,x)$ yields the Cauchy problem

$$\begin{aligned} \partial_t^2 w(t,r) \;-\; \partial_r^2 w(t,r) \;-\; \frac{1}{r}\, \partial_r w(t,r) &= F(t,r), && (t,r) \in (0,\infty) \times [0,\infty), \\ w(0,r) &= w_0(r), && r \geq 0, \\ \partial_t w(0,r) &= w_1(r), && r \geq 0. \end{aligned}$$



(a)



(b)

Figure 9.5: (a) The neural network solution $u_\theta(t, x)$ and the exact solution at instances $t \in \{0.1, 0.2, 0.5, 0.7, 1.0\}$; (b) The associated loss.

We approximate w by

$$w_\theta(t, r) = w_0(r) + t\, w_1(r) + t^2\, \mathrm{NN}(\theta; t, r),$$

so the initial data are embedded. A practical choice that enforces regularity at the origin is to factor out r^2:

$$w_\theta(t, r) = w_0(r) + t\, w_1(r) + t^2\, r^2\, \widetilde{\mathrm{NN}}(\theta; t, r),$$

which guarantees $\partial_r w_\theta(t, 0) = 0$ and avoids the $1/r$ singularity.

Given $N \times M$ collocation points $(t_i, r_j) \in [0,T] \times [0,R]$, the PDE loss reads

$$\mathcal{L}_{\mathrm{PDE}}(\theta) = \frac{1}{NM} \sum_{i=1}^{N} \sum_{j=1}^{M} \Big(\partial_t^2 w_\theta(t_i, r_j) - \partial_r^2 w_\theta(t_i, r_j) - \tfrac{1}{r_j}\, \partial_r w_\theta(t_i, r_j) - F(t_i, r_j) \Big)^2,$$

with the convention that at $r_j = 0$ we replace the radial operator by its regular limit $\partial_r^2 w(t,0) := \lim_{r \to 0} \big(\partial_r^2 w + \frac{1}{r} \partial_r w \big)$, which is automatically satisfied by the r^2-factored ansatz.

To control the artificial outer boundary $r = R$ one may add a soft *absorbing* penalty of Sommerfeld-type (see Section 9.5.2 for more details)

$$\mathcal{L}_{\mathrm{abs}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \Big(\partial_r w_\theta(t_i, R) - \partial_t w_\theta(t_i, R) \Big)^2.$$

The total loss is

$$\mathcal{L}_{\mathrm{tot}}(\theta) = \mathcal{L}_{\mathrm{PDE}}(\theta) + \lambda_{\mathrm{abs}}\, \mathcal{L}_{\mathrm{abs}}(\theta),$$

(with $\lambda_{\mathrm{abs}} \geq 0$), noting that no initial-condition term is needed because the ansatz enforces $w(0,\cdot) = w_0$ and $\partial_t w(0,\cdot) = w_1$ exactly.

If data $w(t_i, r_j)$ are available, a data misfit

$$\mathcal{L}_{\mathrm{data}} = \frac{1}{NM} \sum_{i,j} \big(w_\theta(t_i, r_j) - w_{ij} \big)^2$$

can be added with another weight.

We choose R large enough that the support of interest is contained in $[0,R]$, and sample (t_i, r_j) uniformly. Then we train a small neural network with inputs (t, r) on $\mathcal{L}_{\mathrm{tot}}$. Since an exact solution (via Poisson's formula) is available, we can validate by plotting profiles $r \mapsto w_\theta(t, r)$ against $w_{\mathrm{exact}}(t, r)$ at representative times, and monitor $\mathcal{L}_{\mathrm{PDE}}$ across epochs.

Remark 9.5.3 For any smooth radial solution one has $\partial_r w(t, 0) = 0$. The r^2 factor in the ansatz enforces this "hardly"; alternatively, one may omit the factor and add a soft penalty $\frac{1}{N} \sum_i \big(\partial_r w_\theta(t_i, 0) \big)^2$.

Implementation example Consider the Cauchy problem

$$\begin{aligned} \partial_t^2 u(t,x) - \Delta u(t,x) &= t\, \|x\|^2, && (t,x) \in (0,\infty) \times \mathbb{R}^2, \\ u(0,x) &= 1, && x \in \mathbb{R}^2, \\ \partial_t u(0,x) &= 0, && x \in \mathbb{R}^2. \end{aligned}$$

By radial symmetry, with $r = \|x\|$ and $w(t,r) = u(t,x)$, this reduces to

$$\begin{aligned} \partial_t^2 w(t,r) - \partial_r^2 w(t,r) - \frac{1}{r}\, \partial_r w(t,r) &= t\, r^2, && (t,r) \in (0,\infty) \times [0,\infty), \\ w(0,r) &= 1, && r \geq 0, \\ \partial_t w(0,r) &= 0, && r \geq 0. \end{aligned}$$

We approximate the solution with the embedded ansatz

$$w_\theta(t,r) \;=\; 1 \;+\; t^2(s_\theta(t) + r^2\,\mathrm{nn}(\theta;t,r)),$$

which enforces the initial conditions and guarantees that $\partial_r w_\theta(t,0) = 0$. Here, $s_\theta(t)$ is a neural network which depends just on t, and $\mathrm{nn}(\theta;t,r)$ is a second neural network depending on both t and r. The reason for considering the extra s-network is to increase the versatility of the w-network in matching better the solution.

The total loss combines the PDE residual with a first-order absorbing boundary penalty at $r = R$:

$$\mathcal{L}_{\mathrm{tot}}(\theta) = \mathcal{L}_{\mathrm{PDE}}(\theta) + \lambda\,\mathcal{L}_{\mathrm{abs}}(\theta), \qquad \lambda > 0,$$

where, for collocation points $(t_i, r_j) \in [0,T] \times [0,R]$,

$$\mathcal{L}_{\mathrm{PDE}}(\theta) = \frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M}\Big(\partial_t^2 w_\theta(t_i,r_j) - \partial_r^2 w_\theta(t_i,r_j) - \tfrac{1}{r_j}\,\partial_r w_\theta(t_i,r_j) - t_i\,r_j^2\Big)^2,$$

$$\mathcal{L}_{\mathrm{abs}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\Big(\partial_r w_\theta(t_i,R) + \partial_t w_\theta(t_i,R)\Big)^2,$$

with the + sign corresponding to outward-traveling signals for unit wave speed.

The exact solution. The classical solution given by Poisson's formula is

$$u_{\mathrm{exact}}(t,x) = 1 \;+\; \frac{t^3}{6}\,\|x\|^2 \;+\; \frac{t^5}{30}.$$

One can obtain this either by evaluating the Poisson integrals explicitly or by direct verification:

$$\Delta u_{\mathrm{exact}} = \frac{2}{3}\,t^3, \qquad \partial_t^2 u_{\mathrm{exact}} = t\,\|x\|^2 \;+\; \frac{2}{3}\,t^3,$$

hence $\partial_t^2 u_{\mathrm{exact}} - \Delta u_{\mathrm{exact}} = t\,\|x\|^2$, with $u_{\mathrm{exact}}(0,x) = 1$ and $\partial_t u_{\mathrm{exact}}(0,x) = 0$. In radial form, with $r = \|x\|$, we have

$$w_{\mathrm{exact}}(t,r) = 1 \;+\; \frac{t^3}{6}\,r^2 \;+\; \frac{t^5}{30},$$

which we will use to validate the neural-network solution.

NN implementation. The time-only network $s(\cdot)$ has two hidden layers with tanh activations, and the space–time network $n(t,r)$ has three hidden layers with 64 tanh neurons each. We train on $[0,T] \times [0,R] = [0,1] \times [0,2]$ using $N = 128$ time and $M = 256$ radial collocation points, for $3,000$ epochs with Adam (learning rate 10^{-3}). The absorbing penalty weight is $\lambda = 1$. Validation is performed at $t = 0.1$, 0.5, 1.0. For small r the NN and exact profiles nearly coincide; for larger r the discrepancy grows, more noticeably at larger t, see Fig. 9.6.

The code details can be found at:

`https://tinyurl.com/9xkm4vdc`

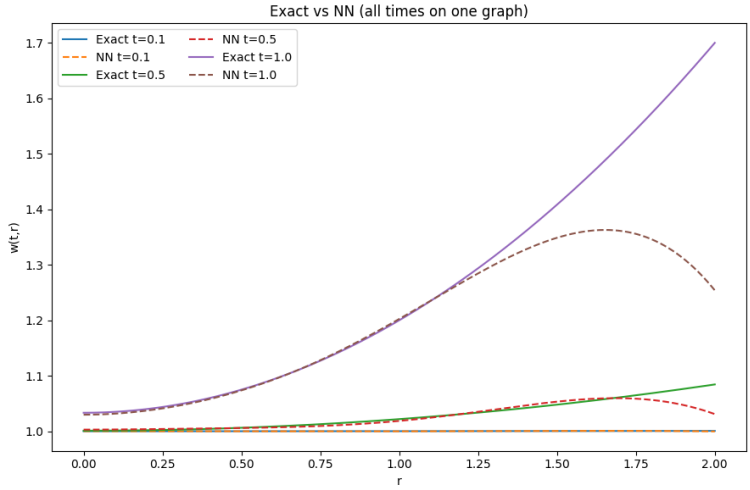


Figure 9.6: The exact solution versus the NN-solution profiles for $t = 0.1,\ 0.5,\ 1.0$.

The case $n = 3$. The initial value problem is well-posed in three spatial dimensions:

Proposition 9.5.4 (Kirchhoff's formula). Let $f \in C^2\big([0,\infty)\times\mathbb{R}^3\big)$, $u_0 \in C^3(\mathbb{R}^3)$, and $u_1 \in C^2(\mathbb{R}^3)$. Then the Cauchy problem

$$\begin{aligned} \partial_t^2 u(t,x) - \Delta u(t,x) &= f(t,x), && (t,x)\in(0,\infty)\times\mathbb{R}^3,\\ u(0,x) &= u_0(x), && x\in\mathbb{R}^3,\\ \partial_t u(0,x) &= u_1(x), && x\in\mathbb{R}^3, \end{aligned}$$

admits a unique classical solution $u \in C^2\big([0,\infty)\times\mathbb{R}^3\big)$, given by Kirchhoff's formula:

$$u(t,x) = \frac{1}{4\pi}\,\frac{\partial}{\partial t}\Big(\frac{1}{t}\int_{\partial B_t(x)} u_0(y)\,d\sigma_y\Big) \;+\; \frac{1}{4\pi t}\int_{\partial B_t(x)} u_1(y)\,d\sigma_y \\ +\; \frac{1}{4\pi}\int_{B_t(x)} \frac{f(t-\|x-y\|,y)}{\|x-y\|}\,dy,$$

where $\partial B_t(x) := \{\, y\in\mathbb{R}^3 :\ \|y-x\| = t \,\}$ denotes the sphere of radius t centered at x.

To solve the problem, we also adopt a neural-network ansatz that *hard-embeds* the initial conditions,

$$u_\theta(t,x) = u_0(x) + t\,u_1(x) + t^2\,\mathrm{nn}(\theta;t,x),$$

and train by minimizing the associated PDE residual loss.

In the presence of rotational symmetry, the problem reduces from four variables (time and three spatial coordinates) to two (time and radius). Let $r = \|x\|$ and assume $f(t,x) = F(t,r)$, $u_0(x) = w_0(r)$, and $u_1(x) = w_1(r)$. Seeking a radial solution $w(t,r) =$

$u(t,x)$ yields the Cauchy problem

$$\begin{aligned} \partial_t^2 w(t,r) \;-\; \partial_r^2 w(t,r) \;-\; \frac{2}{r}\,\partial_r w(t,r) = F(t,r), \qquad & (t,r)\in(0,\infty)\times[0,\infty), \\ w(0,r) = w_0(r), \qquad & r\ge 0, \\ \partial_t w(0,r) = w_1(r), \qquad & r\ge 0, \end{aligned}$$

where the radial Laplacian in $\mathbb{R}^3$ appears as $\Delta_r w = \partial_r^2 w + \frac{2}{r}\partial_r w$.

The training procedure mirrors the $n=2$ case: use the embedded ansatz (optionally with an r^2 factor to enforce $\partial_r w(t,0)=0$), minimize the PDE residual on $(t,r)\in[0,T]\times[0,R]$, and include a first-order absorbing boundary penalty at $r=R$ to suppress artificial reflections. Further worked examples are deferred to the exercises section.

9.5.2 Absorbing Boundary Condition

In computations on a truncated radial domain $[0,R]$, outgoing waves reflect from the artificial boundary unless a nonreflecting boundary condition is imposed. To suppress spurious echoes and emulate the infinite exterior, we add a Sommerfeld-type absorbing penalty at $r=R$ to the training objective of the neural approximation w_θ.

Sommerfeld radiation condition The Sommerfeld radiation condition specifies the asymptotic behavior of *outgoing* waves at infinity and is central to uniqueness, well-posedness, and nonreflecting truncations of unbounded domains.

For the wave equation $\partial_t^2 w - c^2\Delta w = 0$ in $\mathbb{R}^n$, an outgoing solution has the large-r form

$$w(t,r) \;\sim\; \frac{F\big(t-\frac{r}{c}\big)}{r^{(n-1)/2}} \quad (r\to\infty),$$

and one has

$$\Big(\partial_r + \tfrac{1}{c}\partial_t\Big)w(t,r) = -\,\frac{n-1}{2}\,\frac{F\big(t-\frac{r}{c}\big)}{r^{(n+1)/2}} \;\xrightarrow{\;r\to\infty\;}\; 0.$$

Thus, the *Sommerfeld radiation condition* is

$$(\partial_r + \tfrac{1}{c}\partial_t)w = 0 \quad \text{as } r\to\infty \tag{9.5.17}$$

(up to lower-order $1/r$ terms when $n\ge 2$). On a finite boundary $r=R$ this yields a first-order absorbing boundary condition (ABC).

We enforce the ABC weakly by adding to the loss

$$\mathcal{L}_{\text{abs}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\Big(\partial_r w_\theta(t_i,R) \;+\; \tfrac{1}{c}\,\partial_t w_\theta(t_i,R)\Big)^2,$$

with collocation times $\{t_i\}_{i=1}^N$. The sign "+" corresponds to outward-traveling signals of the form $F(t-r/c)$. (If an inward-wave convention is used, flip the sign.)

The motivation of using the ABC is to eliminate artificial reflections at $r=R$, improving accuracy near the boundary and allowing smaller R. Then we use

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{PDE}}(\theta) + \lambda_{\text{abs}}\,\mathcal{L}_{\text{abs}}(\theta),$$

with $\lambda_{\text{abs}}>0$ tuning the strength of absorption.

9.5.3 The Mixed Problem

Let $\Omega \subset \mathbb{R}^n$ be a bounded open set with C^1-boundary $\partial\Omega$. For $T > 0$ define the space–time cylinder $Q_T = \Omega \times (0,T)$ and its lateral boundary $S_T = \partial\Omega \times (0,T)$. Given the functions

$$f : Q_T \to \mathbb{R}, \qquad u_0, u_1 : \Omega \to \mathbb{R}, \qquad g : S_T \to \mathbb{R},$$

the *mixed (initial-boundary value)* problem with Dirichlet condition for the wave equation is

$$\begin{aligned} \partial_t^2 u - \Delta u &= f && \text{in } Q_T, \\ u(0,x) = u_0(x), \quad \partial_t u(0,x) &= u_1(x) && \text{for } x \in \Omega, \\ u &= g && \text{on } S_T. \end{aligned}$$

A *classical solution* of the mixed problem is a function

$$u \in C^2(Q_T) \cap C(\overline{Q_T}), \qquad \partial_t u \in C(\overline{Q_T}),$$

that satisfies the PDE pointwise in Q_T together with the initial conditions on $\{0\} \times \Omega$ and the boundary condition on S_T.

If a classical solution exists, the following compatibility conditions at $t = 0$ must hold:

$$u_0|_{\partial\Omega} = g(0,\cdot), \qquad u_1|_{\partial\Omega} = \partial_t g(0,\cdot).$$

The mixed problem in one space dimension

We analyze the initial-boundary value problem for a vibrating string of length L, which is modeled by

$$\begin{cases} \partial_t^2 u(t,x) - \partial_x^2 u(t,x) = f(t,x), & (t,x) \in (0,T) \times (0,L), \\ u(0,x) = u_0(x), \quad \partial_t u(0,x) = u_1(x), & x \in [0,L], \\ u(t,0) = h_1(t), \quad u(t,L) = h_2(t), & t \in [0,T]. \end{cases}$$

Reduction to homogeneous boundary data Set $u = v + w$, where we choose a simple lifting w that matches the boundary values,

$$w(t,x) = \Big(1 - \frac{x}{L}\Big) h_1(t) + \frac{x}{L}\, h_2(t).$$

Then $w(t,0) = h_1(t)$ and $w(t,L) = h_2(t)$, and $v := u - w$ satisfies

$$\begin{cases} \partial_t^2 v - \partial_x^2 v = F, & (t,x) \in (0,T) \times (0,L), \\ v(0,x) = f(x), & x \in [0,L], \\ \partial_t v(0,x) = g(x), & x \in [0,L], \\ v(t,0) = v(t,L) = 0, & t \in [0,T], \end{cases}$$

with modified source

$$F(t,x) := f(t,x) - \Big(1 - \frac{x}{L}\Big) h_1''(t) - \frac{x}{L}\, h_2''(t),$$

and modified initial conditions

$$\begin{aligned} f(x) &:= u_0(x) - w(0,x) \\ g(x) &:= u_1(x) - \partial_t w(0,x). \end{aligned}$$

The reflection method We shall perform an odd reflection and a periodic extension, which will transform the previous problem into a Cauchy problem on the real line, solvable using d'Alembert's formula. To this end, we extend the data *oddly* across $x = 0$ and $x = L$ and then $2L$-periodically as follows.

First, we assume the following regularity conditions: $F \in C^1([0,T] \times [0,L])$, $f \in C^2([0,L])$, $g \in C^1([0,L])$, and

$$F(t,0) = F(t,L) = 0, \quad \forall\, t \in [0,T],$$

$$f(0) = f(L) = f''(0) = f''(L) = 0,$$

$$g(0) = g(L) = 0.$$

These conditions are required for obtaining a classical solution after extension to the entire real line.

The odd extension across $x = 0$ for f is

$$f^*(x) = \begin{cases} f(x), & x \in [0,L], \\ -f(-x), & x \in [-L,0]. \end{cases}$$

Then we extend, with period $2L$, the function f^* to the entire line and obtain $f^* \in C^2(\mathbb{R})$. Similarly, the extensions F^* and g^* of F and g satisfy $2L$-periodicity and regularity, with $F^* \in C^1([0,T]\times\mathbb{R})$ and $g^* \in C^1(\mathbb{R})$. Under these extension hypotheses, v solves the forced wave equation on $\mathbb{R}$ with $2L$-periodic data. One may then apply d'Alembert's formula on $\mathbb{R}$ to obtain

$$\begin{aligned} v^*(t,x) \;=\; & \frac{1}{2}\big(f^*(x+t) + f^*(x-t)\big) + \frac{1}{2}\int_{x-t}^{x+t} g^*(y)\, dy \\ & + \frac{1}{2}\int_0^t \int_{x-(t-s)}^{x+(t-s)} F^*(s,y)\, dy\, ds, \quad x \in \mathbb{R}. \end{aligned}$$

Taking the restriction

$$v(t,x) = v^*(t,x)\big|_{[0,T]\times[0,L]}$$

we obtain the solution to the homogeneous boundary-condition mixed problem. Finally, we recover $u = v + w$ to restore the original boundary values. This procedure yields a classical solution $u(t,x)$.

Implementation example We consider the mixed homogeneous mixed problem:

$$\begin{cases} \partial_t^2 v - \partial_x^2 v = \sin(\pi x)\cos(\pi t), & (t,x) \in (0,\infty)\times(0,1), \\ v(0,x) = -\frac{1}{2\pi^2}\sin(\pi x), & x \in [0,1], \\ \partial_t v(0,x) = \pi \sin(\pi x), & x \in [0,1], \\ v(t,0) = v(t,1) = 0, & t \in [0,1]. \end{cases}$$

Since all the hypotheses of the reflection method are satisfied, then d'Alembert's formula provides the following classical solution

$$v(t,x) = \Big[-\frac{1}{2\pi^2}\cos(\pi t) \ + \ \sin(\pi t) \ + \ \frac{t}{2\pi}\sin(\pi t)\Big]\sin(\pi x), \quad (t,x) \in [0,\infty)\times[0,1].$$

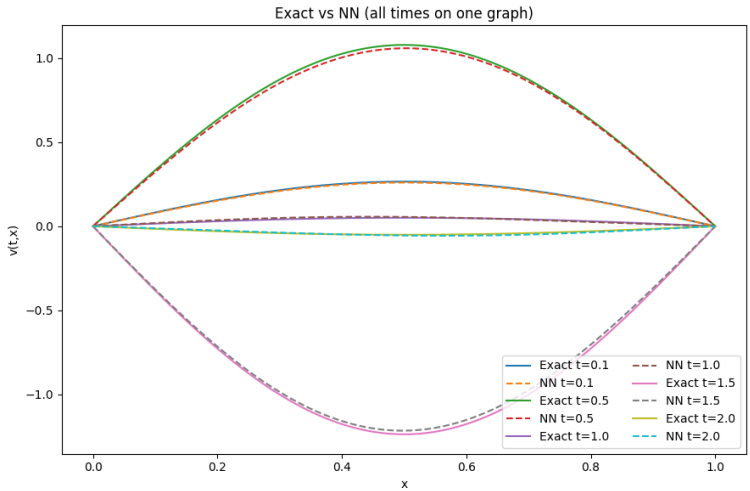


Figure 9.7: The network solution $v_\theta(t,x)$ and the exact solution are plotted together for $t = 0.1,\ 0.5,\ 1.0,\ 1.5,\ 2.0$. Their match is very close.

To solve this problem with a neural network, we use the hard-embedded ansatz

$$\begin{aligned} v_\theta(t,x) &= v(0,x) + t\,\partial_t v(0,x) + t^2 x(1-x)\,\mathrm{nn}(\theta;t,x) \\ &= -\frac{1}{2\pi^2}\sin(\pi x) + \pi t\,\sin(\pi x) + t^2 x(1-x)\,\mathrm{nn}(\theta;t,x). \end{aligned}$$

The network has three hidden layers with 64 tanh-activated neurons each, and is trained for 3,000 epochs using Adam with learning rate 10^{-3}. We sample $N \times M = 128 \times 256$ interior collocation points $(t_i, x_j) \in (0,2)\times(0,1)$. The NN and exact solutions are plotted together for $t = 0.1,\ 0.5,\ 1.0,\ 1.5,\ 2.0$, and the agreement is very close, see Fig. 9.7.
The code implementation details are available at

`https://tinyurl.com/mruxvz2z`

Further reading Further details on these equations can be found in the classical book by Vladimirov [97], which treats the standard equations of Physics and keeps a close connection to physical models and explicit constructions. Guenther and Lee [43] also provide an excellent bridge between PDE theory and applied Mathematical Physics. For a more theoretical perspective, Mikhlin's monograph [72] offers a rigorous, analysis-first treatment of linear PDEs with a strong emphasis on functional-analytic tools. Finally, Tikhonov and Samarskii [92] devote substantial attention to qualitative properties and numerical methods. The theory of second-order elliptic partial differential equations, covering existence, uniqueness, regularity, and maximum principles for both linear and nonlinear cases can be found in the classical text of Gilbarg and Trudinger [38].

9.5.4 Determining an Electromagnetic Field

Let $(E(t,x), H(t,x))$ be an electromagnetic field, where

$$E(t,x) = (E^1(t,x), E^2(t,x), E^3(t,x)) \text{ and } H(t,x) = (H^1(t,x), H^2(t,x), H^3(t,x))$$

denote the electric and magnetic fields, respectively. Assume we have n sensors located at spatial points $\{x_1, \dots, x_n\}$ in $\mathbb{R}^3$, which record the field components at m time instances $\{t_1, \dots, t_m\}$. Set

$$E_{ij} = E(t_i, x_j), \qquad H_{ij} = H(t_i, x_j),$$

for $i = 1, \dots, m$ and $j = 1, \dots, n$.

Problem *Given the measurements $\{E_{ij}\}$ and $\{H_{ij}\}$, determine the time evolution of the electromagnetic field (E, H).*

1. A data-driven approach In this approach we learn the fields directly from data by training two neural networks with three-dimensional outputs,

$$E_\theta(t,x) \in \mathbb{R}^3, \qquad H_\theta(t,x) \in \mathbb{R}^3,$$

simultaneously to minimize the empirical data loss

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{mn} \sum_{i=1}^{m} \sum_{j=1}^{n} \big\| E_\theta(t_i, x_j) - E_{ij} \big\|^2 + \frac{1}{mn} \sum_{i=1}^{m} \sum_{j=1}^{n} \big\| H_\theta(t_i, x_j) - H_{ij} \big\|^2.$$

This produces an approximation (E_θ, H_θ) to the true fields on the sampled space–time grid, with interpolation to unseen (t,x) supplied by the networks' inductive bias. However, this is only an approximate approach; we can improve it substantially by enforcing Maxwell's equations, which the fields must satisfy.

Maxwell's equations In the absence of sources (no charges or currents), the electromagnetic fields satisfy

$$\operatorname{curl} H = \frac{1}{c} \frac{\partial E}{\partial t}, \qquad \operatorname{curl} E = -\frac{1}{c} \frac{\partial H}{\partial t},$$
$$\operatorname{div} H = 0, \qquad \operatorname{div} E = 0,$$

where c is the speed of light in vacuum.

Rather than imposing these as four soft constraints, we refine the model by observing that both E and H obey homogeneous wave equations:

$$\Box E = 0, \qquad \Box H = 0, \tag{9.5.18}$$

with the d'Alembert operator $\Box := \partial_t^2 - c^2\Delta$ on $\mathbb{R}^3$.

This follows from the vector identity

$$\operatorname{curl}\operatorname{curl} H = \operatorname{grad}\operatorname{div} H - \Delta H, \tag{9.5.19}$$

together with $\operatorname{div} H = 0$. Applying curl to the first Maxwell equation and then using the second gives

$$\operatorname{curl}\operatorname{curl} H = \frac{1}{c}\frac{\partial}{\partial t}\big(\operatorname{curl} E\big) = -\frac{1}{c^2}\frac{\partial^2 H}{\partial t^2}.$$

Since $\operatorname{div} H = 0$, the left-hand side of (9.5.19) reduces to $-\Delta H$, and hence

$$\partial_t^2 H - c^2\Delta H = 0.$$

The same argument, starting from $\operatorname{curl} E = -\frac{1}{c}\partial_t H$ and using $\operatorname{div} E = 0$, yields $\Box E = 0$.

2. Using the wave equation as a soft constraint A more accurate way to predict the electromagnetic field $(E(t,x), H(t,x))$ from measurements is to exploit that both E and H satisfy the wave equations. Adding these as soft constraints to the data loss regularizes the reconstruction. The total loss is

$$\mathcal{L}(\theta) \;=\; \mathcal{L}_{\text{data}}(\theta) \;+\; \lambda\,\mathcal{L}_{\text{wave}}(\theta),$$

where

$$\mathcal{L}_{\text{wave}}(\theta) \;=\; \frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}\big(\Box E_\theta(t_i,x_j)\big)^2 \;+\; \frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}\big(\Box H_\theta(t_i,x_j)\big)^2.$$

Here $\lambda > 0$ balances data fidelity and the wave-equation regularization.

3. Using hard embedding We learn two scalar potentials φ and ψ and two vector potentials A and B with neural networks. In the source-free case, both E and H are divergence-free, hence (on a simply connected domain) they are purely rotational:

$$H = \operatorname{curl} A, \qquad E = \operatorname{curl} B,$$

for vector fields A, B on $\mathbb{R}^3$.
From Maxwell's equations we obtain

$$c\,\operatorname{curl} H = \partial_t E = \partial_t(\operatorname{curl} B) = \operatorname{curl}(\partial_t B),$$

hence

$$\operatorname{curl}\big(cH - \partial_t B\big) = 0.$$

Therefore there exists a space-dependent scalar potential φ such that

$$cH - \partial_t B = \nabla\varphi \quad\Longrightarrow\quad H = \tfrac{1}{c}\big(\nabla\varphi + \partial_t B\big).$$

Similarly, using $H = \operatorname{curl} A$ in $\operatorname{curl} E = -\frac{1}{c}\partial_t H$ gives

$$E = \tfrac{1}{c}\big(\nabla\psi - \partial_t A\big),$$

for some space-dependent scalar potential ψ.

We parameterize the potentials with neural networks:

$$\varphi_\theta(t,x),\ \psi_\theta(t,x) \in \mathbb{R}, \qquad A_\theta(t,x),\ B_\theta(t,x) \in \mathbb{R}^3,$$

and *hard-embed* the electromagnetic fields via

$$H_\theta(t,x) = \tfrac{1}{c}\big(\nabla\varphi_\theta(t,x) + \partial_t B_\theta(t,x)\big), \qquad E_\theta(t,x) = \tfrac{1}{c}\big(\nabla\psi_\theta(t,x) - \partial_t A_\theta(t,x)\big).$$

Given samples $\{(t_i, x_j)\}$ with measurements H_{ij} and E_{ij}, we enforce the embeddings with the loss

$$\begin{aligned}\mathcal{L}_{\text{embed}}(\theta) = \frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}\big\|\nabla\varphi_\theta(t_i,x_j) + \partial_t B_\theta(t_i,x_j) - c\,H_{ij}\big\|^2 \\ + \frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}\big\|\nabla\psi_\theta(t_i,x_j) - \partial_t A_\theta(t_i,x_j) - cE_{ij}\big\|^2.\end{aligned}$$

The total loss combines data fidelity with the hard-embedding term:

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \mathcal{L}_{\text{embed}}(\theta).$$

Orthogonality prior In a source-free homogeneous medium, a plane electromagnetic wave satisfies $E(t,x) \perp H(t,x)$, i.e., $E \cdot H = 0$. To encourage this behavior, we add a soft penalty

$$\begin{aligned}\mathcal{L}_\perp(\theta) &= \frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}\big(E(t_i,x_j)\cdot H(t_i,x_j)\big)^2 \\ &= \frac{1}{mn}\sum_{i,j}\big(E^1H^1 + E^2H^2 + E^3H^3\big)^2(t_i,x_j),\end{aligned}$$

which biases the learned fields toward orthogonality in the plane-wave regime.

9.6 Exercises

Example 9.6.1 Consider the Cauchy problem

$$\begin{cases}\partial_t^2 u - \partial_x^2 u = 0, & (t,x) \in (0,\infty)\times\mathbb{R},\\ u(0,x) = 0, & x \in \mathbb{R},\\ \partial_t u(0,x) = \sin x, & x \in \mathbb{R}.\end{cases}$$

(a) Show that the exact solution is $u_{\text{exact}}(t,x) = \sin t\,\sin x$.

(b) Approximate the solution with a neural network by embedding the initial data, e.g.

$$u_\theta(t,x) = t\,\sin x + t^2\,\mathrm{nn}_\theta(t,x),$$

and train by minimizing the PDE residual $\left(\partial_t^2 u_\theta - \partial_x^2 u_\theta\right)^2$ on a suitable space–time collocation grid. Validate your approximation by comparing $u_\theta(t,\cdot)$ with $u_{\text{exact}}(t,\cdot)$ for several values of t (e.g., $t = 0.1,\ 0.5,\ 1.0,\ 1.5,\ 2.0$).

Example 9.6.2 Consider the Cauchy problem on $\mathbb{R}^2$:

$$\begin{cases} \partial_t^2 u(t,x) - \Delta u(t,x) = t\,x_1 x_2, & (t,x) \in (0,\infty)\times\mathbb{R}^2, \\ u(0,x) = x_1^2 - x_2^2, & x \in \mathbb{R}^2, \\ \partial_t u(0,x) = x_1 x_2, & x \in \mathbb{R}^2. \end{cases}$$

(a) Show that the problem admits a unique classical solution and verify that

$$u_{\text{exact}}(t,x) = x_1^2 - x_2^2 \;+\; t\,x_1 x_2 \;+\; \frac{t^3}{6}\,x_1 x_2, \qquad x = (x_1,x_2) \in \mathbb{R}^2,\ \ t \geq 0,$$

indeed satisfies the PDE and the initial conditions.

(b) Approximate the solution with a neural network using a hard-embedded ansatz

$$u_\theta(t,x) = \left(x_1^2 - x_2^2\right) + t\,x_1 x_2 + t^2\,\mathrm{nn}_\theta(t,x),$$

and train by minimizing the PDE residual

$$\left(\partial_t^2 u_\theta - \Delta u_\theta - t\,x_1 x_2\right)^2$$

on a suitable space–time collocation grid (e.g., $t \in [0,2]$, $x \in [-L,L]^2$). Validate your approximation by comparing $u_\theta(t,\cdot)$ with $u_{\text{exact}}(t,\cdot)$ at several times (e.g., $t = 0.1,\ 0.5,\ 1.0,\ 1.5,\ 2.0$), and report the training loss across epochs.

Example 9.6.3 Consider the Cauchy problem on $\mathbb{R}^3$:

$$\begin{cases} \partial_t^2 u(t,x) - \Delta u(t,x) = 2\,x_1 x_2 x_3 + t, & (t,x) \in (0,\infty)\times\mathbb{R}^3, \\ u(0,x) = x_1^2 + x_2^2 - 2x_3^2, & x \in \mathbb{R}^3, \\ \partial_t u(0,x) = x_1 + x_2^2, & x \in \mathbb{R}^3. \end{cases}$$

(a) Show that the problem admits a classical solution and verify that

$$u_{\text{exact}}(t,x) = x_1^2 + x_2^2 - 2x_3^2 \;+\; t\,(x_1 + x_2^2) \;+\; t^2\,x_1 x_2 x_3 \;+\; \tfrac{1}{2}\,t^3$$

indeed satisfies the PDE and the initial conditions.

(b) Recover the solution using a neural network with a hard-embedded ansatz

$$u_\theta(t,x) = \left(x_1^2 + x_2^2 - 2x_3^2\right) \;+\; t\,(x_1 + x_2^2) \;+\; t^2\,\mathrm{nn}_\theta(t,x),$$

and train by minimizing the PDE residual

$$\left(\partial_t^2 u_\theta - \Delta u_\theta - (2x_1 x_2 x_3 + t)\right)^2$$

on a space–time collocation grid (e.g., $t \in [0,2]$, $x \in [-L,L]^3$ for some $L > 0$). Validate your approximation by comparing $u_\theta(t,\cdot)$ with $u_{\text{exact}}(t,\cdot)$ for several times (e.g., $t = 0.1,\ 0.5,\ 1.0,\ 1.5,\ 2.0$), and plot the training loss across epochs.

Example 9.6.4 Consider the wave equation in $\mathbb{R}^3$ with *radial* initial data:

$$\begin{cases} \partial_t^2 u(t,x) - c^2\,\Delta u(t,x) = 0, & (t,x) \in (0,\infty)\times\mathbb{R}^3, \\ u(0,x) = w_0(\|x\|), \qquad \partial_t u(0,x) = w_1(\|x\|), \end{cases}$$

where $w_0, w_1 : [0,\infty) \to \mathbb{R}$ are given.

(a) Show that the solution remains radial, $u(t,x) = w(t,r)$ with $r = \|x\|$, and that w solves

$$\partial_t^2 w(t,r) - c^2\left(\partial_r^2 w(t,r) + \frac{2}{r}\,\partial_r w(t,r)\right) = 0, \qquad r>0,\ t>0,$$

with initial data $w(0,r) = w_0(r)$, $\partial_t w(0,r) = w_1(r)$ and regularity at the origin $\partial_r w(t,0) = 0$.

(b) Define $v(t,r) := r\,w(t,r)$. Prove that v satisfies the 1D wave equation on the half-line,

$$\partial_t^2 v(t,r) - c^2\,\partial_r^2 v(t,r) = 0, \qquad r>0,\ t>0,$$

with $v(0,r) = r\,w_0(r)$, $\partial_t v(0,r) = r\,w_1(r)$ and boundary condition $v(t,0) = 0$.

(c) By odd extension of the initial data to $r<0$, deduce the d'Alembert formula

$$v(t,r) = \tfrac{1}{2}V_0(r-ct) + \tfrac{1}{2}V_0(r+ct) + \frac{1}{2c}\int_{r-ct}^{r+ct} V_1(s)\,ds,$$

where $V_0(r) = r\,w_0(|r|)\,\mathrm{sgn}(r)$ and $V_1(r) = r\,w_1(|r|)\,\mathrm{sgn}(r)$. Conclude that

$$w(t,r) = \frac{1}{2r}\Big[(r-ct)\,w_0(|r-ct|) + (r+ct)\,w_0(r+ct)\Big] + \frac{1}{2cr}\int_{|r-ct|}^{r+ct} s\,w_1(s)\,ds.$$

(d) We consider the following explicit profile: take $w_1 \equiv 0$ and $w_0(r) = e^{-r^2}$. Use the formula in (b) to write $w(t,r)$ explicitly and verify that it satisfies the radial PDE and the origin regularity. Implement a PINN in variables $(t,r) \in [0,T]\times[0,R]$ with the ansatz $w_\theta(t,r) = w_0(r) + t\,w_1(r) + t^2 r^2\,\mathrm{nn}_\theta(t,r)$ and compare w_θ to the explicit w for several time instances t.

Example 9.6.5 Use a neural network to solve the mixed problem

$$\begin{cases} \partial_t^2 u - \partial_x^2 u = x^2 - t^2, & (t,x) \in (0,\infty)\times(0,1), \\ u(0,x) = 0, & x \in (0,1), \\ \partial_t u(0,x) = \pi\sin(\pi x), & x \in (0,1), \\ u(t,0) = 0, & t>0, \\ u(t,1) = \tfrac{1}{2}t^2, & t>0. \end{cases}$$

(a) Show that the exact classical solution is

$$u_{\text{exact}}(t,x) = \tfrac{1}{2}\,t^2x^2 \;+\; \sin(\pi x)\,\sin(\pi t).$$

(b) Construct a PINN with a hard-embedded ansatz that enforces the boundary and initial conditions that guarantees $u_\theta(0,x) = 0$, $\partial_t u_\theta(0,x) = \pi \sin(\pi x)$, $u_\theta(t,0) = 0$, and $u_\theta(t,1) = \frac{1}{2}t^2$. Train by minimizing the interior PDE residual

$$\left(\partial_t^2 u_\theta - \partial_x^2 u_\theta - (x^2 - t^2)\right)^2$$

on a collocation grid in $(0,T] \times (0,1)$ (e.g., $T = 2$). Validate by plotting $u_\theta(t,\cdot)$ against $u_{\text{exact}}(t,\cdot)$ for several times (e.g., $t = 0.25$, 0.5, 1, 1.5, 2), and report the training loss across epochs.

This page intentionally left blank

Part III

Inverse Problems: Inferring Physics from Data

This page intentionally left blank

Chapter 10

Introduction to Inverse Problems in Mechanics

10.1 Why Deep Learning in Inverse Problems?

While direct problems in Physics can often be solved using a wide range of traditional numerical methods, it is in the realm of inverse problems where deep learning proves to be especially valuable.

Inverse problems are central to scientific discovery, especially in Physics, where one seeks to infer underlying laws or governing equations from observed data. A typical inverse problem involves learning a differential equation or a conservation law–often expressed in terms of position and momentum–for a dynamical system that has been empirically observed across a large number of trials. Deep Learning offers powerful tools for solving such problems by approximating complex functional relationships directly from data.

In favorable cases, the structure of the physical law is known, but certain parameters remain undetermined. This scenario, often referred to as calibration by Deep Learning, involves training neural networks to estimate the unknown parameters that best align the model with empirical observations. This process can be seen as a modern, data-driven variant of the statistical bootstrapping method.

In more challenging situations, the inverse problem may be ill-posed meaning, for instance, that a unique solution does not exist. In such cases, Deep Learning tends to converge to the solution that best matches the data in a least-squares or best-fit sense, effectively regularizing the ill-posedness by favoring data-consistent approximations.

In more advanced scenarios, inverse problems in Physics involve using Deep Learning to discover the underlying Lagrangians or Hamiltonians that govern the system's dynamics.

10.2 Learning a Velocity Field from Position Data

Reconstructing trajectories from a known velocity field is a straightforward integration problem in ODE theory. Here, we tackle the inverse problem: inferring the velocity

field from position measurements. For example, if we photograph snowflakes at various times, can we use those snapshots to recover the wind velocity field?

Problem *Given measurements* $\{(t_j, x_j)\}_{j=1}^N \subset \mathbb{R} \times \mathbb{R}^n$*, find a function*

$$V : \mathbb{R} \times \mathbb{R}^n \to \mathbb{R}^n$$

so that the solution of

$$\dot{x}(t) = V\big(t, x(t)\big)$$

matches the data, i.e., we have $x(t_j) = x_j, \quad j = 1, \dots, N.$

To solve this, we define a feed-forward neural network

$$\mathrm{NN}(t, y; \theta) = \big(x_\theta(t),\, V_\theta(t, y)\big),$$

and train it by minimizing the composite loss

$$\mathcal{L}(\theta) = \sum_{j=1}^{M} \|\dot{x}_\theta(u_j) - V_\theta\big(u_j, x_\theta(u_j)\big)\|^2 \;+\; \lambda \sum_{k=1}^{N} \|x_\theta(t_k) - x_k\|^2,$$

where $\{u_j\}_{j=1}^M$ are collocation points enforcing the ODE, and $\lambda > 0$ balances the ODE and data terms. The optimal parameter $\theta^* = \arg\min_\theta \mathcal{L}(\theta)$ yields the learned velocity field $V_{\theta^*}(t, y)$. As a consequence, the neural network learns also the trajectory, $x_{\theta^*}(t)$.

10.2.1 Implementation

Discovery of equation $\dot{x} = x^2 + 1$ **from data**
(*i*) *Data collection.* We shall start by generating our own synthetic data by collecting $N = 30$ measurements $\{(t_j, x_j)\}$ from the solution $x(t) = \tan(x + \pi/4)$ of the equation

$$\dot{x}(t) = x^2 + 1, \quad x(0) = 1, \quad t \in [0, 0.75].$$

These data are collected and graphed by the following snippet:

```
N = 30
t_data = np.linspace(0, 0.75, N, dtype=np.float32).reshape(-1,1)
x_data = np.tan(t_data + np.pi/4)

plt.figure(figsize=(6,4))
plt.scatter(t_data, x_data, c='black', s=20, label='Data $(t_j, x_j)$')
plt.xlabel('t')
plt.ylabel('x(t)')
plt.title('Data from $dx/dt = x^2 + 1$')
plt.grid(True)
plt.legend()
plt.show()
```

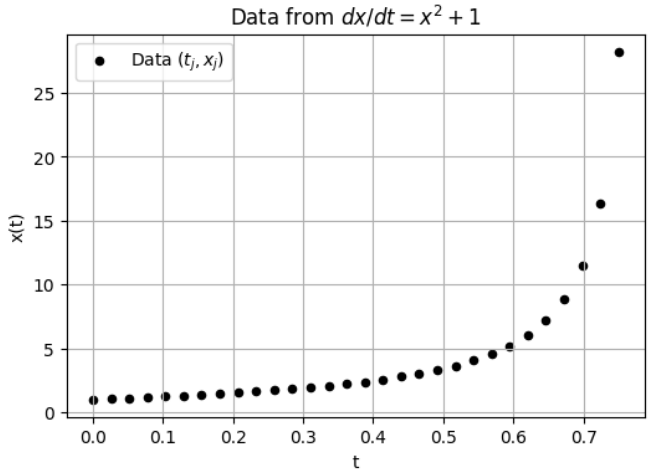


Figure 10.1: 30 sample equidistant points from the graph of the function $x(t) = \tan(x + \pi/4)$.

The graph of the scatter plot is given by Fig. 10.1. We note that due to the curvature of the solution the points on the right end look more "distant"; this effect will negatively impact the learning later. To enforce the differential equation we use 200 points equidistantly picked in the interval $[0, 0.75]$. The right endpoint was chosen such way that we are not reaching the solution singularity, $0.75 < \pi/2$:

```
M = 200
t_colloc = np.linspace(0, 0.75, M, dtype=np.float32).reshape(-1,1)
```

(*ii*) *Equation discovery.* In order to discover the right side $f(x) = x^2 + 1$ of the equation, we employ two neural networks, which simultaneously learn the solution $x(t) = \tan(x + \pi/4)$ and the function $f(x)$. Each network contains two 64-neuron hidden layers with `swish` activation functions. It looks that `swish` function provides better learning than `relu` or `tanh` activation functions:

```
#  Build model_sol: t -> x_pred
def build_model_sol():
    inp = layers.Input(shape=(1,))
    x = layers.Dense(64, activation='swish')(inp)
    x = layers.Dense(64, activation='swish')(x)
    out = layers.Dense(1)(x)
    return Model(inp, out, name='model_sol')

#  Build model_f: x_pred -> f_pred
def build_model_f():
    inp = layers.Input(shape=(1,))
    x = layers.Dense(64, activation='swish')(inp)
```

```
    x = layers.Dense(64, activation='swish')(x)
    out = layers.Dense(1)(x)
    return Model(inp, out, name='model_f')

model_sol = build_model_sol()
model_f   = build_model_f()
```

Next, we chose a smaller learning rate

```
optimizer = optimizers.Adam(learning_rate=0.75e-4)
```

but a larger number of training epochs, `epochs = 300,000`. Then we choose the hyperparameter `w_phys = 0.2` to preserve proportions of measurements in the total loss formula

```
loss_total = loss_data + w_phys * loss_phys
```

where the data loss is implemented by

```
        x_pred = model_sol(t_train_tf, training=True)   # (N,1)
        loss_data = tf.reduce_mean((x_pred - x_train_tf)**2)
```

and the physics-loss as

```
        with tf.GradientTape() as g:
            g.watch(t_colloc_tf)
            x_coll = model_sol(t_colloc_tf, training=True) # (M,1)
        dx_dt = g.gradient(x_coll, t_colloc_tf)     # (M,1)

        f_pred = model_f(x_coll, training=True)     # (M,1)
        loss_phys = tf.reduce_mean((dx_dt - f_pred)**2)
```

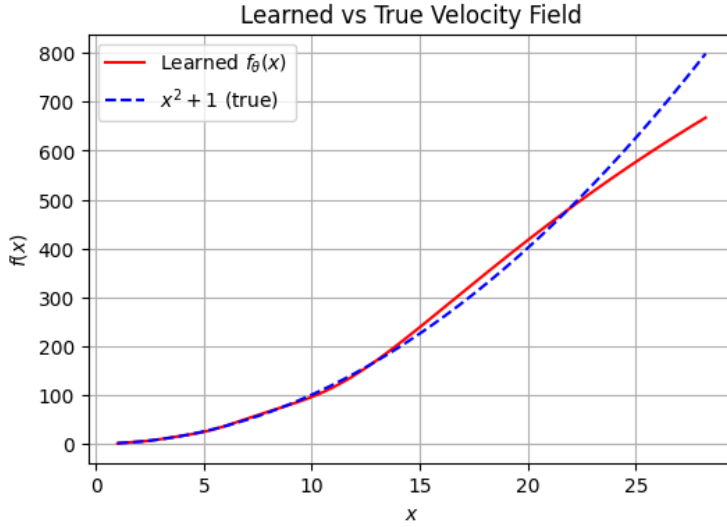
After training the total error decreased to 5 digits accuracy

```
Epoch 300000: total=5.5067e-05, data=4.7168e-05, phys=3.9496e-05
```

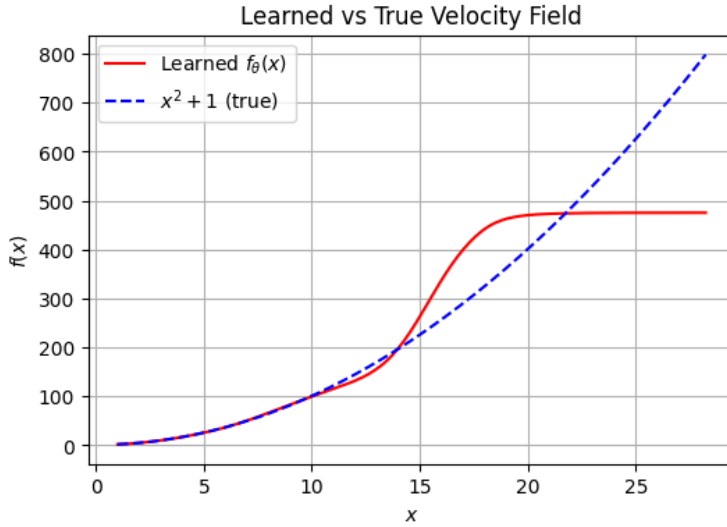
Figure 10.2(a) shows the function $f(x) = x^2 + 1$ learned by the network when using the `swish` activation. For comparison, Fig. 10.2(b) displays the same experiment with the `tanh` activation. We see that `swish` clearly outperforms `tanh`, although in both cases the network struggles for $x \gtrsim 15$. This degradation at large x reflects the curvature effect mentioned above, where the learning is done from to few data.

Consequently, from our synthetic data we recovered the ODE $\dot{x} = f(x)$, with the approximation $f(x) \approx x^2 + 1$ for $0 \leq x \leq 15$. The details of the working code can be found at:

```
https://tinyurl.com/3z9ww3z3
```



(a)



(b)

Figure 10.2: (a) The case when swish activations were used. (b) The case when tanh activations were used.

10.3 Learning a Force Field from Position Data

We tackle the inverse problem of learning a force field from position measurements:

Problem *Given measurements* $\{(t_j, x_j)\}_{j=1}^N \subset \mathbb{R} \times \mathbb{R}^n$*, find a function*

$$F\colon \mathbb{R} \times \mathbb{R}^n \to \mathbb{R}^n$$

such that the solution of the Newton equation

$$\ddot{x}(t) = F\big(t, x(t)\big)$$

satisfies $x(t_j) = x_j$ *for* $j = 1, \ldots, N$.

This problem can be approached in two deep-learning variants:

1. Direct Second-Order PINN

Define a neural network

$$\mathrm{NN}(t, y; \theta) = \big(x_\theta(t),\, F_\theta(t, y)\big),$$

where $(t, y) \in \mathbb{R} \times \mathbb{R}^n$. Using automatic differentiation to compute $\ddot{x}_\theta(t)$, we minimize

$$\mathcal{L}(\theta) = \sum_{j=1}^{M} \|\ddot{x}_\theta(u_j) - F_\theta(u_j, x_\theta(u_j))\|^2 \;+\; \lambda \sum_{k=1}^{N} \|x_\theta(t_k) - x_k\|^2,$$

where $\{u_j\}_{j=1}^M$ are collocation points enforcing the ODE. The learned force field is F_{θ^*} at the optimizer $\theta^* = \arg\min_\theta \mathcal{L}(\theta)$.

2. First-Order Reformulation via Velocity

We introduce the auxiliary velocity $v(t) = \dot{x}(t)$, converting the second-order ODE into the system

$$\begin{cases} \dot{x}(t) = v(t), \\ \dot{v}(t) = F\big(t, x(t)\big). \end{cases}$$

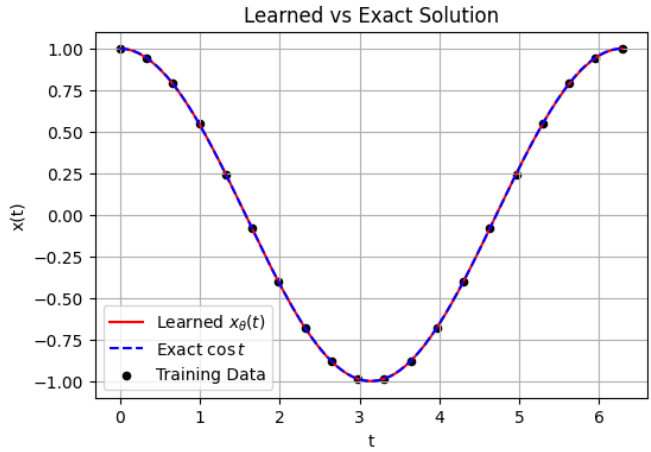
We model this with the neural network

$$\mathrm{NN}(t, y; \theta) = \big(x_\theta(t),\, v_\theta(t),\, F_\theta(t, y)\big),$$

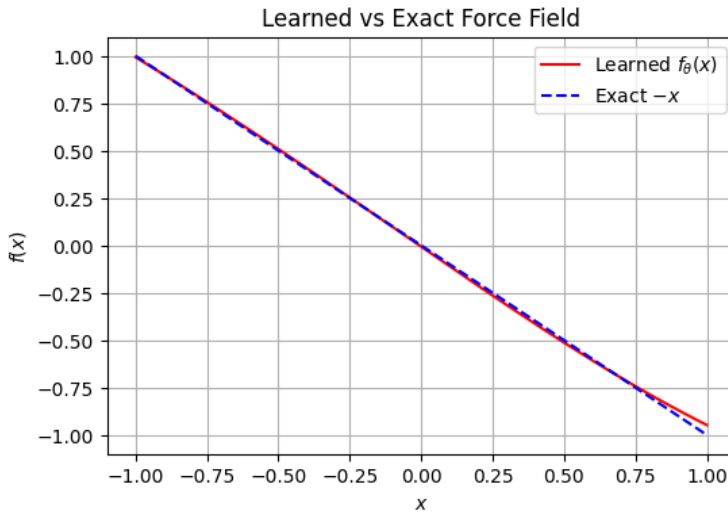
and train by minimizing

$$\mathcal{L}(\theta) = \sum_{j=1}^{M} \|\dot{x}_\theta(u_j) - v_\theta(u_j)\|^2 + \sum_{j=1}^{M} \|\dot{v}_\theta(u_j) - F_\theta(u_j, x_\theta(u_j))\|^2 + \lambda \sum_{k=1}^{N} \|x_\theta(t_k) - x_k\|^2.$$

Again, $\lambda > 0$ balances data-fit and physics terms (roughly $\lambda \approx M/N$), and the optimal parameter θ^* yields both the force and velocity fields, F_{θ^*} and v_{θ^*}.



(a)



(b)

Figure 10.3: (a) The solution model $x_\theta(t)$ learned by the network and the exact solution $x_{exact}(t) = \cos t$ are both passing through the data points $\{(t_j, x_j)\}$. (b) The learned force field $f_\theta(x)$ versus the true field $f(x) = -x$.

10.3.1 Implementation

1. Discovery of equation $\ddot{x} = -x$ from data

(*i*) *Data collection.* We generate first some synthetic data by collecting $N = 20$ measurements $\{(t_j, x_j)\}$ from the solution $x(t) = \cos t$ of the equation

$$\ddot{x}(t) = -x, \quad x(0) = 1.$$

These data are collected by

```
N = 20
t_data = np.linspace(0, 2*np.pi, N, dtype=np.float32).reshape(-1,1)
x_data = np.cos(t_data)
```

(*ii*) *Equation discovery.* Then we build a physics-informed neural network to learn both the solution and the acceleration field. The equation is enforced at `M = 100` collocation points. The neural models are as follows:

```
# Model for solution x(t)
def build_model_sol():
    inp = layers.Input(shape=(1,))
    x = layers.Dense(32, activation='swish')(inp)
    x = layers.Dense(32, activation='swish')(x)
    out = layers.Dense(1)(x)
    return Model(inp, out, name='model_sol')

# Model for f(x)
def build_model_f():
    inp = layers.Input(shape=(1,))
    x = layers.Dense(64, activation='swish')(inp)
    x = layers.Dense(64, activation='swish')(x)
    out = layers.Dense(1)(x)
    return Model(inp, out, name='model_f')

model_sol = build_model_sol()
model_f   = build_model_f()
```

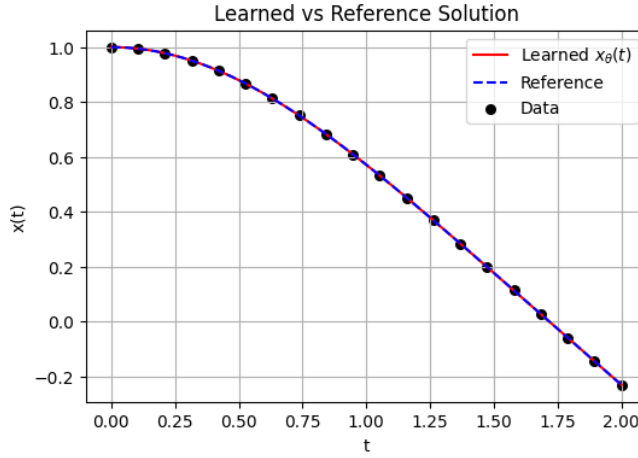
We use Adam optimizer with learning rate 0.001 to minimize the composite loss

```
loss_total = loss_data + w_phys * loss_phys
```

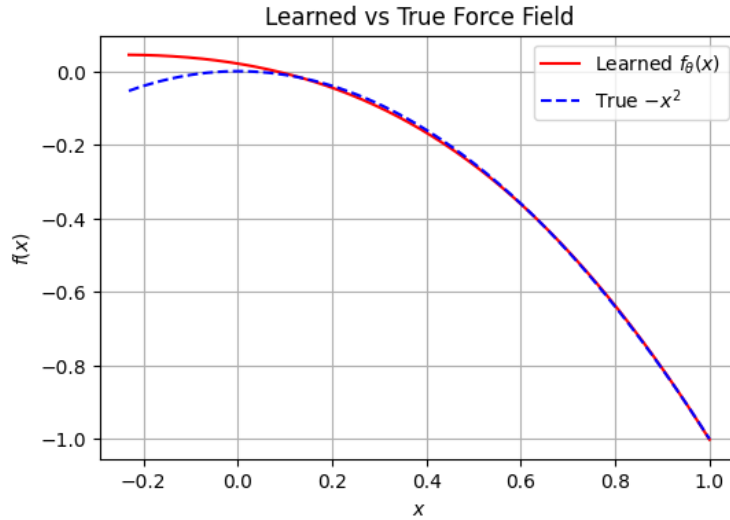
with `w_phys = 5.0`. Training for $5,000$ epochs we obtain small losses for the total, data and physics-loss as follows:

```
Epoch 50000: total=4.0114e-06, data=1.5392e-06, phys=4.9443e-07
```

The solution $x_\theta(t)$ learned by the network and the exact solution $x_{exact}(t) = \cos t$ are given in Fig. 10.3(a). The learned force field $f_\theta(x)$ versus the true field $f(x) = -x$ are given in Fig. 10.3(b). We note a very good fit and recovery of the force field.



(a)



(b)

Figure 10.4: (a) The solution model $x_\theta(t)$ learned by the network and the exact solution $x_{exact}(t)$ are both passing through the data points $\{(t_j, x_j)\}$. (b) The learned force field $f_\theta(x)$ versus the true field $f(x) = -x^2$. We notice a mismatch for small values of x.

The next example shows that even a non-linear force field can be learned a similar methodology.

2. Discovery of equation $\ddot{x} = -x^2$ from data Applying the same neural-network architecture to the ODE

$$\ddot{x} = -x^2, \quad x(0) = 1, \quad \dot{x}(0) = 0,$$

we obtain a reasonable match both for the learned solution and for the recovered force field as in Fig. 10.4.

The details of the working code can be found at:

`https://tinyurl.com/3z9ww3z3`

10.3.2 Exercises

Exercise 10.3.1 The following ODE

$$\dot{x} = \sqrt{1 - x^2}, \quad x(0) = 0,$$

has the analytical solution on $0 \le t \le \pi/2$ given by

$$x(t) = \sin t.$$

1. Generate $N = 20$ synthetic observations $\{(t_j, x_j)\}_{j=1}^{N}$ by sampling the exact solution of the previous ODE.
2. Plot the raw data points $\{(t_j, \sin t_j)\}$.
3. Construct two neural networks:

   $$\mathrm{NN}_\theta : \ t \mapsto x_\theta(t), \quad f_\theta : \ x \mapsto f_\theta(x),$$

   and select M collocation times $\{\tau_k\} \subset [0, \pi/2]$.
4. Define the composite loss

   $$\mathcal{L}(\theta) = \underbrace{\frac{1}{N}\sum_{j=1}^{N}\big(x_\theta(t_j) - x_j\big)^2}_{\text{data term}} + \underbrace{\frac{1}{M}\sum_{k=1}^{M}\big(\dot{x}_\theta(\tau_k) - f_\theta\big(x_\theta(\tau_k)\big)\big)^2}_{\text{physics term}}.$$

5. Train θ by gradient descent to minimize $\mathcal{L}(\theta)$.
6. Plot the learned field $f_\theta(x)$ against the true $\sqrt{1 - x^2}$.
7. Plot the learned trajectory $x_\theta(t)$ versus the exact $\sin t$.
8. Verify that $f_\theta(x) \approx \sqrt{1 - x^2}$ over the range $0 \le x \le 1$.

t_j	x_j
0.000	1.000
0.164	0.989
0.330	0.954
0.494	0.899
0.660	0.820
0.824	0.722
0.990	0.611
1.154	0.494
1.320	0.376
1.484	0.264
1.650	0.161
1.814	0.070
1.980	-0.006
2.144	-0.065
2.310	-0.106
2.474	-0.129
2.640	-0.136
2.804	-0.127
2.970	-0.103
3.134	-0.069

Table 10.1: Sample points (t_j, x_j) from the solution of $\ddot{x} = -\sin(x)$.

Exercise 10.3.2 The solution to the nonlinear pendulum equation $\ddot{x} = -\sin(x)$ with initial conditions $x(0) = 1$, $\dot{x}(0) = 0$ has been computed numerically on the interval $[0, \pi]$ using the Runge–Kutta method. Then a table of 20 evenly spaced sample points (t_j, x_j) from the solution is provided in Table 10.1.

(a) Plot the 20 sample points (t_j, x_j) to visualize the behavior of the pendulum over time.

(b) Build and train a feed-forward neural network $x_\theta(t)$ to learn the function $x(t)$, using the 20 data points.

(c) Use the trained model $x_\theta(t)$ to compute $\ddot{x}_\theta(t_j)$ using automatic differentiation or finite differences.

(d) Construct the dataset $\{x_\theta(t_j), \ddot{x}_\theta(t_j)\}_{j=1}^{20}$.

(e) Train a second neural network $f_\phi(x)$ to approximate the force law from the data $\{x_j, \ddot{x}_j\}$ using the loss function

$$\mathcal{L}_2 = \frac{1}{N} \sum_{j=1}^{N} (f_\phi(x_j) - \ddot{x}_j)^2.$$

(f) Plot the learned function $f_\phi(x)$ and the true function $f(x) = -\sin(x)$ on the interval $x \in [-1.5, 1.5]$.

(g) Compute and report the mean squared error:

$$\text{MSE} = \frac{1}{N}\sum_{j=1}^{N}(f_\phi(x_j) + \sin(x_j))^2.$$

(h) Repeat the training of f_ϕ using each of the following activation functions: ReLu, tanh, logistic sigmoid, swish. Discuss which activation function performs best for this regression task and provide reasoning.

Chapter 11

Identifying Conservation Laws

In Chapter 6, we demonstrated how conservation laws can serve as effective regularizers for neural networks, either by introducing them as soft constraints or by incorporating them directly through hard embeddings. We also showed that such conservation laws can be derived from symmetries of the Lagrangian.

In this chapter, however, we take a different perspective: instead of assuming knowledge of the governing dynamical equations we aim to construct conservation laws directly from data. One objective is to identify consistent patterns and invariants solely from measurements. This can be done either locally, over short time windows, or globally, across the entire dataset.

11.1 Mechanical Systems

The first application considers data collected from a mechanical system, given as pairs of position and velocity $(x_i, \dot{x}_i)$. Our goal is to identify a conservation law that represents a functional relationship involving time t, position $x(t)$, and velocity $\dot{x}(t)$. Such a conservation law can be expressed as

$$F\big(t, x(t), \dot{x}(t)\big) = C, \qquad \forall t \geq 0,$$

where C is a constant. Since the constant can be absorbed into the definition of F, without loss of generality we may rewrite the conservation law in the simplified form

$$F\big(t, x(t), \dot{x}(t)\big) = 0. \tag{11.1.1}$$

We now formulate the following problem:

Problem *Given measurements of the state of a particle–its position and velocity–denoted by $(x_i, \dot{x}_i)$ at N collocation instances t_i, determine the function F that satisfies condition (11.1.1).*

If there exists an instance $t_0 > 0$ such that

$$\partial_{\dot{x}} F\big(t_0, x(t_0), \dot{x}(t_0)\big) \neq 0,$$

then equation (11.1.1) can be solved locally around t_0. In particular, there exists $\epsilon > 0$ such that

$$\dot{x}(t) \;=\; G(t, x(t)), \qquad t \in (t_0 - \epsilon,\, t_0 + \epsilon).$$

Since this relation holds only on a short time interval, we refer to it as a *local conservation law*, meaning a local pattern where the velocity $\dot{x}(t)$ can be expressed explicitly in terms of position and time.

Explicit conservation laws. We consider training data of the form $(t_i, x_i; \dot{x}_i)$ and train a neural network in a supervised manner, taking $\{t_i, x_i\}$ as inputs and $\dot{x}_i$ as labels. The neural network is denoted by $f_\theta(t, x)$, and the associated loss is

$$\mathcal{L}(\theta) \;=\; \frac{1}{N} \sum_{i=1}^{N} \big(f_\theta(t_i, x_i) - \dot{x}_i\big)^2. \tag{11.1.2}$$

Let $\theta^* = \arg\min \mathcal{L}(\theta)$. Then we obtain the relation $f_{\theta^*}(t, x) \;\approx\; \dot{x}$, and we can regard

$$F(t, x, \dot{x}) \;=\; \dot{x} - f_{\theta^*}(t, x)$$

as a candidate conservation law.

Since this relation is not expected to hold globally for all t, we may restrict attention to a local time segment by defining the localized loss

$$\mathcal{L}_{j,k}(\theta) \;=\; \frac{1}{k - j + 1} \sum_{i=j}^{k} \big(f_\theta(t_i, x_i) - \dot{x}_i\big)^2, \qquad 1 \le j \le k \le N.$$

This yields a conservation law on the time interval (t_j, t_k).

Implicit conservation laws. An implicit functional relation among the variables t, x, and $\dot{x}$ can be discovered by training a neural network $F_\theta(t, x, \dot{x})$ in an unsupervised manner, with the goal that its output vanishes, or it is constant. This is achieved by minimizing the loss

$$\mathcal{L}(\theta) \;=\; \frac{1}{N} \sum_{i=1}^{N} F_\theta(t_i, x_i, \dot{x}_i)^2.$$

In this case, the learned relation

$$F_{\theta^*}(t, x, \dot{x}) \;=\; 0$$

defines a global conservation law.

Alternatively, we may enforce constancy by considering the variance-type loss

$$\mathcal{L}(\theta) \;=\; \frac{1}{N} \sum_{i=1}^{N} \Big(F_\theta(t_i, x_i, \dot{x}_i) - \overline{F}_\theta\Big)^2, \qquad \overline{F}_\theta = \frac{1}{N} \sum_{i=1}^{N} F_\theta(t_i, x_i, \dot{x}_i). \tag{11.1.3}$$

Here, the network learns a functional relation F_θ that remains approximately constant across the data, which again represents a conservation law.

Remark 11.1.1 It is important to note that a dynamical system may admit multiple, independent conservation laws. Accordingly, distinct neural network architectures, or even different training initializations, may converge to and uncover different conservation laws inherent in the system.

We now formulate another problem, which involves also acceleration measurements $\ddot{x}_i = \ddot{x}(t_i)$:

Problem *Given measurements of the position, velocity and acceleration, denoted by* $(x_i, \dot{x}_i, \ddot{x}_i)$ *at* N *collocation instances* t_i*, determine the function* F *that satisfies condition (11.1.1).*

We begin by observing that differentiating relation (11.1.1) with respect to time yields

$$0 \;=\; \frac{d}{dt} F\big(t, x(t), \dot{x}(t)\big) \;=\; \frac{\partial F}{\partial t} \;+\; \frac{\partial F}{\partial x}\,\dot{x}(t) \;+\; \frac{\partial F}{\partial \dot{x}}\,\ddot{x}(t).$$

On the other hand, suppose that the dynamics of the system are governed by a Newton-type equation

$$a\,\ddot{x}(t) \;+\; b\,\dot{x}(t) \;+\; c \;=\; 0,$$

where $a = a(t, x, \dot{x})$, $b = b(t, x, \dot{x})$, and $c = c(t, x, \dot{x})$ are known smooth functions.

By comparing coefficients in the two relations, we obtain a system of PDEs for F:

$$\frac{\partial F}{\partial \dot{x}} = a, \qquad \frac{\partial F}{\partial x} = b, \qquad \frac{\partial F}{\partial t} = c.$$

For such a function F to exist, a compatibility condition must hold. Specifically, the curl of the vector field (a, b, c) must vanish, which requires

$$\frac{\partial a}{\partial x} = \frac{\partial b}{\partial \dot{x}}, \qquad \frac{\partial a}{\partial t} = \frac{\partial c}{\partial \dot{x}}, \qquad \frac{\partial b}{\partial t} = \frac{\partial c}{\partial x}.$$

To determine the function F, we proceed in two steps.

Step 1. Learning the coefficients We first learn the coefficient functions a, b, and c by employing a neural network

$$NN(\theta; t, x, \dot{x}) \;=\; \big(a_\theta(t, x, \dot{x}),\, b_\theta(t, x, \dot{x}),\, c_\theta(t, x, \dot{x})\big),$$

which is trained to minimize the total loss

$$\mathcal{L}_{\text{tot}}(\theta) \;=\; \mathcal{L}_{\text{ODE}}(\theta) + \mathcal{L}_{\text{curl}}(\theta).$$

The *ODE loss* enforces the governing equation along the training data set $\{(t_i, x_i, \dot{x}_i)\}$:

$$\mathcal{L}_{\text{ODE}}(\theta) \;=\; \frac{1}{N} \sum_{i=1}^{N} \Big(a_\theta(t_i, x_i, \dot{x}_i)\,\ddot{x}_i + b_\theta(t_i, x_i, \dot{x}_i)\,\dot{x}_i + c_\theta(t_i, x_i, \dot{x}_i) \Big)^2.$$

The *curl loss* enforces the compatibility condition (vanishing curl):

$$\begin{aligned}\mathcal{L}_{\text{curl}}(\theta) \;&= \frac{1}{N}\sum_{i=1}^{N}\Big(\tfrac{\partial}{\partial x}a_\theta(t_i,x_i,\dot{x}_i) - \tfrac{\partial}{\partial \dot{x}}b_\theta(t_i,x_i,\dot{x}_i)\Big)^2 \\ &+ \frac{1}{N}\sum_{i=1}^{N}\Big(\tfrac{\partial}{\partial t}a_\theta(t_i,x_i,\dot{x}_i) - \tfrac{\partial}{\partial \dot{x}}c_\theta(t_i,x_i,\dot{x}_i)\Big)^2 \\ &+ \frac{1}{N}\sum_{i=1}^{N}\Big(\tfrac{\partial}{\partial t}b_\theta(t_i,x_i,\dot{x}_i) - \tfrac{\partial}{\partial x}c_\theta(t_i,x_i,\dot{x}_i)\Big)^2.\end{aligned}$$

Once training is complete, the learned coefficients are given by

$$a = a_{\theta^*}, \qquad b = b_{\theta^*}, \qquad c = c_{\theta^*}.$$

Step 2. Learning the function F With the coefficient functions a, b, and c obtained in Step 1, we now employ a neural network

$$NN(w;t,x,\dot{x}) \;=\; F_w(t,x,\dot{x}),$$

which is trained to minimize the loss

$$\begin{aligned}\mathcal{L}(w) \;&= \frac{1}{N}\sum_{i=1}^{N}\Big(\tfrac{\partial}{\partial \dot{x}}F_w(t_i,x_i,\dot{x}_i) - a(t_i,x_i,\dot{x}_i)\Big)^2 \\ &+ \frac{1}{N}\sum_{i=1}^{N}\Big(\tfrac{\partial}{\partial x}F_w(t_i,x_i,\dot{x}_i) - b(t_i,x_i,\dot{x}_i)\Big)^2 \\ &+ \frac{1}{N}\sum_{i=1}^{N}\Big(\tfrac{\partial}{\partial t}F_w(t_i,x_i,\dot{x}_i) - c(t_i,x_i,\dot{x}_i)\Big)^2.\end{aligned}$$

If $w^* = \arg\min \mathcal{L}(w)$, then the learned conservation law is given by

$$F_{w^*}(t,x,\dot{x}) \;=\; C,$$

for some constant C.

The conservation of energy Sometimes, if prior knowledge of the dynamical system is available-such as its underlying Lagrangian or Hamiltonian-then the associated conservation laws can be obtained directly. In this section we assume a dynamical system whose evolution is described by the natural Lagrangian

$$\mathcal{L}(x,\dot{x}) = \tfrac{1}{2}\dot{x}^2 - U(x),$$

where $U(x)$ is the potential in which a unit-mass particle moves.

Problem *Given N measurements of the particle's state $(x_i,\dot{x}_i)$, recover the conservation of energy for the system.*

It is known that in this case the conserved energy takes the form

$$E = \tfrac{1}{2}\dot{x}^2 + U(x) = \text{constant}.$$

Thus, it suffices to identify the potential $U(x)$. To this end, we parameterize the potential with a neural network,

$$NN(\theta; x) = U_\theta(x),$$

and define the corresponding energy function

$$F_\theta(x, \dot{x}) = \tfrac{1}{2}\dot{x}^2 + U_\theta(x).$$

Since $F_\theta(x, \dot{x})$ should be constant along the trajectory, we minimize the variance-type loss

$$\mathcal{L}_{\text{var}}(\theta) = \text{Var}\big(F_\theta(x, \dot{x})\big) = \frac{1}{N}\sum_{i=1}^{N}\Big(F_\theta(x_i, \dot{x}_i) - \overline{F}_\theta\Big)^2,$$

where

$$\overline{F}_\theta = \frac{1}{N}\sum_{i=1}^{N} F_\theta(x_i, \dot{x}_i),$$

plus a soft anchor penalty enforcing $U(0) = U_0$. This anchor is needed since the potential is identifiable up to an additive constant. The total loss becomes

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{var}}(\theta) + \lambda(U_\theta(0) - U_0)^2.$$

Let $\theta^* = \arg\min_\theta \mathcal{L}_{\text{total}}(\theta)$. Then $U_{\theta^*}(x)$ approximates the potential energy, and the conserved energy is $E = \overline{F}_{\theta^*}$, so that the conservation of energy becomes

$$\tfrac{1}{2}\dot{x}^2 + U_{\theta^*}(x) \;=\; \overline{F}_{\theta^*}.$$

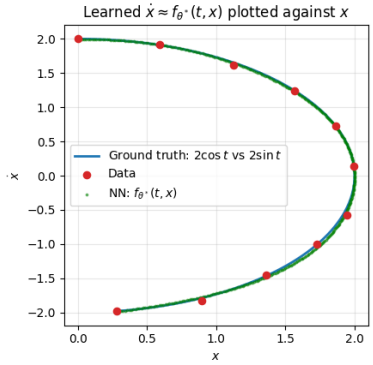
11.1.1 Implementations

Implementation for one-dimensional harmonic oscillator Some data were generated synthetically from the trajectory of a one-dimensional harmonic oscillator:

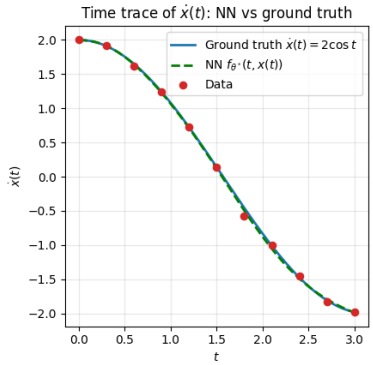
We aim to recover the energy conservation law using both supervised and unsupervised approaches. The code for this section is available at

`https://tinyurl.com/5fkba2m6`

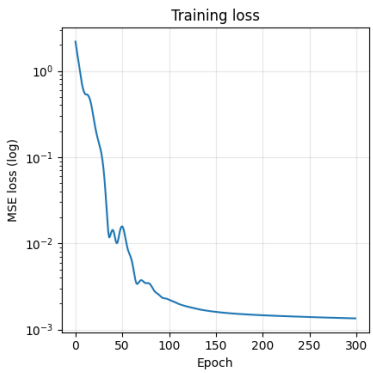
Explicit (supervised) conservation law. We train a neural network $f_\theta(t, x)$ in a supervised manner with the loss in (11.1.2). The model is a feed-forward network with four hidden layers of 64 `tanh`-activated neurons each, optimized with Adam having learning rate 10^{-3} for 300 epochs. Figure 11.1(a) shows that the learned relation closely matches both the data $(x_i, \dot{x}_i)$ and the exact algebraic curve. In Fig. 11.1(b) the network's predicted velocity $\dot{x}$ aligns well with the measurements and the exact solution $\dot{x}(t) = 2\cos t$. The training loss, plotted in Fig. 11.1(c), saturates after roughly 250 epochs.



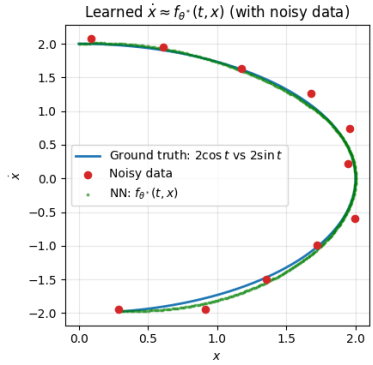
(a)



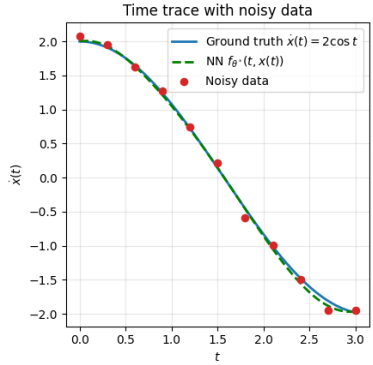
(b)



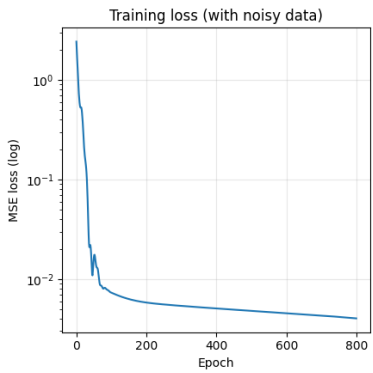
(c)



(d)



(e)



(f)

Figure 11.1: Noiseless data: (a) Recovered $\dot{x} = f_\theta(t, x)$ versus the exact solution; (b) Recovered $\dot{x}(t)$ compared with the exact $\dot{x}_{\text{exact}}(t) = 2\cos t$; (c) Associated loss versus epochs. Noisy data: (d) Recovered $\dot{x} = f_\theta(x)$ versus the exact solution; (e) Recovered $\dot{x}(t)$ compared with the exact $\dot{x}_{\text{exact}}(t) = 2\cos t$; (f) Associated loss versus epochs.

t_j	$x(t_j) = 2\sin t_j$	$\dot{x}(t_j) = 2\cos t_j$
0.0	0.0000	2.0000
0.3	0.5910	1.9107
0.6	1.1280	1.6209
0.9	1.5656	1.2449
1.2	1.8632	0.7247
1.5	1.9940	0.1411
1.8	1.9468	−0.5817
2.1	1.7270	−1.0009
2.4	1.3624	−1.4560
2.7	0.8962	−1.8226
3.0	0.2822	−1.9799

Table 11.1: Measurements of position $x(t)$ and velocity $\dot{x}(t)$ for $t_j \in [0, 3]$.

To test the robustness of the method, we added a 5% Gaussian noise to the data and attempted to recover the explicit conservation law. As shown in Figs. 11.1(d)–(f), the recovery remains highly accurate, even in the presence of noise.

Implicit (unsupervised) conservation law. Using the data described above, we seek an implicit functional relation among the variables t, x, and $\dot{x}$ by training a neural network $F_\theta(t, x, \dot{x})$ in an unsupervised manner with the variance-type loss function given in (11.1.3). As shown in Fig. 11.2(a), this network does not succeed in learning the underlying relation. The reason is that the variable t does not enter the implicit conservation law.

We therefore remove t and consider instead a neural network of the form $F_\theta(x, \dot{x})$, with only two inputs. In this case the network learns the relation successfully and approximates the exact trajectory in the phase space, which corresponds to an arc of the circle $x^2 + \dot{x}^2 = 4$, see Fig. 11.2(b).

Since the available data cover only part of the trajectory, the implicit plot of F follows the circle only locally. With a more balanced data set covering the entire trajectory, the network would recover the full circle.

Implementation for pendulum Table 11.2 is generated by solving the simple nonlinear pendulum equation

$$\ddot{x}(t) = -\sin x(t), \qquad x(0) = 0, \qquad \dot{x}(0) = 1,$$

by the Runge–Kutta method, taking $t \in [0, 3]$ and $\Delta t = 0.2$.

The potential network $U_\theta(x)$ was implemented as a feed-forward neural network with two hidden layers of 64 neurons each, using tanh activation. An anchor condition $U_\theta(0) = -1$ was enforced as a soft constraint in addition to the variance loss. We further included an L^2-regularization term with weight 10^{-6} to improve robustness. Training was carried out for $8,000$ epochs using the Adam optimizer with a learning rate of 10^{-4}.

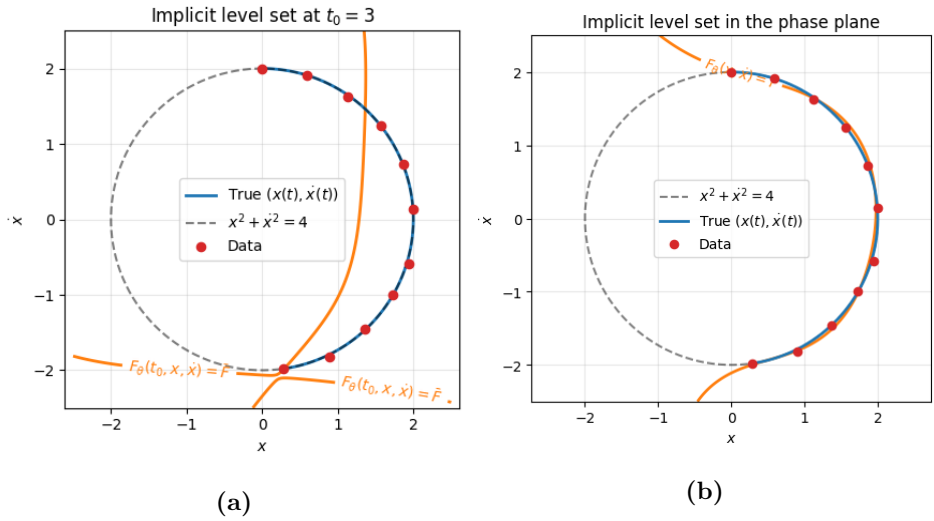


Figure 11.2: Plots in the phase space: (a) The implicit plot $F(t_0, x, \dot{x}) = \overline{F}$ at the time slice $t_0 = 3$; (b) The implicit plot $F(x, \dot{x}) = \overline{F}$.

t	$x(t)$	$\dot{x}(t)$
0.000000	0.000000	1.000000
0.200000	0.198669	0.980132
0.400000	0.389494	0.922062
0.600000	0.565220	0.830020
0.800000	0.719632	0.709991
1.000000	0.847790	0.568572
1.200000	0.946054	0.412029
1.400000	1.011955	0.245764
1.600000	1.044009	0.074227
1.800000	1.041545	-0.098823
2.000000	1.004614	-0.269850
2.200000	0.933994	-0.435024
2.400000	0.831298	-0.589766
2.600000	0.699151	-0.728537
2.800000	0.541372	-0.844981
3.000000	0.363090	-0.932522

Table 11.2: RK4 solution of $\ddot{x} = -\sin x$, with $x(0) = 0$, $\dot{x}(0) = 1$, $\Delta t = 0.2$.

The learned model produced an estimated conserved energy

$$\overline{E} = -0.4995627541810287,$$

which is in excellent agreement with the true value of -0.5.

Figure 11.3(a) shows that the learned potential $U_{\theta^*}(x)$ overlaps almost perfectly with the true potential curve $-\cos x$. Figure 11.3(b) illustrates how the energy values along the trajectory fluctuate slightly around the mean, while Fig. 11.3(c) demonstrates that the training losses have already reached their thresholds. The corresponding code can be found at the page

`https://tinyurl.com/53kmdwn9`

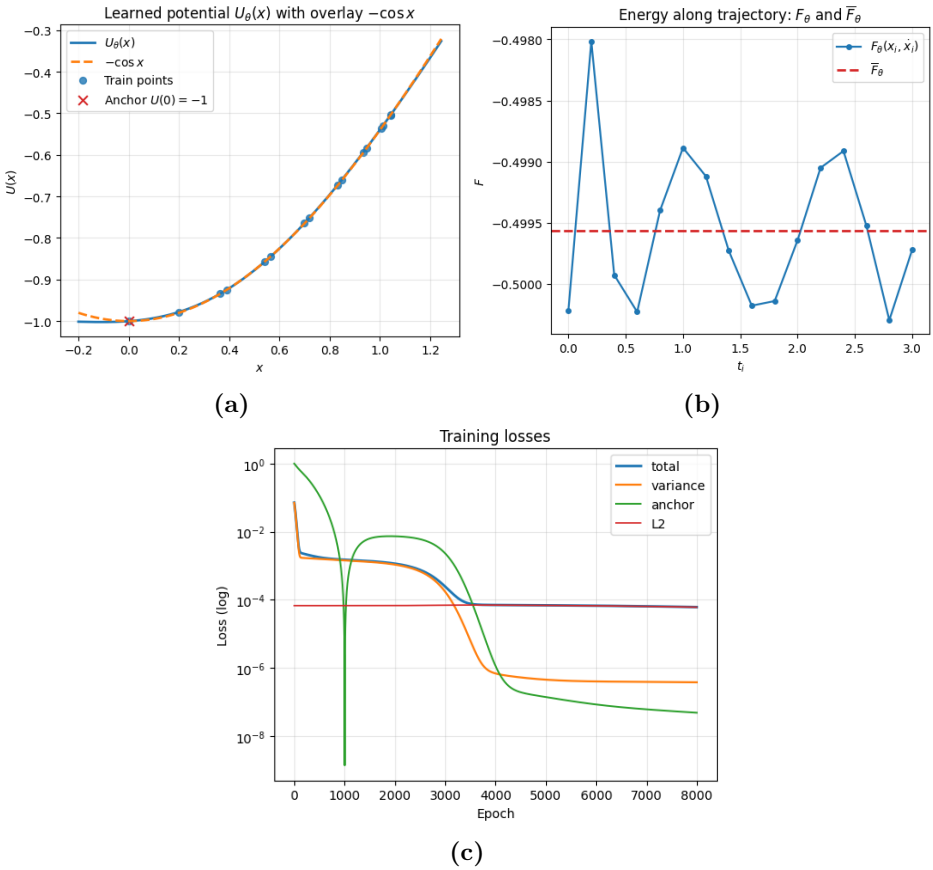


Figure 11.3: Noiseless data: (a) The potential $U_{\theta^*}(x)$ versus $y = -\cos x$; (b) The energy value oscillations about the mean; (c) Associated losses versus epochs.

11.2 Fluid Dynamics

The motion of an ideal fluid is governed by several fundamental conservation laws:

1. Conservation of mass This is expressed by the *continuity equation* for a sourceless

fluid:

$$\frac{\partial \rho}{\partial t} + \operatorname{div}(\rho V) = 0, \tag{11.2.4}$$

where $\rho(t, x)$ denotes the fluid density at point x and time t, and $V(t, x) = (V_1, V_2, V_3)$ is the velocity vector field of the fluid particles.

2. Conservation of momentum This is given by *Euler's equation*:

$$\frac{\partial V}{\partial t} + (V \cdot \nabla)V + \frac{1}{\rho}\nabla p = F, \tag{11.2.5}$$

where V is the velocity field, ρ the fluid density, $p(t, x)$ the pressure, and $F(t, x) = (F_1, F_2, F_3)$ the external force field (such as gravity). The term $(V \cdot \nabla)V$ represents the directional derivative of V along itself, i.e. the convective acceleration.

3. Equation of state This relation connects the density and the pressure of the fluid:

$$\Phi(\rho, p) = 0. \tag{11.2.6}$$

11.2.1 Conservation of mass

Using the vector calculus identity

$$\operatorname{div}(\rho V) = \rho \operatorname{div} V + \langle \nabla \rho, V \rangle,$$

where $\nabla \rho = (\partial_{x_1}\rho, \partial_{x_2}\rho, \partial_{x_3}\rho)$ denotes the gradient of the density, the continuity equation (11.2.4) takes the form

$$\frac{\partial \rho}{\partial t} + \langle \nabla \rho, V \rangle = -\rho \operatorname{div} V. \tag{11.2.7}$$

The left-hand side admits a natural geometric interpretation. Let $c(t) = (c_1(t), c_2(t), c_3(t))$ be an integral curve of the velocity field V, so that

$$\dot{c}(t) = V(t, c(t)).$$

The particle trajectories in the fluid follow such curves $c(t)$. Applying the chain rule, the rate of change of the density along these trajectories is

$$\begin{aligned} \frac{d}{dt}\rho(t, c(t)) &= \frac{\partial \rho}{\partial t} + \sum_i \frac{\partial \rho}{\partial x_i}\, \dot{c}_i(t) \\ &= \frac{\partial \rho}{\partial t} + \langle \nabla \rho, \dot{c}(t) \rangle \\ &= \frac{\partial \rho}{\partial t} + \langle \nabla \rho, V \rangle \big|_{c(t)}. \end{aligned}$$

Substituting this into (11.2.7) yields an alternative form of the continuity equation:

$$\frac{d}{dt}\,\rho(t, c(t)) = -\rho \operatorname{div} V\big|_{c(t)}.$$

This relation connects the rate of change of density along particle paths with the local expansion rate of the velocity field, given by its divergence.

A common simplification is the case of an *incompressible fluid*, for which $\operatorname{div} V = 0$. In this case the continuity equation reduces to

$$\frac{\partial \rho}{\partial t} + \langle \nabla \rho, V \rangle = 0, \tag{11.2.8}$$

which implies

$$\frac{d}{dt}\, \rho(t, c(t)) = 0.$$

Thus the density of the fluid remains constant along particle trajectories, leading to the conservation law for incompressible fluids:

$$\rho(t, c(t)) = \rho(0, c(0)), \qquad \forall\, t \geq 0. \tag{11.2.9}$$

The use of neural nets Since the form of the mass conservation law is already known, the quantities that must be inferred from data are the density ρ and the divergence-free velocity field V. In practice, one of these may be directly measurable, while the other must be reconstructed, for example using Deep Learning methods. This leads us to the following formulation:

Problem *Given $N \times M$ measurements of the density function, $\rho_{ij} = \rho(t_i, x_j)$, at N time instances t_i and M spatial locations x_j, determine the velocity field V such that* $\operatorname{div} V = 0$ *and the continuity equation (11.2.8) holds.*

For precision, we assume that the collocation points lie within a cylindrical domain

$$\mathcal{C} = \{(t_i, x_j) \in [0, T] \times \Omega\},$$

where $\Omega \subset \mathbb{R}^3$ is a compact spatial region. Our goal is to determine the velocity field inside this cylinder. In layman's terms, the problem can be stated as follows: *Given the snowflake density at any time t, determine the wind velocity.*

We shall adopt two approaches:

Method 1 We shall follow two steps.

Step 1: Learning the Density. We use a neural network $\rho_w(t, x)$ to approximate the density from the data ρ_{ij}. The architecture is a standard feed-forward network, trained by minimizing the following loss function:

$$\mathcal{L}_{\text{density}}(w) = \frac{1}{NM} \sum_{i,j} \left(\rho_w(t_i, x_j) - \rho_{ij}\right)^2.$$

After training, the density is approximated by $\rho \approx \rho_{w^*}$, where

$$w^* = \arg\min \mathcal{L}_{\text{density}}(w).$$

Step 2: Learning the Velocity Field. The velocity field is modeled using a neural network, with the divergence-free constraint incorporated directly into the network

architecture to ensure physical consistency. To achieve this, we introduce a latent vector potential $A = (A_1, A_2, A_3)$ such that

$$V = \mathrm{curl} A = \nabla \times A,$$

which automatically guarantees that $\mathrm{div}\, V = 0$. We then employ a neural network A_θ to approximate the latent potential A, so that the velocity field is represented as

$$V_\theta = \mathrm{curl} A_\theta = \nabla \times A_\theta.$$

This formulation ensures that the learned velocity field is divergence-free by design, while its parameters are optimized to satisfy the continuity equation using the available data by minimizing the following loss function

$$\mathcal{L}(\theta) = \frac{1}{NM} \sum_{i,j} \Big(\frac{\partial \rho}{\partial t}(t_i, x_j) + \langle \nabla \rho, V_\theta \rangle_{|(t_i, x_j)} \Big)^2.$$

In the previous loss the derivatives of the density are taken by autodifferentiation in the network ρ_{w^*} found at Step 1.

Method 2 Again, we follow several steps:

Step 1: Learning the Density. This is the same as in Method 1.

Step 2: Learning the integral curves. Rather than explicitly solving for the velocity field V, it is sufficient to determine its integral curves; the velocity field can then be recovered by differentiation. To this end, we fix an arbitrary initial point $x_0 \in \{0\} \times \Omega$, and construct the integral curve $c(t)$ of V starting at $c(0) = x_0$ using a neural network that incorporates this initial condition:

$$c_\theta(t) = x_0 + t\, \mathrm{nn}(\theta; t).$$

To enforce equation (11.2.9), we define a variance-type loss function:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(\rho(t_i, c_\theta(t_i)) - \overline{\rho} \right)^2, \qquad \overline{\rho} = \frac{1}{N} \sum_{i=1}^{N} \rho(t_i, c_\theta(t_i)).$$

Or, if $\rho(0, x_0)$ is given, we can take

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(\rho(t_i, c_\theta(t_i)) - \rho(0, x_0) \right)^2.$$

Then $c_{\theta^*}(t)$ is the integral curve of V starting at $c_{\theta^*}(0) = x_0$.

Step 3: Learning the Velocity Field. Although we know that

$$V(t, c_{\theta^*}(t)) = \dot{c}_{\theta^*}(t),$$

we still require a formula for $V(t, x)$. To obtain this, it suffices to find the integral curve that passes through x at time t, i.e., with $c(t) = x$. Since such an integral curve

is uniquely determined by its initial condition $x_0 = c(0)$, the task reduces to recovering this initial value x_0. Once identified, Step 2 can be applied to obtain the corresponding integral curve.

To this end, we treat x_0 as an internal variable $\eta \in \mathbb{R}^3$ of the network and embed both endpoint conditions into the following parameterization of the curve:

$$c_{\theta,\eta}(\tau) = \frac{\tau}{t}x \; + \; \frac{t-\tau}{t}\eta \; + \; \tau(t-\tau)\, NN(\theta, \eta; \tau),$$

where $NN(\theta, \eta; \tau)$ is a free-form neural network with augmented parameters (θ, η).

Training is performed by minimizing the variance-type loss

$$\mathcal{L}(\theta, \eta) = \frac{1}{N}\sum_{i=1}^{N} \left(\rho(t_i, c_{\theta,\eta}(t_i)) - \overline{\rho}\right)^2, \qquad \overline{\rho} = \frac{1}{N}\sum_{i=1}^{N} \rho(t_i, c_{\theta,\eta}(t_i)).$$

At convergence, we obtain $c_{\theta^*,\eta^*}(t) = x$, and we recover also the initial condition

$$x_0 = c_{\theta^*,\eta^*}(0) = \eta^*.$$

Why is mass conserved? For completeness, we explain why the continuity equation expresses conservation of mass for a fluid. The total mass of the substance at time t in a compact domain $\Omega \subset \mathbb{R}^3$ is obtained by integrating the density:

$$m(t) = \int_\Omega \rho(t, x)\, dx.$$

The mass depends on time since fluid may enter or leave Ω. Differentiating under the integral gives the rate of change of mass:

$$m'(t) = \int_\Omega \partial_t \rho(t, x)\, dx. \tag{11.2.10}$$

We can compute the same rate of change in an equivalent physical way. The change in mass over the interval $[t, t + \Delta t]$ is

$$\Delta m(t) = m(t + \Delta t) - m(t).$$

This change corresponds to the net mass flux across the boundary $\partial\Omega$. Consider a surface element $d\sigma$ of $\partial\Omega$ with outward unit normal vector ν. The mass passing through this surface during time Δt is

$$\rho\langle V, \nu\rangle\, d\sigma\, \Delta t,$$

where V is the velocity field. Summing over the boundary, the total variation of mass is

$$\int_{\partial\Omega} \rho\langle V, \nu\rangle\, d\sigma\, \Delta t = \int_{\partial\Omega} \langle \rho V, \nu\rangle\, d\sigma\, \Delta t = \int_\Omega \operatorname{div}(\rho V)\, dx\, \Delta t,$$

where we used Gauss' divergence theorem.

Therefore,

$$\Delta m = -\int_\Omega \operatorname{div}(\rho V)\, dx\, \Delta t,$$

the minus sign reflecting that an increase in mass inside Ω corresponds to inward flux across $\partial\Omega$, opposite to the outward normal ν.

Dividing by Δt and taking the limit as $\Delta t \to 0$, we obtain

$$m'(t) = -\int_\Omega \operatorname{div}(\rho V)\, dx.$$

Comparing with (11.2.10), we find

$$\int_\Omega (\partial_t \rho + \operatorname{div}(\rho V))\, dx = 0.$$

Since this integral identity holds for every compact domain Ω, it follows that

$$\partial_t \rho + \operatorname{div}(\rho V) = 0,$$

which is precisely the continuity equation.

11.2.2 Conservation of Momentum

Euler's equation is a Newton-type equation in the dynamics of ideal fluids. It expresses the conservation of momentum and is written as:

$$\frac{\partial V}{\partial t} + (V \cdot \nabla)V = -\frac{1}{\rho}\nabla p + f, \tag{11.2.11}$$

where V is the fluid velocity field, p is the pressure, ρ is the fluid density, and f represents external forces.

The left-hand side of the equation consists of two components:

- *Temporal acceleration:* $\frac{\partial V}{\partial t}$, representing the rate of change of velocity with respect to time.
- *Convective acceleration:* $(V \cdot \nabla)V$, which accounts for changes in velocity due to spatial movement of the fluid. In Cartesian coordinates, this term expands as:

$$(V \cdot \nabla)V = V_1 \frac{\partial V}{\partial x_1} + V_2 \frac{\partial V}{\partial x_2} + V_3 \frac{\partial V}{\partial x_3}.$$

The right-hand side represents the forces acting on the fluid:

- *The pressure gradient force:* $-\frac{1}{\rho}\nabla p$.
- *External body forces:* such as gravity, f.

Equation (11.2.11) applies to incompressible flows under the additional constraint:

$$\operatorname{div} V = 0.$$

Furthermore, assuming constant density $\rho = 1$ (for simplicity, by choosing appropriate units), and neglecting external forces $f = 0$, the governing equations simplify to the following initial value problem:

Problem *Given a pressure field $p(t,x)$ and the initial velocity field $V_0(x)$, find the velocity field $V(t,x)$ satisfying:*

$$\begin{aligned} \frac{\partial V}{\partial t} + (V \cdot \nabla)V &= -\nabla p, \\ \operatorname{div} V &= 0, \\ V(0,x) &= V_0(x). \end{aligned}$$

In practice the pressure field is known via measurements $p_{ij} = p(t_i, x_j)$ with collocation points $(t_i, x_j) \in \mathcal{C}$. Using a neural network $p_\theta(t,x)$ we can learn the pressure field via previous measurements by minimizing the data loss

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{NM} \sum_{i,j} (p_\theta(t_i, x_j) - p_{ij})^2.$$

From now on we assume that $p(t,x)$ is known for any $(t,x) \in \mathcal{C}$.

There are several ways to approach the problem described above.

Method 1: Using soft constraints We define a neural network approximation for the velocity field:

$$V_\theta(t,x) = \left(V_\theta^1(t,x), V_\theta^2(t,x), V_\theta^3(t,x)\right),$$

where θ denotes the network parameters. Using automatic differentiation, we can compute the necessary partial derivatives to define two key residuals.

Equation residual:

$$R_{\text{eq}}(\theta; t, x) = \frac{\partial V_\theta}{\partial t} + (V_\theta \cdot \nabla)V_\theta + \nabla p.$$

Divergence residual:

$$R_{\text{div}}(\theta; t, x) = \operatorname{div} V_\theta(t,x) = \frac{\partial V_\theta^1}{\partial x_1} + \frac{\partial V_\theta^2}{\partial x_2} + \frac{\partial V_\theta^3}{\partial x_3}.$$

The total loss function combines these residuals with the initial condition:

$$\begin{aligned} \mathcal{L}_{\text{total}}(\theta) &= \frac{\lambda_1}{NM} \sum_{i,j} \|R_{\text{eq}}(\theta; t_i, x_j)\|^2 + \frac{\lambda_2}{NM} \sum_{i,j} \left(R_{\text{div}}(\theta; t_i, x_j)\right)^2 \\ &\quad + \frac{\lambda_3}{M} \sum_j \|V_\theta(0, x_j) - V_0(x_j)\|^2, \end{aligned}$$

where:

- $\{(t_i, x_j)\}$ are sampled training points in time-space,
- $\lambda_1, \lambda_2, \lambda_3$ are weighting hyperparameters,
- N and M are the number of time and space samples, respectively.

The solution to the problem is given by the neural network V_{θ^*}, where

$$\theta^* = \arg\min_{\theta} \mathcal{L}_{\text{total}}(\theta).$$

Method 2: Using hard embedding We begin by observing that the initial condition can be enforced directly by writing

$$V_\theta(t,x) = V_0(x) + t\,\text{nn}(\theta;t,x),$$

where $\text{nn}(\theta;t,x)$ is a neural network correction term.
Next, the divergence-free constraint can be embedded using a vector potential as

$$V_\theta(t,x) = \text{curl}F_\theta(t,x) = \nabla \times F_\theta(t,x),$$

where $F_\theta(t,x)$ is a vector-valued neural network. Our goal is to incorporate both embeddings simultaneously.

Since the velocity field is divergence-free, in particular $\text{div}V_0(x) = 0$. Hence, there exists a vector potential $A_0(x)$ such that

$$V_0(x) = \text{curl}A_0(x) = \nabla \times A_0(x).$$

Similarly, we introduce another vector-valued neural network $A_\theta(t,x)$ and require that

$$\text{nn}(\theta;t,x) = \text{curl}A_\theta(t,x) = \nabla \times A_\theta(t,x).$$

Using the linearity of the curl operator, we obtain

$$\begin{aligned} V_\theta(t,x) &= V_0(x) + t\,\text{nn}(\theta;t,x) = \nabla \times A_0(x) + t\,\nabla \times A_\theta(t,x) \\ &= \nabla \times \big(A_0(x) + tA_\theta(t,x)\big). \end{aligned}$$

Therefore, we may define

$$F_\theta(t,x) = A_0(x) + tA_\theta(t,x),$$

which simultaneously enforces the initial condition and the divergence-free constraint.

This approach can be carried out in two steps:
Step 1: Determining $A_0(x)$. We introduce a vector-valued neural network $A_0(\eta;x)$ and train it to recover the initial velocity field via its curl. Specifically, $A_0(\eta;x)$ is optimized by minimizing the loss

$$\mathcal{L}(\eta) = \int_\Omega \left\| V_0(x) - \text{curl}A_0(\eta;x) \right\|^2 dx.$$

Step 2: Determining $A_\theta(t,x)$. Once $A_0(x)$ has been obtained, the task reduces to learning $F_\theta(t,x) = A_0(x) + tA_\theta(t,x)$. To this end, we train the network A_θ using the loss

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{eq}}(\theta) = \frac{\lambda_1}{NM} \sum_{i,j} \left\| R_{\text{eq}}(\theta;t_i,x_j) \right\|^2,$$

where $R_{\text{eq}}(\theta;t_i,x_j)$ is the equation residual defined in Method 1.

Still under the assumptions of constant density ($\rho = 1$) and no external forces ($f = 0$), we may formulate a more general initial value problem for the Euler equations that can be solved using neural networks:

Problem *Given an initial pressure field $p_0(x)$ and an initial velocity field $V_0(x)$, find the pressure field $p(t, x)$ and velocity field $V(t, x)$ satisfying*

$$\begin{aligned} \frac{\partial V}{\partial t} + (V \cdot \nabla)V &= -\nabla p, \\ \operatorname{div} V &= 0, \\ V(0, x) &= V_0(x), \\ p(0, x) &= p_0(x). \end{aligned}$$

This problem can be addressed in an *unsupervised* manner, i.e., without requiring labeled solution data, by employing a neural network of the form

$$NN(\theta; t, x) = \big(V_\theta(t, x),\, p_\theta(t, x)\big),$$

where θ denotes the trainable parameters. The network is trained by minimizing the total loss

$$\begin{aligned} \mathcal{L}_{\text{total}}(\theta) &= \frac{\lambda_1}{NM} \sum_{i,j} \big\| R_{\text{eq}}(\theta; t_i, x_j) \big\|^2 + \frac{\lambda_2}{NM} \sum_{i,j} \big(R_{\text{div}}(\theta; t_i, x_j) \big)^2 \\ &\quad + \frac{\lambda_3}{M} \sum_{j} \big\| V_\theta(0, x_j) - V_0(x_j) \big\|^2 + \frac{\lambda_4}{M} \sum_{j} \big\| p_\theta(0, x_j) - p_0(x_j) \big\|^2, \end{aligned}$$

where $(t_i, x_j) \in \mathcal{C}$ are collocation points in the space-time domain.

Implementation example We assume the 2-dimensional case when the initial velocity is the divergence-free Taylor-Green vortex

$$V_0(x, y) = (\sin(\pi x)\cos(\pi y), -\cos(\pi x)\sin(\pi y)),$$

and the initial pressure is given by

$$p_0(x, y) = \cos(\pi x)\cos(\pi y).$$

The implementation follows the soft constrains method and code details can be found at:

`https://tinyurl.com/56hrh529`

The solution is represented as a series of heat maps for $V = (u, v)$ and p at several time instances, $t \in \{0.5, 2.0, 4.0, 6.0\}$, see Fig. 11.4. We note how the minimum of the pressure (the blue disk) evolves over time.

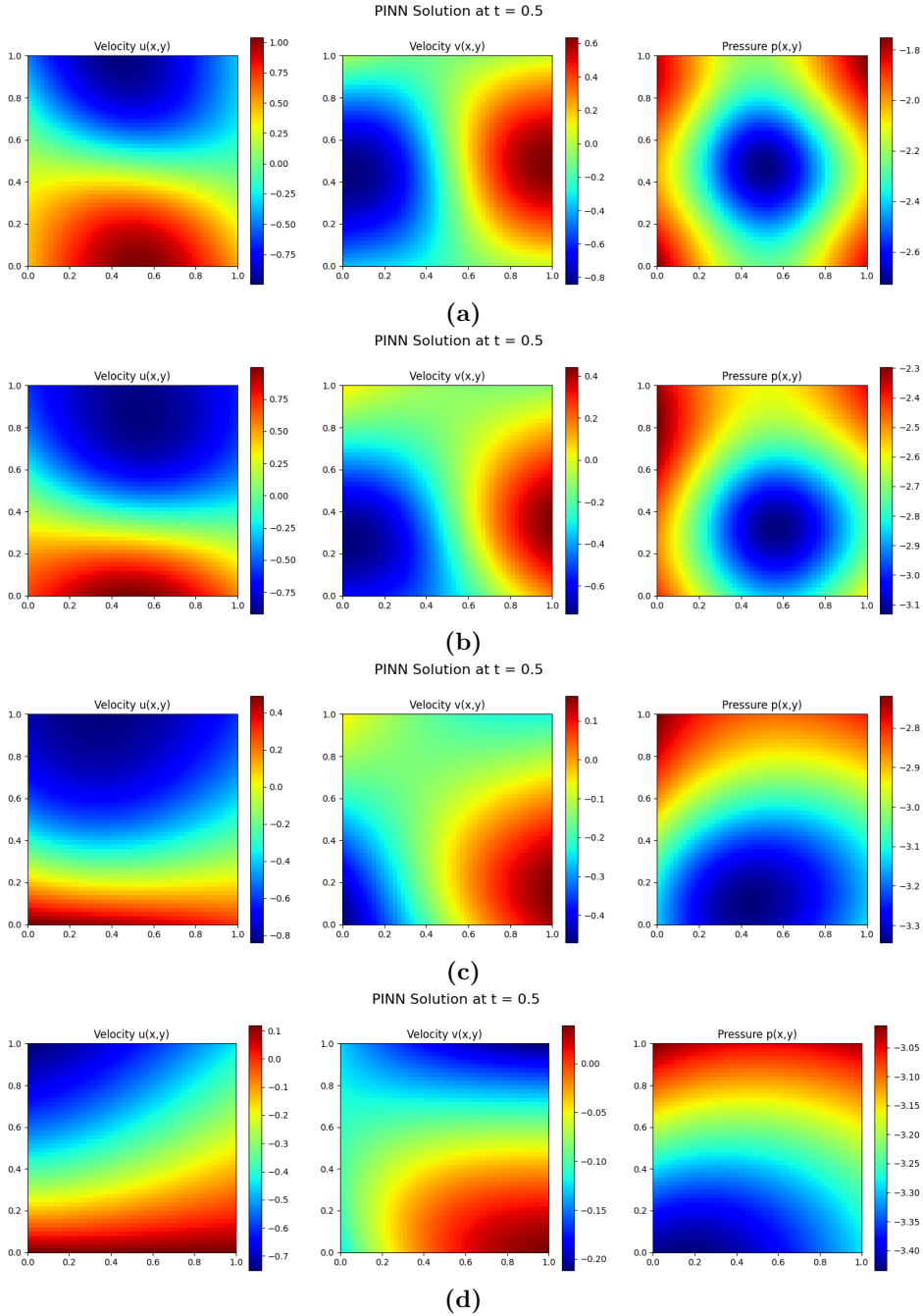


Figure 11.4: Euler's equation plot of heat maps for u, v and p, where $V = (u, v)$, at several time instances: (a) $t = 0.5$; (b) $t = 2.0$; (c) $t = 4.0$; (d) $t = 6.0$.

Physical interpretation We define the *material derivative* along the velocity field V as the operator

$$\frac{D_V}{Dt} = \frac{\partial}{\partial t} + (V \cdot \nabla). \tag{11.2.12}$$

For a smooth function $f(t,x)$, this gives

$$\begin{aligned} \frac{D_V f}{Dt} &= \frac{\partial f}{\partial t} + (V \cdot \nabla) f \\ &= \frac{\partial f}{\partial t} + \sum_i V^i \, \partial_{x_i} f \\ &= \frac{\partial f}{\partial t} + \langle V, \nabla f \rangle. \end{aligned}$$

If $c(t)$ is an integral curve of the vector field V, then this becomes

$$\frac{D_V}{Dt} f(t, c(t)) = \frac{\partial f}{\partial t} + \sum_i \dot{c}^i(t) \, \partial_{x_i} f = \frac{d}{dt} f(t, c(t)).$$

Thus, the material derivative represents the *total derivative* of f along the flow of V.

Next, let us compute the material derivative of the density $\rho(t,x)$. From the continuity equation we have

$$\begin{aligned} 0 &= \frac{\partial \rho}{\partial t} + \operatorname{div}(\rho V) \\ &= \frac{\partial \rho}{\partial t} + \langle V, \nabla \rho \rangle + \rho \operatorname{div} V \\ &= \frac{D_V \rho}{Dt} + \rho \operatorname{div} V, \end{aligned}$$

so that

$$\frac{D_V \rho}{Dt} = -\rho \operatorname{div} V. \tag{11.2.13}$$

In fluid mechanics, a fluid is called *incompressible* if its density is constant following a fluid particle

$$\frac{D_V \rho}{Dt} = 0.$$

Then from (11.2.13) if follows that the incompressibility can be written equivalently as $\operatorname{div} V = 0$, relation which has been used before.

Now consider the momentum of a fluid parcel Ω at time t:

$$M(t; \Omega) = \int_\Omega \rho V \, dx.$$

Its material derivative is

$$\begin{aligned}\frac{D_V}{Dt}M(t;\Omega) &= \int_\Omega \frac{D_V}{Dt}(\rho V)\,dx \\ &= \int_\Omega \frac{D_V\rho}{Dt}\,V\,dx + \int_\Omega \rho\frac{D_V V}{Dt}\,dx \\ &= \int_\Omega \rho\frac{D_V V}{Dt}\,dx,\end{aligned}$$

where in the last step we used $\frac{D_V\rho}{Dt} = -\rho\,\mathrm{div}\,V = 0$ for incompressible flow.

By Newton's second law, the rate of change of momentum of the parcel Ω equals the sum of the forces acting on it. These are:

1. *Pressure forces*, acting through the boundary $\partial\Omega$, given by

$$-\int_{\partial\Omega} p\,\nu\,d\sigma = -\int_\Omega \nabla p\,dx,$$

where we used the divergence theorem. The negative sign indicates that the pressure force acts opposite to the outward normal ν.

2. *External forces*, such as gravity or electromagnetic fields, acting throughout Ω, given by

$$\int_\Omega f(t,x)\,dx.$$

Equating the rate of change of momentum with the total force gives

$$\int_\Omega \rho\frac{D_V V}{Dt}\,dx = -\int_\Omega \nabla p\,dx + \int_\Omega f(t,x)\,dx.$$

Since this holds for every domain Ω, we conclude that the integrands must coincide:

$$\rho\frac{D_V V}{Dt} = -\nabla p + f(t,x).$$

Finally, dividing by ρ and using the definition of the material derivative, we recover *Euler's equation* (11.2.11).

Remark 11.2.1 If the fluid is not ideal, the effect of viscosity must be taken into account. In this case, an additional term appears in Euler's equation, leading to the celebrated *Navier–Stokes equations* for an incompressible viscous fluid:

$$\frac{\partial V}{\partial t} + (V\cdot\nabla)V = -\frac{1}{\rho}\nabla p + \nu\Delta V + f, \quad \mathrm{div}V = 0,$$

where ν is the kinematic viscosity of the fluid, Δ is the Laplacian operator, and f denotes external body forces.

Compared to Euler's equation, the new term $\nu\Delta V$ accounts for viscous diffusion of momentum inside the fluid. This term is crucial for describing real-world phenomena such as turbulence, boundary layers, and energy dissipation.

Despite its apparent simplicity, the Navier–Stokes system in three dimensions remains one of the most challenging problems in mathematics. A central open question is whether smooth (globally defined and infinitely differentiable) solutions always exist for all time, given smooth initial data, or whether singularities (blow-ups) can develop in finite time. This question is so fundamental that it has been named one of the *Clay Millennium Prize Problems*, and a complete resolution is awarded with a prize of $1 million by the Clay Mathematics Institute.

11.2.3 The State Equation

The relation between the pressure and density of an ideal fluid is given by the *state equation*

$$\Phi(\rho, p) = 0.$$

In cases where $\partial_p \Phi \neq 0$, the Implicit Function Theorem guarantees that the state equation can be locally solved in the form $p = p(\rho)$. A particularly important situation arises in the case of adiabatic processes, where no heat is exchanged with the surroundings. In this setting, the state equation takes the explicit form

$$p = c\rho^{\gamma},$$

with c a positive constant determined by the initial conditions of the fluid, and $\gamma = c_p/c_v$ denoting the adiabatic index, i.e., the ratio between the specific heat at constant pressure and the specific heat at constant volume. This relation reflects the compressibility of the fluid under adiabatic transformations and plays a fundamental role in modeling compressible flows.

Remark 11.2.2 In more general thermodynamic contexts, the state equation can also involve the temperature, leading to the well-known ideal gas law $p = \rho RT$, which provides a more complete description of the fluid's thermodynamic behavior.

Using neural networks A neural network can learn the equation of state by treating it as a function-approximation problem: it takes pressure as input and predicts density. Training is performed by minimizing the discrepancy between predictions and measurements. More precisely, let $\rho_{ij} = \rho(t_i, x_j)$ and $p_{ij} = p(t_i, x_j)$ denote $N \times M$ measurements at collocation times $\{t_i\}$ and spatial locations $\{x_j\}$.

Since we seek a functional relation $p = F(\rho)$, we approximate the unknown mapping F using a neural network $NN(\theta; \rho)$ with parameters θ and input ρ. Training is carried out in a supervised manner with inputs ρ_{ij} and corresponding labels p_{ij}. The loss function penalizes the difference between predicted and measured densities:

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{NM} \sum_{i,j} \Big(NN(\theta; \rho_{ij}) - p_{ij} \Big)^2.$$

The previous purely data-driven approach yields an empirical relation between ρ and p. However, it is also a well-established physical fact that pressure increases with density. This prior knowledge can be incorporated into the learning process by augmenting the loss function with an additional term that enforces monotonicity. In particular, we require the neural network approximation to satisfy the constraint that density is a non-decreasing function of pressure. Since the state equation is approximated as $p = NN(\theta; \rho)$, we require

$$\frac{dp}{d\rho} \geq 0.$$

This condition can be incorporated in the loss function as an additional penalty term:

$$\mathcal{L}_{\text{mono}}(\theta) = \frac{1}{NM} \sum_{i,j} \max\left(0, -\frac{d}{d\rho} NN(\theta; \rho_{ij})\right)^2.$$

The total loss is then defined as

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \lambda_{\text{mono}} \mathcal{L}_{\text{mono}}(\theta),$$

where $\lambda_{\text{mono}} > 0$ is a tunable hyperparameter controlling the strength of the monotonicity constraint. Once trained, the neural network serves as a surrogate model for the state equation.

11.2.4 One-Dimensional Adiabatic Process

We consider an adiabatic process for an ideal fluid confined in a long, thin tube, so that the motion can be modeled using a single spatial coordinate x. Let $v(t, x)$, $\rho(t, x)$, and $p(t, x)$ denote the velocity, density, and pressure, respectively, at position x and time t. The initial state of the gas is prescribed by

$$v(x, 0) = v_0(x), \qquad \rho(x, 0) = \rho_0(x), \qquad p(x, 0) = p_0(x).$$

The system admits solutions only when certain compatibility conditions on the initial data are satisfied, which we shall determine later.

We assume that there is no heat exchange with the environment, so the process is adiabatic, and that the ideal fluid is incompressible. Under these hypotheses, the objective is to determine the velocity, density, and pressure of the fluid at any future time.

The problem can be approached using neural networks in an unsupervised manner, and the resulting solution can be validated against the exact analytical solution, which is available in this case.

Explicit dynamics The dynamics of the fluid are obtained from the continuity equation, Euler's equation, the equation of state, and the incompressibility condition. In one spatial dimension, these equations take a particularly simple form.

• The *incompressibility condition* $\operatorname{div} v = 0$ becomes $\partial_x v(t, x) = 0$, which implies that the velocity is spatially homogeneous, $v(t, x) = v(t)$. Thus the velocity depends only on time, and the initial condition is $v(x, 0) = v_0$, a constant.

- The *continuity equation* $\partial_t \rho + \partial_x(\rho v) = 0$ reduces to

$$\partial_t \rho + v(t)\, \partial_x \rho = 0. \tag{11.2.14}$$

- The *Euler equation* $\partial_t v + v\partial_x v + \frac{1}{\rho}\, \partial_x p = 0$ simplifies to

$$v'(t) + \frac{1}{\rho}\, \partial_x p = 0. \tag{11.2.15}$$

- The *equation of state* for an adiabatic process is $p = c\,\rho^\gamma$, with $c > 0$ and $1 < \gamma < 2$. Differentiating gives

$$\partial_x p = c\gamma \rho^{\gamma-1}\, \partial_x \rho.$$

Substituting into (11.2.15) yields

$$v'(t) + c\gamma \rho^{\gamma-2}\, \partial_x \rho = 0. \tag{11.2.16}$$

Eliminating $\partial_x \rho$ between (11.2.14) and (11.2.16), we obtain

$$v(t)\, v'(t) = c\gamma \rho^{\gamma-2}\, \partial_t \rho.$$

This can be written as an exact derivative in t:

$$\partial_t \Big(\tfrac{1}{2} v(t)^2 - \tfrac{c\gamma}{\gamma-1}\, \rho^{\gamma-1} \Big) = 0.$$

Defining

$$F(t,x) = \tfrac{1}{2} v(t)^2 - \tfrac{c\gamma}{\gamma-1}\, \rho^{\gamma-1}, \tag{11.2.17}$$

we obtain

$$\begin{aligned} \partial_x F &= \partial_x \Big(\tfrac{1}{2} v(t)^2 - \tfrac{c\gamma}{\gamma-1}\, \rho^{\gamma-1} \Big) \\ &= -c\gamma \rho^{\gamma-2}\, \partial_x \rho \\ &= v'(t), \end{aligned}$$

where we used (11.2.16). Thus, the system of partial differential equations governing the dynamics is

$$\partial_t F = 0, \qquad \partial_x F = v'(t).$$

From $\partial_t F = 0$ we get that F is independent of t, hence $F(t,x) = G(x)$ for some function G. Plugging this into $\partial_x F = v'(t)$ gives

$$\frac{d}{dx} G(x) = v'(t).$$

The left-hand side depends only on x, the right-hand side only on t; therefore each side must be a constant. Denote this constant by $C \in \mathbb{R}$. Thus

$$v'(t) = C \quad \Longrightarrow \quad v(t) = v_0 + Ct, \tag{11.2.18}$$

and

$$\frac{d}{dx}G(x) = C \quad \Longrightarrow \quad F(t,x) = G(x) = Cx + K,$$

for some constants $v_0, K \in \mathbb{R}$.
In particular, with the definition (11.2.17) we obtain the explicit relation

$$\frac{1}{2}\big(v_0 + Ct\big)^2 \; - \; \frac{c\gamma}{\gamma - 1}\,\rho(t,x)^{\gamma-1} \; = \; Cx + K.$$

Solving for density we obtain

$$\rho(t,x) \; = \; \left[\frac{\gamma-1}{c\gamma}\left(\tfrac{1}{2}(v_0+Ct)^2 - Cx - K\right)\right]^{1/(\gamma-1)}, \tag{11.2.19}$$

whenever the bracket is positive. The constants C and K are fixed by the initial data via

$$\frac{1}{2}v_0^2 - \frac{c\gamma}{\gamma-1}\,\rho_0(x)^{\gamma-1} = Cx + K, \tag{11.2.20}$$

which shows that a necessary compatibility for arbitrary x is that the combination on the left be affine in x (equivalently, $\rho_0(x)$ is of the form $[A - Bx]^{1/(\gamma-1)}$).

To conclude, we have:

Proposition 11.2.3 *A solution exists only when the initial data satisfy the following compatibility conditions:*

$$\begin{aligned} v_0(x) &= v_0 \\ \rho_0(x) &= [A - Bx]^{1/(\gamma-1)} \\ p_0(x) &= c[A - Bx]^{\gamma/(\gamma-1)}, \end{aligned}$$

with v_0, A and B constants. In this case the solutions are given by (11.2.18) and (11.2.19), with constants C and K determined by relation (11.2.20).

The use of neural networks The problem can also be approached using neural network methods. To enforce the divergence-free condition on the velocity field $v(t,x)$, one may embed this property directly into the representation by restricting to spatially independent velocity fields of the form $v(t,x) = v(t)$. The solution is then approximated by a neural network with three outputs,

$$NN(\theta; t, x) = \big(v_\theta(t),\, \rho_\theta(t,x),\, p_\theta(t,x)\big),$$

where θ denotes the trainable parameters. The initial conditions are encoded into the network architecture as follows:

$$\begin{aligned} v_\theta(t) &= v_0 + t\,\mathrm{nn}^1(\theta; t), \\ \rho_\theta(t,x) &= \rho_0(x) + t\,\mathrm{nn}^2(\theta; t, x), \\ p_\theta(t,x) &= p_0(x) + t\,\mathrm{nn}^3(\theta; t, x). \end{aligned}$$

The parameters θ are trained to minimize a composite loss function,

$$\mathcal{L}_{\text{total}}(\theta) = \lambda_1 \mathcal{L}_{\text{cont}}(\theta) + \lambda_2 \mathcal{L}_{\text{Euler}}(\theta) + \lambda_3 \mathcal{L}_{\text{state}}(\theta) + \lambda_4 \mathcal{L}_{\text{data}}(\theta),$$

where the different terms correspond to the continuity equation, Euler's equations, the equation of state, and data measurements respectively, as follows

$$\begin{aligned}
\mathcal{L}_{\text{cont}}(\theta) &= \int_0^L \int_0^T (\partial_t \rho_\theta(t,x) + v_\theta(t) \partial_x \rho_\theta(t,x))^2 dt dx \\
\mathcal{L}_{\text{Euler}}(\theta) &= \int_0^L \int_0^T (\rho_\theta(t,x) v'_\theta(t) + \partial_x p_\theta(t,x))^2 dt dx \\
\mathcal{L}_{\text{state}}(\theta) &= \int_0^L \int_0^T (p_\theta(t,x) - c\rho_\theta^\gamma(t,x))^2 dt dx.
\end{aligned}$$

The *data loss* introduces a soft constraint that incorporates information from available measurements of velocity, pressure, or density. For example, if measurements of velocity $v_j = c(t_j)$ are taken at time instances t_j, the corresponding loss can be written as

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{m} \sum_{j=1}^{m} \big(v_\theta(t_j) - v_j\big)^2.$$

When the weight $\lambda_4 = 0$, the model operates in a purely *unsupervised* mode, relying solely on the governing equations. In contrast, when $\lambda_4 \neq 0$, the model leverages both physics and data, resulting in a *hybrid* learning scheme.

In the next implementation example, we will explore both approaches.

Implementation example We focus on a particular case where the model parameters are chosen as

$$v_0 = 1, \quad c = 1, \quad \gamma = 1.8, \quad A = 2, \quad B = 0.5.$$

The neural network architecture consists of three hidden layers, each with 64 neurons and `swish` activation functions. Training is performed for 6,000 epochs using the Adam optimizer with a small learning rate of 3×10^{-5}.

The weights assigned to the individual loss terms are

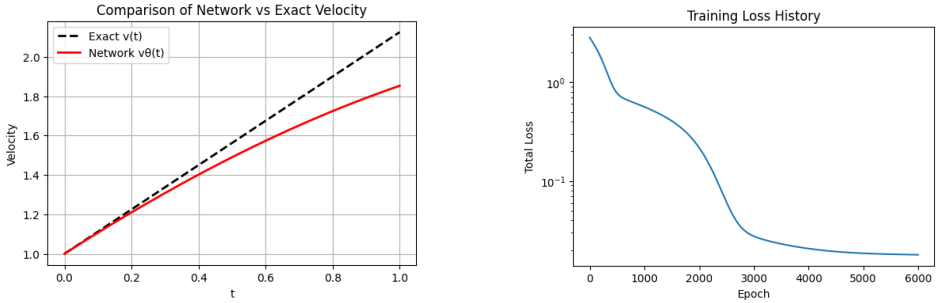
$$\lambda_1 = \lambda_2 = \lambda_3 = 1.$$

First, by setting $\lambda_4 = 0$, we train the neural network in an *unsupervised mode* and compare the predicted solution $v(t)$ with the exact solution over the interval $t \in [0, 1]$. As shown in Fig. 11.5(a), the error grows as t approaches 1.

Training is carried out until the loss reaches saturation. Therefore, the discrepancy between the two solutions cannot be attributed to undertraining, see Fig. 11.5(b).

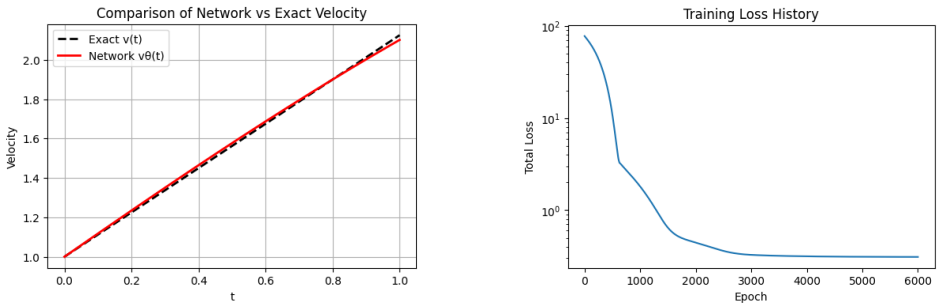
Next, we set $\lambda_4 = 50$ and train the network in a *hybrid mode*, incorporating two observational constraints: $v_1 = v_{\text{exact}}(0.5)$ and $v_2 = v_{\text{exact}}(1)$. In this case, the neural network prediction and the exact velocity solution coincide to within visual indistinguishability, as illustrated in Figs. 11.6(a)–(b).

The reader can find the code detail at `https://tinyurl.com/4nwjupwc`.



(a) **(b)**

Figure 11.5: An unsupervised training: (a) The velocities $v_{\theta^*}(t)$ and $v_{\text{exact}}(t)$ are not the same; (b) The associated total loss.



(a) **(b)**

Figure 11.6: A hybrid training: (a) The velocities $v_{\theta^*}(t)$ and $v_{\text{exact}}(t)$ are indistinguishable; (b) The associated total loss.

Conclusions We conclude this section with a few remarks. A purely unsupervised learning strategy, even when applied to a well-posed initial value problem for fluid dynamics, is unable to maintain accuracy for larger values of time t. In practice, this manifests as an accumulation of errors as time progresses, leading to a visible departure between the neural network prediction and the exact solution.

Theoretically, this behavior can be understood in terms of error propagation: small inaccuracies in the representation of the velocity or density fields, though negligible at early times, become amplified as the solution evolves. In particular, the dynamics are governed by nonlinear equations, so that perturbations do not merely accumulate additively but may grow in a compounded fashion, resulting in degraded accuracy for larger t. This explains why unsupervised training, relying solely on PDE residual minimization, cannot fully eliminate long-term drift from the true solution.

By contrast, when observational data are incorporated into the training process, the resulting *hybrid approach* leverages both the governing physical laws and the available measurements. The data act as "anchors" at selected times or locations, effectively

resetting accumulated errors and constraining the network to remain close to the physically correct trajectory. This method dramatically improves the quality of the learned solution; in our experiments, the hybrid model produces results that are essentially indistinguishable from the true solution, even at longer times.

A few bibliographical notes for further reading are as follows. The classic text by Landau and Lifshitz [58] presents rigorous derivations of the continuity equation, Euler's equations, and the equation of state, including the adiabatic law, and remains a cornerstone of theoretical fluid mechanics.

The well-known book by Liepmann and Roshko provides a foundational reference on compressible and adiabatic flows, offering clear and accessible derivations of Euler's and continuity equations in the context of high-speed gas dynamics.

A mathematical survey of the compressible Euler equations, with emphasis on well-posedness, the formation of shocks, and the role of the adiabatic exponent in stability, is given in the paper by Christodoulou [25].

For a pedagogical introduction oriented toward astrophysical applications, the article by Trac and Pen [94] reviews Eulerian fluid dynamics, including the continuity and Euler equations under adiabatic processes, together with a discussion of numerical methods.

The applications of Deep Learning in fluid dynamics have been explored in several recent works. Alhafiz et al. [4] propose an explainable Deep Learning framework for turbulence modeling, demonstrating how neural networks can identify and interpret physically meaningful features in turbulent flows. Yao et al. [101] apply Deep Learning techniques to optimize cooling efficiency in turbomachinery, effectively combining CFD simulations with data-driven prediction. Li and Hu [60] introduce a physics-informed recurrent neural network (LSTM) for the simulation of nonlinear fluid dynamics, thereby bridging temporal sequence learning with PDE-based constraints. Finally, Wang et al. [98] investigate deep learning–assisted modeling of multiphase boiling processes, showing how hierarchical neural architectures can enhance the description of heat transfer dynamics.

11.3 Exercises

Exercise 11.3.1 Consider a dynamical system described by the initial value ODE

$$\ddot{x} + 2\dot{x} + 1 = 0, \qquad x(0) = 0, \qquad \dot{x}(0) = 1.$$

(a) Use a neural network to learn the coefficients of the equation.

(b) Use a neural network to learn a function F such that $F(t, x, \dot{x}) = C$ and estimate the value of the constant C.

Exercise 11.3.2 Consider the dynamical system given by the initial value problem

$$t\ddot{x} + x\dot{x} + \dot{x} = 0, \qquad x(0) = 0, \qquad \dot{x}(0) = 1.$$

(a) Use a neural network to learn the coefficients of the differential equation.

(*b*) Use a neural network to learn a function F such that

$$F(t, x, \dot{x}) = C,$$

and estimate the value of the constant C.

Exercise 11.3.3 (*a*) Assume $\rho(t, x)$ is constant. Use the continuity equation to show that $\operatorname{div} V = 0$.

(*b*) Assume the density $\rho(t, x) = \rho(x)$ is independent of time. Show that any divergence-free vector field V is tangent to the surfaces of constant density, i.e. to $\rho^{-1}\{c\}$ for $c \in \mathbb{R}$.

(*c*) Assume the density is spatially homogeneous, i.e. $\rho(t, x) = \rho(t)$. Show that $\operatorname{div} V = \dfrac{d}{dt} \ln \rho(t)$. Conclude that ρ becomes constant if V is divergence-free.

Exercise 11.3.4 (*a*) Assume $\rho(t, x)$ is constant. Use the continuity equation to show that $\operatorname{div} V = 0$.

(*b*) Assume the density $\rho(t, x) = \rho(x)$ is independent of time. Show that any divergence-free vector field V is tangent to the surfaces of constant density, i.e. to $\rho^{-1}\{c\}$ for $c \in \mathbb{R}$.

(*c*) Assume the density is spatially homogeneous, i.e. $\rho(t, x) = \rho(t)$. Show that $\operatorname{div} V = \dfrac{d}{dt} \ln \rho(t)$. Conclude that ρ becomes constant if V is divergence-free.

Exercise 11.3.5 The heat maps in Fig. 11.4 were obtained by solving the two-dimensional Euler equations with prescribed initial conditions, using a neural network trained with a loss function based on *soft constraints*.

(*a*) Redo the experiment by implementing the approach with *hard embeddings*, as described in the text. Recompute the neural network approximation of the solution and redraw the corresponding heat maps at the same time instances as in Fig. 11.4.

(*c*) Compare your results with the heat maps obtained using soft constraints, and comment on the differences in accuracy, stability, and physical consistency of the solutions.

Chapter 12

Parameter Estimation

The parameter estimation problem can be formulated as an *inverse problem*, in which the goal is to determine an unknown parameter given two sources of information:

(i) A mathematical model (e.g., a differential equation) that depends on one or more unknown parameters, and

(ii) A body of data obtained from direct measurements, which obeys the model.

The task is then to use suitable methods—in our case, neural networks—to recover the missing parameters.

We illustrate this approach through several examples, beginning with the simplest case and progressing toward more complex parameter estimation problems.

12.1 Newton's Cooling Law

As a first example, we consider the problem of estimating the thermal decay constant k when the governing model is Newton's law of cooling. The corresponding ordinary differential equation is

$$\begin{aligned} u'(t) &= -k^2\big(u(t) - T\big), \\ u(0) &= u_0, \end{aligned}$$

where $u(t)$ denotes the temperature of the body at time t, u_0 is its initial temperature, and T is the constant ambient temperature.

In addition to the model, we assume that experimental data are available in the form of N temperature measurements,

$$u_j = u(t_j), \qquad j = 1, \ldots, N,$$

taken at discrete times $\{t_j\}$. The objective is to infer the value of k^2 from these observations using a neural network framework.

The parameter k can, in principle, be determined exactly if the closed-form solution of the problem is available. However, in practice this is not always the case. To address this, we shall consider three different approaches:

1. Using the exact analytical solution for $u(t)$ and estimating the parameter directly from it;

2. Employing a neural network with soft constraints, where k is inferred by minimizing a composite loss function that balances the ODE residual with data fitting;

3. Treating k as an internal trainable parameter of the network, while using a hard embedding of the initial condition to enforce $u(0) = u_0$ exactly.

We shall discuss each of them as follows.

1. First Variant: Using the closed form formula. The first variant is to solve the differential equation explicitly. Its solution is

$$u(t) = T + (u_0 - T)e^{-k^2 t}.$$

Substituting $t = t_j$ into this expression gives N relations:

$$u_j = T + (u_0 - T)e^{-k^2 t_j}, \quad j = 1, \ldots, N.$$

If k_j^2 denotes the value obtained from the j-th relation, then

$$k_j^2 = \frac{1}{t_j} \ln(u_0 - T) \; - \; \frac{1}{t_j} \ln(u_j - T).$$

Thus, each measurement u_j produces an explicit estimate k_j^2 for the decay constant. Since all partial estimates are equally important, we weight them uniformly. The final estimate of the constant is then taken as the average:

$$k^2 = \frac{1}{N} \sum_{j=1}^{N} k_j^2 = \frac{\ln(u_0 - T)}{N} - \frac{1}{N} \sum_{j=1}^{N} \frac{\ln(u_j - T)}{t_j}.$$

2. Second Variant: Simultaneous learning of $u(t)$ and k^2. In this variant we consider a two-branch neural network with input t,

$$NN(\theta; t) = \big(u_\theta(t),\ k_\theta(t)\big),$$

where the first branch learns the solution $u(t)$ of Newton's cooling law, and the second branch learns the parameter k.

The ODE loss enforces the differential equation:

$$\begin{aligned} \mathcal{L}_{\text{ODE}}(\theta) &= \int_0^\tau \left(\frac{d}{dt} u_\theta(t) + k_\theta(t)^2 \left(u_\theta(t) - T \right) \right)^2 dt \\ &\approx \frac{\tau}{N} \sum_{j=1}^{N} \left(\frac{du_\theta(t_j)}{dt} + k_\theta(t_j)^2 \left(u_\theta(t_j) - T \right) \right)^2, \end{aligned}$$

where $\tau = \max\{t_j\}$.

The initial condition is enforced through

$$\mathcal{L}_{\mathrm{IC}}(\theta) = \big(u_\theta(0) - u_0\big)^2,$$

while the data loss is given by

$$\mathcal{L}_{\mathrm{data}}(\theta) = \frac{1}{N}\sum_{j=1}^{N}\big(u_\theta(t_j) - u_j\big)^2.$$

To enforce that the second branch learns a constant parameter, we minimize the variance of $k_\theta(t)$ over the training points:

$$L_k(\theta) = \frac{1}{N}\sum_{j=1}^{N}\Big(k_\theta(t_j) - \bar{k}_\theta\Big)^2, \qquad \bar{k}_\theta = \frac{1}{N}\sum_{j=1}^{N} k_\theta(t_j).$$

The composite total loss is then

$$\mathcal{L}_{\mathrm{tot}}(\theta) = \mathcal{L}_{\mathrm{data}}(\theta) + \mathcal{L}_{\mathrm{ODE}}(\theta) + \mathcal{L}_{\mathrm{IC}}(\theta) + L_k(\theta).$$

Finally, if

$$\theta^* = \arg\min_\theta \mathcal{L}_{\mathrm{tot}}(\theta),$$

then the estimated decay constant is $k \approx k_{\theta^*}$.

3. Third Variant: Internal Parameter Learning. In this approach, we treat the unknown parameter k as an internal trainable parameter of the network. Let $w = (\theta, k)$ denote the augmented parameter set, where θ are the standard weights of the neural network and k is the additional scalar parameter to be learned.

The network is constructed so that it automatically satisfies the initial condition, by embedding it directly into the architecture as

$$u_w(t) \;=\; u_0 + t\, nn_w(t),$$

where $nn_w(t)$ is an unconstrained neural network.

To enforce the governing Newton's law, we define the total loss as the squared residual of the ODE:

$$\mathcal{L}_{\mathrm{tot}}(w) \;=\; \int_0^\tau \left(\frac{d}{dt}u_w(t) + k^2\big(u_w(t) - T\big)\right)^2 dt.$$

Since

$$\frac{du_w}{dt} \;=\; nn_w(t) + t\,\frac{d}{dt}nn_w(t),$$

this loss can be computed directly using automatic differentiation. Thus, the optimization simultaneously learns the neural representation of $u(t)$ and the parameter k, without requiring additional soft constraints.

Practical Implementations We shall use the same example introduced at page 65, with $u_0 = 90$ and $T = 30$. The decay constant satisfies $k^2 = 0.5$ and it is subject to be recovered from the experimental measurements provided by the Table 2.1. In this case $N = 9$ and $\tau = 10$. We shall implement each of the versions previously introduced theoretically. The code for all three variants can be found at `https://tinyurl.com/ad4ynamp`

1. First Variant: Using the closed form formula. The first variant produces the following individual estimations:

```
Individual k^2 estimates: [0.5006 0.5005 0.5006 0.5006 0.4996
                           0.4998 0.5007 0.5007 0.5011]
```

which leads to the estimation of k^2:

```
Estimated k^2 (average): 0.5005
```

This estimation method is very accurate and straightforward to apply, since it relies on a closed-form formula. However, it comes with several important limitations. First, closed-form solutions for the governing differential equations are not always available, and even when they are, it is not always possible to isolate the parameter explicitly. Moreover, the method becomes impractical in situations where more than one parameter must be estimated simultaneously.
The results obtained with this approach are illustrated in Fig. 12.1.

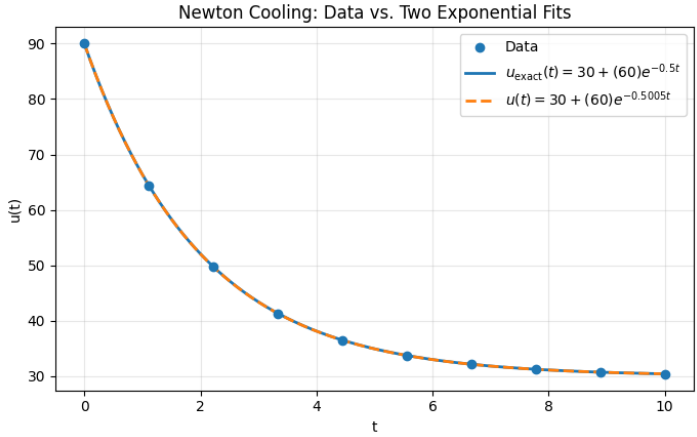


Figure 12.1: The first variant: The exact solution corresponding to $k = 0.5$ and the solution corresponding to $k = 0.5005$. In this case no neural network was used.

2. Second Variant: Simultaneous learning of $u(t)$ and k^2. The second variant simultaneously learns both the solution $u_\theta(t)$ and the parameter $k_\theta(t)^2$. Since the constant must satisfy $k^2 > 0$, we apply the `softplus` activation, with a small epsilon added to avoid exact zero:

```
k_net_raw = mlp(in_dim=1, width=32, depth=3, act="swish", out_dim=1)
softplus = tf.nn.softplus
```

```
def u_theta(t):
    return u_net_raw(t)

def k_theta(t):
    return softplus(k_net_raw(t)) + 1e-6
```

The total loss is defined as

```
L_tot = w_data*L_data + w_ode*L_ode + w_ic*L_ic + w_kvar*L_k
```

with weights chosen as:

```
w_data = 100.0
w_ode  = 1.0
w_ic   = 100.0
w_kvar = 200.0
```

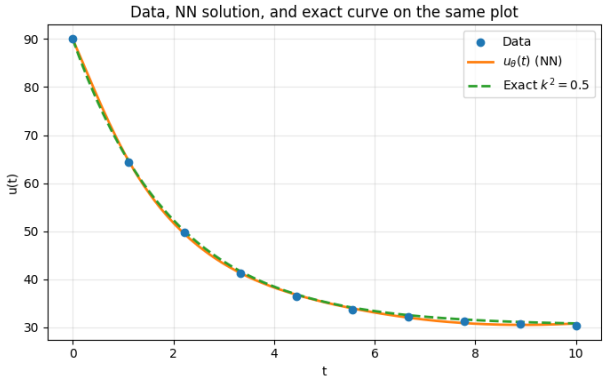
The network is trained using Adam with a learning rate of `1e-2` for 400 epochs. The architecture is a feed-forward network with three hidden layers and 64 neurons in each layer. The resulting estimate was highly accurate:

```
Estimated k^2: 0.491827
```

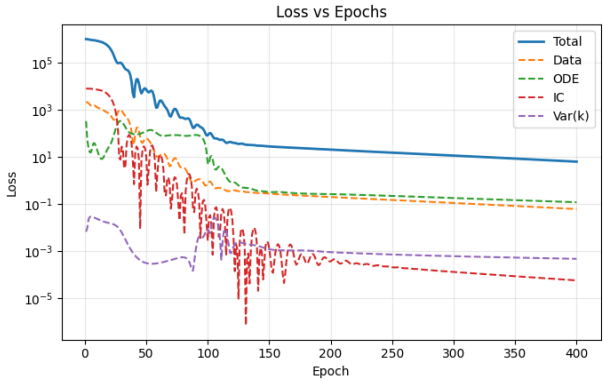
Figure 12.2(a) shows that the neural network reproduces the exact solution with very good accuracy. Figure 12.2(b) contains the losses evolution with respect to the number of epochs. Figure 12.2(c) illustrates the evolution of the estimated k^2 across training epochs. We note that after roughly 400 epochs the estimate stabilizes at the correct target value of 0.5.

Even though this approach may appear to be a clever way of estimating the constant k^2, in practice it tends to be unstable. Running the method multiple times with the same hyperparameters can lead to noticeably different results. Moreover, the accuracy of the estimates is quite sensitive to variations in the learning rate and the relative weights assigned in the loss function.

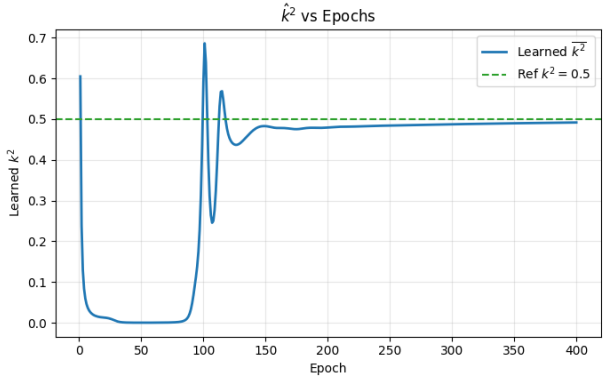
3. Third Variant: Internal Parameter Learning. The third approach treats the unknown parameter k as an internal trainable parameter of the network. Figure 12.3(a) shows that the neural network successfully learns the exact solution, while Fig. 12.3(c) illustrates the evolution of the parameter k^2 during training. We observe that the estimate converges steadily toward the true value of 0.5. After 6,000 epochs, the learned value stabilizes at $k^2 \approx 0.503$.
This method proves to be highly robust: it converges smoothly, without oscillations, and consistently produces reliable results. For these reasons, we recommend this approach as the preferred method going forward.



(a)

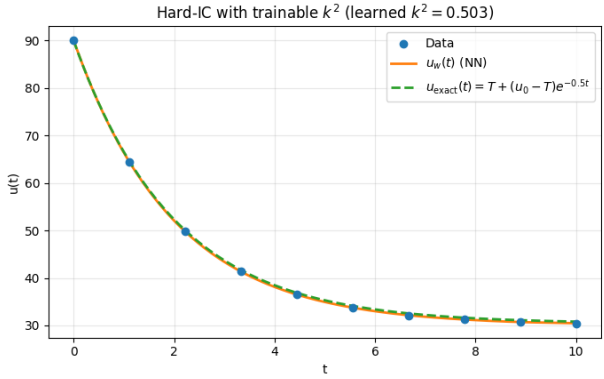


(b)

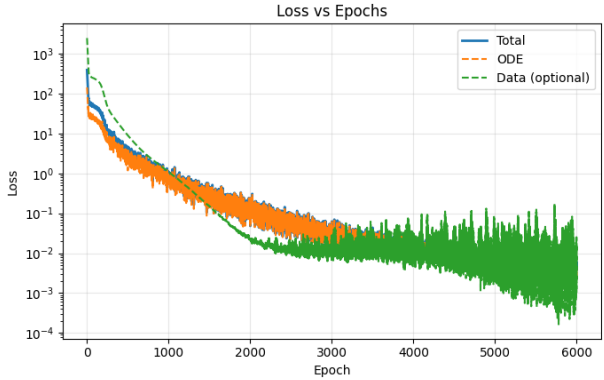


(c)

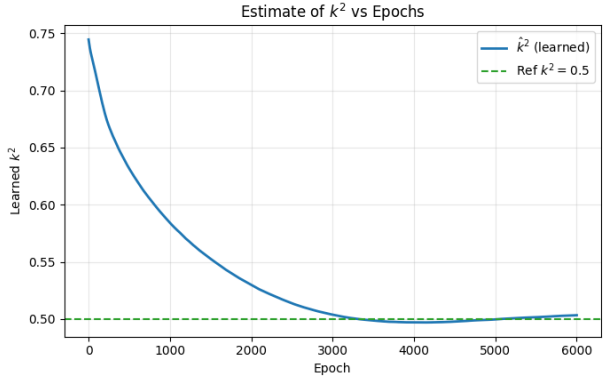
Figure 12.2: The second variant: (a) The neural network solution versus exact solution; (b) The loss versus number of epochs; (c) The evolution of k^2 across epochs.



(a)



(b)



(c)

Figure 12.3: The third variant: (a) The neural network solution versus the exact solution; (b) The losses versus number of epochs; (c) The evolution of k^2 across training.

12.2 Frequency Estimation

We shall estimate the oscillation frequency of a signal from data. The mathematical model is the one-dimensional harmonic oscillator

$$\frac{d^2u(t)}{dt} = -k^2u(t),$$

and the data is given in the Table 12.1.

j	t_j	$u(t_j)$
0	0.00000	0.00000
1	1.39626	1.28558
2	2.79253	1.96962
3	4.18879	1.73205
4	5.58505	0.68404
5	6.98132	-0.68404
6	8.37758	-1.73205
7	9.77384	-1.96962
8	11.17011	-1.28558
9	12.56637	0.00000

Table 12.1: Values of $u(t_j) = 2\sin(t_j/2)$ for $t_j = \frac{4\pi}{9}j$, $j = 0, \ldots, 9$.

This problem involves a single unknown parameter and the governing model is a linear second-order differential equation. The oscillator's frequency is given by $f = k/(2\pi)$. The goal is to recover the parameter k^2 using a neural network that treats k^2 as an internal trainable parameter.

To incorporate the endpoint conditions, we use a *hard embedding*. Since the solution satisfies $u(0) = u(4\pi) = 0$, the embedding takes the form

$$u_\theta(t) \;=\; t\,(4\pi - t)\,nn_\theta(t),$$

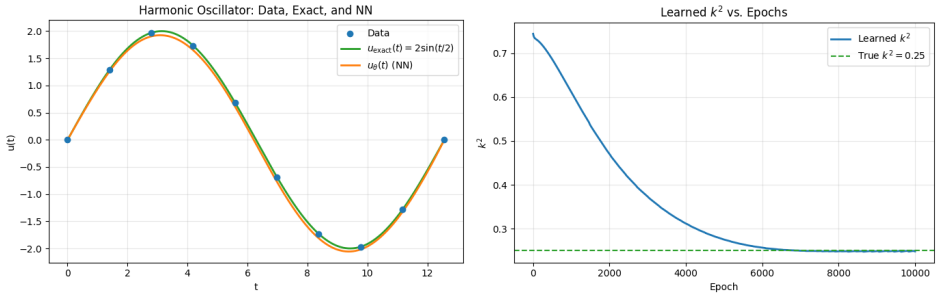
where the auxiliary network $nn_\theta(t)$ captures the interior behavior of the solution.

The loss function combines the ODE residual with the data misfit, ensuring that both the governing equation and the observed measurements are respected

$$\mathcal{L}_{\text{ODE}}(\theta) = \frac{1}{M}\sum_{j=1}^{M}\Big(\frac{d^2u(s_j)}{dt} + k^2u(s_j)\Big)^2 + \lambda_{\text{data}}\frac{1}{m}\sum_{j=0}^{m}\big(u_\theta(t_j) - u_j\big)^2,$$

where $m = 9$ and $\{s_j\}$ are M collocation points where the ODE is reinforced.

In the implementation, we set $M = 256$ and used a feed-forward neural network with three hidden layers, each containing 64 neurons and employing the tanh activation function. The network was trained for 10,000 epochs using the Adam optimizer with a learning rate of 10^{-3}. The network successfully recovers the exact solution, as shown in Fig. 12.4(a), and in the long run it also converges to the true parameter value $k^2 = 0.25$, see Fig. 12.4(b). The code is available at `https://tinyurl.com/dwc44zz9`



(a) **(b)**

Figure 12.4: Learning the oscillator from data: (a) The neural network solution versus exact solution; (b) The evolution of k^2 across epochs.

Remark 12.2.1 The main advantage of this method is that the frequency can be recovered from samples taken within a single period. In contrast, biological systems such as the human ear require observation over several oscillations, effectively sampling across multiple periods before extracting the frequency. Similarly, classical tools like Fourier and wavelet analysis are inherently global in nature: they rely on decompositions over longer intervals in order to resolve frequency components with sufficient precision. As a result, they are less effective when only a short segment of data is available.

Neural approaches, on the other hand, can directly enforce the underlying differential equation structure of the system, allowing the frequency to be identified even from sparse or short-time observations. This makes them particularly suitable for parameter recovery in cases where data is limited, noisy, or truncated, and highlights a key advantage over traditional signal processing techniques.

The next example is very similar, the only difference being that the corresponding ODE is nonlinear and there is no closed-form solution in elementary functions.

12.3 Pendulum Length Estimation

The equation of motion for a simple pendulum of unit mass is

$$\frac{d^2u(t)}{dt^2} + \frac{g}{L}\sin u(t) \;=\; 0,$$

where g is the gravitational acceleration and L is the length of the pendulum.

We collect $N = 10$ experimental measurements of the pendulum angle $u(t)$ at distinct time instances. The data is summarized in Table 12.2.

The objective is to infer the pendulum length L from the experimental data presented in the table. For convenience, we introduce the parameter

$$\kappa^2 = \frac{g}{L},$$

so that the problem reduces to estimating κ^2 using neural networks.

j	t_j	$u(t_j)$
0	0.00000	0.00000
1	0.55556	0.50088
2	1.11111	0.72183
3	1.66667	0.55094
4	2.22222	0.07385
5	2.77778	-0.49472
6	3.33333	-0.72328
7	3.88889	-0.55320
8	4.44444	-0.08436
9	5.00000	0.38555

Table 12.2: Synthetic data obtained as Runge–Kutta (RK4) measurements for the solution of $u''(t) + 2\sin u(t) = 0$ with $u(0) = 0$, $u'(0) = 1$ on $[0, 5]$.

As before, we employ the method by which the constant κ^2 is declared as an internal trainable parameter and it is learned by the neural network together with the solution $u(t)$.

To incorporate the initial condition, we use a *hard embedding*. Since the solution satisfies $u(0) = 0$, the embedding takes the form

$$u_\theta(t) \;=\; t\, nn_\theta(t),$$

where the auxiliary network $nn_\theta(t)$ captures the interior behavior of the solution.

The loss function combines the ODE residual with the data misfit, ensuring that both the governing equation and the observed measurements are respected

$$\mathcal{L}_{\mathrm{ODE}}(\theta) = \frac{1}{M}\sum_{j=1}^{M}\Big(\frac{d^2u(s_j)}{dt} + \kappa^2 \sin u(s_j)\Big)^2 + \lambda_{\mathrm{data}}\frac{1}{m}\sum_{j=0}^{m}\big(u_\theta(t_j) - u_j\big)^2,$$

where $m = 9$ and $\{s_j\}$ are $M = 256$ collocation points where the ODE is reinforced.

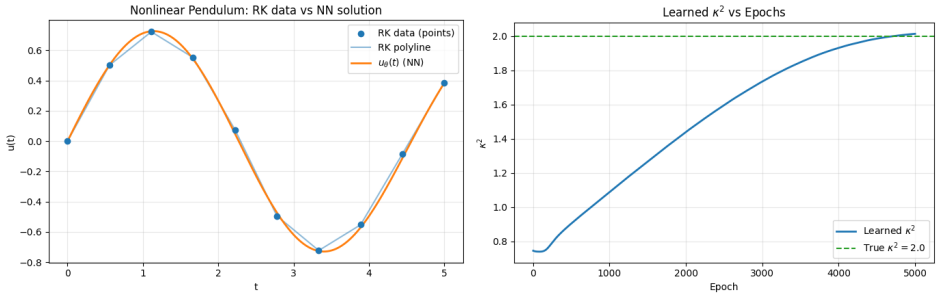
The network uses three hidden layers with 64 tanh-activated neurons and is trained for 5,000 epochs using the Adam optimizer with a learning rate of 10^{-3}. Figure 12.5(a) shows the neural network solution passing through the given data points, while Fig. 12.5(b) depicts the evolution of κ^2 across epochs. The estimate clearly converges toward the true value: after 5,000 epochs we obtain $\kappa^2 = 2.013697$, which is very close to the exact value 2. The complete code can be found at `https://tinyurl.com/3krt86re`

12.4 Estimating Thermal Diffusivity

The temperature distribution $u(x, t)$ in a rod of length L at time t and position x, with an initial temperature profile $\varphi(x)$ and no external heat sources, satisfies the initial value problem

$$\begin{aligned} \partial_t u - \kappa\, \partial_x^2 u &= 0, \qquad t > 0, \\ u(x, 0) &= \varphi(x), \qquad x \in [0, L], \end{aligned}$$

where $\kappa > 0$ is the thermal diffusivity constant.



(a) **(b)**

Figure 12.5: Learning the pendulum length from data: (a) The neural network solution versus given data points; (b) The evolution of k^2 across epochs.

If no heat is exchanged with the environment, we impose the *no-flux* boundary condition

$$\partial_x u(0,t) \;=\; \partial_x u(L,t) \;=\; 0,$$

which means that the ends of the rod at $x = 0$ and $x = L$ are perfectly insulated. To have the relation satisfied for $t = 0$, the initial temperature should satisfy the boundary conditions

$$\varphi'(0) = \varphi'(L) = 0.$$

From Example 6.1.8, the total heat

$$Q(t) \;=\; \int_0^L u(x,t)\,dx \;=\; \int_0^L \varphi(x)\,dx \;=\; Q(0)$$

is conserved over time.

Given the initial temperature distribution $\varphi(x)$ and measurement values $u_{ij} = u(x_i, t_j)$ at collocation points $\{x_i\}_{i=1}^n \subset [0, L]$ and time instances $\{t_j\}_{j=1}^m \subset (0, T]$, the task is to estimate the thermal diffusivity constant κ from these data.

We shall approach the problem in two ways: (i) using a neural network and (ii) using the heat kernel of the interval $[0, L]$.

(i) Using a neural network

1. Soft constraints. We consider a neural network with inputs $(x, t) \in [0, L] \times [0, T]$ and scalar output, with κ treated as an internal trainable parameter. Let $w = (\theta, \kappa)$ denote the augmented parameter set, where θ are the standard network weights. The network is written as

$$NN(w; x, t) = u_w(x, t),$$

which is trained to approximate the true solution $u(x, t)$.

We consider the following loss terms:

- Data loss:
$$\mathcal{L}_{\text{data}}(w) = \frac{1}{nm}\sum_{i=1}^{n}\sum_{j=1}^{m}\big(u_{ij} - u_w(x_i,t_j)\big)^2.$$

- PDE loss:
$$\mathcal{L}_{\text{PDE}}(w) = \frac{1}{nm}\sum_{i=1}^{n}\sum_{j=1}^{m}\big(\partial_t u_w(x_i,t_j) - \kappa\,\partial_x^2 u_w(x_i,t_j)\big)^2.$$

- Initial condition loss:
$$\mathcal{L}_{\text{IC}}(w) = \frac{1}{n}\sum_{i=1}^{n}\big(u_w(x_i,0) - \varphi(x_i)\big)^2.$$

- Conservation loss:
$$\mathcal{L}_{\text{cons}}(w) = \frac{1}{m}\sum_{j=1}^{m}\Big(\int_0^L u_w(x,t_j)\,dx - \int_0^L \varphi(x)\,dx\Big)^2.$$

The total loss combines all contributions:

$$\mathcal{L}_{\text{tot}}(w) = \mathcal{L}_{\text{data}}(w) + \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}}(w) + \lambda_{\text{IC}}\mathcal{L}_{\text{IC}}(w) + \lambda_{\text{cons}}\mathcal{L}_{\text{cons}}(w),$$

with some positive weights. If

$$w^* = (\theta^*, \kappa^*) = \arg\min_w \mathcal{L}_{\text{tot}}(w),$$

then κ^* provides an estimate of the thermal diffusivity constant.

2. Hard embedding. The previous approach relies heavily on soft embeddings. In practice, we usually try to avoid these whenever possible, since they tend to train slowly and often yield lower accuracy.

Both the initial condition and the no-flux boundary conditions can instead be incorporated directly into the solution by adopting the following ansatz:

$$u_w(x,t) \;=\; \varphi(x) \;+\; t\Big(A(w;t) + \int_0^x y\,(T-y)\,\mathrm{nn}(w;y,t)\,dy\Big), \tag{12.4.1}$$

where $w = (\theta, \kappa)$ denotes the augmented set of network parameters. The networks $A(w;t)$ and $\mathrm{nn}(w;y,t)$ are unconstrained networks subject to training. The role of the network $A_w(t)$ is to avoid pinning down $u_w(0,t)$ for all t. Thus, $u_w(0,t) = \varphi(0) + tA(w;t)$ can evolve with time t.

With this embedding, it suffices to minimize only the sum of the data and PDE losses, since the initial and boundary conditions are already satisfied by construction. Optionally, one may also include the heat conservation constraint as a soft penalty term, serving as an additional regularizer.

(*ii*) **Using the heat kernel** We shall show that we can further get rid of the ODE loss, keeping only the data loss. To this end we shall employ the use of the *heat kernel* $K(x, y; t)$. Thus, the solution can be expressed in closed form as

$$u(x,t) \;=\; \int_0^L K(x,y;t)\,\varphi(y)\,dy,$$

where $K(x, y; t)$ is the heat kernel associated with the heat operator on the compact interval $[0, L]$ for the no-flux boundary conditions.

This method is based on the fact that in this setting, the kernel $K(x, y; t)$ is explicitly known and can be written as the following infinite series (see [16] page 50 for details[1]).

$$\begin{aligned} K(x,y;t) =& \frac{1}{\sqrt{4\pi\kappa t}} e^{-\frac{(x-y)^2}{4\kappa t}} \Big[1 + 2\sum_{n\geq 1} e^{-\frac{n^2L^2}{\kappa t}} \cosh\big(\frac{nL}{\kappa t}(x-y)\big)\Big] \\ &+ \frac{1}{\sqrt{4\pi\kappa t}} e^{-\frac{(x+y)^2}{4\kappa t}} \Big[1 + 2\sum_{n\geq 1} e^{-\frac{n^2L^2}{\kappa t}} \cosh\big(\frac{nL}{\kappa t}(x+y)\big)\Big]. \end{aligned}$$

To emphasize the dependence of κ, we shall denote the heat kernel by $K(x, y; t; \kappa)$, so the solution depends on κ as

$$u_\kappa(x,t) = \int_0^L K(x,y;t;\kappa)\,\varphi(y)\,dy.$$

This representation provides an exact embedding of the PDE structure as well as the initial condition and boundary conditions into the solution, eliminating the need for soft constraints. Evaluating at (x_i, t_j) we obtain nm integral equations

$$\int_0^L K(x_i,y;t_j;\kappa)\,\varphi(y)\,dy = u_{ij}.$$

Then the loss depends only on the parameter κ and is constructed to enforce the measurement constraints as

$$\mathcal{L}(\kappa) \;=\; \frac{1}{nm}\sum_{i=1}^{n}\sum_{j=1}^{m}\left(\int_0^L K(x_i,y;t_j;\kappa)\,\varphi(y)\,dy \;-\; u_{ij}\right)^2.$$

The estimate of the thermal diffusivity is then obtained as

$$\kappa^* \;=\; \arg\min_{\kappa} \mathcal{L}(\kappa).$$

Remark 12.4.1 For general domains with irregular geometry, the heat kernel is typically not available in closed form. This makes the method difficult to apply in such cases.

[1] To arrive to the formula we changed t into $2\kappa t$ to reduce our case to the heat kernel of the standard heat operator $\partial_t - \frac{1}{2}\partial_x^2$.

Implementation The code details of this paragraph can be found at the page `https://tinyurl.com/mu366j8j`

We consider the data $u_{ij} = u(x_i, t_j)$ given in the Table 12.3. These data corresponds to a one-dimensional heat equation with *no-flux* boundary condition and initial temperature $\varphi(x) = 3x^2 - 2x^3 + 0.5$. Our task is to estimate the diffusivity parameter κ.

$x\backslash t$	0.05	0.10	0.20	0.30	0.50	0.70	1.00	2.00
0.0	0.5000	0.6143	0.7649	0.8565	0.9465	0.9801	0.9955	1.0000
0.1	0.5280	0.6334	0.7764	0.8635	0.9491	0.9810	0.9957	1.0000
0.2	0.6040	0.6887	0.8098	0.8839	0.9567	0.9839	0.9963	1.0000
0.3	0.7160	0.7743	0.8618	0.9157	0.9686	0.9883	0.9973	1.0000
0.4	0.8520	0.8816	0.9274	0.9557	0.9835	0.9938	0.9986	1.0000
0.5	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
0.6	1.1480	1.1184	1.0726	1.0443	1.0165	1.0062	1.0014	1.0000
0.7	1.2840	1.2257	1.1382	1.0843	1.0314	1.0117	1.0027	1.0000
0.8	1.3960	1.3113	1.1902	1.1161	1.0433	1.0161	1.0037	1.0000
0.9	1.4720	1.3666	1.2236	1.1365	1.0509	1.0190	1.0043	1.0000
1.0	1.5000	1.3857	1.2351	1.1435	1.0535	1.0199	1.0045	1.0000

Table 12.3: Data $u_{ij} = u(x_i, t_j)$ is located at the intersection of the t_jth column with the x_ith row.

1. Soft constraints. We employ a neural network with *soft constraints* to estimate the diffusivity parameter. In this case, the spatial domain has length $L = 1$ and the final time is $T = 2$. The *total heat* is conserved, and by direct integration we obtain

$$Q(t) = Q(0) = \int_0^1 \varphi(x)\, dx = 1.$$

The neural architecture is a feed-forward network with four hidden layers, each containing 64 neurons and using the `tanh` activation. To ensure positivity of the parameter κ, we reparameterized it through a *Softplus function.*

Training was performed in two stages and for different learning rates using the Adam optimizer.
Training for 6,000 epochs with a learning rate of 5×10^{-3}*:* The losses are relatively noisy, as shown in Fig. 12.6(a). The evolution of κ across epochs is displayed in Fig. 12.6(b). Although the curve reaches saturation, the larger learning rate introduces small oscillations around the stable value. The final estimate of κ in this case is 0.34.

Training for 10,000 epochs with a learning rate of 5×10^{-4}*:* The losses are much less noisy, as illustrated in Fig. 12.6(c). The evolution of κ is given in Fig. 12.6(d). Here the curve is smoother and approaches saturation more steadily, which can be attributed to the smaller learning rate. The final estimate of κ in this case is 0.35.

2. Hard embedding. Implementing the formula (12.4.1) we obtain similar results as follows.

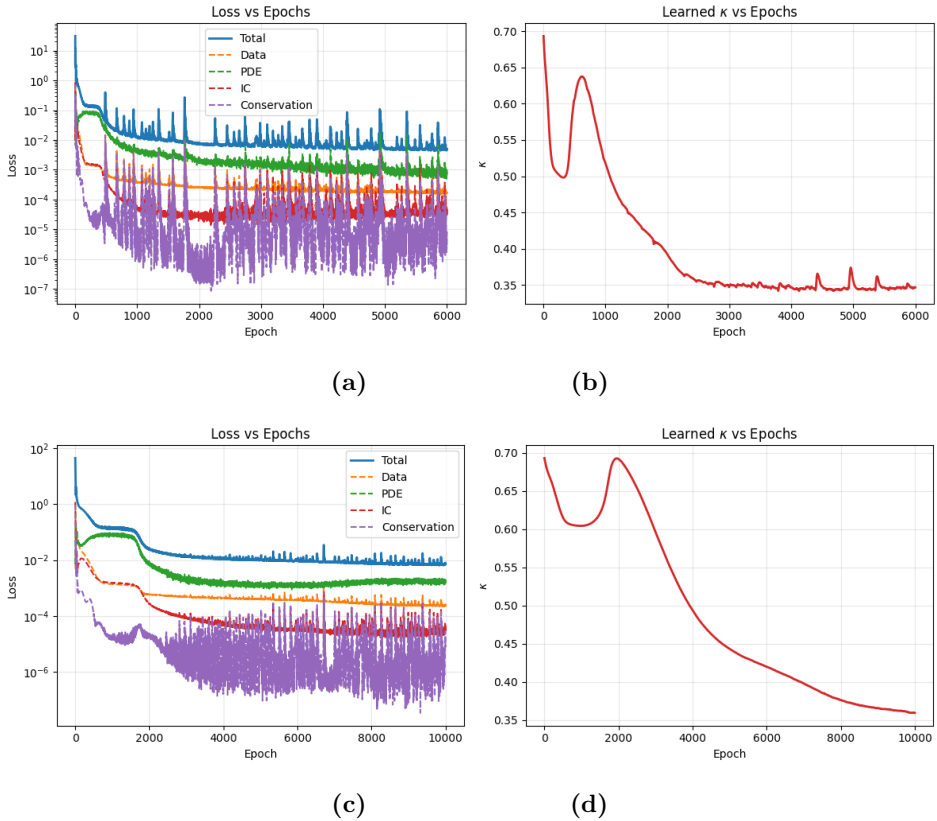


Figure 12.6: The soft constraints variant: (a) Losses for $\text{lr} = 5 \times 10^{-3}$; (b) Evolution of κ over epochs for $\text{lr} = 5 \times 10^{-3}$; (c) Losses for $\text{lr} = 5 \times 10^{-4}$; (d) Evolution of κ over epochs for $\text{lr} = 5 \times 10^{-4}$. We notice that a smaller learning rate produces a smaller variance of the losses and a smoother estimation of the parameter κ. However, the smaller the learning rate, the slower the convergence of the estimation κ.

Training for 15,000 epochs with a learning rate of 5×10^{-3}*:* The losses are relatively noisy, as shown in Fig. 12.7(a). The evolution of κ across epochs is displayed in Fig. 12.7(b). Although the curve seems to reach saturation, the larger learning rate introduces small random perturbations around the stable value. The final estimate of κ in this case is 0.3.

Training for 15,000 epochs with a learning rate of 5×10^{-4}*:* The losses are now much less noisy, as illustrated in Fig. 12.7(c). The evolution of κ is given in Fig. 12.7(d). Here the curve is smooth and approaches saturation more steadily. The final estimate of κ in this case is 0.3.

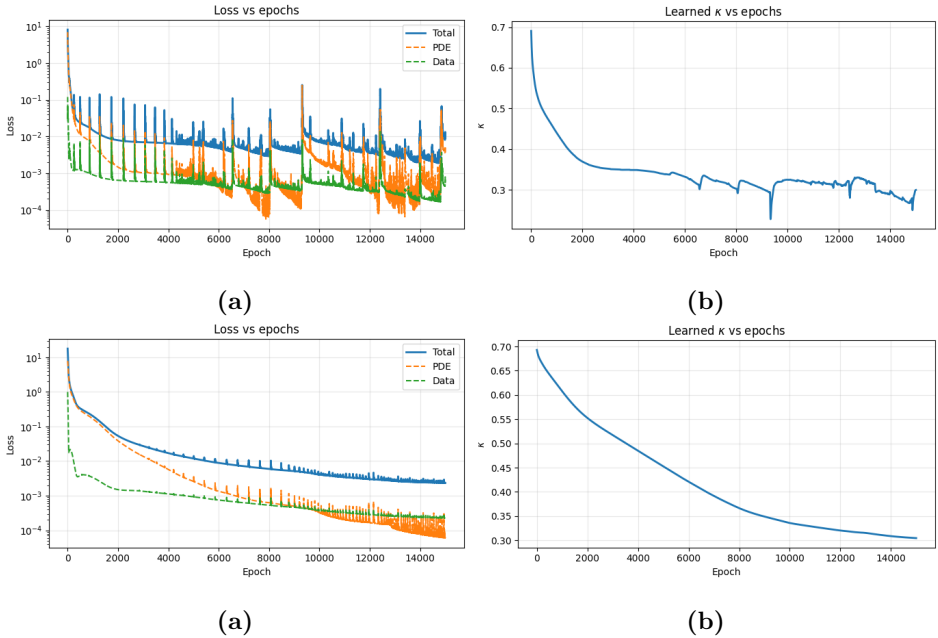


Figure 12.7: The hard embedding variant: (a) Losses for $\mathrm{lr} = 5 \times 10^{-3}$; (b) Evolution of κ over epochs for $\mathrm{lr} = 5 \times 10^{-3}$; (c) Losses for $\mathrm{lr} = 5 \times 10^{-4}$; (d) Evolution of κ over epochs for $\mathrm{lr} = 5 \times 10^{-4}$.

12.5 Parameter Estimation for Lotka–Volterra System

In this section, we use neural networks to recover the four parameters α, β, γ, and δ that appear in the Lotka–Volterra system of equations:

$$\begin{aligned} x'(t) &= \alpha\, x(t) - \beta\, x(t)y(t), \\ y'(t) &= \delta\, x(t)y(t) - \gamma\, y(t), \end{aligned}$$

from observed data $\{(x_i, y_i)\}$, where $x_i = x(t_i)$ and $y_i = y(t_i)$ at a set of collocation times $\{t_i\}$.

To identify the values of the four parameters, we treat them as internal trainable variables of a neural network

$$NN(w;t) = \big(x_w(t),\, y_w(t)\big),$$

where $w = (\theta, \alpha, \beta, \gamma, \delta)$ denotes the augmented parameter set, consisting of both the network weights θ and the unknown system parameters.

The network is trained to fit both the ODE system and the observed data. We introduce two loss components:

- *Data loss:*

$$\mathcal{L}_{\text{data}}(w) \;=\; \frac{1}{N}\sum_{j=1}^{N}\big(x_w(t_j) - x_j\big)^2 \;+\; \frac{1}{N}\sum_{j=1}^{N}\big(y_w(t_j) - y_j\big)^2.$$

- *ODE loss:*

$$\begin{aligned}\mathcal{L}_{\text{ODE}}(w) \;=\; &\frac{1}{M}\sum_{k=1}^{M}\big(x'_w(t_k) - \alpha\, x_w(t_k) + \beta\, x_w(t_k)\, y_w(t_k)\big)^2 \\ &+ \frac{1}{M}\sum_{k=1}^{M}\big(y'_w(t_k) - \delta\, x_w(t_k)\, y_w(t_k) + \gamma\, y_w(t_k)\big)^2.\end{aligned}$$

The total loss combines these contributions:

$$\mathcal{L}_{\text{tot}}(w) \;=\; \mathcal{L}_{\text{ODE}}(w) \;+\; \mathcal{L}_{\text{data}}(w).$$

Implementation We first generated synthetic data from a Lotka–Volterra system with parameter values

$$\alpha = 1.0, \qquad \beta = 0.5, \qquad \gamma = 0.8, \qquad \delta = 0.3,$$

and the resulting values are presented in Table 12.4.

Using this dataset, we employed neural networks to recover the values of the parameters. The initial conditions were hard-embedded into the network architecture by setting

$$x_w(t) = x_0 + t\,\mathrm{nn}_1(w;t), \qquad y_w(t) = y_0 + t\,\mathrm{nn}_2(w;t),$$

where nn_1 and nn_2 are auxiliary neural networks.

The trainable parameters $\alpha, \beta, \gamma, \delta$ were included as internal variables of the network and parametrized through a softplus activation to ensure positivity. The architecture consisted of three hidden layers with 64 neurons each and `tanh` activations. Training was performed over 1,000 epochs using the Adam optimizer with learning rate 10^{-3}, yielding the following estimates:

t	$x(t)$	$y(t)$
0.0	1.200000	0.800000
0.1	1.275291	0.766415
0.2	1.357477	0.735976
0.3	1.447049	0.708569
0.4	1.544533	0.684097
0.5	1.650485	0.662489
0.6	1.765486	0.643692
0.7	1.890145	0.627679
0.8	2.025085	0.614452
0.9	2.170942	0.604042
1.0	2.328349	0.596514

Table 12.4: Data for a Lotka–Volterra system with $\alpha = 1.0$, $\beta = 0.5$, $\gamma = 0.8$, $\delta = 0.3$, $x(0) = 1.2$, $y(0) = 0.8$.

```
Estimated parameters:
alpha = 0.99298, beta = 0.50418, gamma = 0.77088, delta = 0.28394
```

These results are highly accurate, especially given that only ten data points were available to estimate four independent parameters. The neural network solution interpolates the provided data points almost perfectly, as illustrated in Fig. 12.8(a). The loss functions decrease steadily and approach saturation, as shown in Fig. 12.8(b). Moreover, the parameter trajectories across epochs converge to stable values, see Fig. 12.8(c).

Overall, the neural network converges rapidly and provides results that are both stable and very close to the true values, highlighting the effectiveness of this approach.

Remark 12.5.1 (*i*) The natural question is whether the same parameter values can be recovered from fewer measurements. To test this, we selected only three data points and attempted to infer four parameters—an apparently underdetermined task since the number of unknowns exceeds the number of observations.

Surprisingly, the results were excellent. The estimated parameters were

```
Estimated parameters:
alpha = 0.98844, beta = 0.52179, gamma = 0.76305, delta = 0.28263
```

(*ii*) We further pushed the method to its extreme by using only two data points: the initial condition and a single final measurement. Remarkably, even under such scarce information, the results remained quite accurate. The estimated parameters were

```
Estimated parameters:
alpha = 0.98513, beta = 0.57373, gamma = 0.70225, delta = 0.27238
```

(*iii*) The success of this method lies in the fact that the differential equation itself encodes a significant amount of information about the system. By incorporating the ODE loss, the neural network is able to leverage this hidden structure and compensate for the limited data.

The code for this section is available at: `https://tinyurl.com/htda8e7h`

Further readings There is a substantial body of literature on the parameter identification problem. Classical contributions include the monograph by Banks and Kunisch [8], which develops estimation techniques for systems governed by PDEs, framing state and parameter estimation as optimization problems in infinite-dimensional spaces. We also count here Baumeister's book [10], which investigates the ill-posedness of inverse problems and introduces stability concepts and regularization techniques, providing both theoretical foundations and practical tools. We mention the paper by Kravaris and Seinfeld [56], which analyzes parameter identification in PDE systems, highlighting the role of regularization in counteracting ill-posedness and illustrating applications in chemical engineering and process control. Also, we cite Richter's work [80], which also focuses on numerical techniques for identifying spatially varying parameters in PDEs, emphasizing stability and regularization.

In recent years, the use of neural networks for parameter identification has become increasingly prominent, offering robust approaches to handle ill-posedness, noisy data, and complex nonlinearities.

Cen et al. [20] provide a comprehensive survey of unsupervised neural network methods, with particular focus on error bounds and conditional stability for diffusion-type PDEs. Aarset et al. [1] propose an "all-at-once" learning strategy, in which state, parameters, and nonlinearities are estimated simultaneously, thereby avoiding explicit parameter-to-state maps.

Early neural network applications can be traced to González-García et al. [39], who introduced ANN architectures for distributed parameter systems. Dong [32] later reviewed how neural models relate to classical parametric system identification.

With the advent of physics-informed neural networks, Tartakovsky et al. [88] demonstrated parameter identification of space-dependent diffusion coefficients and constitutive laws using sparse and noisy measurements. More recently, Rojas et al. [81] applied ANN-based methods to identify parameters in nonlinear damage phase field models, showing the adaptability of NN methods to complex PDE systems.

12.6 Exercises

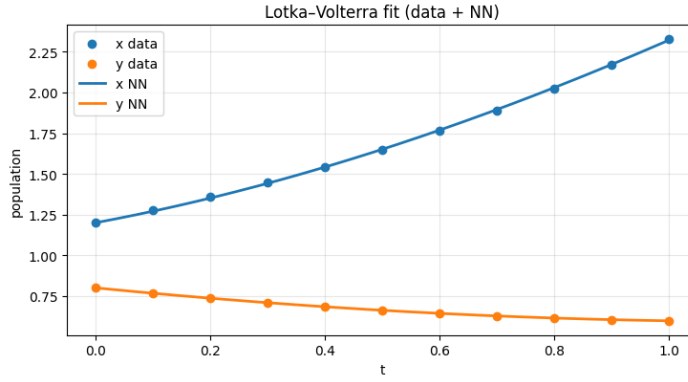
Exercise 12.6.1 (Parameter estimation for logistic growth). Consider the logistic ODE

$$u'(t) \;=\; r\,u(t)\Big(1-\tfrac{u(t)}{K}\Big), \qquad u(0)=u_0,$$

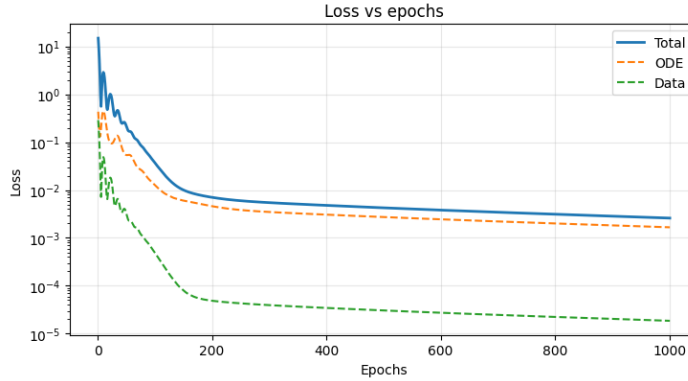
describing population growth with intrinsic rate $r>0$ and carrying capacity $K>0$. Assume $K=10$ and $u_0=1$ are known. Your goal is to estimate the unknown parameter r from noisy measurements $\{(t_j,u_j)\}$ of $u(t)$.

The following data measurements are provided:

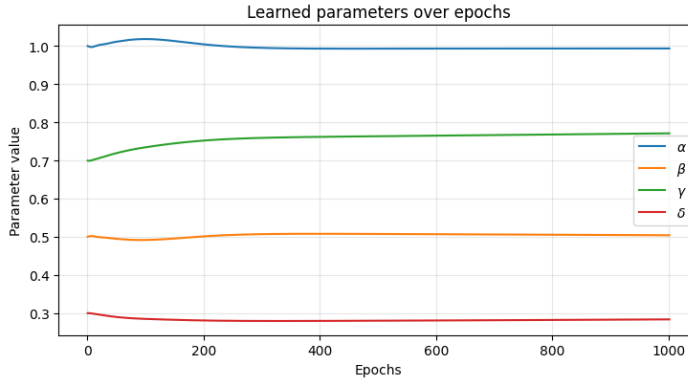
t_j	0.0	0.2	0.4	0.6	0.8	1.0	1.2	1.4	1.6	1.8	2.0	2.2
u_j	1.00	1.13	1.29	1.47	1.69	1.94	2.23	2.57	2.96	3.41	3.92	4.49



(a)



(b)



(c)

Figure 12.8: Retrieving 4 parameters from 10 data points: (a) The neural network solution passing through the given data points; (b) The associated losses reach perfect saturation; (c) The evolution of the parameters across epochs.

(*a*) Build a feed-forward neural network $n_\theta(t)$ and hard-embed the initial condition by defining

$$u_\theta(t) \;=\; u_0 \;+\; t\, n_\theta(t),$$

so that $u_\theta(0) = u_0$ holds exactly. Treat r as an *internal trainable scalar* (parameterize as $r = \mathrm{softplus}(\rho)$ to keep $r > 0$).

(*b*) Train (θ, ρ) by minimizing the sum of

$$\mathcal{L}_{\text{data}} \;=\; \frac{1}{N}\sum_{j=1}^{N}\big(u_\theta(t_j) - u_j\big)^2, \quad \mathcal{L}_{\text{ode}} \;=\; \frac{1}{N}\sum_{j=1}^{N}\Big(\tfrac{d}{dt}u_\theta(t_j) - r\,u_\theta(t_j)\big(1 - \tfrac{u_\theta(t_j)}{K}\big)\Big)^2,$$

where $\frac{d}{dt}u_\theta$ is obtained via automatic differentiation.

(*c*) After training, report the learned parameter $\hat{r}$ and compare it to $r_{\text{true}} = 0.7$. Plot on the same figure: the data points (t_j, u_j), the NN solution $u_\theta(t)$ over a dense grid $t \in [0, 2.2]$, and the exact curve

$$u_{\text{exact}}(t) = \frac{K}{1 + \big(\frac{K-u_0}{u_0}\big)e^{-rt}} \quad \text{with } r = r_{\text{true}}.$$

Also plot the trajectory of the learned r versus epochs.

Exercise 12.6.2 (Parameter estimation for the SIR model). Consider the classical SIR system

$$S'(t) = -\beta\, S(t)\, I(t),$$

$$I'(t) = \beta\, S(t)\, I(t) - \gamma\, I(t),$$

$$R'(t) = \gamma\, I(t),$$

where $S(t)$, $I(t)$, and $R(t)$ represent the fractions of susceptible, infected, and recovered individuals in a closed population. The parameters $\beta > 0$ (infection rate) and $\gamma > 0$ (recovery rate) are unknown and must be inferred from data.

(*a*) Take $S(0) = 0.99$, $I(0) = 0.01$, $R(0) = 0$, and parameters

$$\beta_{\text{true}} = 0.9, \qquad \gamma_{\text{true}} = 0.3.$$

Generate synthetic observations $\{(t_j, I_j)\}$ for $t_j \in [0, 10]$ (e.g., 20 equally spaced points), using a numerical ODE solver for the system. This mimics the realistic scenario where only infection counts are available.

(*b*) Construct a neural network with time t as input and outputs

$$NN(w; t) = \big(S_w(t), I_w(t), R_w(t)\big),$$

with $w = (\theta, \beta, \gamma)$ where θ are the NN weights and (β, γ) are treated as internal trainable scalars. Use a softplus activation on (β, γ) to ensure positivity.

(*c*) Enforce the initial conditions by embedding them into the architecture, e.g.

$$S_w(t) = S(0) + t\, n_1(w; t), \quad I_w(t) = I(0) + t\, n_2(w; t), \quad R_w(t) = R(0) + t\, n_3(w; t).$$

(*d*) Combine:

- Data loss (matching $I(t)$ to the observed values I_j),

$$\mathcal{L}_{\text{data}}(w) = \frac{1}{N}\sum_{j=1}^{N}\big(I_w(t_j) - I_j\big)^2,$$

- ODE loss (residuals of the differential equations),

$$\begin{aligned}\mathcal{L}_{\text{ODE}}(w) \;=\; \frac{1}{M}\sum_{k=1}^{M}\Big[&(S_w'(t_k) + \beta S_w(t_k) I_w(t_k))^2 \\ &+ (I_w'(t_k) - \beta S_w(t_k) I_w(t_k) + \gamma I_w(t_k))^2 \\ &+ (R_w'(t_k) - \gamma I_w(t_k))^2\Big],\end{aligned}$$

  with derivatives computed via automatic differentiation.

The total loss is

$$\mathcal{L}_{\text{tot}}(w) = \mathcal{L}_{\text{data}}(w) + \lambda \mathcal{L}_{\text{ODE}}(w).$$

Then train the model using Adam with a suitable learning rate. Plot:

- the learned trajectories $(S_w(t), I_w(t), R_w(t))$ against the numerical solution,
- the estimated parameters $(\hat{\beta}, \hat{\gamma})$ across epochs.

(*e*) Compare the learned parameters $\hat{\beta}, \hat{\gamma}$ with the true ones

$$\beta_{\text{true}} = 0.9, \qquad \gamma_{\text{true}} = 0.3.$$

Discuss how the ODE structure compensates for the small number of data points.

Exercise 12.6.3 (Estimating diffusivity in a 1D heat equation). Consider the 1D heat equation on the interval $[0, L]$ without external sources

$$\partial_t u - \kappa\, \partial_x^2 u \;=\; 0, \qquad x \in (0, L),\ t > 0,$$

with Dirichlet boundary conditions

$$u(0, t) = 0, \qquad u(L, t) = 0 \quad \text{for all } t \geq 0,$$

and initial condition

$$u(x, 0) = \varphi(x), \qquad x \in [0, L],$$

with $\varphi(x)$ smooth function satisfying $\varphi(0) = \varphi(L) = 0$.

(*a*) Show that the neural ansatz

$$u_w(x, t) \;=\; \varphi(x) \;+\; t\, x(x - L)\, \mathrm{nn}(w;\, x, t)$$

is a hard embedding of the boundary and initial conditions.

(*b*) Suppose you are given noisy measurements

$$u_{ij} \approx u(x_i, t_j), \qquad i = 1, \ldots, n, \quad j = 1, \ldots, m,$$

at collocation points $\{(x_i, t_j)\} \subset [0, L] \times (0, T]$. Treat κ as a trainable scalar parameter together with the network weights w, and define the model

$$u_{w,\kappa}(x,t) \;=\; \varphi(x) \;+\; t\,x(x-L)\,\mathrm{nn}(w;\,x,t).$$

Estimate κ by minimizing a composite loss

$$\mathcal{L}(w,\kappa) \;=\; \mathcal{L}_{\mathrm{data}}(w,\kappa) \;+\; \lambda_{\mathrm{PDE}}\,\mathcal{L}_{\mathrm{PDE}}(w,\kappa),$$

where

$$\mathcal{L}_{\mathrm{data}}(w,\kappa) \;=\; \frac{1}{nm}\sum_{i=1}^{n}\sum_{j=1}^{m}\big(u_{w,\kappa}(x_i,t_j) - u_{ij}\big)^2,$$

and the physics-informed residual enforces the heat equation at the same (or additional) collocation points

$$\mathcal{L}_{\mathrm{PDE}}(w,\kappa) \;=\; \frac{1}{nm}\sum_{i=1}^{n}\sum_{j=1}^{m}\Big(\partial_t u_{w,\kappa}(x_i,t_j) \;-\; \kappa\,\partial_x^2 u_{w,\kappa}(x_i,t_j)\Big)^2.$$

Measuring the capacitance of an electrical circuit is trickier than measuring resistance and inductance. The next exercise proposes a method of estimating the capacitance C in a parallel RLC circuit via PINNs.

Exercise 12.6.4 Consider a parallel RLC circuit with known resistance $R > 0$ and inductance $L > 0$. Let $v(t)$ denote the node voltage. We assume there are no outside current sources acting on the circuit. Kirchhoff's current law yields the ODE

$$C\,\ddot{v}(t) \;+\; \frac{1}{R}\,\dot{v}(t) \;+\; \frac{1}{L}\,v(t) \;=\; 0,$$

where $C > 0$ (capacitance) is unknown. You are given noisy voltage measurements $\{(t_j,\, v_j)\}_{j=1}^{N}$ on a time window $[0, T]$, and initial data $v(0) = v_0$.

(a) Define the physics residual

$$\mathcal{R}(t; C, v) \;=\; C\,\ddot{v}(t) \;+\; \frac{1}{R}\,\dot{v}(t) \;+\; \frac{1}{L}\,v(t).$$

Let $v_\theta(t)$ be a feed-forward network (e.g. 3–4 hidden layers, 32–64 neurons, `tanh` activation). Treat C as a trainable scalar via the positive parameterization $C = \mathrm{softplus}(c_{\mathrm{raw}}) + \varepsilon$ with small $\varepsilon > 0$. Use the hard-embedding by setting

$$v_\theta(t) = v_0 + t\,n_\theta(t)$$

where n_θ is an unconstrained network.

(b) Construct the total loss

$$\mathcal{L}_{\mathrm{tot}}(\theta, c_{\mathrm{raw}}) \;=\; \lambda_{\mathrm{data}}\,\frac{1}{N}\sum_{j=1}^{N}\big(v_\theta(t_j) - v_j\big)^2 \;+\; \lambda_{\mathrm{pde}}\,\frac{1}{K}\sum_{k=1}^{K}\mathcal{R}(t_k; C, v_\theta)^2\,,$$

where $\{t_k\}_{k=1}^K$ are collocation times (you may include the data times and extra random points in $[0, T]$). Then implement the model in Keras/TensorFlow with automatic differentiation to obtain $\dot{v}_\theta, \ddot{v}_\theta$. Train with Adam (e.g. learning rate 10^{-3} to 10^{-4}) for 5,000–20,000 epochs.

(*c*) Track and plot: (i) $\mathcal{L}_{\text{tot}}$, data loss, and PDE loss vs. epochs; and (ii) the learned C versus epochs. Report the final estimate C^*. Then plot the fitted voltage $v_\theta(t)$ against the measurements (t_j, v_j).

(*d*) Choose $R = 100\ \Omega$, $L = 10$ mH, true $C = 1\ \mu$F, unforced $I \equiv 0$, and generate $v(t)$ by integrating the ODE with initial conditions $v(0) = 1$ V, $\dot{v}(0) = 0$. Sample $N = 10$ points on $[0, 5]$, add small Gaussian noise, and run your PINN to recover C.

Exercise 12.6.5 (Skydiving). A skydiver of mass m jumps vertically out of a plane with zero initial vertical velocity, $v_0 = 0$. His velocity at time t satisfies the ODE

$$v'(t) + \frac{k}{m}\, v(t) = g,$$

where g is the gravitational acceleration and k is a constant that depends on the skydiver's cross-sectional area.

The velocity is measured at discrete time instances t_j, yielding data points $v_j = v(t_j)$ for $1 \le j \le n$.

(a) Verify that the function

$$v(t) \;=\; \frac{mg}{k}\Big(1 - e^{-\frac{k}{m}t}\Big)$$

is a solution of the ODE with $v(0) = 0$.

(b) Show that

$$f_{m,g,t}(k) \;=\; \frac{mg}{k}\Big(1 - e^{-\frac{k}{m}t}\Big)$$

is strictly decreasing in k. Conclude that for fixed (m, g, t, v) there exists a unique solution $k = k(m, g, t, v)$ of the equation in part (a), but that this solution does not admit an explicit elementary expression.

(c) Explain how the constant k can be estimated from the measurements $\{v_j\}$ by minimizing a least-squares loss based on the formula in part (a).

(d) Consider a neural network in which k is treated as an internal trainable parameter. Explain how k can be estimated by minimizing a combination of a physics-informed loss (based on the ODE residual) and a data loss (based on the observations).

Exercise 12.6.6 Consider the one-dimensional Klein–Gordon equation for a wave function $\varphi(t, x)$ describing a free relativistic particle of mass m_0:

$$(\Box + m_0^2)\, \varphi(t, x) = 0, \qquad \Box = \partial_t^2 - \partial_x^2.$$

(a) Given measurements of the wave function at a discrete set of space–time points $\{\varphi_{ij} = \varphi(t_i, x_j)\}$, formulate a neural network–based approach to estimate the unknown mass parameter m_0 from these observations.

(b) Then train a neural network $\varphi_\theta(t, x)$ to approximate the solution of the PDE while treating m_0 as an additional trainable parameter. The loss function should include both a data loss, enforcing agreement with the observed values and a PDE loss, enforcing the Klein–Gordon equation. During training, both θ and m_0 are updated, allowing the network to infer the unknown mass m_0 from the data.

Exercise 12.6.7 Consider the two-dimensional Helmholtz equation for a wave function $u(x, y)$:

$$\Delta u(x, y) + k^2 u(x, y) = 0, \qquad (x, y) \in \mathbb{R}^2,$$

where $\Delta = \partial_x^2 + \partial_y^2$ is the Laplacian and $k > 0$ is the wave number.

Suppose we have measurements of the wave field at a discrete set of points $\{u(x_i, y_j)\}$.

(a) Formulate a neural network–based approach to estimate the unknown parameter k from these observations.

(b) Train a neural network $u_\theta(x, y)$ to approximate the solution while treating k as an additional trainable parameter. The loss function should include both a data loss, enforcing agreement with the observed values $u(x_i, y_j)$, and a PDE loss, enforcing the Helmholtz equation. During training, both θ and k are updated, allowing the network to infer the wave number k directly from the measured data.

This page intentionally left blank

Chapter 13

Equations Discovery

In Chapter 12 we considered the problem of parameter estimation from data under the assumption that the governing mathematical model was already known. In this chapter we take a more ambitious step: we assume that the differential equation itself is unknown, and Deep Learning must be used to infer the underlying model that best fits the observed data.

This task may involve estimating variable coefficients in a given class of differential equations, or even identifying the entire equation from data. Conceptually, learning a differential equation from data is analogous to determining a polynomial that interpolates data points: in the polynomial case one only needs to determine the coefficients; in the differential equation case the "coefficients" are typically functions, which must themselves be estimated.

From a statistical perspective, classical regression discovers laws that interpolate data using simple functional forms (polynomials, exponentials, logarithms, etc.). In Physics, however, the governing laws are expressed not as functions but as differential equations. Learning such laws requires specifying the order of the highest derivative (usually first or second order, as in Newton's laws of motion), and estimating the coefficients of these derivatives, which may be constants or functions of one or more variables.

13.1 Estimation of a Time-dependent Coefficient

We begin with the simpler setting where the form of the governing PDE (or ODE) is known, but one of its coefficients is an unknown function to be learned from data. Specifically, consider a mechanical oscillator of mass m and elastic constant k, both known. The system is subject to a damping force whose coefficient $\delta(t)$ is unknown and varies in time. Our goal is to recover the function $\delta(t)$ from measurements.
Suppose we are given N observations of the state of the oscillator,

$$(u_i, v_i) = \big(u(t_i), u'(t_i)\big), \qquad t_i \in [0, T],\ i = 1, \ldots, N,$$

where $u(t)$ satisfies the second-order ODE

$$mu''(t) + \delta(t)u'(t) + ku(t) = 0,$$

subject to the initial conditions $u(0) = u_0$ and $u'(0) = v_0$.

Method 1 (Brute force method) We approximate the solution directly from the data using a neural network with initial conditions embedded

$$u_\theta(t) = u_0 + tv_0 + \frac{1}{2}t^2 NN(\theta; t).$$

The parameters θ are trained by minimizing the data mismatch loss

$$\mathcal{L}_{\text{data}}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N}\big(u_\theta(t_i) - u_i\big)^2 \;+\; \frac{1}{N}\sum_{i=1}^{N}\big(u'_\theta(t_i) - v_i\big)^2,$$

where (u_i, v_i) are the observed states.

Once the optimal parameters θ^* are obtained, the damping coefficient can be reconstructed pointwise from the governing equation as

$$\delta(t) \;=\; -\,\frac{m\,u''_{\theta^*}(t) + k\,u_{\theta^*}(t)}{u'_{\theta^*}(t)}.$$

Remark 13.1.1 This reconstruction formula can become numerically unstable if $u'_{\theta^*}(t)$ is close to zero, since division by small values amplifies errors in the numerator. In practice, a regularization term $u'_{\theta^*}(t) + \epsilon$ replaces $u'_{\theta^*}(t)$ for $\epsilon > 0$ small. We will see from the implementation that this is the most problematic of all methods.

Method 2 (Polynomial approximation of $\delta(t)$) We first assume that the damping coefficient $\delta(t)$ varies slowly with time. In this case, a quadratic approximation is expected to provide a good model:

$$\delta(t) \;\approx\; \delta_0 + \delta_1 t + \tfrac{1}{2}\delta_2 t^2.$$

The constants $(\delta_0, \delta_1, \delta_2)$ are treated as trainable parameters together with the neural network weights. Denoting the collection of all parameters by

$$w = (\theta, \delta_0, \delta_1, \delta_2),$$

we approximate the solution $u(t)$ using a neural network with embedded initial conditions:

$$u_w(t) \;=\; u_0 + v_0 t + \tfrac{1}{2}t^2\, NN(w; t).$$

The parameters are trained by minimizing the composite loss

$$\mathcal{L}(w) = \mathcal{L}_{\text{ODE}}(w) + \lambda\,\mathcal{L}_{\text{data}}(w),$$

where the PDE residual loss is

$$\mathcal{L}_{\text{ODE}}(w) \;=\; \frac{1}{M}\sum_{j=1}^{M}\Big(mu''_w(t_j) + \delta(t_j)u'_w(t_j) + ku_w(t_j)\Big)^2,$$

and the data mismatch loss is

$$\mathcal{L}_{\text{data}}(w) \;=\; \frac{1}{N}\sum_{i=1}^{N}\big(u_w(t_i) - u_i\big)^2 \;+\; \frac{1}{N}\sum_{i=1}^{N}\big(u'_w(t_i) - v_i\big)^2.$$

Here M denotes the number of collocation points for the ODE residual, which are not necessarily the same as the N data points.

At convergence, the optimal parameter set is

$$w^* = (\theta^*, \delta_0^*, \delta_1^*, \delta_2^*),$$

yielding the estimated damping coefficient

$$\delta(t) \;\approx\; \delta_0^* + \delta_1^* t + \tfrac{1}{2}\delta_2^* t^2.$$

Remark 13.1.2 1. If $\delta(t)$ varies linearly in time, then the last optimal parameter satisfies $\delta_2^* = 0$.

2. For more complex variations of $\delta(t)$, the same approach extends by using an nth-order Taylor polynomial approximation,

$$\delta(t) \;\approx\; \delta_0 + \delta_1 t + \cdots + \frac{t^n}{n!}\,\delta_n,$$

where the coefficients δ_j are treated as additional trainable parameters in the neural network framework.

Method 3 (Cosine approximation of $\delta(t)$) We next consider the case where the damping coefficient $\delta(t)$ varies periodically in time. A natural model in this setting is

$$\delta(t) \;\approx\; \delta_2 \cos(\delta_1 t + \delta_0),$$

where $(\delta_0, \delta_1, \delta_2)$ are unknown constants to be determined. As in Method 1, these constants are treated as trainable parameters alongside the neural network weights θ. The training procedure and loss formulation then follow exactly the same structure as in Method 3.

Method 4 (Neural network approximation of $\delta(t)$) If no prior information is available about whether $\delta(t)$ varies slowly or periodically, we adopt a more general approach and approximate it directly using a neural network. In this setting, the network outputs both the damping coefficient and the solution:

$$NN(\theta; t) \;=\; \big(\delta_\theta(t),\, u_\theta(t)\big),$$

where the solution component is expressed with embedded initial conditions as

$$u_\theta(t) \;=\; u_0 + v_0 t + \tfrac{1}{2} t^2\, \mathrm{nn}(\theta; t).$$

The network parameters θ are trained to minimize the composite loss

$$\mathcal{L}(\theta) \;=\; \mathcal{L}_{\mathrm{ODE}}(\theta) + \lambda\, \mathcal{L}_{\mathrm{data}}(\theta),$$

where the ODE residual loss is

$$\mathcal{L}_{\mathrm{ODE}}(\theta) \;=\; \frac{1}{M} \sum_{j=1}^{M} \Big(m u_\theta''(t_j) + \delta_\theta(t_j) u_\theta'(t_j) + k u_\theta(t_j) \Big)^2,$$

and the data loss is

$$\mathcal{L}_{\text{data}}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N}\big(u_\theta(t_i)-u_i\big)^2 \;+\; \frac{1}{N}\sum_{i=1}^{N}\big(u'_\theta(t_i)-v_i\big)^2.$$

At convergence, the optimal damping coefficient is given by $\delta_{\theta^*}(t)$, where $\theta^* = \arg\min_\theta \mathcal{L}(\theta)$.

Implementation examples We consider the ordinary differential equation

$$u''(t) + (1+t)\,u'(t) + 2u(t) = 0, \qquad u(0) = 0,\; u'(0) = 1.$$

Here, the friction coefficient increases linearly with time, which causes the oscillations to be progressively damped until they vanish in the long run.

Using the Runge–Kutta numerical method (RK45), we obtain the solution shown in Fig. 13.1. From this example, we synthetically generate the dataset that is provided in Table 13.1.

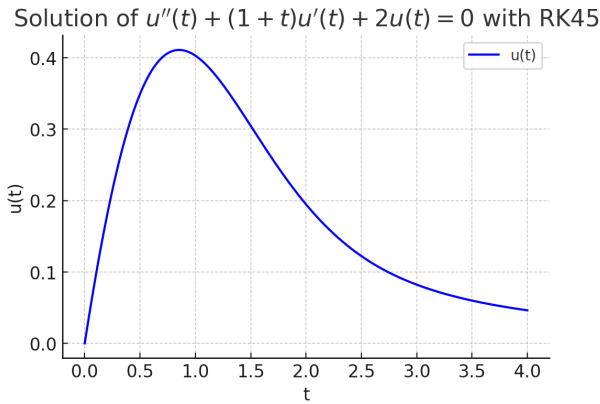


Figure 13.1: Solution $u(t)$ of a damped oscillator with a linearly increasing damping coefficient.

The code for all the methods in this section can be found at `https://tinyurl.com/5n8pd7dw`.

Method 1 We use a feed-forward neural network with 3 hidden layers and 64 neurons each to learn $u(t)$ from data (u_i, v_i). This part is accomplished successfully, as we can see from Fig. 13.2.

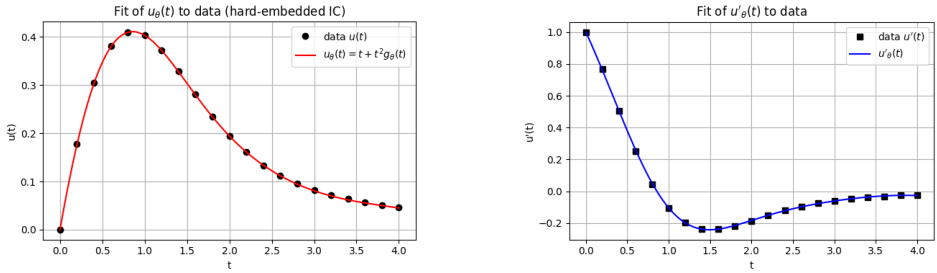
When we apply the formula

$$\delta(t) = -\frac{u''(t)+u'(t)}{u'(t)+10^{-6}},$$

we obtain the graph in Fig. 13.3(a), which exhibits a spurious oscillation near $t \approx 0.75$, precisely where $u'(t)$ is small. To reduce the influence of this instability, we restrict

t	$u(t)$	$u'(t)$
0.0	0.000000	1.000000
0.2	0.177684	0.768068
0.4	0.305064	0.504923
0.6	0.380565	0.254028
0.8	0.409627	0.045882
1.0	0.402791	-0.104120
1.2	0.372009	-0.194722
1.4	0.328383	-0.234554
1.6	0.280667	-0.237132
1.8	0.235033	-0.216965
2.0	0.194623	-0.185629
2.2	0.160880	-0.152011
2.4	0.133640	-0.120924
2.6	0.112141	-0.094754
2.8	0.095344	-0.074033
3.0	0.082209	-0.058261
3.2	0.071800	-0.046348
3.4	0.063464	-0.037521
3.6	0.056653	-0.030875
3.8	0.051008	-0.025857
4.0	0.046244	-0.021951

Table 13.1: Synthetic dataset of solution values $u(t)$ and derivatives $u'(t)$ obtained from the damped oscillator with linearly increasing damping.



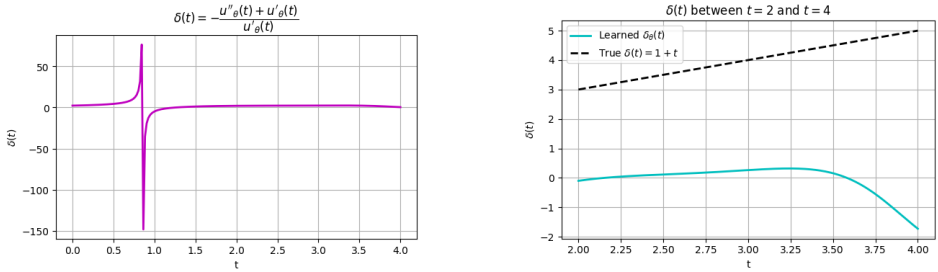
(a) **(b)**

Figure 13.2: A supervised training using data: (a) The recovery of $u(t)$; (b) The recovery of $u'(t)$.

attention to the interval $[2, 4]$ and plot $\delta(t)$ together with the true damping coefficient $\delta_{\text{true}}(t) = 1 + t$ in Fig. 13.3(b). The comparison shows that the recovered function does not match the expected result at all!

The reason is that this approach cannot recover the true $\delta(t)$: the governing ODE is never enforced in the training process. Instead, the network is only fitted to trajectory

data, and $\delta(t)$ is subsequently reverse-engineered from an approximate solution using a formula that is highly sensitive to numerical error. In particular, small discrepancies in $u_\theta(t)$ and $u'_\theta(t)$ become amplified when computing the second derivative $u''_\theta(t)$, leading to inaccurate and unstable estimates of $\delta(t)$.



(a) **(b)**

Figure 13.3: (a) The graph of $\delta_\theta(t)$; (b) The graph of $\delta_\theta(t)$ versus $\delta_{\text{true}}(t) = 1 + t$.

Method 2 We set the weight of the data term to $\lambda = 100$ and used a neural network architecture with three hidden layers of 64 neurons each, activated by the `swish` function. The trainable parameters for the damping coefficient were δ_0, δ_1, and δ_2, and the model was trained for $10,000$ epochs using the Adam optimizer with learning rate 3×10^{-3}.

The neural network successfully learns the solution $u(t)$, as shown in Fig. 13.4(a), while the learned damping coefficient $\delta(t)$ is displayed in Fig. 13.4(b) and closely matches the line $y = 1 + t$. The evolution of the coefficients δ_i over the training epochs is presented in Fig. 13.4(c). We observe that δ_0 converges most rapidly to 1, followed by δ_1, while δ_2 tends to 0. After $10,000$ epochs the coefficients are
`Final coefficients:delta0 = 0.991, delta1 = 0.998, delta2 = 0.005,`
which are very accurate approximations of the true values.

Remark 13.1.3 1. The higher the degree of the Taylor polynomial used to approximate $\delta(t)$, the slower the training becomes, requiring more epochs for convergence.

2. An interesting observation is that, even in the absence of data, using only the initial condition together with the ODE, the same method is able to recover the correct values of δ_0 and δ_1, if enough training is provided, see Fig. 13.4(d).

Method 3 This method is very similar to Method 2. See also Exercise 13.10.3.

Method 4 This method is more robust in the sense that it requires no prior assumptions on the parametrization of the function $\delta(t)$. The cost of this generality is a significantly longer training time in order to obtain an accurate solution.

We employed a feed-forward neural network with three hidden layers of 64 `swish`-activated neurons, with the initial conditions hard-embedded into the solution. Training was carried out for 50,000 epochs using the Adam optimizer with a learning rate of 10^{-5}. An L^2-regularization has been applied with the small weight $\lambda = 10^{-5}$. The

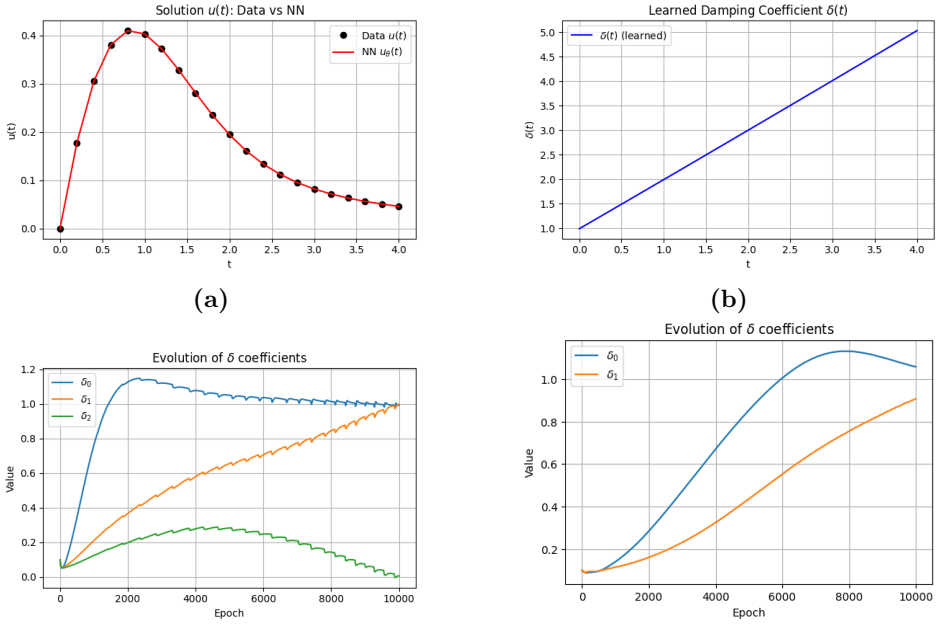


Figure 13.4: (a) The graph of $u_\theta(t)$; (b) The graph of $\delta_\theta(t)$ matches $\delta_{\text{true}}(t) = 1 + t$; (c) The evolution of δ_i, $i = 0, 1, 2$, across epochs; (d) The evolution of δ_0 and δ_1 in an unsupervised mode.

results are nearly exact, as illustrated in Fig. 13.5(a), and the loss successfully reached its threshold, as shown in Fig. 13.5(b).

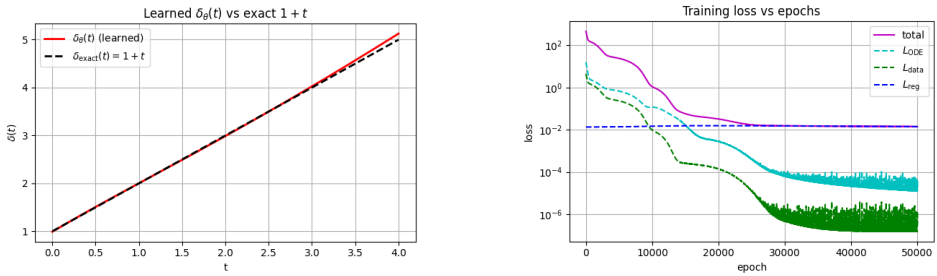


Figure 13.5: (a) The graph of $\delta_\theta(t)$ versus the exact solution; (b) The associated loss functions.

13.2 Finding all Time-dependent Coefficients

The previous method can be extended from identifying a single functional coefficient to determining all coefficients, provided they are either functions of time or constants.

We consider measurement data $(u_i, v_i) = (u(t_i), u'(t_i))$ of a one-dimensional dynamical system and aim to recover the differential equation satisfied by the solution. In other words, our goal is to identify the underlying physical law governing the system.

Since the equation represents a balance of forces, we include the following contributions:

- The inertial term $m(t)u''(t)$, where $m(t)$ is the (possibly time-varying) mass of the system, as in the case of a rocket losing fuel,
- The damping term $\delta(t)u'(t)$, proportional to velocity,
- The elastic term $k(t)u(t)$, proportional to displacement, where the elastic constant $k(t)$ may vary with elongation.

Let

$$P(m, \delta, k) \;=\; m(t)\frac{d^2}{dt^2} \;+\; \delta(t)\frac{d}{dt} \;+\; k(t)$$

denote the differential operator with time-dependent coefficients. The problem is then to determine the coefficient functions such that

$$Pu(t) \;=\; 0,$$

where $u(t)$ is consistent with the observed measurements.

We address this problem by considering cases of increasing complexity for the coefficients as follows:

Step 1. Constant coefficients Assume that m, δ, and k are constant. We learn the solution $u(t)$ through a hybrid approach (PINN plus data), using a neural network $u_w(t)$ with augmented parameters

$$w = (\theta, m, \delta, k),$$

where the unknown coefficients are treated as trainable variables of the network. If this procedure does not yield convergent values for the coefficients, we proceed to the next step.

Step 2. Linear coefficients Suppose that the coefficients vary at most linearly in time, so that

$$\begin{aligned} m(t) &= m_0 + m_1 t, \\ \delta(t) &= \delta_0 + \delta_1 t, \\ k(t) &= k_0 + k_1 t. \end{aligned}$$

In this case, we approximate the coefficients by training a neural network with augmented parameters

$$w = (\theta, m_0, m_1, \delta_0, \delta_1, k_0, k_1),$$

where the network $u_w(t)$ is used to approximate the solution $u(t)$. If the training fails to converge for all extra parameters, it indicates that a linear model is not appropriate for these coefficients. Then we move to the next step.

Step 3. General functional coefficients If the coefficients exhibit more complex, unknown variations, we approximate them directly with neural networks:

$$\begin{aligned} m(t) &= m_\theta(t), \\ \delta(t) &= \delta_\theta(t), \\ k(t) &= k_\theta(t). \end{aligned}$$

We then train the network

$$N(\theta; t) = \big(u_\theta(t),\, m_\theta(t),\, \delta_\theta(t),\, k_\theta(t)\big)$$

using both the ODE and the observed data. If from the previous step we identified that one or more coefficients can be modeled linearly, we retain those as linear functions (with their parameters treated as augmented network variables), while representing only the remaining coefficients by neural networks. This case requires the longest training time, as it provides the greatest modeling flexibility.

13.3 First Order ODE Discovery

We assume that a dataset $\{(t_i, u_i)\}_{i=1}^N$ is given, and our task is to identify a first-order ODE whose solution $u(t)$ interpolates the data, i.e. $u(t_i) = u_i$. The underlying equation may be linear or nonlinear in u, and it may be either autonomous or explicitly time-dependent (non-autonomous).

The autonomous case. We seek a smooth function $f(u)$ such that the dynamics of the system are governed by the autonomous ODE

$$u'(t) \;=\; f\big(u(t)\big),$$

with a solution satisfying the conditions $u(t_i) = u_i$ at given data points.

To solve this problem, we employ two neural networks:

(1) The network $u_\theta(t)$, which approximates the trajectory $u(t)$,

(2) The network $f_\theta(u)$, which learns the underlying function $f(u)$.

The two networks are trained simultaneously by minimizing a composite loss that combines a data loss with an ODE consistency loss:

$$\mathcal{L}_{\text{tot}}(\theta) \;=\; \mathcal{L}_{\text{data}}(\theta) \;+\; \mathcal{L}_{\text{ODE}}(\theta).$$

The data loss enforces agreement with the observed measurements:

$$\mathcal{L}_{\text{data}}(\theta) \;=\; \frac{1}{N} \sum_{i=1}^{N} \big(u_\theta(t_i) - u_i\big)^2,$$

while the ODE loss enforces consistency with the learned dynamics:

$$\mathcal{L}_{\mathrm{ODE}}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N}\big(u'_\theta(t_i) - f_\theta(u_i)\big)^2 \;+\; \frac{1}{M}\sum_{j=1}^{M}\big(u'_\theta(\tau_j) - f_\theta(u_\theta(\tau_j))\big)^2.$$

The first term of $\mathcal{L}_{\mathrm{ODE}}$ uses only the collocation points (t_i, u_i) from the dataset, while the second term optionally includes additional time instances τ_j, at which the trial solution $u_\theta(t)$ must be evaluated.

After training, the optimal parameters θ^* yield both the solution $u_{\theta^*}(t)$ and the recovered dynamical law $f_{\theta^*}(u)$.

The non-autonomous case. We now seek a smooth function $f(u,t)$ such that the system is governed by the ODE

$$u'(t) \;=\; f\big(u(t), t\big).$$

This case follows the same approach as in the autonomous setting. The data loss remains unchanged, while the ODE consistency loss becomes

$$\mathcal{L}_{\mathrm{ODE}}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N}\big(u'_\theta(t_i) - f_\theta(u_i, t_i)\big)^2 \;+\; \frac{1}{M}\sum_{j=1}^{M}\big(u'_\theta(\tau_j) - f_\theta(u_\theta(\tau_j), \tau_j)\big)^2.$$

Here the function f_θ explicitly depends on both the state u and time t, allowing the learned dynamics to capture non-autonomous behavior.

Implementation example We aim to recover the first-order ODE

$$u'(t) \;=\; -\tfrac{1}{2}\big(u(t) - 50\big),$$

using 15 measurements (t_i, u_i) reported in Table 13.2.

We adopt the autonomous formulation, where $u_\theta(t)$ approximates the trajectory and $f_\theta(u)$ approximates the underlying dynamical law. Both are modeled by feed-forward neural networks with three hidden layers of 64 `tanh`-activated neurons. Training was carried out for 12,000 epochs using the Adam optimizer with a learning rate of 10^{-3}.

The resulting learned trajectory overlapped almost exactly with the true analytical solution

$$u_{\mathrm{true}}(t) = 50 + 50e^{-0.5t},$$

as shown in Fig. 13.6(a). The recovered function $f_\theta(u)$ is nearly linear; in Fig. 13.6(b) we compare it against the exact relation

$$f_{\mathrm{true}}(u) = -0.5\,(u - 50).$$

The fit is very accurate for smaller values of u (where $u(t)$ varies slowly), while the discrepancy increases near $u \approx 100$, corresponding to the region where the solution decays most rapidly.

Keeping the same architectures for $u_\theta(t)$ and $f_\theta(u)$ but changing only the activation function from `tanh` to `swish` yields qualitatively better results, including in the region $u \approx 100$, see Fig. 13.6(c). This highlights the fact that certain activation functions can capture the dynamics more effectively than others. The reader can find the code at:

`https://tinyurl.com/3e2af3h3`

t	$u(t)$
0.000	100.000
0.714	84.984
1.429	74.477
2.143	67.126
2.857	61.983
3.571	58.384
4.286	55.866
5.000	54.104
5.714	52.872
6.429	52.009
7.143	51.406
7.857	50.984
8.571	50.688
9.286	50.482
10.000	50.337

Table 13.2: Numerical values of the solution $u(t)$ for $u'(t) = -0.5(u(t)-50)$, $u(0) = 100$.

13.4 Second Order ODE Discovery

We assume that a dataset $\{(t_i, u_i, u_i')\}_{i=1}^N$ is given, and our task is to identify a second-order ODE whose solution $u(t)$ and derivative $u'(t)$ interpolate the data, i.e. $u(t_i) = u_i$ and $u'(t_i) = u_i'$. The underlying equation may be linear or nonlinear in u, and may be either autonomous or explicitly time-dependent (non-autonomous).

The autonomous case. We seek two neural networks, $u_\theta(t)$ and $f_\theta(u, u')$, trained simultaneously so that

$$u''(t) = f(u, u'), \qquad u(t_i) = u_i, \quad u'(t_i) = u_i', \quad i = 1, \ldots, N.$$

The training objective is given by the composite loss

$$\begin{aligned}\mathcal{L}(\theta) \;&=\; \frac{1}{N}\sum_{i=1}^{N}\big(u_\theta(t_i) - u_i\big)^2 + \frac{1}{N}\sum_{i=1}^{N}\big(u_\theta'(t_i) - u_i'\big)^2 \\ &+\; \frac{1}{N}\sum_{i=1}^{N}\big(u_\theta''(t_i) - f_\theta(u_i, u_i')\big)^2 \;+\; \frac{1}{M}\sum_{j=1}^{M}\big(u_\theta''(\tau_j) - f_\theta(u_\theta(\tau_j), u_\theta'(\tau_j))\big)^2.\end{aligned}$$

The final term enforces the ODE at additional collocation points $\tau_j \notin \{t_i\}$ and is optional, to be included when higher fidelity is required.

The non-autonomous case. The formulation is entirely analogous to the autonomous case, except that the right-hand side depends explicitly on time, i.e.

$$u''(t) = f(u, u', t).$$

Accordingly, the neural network f_θ is taken as a function of (u, u', t), and the same loss structure applies, with the additional time variable included.

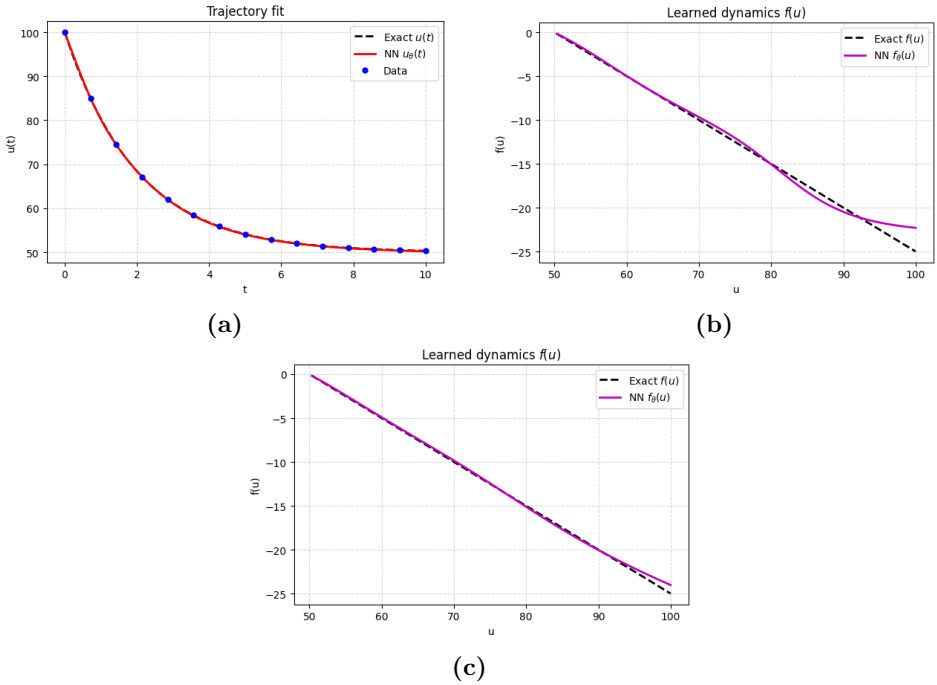


Figure 13.6: Hybrid learning for a first-order ODE: (a) Comparison between the theoretical and neural network solutions, showing perfect overlap; (b) Recovered function $f_{\theta^*}(u)$ versus the true function $f(u)$ for *tanh*-activated neurons; (c) Recovered function $f_{\theta^*}(u)$ versus the true function $f(u)$ for *swish*-activated neurons.

Implementation example We aim to recover the second-order ODE

$$u''(t) \;=\; 3u'(t) - 2u(t),$$

from a finite set of measurements. For validation purposes, we use the exact analytical solution

$$u_{\text{exact}}(t) \;=\; e^t - e^{2t}.$$

To construct a dataset, we sample $u(t)$ and $u'(t)$ at 10 equally spaced points in the interval $t \in [-2, 0.5]$. These synthetic measurements will serve as training data for the neural network approach. The computed values are presented in Table 13.3.

Training setup. We trained two networks, u_θ and f_θ, each with three hidden layers of 64 `swish` units, for 12,000 epochs using the learning-rate schedule

$$\text{learning rate} = \begin{cases} 10^{-3}, & \text{if epochs} \le 2,000 \\ 6\times 10^{-4}, & \text{if } 2,000 < \text{epochs} \le 5,000 \\ 10^{-4}, & \text{if } 5,000 < \text{epochs} \le 8,000 \\ 5\times 10^{-5}, & \text{if } 8,000 < \text{epochs} \le 10,000 \\ 10^{-5}, & \text{if } 10,000 < \text{epochs} \le 15,000 \end{cases}$$

t	$u(t)$	$u'(t)$
-2.000	0.11702	0.09870
-1.722	0.14675	0.11482
-1.444	0.18024	0.12460
-1.167	0.21443	0.11746
-0.889	0.24210	0.07309
-0.611	0.24817	-0.04640
-0.333	0.20311	-0.31030
-0.056	0.05112	-0.84372
0.222	-0.31077	-1.87040
0.500	-1.06956	-3.78784

Table 13.3: Synthetic dataset for the ODE $u''(t) = 3u'(t) - 2u(t)$ with exact solution $u(t) = e^t - e^{2t}$, sampled at 10 equally spaced points in $t \in [-2, 0.5]$.

which facilitates a smooth non-oscillating loss function. Consequently, the learned solution $u_\theta(t)$ matches the exact $u(t)$ essentially indistinguishably over the interval $[-2, 0.5]$, see Fig. 13.7.

Field recovery and local linearization. However, the velocity field $f_\theta(u, u')$ is less accurate. A linearization of f_θ at $t = -1.167$, yields

$$(a_{00}, a_{10}, a_{01}) = (-0.031,\ -1.946,\ 3.135),$$

so that, in the non-centered form,

$$f_\theta(u, u') \approx a_{00} + a_{10}\, u + a_{01}\, u' \ \approx\ -0.031 \ -\ 1.946\, u \ +\ 3.135\, u'.$$

This is close, but not identical, to the true field $f(u, u') = 3u' - 2u$.

Why this happens and how to fix it? The data $\{(u(t_i), u'(t_i))\}$ come from a single trajectory

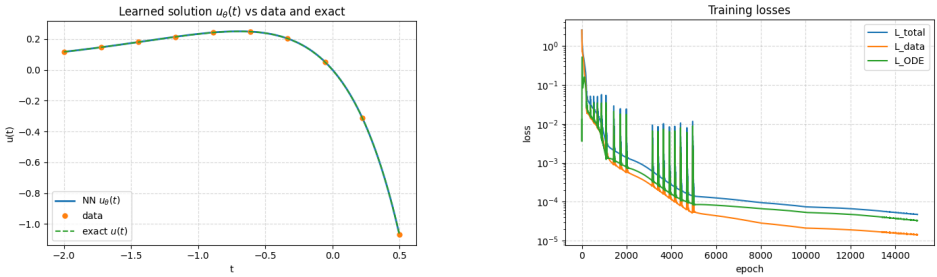
$$\Gamma \ =\ \{\, (u(t), u'(t)) \,\} \ \subset\ \mathbb{R}^2,$$

i.e., a one-dimensional curve in the (u, u') plane. Recovering $f(u, u')$ means identifying a surface over a two-dimensional domain. With only one curve, the problem is underdetermined off Γ: many distinct fields agree along the observed trajectory but differ elsewhere. Consequently, $u_\theta(t)$ can fit well, while f_θ away from the curve is weakly constrained. A practical remedy is to provide data from multiple trajectories (diverse initial conditions) so that samples populate a 2D region in (u, u'), making the regression for $f(u, u')$ identifiable and improving recovery of the true velocity field.

The code details can be found at `https://tinyurl.com/5ycs49hy`

13.5 Euler-Lagrange Equation Discovery

The goal is to recover both the Lagrangian and the associated Euler–Lagrange equations that generate a trajectory $u(t)$, given N observed states (u_i, u_i'). Since the Euler–Lagrange equations are second-order ODEs, this naturally leads to a physics-informed learning framework.



(a) **(b)**

Figure 13.7: (a) The perfect recovery of the solution $u_{\text{exact}}(t) = e^t - e^{2t}$; (b) The associated total loss reaches saturation in an free-oscillations mode.

The natural Lagrangian case. We begin with the assumption that the observed states correspond to a solution of the Euler–Lagrange equations associated with the natural Lagrangian

$$L(q, \dot{q}) \;=\; \tfrac{1}{2}\dot{q}^2 - U(q).$$

For simplicity we present the one-dimensional case, but the extension to n dimensions is straightforward and left to the reader.

Learning the Lagrangian is therefore equivalent to learning the potential function $U(q)$. To achieve this, we introduce two neural networks: $t \mapsto q_\theta(t)$ to approximate the trajectory, and $q \mapsto U_\theta(q)$ to approximate the potential.

The networks are trained to satisfy the data conditions

$$q_\theta(t_i) \approx q_i, \qquad q'_\theta(t_i) \approx q'_i,$$

and to enforce the Euler–Lagrange equation

$$\ddot{q}(t) = -U'(q(t)).$$

The total loss is

$$\mathcal{L}(\theta) \;=\; \mathcal{L}_{\text{data}}(\theta) + \mathcal{L}_{\text{EL}}(\theta),$$

with

$$\mathcal{L}_{\text{data}}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N} \big(q_\theta(t_i) - q_i\big)^2 + \frac{1}{N}\sum_{i=1}^{N} \big(q'_\theta(t_i) - q'_i\big)^2,$$

and

$$\mathcal{L}_{\text{EL}}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N} \big(\ddot{q}_\theta(t_i) + U'_\theta(q_i)\big)^2 + \frac{1}{M}\sum_{j=1}^{M} \big(\ddot{q}_\theta(\tau_j) + U'_\theta(q_\theta(\tau_j))\big)^2.$$

After training, we obtain both the learned trajectory $q_{\theta^*}(t)$ and the potential $U_{\theta^*}(q)$. From these, the Euler–Lagrange equations can be readily deduced.

The general Lagrangian case. We now move beyond the natural Lagrangian and allow for a more general form that may depend explicitly on time, on the coordinate q, and on the velocity $\dot{q}$. To this end, we employ two neural networks:

- $t \mapsto q_\theta(t)$, which approximates the trajectory of the system,
- $(q, \dot{q}, t) \mapsto L_\theta(q, \dot{q}, t)$, which approximates the Lagrangian itself.

The *data loss* remains the same as in the natural case:

$$\mathcal{L}_{\text{data}}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N}\big(q_\theta(t_i) - q_i\big)^2 + \frac{1}{N}\sum_{i=1}^{N}\big(q'_\theta(t_i) - q'_i\big)^2.$$

The Euler–Lagrange equations in this setting read

$$\frac{d}{dt}\Big(\frac{\partial L}{\partial \dot{q}}(q, \dot{q}, t)\Big) - \frac{\partial L}{\partial q}(q, \dot{q}, t) \;=\; 0.$$

To enforce them, we define the *Euler–Lagrange loss*:

$$\mathcal{L}_{\text{EL}}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N}\left[\frac{d}{dt}\Big(\frac{\partial L_\theta}{\partial \dot{q}}(q_i, q'_i, t_i)\Big) - \frac{\partial L_\theta}{\partial q}(q_i, q'_i, t_i)\right]^2.$$

As in the previous case, additional collocation points τ_j may be introduced to further enforce the Euler–Lagrange structure across the entire time domain:

$$+\,\frac{1}{M}\sum_{j=1}^{M}\left[\frac{d}{dt}\Big(\frac{\partial L_\theta}{\partial \dot{q}}(q_\theta(\tau_j), q'_\theta(\tau_j), \tau_j)\Big) - \frac{\partial L_\theta}{\partial q}(q_\theta(\tau_j), q'_\theta(\tau_j), \tau_j)\right]^2.$$

The total training objective is then

$$\mathcal{L}(\theta) \;=\; \mathcal{L}_{\text{data}}(\theta) + \mathcal{L}_{\text{EL}}(\theta),$$

whose minimizer θ^* simultaneously yields an approximation of the trajectory $q_{\theta^*}(t)$ and the underlying Lagrangian $L_{\theta^*}(q, \dot{q}, t)$.

This approach allows us not only to recover the equations of motion but also to uncover the underlying physical law encoded in the Lagrangian itself, even when it depends explicitly on time or when the system has nonlinear and non-standard dynamics.

Implementation example We consider a particle of unit mass moving under the influence of a one-dimensional potential field $U(q)$. Given 10 measurements of the particle's state $(t_i, q(t_i), q'(t_i))$, as listed in Table 13.4, our objectives are:

(*i*) To recover the potential field $U(q)$;

(*ii*) To reconstruct the trajectory of the particle $q(t)$.

We aim to identify the closest trajectory consistent with the measurements that can be described by a natural Lagrangian, while simultaneously learning the associated potential function $U(q)$.

To achieve this, we employed two neural networks: a *q-net* to approximate the trajectory and a *U-net* to approximate the potential. Both networks consist of three

t	$q(t)$	$q'(t)$
0.000	1.000	-0.750
0.556	1.046	-0.376
1.111	1.209	0.416
1.667	1.534	1.623
2.222	2.085	3.303
2.778	2.965	5.565
3.333	4.330	8.594
3.889	6.404	12.664
4.444	9.514	18.170
5.000	14.141	25.663

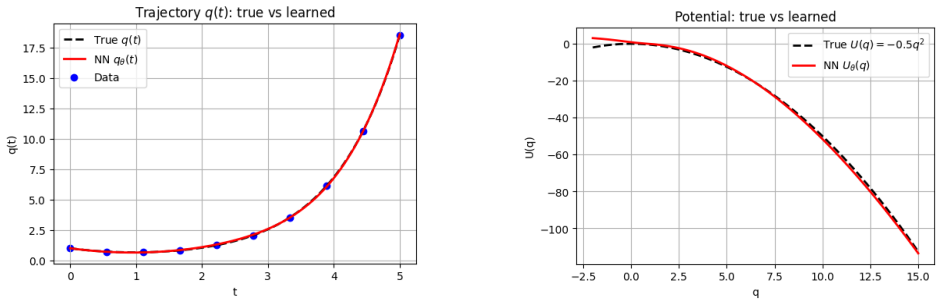
Table 13.4: Synthetic dataset generated from $q(t) = \cosh(t) - 0.75\sinh(t)$ and its derivative $q'(t) = \sinh(t) - 0.75\cosh(t)$, sampled at 10 equally spaced points in $[0, 5]$.

hidden layers with 64 neurons each, using the swish activation function. In the q-net, the initial conditions are hard-embedded to guarantee that the learned solution respects the prescribed boundary data.

Training was performed for $7,000$ epochs using the Adam optimizer with a scheduled learning rate. The results indicate that the q-net successfully reproduces the exact trajectory $q(t)$, see Fig. 13.8(a), while the U-net closely recovers the true potential $U(q) = -\frac{1}{2}q^2$, see Fig. 13.8(b). Moreover, the phase-space trajectory $(q(t), q'(t))$ predicted by the networks and the analytical solution are virtually indistinguishable, as shown in Fig. 13.9(a). The loss reaches the saturation threshold and the learning rate schedule helps to remove spurious oscillations, see Fig. 13.9(b). The reader can find the appropriate code at:

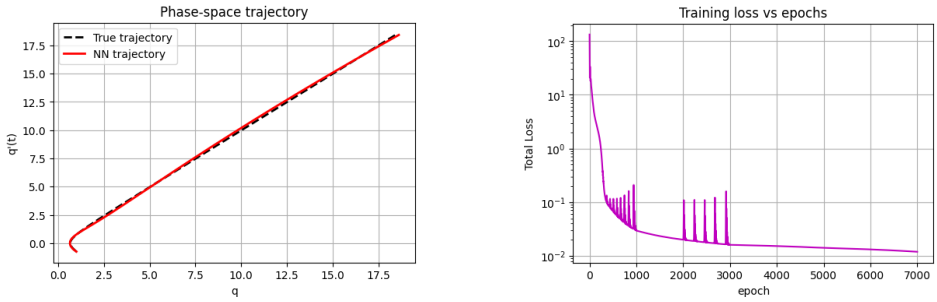
`https://tinyurl.com/bddb7rvu`

These findings demonstrate that the proposed method can accurately reconstruct both the particle trajectory and the governing potential from discrete state measurements.



(a) **(b)**

Figure 13.8: (a) The perfect recovery of the solution $u_{\text{exact}}(t) = \cosh t - 0.75\sin t$; (b) The almost perfect recovery of the quadratic potential.



(a) **(b)**

Figure 13.9: (a) The solution in phase space closely follows the true trajectory; (b) The loss function reaches saturation without oscillations.

13.6 Hamiltonian System Discovery

Suppose we are given measurements of position–momentum pairs (q_i, p_i) at discrete time instances t_i. Our goal is to reconstruct the closest Hamiltonian trajectory to these data. Equivalently, we seek a curve $(q(t), p(t))$ that not only approximates the observed data but is also generated by a Hamiltonian system, i.e. it satisfies

$$\dot{q}(t) \;=\; \partial_p H(q(t), p(t), t), \qquad \dot{p}(t) \;=\; -\partial_q H(q(t), p(t), t),$$

for some function $H(q, p)$. In cases where measurements at $t = 0$ are available, we may also embed the initial conditions

$$q(0) = q_0, \qquad p(0) = p_0.$$

Raw solution. As a first step, we construct a regression curve that interpolates the given data without yet imposing any Hamiltonian structure. For this, we introduce a neural network

$$NN(\theta; t) \;=\; \big(q_\theta(t),\, p_\theta(t)\big),$$

with embedded initial conditions

$$q_\theta(t) = q_0 + t\,\mathrm{nn}^1(\theta; t), \qquad p_\theta(t) = p_0 + t\,\mathrm{nn}^2(\theta; t).$$

The network is trained by minimizing the data loss

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\Big[\big(q_\theta(t_i) - q_i\big)^2 + \big(p_\theta(t_i) - p_i\big)^2\Big].$$

The resulting curve

$$(q_{\text{raw}}(t), p_{\text{raw}}(t)) \;=\; (q_{\theta^*}(t), p_{\theta^*}(t)), \quad \theta^* = \arg\min \mathcal{L}_{\text{data}}(\theta),$$

is a purely statistical regression fit to the data. At this stage, no dynamical information from Hamiltonian mechanics has been incorporated.

Hamiltonian solution. To enforce the Hamiltonian structure, we introduce two coupled neural networks:

$$t \;\mapsto\; (q_\theta(t), p_\theta(t)), \qquad (q,p) \;\mapsto\; H_\theta(q,p),$$

which learn simultaneously both the trajectory and the underlying Hamiltonian. The total loss combines the data fidelity and the Hamiltonian consistency:

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \mathcal{L}_{\text{Ham}}(\theta),$$

where

$$\mathcal{L}_{\text{Ham}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\Big(\dot{q}_\theta(t_i) - \partial_p H_\theta(q_i,p_i)\Big)^2 + \frac{1}{N}\sum_{i=1}^{N}\Big(\dot{p}_\theta(t_i) + \partial_q H_\theta(q_i,p_i)\Big)^2.$$

Remark 13.6.1 While the raw solution may interpolate the observed data more flexibly, the Hamiltonian solution enforces the conservation laws and structural properties of Hamiltonian mechanics, potentially yielding more physically meaningful dynamics and better extrapolation beyond the training data.

Natural Hamiltonian. A substantial simplification arises when the dynamics are known to be governed by a *natural Hamiltonian* of the form

$$H(q,p) \;=\; \tfrac{1}{2}p^2 \;+\; U(q),$$

where $U(q)$ denotes the potential energy. In this setting, the problem reduces to recovering the potential function $U(q)$.

To achieve this, we employ two coupled neural networks:

$$t \;\mapsto\; (q_\theta(t), p_\theta(t)), \qquad q \;\mapsto\; U_\theta(q),$$

which are trained simultaneously to approximate both the trajectory $(q(t), p(t))$ and the potential energy $U(q)$.

The total loss is defined as

$$\mathcal{L}_{\text{tot}}(\theta) \;=\; \mathcal{L}_{\text{data}}(\theta) + \mathcal{L}_{\text{Ham}}(\theta),$$

where $\mathcal{L}_{\text{data}}(\theta)$ enforces agreement with the measured trajectory data, and

$$\mathcal{L}_{\text{Ham}}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N}\Big(\dot{q}_\theta(t_i) - p_i\Big)^2 \;+\; \frac{1}{N}\sum_{i=1}^{N}\Big(\dot{p}_\theta(t_i) + \partial_q U_\theta(q_i)\Big)^2,$$

which enforces consistency with Hamilton's equations for a natural Hamiltonian system.

Thus, by learning both the trajectory and the potential energy function, this approach reconstructs the natural Hamiltonian structure underlying the observed data.

13.7 PDE Discovery

In many situations in Physics and Engineering, phenomena are governed by partial differential equations (PDEs) whose coefficients may vary with respect to the spatial coordinates. For clarity, we restrict our discussion here to the one-dimensional spatial case, though the same considerations extend naturally to the n-dimensional setting.

We denote by

$$\mathcal{P}(x, \partial_x) \;=\; a(x)\,\partial_x^2 \;+\; b(x)\,\partial_x \;+\; c(x)$$

a second-order differential operator with smooth coefficient functions $a(x)$, $b(x)$, and $c(x)$.

Evolution equation A frequently encountered problem in Physics is the description of time-evolving phenomena, modeled by equations of the form

$$\begin{aligned} \partial_t u(t,x) &= \mathcal{P}(x, \partial_x) u(t,x), \\ u(0,x) &= \varphi(x), \end{aligned}$$

where $x \in \Omega = [a, b]$ and $t \geq 0$. Here $\varphi(x)$ represents the initial condition of the state variable $u(t, x)$. For instance, one may interpret $u(t, x)$ as the temperature distribution at location x and time t, with $\varphi(x)$ specifying the initial temperature profile.

The central problem is the following: *Given a set of measurements $u_{ij} = u(t_i, x_j)$ at N temporal collocation points $\{t_i\}_{i=1}^N$ and M spatial collocation points $\{x_j\}_{j=1}^M$, determine:*

1. *The solution $u(t, x)$;*
2. *The evolution operator $\mathcal{P}(x, \partial_x)$.*

Depending on the level of complexity assumed for the operator $\mathcal{P}(x, \partial_x)$, we shall discuss the solution in several stages:

Method 1. In this simplest setting, we assume that the coefficients of the operator are constants, namely $a(x) = a$, $b(x) = b$, and $c(x) = c$. Thus, the augmented parameter set of the neural network is $w = (\theta, a, b, c)$, where θ denotes the standard network weights.
We construct a neural network with the initial condition hard-embedded

$$u_w(t,x) \;=\; \varphi(x) + t\,\mathrm{nn}(w;\, t, x),$$

and train it by minimizing the total loss

$$\mathcal{L}_{\mathrm{tot}}(w) \;=\; \mathcal{L}_{\mathrm{PDE}}(w) \;+\; \lambda\,\mathcal{L}_{\mathrm{data}}(w),$$

where the PDE residual loss is

$$\mathcal{L}_{\mathrm{PDE}}(w) \;=\; \frac{1}{NM} \sum_{i=1}^{N} \sum_{j=1}^{M} \Big(\partial_t u_w(t_i, x_j) - \mathcal{P}(x_j, \partial_x) u_w(t_i, x_j) \Big)^2,$$

and the data mismatch loss is

$$\mathcal{L}_{\text{data}}(w) \;=\; \frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M}\big(u_w(t_i,x_j)-u_{ij}\big)^2.$$

A similar approach can be used when the coefficient functions are assumed to belong to a known family of parameterized functions. For instance, if the coefficients are known to be linear in x, we may write

$$a(x)=a_1x+a_2, \qquad b(x)=b_1x+b_2, \qquad c(x)=c_1x+c_2,$$

and extend the augmented parameter set to

$$w=(\theta,a_1,a_2,b_1,b_2,c_1,c_2).$$

Method 2. In this setting, no prior knowledge about the coefficients is assumed. Instead, each coefficient is learned as the output of a dedicated branch of a neural network:

$$NN(\theta;t,x) \;=\; \big(a_\theta(x),\, b_\theta(x),\, c_\theta(x),\, u_\theta(t,x)\big).$$

The solution is constructed with hard-embedded initial conditions as

$$u_\theta(t,x) \;=\; \varphi(x)+t\,\mathrm{nn}(\theta;t,x),$$

and the training minimizes the total loss

$$\mathcal{L}_{\text{tot}}(\theta) \;=\; \mathcal{L}_{\text{PDE}}(\theta)+\lambda\,\mathcal{L}_{\text{data}}(\theta),$$

as in Method 1. This approach is more flexible but typically requires more training epochs than the constant or linear coefficient cases.

Method 3. In certain areas of Physics—for instance, shock waves or nonlinear diffusion processes—the time evolution is governed by a *nonlinear differential operator*. In this case we assume that the PDE can be expressed in the general form

$$\begin{aligned}\partial_t u &= F\big(x,u,\partial_x u,\partial_x^2 u\big), \quad t\ge 0\\ u(0,x) &= \varphi(x), \qquad x\in\Omega,\end{aligned}$$

where the nonlinear function F is unknown and must be learned from the data.

To solve this problem we introduce two neural networks:

- a *u-network*, $(t,x)\mapsto u_\theta(t,x)=\varphi(x)+t\,\mathrm{nn}(\theta;t,x)$, with the initial condition embedded by construction,
- and an *F-network*, $(x,u,v,w)\mapsto F_\theta(x,u,v,w)$, which learns the nonlinear operator directly from its arguments $(u,\partial_x u,\partial_x^2 u)$.

The networks are trained jointly by minimizing the composite loss

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \frac{1}{N \cdot M} \sum_{i,j} \Big[\partial_t u_\theta(t_i, x_j) - F_\theta\big(x_j, u_{ij}, \partial_x u_\theta(t_i, x_j), \partial_x^2 u_\theta(t_i, x_j)\big) \Big]^2$$

where $\mathcal{L}_{\text{data}}$ enforces consistency with the measurements u_{ij} as before.

This approach is significantly more challenging than in the linear case. The main difficulty is that F_θ represents a function on a 4-dimensional space $(x, u, \partial_x u, \partial_x^2 u)$, while the training data u_{ij} live on the 2-dimensional surface (t, x). This mismatch often results in slower training and lower accuracy.

Improving accuracy. A natural way to enhance performance is to provide more data by considering multiple solutions corresponding to different initial conditions. Suppose we are also given measurements $\tilde{u}_{ij} = \tilde{u}(t_i, x_j)$ for another trajectory governed by the same nonlinear operator:

$$\begin{aligned} \partial_t \tilde{u} &= F(x, \tilde{u}, \partial_x \tilde{u}, \partial_x^2 \tilde{u}), \\ \tilde{u}(0, x) &= \tilde{\varphi}(x), \qquad x \in \Omega. \end{aligned}$$

In this case we augment the u-network to predict both trajectories:

$$(t, x) \;\mapsto\; \big(u_\theta(t, x),\ \tilde{u}_\theta(t, x)\big) = \big(\varphi(x) + t\,\mathrm{nn}(\theta; t, x),\ \tilde{\varphi}(x) + t\,\tilde{\mathrm{nn}}(\theta; t, x)\big),$$

while retaining a shared F_θ.

The updated loss then combines both datasets:

$$\mathcal{L}_{\text{tot}}(\theta) \;=\; \mathcal{L}_{\text{data}}(\theta) + \tilde{\mathcal{L}}_{\text{data}}(\theta) + \mathcal{L}_{\text{PDE}}(\theta) + \tilde{\mathcal{L}}_{\text{PDE}}(\theta),$$

where each component enforces consistency with the observed data and the PDE for u and $\tilde{u}$, respectively.

In practice, using multiple trajectories from different initial conditions provides richer coverage of the input space of $F(x, u, u_x, u_{xx})$, which greatly improves the stability and accuracy of the learned nonlinear operator.

Tricomi equation A classical example of a mixed-type PDE is the *Tricomi equation*,

$$y\,\partial_x^2 u(x, y) \;+\; \partial_y^2 u(x, y) \;=\; 0,$$

which arises in the mathematical theory of transonic fluid flow. In aerodynamics, when the velocity of a flow transitions from subsonic to supersonic, the governing PDE changes type:

- For $y > 0$, the equation is elliptic, describing subsonic flow.
- At $y = 0$, it degenerates along the sonic line.
- For $y < 0$, it becomes hyperbolic, corresponding to supersonic flow.

In what follows, we restrict attention to the elliptic region. Specifically, data was collected from

$$\Omega = [0,1] \times \left[\tfrac{1}{2}, 1\right],$$

which is contained in $\{y > 0\}$. In this regime the Tricomi equation is elliptic, and the boundary value problem is well posed. Our aim is to use Deep Learning to recover the coefficient $a(x,y)$ in the generalized equation

$$a(x,y)\,\partial_x^2 u(x,y) \;+\; \partial_y^2 u(x,y) \;=\; 0, \quad (x,y) \in \Omega,$$

from measurements of the solution $u(x,y)$ at collocation points (x_i, y_j), together with suitable boundary conditions.

Data generation. To illustrate, we generate synthetic data from a separable solution of the Tricomi equation in the elliptic region. The function is chosen as

$$u(x,y) \;=\; a(x)\,b(y), \qquad a(x) = \sin(\pi x),$$

with $b(y)$ satisfying

$$b''(y) \;=\; \pi^2 y\, b(y), \qquad b(0.5) = 0, \;\; b(1) = 1.$$

The resulting values $u_{ij} = u(x_i, y_j)$ are shown in the Table 13.5.

	$y_1 = 0.500$	$y_2 = 0.625$	$y_3 = 0.750$	$y_4 = 0.875$	$y_5 = 1.000$
$x_1 = 0.000$	0.000	0.000	0.000	0.000	0.000
$x_2 = 0.125$	0.000	0.146	0.273	0.355	0.383
$x_3 = 0.250$	0.000	0.271	0.505	0.656	0.707
$x_4 = 0.375$	0.000	0.354	0.660	0.858	0.924
$x_5 = 0.500$	0.000	0.383	0.707	0.924	1.000
$x_6 = 0.625$	0.000	0.354	0.660	0.858	0.924
$x_7 = 0.750$	0.000	0.271	0.505	0.656	0.707
$x_8 = 0.875$	0.000	0.146	0.273	0.355	0.383
$x_9 = 1.000$	0.000	0.000	0.000	0.000	0.000

Table 13.5: Synthetic data for $u(x,y) = a(x)b(y)$ with $a(x) = \sin(\pi x)$ and $b(y)$ solving $b''(y) = \pi^2 y b(y)$, $b(0.5) = 0$, $b(1) = 1$, collected from $\Omega = [0,1] \times [0.5, 1]$.

Task: The learning task is then to recover the true coefficient $a(x,y)$ from these data.

Method. Using the method introduced earlier, we approximate the coefficient $a(x,y)$ by a quadratic polynomial:

$$a(x,y) \;\approx\; a_{00} + a_{10}x + a_{01}y + a_{20}x^2 + a_{02}y^2 + a_{11}xy,$$

and treat the coefficients a_{ij} as trainable parameters. These are learned simultaneously with the neural network that approximates the solution. Specifically, we define

$$NN(w; x, y) \;=\; u_w(x,y),$$

where the augmented parameter set is $w = (\theta, a_{ij})$, with θ denoting the network weights and a_{ij} the polynomial coefficients.

The homogeneous Dirichlet boundary conditions are enforced by embedding them directly into the solution ansatz:

$$u_w(x,y) \;=\; x(1-x)\,(y-\tfrac{1}{2})\,\mathrm{nn}(w;x,y),$$

where $\mathrm{nn}(w;x,y)$ is a free neural network.

The training is guided by three loss components:

$$\mathcal{L}_{\mathrm{data}}(w) \;=\; \sum_{i,j} \big(u_w(x_i,y_j) - u_{ij}\big)^2,$$

which enforces agreement with the measured data,

$$\mathcal{L}_{\mathrm{PDE}}(w) \;=\; \sum_{i,j} \Big(a(x_i,y_j)\,\partial_x^2 u_w(x_i,y_j) \;+\; \partial_y^2 u_w(x_i,y_j)\Big)^2,$$

which penalizes violations of the governing PDE and

$$\mathcal{L}_{\mathrm{sym}}(w) = \sum_{i,j} \big(u_w(x_i,y_j) - u(1-x_i,y_j)\big)^2,$$

which enforces the fact that the solution is symmetric with respect to the line $x = 0.5$, see Table 13.5. The total loss is then

$$\mathcal{L}_{\mathrm{total}}(w) \;=\; \mathcal{L}_{\mathrm{PDE}}(w) \;+\; \lambda\,\mathcal{L}_{\mathrm{data}}(w),$$

with λ a weighting parameter that balances physics-based and data-based constraints. See Exercise 13.10.4.

13.8 Recovering Heat Kernels

In this section we show that heat kernels can be obtained either from experimental or simulated temperature data in a *data-driven* fashion, or in an *operator-driven* way by enforcing directly the differential equation and its defining properties. In both cases one recovers the fundamental solution $K(t,x,y)$ of the heat equation associated with a given operator $\mathcal{P}(x,\partial)$.

An important question, however, is whether one can also recover the underlying physical law itself, namely the operator $\mathcal{P}$, from knowledge of its heat kernel. As we shall show, this problem is not always well posed.

Definitions and properties Let

$$\mathcal{P}(x,\partial) = \sum_{i,j=1}^{n} a_{ij}(x)\,\partial_{x_i}\partial_{x_j} + \sum_{k=1}^{n} b_k(x)\,\partial_{x_k} + c(x), \tag{13.8.1}$$

be a second-order differential operator on $\mathbb{R}^n$, where the coefficients $a_{ij}(x)$, $b_k(x)$, and $c(x)$ are smooth functions. The *heat kernel* is a function

$$K : [0, \infty) \times \mathbb{R}^n \times \mathbb{R}^n \longrightarrow \mathbb{R} \tag{13.8.2}$$

satisfying the following conditions:

1. *Heat equation:*

$$\partial_t K(t, x, y) - \mathcal{P}(x, \partial) K(t, x, y) = 0, \quad t > 0. \tag{13.8.3}$$

2. *Initial condition:*

$$\lim_{t \to 0^+} K(t, x, y) = \delta(x - y), \tag{13.8.4}$$

where $\delta(x - y) = \delta_y(x)$ denotes the Dirac delta distribution centered at y.

Physical interpretation. The function $K(t, x, y)$ represents the amount of heat observed at position x and time t, given that at time $t = 0$ a unit heat impulse is applied at point y. Since the amount of heat must be positive, we have $K(t, x, y) > 0$, fact that can be enforced using a softplus function in the last layer of the network.

If, during the heat diffusion over $\mathbb{R}^n$, the total amount of heat is conserved, then the normalization condition

$$\int_{\mathbb{R}^n} K(t, x, y)\, dx = 1, \quad \forall\, t > 0, \tag{13.8.5}$$

holds for every fixed $y \in \mathbb{R}^n$. It can be shown, see Exercise 13.10.6, that this requirement is implied by the following relation among the coefficient functions of the operator:

$$c(x) \;=\; \sum_k \partial_{x_k} b_k(x) \;-\; \sum_{i,j} \partial_{x_i x_j} a_{ij}(x). \tag{13.8.6}$$

This condition holds for non-dissipative systems, namely systems for which

$$Q(t) = \int_{\mathbb{R}^n} u(t, x) dx = Q(0) = \int_{\mathbb{R}^n} \varphi(y) dy.$$

In addition, one expects the kernel to satisfy the *symmetry property*

$$K(t, x, y) = K(t, y, x), \quad t > 0. \tag{13.8.7}$$

Role in solving the heat equation. The knowledge of the heat kernel provides a powerful representation formula for solutions of the Cauchy problem

$$\begin{aligned} \partial_t u(t, x) &= \mathcal{P}(x, \partial) u(t, x), \quad t > 0, \\ u(0, x) &= \varphi(x), \end{aligned}$$

where φ is a given smooth initial condition on $\mathbb{R}^n$. In this setting, the solution can be written explicitly in terms of the heat kernel:

$$u(t, x) = \int_{\mathbb{R}^n} K(t, x, y)\, \varphi(y)\, dy. \tag{13.8.8}$$

This integral formula expresses the value of $u(t, x)$ as a superposition of the effects of unit impulses, propagated through time by the heat kernel.

ML approach to heat kernels In geometric analysis, the construction of the heat kernel K relies on the intrinsic properties of the underlying differential operator $\mathcal{P}$—for example, its symmetries or the geometric structures it induces, such as distances. In Quantum Physics, the heat kernel is interpreted as a propagator and can be obtained through Feynman path integrals. Several methods for computing and approximating the heat kernel are developed in the monographs of Avramidi [7] and Calin et al. [16].

In contrast, Deep Learning offers alternative strategies for discovering the heat kernel:

1. *Data-driven (supervised) learning.* If temperature measurements $u_{ij} = u(t_i, x_j)$ are available, along with initial conditions $\varphi(x)$ for a sufficiently rich set of initial profiles, neural networks can be trained to approximate the heat kernel directly from data. This corresponds to a supervised learning approach.

2. *Operator-driven (unsupervised) learning.* If only the differential operator $\mathcal{P}$ is known, the heat kernel can be learned by training neural networks to satisfy the defining PDE and associated constraints, without explicit labeled data.

3. *Hybrid learning.* When both the operator $\mathcal{P}$ and measurement data u_{ij} are available, one can combine physics-informed learning with data supervision. This hybrid strategy leverages both theoretical structure and empirical evidence to approximate K more effectively.

We shall discuss each of them in the next paragraphs.

1. Data-driven (supervised) learning Given the temperature $u(t, x)$ and the initial condition $\varphi(x)$, the integral relation

$$u(t, x) = \int_{\mathbb{R}^n} K(t, x, y)\, \varphi(y)\, dy \tag{13.8.9}$$

is not necessarily well-posed. In fact, the associated integral operator

$$T(\varphi)(x) \;=\; \int_{\mathbb{R}^n} K(t, x, y)\, \varphi(y)\, dy$$

is typically *not invertible* in most standard function space settings.

To address this issue, we enrich the data by considering the temperature evolution from several different initial profiles $\varphi^k(x)$. This leads to a family of m integral equations that share the same kernel:

$$u^k(t, x) \;=\; \int_{\mathbb{R}^n} K(t, x, y)\, \varphi^k(y)\, dy, \quad 1 \le k \le m.$$

We then record the temperature fields $u^k(t, x)$ at a collection of collocation points (t_i, x_j),

$$u^k_{i,j} \;=\; u^k(t_i, x_j).$$

The problem is now reformulated as follows:

Problem *Determine the heat kernel $K(t,x,y)$ given the measured data $(u^k_{i,j}, \varphi^k)$, subject to the consistency relations*

$$\int_{\mathbb{R}^n} K(t_i, x_j, y)\, \varphi^k(y)\, dy \;=\; u^k_{i,j}, \quad 1 \le k \le m.$$

We shall learn the kernel using neural networks. Sometimes, it is useful to consider the following natural hard embedding of the kernel

$$K_\theta(t,x,y) \;=\; V_\theta(t)\, e^{S_\theta(t,x,y)},$$

where the two neural networks are:

- a *volume network* $t \mapsto V_\theta(t) = \mathrm{sp}(\mathrm{nn}(\theta; t)) > 0$,
- an *action network* $(t,x,y) \mapsto S_\theta(t,x,y)$.

In order to better reflect geometric structure, the action network can itself be written in the hard-embedded form

$$S_\theta(t,x,y) \;=\; (x-y)\, \mathrm{nn}(\theta; x, y),$$

where $\mathrm{nn}(\theta; x, y)$ is a standard feed-forward neural network.

The kernel must reproduce the observed temperature data through the relation

$$V_\theta(t_i) \int_{\mathbb{R}^n} e^{S_\theta(t_i, x_j, y)}\, \varphi^k(y)\, dy \;=\; u^k_{ij}.$$

This motivates the data loss

$$\mathcal{L}_{\mathrm{data}}(\theta) = \frac{1}{nmN} \sum_{k,i,j} \left(u^k_{ij} - V_\theta(t_i) \int_{\mathbb{R}^n} e^{S_\theta(t_i, x_j, y)}\, \varphi^k(y)\, dy \right)^2, \tag{13.8.10}$$

where $1 \le i \le n$, $1 \le j \le m$, and $1 \le k \le N$.

To incorporate the *symmetry property* (13.8.7), we add the loss

$$\mathcal{L}_{\mathrm{sym}}(\theta) = \frac{1}{nm^2} \sum_{i,j,r} \Big(S_\theta(t_i, x_j, y_r) - S_\theta(t_i, y_r, x_j) \Big)^2. \tag{13.8.11}$$

Equivalently, we can bypass the symmetry loss, imposing the symmetry by the hard embedding

$$S_\theta(t,x,y) = \frac{1}{2}(g_\theta(t,x,y) + g_\theta(t,y,x))$$

for a neural network $(t,x,y) \to g_\theta(t,x,y)$.

The total loss function is then defined as a weighted combination of these three contributions, using the symmetry and normalization losses as regularizers:

$$\mathcal{L}_{\mathrm{tot}}(\theta) = \mathcal{L}_{\mathrm{data}}(\theta) + \lambda\, \mathcal{L}_{\mathrm{sym}}(\theta), \tag{13.8.12}$$

where $\lambda > 0$ is a hyperparameter controlling the relative importance of each constraint.

2. Operator-driven (unsupervised) learning In the absence of measurement data, the heat kernel $K(t, x, y)$ can be learned in an *unsupervised* manner by approximating it with a neural network $K_\theta(t, x, y)$. The training is therefore *operator-driven*, as it relies exclusively on the structure of the differential operator $\mathcal{P}(x, \partial)$ and the defining properties of the heat kernel, such as the heat equation, the initial condition, normalization, and symmetry.

To ensure positivity of the learned kernel, one may parameterize it using the softplus activation:

$$K_\theta(t, x, y) = \mathrm{sp}\big(\mathrm{nn}(\theta; t, x, y)\big), \tag{13.8.13}$$

where $\mathrm{nn}(\theta; \cdot)$ denotes the output of a neural network.

The following loss terms are considered:

PDE residual loss. We enforce the heat equation at collocation points (t_i, x_j, y_r):

$$\mathcal{L}_{\mathrm{PDE}}(\theta) = \frac{1}{nm^2} \sum_{(t_i, x_j, y_r)} \left(\partial_t K_\theta(t_i, x_j, y_r) - \mathcal{P}(x_j, \partial)\, K_\theta(t_i, x_j, y_r)\right)^2.$$

Initial condition loss. The initial condition is enforced by approximating the Dirac delta $\delta(x - y)$ with a narrow Gaussian $G_\varepsilon(x - y)$

$$\mathcal{L}_{\mathrm{init}}(\theta) = \frac{1}{m^2} \sum_{(x_j, y_r)} \Big(K_\theta(0, x_j, y_r) - G_\varepsilon(x_j - y_r)\Big)^2,$$

where

$$G_\varepsilon(x - y) = \frac{1}{(4\pi\varepsilon)^{n/2}} \exp\left(-\frac{\|x - y\|^2}{4\varepsilon}\right),$$

with $\varepsilon > 0$ a small parameter controlling the width of the Gaussian.[1]

Symmetry loss. The kernel must satisfy the symmetry relation $K(t, x, y) = K(t, y, x)$:

$$\mathcal{L}_{\mathrm{sym}}(\theta) = \frac{1}{nm^2} \sum_{(t_i, x_j, y_r)} \Big(K_\theta(t_i, x_j, y_r) - K_\theta(t_i, y_r, x_j)\Big)^2.$$

Normalization loss. Moreover, if conservation condition (13.8.6) holds, then the normalization condition (13.8.5) is enforced by

$$\mathcal{L}_{\mathrm{norm}}(\theta) = \frac{1}{nm} \sum_{i,j} \left(\int_{\mathbb{R}^n} K_\theta(t_i, x_j, y)\, dy - 1\right)^2. \tag{13.8.14}$$

The total loss functional is then given by

$$\mathcal{L}_{\mathrm{tot}}(\theta) = \mathcal{L}_{\mathrm{PDE}}(\theta) + \lambda_1 \mathcal{L}_{\mathrm{init}}(\theta) + \lambda_2 \mathcal{L}_{\mathrm{sym}}(\theta) + \lambda_3 \mathcal{L}_{\mathrm{norm}}(\theta),$$

where $\lambda_1, \lambda_2, \lambda_3 > 0$ are balancing parameters.

[1] This is based on the fact that as $\varepsilon \to 0^+$, we recover the Dirac delta in the sense of distributions: $G_\varepsilon(x - y) \longrightarrow \delta(x - y)$.

Training is performed by sampling collocation points (t_i, x_j, y_r), computing the loss terms above, and updating the network parameters θ via backpropagation. After convergence, K_θ serves as an approximation of the true heat kernel.

3. Hybrid learning of the heat kernel We combine operator-driven constraints with measurement data. Let $K_\theta(t, x, y)$ be a neural parametrization of the heat kernel (e.g. $K_\theta(t, x, y) = V_\theta(t)\, e^{S_\theta(t,x,y)}$). Given initial profiles $\{\varphi^k\}_{k=1}^N$ and temperature measurements $u_{ij}^k = u^k(t_i, x_j)$, we minimize a joint loss:

$$\mathcal{L}_{\text{tot}}(\theta) = \alpha\, \mathcal{L}_{\text{data}}(\theta) + \beta\, \mathcal{L}_{\text{PDE}}(\theta) + \gamma\, \mathcal{L}_{\text{init}}(\theta) + \mu\, \mathcal{L}_{\text{sym}}(\theta) + \nu\, \mathcal{L}_{\text{norm}}(\theta),$$

with positive weights $\alpha, \beta, \gamma, \mu, \nu$.

Training. We alternate mini-batches of data points (t_i, x_j) (for $\mathcal{L}_{\text{data}}$) and physics collocation triples (t_i, x_j, y_r) (for $\mathcal{L}_{\text{PDE}}, \mathcal{L}_{\text{init}}, \mathcal{L}_{\text{sym}}, \mathcal{L}_{\text{norm}}$), and update θ by stochastic gradient descent. This hybrid objective exploits both measurement information and the analytic structure imposed by the operator $\mathcal{P}(x, \partial)$.

Non-uniqueness of the differential operator We have seen that, given a collection of initial temperature profiles $\varphi^k(x)$ and the corresponding temperature measurements $u_{ij}^k = u^k(t_i, x_j)$, one can reconstruct the associated heat kernel $K(t, x, y)$. A natural next question is whether one can also recover the underlying physical law itself–namely, the differential operator $\mathcal{P}(x, \partial)$ whose heat kernel is $K(t, x, y)$.

Unfortunately, this inverse problem is not, in general, well-posed. The following counterexample illustrates the issue.

Consider two nontrivial one-dimensional operators:

$$\mathcal{P}_1(x, \partial) \;=\; \partial_x^2 - \tfrac{1}{4}x^2, \quad \text{(the Hermite operator)},$$

and

$$\mathcal{P}_2(x, \partial) \;=\; \partial_x^2 + x\, \partial_x.$$

At first sight, $\mathcal{P}_1$ and $\mathcal{P}_2$ appear to be very different. Yet their heat kernels are both known in closed form and–quite remarkably–they coincide exactly (see Calin and Chang [17], pp. 190 and 223):

$$K(t, x, y) \;=\; \frac{1}{\sqrt{4\pi \sinh t}}\, \exp\!\left(-\frac{1}{4 \sinh t}\, \left[(x^2 + y^2) \cosh t - 2xy \right] \right), \quad t > 0.$$

Therefore, even if the heat kernel $K(t, x, y)$ can be uniquely reconstructed from data, the differential operator that generates it is not necessarily unique. Different physical laws may correspond to the same kernel, making the inverse problem of recovering $\mathcal{P}(x, \partial)$ from the kernel fundamentally ill-posed.

13.9 The Heat Kernel on $\mathbb{S}^1$

We seek the heat kernel on the unit circle $\mathbb{S}^1$, parameterized by the angle $s \in [0, 2\pi)$. By rotational invariance we may take the source at $s_0 = 0$, so we look for a 2π-periodic in s function $K(t; s)$ such that

$$\begin{aligned}(\partial_t - \partial_s^2)\, K(t;s) &= 0, \quad t > 0,\ s \in [0, 2\pi),\\ K(0;s) &= \delta(s)\\ K(t;0) &= K(t;2\pi)\\ \partial_s K(t;0) &= \partial_s K(t;2\pi)\\ \partial_s^2 K(t;0) &= \partial_s^2 K(t;2\pi).\end{aligned}$$

Then the heat kernel from s_0 to s will be

$$K(t; s_0, s) = K(t; 0, s - s_0) = K(0; s - s_0).$$

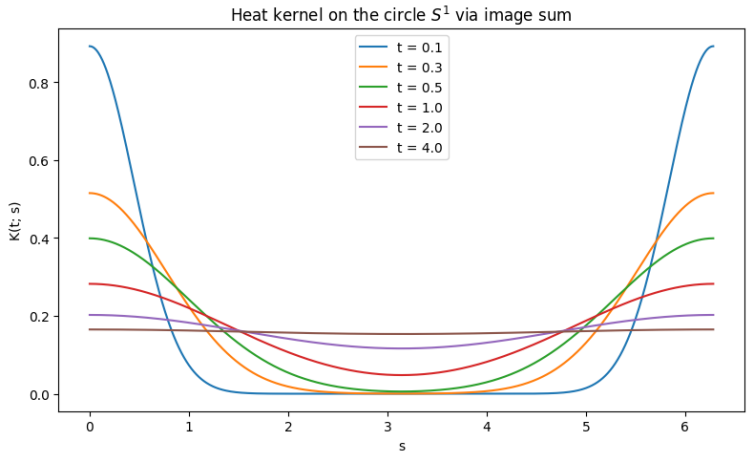


Figure 13.10: The heat kernel profiles for $t \in \{0.1, 0.3, 0.5, 1.0, 2.0, 4.0\}$ on the interval $[0, 2\pi]$.

The analytic formula of the heat kernel, obtained by a reflective geometric method, is given by a wrapped Gaussian contributions

$$K(t; s) = \frac{1}{\sqrt{4\pi t}} \sum_{k \in \mathbb{Z}} \exp\left(-\frac{(s + 2\pi k)^2}{4t}\right), \qquad s \in \mathbb{R}$$

see for instance Calin et al. [16], page 49. A few profiles for this heat kernel are plotted for $t \in \{0.1, 0.3, 0.5, 1.0, 2.0, 4.0\}$ on the interval $[0, 2\pi]$, see Fig 13.10. We note that in the long run the temperature stabilizes to the uniform distribution.

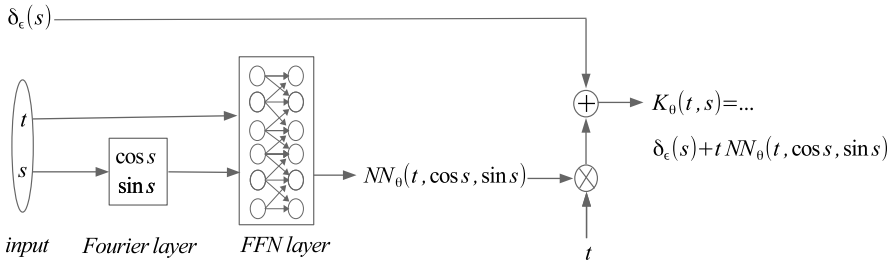


Figure 13.11: The schematic of a hard-embedding for the heat kernel neural network.

NN Approach We shall construct the heat kernel via neural networks and compare it against the aforementioned closed form expression.

Hard embeddings. We approximate the Dirac mass at $s_0 = 0$ on the circle (identified with $[0, 2\pi)$) by the periodic, piecewise Gaussian

$$\delta_\varepsilon(s) = \begin{cases} \dfrac{1}{\sqrt{4\pi}\,\varepsilon}\,\exp\Big(-\dfrac{s^2}{4\varepsilon^2}\Big), & 0 < s < \pi, \\ \dfrac{1}{\sqrt{4\pi}\,\varepsilon}\,\exp\Big(-\dfrac{(2\pi - s)^2}{4\varepsilon^2}\Big), & \pi < s < 2\pi. \end{cases}$$

This places half of the Gaussian "bell" to the right of $s_0 = 0$ (on $(0, \pi)$) and the other half to the left, wrapped around near 2π (on $(\pi, 2\pi)$), producing a 2π-periodic mollifier concentrated at $s = 0$.

To enforce periodicity on $K(t, \cdot)$, we feed the network with a Fourier layer $(\cos s, \sin s)$, and to encode the initial condition we use the aforementioned mollifier

$$K_\eta(t; s) \;=\; \delta_\varepsilon(s) \;+\; t\,\mathrm{NN}_\eta\big(t, \cos s, \sin s\big),$$

so that $K_\eta(0; s) \approx \delta_\varepsilon(s)$ and K_η is automatically 2π-periodic in s. See the schematic in Fig. 13.11.

Training objective. On a set of interior collocation points $\{(t_i, s_j)\}$ with $t_i > 0$, minimize the PDE residual subject to a normalization constraints (conservation of heat)

$$\mathcal{L}(\eta) \;=\; \frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M}\big(\partial_t K_\eta(t_i; s_j) - \partial_s^2 K_\eta(t_i; s_j)\big)^2 + \frac{1}{N}\sum_{i=1}^{N}\Big(\int_0^{2\pi} K_\eta(t_i, s)ds - 1\Big)^2.$$

Implementation. We used a shallow network NN_η with two hidden layers of 64 tanh-activated neurons. Training was performed for 5,000 epochs with the Adam optimizer and a small learning rate of 5×10^{-4}. This choice was made empirically after observing loss oscillations at higher learning rates. The learned kernel was validated against the

analytic expression, and profiles for $t = 0.1,\ 0.3,\ 1.0$ were plotted, see Fig. 13.12. We noticed a very good match between the analytic formula and the NN-formula. The code can be found at:

`https://tinyurl.com/34dhwxw8`

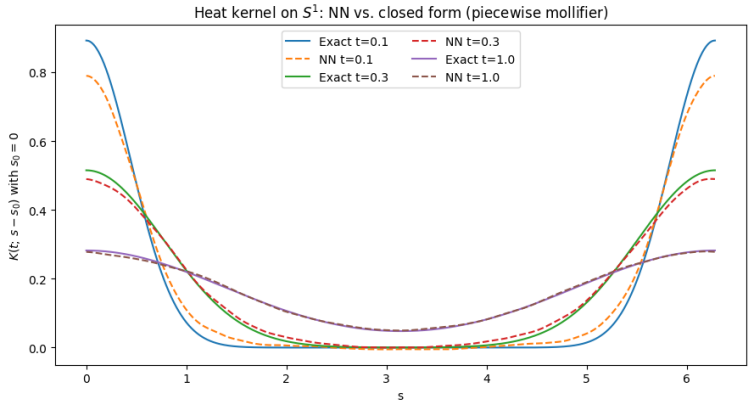


Figure 13.12: The NN-heat kernel profiles versus the analytic formula for $t \in \{0.1, 0.3, 1.0\}$ on the interval $[0, 2\pi]$.

13.10 Exercises

Exercise 13.10.1 Consider the first-order ODE with constant coefficients

$$au'(t) + bu(t) + c = 0.$$

(a) Collect 10 equally spaced data points from the curve

$$u(t) = e^{-t/2} - 1, \qquad t \in [0, 4],$$

and organize them into a table of measurements (t_i, u_i), $i = 1, \ldots, 10$.

(b) Using these measurements (t_i, u_i), train a neural network model to recover the coefficients a, b, and c of the given differential equation.

(c) Plot the evolution of the coefficients a, b, and c during training, and analyze their convergence behavior over the epochs. Compare with the ground truth coefficients $(a, b, c) = (2, 1, 1)$.

Exercise 13.10.2 (a) Collect 10 equally spaced data points from the curve

$$(q(t), p(t)) \ = \ (1 - t^2,\ -2t), \qquad t \in [0, 4],$$

and organize them into a table of measurements (q_i, p_i), $i = 1, \ldots, 10$.

(b) Using these measurements (q_i, p_i), train neural networks to recover the *natural Hamiltonian* that generates the corresponding Hamiltonian dynamics.

(c) Plot the neural network trajectory $(q_\theta(t), p_\theta(t))$ alongside the exact solution, and verify that the two curves are visually indistinguishable.

Exercise 13.10.3 Consider the oscillatory equation of a piston subject to time-dependent friction:

$$u''(t) + 2\cos(t + 0.5)\, u'(t) + u(t) = 0, \qquad u(0) = 0,\ u'(0) = 1.$$

(a) Solve the equation numerically using the Runge–Kutta method and collect 15 equally spaced data points

$$(u_i, u_i') = \big(u(t_i), u'(t_i)\big)$$

from the computed trajectory.

(b) Using the data from part (a), train a neural network to recover the friction coefficient in the form

$$\delta(t) \ = \ \delta_2 \cos(\delta_1 t + \delta_0),$$

treating $\delta_0, \delta_1, \delta_2$ as learnable parameters of the network.

(c) Plot the neural network solution $u(t)$ together with the reference solution obtained by the Runge–Kutta method, and show that the two curves are indistinguishable.

Exercise 13.10.4 Consider the Tricomi problem on $\Omega = [0, 1] \times \left[\frac{1}{2}, 1\right]$ with data collected as in Table 13.5.

(a) Use a neural network to recover a quadratic approximation of the coefficient $a(x, y)$:

$$a(x, y) \ \approx \ a_{00} + a_{10}x + a_{01}y + a_{20}x^2 + a_{02}y^2 + a_{11}xy,$$

treating the coefficients a_{ij} as trainable parameters alongside the network weights.

(b) Plot the evolution of the coefficients a_{ij} during training and analyze their convergence behavior over the epochs.

(c) If convergence stalls or thresholds are not reached, implement a scheduled learning rate strategy to accelerate training and improve accuracy.

Exercise 13.10.5 Let

$$V(x, y, z) = \big(\tfrac{yz}{x},\ z,\ -2y\big)$$

be a vector field on $\mathbb{R}^3$, and consider the equation

$$\operatorname{div}\big(a(x, y, z)\, V(x, y, z)\big) = 0.$$

We aim to determine the function $a(x, y, z)$ as follows.

(a) Collect N^3 velocity data points

$$V_{ijk} = V(x_i, y_j, z_k)$$

at collocation nodes (x_i, y_j, z_k), for some suitable integer N.

(b) We construct a latent A-net, which is a neural network vector potential on $\mathbb{R}^3$,

$$(x, y, z) \;\mapsto\; A_\theta(x, y, z),$$

such that the following hard constraint is enforced:

$$a_\theta(x, y, z)\, V(x, y, z) \;=\; \operatorname{curl} A_\theta(x, y, z),$$

where

$$a_\theta(x, y, z) \;\approx\; \alpha + \beta x + \gamma y + \delta z.$$

Treat the coefficients $(\alpha, \beta, \gamma, \delta)$ as internal variables of the network and write down a suitable loss function. Then implement a Keras program to minimize this loss and recover the function $a(x, y, z)$.

(c) Plot the evolution of the coefficients $(\alpha, \beta, \gamma, \delta)$ during training, and analyze their convergence behavior over the epochs.

(d) If convergence stalls or thresholds are not reached, implement a scheduled learning rate strategy to accelerate training and improve accuracy.

Exercise 13.10.6 Consider the second-order differential operator

$$\mathcal{P}(x, \partial) = \sum_{i,j=1}^{n} a_{ij}(x)\, \partial_{x_i} \partial_{x_j} + \sum_{k=1}^{n} b_k(x)\, \partial_{x_k} + c(x),$$

with smooth coefficient functions a_{ij}, b_k, c. We shall investigate the relation between the *conservation condition*

$$c(x) \;=\; \sum_{k=1}^{n} \partial_{x_k} b_k(x) \;-\; \sum_{i,j=1}^{n} \partial_{x_i x_j} a_{ij}(x), \tag{13.10.15}$$

and the *normalization property* of the heat kernel

$$\int_{\mathbb{R}^n} K(t, x, y)\, dx = 1, \quad t > 0.$$

Let $Q(t) = \int_{\mathbb{R}^n} u(t, x)\, dx$ denote the total heat at time t, where $u(t, x)$ solves the Cauchy problem

$$\partial_t u(t, x) \;=\; \mathcal{P}(x, \partial)\, u(t, x), \qquad u(0, x) = \varphi(x),$$

and let $Q_0 = \int_{\mathbb{R}^n} \varphi(x)\, dx$ be the initial heat.

(a) Assume that the conservation condition (13.10.15) holds. Show that

$$Q(t) = Q_0, \qquad \forall\, t > 0,$$

for every smooth initial condition φ.

Hint: Differentiate $Q(t)$ with respect to t and use integration by parts to prove that

$$Q'(t) = \int_{\mathbb{R}^n} \left(\sum_{i,j} \partial_{x_i x_j} a_{ij}(x) - \sum_{k} \partial_{x_k} b_k(x) + c(x) \right) u(t, x)\, dx = 0.$$

(b) Show that

$$\int_{\mathbb{R}^n} K(t,x,y)\,dx = 1, \quad \forall\, t > 0,\ y \in \mathbb{R}^n,$$

if and only if

$$Q(t) = Q_0, \qquad \forall\, t > 0,$$

for any smooth initial temperature φ.

Chapter 14

Inverse Problems for Classical PDEs

This chapter addresses the inverse task of recovering *initial* and *boundary* data from measurements of the solution for three model equations: the Laplace equation, the heat equation, and the wave equation. Unlike the forward problem, these inverse formulations are often ill-posed: small measurement noise can produce large reconstruction errors, uniqueness may fail without additional apriori information, and stability typically requires regularization.

14.1 Learning a Dirichlet Boundary Condition for the Laplace Equation

Let $\Omega \subset \mathbb{R}^2$ be a bounded domain with smooth enough boundary. Consider the Dirichlet problem for the Laplace equation

$$\begin{cases} \Delta u = 0 & \text{in } \Omega, \\ u|_{\partial\Omega} = \varphi & \text{on } \partial\Omega, \end{cases}$$

with $u \in C^2(\Omega) \cap C(\overline{\Omega})$ and $\varphi \in C(\partial\Omega)$.

We ask the following inverse problem:

Problem *Given N interior measurements $u_j = u(x_j)$ at points $x_j \in \Omega$, and M boundary measurements $\varphi_k = u(y_k)$ at boundary points $y_k \in \partial\Omega$, recover the unknown boundary trace φ on $\partial\Omega$.*

The boundary trace $\varphi = u|_{\partial\Omega}$ is recovered from an approximation of the harmonic solution u by a neural network $u_\theta : \overline{\Omega} \to \mathbb{R}$. We determine u_θ by minimizing a loss that enforces harmonicity in the interior and fidelity to the measurements:

$$\mathcal{L}_{\text{tot}}(\theta) \quad = \quad \mathcal{L}_{\text{PDE}}(\theta) + \lambda_{\text{int}}\, \mathcal{L}_{\text{int}}(\theta) + \lambda_{\text{bd}}\, \mathcal{L}_{\text{bd}}(\theta),$$

where, for interior collocation points $\{x_i^{\mathrm{col}}\}_{i=1}^{N_{\mathrm{col}}} \subset \Omega$, interior measurements $\{(x_j, u_j)\}_{j=1}^{N} \subset \Omega \times \mathbb{R}$, and boundary samples $\{(y_k, \varphi_k)\}_{k=1}^{M} \subset \partial\Omega \times \mathbb{R}$,

$$\begin{aligned}
\mathcal{L}_{\mathrm{PDE}}(\theta) &= \frac{1}{N_{\mathrm{col}}} \sum_{i=1}^{N_{\mathrm{col}}} \big(\Delta u_\theta(x_i^{\mathrm{col}})\big)^2, \\
\mathcal{L}_{\mathrm{int}}(\theta) &= \frac{1}{N} \sum_{j=1}^{N} \big(u_\theta(x_j) - u_j\big)^2, \\
\mathcal{L}_{\mathrm{bd}}(\theta) &= \frac{1}{M} \sum_{k=1}^{M} \big(u_\theta(y_k) - \varphi_k\big)^2.
\end{aligned}$$

The recovered Dirichlet trace is then $\varphi_\theta := u_\theta|_{\partial\Omega}$.

Disk case and Poisson integral If $\Omega = \{x \in \mathbb{R}^2 : \|x\| < R\}$, the solution with Dirichlet boundary data φ is given by the Poisson formula

$$u(x) = \frac{1}{2\pi R} \int_{\|y\|=R} \frac{R^2 - \|x\|^2}{\|x - y\|^2}\, \varphi(y)\, d\sigma_y, \qquad \|x\| < R.$$

In this setting one can *learn the boundary trace directly.* Parameterize φ by a boundary network $\varphi_\theta : \partial\Omega \to \mathbb{R}$ (e.g., via angle $y = R(\cos\phi, \sin\phi)$), and define the *Poisson loss* that enforces the integral representation against interior measurements $\{(x_k, u_k)\}_{k=1}^{N}$:

$$\begin{aligned}
\mathcal{L}_{\mathrm{Pois}}(\theta) &= \frac{1}{N} \sum_{k=1}^{N} \left(u_k - \frac{1}{2\pi R} \int_{\|y\|=R} \frac{R^2 - \|x_k\|^2}{\|x_k - y\|^2}\, \varphi_\theta(y)\, d\sigma_y \right)^2 \\
&= \frac{1}{N} \sum_{k=1}^{N} \left(u_k - \frac{R^2 - \|x_k\|^2}{m} \sum_{r=0}^{m-1} \frac{\varphi_\theta(R\omega_m^r)}{\|x_k - R\omega_m^r\|^2} \right)^2,
\end{aligned}$$

where $\omega_m^r = (\cos\frac{2\pi r}{m}, \sin\frac{2\pi r}{m})$ are the vertices of a regular m-polygon inscribed in the circle of radius R along which the integral is evaluated.

If boundary measurements $\{(y_k, \varphi_k)\}$ are available, we add a boundary fit term

$$\mathcal{L}_{\mathrm{bd}}(\theta) = \frac{1}{M} \sum_{k=1}^{M} \big(\varphi_\theta(y_k) - \varphi_k\big)^2,$$

and minimize

$$\mathcal{L}_{\mathrm{tot}}(\theta) = \mathcal{L}_{\mathrm{Pois}} + \lambda_{\mathrm{bd}} \mathcal{L}_{\mathrm{bd}}.$$

This *Poisson-layer* approach enforces harmonicity and the Dirichlet map exactly, focusing learning on the unknown boundary trace.

Remark 14.1.1 The one-dimensional case is somehow different. For $\Omega = (0, 1)$ the Laplace equation reduces to $u'' = 0$, with the solution given by an affine function $u(x) = ax + b$. Two interior measurements, u_1, u_2 already determine the parameters pair (a, b) uniquely; an overdetermined set ($N > 2$) is naturally handled by the least-squares fit. The network reduces to a linear model in this case.

14.2 Learning the Initial Condition for the Heat Equation

Consider an infinite rod with unknown initial temperature distribution

$$u(0,x) = \varphi(x), \qquad x \in \mathbb{R}.$$

In the absence of sources, the temperature evolves according to the heat equation

$$\partial_t u(t,x) - \partial_x^2 u(t,x) = 0, \qquad t > 0,\ x \in \mathbb{R}.$$

We consider the following inverse problem:

Problem *Given measurements $u_{ij} = u(t_i, x_j)$ at a set of times $\{t_i\}_{i=1}^N$ and locations $\{x_j\}_{j=1}^M$, recover the initial temperature $\varphi(x)$.*

Variant 1: Learn u directly and read off $\varphi(x) = u(0,x)$. Train a neural network $u_\theta(t,x)$ to represent the temperature field and enforce both the PDE and the data:

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{PDE}}(\theta) + \lambda_{\text{data}}\, \mathcal{L}_{\text{data}}(\theta),$$

with

$$\begin{aligned} \mathcal{L}_{\text{PDE}}(\theta) &= \frac{1}{N_{\text{col}} M_{\text{col}}} \sum_{i=1}^{N_{\text{col}}} \sum_{j=1}^{M_{\text{col}}} \Big(\partial_t u_\theta(\tilde{t}_i, \tilde{x}_j) - \partial_x^2 u_\theta(\tilde{t}_i, \tilde{x}_j) \Big)^2, \\ \mathcal{L}_{\text{data}}(\theta) &= \frac{1}{NM} \sum_{i=1}^{N} \sum_{j=1}^{M} \big(u_\theta(t_i, x_j) - u_{ij} \big)^2, \end{aligned}$$

where $\{(\tilde{t}_i, \tilde{x}_j)\}$ are interior collocation points and $\lambda_{\text{data}} > 0$ balances Physics and data. After training, we estimate the initial profile as $\varphi(x) = u_\theta(0,x)$.

Remark 14.2.1 (i) Because the heat flow is strongly smoothing, the inverse problem is ill-posed. In particular, minimizing only the physics-loss $\mathcal{L}_{\text{PDE}}(\theta)$ is insufficient; the data misfit $\mathcal{L}_{\text{data}}(\theta)$ plays a crucial stabilizing role (often together with regularization).
(ii) In practice we work on a truncated spatial domain $[-L, L]$ with $L > \max_j |x_j|$ and impose a *homogeneous boundary penalty* to suppress edge artifacts. A convenient choice is

$$\mathcal{L}_{\text{bd}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \Big(|u_\theta(t_i, -L)|^2 + |u_\theta(t_i, L)|^2 \Big),$$

which encourages u_θ to remain small at the artificial boundaries for all sampled times.

Variant 2: Learn φ via the heat kernel. For $t > 0$ the solution is the Gaussian convolution of the initial temperature:

$$u(t,x) = \int_{\mathbb{R}} G(t, x-y)\, \varphi(y)\, dy, \qquad G(t,\xi) = \frac{1}{\sqrt{4\pi t}} \exp\!\left(-\frac{\xi^2}{4t} \right).$$

Parameterize φ by a 1D network $\varphi_\theta : \mathbb{R} \to \mathbb{R}$ and fit it to the data through the loss

$$\mathcal{L}(\theta) = \frac{1}{NM} \sum_{i=1}^{N} \sum_{j=1}^{M} \left(u_{ij} - \int_{\mathbb{R}} G(t_i, x_j - y)\, \varphi_\theta(y)\, dy \right)^2 + \lambda \, \|\varphi_\theta\|_{L^2}^2.$$

The role of the L^2-regularizer is to stabilize the inversion against noise and the intrinsic ill-posedness of backward heat problem.

The integral on $\mathbb{R}$ is approximated by a finite grid $\{y_k\}$ covering the effective support of φ and of $G(t_i, \cdot)$ as

$$\int_{\mathbb{R}} G(t_i, x_j - y)\, \varphi_\theta(y)\, dy \;\approx\; \sum_{k} G(t_i, x_j - y_k)\, \varphi_\theta(y_k)\, \Delta y.$$

Since G decays exponentially, it suffices to choose a window $|y| \le L$ with $L \sim 3\sqrt{t_{\max}}$.

Both variants can be combined in a hybrid loss (Physics + kernel forward model + data), with φ_θ tied to u_θ through the constraint $\varphi_\theta(x) = u_\theta(0, x)$.

14.3 Learning the Initial Conditions for Wave Equation

The general dimension case. The Cauchy problem for the wave equation in $\mathbb{R}^n$ is

$$\begin{cases} \partial_t^2 u(t,x) - \Delta u(t,x) = 0, & (t,x) \in (0,\infty) \times \mathbb{R}^n, \\ u(0,x) = \varphi(x), & \\ \partial_t u(0,x) = \psi(x), & x \in \mathbb{R}^n. \end{cases}$$

The inverse task is to approximate the initial conditions φ and ψ from given measurements $u_{ij} = u(t_i, x_j)$, $1 \le i \le N$, $1 \le j \le M$. A natural strategy is to learn u with a neural network and then read off the initial traces.

Let $u_\theta(t,x)$ be a neural network trained to minimize the composite loss

$$\begin{aligned} \mathcal{L}(\theta) \;=\; & \underbrace{\frac{1}{N_{\mathrm{col}} M_{\mathrm{col}}} \sum_{i,j} \big(\partial_t^2 u_\theta(\tilde{t}_i, \tilde{x}_j) - \Delta u_\theta(\tilde{t}_i, \tilde{x}_j) \big)^2}_{\mathcal{L}_{\mathrm{PDE}}(\theta)} \\ & + \lambda \underbrace{\frac{1}{NM} \sum_{i,j} \big(u_\theta(t_i, x_j) - u_{ij} \big)^2}_{\mathcal{L}_{\mathrm{data}}(\theta)} + \lambda_{\mathrm{reg}} \|\theta\|^2, \end{aligned}$$

where $\{(\tilde{t}_i, \tilde{x}_j)\}$ are interior collocation points for enforcing the PDE, while λ and λ_{reg} are positive weights that fit data and regularization.

With $\theta^* = \arg\min_\theta \mathcal{L}(\theta)$, the recovered initial conditions are

$$\widehat{\varphi}(x) = u_{\theta^*}(0, x), \qquad \widehat{\psi}(x) = \partial_t u_{\theta^*}(0, x).$$

The regularization term $\lambda_{\text{reg}}\|\theta\|^2$ helps stabilize the inversion against measurement noise and the mild ill-posedness of the backward reconstruction.

If $n \in \{1, 2, 3\}$, closed-form representation formulas are available for the Cauchy problem of the wave equation. In these cases we can *learn the initial data directly* and enforce the PDE *exactly* by construction via the corresponding representation.

The case $n = 1$ (d'Alembert's formula) For $\partial_t^2 u - \partial_x^2 u = 0$ on $\mathbb{R}$, the solution is

$$u(t,x) = \tfrac{1}{2}\big(\varphi(x+t) + \varphi(x-t)\big) \;+\; \tfrac{1}{2}\int_{x-t}^{x+t} \psi(s)\, ds.$$

We parameterize the unknown initial data by two 1D networks $\varphi_\theta : \mathbb{R} \to \mathbb{R}$ and $\psi_\theta : \mathbb{R} \to \mathbb{R}$. Given samples $\{(t_i, x_j, u_{ij})\}$, we fit θ by minimizing

$$\mathcal{L}(\theta) = \frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M}\left[\tfrac{1}{2}\big(\varphi_\theta(x_j+t_i) + \varphi_\theta(x_j-t_i)\big) + \tfrac{1}{2}\int_{x_j-t_i}^{x_j+t_i}\psi_\theta(s)\, ds - u_{ij}\right]^2 + \lambda\,\|\theta\|_{L^2}^2.$$

In practice, we approximate the integral by trapezoidal quadrature on the interval $[x_j - t_i, x_j + t_i]$. This "d'Alembert layer" guarantees that the learned field satisfies the wave equation and that $u(0,x) = \varphi_\theta(x)$, $\partial_t u(0,x) = \psi_\theta(x)$ by construction.

The case $n = 2$ (Poisson's formula) For $\partial_t^2 u - \Delta u = 0$ in $\mathbb{R}^2$, the representation is

$$u(t,x) = \frac{\partial}{\partial t}\left(\frac{1}{2\pi}\int_{|y-x|<t} \frac{\varphi(y)}{\sqrt{t^2 - |y-x|^2}}\, dy\right) + \frac{1}{2\pi}\int_{|y-x|<t}\frac{\psi(y)}{\sqrt{t^2-|y-x|^2}}\, dy.$$

We parameterize $\varphi_\theta, \psi_\theta : \mathbb{R}^2 \to \mathbb{R}$ and define the data loss

$$\mathcal{L}(\theta) = \frac{1}{NM}\sum_{i,j}\left[\partial_t\left(\frac{1}{2\pi}\int_{|y-x_j|<t}\frac{\varphi_\theta(y)}{\sqrt{t^2-|y-x_j|^2}}\, dy\right)_{|t=t_i} + \frac{1}{2\pi}\int_{|y-x_j|<t_i}\frac{\psi_\theta(y)}{\sqrt{t_i^2-|y-x_j|^2}}\, dy - u_{ij}\right]^2 + \lambda\,\|\theta\|^2.$$

Both integrals are over a disk and can be computed by polar quadrature centered at x_j. In the first integral the t-derivative is taken first and then the value $t = t_i$.

The case $n = 3$ (Kirchhoff's formula) For $\partial_t^2 u - \Delta u = 0$ in $\mathbb{R}^3$, Kirchhoff's formula gives

$$u(t,x) = \frac{\partial}{\partial t}\left(\frac{1}{4\pi t}\int_{|y-x|=t}\varphi(y)\, d\sigma_y\right) + \frac{1}{4\pi t}\int_{|y-x|=t}\psi(y)\, d\sigma_y.$$

Learning proceeds by parameterizing $\varphi_\theta, \psi_\theta : \mathbb{R}^3 \to \mathbb{R}$ and matching the spherical means to the data via a least-squares loss

$$\mathcal{L}(\theta) = \frac{1}{NM} \sum_{i,j} \Big(\partial_t S(\varphi; t_i, x_j) + S(\psi; t_i, x_j) - u_{ij} \Big)^2 + \lambda \, \|\theta\|^2,$$

where we denoted by

$$S(f; t, x) = \frac{1}{4\pi t} \int_{|y-x|=t} f(y) \, d\sigma_y$$

the spherical mean of f on a sphere centered at x of radius t. This sphere integral can be evaluated as

$$\begin{aligned} S(f; t, x) &= \frac{1}{4\pi t} \int_{|y-x|=t} f(y) \, d\sigma_y \approx \frac{1}{4\pi t} \frac{4\pi t^2}{m} \sum_{k=1}^{m} f(y_k) \\ &= \frac{t}{m} \sum_{k=1}^{m} f(x + t n_k) = \frac{r}{m} \sum_{k=1}^{m} f\Big(x + t \frac{g_k}{\|g_k\|} \Big), \end{aligned}$$

with $g_k \sim N(0, I_3)$ normally distributed. The vectors n_k are uniformly distributed on the unit sphere $\mathbb{S}^2$.

The derivative $\partial_t S$ is computed classically using product rule and a change of variables as follows

$$\begin{aligned} \partial_t S(\varphi; t, x) &= \frac{\partial}{\partial t} \left(\frac{1}{4\pi t} \int_{|y-x|=t} \varphi(y) \, d\sigma_y \right) = \frac{\partial}{\partial t} \left(\frac{1}{4\pi t} \int_{\|n\|=1} \varphi(x + tn) \, t^2 d\omega \right) \\ &= \frac{\partial}{\partial t} \left(\frac{t}{4\pi} \int_{\mathbb{S}^2} \varphi(x + tn) \, d\omega \right) \\ &= \frac{1}{4\pi} \int_{\mathbb{S}^2} \varphi(x + tn) \, d\omega + \frac{t}{4\pi} \int_{\mathbb{S}^2} \underbrace{\nabla \varphi(x + tn) \cdot n}_{= \partial_n \varphi(x + tn)} \, d\omega. \end{aligned}$$

14.4 Exercises

Exercise 14.4.1 (Recovering Dirichlet). We consider $B := B(0, 1) \subset \mathbb{R}^2$ and $\partial B = \mathbb{S}^1$.

(a) Synthetic data generation: Consider the interior Dirichlet problem

$$\Delta u = 0 \ \text{ in } B, \qquad u|_{\partial B} = \varphi, \quad \varphi(y) = y_1 y_2.$$

Generate interior samples $u_i = u(x_i)$ at points $x_i \in B$ by evaluating the Poisson integral

$$u(x) = \frac{1}{2\pi} \int_{|y|=1} \frac{1 - |x|^2}{|x - y|^2} \, \varphi(y) \, ds_y, \qquad |x| < 1.$$

Hint: Parametrize the circle by $y(\phi) = (\cos\phi, \sin\phi)$, $ds = d\phi$, and use the periodic trapezoidal rule:

$$u(x) \approx \frac{1}{2\pi} \sum_{k=0}^{N_\phi - 1} \frac{1 - |x|^2}{|x - y(\phi_k)|^2} \, \varphi\big(y(\phi_k)\big) \, \Delta\phi, \quad \phi_k = \tfrac{2\pi k}{N_\phi}.$$

(b) Inverse reconstruction of the boundary condition: Given noisy (or noise-free) interior data $\{(x_i, u_i)\}_{i=1}^N$, recover the boundary trace φ on $\mathbb{S}^1$ by the following two approaches:

Method (i): Learn a harmonic field $u_\theta : \overline{B} \to \mathbb{R}$ by minimizing

$$\mathcal{L}_{\mathrm{PINN}}(\theta) = \underbrace{\frac{1}{N_{\mathrm{col}}} \sum_{m=1}^{N_{\mathrm{col}}} \big(\Delta u_\theta(\tilde{x}_m)\big)^2}_{\text{PDE (interior collocation)}} + \lambda_{\mathrm{data}} \underbrace{\frac{1}{N} \sum_{i=1}^{N} \big(u_\theta(x_i) - u_i\big)^2}_{\text{data fit}} + \lambda_{\mathrm{reg}}\, \mathcal{R}\big(u_\theta|_{S^1}\big),$$

where $\{\tilde{x}_m\} \subset B$ are interior collocation points and $\mathcal{R}$ is a boundary regularizer. Then set $\widehat{\varphi} = u_\theta|_{\mathbb{S}^1}$.

Method (ii): Parameterize the boundary trace by a 2π-periodic network $\varphi_\theta(\phi)$ with $y(\phi) = (\cos\phi, \sin\phi)$, and predict the interior field via the Poisson formula:

$$u_\theta(x) = \frac{1}{2\pi} \int_0^{2\pi} \frac{1 - |x|^2}{|x - y(\phi)|^2}\, \varphi_\theta(\phi)\, d\phi.$$

Fit θ by minimizing

$$\mathcal{L}_{\mathrm{Pois}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \big(u_\theta(x_i) - u_i\big)^2 + \lambda_{\mathrm{reg}}\, \mathcal{R}\big(\varphi_\theta\big),$$

with the integral evaluated by the trapezoidal rule in ϕ. Report the recovered boundary function as $\widehat{\varphi}(\phi) = \varphi_\theta(\phi)$.

(c) Plot the true $\varphi(\phi) = \cos\phi\, \sin\phi$ and the reconstructions from methods (i) and (ii) on $[0, 2\pi)$. Add a brief comparison discussing accuracy, and the effect of the regularization parameter λ_{reg}.

Exercise 14.4.2 (Dirichlet trace on the exterior of the unit disk). Let $D := \{x \in \mathbb{R}^2 : |x| > 1\}$ be the exterior of the unit disk and $\partial D = \mathbb{S}^1$.

(a) Synthetic data generation: Consider the exterior Dirichlet problem

$$\Delta u = 0 \ \text{ in } D, \qquad u|_{|x|=1} = \varphi(x), \qquad u \text{ bounded as } |x| \to \infty,$$

with boundary datum $\varphi(y) = y_1 + y_2$ on $\mathbb{S}^1$. Generate samples $u_i = u(x_i)$ at points $x_i \in D$ using the exterior Poisson representation

$$u(x) = \frac{1}{2\pi} \int_{|y|=1} \frac{1 - |x|^2}{|x - y|^2}\, \varphi(y)\, ds_y, \qquad |x| > 1.$$

(b) Inverse reconstruction of the boundary condition: Given measurements $\{(x_i, u_i)\}_{i=1}^N$ with $x_i \in D$, recover the boundary trace φ on $\mathbb{S}^1$ using two approaches similar with those in Exercise 14.4.1.

(c) Plot the true boundary function $\varphi(\phi) = \cos\phi + \sin\phi$ and the reconstructions from methods (i) and (ii) on $[0, 2\pi)$.

Exercise 14.4.3 (Recovering an impact from wave data). A very large, effectively unbounded lake is initially at rest: $u(0,x) = 0$ for $x \in \mathbb{R}^2$. At $t = 0^+$ a falling rock impacts the surface at some (unknown) point $x^* \in \mathbb{R}^2$, impacting an *initial velocity* $u_1(x) = \partial_t u(0,x)$. For small-amplitude gravity waves over nearly uniform depth, we model the surface elevation $u(t,x)$ by the linear wave equation

$$\partial_t^2 u - c^2 \Delta u = 0, \qquad (t,x) \in (0,\infty) \times \mathbb{R}^2,$$

with wave speed $c > 0$, and initial data $u(0,x) = 0$, $\partial_t u(0,x) = u_1(x)$.

You are given measurements $u_{ij} = u(t_i, x_j)$ at discrete times $\{t_i\}_{i=1}^N$ from a set of buoys at locations $\{x_j\}_{j=1}^M \subset \mathbb{R}^2$ and you will find the *impact location* x^* (center of the impulse), by training a neural network following the steps:

(a) *Forward model.* Work on a truncated disk $\{x : \|x\| \le R\}$ with R large enough to contain all buoys, and impose a first-order absorbing boundary condition (ABC)

$$\partial_r u + \tfrac{1}{c}\, \partial_t u = 0 \quad \text{at } r = R,$$

to reduce reflections.

(b) *Impact ansatz.* Assume a localized Gaussian impulse

$$u_1(x) \approx A \exp\Big(- \tfrac{\|x-p\|^2}{2\sigma^2} \Big),$$

and learn (A, p, σ), optionally allowing a small corrective network $\psi_\theta(x)$ so that $u_1(x) = Ae^{-\|x-p\|^2/(2\sigma^2)} + \psi_\theta(x)$.

(c) *PDE-consistent solution map.* Learn $u_\theta(t,x)$ directly and enforce the wave PDE at interior collocation points, the ABC at $r = R$, and the data fit at (t_i, x_j); then set $\widehat{u_1}(x) = \partial_t u_\theta(0,x)$ and estimate x^* as the maximizer of $\widehat{u_1}$.

(c) *Loss function.* Minimize the loss

$$\mathcal{L}(\theta) = \underbrace{\tfrac{1}{N_c}\sum (\partial_t^2 u_\theta - c^2 \Delta u_\theta)^2}_{\text{PDE}} + \lambda_{\text{bc}} \underbrace{\tfrac{1}{N_b}\sum (\partial_r u_\theta + \tfrac{1}{c}\partial_t u_\theta)^2_{r=R}}_{\text{ABC}} + \lambda_{\text{data}} \underbrace{\frac{1}{NM} \sum_{i,j} \big(u_\theta(t_i,x_j) - u_{ij}\big)^2}_{\text{data}} + \lambda_{\text{reg}}\, \mathcal{R},$$

with an appropriate regularizer $\mathcal{R}$.

(c) Provide plots of the recovered $\widehat{u_1}(x)$ and the inferred impact position $\widehat{x^*}$.

Exercise 14.4.4 (Recovering an initial heat profile in $\mathbb{R}^2$). Consider the heat equation in $\mathbb{R}^2$,

$$\partial_t u(t,x) - \Delta u(t,x) = 0, \qquad t > 0,\ x \in \mathbb{R}^2,$$

with initial condition $u(0,x) = \varphi(x)$. Let the initial profile be

$$\varphi(x) = x_1\, e^{-\|x\|^2}, \qquad x = (x_1, x_2) \in \mathbb{R}^2.$$

We recall the heat kernel in $\mathbb{R}^2$

$$G(t,\xi) = \frac{1}{4\pi t}\,\exp\!\big(-\tfrac{\|\xi\|^2}{4t}\big), \qquad t>0,\ \xi\in\mathbb{R}^2,$$

and that the solution is the Gaussian convolution

$$u(t,x) = \int_{\mathbb{R}^2} G(t,x-y)\,\varphi(y)\,dy.$$

(a) Choose a time set $\{t_i\}_{i=1}^{N}$ and spatial sensor locations $\{x_j\}_{j=1}^{M}\subset\mathbb{R}^2$. Generate synthetic measurements $u_{ij}=u(t_i,x_j)$ by numerically evaluating

$$u(t_i,x_j) \;=\; \int_{\mathbb{R}^2} G(t_i,x_j-y)\,\varphi(y)\,dy.$$

Hint: Truncate the integral to a box $[-L,L]^2$ with L large enough (e.g. $L\gtrsim 4\sqrt{t_{\max}}$).
(b) Using the synthetic data $\{(t_i,x_j,u_{ij})\}$, recover the initial condition φ in the following two ways:

(i) Learn $u_\theta(t,x)$ on a truncated box $[-L,L]^2$ by minimizing

$$\begin{aligned}\mathcal{L}_{\mathrm{PINN}}(\theta) =& \frac{1}{N_c}\sum_{(t,x)\in\mathcal{C}}\big(\partial_t u_\theta-\Delta u_\theta\big)^2+\lambda_{\mathrm{data}}\frac{1}{NM}\sum_{i,j}\big(u_\theta(t_i,x_j)-u_{ij}\big)^2\\ &+\lambda_{\mathrm{bd}}\frac{1}{N_t}\sum_{i}\Big(u_\theta(t_i,\cdot)\Big|_{\partial([-L,L]^2)}\Big)^2\end{aligned}$$

where $\mathcal{C}$ is a set of interior collocation points and we used the notation

$$\Big(u_\theta(t_i,\cdot)\Big|_{\partial([-L,L]^2)}\Big)^2=\frac{1}{N_b}\sum_{b=1}^{N_b}(u_\theta(t_i,x_b))^2,$$

with boundary points $\{x_b\}_{b=1}^{N_b}\subset\partial([-L,L]^2)$. Estimate the initial profile by $\widehat{\varphi}(x)=u_\theta(0,x)$.

(ii) Consider the network $\varphi_\theta:\mathbb{R}^2\to\mathbb{R}$ and fit θ by

$$\mathcal{L}_{\mathrm{conv}}(\theta)=\frac{1}{NM}\sum_{i,j}\left(u_{ij}-\int_{\mathbb{R}^2}G(t_i,x_j-y)\,\varphi_\theta(y)\,dy\right)^2+\lambda\,\|\varphi_\theta\|_{L^2}^2,$$

with the integral evaluated as in part (a).
(c) Plot the recovered profiles $\widehat{\varphi}$ from (i) and (ii) on a grid and compare them to the ground truth $\varphi(x)=x_1e^{-\|x\|^2}$.

Exercise 14.4.5 (Recovering an impact location on a clamped beam). Consider a vibrating rod of length L with clamped ends. The transverse displacement $u(t,x)$ satisfies

$$\partial_t^2u(t,x)+\partial_x^4u(t,x)=0, \qquad (t,x)\in(0,\infty)\times[0,L],$$

with clamped boundary conditions

$$u(t,0) = u(t,L) = 0, \qquad \partial_x u(t,0) = \partial_x u(t,L) = 0.$$

Assume the beam is initially at rest,

$$u(0,x) = 0, \qquad \partial_t u(0,x) = 0.$$

At a (known) time $t_0 > 0$, the beam is struck transversally at an *unknown* point $x^* \in (0,L)$. The instantaneous impact is modeled as an initial velocity impulse at $t = t_0^+$:

$$\partial_t u(t_0^+, x) \;=\; \delta(x - x^*).$$

You are given measurements $u_{ij} = u(t_i, x_j)$ for a set of times $\{t_i\}$ and sensor locations $\{x_j\} \subset (0,L)$. Using the measurements $\{(t_i, x_j, u_{ij})\}$, the goal is to identify the impact location x^* following the next steps:

(a) Replace the Dirac delta by a narrow Gaussian

$$\delta_\sigma(x - x^*) \;=\; \frac{1}{\sqrt{2\pi}\,\sigma} \exp\Big(- \frac{(x-x^*)^2}{2\sigma^2} \Big),$$

with small width $\sigma \ll L$. Thus,

$$\partial_t u(t_0^+, x) \approx \delta_\sigma(x - x^*).$$

Explain how σ trades off spatial resolution and numerical stability.

(b) Parameterize $u_\theta(t,x)$ (a neural network) and the impact location x^* as a trainable scalar. Enforce:

- PDE in $(0,T] \times (0,L)$ except at $t = t_0$:

$$(\partial_t^2 u_\theta + \partial_x^4 u_\theta)^2 \quad \text{at interior collocation points;}$$

- Clamped BCs via penalties at $x = 0, L$ for all sampled t:

$$u_\theta(t,0),\; u_\theta(t,L),\; \partial_x u_\theta(t,0),\; \partial_x u_\theta(t,L);$$

- Pre- and post-impact conditions near t_0: for a small $\varepsilon > 0$,

$$u_\theta(t_0 - \varepsilon, x) \approx u_\theta(t_0 + \varepsilon, x), \qquad \partial_t u_\theta(t_0 + \varepsilon, x) - \partial_t u_\theta(t_0 - \varepsilon, x) \approx \delta_\sigma(x - x^\star);$$

- Data fit at sensor points: $u_\theta(t_i, x_j) \approx u_{ij}$.

Write the total loss as a weighted sum of these terms and train jointly over θ and x^*.

(c) Solve the problem in the following approach: Use the clamped eigenmodes $\{\phi_k\}$ of ∂_x^4 on $[0,L]$ and represent

$$u(t,x) = \sum_{k \geq 1} \big(A_k \cos(\omega_k (t - t_0)) + B_k \sin(\omega_k (t - t_0)) \big)\, \phi_k(x),$$

with ω_k the natural frequencies. Impose continuity of u at t_0 and the jump in $\partial_t u$ induced by $\delta_\sigma(x - x^*)$ to determine (A_k, B_k) as functions of x^* (or treat a truncated set of coefficients as trainable). Fit x^* to the data via least squares.

(d) Compare the results from parts (a) and (b).

Chapter 15

Special Structures

This chapter explores Deep Learning architectures that go beyond simple feed-forward networks by incorporating structure, dynamics, and attention mechanisms. We introduce models designed to capture spatial, temporal, and physical structure, such as attention-based architectures for sequential and relational data, and neural ordinary differential equation (Neural ODE) models for continuous-time dynamics. Within this latter class, we discuss specialized formulations such as Hamiltonian Neural Networks (HNNs) and Lagrangian Neural Networks (LNNs), which embed physical principles like energy conservation and variational structure directly into the learning process.

Attention layers are examined as mechanisms for adaptive focus, enabling networks to dynamically prioritize information flow across space and time. Throughout the chapter, we emphasize that these architectures outperform plain multilayer perceptrons.

15.1 Residual and Neural ODEs

We consider an isolated physical system, whose state at time t is described by a vector $x_t \in \mathbb{R}^d$, where d is the number of state variables. Assuming the dynamic is Markovian[1] and time-homogeneous, we have

$$x_{t+1} = g(x_t), \qquad t = 0, 1, 2, \ldots,$$

for some unknown transition map $g : \mathbb{R}^d \to \mathbb{R}^d$. We have the following discrete-time learning problem:

Problem *Given a trajectory* $\{x_t\}_{t=0}^N$*, learn the transition map* g*.*

We can approach this by fitting a neural network $g_\theta : \mathbb{R}^d \to \mathbb{R}^d$ that minimizes

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{t=0}^{N-1} \left\| x_{t+1} - g_\theta(x_t) \right\|^2.$$

After training, the one-step forecast is $x_{N+1} \approx g_{\theta^*}(x_N)$.

[1] This is a system where the next state depends only on the present state of the system.

It is often convenient to define the increment map

$$f(x) := g(x) - x,$$

so that

$$x_{t+1} = x_t + f(x_t), \qquad \frac{x_{t+1} - x_t}{\Delta t} = f(x_t) \text{ with } \Delta t = 1.$$

Formally passing to a small sampling step $\Delta t \to 0$ suggests the following continuous-time model.

Continuous-time model Let $x(t)$ denote the state at continuous time t. In the limit,

$$x'(t) = f\big(x(t)\big),$$

i.e., $f : \mathbb{R}^d \to \mathbb{R}^d$ is a vector field generating the dynamics. In this case we have the following continuous-time learning problem:

Problem *Given measurements $\{(x_i, x_i')\}_{i=1}^N$ of positions and velocities at times t_i, learn f so that the ODE $x' = f(x)$ matches the data:*

$$x(t_i) \approx x_i, \qquad x'(t_i) \approx x_i'.$$

To approximate f, we fit a neural network $f_\theta : \mathbb{R}^d \to \mathbb{R}^d$ by minimizing

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \big\| x_i' - f_\theta(x_i) \big\|^2.$$

The aforementioned ODE has unique solution in the case of a Lipschitz function f. This is why we shall add a soft constraint to require the network to be Lipschitz. To this end, we recall the Jacobian matrix $J_{f_\theta}(x) \in \mathbb{R}^{d \times d}$ for $x \in \mathbb{R}^d$, given by

$$J_{ij} = [J_{f_\theta}(x)]_{ij} = \frac{\partial (f_\theta^i(x)}{\partial x_j}.$$

Then we use the Frobenius norm

$$\|J\|_F = \sum_{i,j} J_{ij}^2,$$

which measures the total derivative energy. The soft-constraint term is

$$\mathcal{L}_{JF}(\theta) = \frac{1}{M} \sum_{k=1}^{M} \|J_{f_\theta}\|_F^2,$$

which encourages small first derivative, namely a more Lipschitz-like f_θ. Consequently, the total loss becomes

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \lambda_{\text{Lip}} \mathcal{L}_{JF}(\theta).$$

Remark 15.1.1 If the dimension d is large, a more suitable way to compute the Frobenius norm is to use an average of the product between the Jacobian matrix and a Gaussian random variable as

$$\|J\|_F^2 = \mathrm{Tr}(J^T J) = \mathbb{E}_{\xi \sim N(0,\mathbb{I}_d)}[\|J\xi\|_2^2].$$

Remark 15.1.2 The previous approach can be easily extended to the nonautonomous case: Given data $(x_i, x_i')_{i=1}^N$ find the map $f : \mathbb{R} \times \mathbb{R}^d \to \mathbb{R}^d$ such that the equation $x'(t) = f(t, x(t))$ has a solution matching the data $x(t_i) = x_i$ and $x'(t_i) = x_i'$. The loss to be minimized in this case is

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}(x_i' - f_\theta(t_i, x_i))^2 + \lambda_{\mathrm{Lip}}\mathcal{L}_{JF}(\theta).$$

Learning a Netwon-type equation Similarly, we can learn a model for a second order ODE. The problem is:

Problem *Given the measurements of positions, velocities and accelerations at time instances* t_i

$$(x_i, x_i', x_i'')_{i=1}^N$$

find a function f *such that a solution* $x(t)$ *of the second order ODE*

$$x''(t) = f(x, x')$$

satisfies $x_i = x(t_i)$, $x_i' = x'(t_i)$, $x_i'' = x''(t_i)$.

The model function f is approximated by the neural network $f_\theta : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}^d$ with the loss

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}(x_i'' - f_\theta(x_i, x_i'))^2.$$

Implementation example *Data generation.* We consider $x(t) = \frac{1}{1+e^{-t}}$, which is a solution of the ODE $x'(t) = x(t)(1 - x(t))$. We consider ten equally spaced points $t_i \in [0, 2]$ and produce synthetic data as in Table 15.1.

Training. We used an architecture with two hidden layers of 32 `tanh`-activated neurons, trained for 2,000 epochs with Adam optimizer with a learning rate 0.001. The neural network solution and the true solution $f(x) = x(1 - x)$ match pretty decently, see Fig. 15.1.

The code can be found at: `https://tinyurl.com/4nrujf2j`.

Conclusions The approach in this section was to learn naively the model f from data without any Physics involved. We shall approach this problem in the next sections using Hamiltonian neural nets for solving the first order systems and Lagrangian neural nets for the second order ODEs, the advantage being that the function f is related either to a Hamiltonian or a Lagrangian, which makes the model physically plausible.

Here we learned the vector field f directly from data, with no Physics built in. Next, we'll impose a physical structure by using Hamiltonian neural networks for first-order

t_i	$x(t_i)$	$x'(t_i)$
0.000000	0.500000	0.250000
0.222222	0.555328	0.246939
0.444444	0.609318	0.238050
0.666667	0.660756	0.224157
0.888889	0.708661	0.206461
1.111111	0.752336	0.186326
1.333333	0.791391	0.165091
1.555556	0.825715	0.143910
1.777778	0.855422	0.123675
2.000000	0.880797	0.104994

Table 15.1: Samples of $x(t) = \frac{1}{1+e^{-t}}$ and its derivative $x'(t) = x(1-x)$ for ten equally spaced $t_i \in [0, 2]$.

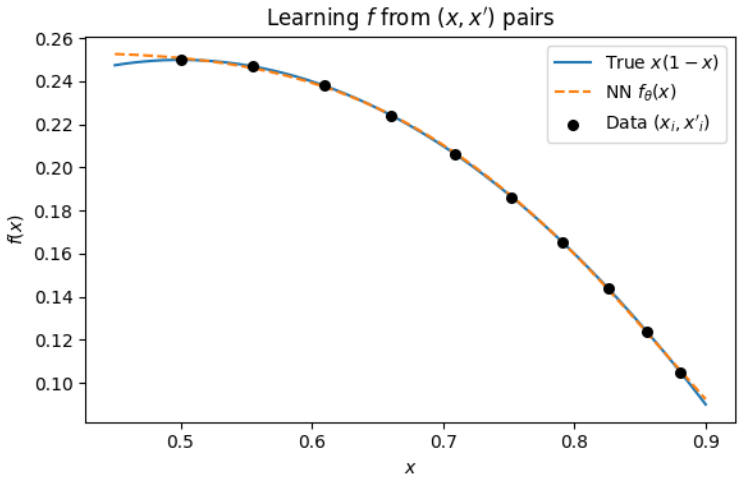


Figure 15.1: The NN-model $f_\theta(x)$ and the exact model $f(x) = x(1-x)$ match decently.

systems and Lagrangian neural networks for second-order ODEs. In these formulations, f is derived from a Hamiltonian or a Lagrangian, yielding models that respect conservation laws and are therefore more physically plausible.

Further Reading on Neural ODEs For readers interested in the broader context of Neural Ordinary Differential Equations, the seminal work by Chen et al. [23] introduced the continuous-depth paradigm, replacing discrete layers with differential equations and linking Deep Learning to dynamical systems. Building upon this foundation, several extensions have incorporated principles from Physics and Geometry. For instance, Greydanus et al. [42] proposed Hamiltonian Neural Networks, which learn energy-conserving dynamics from data by embedding Hamilton's equations into the network structure. Similarly, Sanchez-Gonzalez [83] developed Hamiltonian Graph

Networks that combine graph representations with ODE integrators to capture multi-body interactions, while Toth et al. [93] presented Hamiltonian Generative Networks, employing ODE-based solvers within generative modeling frameworks. Together, these studies demonstrate how the Neural ODE framework unifies learning, dynamics, and geometry, providing a continuous-time perspective that connects modern Deep Learning with classical differential equations and physical laws.

In the next sections, we shall see how these ODE-based networks take different forms, such as Hamiltonian Neural Networks (HNNs) or Lagrangian Neural Networks (LNNs), depending on the physical or geometric constraints they encode.

15.2 Hamiltonian Neural Networks (HNN)

Despite excelling at pattern recognition, plain neural networks often ignore the structure of physical laws. To address this, we introduce models grounded in Hamiltonian mechanics that embed conservation principles directly into the learning process– without requiring supervision from labeled energies. We begin by setting up a baseline network and then introduce *Hamiltonian Neural Networks (HNNs)*, an architecture that incorporates energy conservation into the model itself, yielding more accurate and efficient learning from data. The exposition follows Greydanus et al. [42], whose key idea is to parameterize the Hamiltonian with a neural network and learn it directly from data, rather than hand-crafting it.

Problem Setup We denote by q the position and by p the momentum of a dynamical system, so that the pair (q, p) describes its state. We assume that N observations of the system's state are collected at time instances t_i, yielding the data (q_i, p_i). These serve as inputs to a neural network. At the same time instances t_i, we also observe the derivatives of position and momentum, obtaining (q_i', p_i'), which are used as target data (labels) during supervised training.

A baseline neural network $NN(\theta; q, p)$ takes the inputs (q_i, p_i) and produces the outputs

$$NN(\theta; q, p) = (q'_{\text{pred}}, p'_{\text{pred}}),$$

see Fig. 15.2.

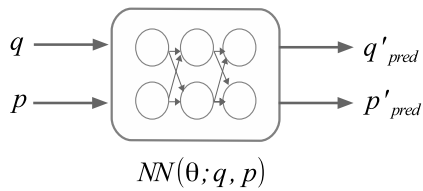


Figure 15.2: The schematic of a baseline neural network, which predicts the direction in the phase space.

Here, (q, p) is a point in the phase space M, while (q', p') represents a tangent vector in $T_{(q,p)}M$. The training objective is to minimize a loss function that aligns the predicted tangent vectors $(q'_{\text{pred}}, p'_{\text{pred}})$ with the ground-truth labels (q', p'). In other words, the baseline neural network is trained to *learn the direction field of the phase-space flow at each point* (q, p).

The associated loss for the baseline network is the mean squared error

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left[|q'_{\text{pred},i} - q'_i|^2 + |p'_{\text{pred},i} - p'_i|^2 \right].$$

One issue with this approach is the presence of noise in the data: measurement errors in the observed states (q, p) propagate to the derivatives, leading to noisy network outputs. The idea is therefore to introduce a filtering mechanism that refines the data by finding the closest noiseless trajectory consistent with an underlying Hamiltonian structure.

Hamiltonian Neural Networks The core idea of HNN is to learn a parametric Hamiltonian function $H_\theta(q, p)$ that captures the dynamics of the system, see Fig. 15.3. The parameters θ are tuned such that the Hamilton system of equations is satisfied

$$q' = \frac{\partial H}{\partial p}, \quad p' = -\frac{\partial H}{\partial q}.$$

The new loss function compares the predicted derivatives

$$q'_{\text{pred}} = \frac{\partial H_\theta}{\partial p}, \quad p'_{\text{pred}} = -\frac{\partial H_\theta}{\partial q}$$

to the true derivatives (q', p') as

$$\begin{aligned} \mathcal{L}(\theta) &= \frac{1}{N} \sum_{i=1}^{N} \left[|q'_{\text{pred},i} - q'_i|^2 + |p'_{\text{pred},i} - p'_i|^2 \right] \\ &= \frac{1}{N} \sum_{i=1}^{N} \left[\left| \frac{\partial H_\theta}{\partial p}(q_i, p_i) - q'_i \right|^2 + \left| \frac{\partial H_\theta}{\partial q}(q_i, p_i) + p'_i \right|^2 \right]. \end{aligned}$$

The expressions $\frac{\partial H_\theta}{\partial p}$ and $\frac{\partial H_\theta}{\partial q}$ are computed using automatic differentiation. Minimizing the loss we obtain the learned Hamiltonian function H_{θ^*}. This comes with a conservation law of energy in the sense that

$$H_{\theta^*}(q(t), p(t)) = \text{constant},$$

fact that is implied from the Hamiltonian system of equations, which is enforced by the loss function.

The Euclidean gradient operator in the phase space is $\nabla = (\partial_q, \partial_p)^T$. The symplectic gradient is given by $\nabla^J = (\partial_p, -\partial_q)^T$, where ^{T} denotes the transpose vector. We note the composition relation

$$\nabla^T = J \circ \nabla,$$

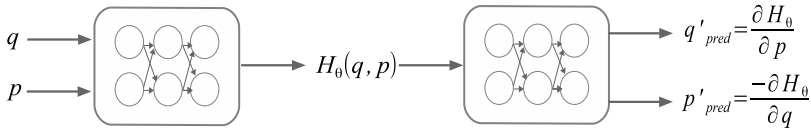


Figure 15.3: The schematic of a HNN.

where $J = \begin{pmatrix} 0 & \mathbb{I} \\ -\mathbb{I} & 0 \end{pmatrix}$ is the symplectic structure of the phase space. With this notations the loss can be written in the equivalent way

$$\mathcal{L}(\theta) = \|(q', p') - \nabla^J H_\theta(q,p)\|_2^2.$$

From this formulation, two key observations become evident:

1. The loss function adjusts the parameters θ so as to align the velocity field in phase space, (q', p'), with the Hamiltonian vector field $\nabla^J H_\theta(q,p)$.

2. The vector field $\nabla^J H$ is tangent to the surfaces of constant energy,

$$S_E = \{(q,p) \mid H(q,p) = E\}.$$

   This follows from the fact that $\nabla^J H$ is orthogonal to ∇H, while ∇H itself is normal to the energy surfaces S_E.

Natural Hamiltonian Functions When it is physically known that the Hamiltonian is *natural*,

$$H(q,p) = \frac{1}{2}p^2 + U(q),$$

it suffices to learn only the potential function using a neural network $U_\theta(q)$. In this case, the loss function becomes

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} |p_i - q_i'|^2 + \frac{1}{N}\sum_{i=1}^{N} |U_\theta'(q_i) + p_i'|^2,$$

where the first summation term is independent of the parameter θ. Consequently, the optimization reduces to minimizing the *reduced loss*

$$\mathcal{L}_{\text{red}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} |U_\theta'(q_i) + p_i'|^2.$$

Implementation Example: The Harmonic Oscillator Many physical systems, such as a spring-mass system, can be modeled by a quadratic Hamiltonian function of the form

$$H(q,p) = \frac{1}{2}p^2 + \frac{1}{2}q^2.$$

Data collection. We consider the corresponding Hamiltonian system

$$q' = p, \quad p' = -q,$$

and solve it for 25 pairs of initial conditions (q_0, p_0) using the analytical solutions

$$q(t) = q_0 \cos t + p_0 \sin t, \qquad p(t) = p_0 \cos t - q_0 \sin t.$$

The energy along each trajectory is given by $H_0 = \frac{1}{2}(q_0^2 + p_0^2)$, with $H_0 \in [0.2, 1]$. For each trajectory, we uniformly sample 30 points $(q(t), p(t))$ and compute the corresponding phase-space derivatives $(q'(t), p'(t))$. To simulate measurement uncertainty, Gaussian noise with variance $\sigma^2 = 0.1$ is added to the data. These noisy pairs constitute the training dataset.

Training. We train two neural network models: (i) A baseline network trained in a purely supervised manner using the previously generated data, and (ii) A Hamiltonian Neural Network (HNN) designed to learn the potential function $U(q)$.

Both models use the Adam optimizer with a learning rate of 10^{-3}. Each network consists of three fully connected layers with 200 hidden units per layer and tanh activations. Training is performed for $2,000$ gradient steps.

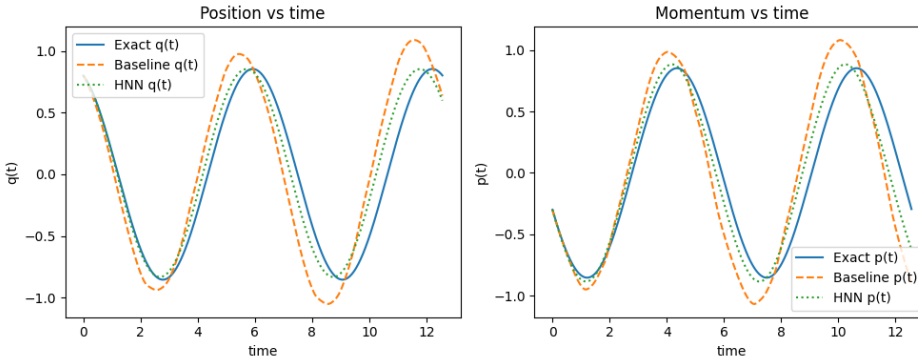
(i) *Baseline model.* The baseline network is trained directly, without incorporating any physical prior, to learn the mapping from system states (q, p) to the corresponding phase-space velocities (q', p').
(ii) *Hamiltonian Neural Network.* We consider a neural network $NN(\theta, q) = U_\theta(q)$ that approximates the potential function, and train it by minimizing the reduced loss

$$\mathcal{L}_{\text{red}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} |U'_\theta(q_i) + p'_i|^2.$$

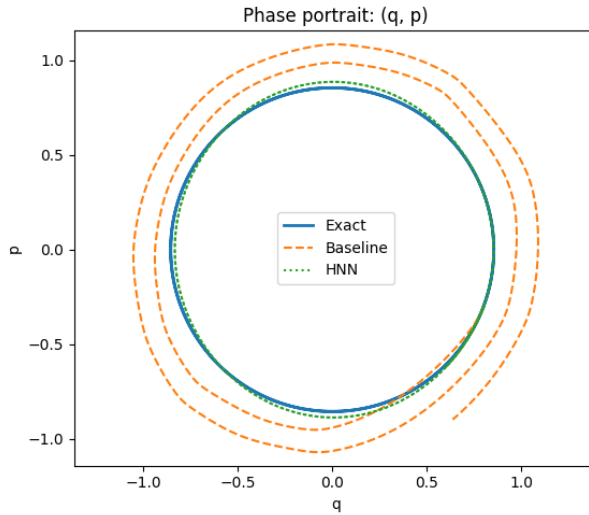
Results. We compare the two networks and validate them against the exact solution for a fixed initial condition (q_0, p_0) in phase space. In a separate plot, we show the energy curves as a function of training epochs for the ground-truth system, the baseline network, and the HNN, and highlight their differences. Overall, the HNN solution approximates the ground-truth dynamics substantially better than the baseline model.

Figure 15.4(a) shows the evolution of the position component $q(t)$ for all three cases (ground truth, baseline, HNN), while Fig. 15.4(b) shows the corresponding momentum component $p(t)$. The phase portrait is given in Fig. 15.4(c); here, the baseline trajectory exhibits noticeable drift away from the ground-truth circular orbit, indicating instability. By contrast, the HNN preserves the energy level, whereas the baseline network displays visible energy drift away from the ground-truth level; see Fig. 15.5.

The code can be found at: `https://tinyurl.com/34wvs5cs`.



(a) **(b)**



(c)

Figure 15.4: Comparison of ground-truth, baseline, and HNN solutions: (a) position $q(t)$; (b) momentum $p(t)$; (c) phase portrait (q, p). The baseline trajectory drifts from the ground-truth orbit, while the HNN closely tracks it.

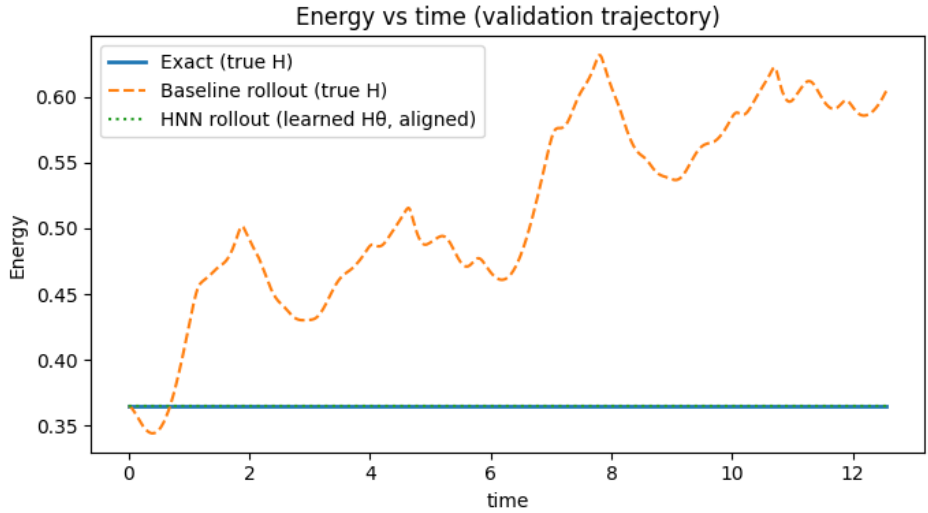


Figure 15.5: Energy curves versus training epochs for the ground-truth system, the baseline network, and the HNN. The HNN maintains the correct energy level, while the baseline exhibits noticeable energy drift.

15.3 Reconstruction of Phase-Space Trajectories

We consider a dynamical system governed by a natural Hamiltonian with an unknown potential function $U(q)$. A collection of N measurements is available in the form of pairs

$$\big((q_i, p_i),\, (q_i', p_i')\big),$$

representing the positions and momenta, together with their time derivatives. A Hamiltonian Neural Network (HNN) is employed to learn the underlying potential function $U(q)$, and consequently to reconstruct the Hamiltonian

$$H(q,p) = \frac{1}{2}p^2 + U(q).$$

Once the Hamiltonian has been learned, the phase-space trajectories can be recovered as level sets of constant energy,

$$H(q(t), p(t)) = c,$$

where c is a constant determined by the initial conditions. These level sets describe the system's motion in the (q,p) plane.

By analyzing the critical points of the potential,

$$\{\, q \;;\; \nabla U(q) = 0 \,\},$$

we can infer the stability and qualitative nature of the corresponding equilibrium points in phase space. This analysis provides valuable insight into the geometry of the energy

landscape and the local and global behavior of the learned dynamics. We conclude this section with an illustrative example.

Implementation example We consider a dynamical system defined by the natural Hamiltonian

$$H(q,p) = \frac{1}{2}p^2 - \frac{1}{2}q^2 + \frac{1}{3}q^3.$$

The corresponding potential energy is

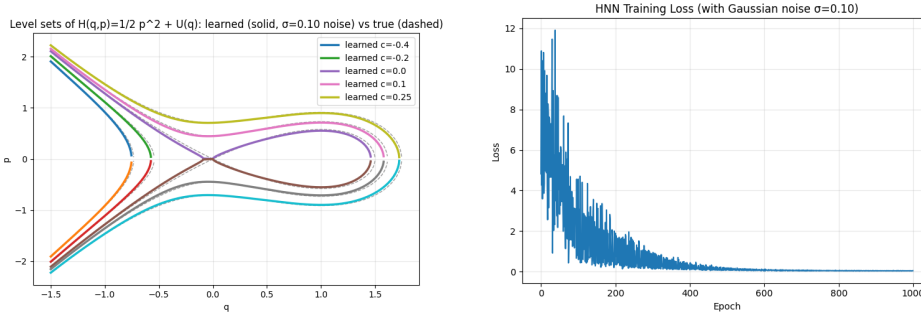
$$U(q) = -\frac{1}{2}q^2 + \frac{1}{3}q^3.$$

The derivative of $U(q)$ vanishes at $q \in \{0, 1\}$, and since $U''(0) = -1 < 0$ and $U''(1) = 1 > 0$, we conclude that $q = 1$ is a stable equilibrium point, while $q = 0$ is unstable. No other equilibrium points exist.

The phase-space trajectories satisfy the energy conservation law

$$\frac{1}{2}p(t)^2 - \frac{1}{2}q(t)^2 + \frac{1}{3}q(t)^3 = c,$$

where c is a constant determined by the initial conditions.



(a) **(b)**

Figure 15.6: (a) Comparison between the Runge–Kuta trajectories (dashed) and the HNN trajectories (solid, color); (b) The loss across epochs.

The associated Hamiltonian system is given by

$$\dot{q} = p,$$
$$\dot{p} = q - q^2.$$

Synthetic Data Generation. To construct the training data, we solve the above Hamiltonian system for 25 distinct initial conditions (q_0, p_0) using a fourth-order Runge–Kutta method. For each trajectory, we uniformly sample 30 points (q_i, p_i) and compute their corresponding derivatives (q_i', p_i'). To simulate measurements errors, we added a Gaussian noise with $\sigma = 0.10$ to all data. These noisy pairs constitute the training dataset.

Training and Reconstruction. Training is performed using a Hamiltonian Neural Network (HNN), assuming that the underlying Hamiltonian is of natural form. Once the Hamiltonian has been learned, we reconstruct the phase-space trajectories by drawing the constant level curves of the learned Hamiltonian. The resulting trajectories, shown in Fig. 15.6(a), accurately reproduce the structure of the original system, confirming the network's ability to recover the underlying dynamics. Training for $1,000$ epochs, the loss reaches its saturation threshold, see Fig. 15.6(b).
The code is available at: `https://tinyurl.com/52fdpxf7`.

15.4 Exercises

Exercise 15.4.1 (Linear Markov transition). Consider a discrete-time Markov process

$$x_{k+1} = f(x_k), \qquad f(x) = ax + b, \quad a = 2,\ b = -3.$$

Take the initial state $x_0 = 0$.

(a) Generate the first $N = 2,000$ states $\{x_k\}_{k=0}^{N-1}$ by iterating $x_{k+1} = 2x_k - 3$.

(b) Using the pairs (x_k, x_{k+1}) for $k = 0, \ldots, N-2$, train a neural network $f_\theta : \mathbb{R} \to \mathbb{R}$ to approximate the transition $x_{k+1} = f(x_k)$ by minimizing

$$\mathcal{L}(\theta) = \frac{1}{N-1} \sum_{k=0}^{N-2} \big(x_{k+1} - f_\theta(x_k)\big)^2.$$

(Use a small network with tanh or ReLU activations and Adam optimizer.)

(c) Performs these tasks:

- Compute the mean squared error of f_θ on a held-out subset (e.g., the last 20% of the pairs).
- Compare $f_\theta(x)$ pointwise to the ground-truth $f(x) = 2x - 3$ on a uniform grid of x values; report the max/mean absolute deviation.

(d) Plot $f_\theta(x)$ and $f(x) = 2x - 3$ over a chosen interval. Scatter plot of $(x_{k+1},\ f_\theta(x_k))$ with $y = x$ for visual alignment.

Exercise 15.4.2 (Nonlinear pendulum). Consider a frictionless unit mass pendulum of unit length whose dynamics is modeled by the Hamiltonian function

$$H(q,p) = \tfrac{1}{2}p^2 + 3(1 - \cos q), \qquad q' = p, \quad p' = -3\sin q.$$

(a) Sample initial conditions (q_0, p_0) with

$$H_0 = \tfrac{1}{2}p_0^2 + 3(1 - \cos q_0) \in [1.3,\ 2.3],$$

generate 25 training trajectories, take 30 samples per trajectory, and add zero-mean Gaussian noise (variance $\sigma^2 = 0.1$) to q, p, q', p'.

(b) Train a neural network $(q, p) \mapsto (q', p')$ (3 layers, 200 hidden units, tanh, Adam 10^{-3}, $\sim$2000 steps). Report train MSE on (q', p').

(c) Assume $H(q, p) = \frac{1}{2}p^2 + U_\theta(q)$ and train U_θ with

$$\mathcal{L}_{\text{red}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \left| U'_\theta(q_i) + p'_i \right|^2,$$

using the same optimizer. Report train $\mathcal{L}_{\text{red}}$ across epochs.

(d) For a chosen (q_0, p_0) in the previous range:

- Roll out three trajectories on the same time grid: (i) ground truth ($q' = p$, $p' = -3\sin q$), (ii) baseline field, (iii) HNN ($q' = p$, $p' = -U'_\theta(q)$).
- Plot $q(t)$, $p(t)$, and the phase portrait (q, p) for all three.
- Plot energies vs. time: $H_{\text{true}} = \frac{1}{2}p^2 + 3(1 - \cos q)$ for ground truth and baseline; $H_\theta = \frac{1}{2}p^2 + U_\theta(q)$ for HNN. Align H_θ by adding a constant so $H_\theta(q_0, p_0) = H_{\text{true}}(q_0, p_0)$.

Exercise 15.4.3 (Two-body problem). Consider two point masses m_1, m_2 moving in the plane under an attractive inverse-distance potential (gravity-like) with constant g. Let $q_i = (x_i, y_i) \in \mathbb{R}^2$ and $p_i = (p_{x,i}, p_{y,i}) \in \mathbb{R}^2$ for $i = 1, 2$. The Hamiltonian in center-of-mass coordinates is

$$H(q_1, q_2, p_1, p_2) = \frac{\|p_{\text{CM}}\|^2}{2(m_1 + m_2)} + \frac{\|p\|^2}{2\mu} - \frac{g\, m_1 m_2}{\|r\|},$$

$$r := q_1 - q_2,\ p := \mu\left(\frac{p_1}{m_1} - \frac{p_2}{m_2}\right),\ p_{\text{CM}} = p_1 + p_2,\ \mu = \frac{m_1 m_2}{m_1 + m_2}.$$

In this exercise take $m_1 = m_2 = g = 1$ and restrict to $p_{\text{CM}} = 0$ (zero total momentum), yielding eight scalar degrees of freedom (two positions and two momenta in 2D for each body). In this setup, the natural form is

$$H = \tfrac{1}{2}\|p_1\|^2 + \tfrac{1}{2}\|p_2\|^2 + U(q_1, q_2), \qquad U(q_1, q_2) = -\frac{1}{\|q_1 - q_2\|}.$$

Hamilton's equations give $q'_i = \partial H/\partial p_i = p_i$ and $p'_i = -\partial U/\partial q_i$.

(a) Generate 200 trajectories with 50 samples each by:

- Initializing with CM at the origin and $p_{\text{CM}} = 0$.
- Choosing radii $r = \|q_2 - q_1\| \sim$ Uniform[0.5, 1.5] and setting initial velocities for *perfectly circular* orbits, then adding zero-mean Gaussian noise to the initial velocities with standard deviation $\sigma = 0.05$ (component-wise).
- Integrating with fixed-step RK4 over a common time grid.

Split trajectories 80/20 into train/test. Record (q_1, q_2, p_1, p_2) and the time derivatives (q_1', q_2', p_1', p_2') along each trajectory.

(b) Train a neural network to map

$$(q_1, q_2, p_1, p_2) \longmapsto (q_1', q_2', p_1', p_2')$$

using the noisy training set (e.g., 3 layers, 200 hidden units, tanh, Adam 10^{-3}, batch size 200, 10,000 steps). Report train/test MSE on all output components.

(c) Assume $H = \frac{1}{2}\|p_1\|^2 + \frac{1}{2}\|p_2\|^2 + U_\theta(q_1, q_2)$ with a neural potential $U_\theta : \mathbb{R}^4 \to \mathbb{R}$. Train U_θ with the reduced loss enforcing $p_i' = -\nabla_{q_i} U_\theta$:

$$\mathcal{L}_{\text{red}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\Big(\big\|\nabla_{q_1} U_\theta(q_{1,i}, q_{2,i}) + p_{1,i}'\big\|^2 + \big\|\nabla_{q_2} U_\theta(q_{1,i}, q_{2,i}) + p_{2,i}'\big\|^2\Big).$$

Use the same optimizer as the baseline network. Report train/test $\mathcal{L}_{\text{red}}$.

(d) Choose a representative initial condition from the test set and produce three rollouts on the same time grid:

- *Ground truth*: integrate $q_i' = p_i$, $p_i' = -\nabla_{q_i} U$ with $U(r) = -1/\|r\|$.
- *Baseline*: integrate using the learned baseline vector field.
- *HNN*: integrate $q_i' = p_i$, $p_i' = -\nabla_{q_i} U_\theta$.

Plot the following: (i) x–y orbits of both bodies, or equivalently the relative orbit $r(t)$; (ii) the phase portrait(s) in (q_i, p_i) or in $(r, \dot{r})$.

(e) Compute the true energy $H_{\text{true}} = \frac{1}{2}\|p_1\|^2 + \frac{1}{2}\|p_2\|^2 - 1/\|r\|$ along the ground-truth and baseline rollouts. Compute $H_\theta = \frac{1}{2}\|p_1\|^2 + \frac{1}{2}\|p_2\|^2 + U_\theta(q_1, q_2)$ along the HNN rollout and *align* it by adding a constant so H_θ matches H_{true} at $t = 0$; then plot all energies vs. time and discuss drift vs. preservation.

(f) Train/test errors, trajectory and phase-space plots, and energy vs. time (with HNN energy aligned), with a brief comment on which model better respects the Hamiltonian structure.

Exercise 15.4.4 (Constant force). Consider the time-independent Hamiltonian

$$H(q, p) = \tfrac{1}{2}p^2 + 3q, \qquad q' = \frac{\partial H}{\partial p} = p, \quad p' = -\frac{\partial H}{\partial q} = -3.$$

The dynamics correspond to a constant force field. The exact solution is

$$p(t) = p_0 - 3t, \qquad q(t) = q_0 + p_0 t - \tfrac{3}{2}t^2,$$

and the energy is conserved: $H(t) \equiv H_0 = \frac{1}{2}p_0^2 + 3q_0$.

(a) Sample initial conditions (q_0, p_0) with energies

$$H_0 = \tfrac{1}{2}p_0^2 + 3q_0 \in [0.5,\, 2.0].$$

Generate 25 training each with 30 samples in a common time window. Record $\big(q(t_i), p(t_i)\big)$ and $\big(q'(t_i), p'(t_i)\big)$, and add zero-mean Gaussian noise with variance $\sigma^2 = 0.1$ to all recorded quantities.

(b) Train a baseline network $(q, p) \mapsto (q', p')$ (3 layers, 200 hidden units, tanh, Adam 10^{-3}, $\sim$2000 steps). Report train MSE on (q', p') across epochs.

(c) Assume $H(q, p) = \frac{1}{2}p^2 + U_\theta(q)$ and learn only U_θ. Train with the reduced loss enforcing $p' = -U'(q)$:

$$\mathcal{L}_{\text{red}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \big|\, U'_\theta(q_i) + p'_i \,\big|^2.$$

Use the Adam optimizer with learning rate 0.001. Report train loss $\mathcal{L}_{\text{red}}$.

(d) Pick a representative (q_0, p_0) in the energy range and produce three rollouts on the same time grid:
(i) *Exact* via $q' = p,\ p' = -3$;
(ii) *Baseline* using the learned field;
(iii) *HNN* via $q' = p,\ p' = -U'_\theta(q)$.
Plot $q(t)$, $p(t)$, and the phase portrait (q, p).

(e) Compute $H_{\text{true}} = \frac{1}{2}p^2 + 3q$ along exact and baseline rollouts. Compute $H_\theta = \frac{1}{2}p^2 + U_\theta(q)$ along the HNN rollout and *align* it by adding a constant so $H_\theta(q_0, p_0) = H_{\text{true}}(q_0, p_0)$. Plot all energies vs. time and comment on drift vs. preservation.

(f) Plot components $q(t)$, $p(t)$, phase portrait (q, p) and energy vs. time and a brief comment on which model best respects the Hamiltonian structure.

15.5 Lagrangian Neural Networks (LNN)

From HNNs to LNNs Standard Hamiltonian Neural Networks (HNNs) are formulated in *canonical* phase-space coordinates (q, p) rather than configuration-velocity coordinates (q, q'). Here q denotes position and p the conjugate momentum, obtained from a Lagrangian L via

$$p_i \;=\; \frac{\partial L}{\partial q'_i}.$$

Canonical coordinates satisfy the Poisson-bracket relations

$$\{q_i, q_j\} = 0, \qquad \{p_i, p_j\} = 0, \qquad \{q_i, p_j\} = \delta_{ij},$$

with

$$\{f, g\} \;=\; \sum_i \frac{\partial f}{\partial q_i}\frac{\partial g}{\partial p_i} \;-\; \frac{\partial f}{\partial p_i}\frac{\partial g}{\partial q_i}.$$

Because HNNs operate natively on (q, p), they cannot directly learn from data given only as (q, q'). To address this, Lagrangian Neural Networks (LNNs) [29] and Deep Lagrangian Networks (DLNs) [65] learn the Lagrangian $L_\theta(q, q')$ as a latent function during training, then recover momenta via $p = \partial_{q'} L_\theta$ to induce Hamiltonian dynamics consistent with the observed (q, q') data.

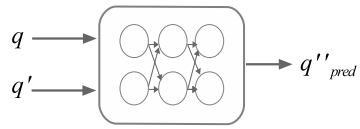


Figure 15.7: The schematic of a baseline network to predict acceleration from the position and velocity.

Problem statement and baseline. Given training data of positions, velocities, and accelerations $\{(q_i, q_i', q_i'')\}_{i=1}^N$, the goal is to learn a model that predicts accelerations q'' from inputs (q, q'). A straightforward baseline is a supervised feed-forward network

$$q''_{\text{pred}} = q''_\theta = f_\theta(q, q'),$$

trained with a regression loss (e.g., MSE) between q''_{pred} and q''. While simple, this approach does not encode the system's symmetries or Physics; in particular, conservation laws (e.g., energy) are not respected unless enforced via additional soft constraints. See Fig. 15.7.

Lagrangian Neural Networks (LNNs). Following Cranmer et al. [29], an LNN instead *learns the Lagrangian* $L_\theta(q, q')$ and derives dynamics from the Euler–Lagrange equations

$$\frac{d}{dt}\Big(\partial_{q'} L_\theta(q, q')\Big) \;-\; \partial_q L_\theta(q, q') \;=\; 0,$$

which yield a mapping $(q, q') \mapsto q''_\theta$ by solving for accelerations. The model is then trained to minimize

$$\mathcal{L}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N} \big\|q''_\theta(q_i, q_i') - q_i''\big\|^2 \;+\; \text{(optional regularizers/soft constraints)}. \quad (15.5.1)$$

The acceleration $q''_\theta(q_i, q_i')$ is obtained from the Lagrangian $L_\theta(q, q')$ as follows.

Using chain rule we compute the left side of the Euler-Lagrange equation as

$$\frac{d}{dt}\Big(\frac{\partial L_\theta}{\partial q_j'}(q, q')\Big) = \frac{\partial^2 L_\theta}{\partial q_j' \partial q_i} q_i' + \frac{\partial^2 L_\theta}{\partial q_j' \partial q_k'} q_k''.$$

This is more convenient to be written in the equivalent matrix way as

$$\frac{d}{dt}\nabla_{q'} L_\theta = (\nabla_{q'}\nabla_{q'}^\top L_\theta) q'' + (\nabla_q \nabla_{q'}^\top L_\theta) q'.$$

Substituting in the Euler–Lagrange equation and solving for the acceleration yields

$$q''_\theta = (\nabla_{q'}\nabla_{q'}^\top L_\theta)^{-1}[\nabla_q L_\theta - (\nabla_q L_\theta \nabla_{q'}^\top L_\theta) q']. \quad (15.5.2)$$

This is the formula which is used in the loss (15.5.1), which minimizes the discrepancy between the true accelerations and the computed accelerations. See Fig. 15.8.

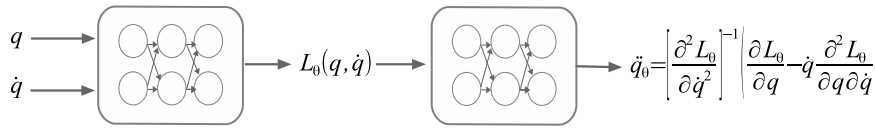


Figure 15.8: The schematic of an LNN.

We note that formula (15.5.2) is a Newton-type equation, the connotation of the right term being a force. We shall provide an example where from a given Lagrangian we can compute explicitly the accelerations.

Example 15.5.1 (2D-harmonic oscillator). Let the potential be quadratic,

$$U(q_1, q_2) = \tfrac{1}{2}\,(q_1^2 + q_2^2),$$

and take the Lagrangian

$$L = \tfrac{1}{2} m\big({q_1'}^2 + {q_2'}^2\big) \;-\; \tfrac{1}{2}\,({q_1}^2 + {q_2}^2).$$

Compute the required derivatives as

$$\nabla_{q'}\nabla_{q'}^{\top} L = \begin{pmatrix} \partial_{q_1'}\partial_{q_1'} L & \partial_{q_1'}\partial_{q_2'} L \\ \partial_{q_2'}\partial_{q_1'} L & \partial_{q_2'}\partial_{q_2'} L \end{pmatrix} = \begin{pmatrix} m & 0 \\ 0 & m \end{pmatrix},$$

$$\nabla_q \nabla_{q'}^{\top} L = 0_{2\times 2}, \qquad \nabla_q L = \begin{pmatrix} \partial_{q_1} L \\ \partial_{q_2} L \end{pmatrix} = \begin{pmatrix} -\,q_1 \\ -\,q_2 \end{pmatrix}.$$

Using the forward model

$$q'' = \big(\nabla_{q'}\nabla_{q'}^{\top} L\big)^{-1}\Big(\nabla_q L - \big(\nabla_q \nabla_{q'}^{\top} L\big) q'\Big),$$

we obtain

$$\begin{pmatrix} q_1'' \\ q_2'' \end{pmatrix} = \begin{pmatrix} m & 0 \\ 0 & m \end{pmatrix}^{-1} \begin{pmatrix} -\,q_1 \\ -\,q_2 \end{pmatrix} = \frac{1}{m} \begin{pmatrix} -\,q_1 \\ -\,q_2 \end{pmatrix}.$$

Thus each coordinate obeys the harmonic-oscillator equation

$$q_i'' + \omega^2 q_i = 0, \qquad \omega = \sqrt{\tfrac{1}{m}}, \quad i = 1, 2.$$

Remark 15.5.2 The invertibility of the Hessian matrix

$$\nabla_{q'}\nabla_{q'}^{\top} L = \Big(\tfrac{\partial^2 L}{\partial q_i' \partial q_j'}\Big)_{i,j}$$

is equivalent to strict convexity of L in the velocities, i.e., an elliptic, or positive-definite kinetic term. When this matrix is singular, the Lagrangian is *degenerate* and the above forward method no longer applies. Below we present a counterexample illustrating the singular case.

Example 15.5.3 Consider the Lagrangian

$$L(q, q') = \tfrac{1}{2}\big(q_1'^{\,2} + q_2'^{\,2}\big) \;+\; \lambda\big(q_3' - 2q_2q_1' + 2q_1q_2'\big),$$

which models sub-Riemannian geodesics on the three-dimensional Heisenberg group, see Calin et al. [18]. It consists of the usual kinetic energy in the (q_1, q_2)-plane together with a linear velocity term enforcing the nonholonomic constraint

$$q_3' = 2\,(q_2q_1' - q_1q_2').$$

We note the Euler-Lagrange equation for q_3 yields $\dot{\lambda} = 0$, so λ is constant along trajectories.

From a Physics viewpoint, this is the Lagrangian of a unit-mass charged particle moving in a constant magnetic field

$$\mathbf{B} = (0, 0,\, 4\lambda).$$

In this example the Hessian matrix is singular:

$$\nabla_{q'}\nabla_{q'}L = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix},$$

reflecting the absence of a quadratic kinetic term in q_3'. Consequently, the LNN cannot be applied for this case. However, we can learn the dynamics by applying HNN with the Hamiltonian

$$H(q, p) = \frac{1}{2}\big(p_1 + 2q_2p_3\big)^2 + \frac{1}{2}\big(p_2 - 2q_1p_3\big)^2.$$

Implementation example We consider the case of the harmonic oscillator.

Data collection. Consider the Lagrangian

$$L(q, q') = \tfrac{1}{2}\,q'^{\,2} - \tfrac{1}{2}\,q^2.$$

The associated Euler–Lagrange equation is $q'' + q = 0$. With initial conditions $q(0) = q_0$ and $q'(0) = v_0$, the solution is

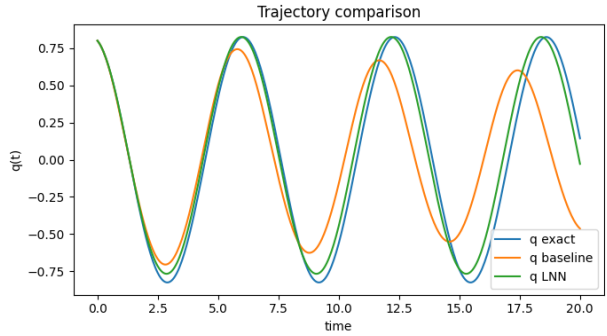
$$q(t) = q_0\cos t + v_0\sin t, \quad q'(t) = -\,q_0\sin t + v_0\cos t, \quad q''(t) = -\,q_0\cos t - v_0\sin t.$$

Along each solution, the energy is conserved:

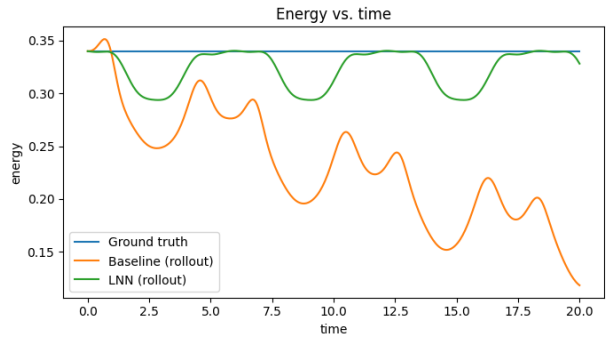
$$E(q_0, v_0) = \tfrac{1}{2}\,q'(t)^2 + \tfrac{1}{2}\,q(t)^2 = \tfrac{1}{2}\,(v_0^2 + q_0^2).$$

We generate 25 pairs of initial conditions (q_0, v_0) with $E(q_0, v_0) \in [0.2, 1]$. For each trajectory, we uniformly sample $N = 30$ points and record the triplets $(q(t_i),\, q'(t_i),\, q''(t_i))$, then add Gaussian noise with variance $\sigma^2 = 0.1$ to the data. These noisy triplets constitute the training set: the first two components are the inputs to the network, and the last one (the acceleration) is the target variable.

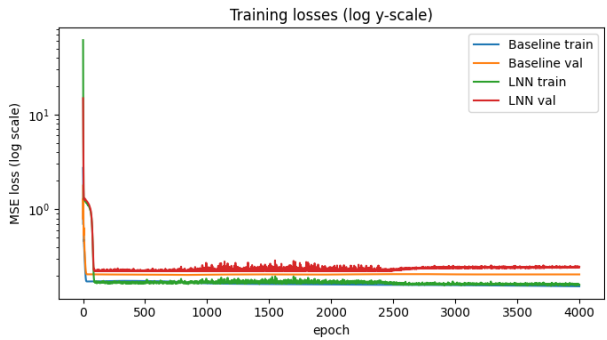
Training. We train two neural networks as follows.



(a)



(b)



(c)

Figure 15.9: (a) The trajectory comparison for the exact, baseline, and LNN solutions; (b) The energy versus time: the baseline energy drifts away from the ground-truth, whereas the LNN energy oscillates within a small neighborhood of the ground-truth curve; (c) The baseline and the LNN achieve comparable training and validation losses.

(i) *Baseline model.* This network is trained in a purely supervised manner, without any physical prior, to learn the mapping $(q, q') \mapsto q''$.

(ii) *Lagrangian Neural Network.* Let $NN(\theta; q, q') = L_\theta(q, q')$ denote a neural network that outputs a scalar Lagrangian. We append a differentiable layer that maps (q, q') to acceleration via

$$q''_\theta = \left(\frac{\partial^2 L_\theta}{\partial q'^2}\right)^{-1} \left[\frac{\partial L_\theta}{\partial q} - \frac{\partial^2 L_\theta}{\partial q \, \partial q'} \, q'\right].$$

The resulting model is trained to minimize

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(q''_\theta(t_i) - q''(t_i)\right)^2,$$

so that the Lagrangian L_θ is learned in the process and then used to generate dynamics.

Architectures. We set the time horizon to $T = 10$ and sample $N = 30$ points along each trajectory. In both cases, the architecture is a feed-forward network with two hidden layers of 164 neurons each, using `softplus` activations. Training is performed for $4,000$ epochs with a batch size of 128.

Results. We compare the two networks and validate them against the exact solution for a given initial condition (q_0, v_0). Figure 15.9(a) shows the trajectory comparison for the exact, baseline, and LNN solutions; the LNN remains much closer to the exact trajectory than the baseline. Figure 15.9(b) plots the energy versus time: the baseline energy *drifts* away from the ground-truth (computed analytically), whereas the LNN energy oscillates within a small neighborhood of the ground-truth curve. Notably, both the baseline and the LNN achieve comparable training and validation losses, see Fig. 15.9(c); thus, the LNN's advantage stems from its built-in Physics (energy-preserving inductive bias), not from lower empirical loss.

The code is available at: `https://tinyurl.com/dt94wyy6`.

15.6 Exercises

Exercise 15.6.1 (A mass-spring system). Consider the ideal mass-spring with Lagrangian

$$L(q, \dot{q}) = \frac{m}{2}\dot{q}^{\,2} - \frac{k}{2}q^2, \qquad k > 0,$$

so that the equation of motion is $\ddot{q} + \omega^2 q = 0$ with $\omega = \sqrt{k/m}$ and energy

$$E(q, \dot{q}) = \frac{m}{2}\dot{q}^{\,2} + \frac{k}{2}q^2.$$

(a) Generate 25 pairs of initial conditions (q_0, v_0) such that the corresponding energies $E_0 = \frac{m}{2}v_0^2 + \frac{k}{2}q_0^2$ lie in a prescribed interval (e.g. $[0.2, 1]$ after choosing $m = k = 1$). For each trajectory, sample $N = 30$ uniform time points t_i on $[0, T]$, with $T = 10$, and record the triplets

$$\big(q(t_i),\, \dot{q}(t_i),\, \ddot{q}(t_i)\big), \qquad i = 1, \ldots, N,$$

where, for the ground truth, $q(t) = q_0 \cos(\omega t) + \frac{v_0}{\omega} \sin(\omega t)$,

$$\dot{q}(t) = -\omega q_0 \sin(\omega t) + v_0 \cos(\omega t), \quad \ddot{q}(t) = -\omega^2 q(t).$$

Add also a small Gaussian noise with standard deviation $\sigma = 0.1$ to all measurements.

(b) Train a baseline feed-forward neural network to learn the mapping $(q, \dot{q}) \longmapsto \ddot{q}$ by minimizing the mean-squared error

$$\mathcal{L}_{\text{base}}(\theta) = \frac{1}{N_{\text{tot}}} \sum_i \left(\hat{\ddot{q}}_\theta(q_i, \dot{q}_i) - \ddot{q}_i \right)^2,$$

where the sum runs over all collected samples from all trajectories.

(c) Let an LNN neural network $L_\theta(q, \dot{q})$ approximate the Lagrangian and predict acceleration via the Euler–Lagrange map

$$\hat{\ddot{q}}_\theta(q, \dot{q}) = \left(\partial_{\dot{q}\dot{q}} L_\theta(q, \dot{q}) \right)^{-1} \left(\partial_q L_\theta(q, \dot{q}) - \partial_{q\dot{q}} L_\theta(q, \dot{q})\, \dot{q} \right),$$

then train by minimizing the loss

$$\mathcal{L}_{\text{LNN}}(\theta) = \frac{1}{N_{\text{tot}}} \sum_i \left(\hat{\ddot{q}}_\theta(q_i, \dot{q}_i) - \ddot{q}_i \right)^2.$$

(d) For evaluation and comparison purposes, choose a test initial condition (q_0^*, v_0^*). Then plot on the same axes:

- $q(t)$ for the ground truth, the baseline solution, and the LNN solution.
- The energy $E(t) = \frac{m}{2} v(t)^2 + \frac{k}{2} q(t)^2$ for all three.

(e) Comment on trajectory accuracy and long-horizon energy behavior. Which model better preserves energy and phase over time? How does the Lagrangian inductive bias affect generalization compared to the baseline?

Exercise 15.6.2 (An ideal pendulum). Consider the ideal pendulum with Lagrangian

$$L(q, \dot{q}) = \frac{m}{2} \dot{q}^{\,2} - 2mg\ell\,(1 - \cos q), \qquad m, \ell, g > 0,$$

so that the equation of motion is

$$\ddot{q} + \omega_0^2 \sin q = 0, \qquad \omega_0 = \sqrt{\tfrac{g}{\ell}},$$

and the (mechanical) energy is

$$E(q, \dot{q}) = \frac{m}{2} \dot{q}^{\,2} + 2mg\ell\,(1 - \cos q).$$

(a) For data collection, we fix $T = 10$ and sample $N = 30$ uniform time points $t_i \in [0, T]$ on each trajectory. Generate 25 pairs of initial conditions (q_0, v_0) whose energies

$$E_0 = \frac{m}{2} v_0^2 + 2mg\ell\,(1 - \cos q_0)$$

lie in a prescribed interval (e.g. $[0.2, 1]$ after non-dimensionalization with $m = \ell = g = 1$). For each (q_0, v_0), obtain the *ground-truth* trajectory by numerically integrating

$$\dot{q} = v, \qquad \dot{v} = -\omega_0^2 \sin q,$$

with a high-accuracy solver (such as RK4 with a small step), and record the triplets

$$\big(q(t_i),\, \dot{q}(t_i),\, \ddot{q}(t_i)\big), \qquad \ddot{q}(t_i) = -\omega_0^2 \sin q(t_i), \quad i = 1, \dots, N.$$

Add small Gaussian noise with standard deviation $\sigma = 0.1$ to all measured quantities.

(b) Perform the supervised training of a baseline feed-forward neural network to learn the mapping

$$(q, \dot{q}) \longmapsto \ddot{q}$$

by minimizing the mean-squared error

$$\mathcal{L}_{\text{base}}(\theta) = \frac{1}{N_{\text{tot}}} \sum_i \Big(\hat{\ddot{q}}_\theta(q_i, \dot{q}_i) - \ddot{q}_i \Big)^2,$$

where the sum runs over all samples from all trajectories.

(c) Let an LNN neural network $\mathcal{L}_\theta(q, \dot{q})$ approximate the Lagrangian and predict acceleration via the Euler–Lagrange map

$$\hat{\ddot{q}}_\theta(q, \dot{q}) = \Big(\partial_{\dot{q}\dot{q}} L_\theta(q, \dot{q}) \Big)^{-1} \Big(\partial_q L_\theta(q, \dot{q}) - \partial_{q\dot{q}} L_\theta(q, \dot{q})\, \dot{q} \Big),$$

then train by minimizing

$$\mathcal{L}_{\text{LNN}}(\theta) = \frac{1}{N_{\text{tot}}} \sum_i \Big(\hat{\ddot{q}}_\theta(q_i, \dot{q}_i) - \ddot{q}_i \Big)^2.$$

(d) To evaluate, choose a test initial condition (q_0^*, v_0^*). Using the *same* numerical integrator and step size for both learned models, roll out trajectories from (q_0^*, v_0^*) for: (i) ground truth, (ii) baseline model, (iii) LNN.

Then plot on the same axes:

- $q(t)$ for all three cases;
- the energy $E(t) = \frac{m}{2} v(t)^2 + 2mg\ell\,(1 - \cos q(t))$ for all three.

(e) Comment on trajectory accuracy (phase and amplitude) and long-horizon energy behavior. Which model better preserves energy and phase over time, especially for larger amplitudes where the dynamics is nonlinear (no small-angle approximation)?

Exercise 15.6.3 (A two-body system). Consider the planar two-body problem with unit masses and Lagrangian

$$L(q_1, q_2, \dot{q}_1, \dot{q}_2) = \tfrac{1}{2}\|\dot{q}_1\|^2 + \tfrac{1}{2}\|\dot{q}_2\|^2 - \frac{1}{\|q_1 - q_2\|^2}, \qquad q_1, q_2 \in \mathbb{R}^2,$$

where $\|\cdot\|$ is the Euclidean norm. Writing $r := q_1 - q_2$, the Euler–Lagrange equations are

$$\ddot{q}_1 = +\frac{2\,r}{\|r\|^4}, \qquad \ddot{q}_2 = -\frac{2\,r}{\|r\|^4},$$

and the energy and (planar) angular momentum are

$$E = \tfrac{1}{2}\|\dot{q}_1\|^2 + \tfrac{1}{2}\|\dot{q}_2\|^2 + \frac{1}{\|r\|^2}, \qquad J = q_1^{\perp}\cdot \dot{q}_1 + q_2^{\perp}\cdot \dot{q}_2 \quad (\text{with } (x_1, x_2)^{\perp} = (-x_2, x_1)).$$

(a) For the data collection generate 25 initial conditions $\big(q_1^0, q_2^0,\ v_1^0, v_2^0\big) \in (\mathbb{R}^2)^4$ satisfying:

- a minimum separation constraint $\|q_1^0 - q_2^0\| \geq \rho_{\min}$ (e.g. $\rho_{\min} = 0.5$) to avoid the singularity,
- a prescribed energy range (after non-dimensionalization, e.g. $E \in [0.2, 1]$).

For each initial condition, integrate the ODE system

$$\dot{q}_i = v_i, \qquad \dot{v}_1 = \frac{2\,r}{\|r\|^4}, \quad \dot{v}_2 = -\frac{2\,r}{\|r\|^4}, \qquad r = q_1 - q_2,$$

on $[0, T]$ with $T = 10$ using a fixed time step. Sample $N = 30$ uniform times t_k and record the triplets, for *each* body $i = 1, 2$,

$$\big(q_i(t_k),\, v_i(t_k),\, a_i(t_k)\big), \qquad a_1(t_k) = \tfrac{2\,r(t_k)}{\|r(t_k)\|^4},\ a_2(t_k) = -\tfrac{2\,r(t_k)}{\|r(t_k)\|^4}.$$

Optionally, add a small Gaussian noise (e.g. $\sigma = 0.05$) to emulate measurement error.
(b) Flatten the state into inputs $x = (q_1, q_2, v_1, v_2) \in \mathbb{R}^8$ and targets $y = (a_1, a_2) \in \mathbb{R}^4$. Train a baseline feed-forward neural network to learn

$$x \longmapsto \hat{y}_\theta = (\hat{a}_{1,\theta}, \hat{a}_{2,\theta})$$

by minimizing the mean–squared error over all samples:

$$\mathcal{L}_{\text{base}}(\theta) = \frac{1}{N_{\text{tot}}} \sum_k \big\|\hat{a}_{1,\theta}(x_k) - a_1(t_k)\big\|^2 + \frac{1}{N_{\text{tot}}} \sum_k \big\|\hat{a}_{2,\theta}(x_k) - a_2(t_k)\big\|^2.$$

(c) Let an LNN neural network $L_\theta(q_1, q_2, v_1, v_2)$ approximate the Lagrangian and define accelerations via the Euler–Lagrange map:

$$\hat{a}_{i,\theta} = \Big(\partial_{v_i v_i} L_\theta\Big)^{-1} \Big(\partial_{q_i} L_\theta - \partial_{q_i v_i} L_\theta\, v_i\Big), \qquad i = 1, 2,$$

(when cross terms $\partial_{v_i v_j}$ for $i \neq j$ are present, use the 4×4 Hessian in block form). Train by the supervised loss

$$\mathcal{L}_{\text{LNN}}(\theta) = \frac{1}{N_{\text{tot}}} \sum_k \big\|\hat{a}_{1,\theta}(x_k) - a_1(t_k)\big\|^2 + \frac{1}{N_{\text{tot}}} \sum_k \big\|\hat{a}_{2,\theta}(x_k) - a_2(t_k)\big\|^2.$$

(d) To evaluate, pick a test initial condition $(q_1^*, q_2^*, v_1^*, v_2^*)$ respecting the separation constraint. From this state, roll out three trajectories on $[0, T_{\text{eval}}]$ with the *same* integrator and step size:

(i) ground truth;

(ii) baseline model $\dot{v}_i = \hat{a}_{i,\theta_{\text{base}}}(q_1, q_2, v_1, v_2)$;

(iii) LNN $\dot{v}_i = \hat{a}_{i,\theta_{\text{LNN}}}(\cdot)$.

Plot on shared axes:

- The trajectories in configuration space (e.g. $q_1(t)$ and $q_2(t)$ or the relative motion $r(t)$).
- The energy $E(t) = \frac{1}{2}\|v_1\|^2 + \frac{1}{2}\|v_2\|^2 + \|r\|^{-2}$ for all three.
- The angular momentum $J(t)$.

(e) Which model better preserves energy and angular momentum?

15.7 Hamilton–Jacobi Neural Networks

We propose a neural architecture that takes as input (t, q)—the time and the position of a dynamical system at time t—and predicts the momentum p at that time. During training the network learns an *action* $S_\theta(t, q)$, realized by a first block, see Fig. 15.10,

$$(t, q) \longmapsto S_\theta(t, q) \in \mathbb{R},$$

which is then followed by a final differentiation layer that outputs momentum via

$$p_\theta(t, q) = \partial_q S_\theta(t, q),$$

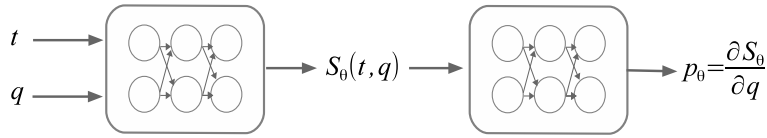


Figure 15.10: The schematic of a HJNN.

Problem *Given N measurements $\{(t_k, q_k; p_k)\}_{k=1}^N$ of time, position, and momentum of a dynamical system described by the Hamiltonian function $H(q, p)$, learn a model that predicts p from (t, q).*

1) Baseline network. A naive supervised approach uses a feed-forward model $NN(\theta; t, q) = p_\theta(t, q)$ trained with

$$\mathcal{L}_{\text{base}}(\theta) = \frac{1}{N} \sum_{k=1}^{N} \big(p_\theta(t_k, q_k) - p_k\big)^2.$$

2) Hamilton–Jacobi Neural Network (HJNN). Here the first network block outputs $S_\theta(t, q)$ and momentum is computed by the differentiation layer $p_\theta = \partial_q S_\theta$. Training minimizes a sum of three losses:

(i) the Hamilton–Jacobi (HJ) residual,

(ii) the momentum matching error, and

(iii) an anchoring term to fix the gauge of S:

$$\mathcal{L}_{\mathrm{HJNN}}(\theta) = \underbrace{\frac{1}{N}\sum_{k=1}^{N}\Big(\partial_t S_\theta(t_k, q_k) + H\big(q_k, \partial_q S_\theta(t_k, q_k)\big)\Big)^2}_{\text{HJ residual}} + \underbrace{\frac{1}{N}\sum_{k=1}^{N}\big(\partial_q S_\theta(t_k, q_k) - p_k\big)^2}_{\text{momentum matching}} + \underbrace{\big(S_\theta(t_0, q_0) - S_0\big)^2}_{\text{anchor}},$$

where $H(q, p)$ is the Hamiltonian governing the dynamics and S_0 is a chosen reference value (since S is defined up to an additive constant). The first term enforces that S_θ satisfies the Hamilton–Jacobi equation $\partial_t S + H(q, \partial_q S) = 0$; the second aligns $\partial_q S_\theta$ with the measured momenta; the third fixes the overall offset of S_θ. If $q_0 = q(t_0)$ denotes the initial position, then $S_0 = S(t_0, q_0) = 0$.

From the nonlinearity of the Hamilton–Jacobi equation, we might have multiple solutions satisfying the same initial solution. Therefore, the momentum, $p = p(t, q)$, obtained by differentiating S in variable q might be multi-valued, fact that is an impediment in finding the correct momentum. We shall see next how this method performs on a simple example.

Implementation example We consider again the case of the harmonic oscillator. The Hamiltonian function is

$$H(q, p) = \frac{1}{2}p^2 + \frac{1}{2}q^2,$$

with the corresponding Hamilton–Jacobi equation

$$\frac{\partial S}{\partial t} + \frac{1}{2}\Big(\frac{\partial S}{\partial q}\Big)^2 + \frac{1}{2}q^2 = 0.$$

Data collection. With initial conditions $q(0) = q_0$ and $p(0) = p_0$, the solution is

$$q(t) = q_0 \cos t + p_0 \sin t, \quad p(t) = p_0 \cos t - q_0 \sin t.$$

Along each solution, the energy is conserved:

$$E(q_0, p_0) = \tfrac{1}{2}\,(p_0^2 + q_0^2).$$

We generate 25 pairs of initial conditions (q_0, p_0) with $E(q_0, p_0) \in [0.2, 1]$. For each trajectory, we uniformly sample $N = 30$ time points and record the triplets $(t_i, q(t_i), p(t_i))$, then add Gaussian noise with variance $\sigma^2 = 0.1$ to the data. These noisy triplets constitute the training set: the first two components are the inputs to the network, and the last one (the momentum) is the target variable.

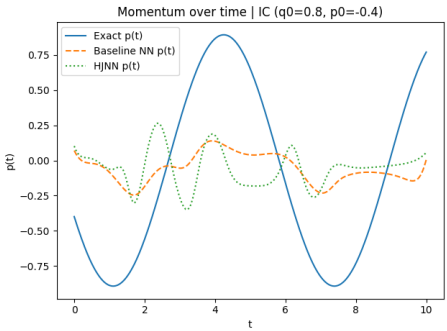
Training. We train again two neural networks:

(i) *Baseline model.* This network is trained in a purely supervised manner, without any physical prior, using a mean squared error to learn the mapping $(t, q) \mapsto p$.

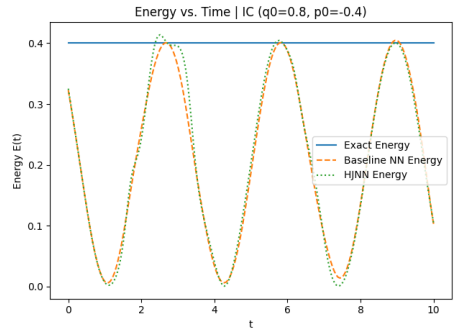
(ii) *Hamilton–Jacobi Neural Network.* Let $NN(\theta; t, q) = S_\theta(q, q')$ denote a neural network that outputs a scalar action. We append a differentiable layer that maps (t, q) to momentum via $p = \partial_q S_\theta(t, q)$.

Architecture. We set the time horizon to $T = 10$ and sample $N = 30$ points along each trajectory. In both cases, the architecture is a feed-forward network with two hidden layers of 164 neurons each, using `softplus` activations. Training is performed for $4,000$ epochs with a batch size of 128. We used Adam optimizer with a learning rate equal to 10^{-3}. We also set the anchor $S_0 = 0$ for $(t_0, q_0) = (0, 0)$.

Results. We evaluate both networks against the exact momentum for a fixed initial condition (q_0, v_0) given by `q0_test, p0_test = 0.8, -0.4`. Figure 15.11(a) compares the trajectories for the exact solution, the baseline, and the HJNN; neither learned model provides an adequate approximation to the ground truth. Figure 15.11(b) shows energy versus time: the energies produced by both the baseline and the HJNN oscillate away from the analytical value, indicating that the HJNN is not energy-preserving. These findings suggest that further improvements to the method are necessary.



(a) Momentum trajectories for the exact, baseline, and HJNN solutions.



(b) Energy versus time for the exact, baseline, and HJNN models.

Figure 15.11: Comparison between the exact, baseline, and HJNN models: (a) momentum trajectories; (b) energy versus time, showing that both the baseline and the HJNN energies deviate from the analytical ground truth.

The code is available at: `https://tinyurl.com/mpdexpjz`.

15.8 Generating-Function Networks

To resolve the multi-valued nature of p at fixed (t, q), we condition on trajectory labels given by constants of motion (such as energy or angular momentum). Instead of learning an action $S(t, q)$, the GFNN learns a *generating function*

$$(t, q) \to G_\theta(t, q; P_0) \in \mathbb{R},$$

where P_0 denotes a (possibly vector-valued) constant of motion that remains invariant along the trajectory. The canonical relations are

$$p_\theta(t, q; P_0) = \partial_q G_\theta(t, q; P_0), \qquad Q_\theta(t, q; P_0) = \partial_{P_0} G_\theta(t, q; P_0), \tag{15.8.3}$$

so that p becomes single-valued when conditioned on P_0:

$$(t, q, P_0) \longmapsto p_\theta(t, q; P_0).$$

To see how the multi-valued momentum p becomes single-valued when we condition on a conserved quantity, we consider the case of a natural Hamiltonian

$$H(q, p) = \frac{1}{2}p^2 + U(q), \qquad q, p \in \mathbb{R},$$

with associated Hamilton–Jacobi equation

$$\frac{\partial S}{\partial t} + \frac{1}{2}\Big(\frac{\partial S}{\partial q}\Big)^2 + U(q) = 0.$$

Seek a separated solution $S(t, q) = A(t) + B(q)$. Substituting and introducing a separation constant E (the energy) yields

$$A'(t) = -E, \qquad \big(B'(q)\big)^2 + 2U(q) = 2E.$$

Integrating gives the one-parameter family of actions

$$S(t, q) = -Et \ \pm \int^q \sqrt{2E - 2U(x)}\, dx + S_0,$$

where $S_0 = S(0, 0)$ and the integrand is real for $E \geq \min U$. Consequently, the momentum

$$p = \partial_q S = \pm\sqrt{2E - 2U(q)}$$

is generally *multi-valued* at fixed (t, q): different energy levels (and branches) yield different p.

Conditioning on the conserved energy E collapses this ambiguity to a single-valued map $(t, q, E) \mapsto p(t, q; E)$. In practice, one also selects a branch (e.g., by imposing a sign convention or a soft constraint such as $p > 0$) to obtain a unique expression

$$p(t, q; E) = \sqrt{2E - 2U(q)} \quad \text{(chosen branch)}.$$

Thus, augmenting the input with the constant of motion (here, E) restores well-posedness for learning p.

Supervision and physics residual Given samples $\{(t_k, q_k, p_k, P_{0,k})\}_{k=1}^N$ from a Hamiltonian system with Hamiltonian $H(q,p)$, we train G_θ by minimizing

$$\mathcal{L}_{\mathrm{GFNN}}(\theta) = \underbrace{\frac{1}{N}\sum_{k=1}^{N}\Big(\partial_t G_\theta(t_k, q_k; P_{0,k}) + H\big(q_k, \partial_q G_\theta(t_k, q_k; P_{0,k})\big)\Big)^2}_{\text{Hamilton–Jacobi residual}} \tag{15.8.4}$$

$$+ \underbrace{\frac{1}{N}\sum_{k=1}^{N}\big\|\partial_q G_\theta(t_k, q_k; P_{0,k}) - p_k\big\|^2}_{\text{momentum matching}} \tag{15.8.5}$$

$$+ \underbrace{\frac{1}{N}\sum_{k=1}^{N}\big\|G_\theta(0, q_{0,k}; P_{0,k}) - S_0\big\|^2}_{\text{per-trajectory anchor}},$$

with $S_0 = 0$ without loss of generality. The first term enforces the Hamilton–Jacobi equation $\partial_t G + H\big(q, \partial_q G\big) = 0$, the second aligns $\partial_q G_\theta$ with measured momenta, and the third fixes the additive constant for each trajectory using its own initial data $(t = 0, q_{0,k}, P_{0,k})$.

Unknown invariants For instance, in the case of the harmonic oscillator, the energy is known, and it can be evaluated from the initial conditions of the kth trajectory as

$$P_{0,k} = \frac{1}{2}(p_{0,k}^2 + q_{0,k}^2).$$

However, in some situations P_0 might be a nontrivial integral of motion, case in which we learn a low-dimensional code $P_{0,\phi}$ with an encoder network $\mathcal{E}_\phi$ from initial conditions as

$$P_{0,\phi} = \mathcal{E}_\phi\big(q_0, p_0\big),$$

and regularize it to be constant along the trajectory:

$$\mathcal{L}_{\mathrm{const}}(\phi) = \frac{1}{N}\sum_{k=1}^{N}\frac{1}{M_k}\sum_{i=1}^{M_k}\big\|P_{0,\phi} - \mathcal{E}_\phi(q_{k,i}, p_{k,i})\big\|^2, \tag{15.8.6}$$

so the total loss is

$$\mathcal{L}_{\mathrm{total}}(\theta, \phi) = \mathcal{L}_{\mathrm{GFNN}}(\theta) + \lambda_{\mathrm{const}}\,\mathcal{L}_{\mathrm{const}}(\phi).$$

Multiple degrees of freedom For d degrees of freedom, we use $q, p \in \mathbb{R}^d$ and a vector of invariants $P_0 \in \mathbb{R}^r$. The generating function $G_\theta(t, q; P_0)$ returns $p = \nabla_q G_\theta$ and $Q = \nabla_{P_0} G_\theta$ by (15.8.3), and the Hamilton–Jacobi residual in (15.8.4) becomes

$$\partial_t G_\theta(t, q; P_0) + H\big(q, \nabla_q G_\theta(t, q; P_0)\big) = 0.$$

Remark 15.8.1 (i) Conditioning on P_0 resolves the ill-posedness of $(t, q) \mapsto p$ and restores a well-defined mapping $(t, q, P_0) \mapsto p$.

(ii) The per-trajectory anchor avoids a global gauge issue for G_θ.

(iii) In practice, we augment the HJ residual with collocation points off the data manifold and use smooth activations to ensure stable derivatives of G_θ.

Implementation example We shall use the same example as before — the harmonic oscillator. Under the same data collection as in the previous section, and the same architecture where we used `Swish` function instead of `Softplus` and performed training for only $1,000$ epochs with Adam with lr=0.01 and batch size of 81. We obtain the results in Fig. 15.12. The GFNN performs better than the baseline, and it is also an improvement over the HJNN of the last section.

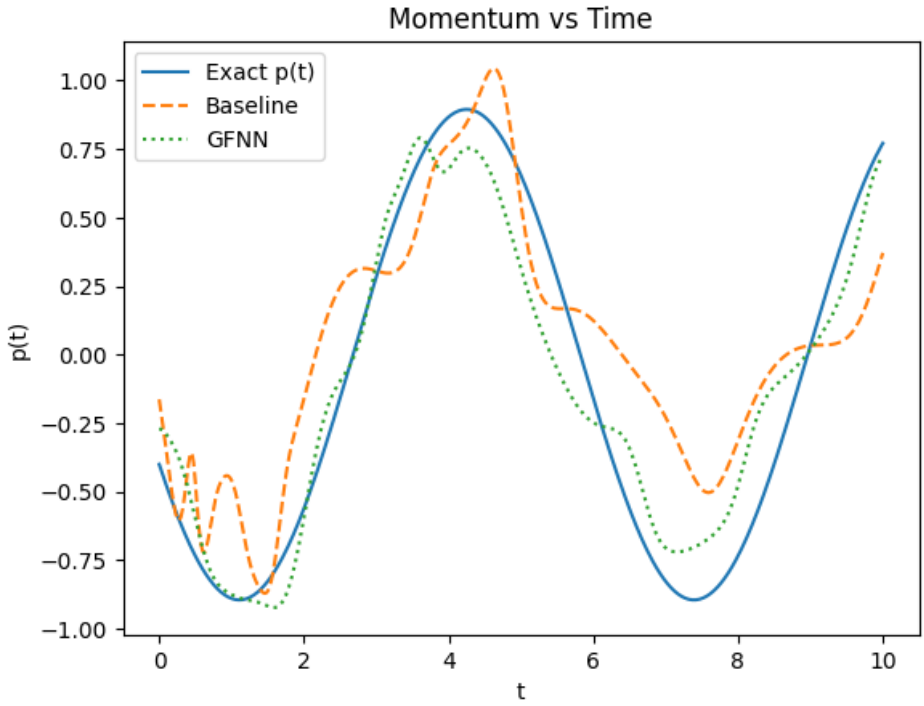


Figure 15.12: Comparison between the momentum trajectories for the exact, baseline, and GFNN models.

Implementation example We revisit the harmonic oscillator under the same data collection protocol as in the previous section. Using the same feed-forward architecture, we replace the activation with `Swish` (instead of `Softplus`) and train for 1,000 epochs with Adam (learning rate 10^{-2}) and a batch size of 81. The results are shown in Fig. 15.12. The GFNN outperforms the baseline and also improves upon the HJNN from the previous section.

The code is available at: `https://tinyurl.com/5ajznjx4`.

15.9 Attention Mechanism along Trajectories

Attention began as a way to let models focus selectively on the most relevant parts of their input. It was first explored in both vision and sequence-to-sequence NLP. Its modern form, self-attention transformed the field by replacing recurrence with parallel interactions, enabling models to capture long-range dependencies efficiently. This shift powered the Transformer architecture, see Vaswani et al. [95], and, in turn, landmark systems such as BERT (for deep bidirectional language understanding), see Devlin et al. [31] and the GPT family, see Radford et al. [78] and Brown et al. [13]. Today, attention is less a single technique than a design principle–one that unifies perception across text, images, audio, and beyond.

In this section, we design an attention mechanism to identify "hot spots"–the most informative points–along a trajectory. Consider a curve $x : [0, T] \to \mathbb{R}^d$ with initial state $x_0 = x(0)$. Let $x_j = x(t_j)$, $1 \le j \le N$, be sampled observations with $0 < t_j < T$. Fix a reference point x_i. Our goal is to quantify how the rest of the trajectory $\{x_j\}_{j \ne i}$ relates to x_i. We do so by assigning a probability distribution over indices j that measures the relevance of each x_j to x_i; this distribution constitutes the attention at x_i from the perspective of x_0.

Scores We score the relationship between two samples x_i and x_j using cosine similarity computed relative to the reference x_0:

$$\text{score}(x_i, x_j) = \frac{\langle x_j - x_0,\ x_i - x_0 \rangle}{\|x_i - x_0\|\,\|x_j - x_0\|} = \cos\varphi_{ij} \in [-1, 1],$$

where φ_{ij} is the angle between the vectors $x_j - x_0$ and $x_i - x_0$, see Fig. 15.13. For $i = j$ we take $\varphi_{ii} = 0$. Note that the choice of x_0 biases the score: changing x_0 changes all $\text{score}(x_i, x_j)$.

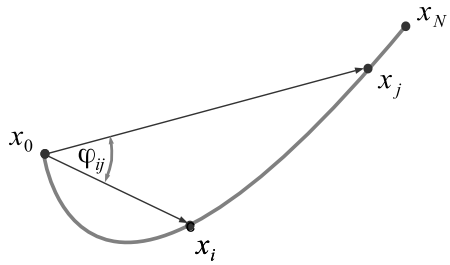


Figure 15.13: The angle φ_{ij} between the vectors $x_j - x_0$ and $x_i - x_0$ needed to construct $\text{score}(x_i, x_j)$.

Attention distribution We convert these scores into an attention distribution at x_i via a `Softmax` over $\cos\varphi_{ij}$:

$$p_{x_i}(x_j) = \frac{e^{\cos\varphi_{ij}}}{\sum_{k=1}^{N} e^{\cos\varphi_{ik}}}.$$

Since $p_{x_i}(x_j) > 0$ and $\sum_{j=1}^{N} p_{x_i}(x_j) = 1$, this defines a probability distribution over $\{x_j\}_{j=1}^{N}$. Let $a_{ij} := p_{x_i}(x_j)$. The resulting attention matrix $A = (a_{ij})$ is generally non-symmetric. To emphasize the dependence on the reference, we write $A(x_0)$ and refer to it as the attention anchored at x_0. The matrix A thus encodes a set of N probability distributions over the samples along the trajectory from the perspective of x_0.

The observer The anchor point x_0 serves as an *observer*: attention is defined with respect to its point of view. This is analogous to human vision, where perception depends on viewing geometry, where the visual system integrates stimuli over a solid angle, and cortical processing accentuates certain features, producing a sense of "surprise" or hot spots in the visual field. Similar to the visual cortex function, the ML system receives information which is processes to build a score between each region and all other regions, and if the entropy of this distribution is small, it is classified as a surprise factor or as a hot spot.

The entropy The average surprise of the observer associated with a point x_i with respect to the other points on the trajectory is quantified by the *entropy* of its attention distribution:

$$H(x_i) \;=\; -\sum_{j=1}^{N} p_{x_i}(x_j)\,\ln p_{x_i}(x_j).$$

A *hot spot* is a point x_i whose attention distribution has *low entropy*–that is, it is sharply peaked–indicating a strong, selective relationship to only a few trajectory points and hence greater "surprise" than typical locations. Because this entropy is defined relative to the observer x_0, it depends on where attention is anchored. To better capture the trajectory's geometry, we can place a *system of observers* at different anchor points and aggregate their views.

Example 15.9.1 (A linear trajectory). Suppose the trajectory $x(t)$ is a straight line and consider the observer anchored at $x_0 = x(0)$. Then every sample x_j is collinear with the initial point x_0 and any reference point x_i, so $\varphi_{ij} = 0$ for all j and hence $\cos\varphi_{ij} = 1$. The attention distribution at x_i becomes

$$p_{x_i}(x_j) = \frac{e^{1}}{\sum_{k=1}^{N} e^{1}} = \frac{1}{N},$$

i.e., the uniform distribution. Thus, from the attention perspective, x_i relates *equally* to all other points. This uniform distribution has maximal entropy $H(p_{x_i}) = \log N$ among all distributions on $\{x_j\}_{j=1}^{N}$, so no location on the line is distinguished, as every x_i yields the same uniform attention.

However, once the observer is moved off the line, the attention distribution becomes *non-uniform*. To show this, we consider the segment $\{\, y = 1 \;:\; 0 < x < 6 \,\}$ with the observer at $(4, 0)$. The entropy of the attention distribution at each point along the segment is shown in Fig. 15.14. The entropy attains its *maximum* near $(3.746,\, 1)$. It is worth noting that this point is not at the orthogonal projection $(4, 1)$ of the observer onto the line. The entropy *minimum* occurs at the right endpoint $(6, 1)$. This indicates that, from this point of view, the endpoint closest to the observer is the "hottest" spot. The code can be found at: `https://tinyurl.com/4h99zkum`.

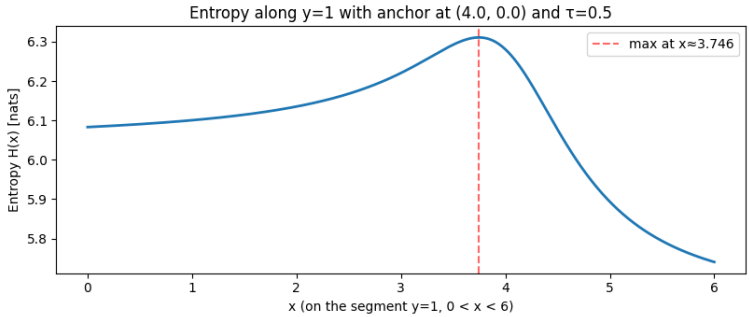


Figure 15.14: The entropy of the points on the line segment $\{\, y = 1 \;:\; 0 < x < 6 \,\}$ with the observer at $(4, 0)$.

Example 15.9.2 (A circular trajectory). We consider the unit-circle trajectory given by $x(\phi) = (\cos\phi, \sin\phi)$ for $0 \le \phi < 2\pi$. Let $\{\phi_k\}_{k=1}^N$ be sampling angles and define the N points on the circle by $x_j = e^{\mathrm{i}\phi_j}$ (equivalently, $(\cos\phi_j, \sin\phi_j)$). Fix a reference point $x_i = e^{\mathrm{i}u}$ with angle u (for readability we write u instead of ϕ_i). Anchoring the observer at the origin $x_0 = (0, 0)$, the cosine similarity between x_i and x_j is

$$\cos\varphi_{ij} \;=\; \cos(u - \phi_j).$$

The attention distribution of x_i with respect to the origin is the softmax

$$p_{x_i}(x_j) \;=\; \frac{e^{\cos(u-\phi_j)}}{Z_N(u)}, \qquad Z_N(u) \;=\; \sum_{k=1}^{N} e^{\cos(u-\phi_k)}.$$

Its entropy is

$$\begin{aligned} H(x_i) &= \ln Z_N(u) \;-\; \mathbb{E}_{p_{x_i}}\big[\cos(u-\phi)\big] \\ &= \ln Z_N(u) \;-\; \frac{1}{Z_N(u)} \sum_{j=1}^{N} \cos(u-\phi_j)\, \exp\big(\cos(u-\phi_j)\big). \end{aligned}$$

We shall compute this entropy for the case of uniform sampling $\phi_j = \frac{2\pi j}{N}$, for N large. Given that the modified Bessel function of order zero I_0 verifies

$$\int_0^{2\pi} e^{\cos z}\, dz = 2\pi I_0(1),$$

then the softmax normalizer satisfies

$$Z_N(u) = \sum_{k=1}^{N} e^{\cos(u-\phi_k)} \;\sim\; \frac{N}{2\pi}\int_0^{2\pi} e^{\cos(u-\phi)}\,d\phi = N\,I_0(1), \text{ as } N\to\infty,$$

where we used the substitution $z = \phi - u$ and the aforementioned formula. We note that this expression is independent of u.

Similarly, using that the modified Bessel function of order one I_1 satisfies

$$\int_0^{2\pi} \cos z\, e^{\cos z}\,dz = 2\pi I_1(1),$$

we have

$$\sum_{j=1}^{N} \cos(u-\phi_j)\, e^{\cos(u-\phi_j)} \sim \frac{N}{2\pi}\int_0^{2\pi} \cos(u-\phi)\, e^{\cos(u-\phi)}\,d\phi = N I_1(1).$$

Therefore, the entropy satisfies

$$\begin{aligned} H(x_i) &= \ln Z_N(u) \;-\; \mathbb{E}_{p_{x_i}}[\cos(u-\phi)] \\ &= \ln N + \ln I_0(1) - \frac{I_0(1)}{I_1(1)} \approx \ln N - 0.2103, \text{ as } N\to\infty. \end{aligned}$$

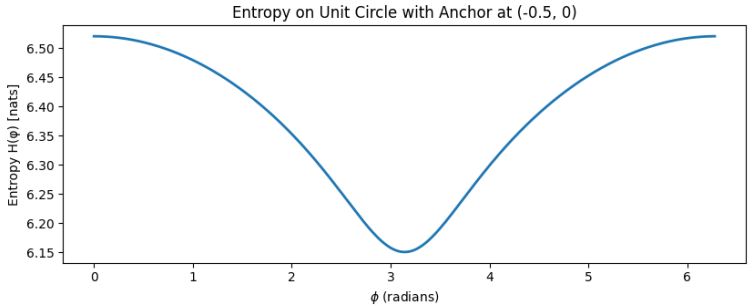
This entropy does not depend on the reference angle u, reflecting the rotational symmetry. Hence, when the observer is at the center of the circle, all points share the same entropy, and there are no hot spots on the unit circle, as expected. Hence, symmetry with respect to the observer makes all points equally informative.

When the observer is off-center, the entropy becomes *non-uniform*. For an observer at $(-0.5, 0)$, the entropy profile achieves its *minimum* at $\phi = \pi$ (the closest point $(-1, 0)$), see Fig. 15.15(a). Figure 15.15(b) presents a heat map where "hot" denotes low entropy: the hottest region concentrates near $(-1, 0)$, whereas the coldest (highest-entropy) region lies farthest from the observer, near $(1, 0)$.

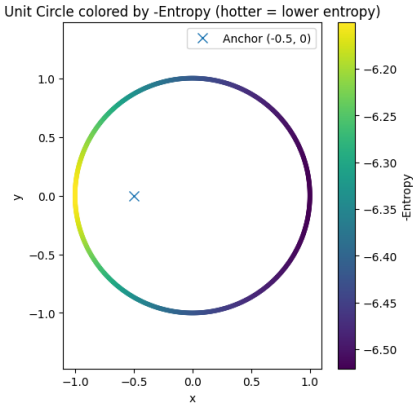
Temperature τ We introduce a temperature parameter $\tau > 0$ by scaling the scores before the `Softmax`:

$$p_{x_i}(x_j;\tau) \;=\; \frac{e^{\text{score}(x_i,x_j)/\tau}}{\sum_{k=1}^{N} e^{\text{score}(x_i,x_k)/\tau}}.$$

This mirrors the Gibbs-Boltzmann distribution with "energy" $E_j = -\text{score}(x_i, x_j)$ and inverse temperature $\beta = 1/\tau$. Smaller τ produces *sharper* distributions that concentrate on the largest scores (lower entropy H), while larger τ yields *smoother*, more diffuse distributions (higher H); in the limits, $\tau \to 0$ selects the argmax and $\tau \to \infty$ approaches the uniform distribution. In practice, a useful sweep is $\tau \in [0.2, 1]$ to trade off selectivity and robustness.



(a)



(b)

Figure 15.15: (a) The entropy profile on the unit circle with an observer at $(-0.5, 0)$; (b) The heat map of the entropy on the unit circle. The yellow corresponds to hot spots (low entropy); The cross represents the location of the observer.

Query and key When trajectories lie in a high-dimensional space $\mathbb{R}^d$, it is convenient to project them into a lower-dimensional space while retaining as much information as possible. We do this via learned linear maps:

$$q_i = W_Q\,(x_i - x_0), \qquad k_j = W_K\,(x_j - x_0),$$

where $W_Q, W_K \in \mathbb{R}^{d\times c}$ with $c \ll d$. Thus $q_i, k_j \in \mathbb{R}^c$. Given a similarity score $\text{score}(q_i, k_j)$ (e.g., cosine similarity or a scaled dot product), we define the attention distribution at x_i by

$$p_{x_i}(x_j) = \frac{e^{\text{score}(q_i,k_j)}}{\sum_{r=1}^{N} e^{\text{score}(q_i,k_r)}}.$$

The vectors q_i and k_j are referred to as the *query* and the *keys*, respectively.

Attention layer Let $A(x_0) \in \mathbb{R}^{N\times N}$ be the attention matrix anchored at x_0. Stack the N trajectory samples as a matrix $X \in \mathbb{R}^{N\times d}$ whose j-th row is $x_j^\top$. The (single-observer) *attention map* is the linear transform

$$Y \;=\; A(x_0)\,X, \qquad \text{i.e.,} \quad y_k \;=\; \sum_{j=1}^{N} a_{kj}\,x_j,$$

where $Y \in \mathbb{R}^{N\times d}$ and $y_k^\top$ is the k-th row.

An *attention layer* aggregates multiple observers $\Omega = \{\omega_m\}_{m=1}^M$, each inducing its own attention matrix $A(\omega_m)$. Its outputs are

$$Y_m \;=\; A(\omega_m)\,X, \qquad m = 1, \ldots, M.$$

Collecting these provides a tensor $(Y_m)_{m=1}^M \in \mathbb{R}^{M\times N\times d}$, which can be used as input to a feed-forward neural network for further analysis. For instances, if F_θ is a classifier network, we can use the compound network $F_\theta(AX)$ to classify trajectories into classes, given a sample of the trajectory, X. For an example, see Exercise 15.14.3.

Curvature and hot spots What is the link between trajectory curvature and the low-entropy hot spots defined above? They may coincide, but they quantify different phenomena. *Curvature* is purely geometric, local, and observer-invariant; *attention hotness* (low entropy) is information-theoretic, observer-dependent, and global. They often align at cusps or corners, where curvature is very large (even infinite) because, from most anchors, the similarity field becomes highly selective and the entropy drops, producing a hot spot.

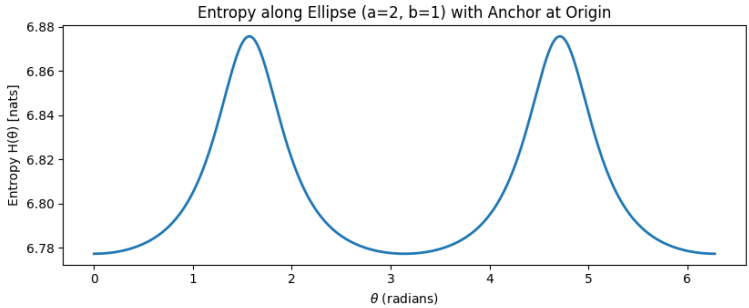
Given this complementarity, it is natural to combine both signals: use curvature to capture intrinsic geometry and entropy to capture observer-relative saliency. A simple hybrid scorer is

$$S(x_i) = \alpha\,\kappa(x_i) \;-\; \beta\,H(x_i),$$

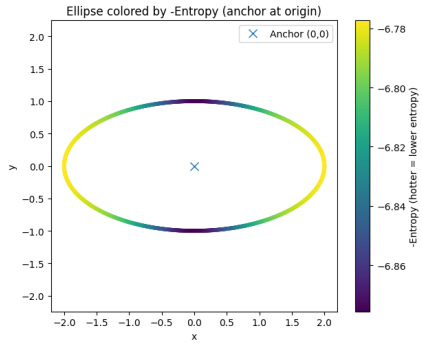
with weights $\alpha, \beta \geq 0$, which ranks by *high* κ and *low* H.

Examples of trajectories and their hot spots We now illustrate several trajectories and highlight their hot spots using the entropy-based attention with selected observers. As a guiding principle, when a trajectory exhibits symmetry, we anchor the observer at a symmetry point; when it has a symmetry axis, we place the observer on that axis. The examples below–drawn from gravitational dynamics and classical mechanical curves–use well-known planar paths to demonstrate how geometry and observer choice shape the saliency (low-entropy) landscape. The code for the following examples can be found at: `https://tinyurl.com/4h99zkum`.

Ellipse. Consider the ellipse $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$ with $a = 2$, $b = 1$, and the observer anchored at the origin $(0, 0)$. Here the entropy is *lowest* at the high-curvature vertices $(\pm 2, 0)$ and *highest* at the low-curvature vertices $(0, \pm b)$, see Fig. 15.16. Thus, in this symmetric setting, geometric saliency (curvature) and informational saliency (low entropy) align.



(a)



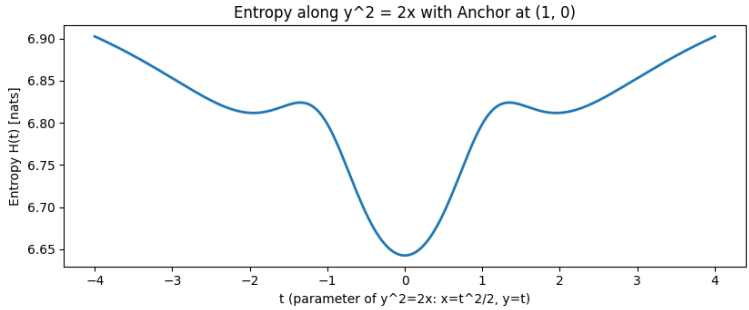
(b)

Figure 15.16: (a) The entropy profile on the ellipse $x^2/4 + y^2 = 1$ with an observer at the origin $(0, 0)$; (b) The heat map of the entropy on the ellipse.

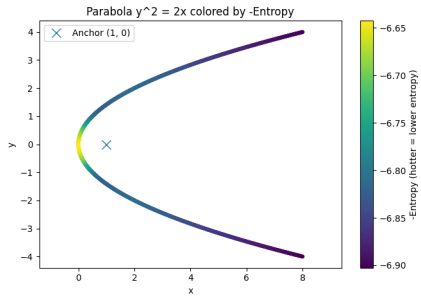
Parabola. Consider the parabola $y^2 = 2x$ with the observer at $(0, 1)$. The entropy profile in Fig. 15.17(a) shows a global minimum at the vertex ($x = 0$), along with a pair of symmetric local minima about the y-axis. The corresponding heat map in Fig. 15.17(b) highlights the vertex as the dominant hot spot-consistent with its maximal curvature among points on the curve.

Astroid. The astroid is a symmetric, closed curve with four cusps, given by the parametric equations $x = \cos^3 t$, $y = \sin^3 t$ for $0 \leq t \leq 2\pi$. With the observer fixed at the center $(0, 0)$, the entropy attains four minima, located at the cuspidal points where the geometry changes most sharply; see Fig. 15.18.

Lemniscate. Consider the Bernoulli lemniscate, e.g., $(x^2 + y^2)^2 = 2a^2(x^2 - y^2)$ for $a > 0$, a figure-eight curve with a self-intersection at the origin. With the observer anchored at $(0, 0)$, the entropy is highest near the tips of the two lobes (the points farthest from the origin) and lowest in a neighborhood of the crossing, where directions change rapidly. Thus the entropy heat map highlights the lobe extrema as cold regions and the self-intersection region as a hot spot area; see Fig. 15.19.



(a)



(b)

Figure 15.17: (a) The entropy profile on the parabola $y^2 = 2x$ with an observer at $(1, 0)$; (b) The heat map of the entropy on the parabola.

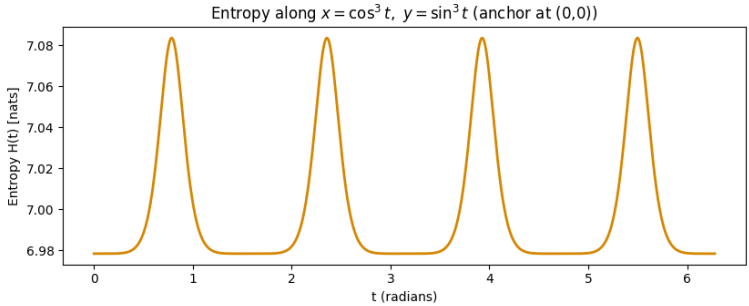
Several observers Sometimes a *family of observers* placed around the trajectory will perceive different hot-spot regions. Aggregating their views yields a consensus: take the *average entropy*

$$\overline{H}(x_i) \;=\; \frac{1}{M}\sum_{m=1}^{M} H^{(m)}(x_i),$$

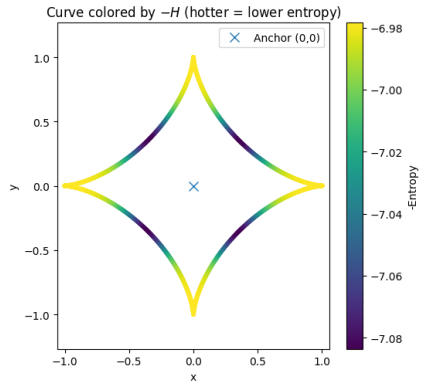
(or a weighted average if certain observers are more reliable), where $H^{(m)}$ denotes the entropy corresponding to the observer ω_m. The minima of $\overline{H}$ identify the *committee's* agreed-upon hot spots-locations that remain salient across viewpoints rather than from a single vantage.

We shall exemplify this in the case of a Bernoulli lemniscate across 12 observers placed on a circle of radius 2 centered at the origin. The result of the average entropy in this case is very different than the result obtained with only one observer, see Fig. 15.20.

The optimal observer Suppose we have a set of observers $\Omega = \{\omega_1, \ldots, \omega_m\}$. For each point x_i on a trajectory and each observer ω, the attention mechanism yields a



(a)



(b)

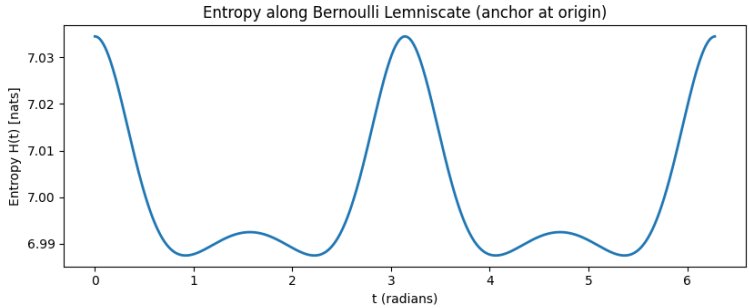
Figure 15.18: (a) The entropy profile on the astroid with an observer at $(0, 0)$; (b) The heat map of the entropy on the astroid.

probability distribution over the sampled points,

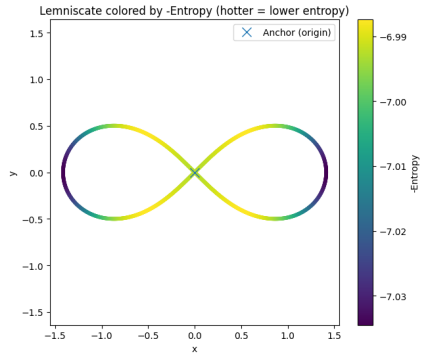
$$p_{x_i}^{(\omega)}(x_j), \qquad j = 1, \ldots, N, \quad \sum_{j=1}^{N} p_{x_i}^{(\omega)}(x_j) = 1.$$

To compare two observers ω_a (taken as reference) and ω_b at the same point x_i, we measure the *inefficiency* of using ω_b in place of ω_a via the Kullback–Leibler (KL) divergence

$$D_{x_i}(\omega_a \,\|\, \omega_b) \;=\; D_{\mathrm{KL}}\big(p_{x_i}^{(\omega_a)} \,\|\, p_{x_i}^{(\omega_b)}\big) \;=\; \sum_{j=1}^{N} p_{x_i}^{(\omega_a)}(x_j)\, \ln \frac{p_{x_i}^{(\omega_a)}(x_j)}{p_{x_i}^{(\omega_b)}(x_j)} \;\geq 0,$$



(a)



(b)

Figure 15.19: (a) The entropy profile on the lemniscate with an observer at $(0, 0)$; (b) The heat map of the entropy on the lemniscate.

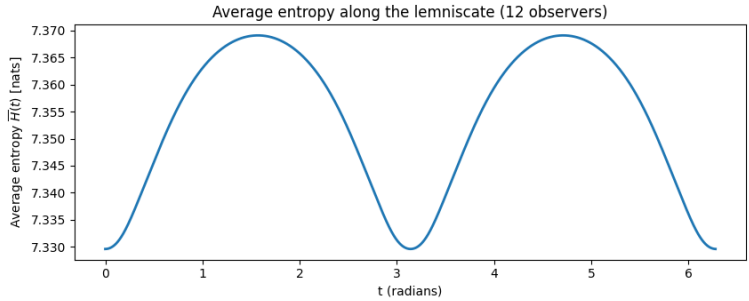
with equality iff the two distributions coincide at x_i. Because this comparison is pointwise, we aggregate over the trajectory to obtain an average dissimilarity

$$\overline{D}(\omega_a \,\|\, \omega_b) \;=\; \frac{1}{N}\sum_{i=1}^{N} D_{x_i}(\omega_a \,\|\, \omega_b).$$

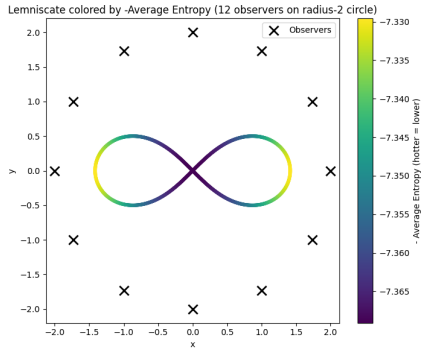
If a single "best" observer is desired and no observer is privileged a priori, a natural choice is the *information method*: the observer that is, on average, closest (in KL) to all the others, ω_k, with

$$k \;=\; \arg\min_{s\in\{1,\dots,m\}} \sum_{\substack{t=1\\ t\neq s}}^{m} \overline{D}(\omega_s \,\|\, \omega_t).$$

This selects the observer whose attention distributions are most representative of the committee.



(a)



(b)

Figure 15.20: (a) The entropy profile on the lemniscate with 12 observers situated on a circle of radius 2, centered at $(0, 0)$; (b) The heat map of the aggregated entropy from 12 observers.

Remark 15.9.1 If symmetry is preferred, we can replace KL with a symmetric divergence, e.g., Jensen–Shannon divergence, or minimize $\overline{D}(\omega_s \,\|\, \omega_t) + \overline{D}(\omega_t \,\|\, \omega_s)$.

15.10 Neural ODEs with Attention

We model the velocities of N particles with positions $x_i(t) \in \mathbb{R}^d$ by a first-order system

$$\dot{x}_i(t) \;=\; F_i\big(x_1(t), \ldots, x_N(t)\big), \qquad i = 1, \ldots, N.$$

When particles are indistinguishable (same mass and interaction law) in a homogeneous environment, it is natural to use a single, shared dynamics function. Moreover, the velocity of particle i should depend on its *local* state $x_i(t)$ and on a *nonlocal* context aggregating the states of the other particles. We realize this context with an attention

mechanism and write a Neural ODE

$$\dot{x}_i(t) \;=\; f_\theta\Big(x_i(t),\,(AX)_i(t)\Big), \qquad i=1,\dots,N,$$

where f_θ is a neural network shared across i, $X(t)=\big[x_1(t),\dots,x_N(t)\big]^\top$, and $(AX)_i(t)$ denotes the attention-weighted aggregation for particle i.

Attention construction Given queries and keys

$$q_i(t)=W_Q\,x_i(t), \qquad k_j(t)=W_K\,x_j(t),$$

with weights matrices $W_Q, W_K \in \mathbb{R}^{d\times c}$, where c is the attention channel dimension, the attention weights are

$$a_{ij}(t) \;=\; \frac{e^{\,q_i(t)^\top k_j(t)/(\sqrt{c}\tau)}}{\displaystyle\sum_{r=1}^{N} e^{\,q_i(t)^\top k_r(t)/(\sqrt{c}\tau)}},$$

with temperature $\tau>0$. Using values $v_j(t)$ (commonly $v_j(t)=W_V x_j(t)$ or simply $x_j(t)$), the contextual average is

$$(AX)_i(t) \;=\; \sum_{j=1}^{N} a_{ij}(t)\,v_j(t).$$

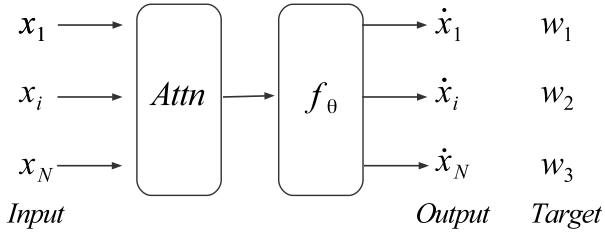


Figure 15.21: The input-output mapping is trained by supervised learning.

Training Suppose we observe positions and velocities at times $0<t_1<\cdots<t_n$,

$$x_i^j := x_i(t_j), \qquad w_i^j := \dot{x}_i(t_j).$$

A supervised derivative-matching loss fits both f_θ and the attention parameters, see Fig. 15.21

$$\mathcal{L}(\theta,W_Q,W_K) \;=\; \sum_{j=1}^{n}\sum_{i=1}^{N}\Big(f_\theta\big(x_i^j,\,(AX)_i(t_j)\big) \;-\; w_i^j\Big)^2.$$

Example The following system models diffusive coupling on a network together with a uniform relaxation toward a reference, encompassing many physical settings (heat flow with sinks, overdamped coupled elements, resistor networks, etc.):

$$\dot{x}_i(t) \;=\; -k\,x_i(t) \;+\; \sum_{j=1}^{N} b_{ij}\,\big(x_i(t) - x_j(t)\big), \qquad i = 1,\dots,N, \quad t \in [0,T].$$

Synthetic data collection. Set $N = 10$, $k = 1$, $T = 2$, and choose symmetric couplings

$$b_{ij} \;=\; \begin{cases} 1, & i \neq j, \\ -(N-1), & i = j, \end{cases} \qquad i,j = 1,\dots,N.$$

Partition $[0,T]$ into $n = 20$ steps with $t_j = j\Delta t$, $\Delta t = T/n$. For each i and t_j, record the state $x_i(t_j)$ and its velocity $\dot{x}_i(t_j)$ obtained via a high-order numerical solver. The resulting pairs $\big\{(x_i(t_j), \dot{x}_i(t_j))\big\}_{i=1,\dots,N}^{j=0,\dots,n}$ constitute the training dataset.

Implementation. The vector field f_θ is a feed-forward network with two hidden layers (128 neurons each, `tanh` activations). The attention block uses query and key projections $W_Q, W_K \in \mathbb{R}^{N\times c}$ with $N = 5$ particles and attention width $c = 8$, and a temperature hyperparameter $\tau = 0.85$. Training minimizes the mean-squared error with Adam (learning rate 10^{-3}) for $3,600$ epochs. For validation we integrate the given system with a high-order Runge–Kutta (RK4) solver and compare against the Neural ODE rollout.

Figure 15.22(a) shows the RK4 trajectories and the learned trajectories for the first three particles, exhibiting close agreement. Figure 15.22(b) displays the exact full heat map dynamics for all five particles, while Fig. 15.22(c) represents the learned neural-ODE heat map of the same particles. We note that the heat map shows strong divergence by $t \approx 1$, since some particle trajectories become very positive (bright band), while others negative (dark band). Since they are visually indistinguishable, it means that the learned dynamics matches the ground-truth dynamics.
The code is available at: `https://tinyurl.com/42xzp9vf`.

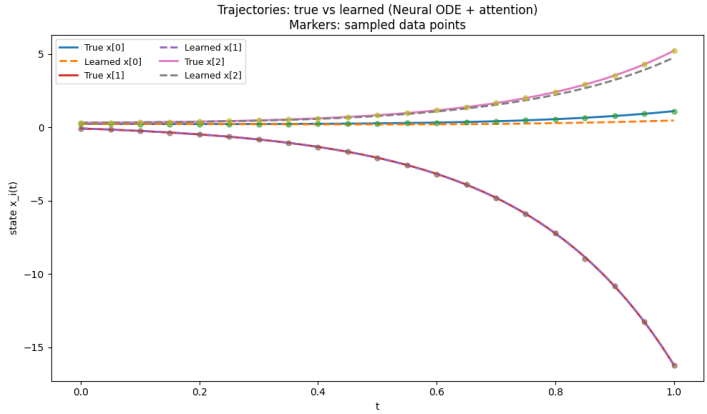
15.11 Attention and Hamiltonian Neural Networks

We enhance the HNN framework of Section 15.2 by incorporating an attention mechanism. To avoid overloading the symbol q (used for *queries* in attention), we denote particle positions by x in this section, so the Hamiltonian is written as $H(x,p)$.

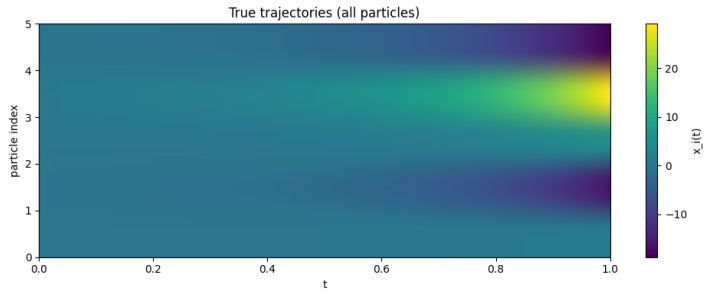
Consider N particles in $\mathbb{R}^d$ with masses m_i and states $z_i(t) = (x_i(t), p_i(t))$, $i = 1,\dots,N$. We assume m measurement times $0 < t_1 < \cdots < t_m$, yielding the training set $\{(z_i(t_k),\, z_i'(t_k))\}$ with $z_i'(t) = (x_i'(t), p_i'(t))$. We seek to learn a Hamiltonian of the form

$$H_\theta(x,p) \;=\; \underbrace{\sum_{i=1}^{N} \frac{\|p_i\|^2}{2m_i}}_{\text{kinetic}} \;+\; \underbrace{V_\theta(x_1,\dots,x_N)}_{\text{potential}},$$

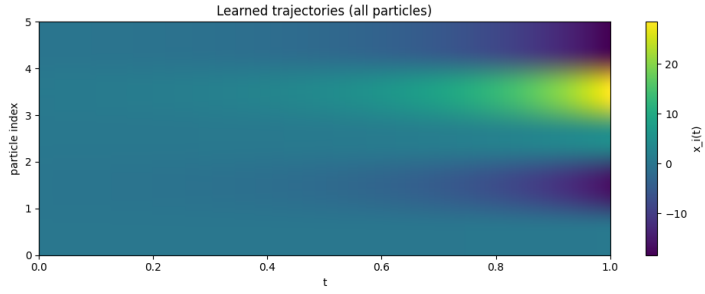
so the learning task reduces to identifying an appropriate potential V_θ.



(a)



(b)



(c)

Figure 15.22: (a) The RK4 trajectories and the learned trajectories for the first three particles are in close agreement; (b) The exact heat map dynamics for all five particles; (c) The learned Neural-ODE heat map dynamics for all five particles.

Because the particles interact nonlocally, we use attention to discover *which* pairs interact and *with what strength*. We parameterize the potential as an attention-weighted sum of pairwise terms:

$$V_\theta(x_1,\dots,x_N) \;=\; \sum_{i<j} \alpha_{ij}(x)\, U_\theta\big(\,\|x_i - x_j\|\,\big), \tag{15.11.7}$$

where $\alpha_{ij}(x) \in [0,1]$ are attention weights (optionally constrained so that $\alpha_{ij} = \alpha_{ji}$) and $U_\theta(\cdot)$ is a small neural network that models the pairwise potential as a function of distance. In this construction, all particle pairs share the same learned radial potential U_θ, while the configuration-dependent weights $\alpha_{ij}(x)$ modulate the relative influence of each pair.

Attention block. The weights are defined by a scaled dot-product attention as

$$\alpha_{ij} \;=\; \mathrm{S}oftmax_j\Big(\frac{q_i^\top k_j}{\sqrt{c}\,\tau}\Big) \;=\; \frac{\exp\Big(\frac{q_i^\top k_j}{\sqrt{c}\,\tau}\Big)}{\sum_r \exp\Big(\frac{q_i^\top k_r}{\sqrt{c}\,\tau}\Big)},$$

where c is the channel dimension and $\tau > 0$ is the temperature hyperparameter (taken in between 0 and 1).

Since physical interactions are typically symmetric (each pair influences one another equally), it is natural to enforce $\alpha_{ij} = \alpha_{ji}$. A simple symmetrization is

$$\alpha_{ij} \;:=\; \tfrac{1}{2}\big(\alpha_{ij} + \alpha_{ji}\big),$$

or, at the score level, use the same embedding for queries and keys so that $q_i = k_i$, which makes the similarity kernel symmetric by construction.

The queries and keys are produced by a shared feature map ψ_θ followed by linear projections:

$$q_i \;=\; W_Q\,\psi_\theta(x_i), \qquad k_j \;=\; W_K\,\psi_\theta(x_j),$$

where ψ_θ is a neural network and $W_Q, W_K \in \mathbb{R}^{d\times c}$ are learnable projection matrices (with c the attention width).

Training. We learn the parameters by minimizing a total loss

$$\mathcal{L}_{\mathrm{tot}} \;=\; \mathcal{L}_{\mathrm{Ham}} \;+\; \lambda\,\mathcal{L}_{\mathrm{reg}} \;+\; \lambda'\,\mathcal{L}_{\mathrm{var}} \quad \text{(optional)},$$

where the first term enforces Hamilton's equations $\dot z = \nabla^J H(z)$, with $\nabla^J := J\nabla$ and $J = \begin{bmatrix} 0 & I \\ -I & 0 \end{bmatrix}$ the symplectic matrix and is given by

$$\mathcal{L}_{\mathrm{Ham}}(\theta, c, \tau, W_Q, W_K) \;=\; \frac{1}{mN}\sum_{k=1}^{m}\sum_{i=1}^{N} \big\|z_i'(t_k) \;-\; \nabla^J H_\theta\big(z_i(t_k)\big)\big\|^2.$$

The second term is the energy-drift regularization term, which penalizes non-conservation of the learned Hamiltonian along short steps:

$$\mathcal{L}_{\mathrm{reg}}(\theta) \;=\; \frac{1}{m-1}\sum_{k=1}^{m-1} \Big(H_\theta\big(z(t_{k+1})\big) \;-\; H_\theta\big(z(t_k)\big)\Big)^2.$$

The third term is optional and deals with the energy-variance regularization. This is a stronger constraint that encourages constant energy over a window:

$$\mathcal{L}_{\mathrm{var}}(\theta) \;=\; \frac{1}{m}\sum_{k=1}^{m}\Big(H_\theta\big(z(t_k)\big) \;-\; \overline{H}\Big)^2, \qquad \overline{H} \;=\; \frac{1}{m}\sum_{k=1}^{m} H_\theta\big(z(t_k)\big).$$

Here $\lambda, \lambda' > 0$ are hyperparameters, c is the attention width, τ the temperature, and W_Q, W_K are the query/key projection matrices.

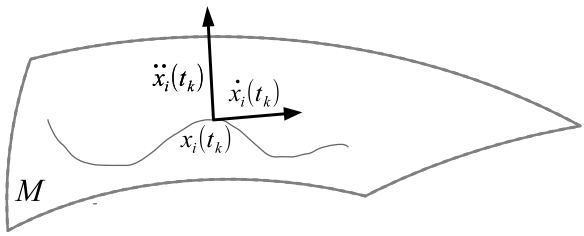


Figure 15.23: The measurements $(x_i(t_k), \dot{x}_i(t_k), \ddot{x}_i(t_k))$ at time t_k along the ith trajectory.

15.12 Attention and Lagrangians

We consider N particles constrained to move on a smooth curved surface $\mathcal{M} \subset \mathbb{R}^{d+1}$ under a nonlocal potential that depends on inter-particle separation. As a motivating example, take N identical negatively charged particles constrained to the unit sphere in $\mathbb{R}^3$, repelling via a Coulomb-type interaction. For $N = 2$, in the presence of small friction, the particles relax to a pair of antipodal points; for large N, the dynamics becomes highly nontrivial.

Problem setup We are given m observation times $0 < t_1 < \cdots < t_m$ and, for each particle $i = 1, \dots, N$, measurements of position, velocity, and acceleration

$$\big(x_i(t_k),\, \dot{x}_i(t_k),\, \ddot{x}_i(t_k)\big)$$

on $\mathcal{M}$, see Fig. 15.23. Our goal is to learn a Lagrangian whose Euler–Lagrange equations reproduce the observed dynamics.

We consider the notations $\mathbf{x} = (x_1, \dots, x_N)^\top$, $\dot{\mathbf{x}} = (\dot{x}_1, \dots, \dot{x}_N)^\top$ and adopt the classical form of the Lagrangian

$$L_\theta(\mathbf{x}, \dot{\mathbf{x}}) \;=\; \underbrace{\tfrac{1}{2}\,\dot{\mathbf{x}}^\top G_\theta(\mathbf{x})\,\dot{\mathbf{x}}}_{\text{kinetic}} \;-\; \underbrace{V_\theta(\mathbf{x})}_{\text{potential}},$$

where $G_\theta(\mathbf{x}) \in \mathbb{R}^{d\times d}$ is a learned Riemannian metric on the space $\mathcal{M}$ and V_θ is a learned nonlocal potential.

Metric parameterization. To ensure symmetry and positive-definiteness we hard-embed

$$G_\theta(\mathbf{x}) \;=\; M_\theta(\mathbf{x})\, M_\theta(\mathbf{x})^\top,$$

where M_θ is produced by a neural network. In the case of a flat space the kinetic term takes the form

$$K = \frac{1}{2}\sum_{i=1}^{N} m_i x_i^2$$

fact that implies that the metric is diagonal and independent of $\mathbf{x}$, given by $G_\theta = \mathrm{D}iag(m_1, \cdots, m_N)$, where m_i is the mass of the ith particle. If the masses are not provided, we can learn them via the neural network G_θ from the given data.

Attention-weighted potential. We model nonlocal interactions via the attention

$$V_\theta(\mathbf{x}) \;=\; \sum_{i<j} \alpha_{ij}(\mathbf{x})\; U_\theta\big(\,\mathrm{dist}_\theta(x_i, x_j)\,\big),$$

where $\mathrm{dist}_\theta(x_i, x_j)$ may represent either the geodesic distance from x_i to x_j measured along $\mathcal{M}$ with respect to the Riemannian metric G_θ, or more simple, if the particle interaction is applied in a straight direction, the Euclidean radial distances, $\mathrm{dist}(x_i, x_j) = \|x_i - x_j\|$.

Here, $U_\theta : \mathbb{R}_+ \to \mathbb{R}$ is a shared pairwise network of the inter-particle separation (geodesic or chordal), and $\alpha_{ij}(\mathbf{x}) \in [0, 1]$ are attention weights computed from queries/keys built on invariant features (e.g., relative directions and distances). We enforce symmetry by construction using $q_i = k_i$.

Training. We learn θ and attention parameters by minimizing a composite loss that blends data fidelity with physics residuals and a regularization term involving attention entropy:

$$\mathcal{L}_{\mathrm{tot}} \;=\; \lambda_{\mathrm{EL}}\,\mathcal{L}_{\mathrm{EL}} \;+\; \lambda_{\mathrm{ent}}\,\mathcal{L}_{\mathrm{ent}}.$$

(i) The Euler–Lagrange residual enforces the equations of motion. Let

$$\mathcal{R}_{\mathrm{EL}}(\mathbf{x}, \dot{\mathbf{x}}, \ddot{\mathbf{x}}) \;=\; \frac{d}{dt}\,\partial_{\dot{\mathbf{x}}} L_\theta(\mathbf{x}, \dot{\mathbf{x}}) \;-\; \partial_{\mathbf{x}} L_\theta(\mathbf{x}, \dot{\mathbf{x}}),$$

computed by automatic differentiation. The residual loss is

$$\mathcal{L}_{\mathrm{EL}} \;=\; \frac{1}{m}\sum_{k=1}^{m} \big\|\mathcal{R}_{\mathrm{EL}}\big(\mathbf{x}(t_k), \dot{\mathbf{x}}(t_k), \ddot{\mathbf{x}}(t_k)\big)\big\|^2.$$

(ii) Regularizer. We encourage parsimonious, stable attention via an entropy term $\mathcal{L}_{\mathrm{ent}} = \sum_i \sum_j \alpha_{ij} \log \alpha_{ij}$.

Since L and $L + c$, with c constant, generate the same Euler–Lagrange equations, in order to avoid drift, we anchor the Lagrangian by fixing a value at an initial batch of values.

To conclude, this attention-augmented Lagrangian learns both the *intrinsic geometry* (via G_θ) and the *interaction law* (via U_θ and α_{ij}). The extrinsic geometry can be found by learning the surface $\mathcal{M}$. We shall do this in the next section.

15.13 Learning the Constraint Manifold

An *admissible trajectory* $x(t)$ on $\mathcal{M}$ is the motion of a particle in the absence of external forces or interactions, i.e., it evolves solely by minimizing the action determined by the Riemannian metric on $\mathcal{M}$ (geodesic motion). Our objective is to recover the surface $\mathcal{M}$ from measurements of such admissible trajectories sampled at multiple time instances.

We model locally the constraint set $\mathcal{M} \subset \mathbb{R}^{d+1}$ as the zero level of an implicit function $\phi_\theta : \mathbb{R}^{d+1} \to \mathbb{R}$, $\mathcal{M} = \{x : \phi_\theta(x) = 0\}$. Its (unnormalized) normal is $\nabla\phi_\theta(x)$. Any admissible trajectory $t \mapsto x(t)$ must satisfy:

(*i*) *Incidence property:* $x(t) \in \mathcal{M}$ (the particle does not slip off the surface);

(ii) *Tangency of velocities:* $\dot{x}(t)^\top \nabla\phi_\theta\big(x(t)\big) = 0$;

(iii) *Geodesic motion*: we assume a vanishing geodesic curvature, namely the ambient acceleration $\ddot{x}(t)$ is normal to $\mathcal{M}$. In $\mathbb{R}^3$ this condition can be written as a vanishing cross product $\ddot{x}(t) \times \nabla\phi_\theta\big(x(t)\big) = 0$. In general dimension, for a hypersurface $\mathcal{M}$ of codimension 1, letting $n_\theta(x) = \nabla\phi_\theta(x)/\|\nabla\phi_\theta(x)\|$, we enforce the vanishing of the tangential component, see Remark 15.13.1

$$\big\|(I - n_\theta n_\theta^\top)\,\ddot{x}(t)\big\| = 0.$$

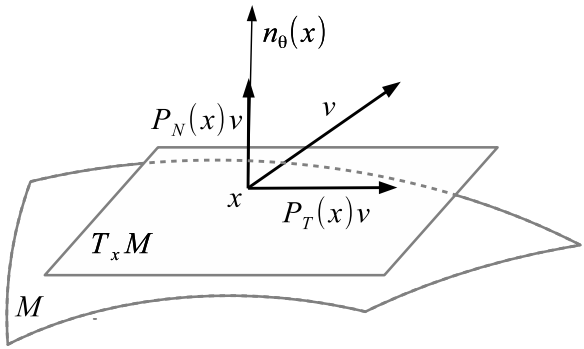


Figure 15.24: The decomposition of a vector $v \in T_x\mathcal{M}$ into a tangent and a normal component.

Training. Given samples $\{x(t_k), \dot{x}(t_k), \ddot{x}(t_k)\}_{k=1}^m$, we learn ϕ_θ by minimizing the loss

$$\mathcal{L}(\theta) = \frac{1}{m}\sum_{k=1}^{m}\Big[\underbrace{\phi_\theta\big(x(t_k)\big)^2}_{\text{on-surface}} + \underbrace{\big(\dot{x}(t_k)^\top \nabla\phi_\theta\big(x(t_k)\big)\big)^2}_{\text{velocity tangency}} + \underbrace{\big\|(I - n_\theta n_\theta^\top)\,\ddot{x}(t_k)\big\|^2}_{\text{geodesic normality}}\Big].$$

Remark 15.13.1 Let $\mathcal{M} \subset \mathbb{R}^n$ be (locally) a smooth hypersurface given by $\phi_\theta(x) = 0$, with the unit normal $n_\theta(x)$. Any vector $v \in \mathbb{R}^n$ decomposes into tangent and normal

parts, see Fig. 15.24, as

$$v \;=\; \underbrace{P_T(x)\,v}_{\text{tangent}} \;+\; \underbrace{P_N(x)\,v}_{\text{normal}},$$

where P_T and P_N are the orthogonal projectors onto the tangent space $T_x\mathcal{M}$ and the normal space $N_x\mathcal{M}$, respectively, given by

$$P_T(x) \;=\; I - n_\theta(x)\,n_\theta(x)^\top, \qquad P_N(x) \;=\; n_\theta(x)\,n_\theta(x)^\top.$$

To show this, we write

$$P_N(x)v \;=\; n_\theta(x)\,n_\theta(x)^\top v = n_\theta(x)\langle n_\theta(x), v\rangle = \langle v, n_\theta(x)\rangle n_\theta(x),$$

which is the orthogonal projection of v onto $n_\theta(x)$ along the normal direction. Since $P_T(x) + P_N(x) = I(x)$, the formula of $P_T(x)$ follows by subtraction.

For a particle constrained to $\mathcal{M}$ and moving geodesically (no external forces), the ambient acceleration $\ddot{x}(t)$ must be purely normal; equivalently, its tangential component vanishes

$$\big\|(I - n_\theta(x)\,n_\theta(x)^\top)\,\ddot{x}(t)\big\| \;=\; 0.$$

15.14 Exercises

Exercise 15.14.1 Highlight the hot spots using entropy-based attention with the selected observers at the origin $(0, 0)$ for the following curves:

(a) The hyperbola: $\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1$, with $a = 1$, $b = 2$.

(b) The cycloid: $x = t - \sin t$, $y = 1 - \cos t$, $t \in \mathbb{R}$;

(c) Descartes' folium: $x^3 + y^3 - 3xy = 0$;

(d) Single cusp: $y = |x|^{1/2}$.

Exercise 15.14.2 Redo Exercise 15.14.1 using a hybrid approach using curvature and entropy by looking for the hot spots as minima for the combination

$$S(x, y) = \alpha\kappa(x, y) - \beta H(x, y),$$

with suitable $\alpha, \beta > 0$.

Exercise 15.14.3 (Classifying conics with neural networks). A planar conic in Cartesian coordinates satisfies

$$Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0,$$

with real coefficients $(A, B, C, D, E, F) \neq 0$, defined up to a nonzero scalar multiple. Given noisy samples $\{(x_i, y_i)\}_{i=1}^N$ from an unknown conic, we shall design neural methods to classify it as *ellipse* (special case: *circle*), *parabola*, or *hyperbola*.

(a) Generate synthetic datasets for each conic type with random poses (translations, rotations) and scales. Add Gaussian noise to points. Split into train/val/test data and ensure balanced classes.

The goal is to compare two approaches:

(a) *Coefficient regression with decision layer.*

1. To predict conic coefficients $\hat{\theta} = (\hat{A}, \hat{B}, \hat{C}, \hat{D}, \hat{E}, \hat{F})$ we employ a feed-forward neural network $NN(\hat{\theta}; x, y)$ trained to minimize the algebraic residual

$$\mathcal{L}_{\text{alg}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \frac{\left(\hat{A}x_i^2 + \hat{B}x_i y_i + \hat{C}y_i^2 + \hat{D}x_i + \hat{E}y_i + \hat{F}\right)^2}{(x_i^2 + y_i^2 + 1)} + \lambda(\|\hat{\theta}\|_2 - 1)^2.$$

2. Append a *non-trainable decision layer* to classify:

$$\Delta = \hat{B}^2 - 4\hat{A}\hat{C}.$$

   If $\Delta < 0 \Rightarrow$ ellipse; if additionally $\hat{A} = \hat{C}$ and $\hat{B} = 0 \Rightarrow$ circle; if $\Delta = 0 \Rightarrow$ parabola; if $\Delta > 0 \Rightarrow$ hyperbola.

3. Report classification accuracy and also coefficient errors on a held-out test set.

(b) *Attention-based end-to-end classifier.*

1. Apply an attention layer to the sequence of training points $\{(x_i, y_i)\}$, then pass the attended representations through a small Transformer-style feed-forward encoder, followed by a classifier head that outputs class probabilities.

2. Train the whole model end-to-end with cross-entropy loss using the ground-truth conic labels.

3. Evaluate classification accuracy.

Exercise 15.14.4 (Classifying triangles with attention). A triangle in the plane is uniquely determined by the coordinates of its three vertices

$$\big(x_1, y_1\big), \quad \big(x_2, y_2\big), \quad \big(x_3, y_3\big) \in \mathbb{R}^2.$$

We aim to classify a given triangle as *acute* or *obtuse* using neural networks.

(a) For the data collection we generate a dataset of N triangles

$$\{(x_1^i, y_1^i), (x_2^i, y_2^i), (x_3^i, y_3^i)\}_{i=1}^N$$

by sampling random vertices in a bounded region (e.g. $[-1, 1]^2$). Remove degenerate or nearly collinear cases by enforcing a minimum area constraint. Label each triangle as *acute* or *obtuse* according to its internal angles:

$$\text{acute} \iff \text{all angles} < 90^\circ, \qquad \text{obtuse} \iff \text{one angle} > 90^\circ$$

using the cosine rule on side lengths.

(b) Represent each triangle as a sequence of three points

$$\big\{(x_1, y_1), (x_2, y_2), (x_3, y_3)\big\}.$$

Feed this sequence to an *attention layer* that learns interactions between the vertices (permutation-invariant design recommended). Pass the attended features through a

small feed-forward encoder (e.g. a Transformer-style block) to obtain a global embedding, and then apply a classifier layer that outputs the probability of each class.

(c) Train the model in a supervised manner using cross-entropy loss and evaluate its accuracy on a held-out test set.

(d) Which triangle configurations are most challenging to classify (e.g. near-right triangles)? How does the model handle invariances such as rotations, translations, and permutations of vertices?

Exercise 15.14.5 (Geometric and informational observers). We consider $S \subset \mathbb{R}^2$ be a bounded curve and let a *observer* be a point $z \in \mathbb{R}^2$. Consider the following generic performance criteria:

$$\Phi_p(z) = \int_S \|x - z\|^p \, d\mu(x) \quad (p \geq 1), \qquad \Phi_\infty(z) = \sup_{x \in S} \|x - z\|,$$

where μ is either the uniform *arc-length* measure on the curve S. We say a "geometric optimal observer" is any minimizer of the chosen Φ. Meanwhile, "an informational optimal observer" is a point z satisfying the definition given at page 477.

(a) Let S be a circle of radius R centered at c.
- Show that $z = c$ minimizes each of Φ_p $(p \geq 1)$ and Φ_∞.
- Show that $z = c$ is an informational optimal observer for the circle.

(b) Assume S is *centrally symmetric* with center c, i.e. $x \in S \iff 2c - x \in S$.
- Show that $z = c$ minimizes Φ_p $(p \geq 1)$ and, if S is convex, also minimizes Φ_∞.
- Is $z = c$ an informational optimal observer?

(c) For reflection-symmetric (but not centrally symmetric) shapes, discuss the set of minimizers of Φ_p. Do they always coincide with the informational optimal observer?

Exercise 15.14.6 (Heat-exchange with an attention Neural ODE). Let $x_i(t)$ denote the temperature of body i $(i = 1, \ldots, N)$ in a large room of ambient temperature T. Assume Newtonian heat exchange with the room and pairwise exchange between bodies is given by

$$C_i \, \dot{x}_i(t) \;=\; h_i\big(T - x_i(t)\big) \;+\; \sum_{j \neq i} k_{ij} \, \big(x_j(t) - x_i(t)\big),$$

with $k_{ij} = k_{ji} \geq 0$, $h_i \geq 0$, $C_i > 0$.

(a) In order to perform a synthetic data collection do the following:

1. Fix N (e.g. $N = 10$). Sample positive capacities C_i and room couplings h_i, and sample a symmetric nonnegative matrix $K = [k_{ij}]$ with $k_{ii} = 0$ (e.g. $k_{ij} \sim$ Uniform$[0, \kappa]$ and symmetrize).

2. Draw initial temperatures $x_i(0)$ from a range (e.g. $[15, 40]^\circ$C). Set a constant T.

3. Simulate the ODE on $[0, T_{\max}]$ (e.g. $T_{\max} = 10$) with a fixed step using a high-accuracy solver.

4. Record time series $\{(t_\ell, x(t_\ell))\}_{\ell=0}^{L}$; optionally add small Gaussian noise to emulate measurement error.

(b) We shall use a neural ODE with attention to learn k_{ij}. To this end, we model the vector field as

$$\dot{x}_i \;=\; \frac{h_i}{C_i}\big(T - x_i\big) \;+\; \frac{1}{C_i}\sum_{j\neq i} \underbrace{\tilde{k}_{ij}(x)}_{\text{to be learned}} \;(x_j - x_i), \qquad \tilde{k}_{ij}(x) \;\approx\; k_{ij},$$

where $\tilde{k}_{ij}(x) \geq 0$ is produced by an attention mechanism:

$$a_{ij}(x) = \frac{\exp\big(\langle W_q e_i(x),\, W_k e_j(x)\rangle/\sqrt{d}\big)}{\sum_{m=1}^{N}\exp\big(\langle W_q e_i(x),\, W_k e_m(x)\rangle/\sqrt{d}\big)}, \qquad \tilde{k}_{ij}(x) = \mathrm{Softplus}\big(s_{ij}\big)\;\cdot\; a_{ij}(x),$$

with learnable symmetric interaction strengths $s_{ij} = s_{ji}$, and point embeddings $e_i(x) = \phi(x_i)$ from a small MLP ϕ. This yields an *attention Neural ODE* given by $\dot{x}(t) = f_\theta\big(x(t)\big)$, with $x(0) = x_0$.

(c) Train $\theta = \{W_q, W_k, \phi, s_{ij}, (h_i)\}$ (keeping C_i known) by minimizing a trajectory loss

$$\mathcal{L}(\theta) = \frac{1}{L+1}\sum_{\ell=0}^{L} \big\|\hat{x}_\theta(t_\ell) - x(t_\ell)\big\|_2^2,$$

where $\hat{x}_\theta$ is obtained by integrating the Neural ODE from $x(0)$. Enforce symmetry and nonnegativity via $s_{ij} = s_{ji}$ and the softplus; optionally add an L_1 penalty on $\mathrm{softplus}(s_{ij})$ to promote sparsity.

(d) To recover coefficients after training, define the estimated static couplings by time-averaging the attention,

$$\widehat{k}_{ij} \;:=\; \mathrm{softplus}(s_{ij}) \cdot \frac{1}{L+1}\sum_{\ell=0}^{L} a_{ij}\big(x(t_\ell)\big),$$

and compare $\widehat{k}_{ij}$ to the ground-truth k_{ij}.

This page intentionally left blank

Chapter 16

Further Applications

16.1 Abel's Mechanical Problem

This section introduces an inverse problem first posed by N. H. Abel, which reduces to solving a family of Abel integral equations. Our goal will be to address this problem using neural networks.

Abel's integral equation originates from Abel's celebrated 1826 paper [2], where it appeared in the study of the tautochrone and related inverse problems in mechanics. Abel's integral equation provides a direct link between experimental measurements (for instance, descent times along a curve) and an unknown function to be reconstructed. Its inversion formula is recognized today as the prototype of a fractional derivative of order one-half.

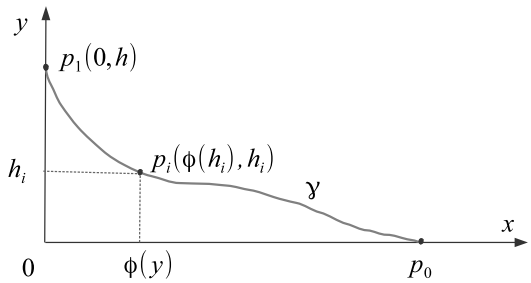


Figure 16.1: Abel's mechanical problem: determine the shape of the curve γ from the descending times corresponding to heights h_i.

Problem formulation Consider a particle of mass m moving along a plane curve γ from an upper point p_1 situated at height $h > 0$ to a lower point p_0 at ground level $h = 0$, under the action of gravity mg and without friction, see Fig. 16.1.

Problem *Let*

$$0 = h_0 < h_1 < \cdots < h_n = h$$

be a sequence of different heights, and suppose we measure the descending times

$$T_i = T(h_i), \qquad i = 0, 1, \ldots, n,$$

needed for the particle to move from height h_i down to ground level. The problem is to determine the curve γ from the measurements (h_i, T_i).

A few particular cases are noteworthy:

- If the measured times are proportional to the square root of the heights, i.e.

$$T_i = c\sqrt{h_i},$$

  then the corresponding curve γ is a straight line.

- If the measured times T_i are all equal, i.e. independent of the heights h_i, then the curve γ is an isochrone curve, which is an arc of an inverted cycloid.

To solve the problem, we begin with the law of conservation of energy

$$E \;=\; \tfrac{m}{2}v^2 + mgy \;=\; mgh \quad \text{(constant)},$$

from which we obtain the speed of the particle in terms of its altitude

$$v = \sqrt{2g(h-y)}.$$

The total time of descent from point p_1 (at height h) to point p_0 (at ground level) is

$$T(h) \;=\; \int_\gamma \frac{ds}{v} = \int_\gamma \frac{\sqrt{(dx)^2 + (dy)^2}}{\sqrt{2g(h-y)}} = \int_0^h \frac{\sqrt{1+\phi'(y)^2}}{\sqrt{2g(h-y)}}\, dy > 0,$$

where the curve γ has been parameterized in terms of the vertical coordinate y as

$$\gamma(y) = \big(\phi(y),\, y\big), \qquad 0 \le y \le h.$$

If we define

$$f(h) = T(h)\sqrt{2g}, \qquad f_i = f(h_i),$$

and introduce

$$\psi(y) = \sqrt{1+\phi'(y)^2},$$

then the problem can be reformulated as follows:

Given n heights h_i, determine the function ψ that simultaneously satisfies the system of n Abel integral equations

$$\int_0^{h_i} \frac{\psi(y)}{\sqrt{h_i - y}}\, dy = f_i, \qquad i = 1, \ldots, n. \tag{16.1.1}$$

Once the function $\psi(y)$ is obtained, the curve γ can be reconstructed by solving the ODE

$$\frac{d\phi}{dy} = \sqrt{\psi(y)^2 - 1}, \qquad \phi(h) = 0, \tag{16.1.2}$$

where we assume that the upper endpoint p_1 has coordinates $(0, h)$.

Remark 16.1.1 If a continuum of measurements is available, i.e. the descent time is monitored continuously, then the problem reduces to Abel's integral equation

$$\int_0^\tau \frac{\psi(y)}{\sqrt{\tau - y}}\, dy = f(\tau), \qquad 0 \le \tau \le h.$$

Assuming that $f(\tau)$ is differentiable, the equation can be inverted to give

$$\psi(y) = \frac{1}{\pi} \int_0^y \frac{f'(\tau)}{\sqrt{y - \tau}}\, d\tau.$$

However, in practice the measured descent times $f_i = f(h_i)$ are subject to noise, and the direct inversion formula is ill-posed. This motivates the use of neural networks, which can provide a stable approximation of the inverse problem even in the presence of noisy data.

Neural networks approach To solve the problem we employ two neural networks:

(1) a ψ-network, $y \mapsto \psi_\theta(y)$, which solves the Abel integral equations (16.1.1);
(2) a ϕ-network, $y \mapsto \phi_w(y)$, which solves the ODE (16.1.2).

The procedure is carried out in two steps:

Step 1. Training the ψ-network. We train $\psi_\theta(y)$ in a supervised manner using the loss function

$$\mathcal{L}_{\text{Abel}}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \left(f_i - \int_0^{h_i} \frac{\psi_\theta(y)}{\sqrt{h_i - y}}\, dy \right)^2.$$

Unfortunately, the integrals in this formulation are singular at $y = h_i$, which makes their numerical evaluation unstable. To address this, we apply the substitution $y = h_i(1 - s^2)$ with $s \in [0, 1]$, which removes the singularity and yields

$$\int_0^{h_i} \frac{\psi_\theta(y)}{\sqrt{h_i - y}}\, dy = 2\sqrt{h_i} \int_0^1 \psi_\theta\big(h_i(1 - s^2)\big)\, ds.$$

If we further define the rescaled data

$$\tilde{f}_i = \frac{f_i}{2\sqrt{h_i}},$$

then the loss takes the more convenient form

$$\tilde{\mathcal{L}}_{\text{Abel}}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \left(\tilde{f}_i - \int_0^1 \psi_\theta\big(h_i(1 - s^2)\big)\, ds \right)^2.$$

Let $\psi_{\theta^*}(y)$ denote the trained network with parameters $\theta^* = \arg\min \tilde{\mathcal{L}}_{\text{Abel}}(\theta)$.

Step 2. Training the ϕ-network. Using the output $\psi_{\theta^*}(y)$ from Step 1, we solve the ODE (16.1.2). We hard-embed the initial condition by writing

$$\phi_w(y) = (y - h)\, \text{nn}(w; y),$$

and train this network with the loss

$$\mathcal{L}(w) = \int_0^h \Big(\phi'_w(y) - \sqrt{\psi_{\theta^*}(y)^2 - 1}\Big)^2 \, dy.$$

The reconstructed curve is then given by

$$\gamma(y) = \big(\phi_{w^*}(y),\, y\big),$$

where $w^* = \arg\min \mathcal{L}(w)$.

Implementation example We generate first some synthetic data (h_i, f_i) for $\phi(y) = \sqrt{1-y^2}$, $0 \le y \le 1$ and $f_i = \displaystyle\int_0^{h_i} \frac{\sqrt{1+\phi'(y)^2}}{\sqrt{h_i - y}}\, dy$, see Table 16.1. The curve γ corresponds in this case to the first quadrangle of the unit circle. We note that as

h_i	f_i
0.1	0.634152
0.2	0.904192
0.3	1.123179
0.4	1.324445
0.5	1.524887
0.6	1.739241
0.7	1.987479
0.8	2.308899
0.9	2.823767
0.99	4.46184
0.999	6.08905

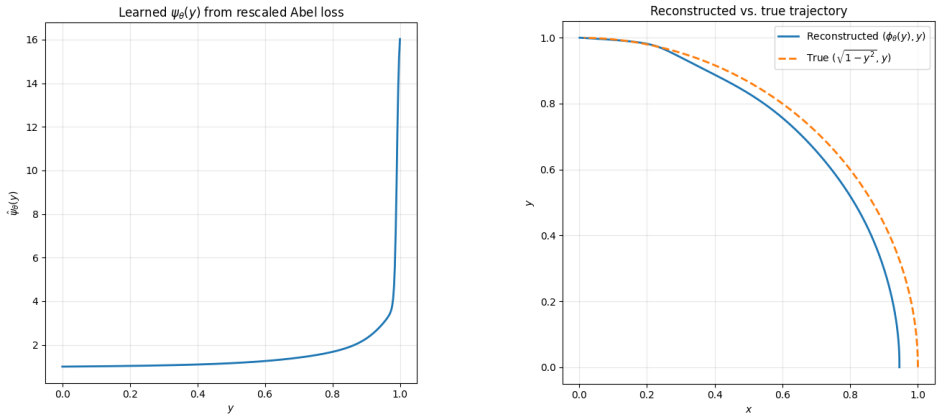
Table 16.1: Values of (h_i, f_i) for $\phi(y) = \sqrt{1-y^2}$.

$h_i \to 1$, we have $f_i \to \infty$. This indicates that the tangent to the curve γ at the point p_1 becomes horizontal, so the particle remains at p_1 in an (unstable) equilibrium.

The ψ-network architecture is a feed-forward neural network with three hidden layers: 128 neurons in the first layer, 128 in the second, and 64 in the third. We use tanh activations for the hidden layers. The input layer rescales the data from $[0, 1]$ to $[-1, 1]$, while the output layer applies a softplus activation to ensure that $\psi(y)$ remains positive. Training was performed for 10,000 epochs using the Adam optimizer with a small learning rate ($\mathrm{lr} = 10^{-4}$) in order to reduce oscillations of the loss function. The resulting graph of $\psi(y)$ is an increasing function, see Fig. 16.2(a).

The ϕ-network has a very similar architecture, with the difference that the initial conditions are hard-embedded, and the output layer uses a linear activation. This network was trained for 5,000 epochs with the same learning rate ($\mathrm{lr} = 10^{-4}$). The reconstructed curve compared with the true trajectory is shown in Fig. 16.2(b). Further training does not improve the approximation, since the loss has already reached its threshold; the remaining mismatch arises from the limited amount of data (h_i, f_i) available.

The reader can find the detailed code at `https://tinyurl.com/38yr2mpx`.



(a) **(b)**

Figure 16.2: (a) The function $\psi_\theta(y)$ is increasing on $[0, 1]$; (b) Reconstructed trajectory $(\phi(y), y)$ of the curve.

Further reading For classical references on inverse problems leading to Abel's integral equations, we refer the reader to the works of Gorenflo and Vessella [41], Murio [73], and Wing [99].

More recent advances combine Abel's integral equation with Deep Learning methods. One of the early contributions is due to Ma et al. [66], who demonstrated how multilayer feed-forward neural networks can be trained to perform Abel inversion in plasma diagnostics. A neural network approach to Abel integral operators, with comparisons to classical numerical methods, is presented in Chouzenoux, Della Valle, and Pesquet [24]. A Deep Learning method for inverse Abel transforms applied to experimental imaging data is described in [86]. Finally, while not based on Deep Learning, the work of Hosseinian et al. [49] develops efficient numerical schemes for generalized Abel equations, providing useful benchmarks for evaluating machine learning approaches.

16.2 Inverse Sturm–Liouville Problem

This section addresses the classical question of whether one can "hear" the density of a vibrating string from its sound. More precisely, we consider a string of length L, held under constant tension and fixed at both endpoints, with a positive weight density $\rho(x) > 0$. The total weight is normalized,

$$\int_0^L \rho(x)\, dx = 1.$$

The transverse displacement of the string satisfies the weighted wave equation with Dirichlet boundary conditions,

$$\rho(x)\, \partial_t^2 u(x,t) = \partial_x^2 u(x,t), \qquad 0 < x < L,\ t > 0,$$
$$u(0,t) = u(L,t) = 0, \qquad t > 0.$$

Applying separation of variables $u(x,t) = X(x)T(t)$ leads to the weighted Sturm–Liouville problem

$$X''(x) = -\omega^2 \rho(x) X(x), \qquad X(0) = X(L) = 0,$$

together with the temporal part

$$T(t) = A\cos(\omega t) + B\sin(\omega t).$$

The frequencies of vibration are determined by the eigenvalues $\{\omega_n\}$ satisfying

$$X_n''(x) = -\omega_n^2 \rho(x) X_n(x), \qquad X_n(0) = X_n(L) = 0,$$

with associated eigenfunctions $X_n(x)$. Sturm–Liouville theory ensures that the eigenvalues are real, positive, simple, and increase to infinity. Moreover, the eigenfunctions form an orthogonal family with respect to the ρ-weighted inner product,

$$\int_0^L X_n(x) X_m(x)\, \rho(x)\, dx = 0, \qquad n \neq m.$$

The solution of the wave equation is obtained by superposition of vibration modes:

$$u(x,t) = \sum_{n=1}^{\infty} \big(A_n \cos(\omega_n t) + B_n \sin(\omega_n t)\big) X_n(x).$$

The inverse problem may now be stated as follows:

Problem *Given the frequency spectrum $\{\omega_n\}$, determine the weight density $\rho(x)$.*

Non-uniqueness In general, this problem is not well-posed, since $\rho(x)$ may not be uniquely determined from a single spectrum. For example, the symmetric load $r(x) = \rho(L - x)$ produces the same spectrum, since if $X(x)$ is an eigenfunction for $\rho(x)$, then $Y(x) = X(L-x)$ is an eigenfunction for $r(x)$. Thus, additional information is required to ensure uniqueness, such as a second spectrum under different boundary conditions (e.g., Dirichlet–Neumann), or symmetry assumptions on $\rho(x)$ with respect to the midpoint of the string.

Historical remarks The systematic study of such inverse problems goes back to the pioneering work of Borg [11], who proved that two spectra (e.g., Dirichlet–Dirichlet and Dirichlet–Neumann) determine the potential or weight uniquely. Later, Gel'fand and Levitan [37] and Marchenko [67] developed integral equation methods for reconstructing coefficients from spectral data. In the context of vibrating strings, these results form the mathematical foundation for the idea of "hearing the density," showing both the possibilities and limitations of the inverse problem.

While classical theory establishes uniqueness and reconstruction formulas, practical computation from noisy or incomplete spectral data is challenging. Modern approaches employ numerical optimization and machine learning techniques to address this ill-posedness. For example, spectral fitting methods approximate $\rho(x)$ by adjusting parameters until the computed eigenvalues match the observed ones.

In this section, we employ physics-informed neural networks that embed the Sturm–Liouville operator directly into the training loss to learn $\rho(x)$ from spectral measurements. Since the weight $\rho(x)$ is represented by a neural network, we assume it to be sufficiently smooth on $[0, L]$.

The use of PINNs Suppose we are given N frequencies $\omega_1, \ldots, \omega_N$, which correspond to the eigenvalue problems

$$X_k''(x) = -\omega_k^2 \rho(x) X_k(x), \qquad X_k(0) = X_k(L) = 0, \qquad 1 \le k \le N,$$

for eigenfunctions X_k. The density function $\rho(x)$ is the same across all these N problems.

We introduce the $(N+1)$-dimensional neural network

$$x \to NN(\theta; x) = \big(\rho_\theta(x),\, X_1(\theta; x),\, \ldots,\, X_N(\theta; x)\big),$$

which is trained to learn simultaneously the density function and the eigenfunctions.

Density normalization. To enforce the total mass constraint $\int_0^L \rho(x)\, dx = 1$, we set

$$\rho_\theta(x) = \frac{r_\theta(x)}{\int_0^L r_\theta(y)\, dy},$$

where $r_\theta(x) = \mathrm{sp}(\mathrm{nn}(\theta; x))$ is the output of a feed-forward neural network passed through a softplus activation, ensuring positivity.

Boundary conditions. The boundary conditions for each eigenfunction are hard-embedded by writing

$$X_k(\theta; x) = x(L - x)\, \mathrm{nn}_k(\theta; x),$$

so that $X_k(\theta; 0) = X_k(\theta; L) = 0$ automatically.

Loss functions. For each eigenvalue problem we define the residual loss

$$\mathcal{L}_k(\theta) = \frac{1}{m} \sum_{j=1}^{m} \Big(X_k''(\theta; x_j) + \omega_k^2 \rho_\theta(x_j) X_k(\theta; x_j) \Big)^2,$$

using m collocation points $\{x_j\} \subset [0, L]$.

If we assume symmetry of the density about the midpoint, $\rho(x) = \rho(L - x)$, we enforce it with an additional penalty:

$$\mathcal{L}_{\mathrm{sym}}(\theta) = \frac{1}{m} \sum_{j=1}^{m} \big(\rho_\theta(x_j) - \rho_\theta(L - x_j) \big)^2.$$

Total loss. The overall loss is then

$$\mathcal{L}_{\mathrm{tot}}(\theta) = \sum_{k=1}^{N} \mathcal{L}_k(\theta) \;+\; \lambda\, \mathcal{L}_{\mathrm{sym}}(\theta),$$

with $\lambda > 0$ a penalty weight. Minimizing $\mathcal{L}_{\text{tot}}$ enforces all N boundary value problems simultaneously while respecting symmetry. Because the network must learn $N + 1$ functions at once (the density and N eigenfunctions), training can be slow, but the formulation guarantees consistency across all spectral data.

To accelerate training, instead of enforcing the symmetry condition through the soft constraint $\mathcal{L}_{\text{sym}}(\theta)$, we can impose it directly by a hard embedding using `softplus` of the form

$$r_\theta(x) = \text{sp}\big(NN\big(\theta;\, (x - \tfrac{1}{2})^2\big)\big)\,,$$

so that symmetry about the midpoint is built into the network architecture. Thus, both the symmetry and the normality of $\rho(x)$ can be simultaneously hard embedded.

If the symmetry condition is omitted, the problem admits two symmetric solutions. In this case, the neural network will converge to whichever solution lies closer to the initialization or the training data.

Well-posedness We now address the question of uniqueness. Under the additional assumption that the density is symmetric with respect to the midpoint of the string, i.e.

$$\rho(x) = \rho(L - x), \qquad 0 \le x \le L,$$

the inverse problem admits a unique solution.

The main idea is to transform the boundary value problem on $[0, L]$,

$$X''(x) = -\omega^2 \rho(x)\, X(x), \qquad X(0) = X(L) = 0,$$

into a canonical Sturm–Liouville eigenvalue problem on the normalized interval $[0, 1]$:

$$\phi''(s) = -\lambda\, q(s)\, \phi(s), \qquad \phi(0) = \phi(1) = 0, \quad 0 < s < 1,$$

for a suitable weight $q(s)$. For this standard form, uniqueness of the coefficient under the symmetry condition

$$q(s) = q(1 - s)$$

is a well-known result; see, for example, Kirsch [55, p. 159].

To carry out the reduction, we apply the classical Liouville transform. Define

$$S = \int_0^L \sqrt{\rho(\xi)}\, d\xi, \qquad s(x) = \frac{1}{S} \int_0^x \sqrt{\rho(\xi)}\, d\xi,$$

which maps $[0, L]$ onto $[0, 1]$. Next, set

$$\phi(s) = f(x)\, X(x), \qquad f(x) = \rho(x)^{1/4}.$$

One checks that $\phi(s)$ satisfies the Sturm–Liouville problem above with

$$q(s) = S^2 \left[\frac{f(x)}{\rho(x)} \left(\frac{f'(x)}{f(x)^2} \right)' \right]_{x = x(s)}, \qquad \lambda = S^2 \omega^2.$$

In this way, the original inverse problem for the string is reduced to an equivalent inverse Sturm–Liouville problem in canonical form. The symmetry condition on ρ translates

into the symmetry of q, which by the cited uniqueness theorem guarantees that the density $\rho(x)$ can be uniquely recovered from the spectrum.

Implementation example

Generating synthetic data. Consider a string of length $L = 10$ with the initial density

$$\rho(x) = \frac{(x-5)^2 + 1}{10 + \frac{10^3}{12}}, \qquad 0 \le x \le 10,$$

which is symmetric with respect to the vertical line $x = 5$ and normalized so that $\int_0^{10} \rho(x)\, dx = 1$.

We are interested in computing the first few eigenvalues of the boundary value problem

$$X''(x) = -\omega^2 \rho(x)\, X(x), \qquad X(0) = X(10) = 0.$$

This is the forward spectral problem, which can be solved numerically, for instance, by discretizing the operator using a finite-difference scheme and assembling the generalized eigenvalue problem

$$-AX = \omega^2 BX,$$

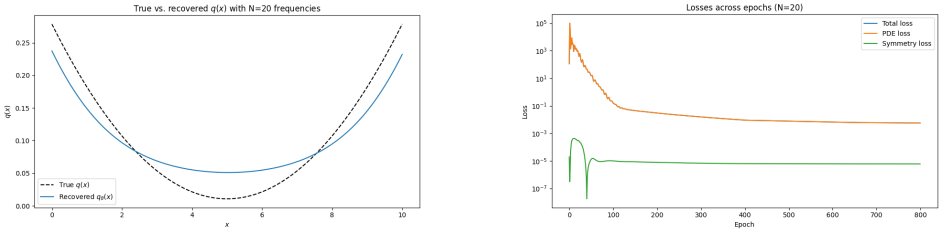
where A is the stiffness matrix (discretization of $-\frac{d^2}{dx^2}$) and B is the mass matrix weighted by $\rho(x)$. The resulting frequencies $\{\omega_n\}$ are given in Table 16.2 and will serve as the synthetic spectral data for testing the inverse reconstruction of $\rho(x)$ using neural networks.

n	ω_n	n	ω_n
1	1.3982	11	12.0849
2	2.0369	12	13.0843
3	3.5183	13	14.2485
4	4.2672	14	15.2750
5	5.6452	15	16.4175
6	6.4840	16	17.4631
7	7.7815	17	18.5904
8	8.6906	18	19.6495
9	9.9285	19	20.7660
10	10.8900	20	21.8346

Table 16.2: First 20 frequencies ω_n of the loaded string.

Architecture and results. We used a feed-forward neural network with two hidden layers, each containing 64 `swish`-activated neurons. Training was performed for 800 epochs with a learning rate of 2.2×10^{-2}, and the symmetry constraint was enforced with weight $\lambda = 0.85$. The recovered profile has a parabolic shape similar to the initial weight function, see Fig. 16.3(a), though it does not match it closely even after extensive training, see Fig. 16.3(b). This level of recovery may still be considered reasonable, since the reconstruction relies only on a finite set of frequencies and thus inevitably lacks complete information.

The code can be found at: `https://tinyurl.com/24fctkvn`.



(a) **(b)**

Figure 16.3: (a) The reconstructed density function $\rho_\theta(x)$ and the true density $\rho(x) = \frac{(x-5)^2+1}{10+\frac{10^3}{12}}$; (b) The associated losses.

Limitations. If the density is discrete, consisting of two point masses of weight $1/2$ placed symmetrically with respect to the midpoint of the string, that is,

$$\rho(x) = \tfrac{1}{2}\big(\delta(x - \tfrac{L}{2} - a) + \delta(x - \tfrac{L}{2} + a)\big),$$

where $\delta(x - x_0)$ denotes the Dirac delta function centered at x_0, then the neural network is unable to recover ρ exactly, since neural networks cannot approximate singular measures. Instead, the network will approximate it by learning a smoothed version of the measure.

16.3 Computer Tomography

This section addresses the medical problem of tomographic reconstruction, namely the recovery of a body's internal density distribution from the attenuation of X-ray beams passing through the body at various angles. We begin with the definition of the Radon transform and its geometric setting, then proceed to the inverse problem of reconstructing the density using neural networks.

Radon Transform Let $f : \mathbb{R}^2 \to [0, +\infty)$ be a given integrable function. For simplicity and practical purposes, we assume that $f(x, y)$ has compact support. To build intuition, one may think of $f(x, y)$ as the grayscale intensity of an image at the pixel located at (x, y). For a fixed value of x, the integral

$$F(x) = \int_{\mathbb{R}} f(x, y)\, dy$$

represents the marginal distribution of the image intensity along the x-axis. In other words, $F(x)$ collects cumulative information along vertical lines and can be interpreted as the projection of f onto the x-axis, see Fig. 16.4, left.

Consider now the rotation by an angle δ about the origin, $\mathrm{Rot}_\delta : \mathbb{R}^2 \to \mathbb{R}^2$, defined by

$$\mathrm{Rot}_\delta(x, y)^T = \begin{pmatrix} \cos\delta & -\sin\delta \\ \sin\delta & \cos\delta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}.$$

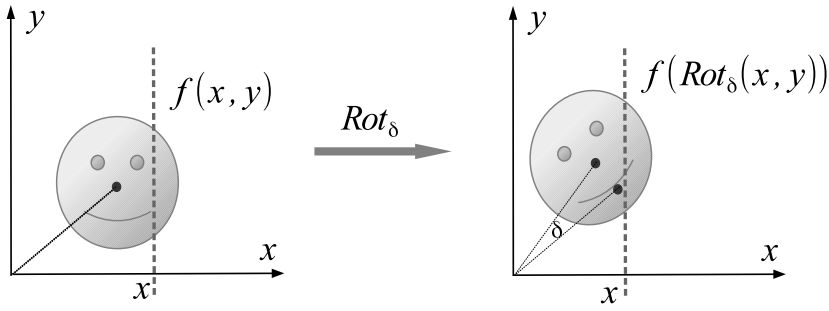


Figure 16.4: Integration along a vertical line in an image before and after rotation.

The rotated image is then given by

$$f(x', y') = f\big(\mathrm{Rot}_\delta(x, y)^T\big),$$

which corresponds to the intensity of the pixel (x', y') obtained after rotation by angle δ, which explicitly means

$$f(x', y') = f(x\cos\delta - y\sin\delta,\ x\sin\delta + y\cos\delta).$$

As before, we define the marginal integral with respect to y

$$(R_\delta f)(x) = \int_{\mathbb{R}} f\big(\mathrm{Rot}_\delta(x, y)^T\big)\, dy. \tag{16.3.3}$$

This expression represents the projection of the rotated image at angle δ onto the x-axis, see Fig. 16.4, right. The function $R_\delta f : \mathbb{R} \to \mathbb{R}$ is called the *Radon transform* of f at angle δ.

We shall see that $R_\delta f$ represents the integral of f along a line. Equivalently, instead of rotating the picture and integrating along the vertical direction, one may rotate the line itself and integrate f along that line. We will develop this idea in the next paragraph.

Some geometry Consider the vertical line $\{x = s\}$, parameterized by the vertical coordinate $y = u$, and denote it by ℓ_0. Each point on this line has coordinates (s, u), where s is fixed. Let ℓ_δ be the line obtained by rotating ℓ_0 by an angle δ, see Fig. 16.5. The coordinates (x, y) of a point on ℓ_δ are then given by

$$\begin{aligned} x &= s\cos\delta - u\sin\delta, \\ y &= s\sin\delta + u\cos\delta. \end{aligned}$$

From simple geometric considerations, we observe that:

(i) s represents the distance from the origin $(0, 0)$ to the line ℓ_δ,

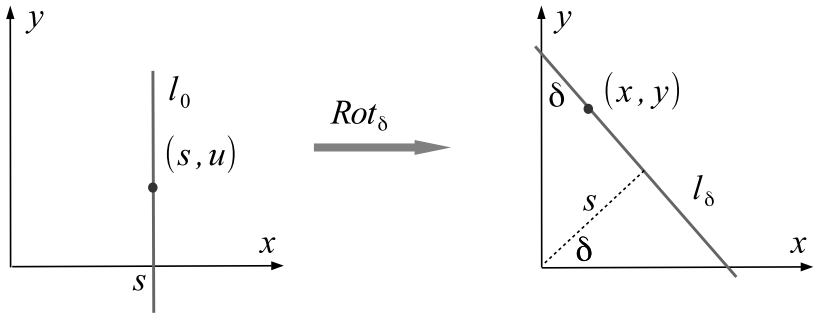


Figure 16.5: The line ℓ_δ is obtained from the line ℓ_0 by a rotation of angle δ, and the point (s, u) is mapped to (x, y).

(ii) u is the parameter along the line,

(iii) δ is the angle made by ℓ_δ with the vertical axis.

Thus, the pair (s, u) can be used as parameters for the line ℓ_δ. Integrating $f(x, y)$ along ℓ_δ gives

$$\begin{aligned} \int_{\ell_\delta} f\, ds &= \int_{\mathbb{R}} f(s\cos\delta - u\sin\delta, s\sin\delta + u\cos\delta)\, du \\ &= \int_{\mathbb{R}} f(\mathrm{Rot}_\delta(s,u)^T)\, du = (R_\delta)(s). \end{aligned}$$

This recovers the Radon transform (16.3.3). Hence, the Radon transform of f can be interpreted as the integral of f along a line making an angle δ with the vertical axis.

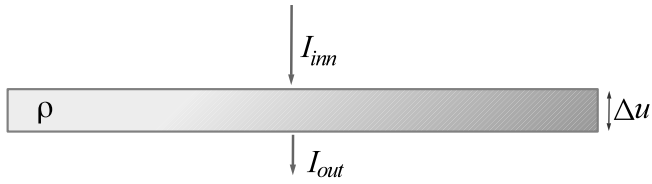


Figure 16.6: The attenuation of intensity in a slab of thickness Δu and density ρ.

Setting up the Problem Let I denote the intensity of an X-ray beam passing through a slab of density ρ and thickness Δu. Denote by I_{in} and I_{out} the incoming

and outgoing intensities of the beam before and after traversing the slab, see Fig. 16.6. Since radiation is absorbed in the material, we have $I_{\text{out}} < I_{\text{in}}$.

The attenuation $\Delta I = I_{\text{out}} - I_{\text{in}}$ is proportional to:

- ρ, since denser material produces greater attenuation,
- Δu, as a thicker slab absorbs more radiation,
- I, because higher incoming intensity leads to greater loss.

Introducing a proportionality constant $\gamma > 0$, we write

$$\Delta I = -\gamma \rho I \, \Delta u.$$

Taking the limit $\Delta u \to 0$ leads to the differential equation

$$\frac{dI}{du} = -\gamma \rho I.$$

Integrating from u_0 to u gives

$$\ln \frac{I(u)}{I(u_0)} = -\gamma \int_{u_0}^{u} \rho\big(s\cos\delta - u\sin\delta,\ s\sin\delta + u\cos\delta\big)\, du.$$

Extending $u_0 \to -\infty$ and $u \to +\infty$ (which, due to the compact support of ρ, simply means taking u_0 sufficiently small and u sufficiently large) yields

$$-\frac{1}{\gamma}\big(\ln I(\infty) - \ln I(-\infty)\big) = \int_{\mathbb{R}} \rho\big(s\cos\delta - u\sin\delta,\ s\sin\delta + u\cos\delta\big)\, du = (R_\delta \rho)(s).$$

The left-hand side, which will be denoted by $g(s, \delta)$, is a measurable physical quantity, called the *sinogram*, representing the total attenuation of the beam intensity, while the right-hand side is exactly the Radon transform of the density function ρ.

The Use of Neural Networks The tomographic reconstruction problem can be formulated as follows:

Recover the density function ρ, given the following:

(i) The density values at some sample points (x_i, y_j) inside the body,

$$\rho_{ij} = \rho(x_i, y_j), \quad 1 \le i \le n,\ 1 \le j \le m.$$

(ii) The Radon transform of ρ along a collection of lines $\ell(\delta_k, s_r)$,

$$(R_{\delta_k}\rho)(s_r) = g_{rk}, \quad 1 \le r \le N,\ 1 \le k \le M,$$

where $g_{rk} = g(s_r, \delta_k)$ are some readings from the attenuation function g.

To this end, we employ a neural network

$$(x, y) \mapsto NN(\theta; x, y) = \rho_\theta(x, y),$$

as an approximation of the density function.

We then define the following losses:

1. A data loss enforcing agreement with known pointwise values,

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{nm} \sum_{i=1}^{n} \sum_{j=1}^{m} \Big(\rho_\theta(x_i, y_j) - \rho_{ij} \Big)^2.$$

2. A Radon loss enforcing agreement with measured projections,

$$\mathcal{L}_{\text{Radon}}(\theta) = \frac{1}{NM} \sum_{r=1}^{N} \sum_{k=1}^{M} \Big(R_{\delta_k} \rho_\theta(s_r) - g_{rk} \Big)^2.$$

The total loss is then given by

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{Radon}}(\theta) + \lambda \, \mathcal{L}_{\text{data}}(\theta),$$

where $\lambda > 0$ balances the contribution of the two terms. If no data is available, we take $\lambda = 0$.

The neural network method just described tends to minimize low-frequency content and stops there. To improve and catch higher frequency and detail we need to add one more ingredient related to the Fourier transform.

Fourier Slice Theorem A fundamental property in tomography relates the 1D Fourier transform of the Radon transform at angle δ to a radial line through the 2D Fourier transform of ρ:

$$\mathcal{F}_s(R_\delta \rho(s))\,(\omega) = \hat{\rho}(\omega \cos\delta,\ \omega \sin\delta)\,,$$

where $\hat{\rho}$ denotes the 2D Fourier transform of ρ. We explain this formula as follows.

The left-hand side represents the Fourier transform of the Radon projections and is a computable expression

$$\mathcal{F}_s(R_\delta \rho(s))\,(\omega) = \mathcal{F}_s\big(g(s,\delta)\big)(\omega) = \int_{\mathbb{R}} g(s,\delta)\, e^{-i\omega s}\, ds.$$

The right-hand side, by the definition of the 2D Fourier transform, is given by

$$\hat{\rho}(\omega \cos\delta,\ \omega \sin\delta) \quad = \quad \int_{\mathbb{R}^2} \rho(x,y)\, e^{-i\omega(x\cos\delta + y \sin\delta)}\, dx\, dy.$$

To enforce the Fourier Slice Theorem during reconstruction with a neural network approximation ρ_θ, we introduce the following loss:

$$\mathcal{L}_{\text{Fourier}}(\theta) = \frac{1}{NM_\omega} \sum_{j=1}^{M} \sum_{k=1}^{M_\omega} \Big| \psi_\theta(\omega_k, \delta_j) - \varphi(\omega_k, \delta_j) \Big|^2,$$

with

$$\psi_\theta(\omega,\delta) = \int_{\mathbb{R}^2} \rho_\theta(x,y)\, e^{-i\omega(x\cos\delta + y\sin\delta)}\, dx\, dy, \qquad \varphi(\omega,\delta) = \int_{\mathbb{R}} g(s,\delta)\, e^{-i\omega s}\, ds$$

and $|\cdot|$ represents the modulus of a complex expression.

Combining Radon-domain and Fourier-domain losses leads to more robust and sharper reconstructions of ρ. We thus combine the penalties into a single unified objective:

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{Radon}}(\theta) + \mu\,\mathcal{L}_{\text{Fourier}}(\theta) + \lambda\,\mathcal{L}_{\text{data}}(\theta),$$

where $\mu, \lambda \geq 0$ balance the frequency- and pointwise-constraints.

Remark 16.3.1 The Fourier loss $\mathcal{L}_{\text{Fourier}}(\theta)$ complements the Radon loss introduced earlier. While the Radon loss enforces agreement in the spatial domain by matching line integrals of the reconstruction ρ_θ with the measured projections $g(s, \delta)$, the Fourier loss enforces agreement in the frequency domain by requiring the Fourier slices of ρ_θ to match the Fourier transforms of the measured sinograms. This dual enforcement has two advantages:

- It stabilizes training by providing constraints both in the spatial and frequency domains, reducing the risk of overfitting to noise in the sinogram.
- It encourages the network to recover high-frequency components more accurately, which are often lost or blurred when training only with Radon-domain losses.

Training a neural network with the above architecture is computationally intensive and requires large amounts of RAM - typically exceeding what Google Colab can provide. For practical experimentation, the code is available at `https://tinyurl.com/3js5s9ts` and should be run on a system equipped with substantial GPU resources and memory.

Further readings Classical computed tomography (CT) is well documented in several foundational works. A standard reference on the mathematics and algorithms is the book by Kak and Slaney [52]; a more rigorous mathematical treatment is provided by Natterer [76]. Connections to practical aspects and the underlying physics are discussed extensively in Herman [46] and Buzug [14].

While classical approaches rely mainly on Radon transforms, filtered backprojection, and iterative methods, more recent research has incorporated deep learning into CT reconstruction. Notable developments include the introduction of deep convolutional neural networks for CT reconstruction by Jin et al. [50], and the residual encoder–decoder CNN proposed by Chen et al. [21] for noise reduction. Zhu et al. [104] presented a high-profile demonstration that deep learning can surpass classical compressed sensing methods in medical imaging reconstruction. Further contributions that explore the integration of deep learning with CT reconstruction include the works of Adler and Öktem [3], He et al. [44], and Li et al. [59].

16.4 Deep Learning for GPS–INS Sensor Fusion

A modern application of Deep Learning arises in the context of GPS-based localization, particularly when combined with inertial navigation systems (INS). An INS is equipped with transducers such as accelerometers and gyroscopes that measure quantities like

linear acceleration and angular velocity. In principle, the position of a vehicle can be determined from the accelerometer by performing a double integration of the measured acceleration, provided that suitable initial conditions for position and velocity are known.

However, in practice these measurements are corrupted by sensor noise and bias, which leads to drift and ambiguity in the estimated position over time. To mitigate this error accumulation, the INS estimate can be corrected periodically by observations from a GPS receiver. The GPS signal itself is also susceptible to noise, multipath effects, or temporary unavailability (e.g., in tunnels or urban canyons). The challenge, therefore, is to combine both measurement sources — each of which is individually noisy — into a more reliable trajectory estimate.

Traditionally, this problem is addressed using stochastic filtering methods, such as the Kalman filter and its nonlinear variants, which fuse INS and GPS measurements into a single coherent estimate. A Deep Learning approach can enhance this process by learning nonlinear correction models that compensate for noise and bias patterns more effectively than classical linear filters.

Neural Networks approach Concretely, suppose the INS provides acceleration measurements

$$\ddot{x}_j = \ddot{x}(t_j)$$

at discrete time instances t_j along the true trajectory $x(t)$. Given the initial conditions $x(0) = x_0$ and $\dot{x}(0) = v_0$, one could in principle recover the trajectory by double integration. In practice, however, these measurements are contaminated by noise, so that

$$\ddot{x}_j = \ddot{x}(t_j) + \epsilon_j,$$

where ϵ_j denotes the measurement error at time t_j. As a consequence, the reconstructed trajectory drifts over time. To reduce this drift, GPS observations are used to periodically recalibrate the estimate by providing location measurements x_i at times τ_i. Since they are also susceptible to noise, we have

$$x_i = x(\tau_i) + \eta_i,$$

with η_i GPS error at time τ_i.

Deep Learning offers an alternative framework for this data fusion task. By training neural networks on sequences of noisy acceleration data together with periodic GPS corrections, one can construct robust estimators of the underlying trajectory. Such approaches have the potential to outperform classical filtering techniques, especially when the measurement errors are non-Gaussian, correlated, or exhibit complex temporal patterns that traditional methods fail to capture.

We model the trajectory using a neural network with initial conditions embedded

$$NN(\theta; t) = x_\theta(t) = x_0 + tv_0 + t^2\,\mathrm{nn}(\theta; t),$$

with $\mathrm{nn}(\theta; t)$ free neural network.

We assume INS data are collected at high frequency for times $t_j \in [0,T]$, with $j \leq N_T$, while GPS data are available at sparser time instances $\tau_i \in [0,T]$, with $i \leq M_T$ and $M_T \ll N_T$. We define two loss terms, one corresponding to INS data and the other to GPS observations:

$$\mathcal{L}_{\text{INS}}(\theta) = \frac{1}{N_T}\sum_{j=1}^{N_T}\Big(\ddot{x}_j - \ddot{x}_\theta(t_j)\Big)^2,$$

$$\mathcal{L}_{\text{GPS}}(\theta) = \frac{1}{M_T}\sum_{i=1}^{M_T}\Big(x_i - x_\theta(\tau_i)\Big)^2.$$

The total loss is then defined as

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{INS}}(\theta) + \lambda\mathcal{L}_{\text{GPS}}(\theta),$$

with the positive hyperparameter of the order of magnitude $\lambda \approx N_T/M_T$. The optimal trajectory estimate on $[0,T]$ is given by the neural network $x_{\theta^*}(t)$, with $\theta^* = \arg\min_\theta \mathcal{L}_{\text{total}}(\theta)$.

Noise regularization Since $x(t)$ represents the trajectory of a vehicle with non-negligible mass, its evolution is governed by inertia. This implies that the trajectory should be sufficiently smooth in time: it cannot exhibit abrupt oscillations or high-frequency fluctuations that are physically unrealistic. However, when training a neural network on noisy sensor data, the network may overfit the noise and produce such spurious oscillations in the predicted trajectory.

A standard way to mitigate this problem is to introduce a smoothness prior through regularization. In practice, one can add an L^2-penalty (also called weight decay) to the network parameters, which discourages overly complex models and suppresses noise amplification. This enforces that the learned trajectory $x_\theta(t)$ remains close to a physically plausible smooth curve, consistent with the inertial dynamics of the vehicle.

Implementation example We consider a vehicle moving in a plane with unit speed along the unit circle

$$x(t) = (\cos t, \sin t), \qquad t \in [0, 2\pi].$$

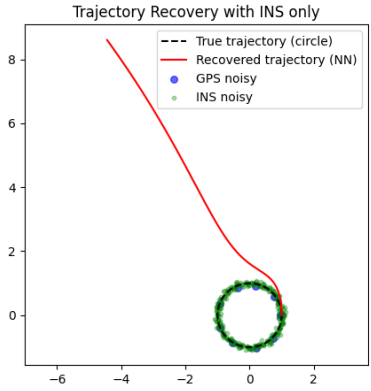
The initial conditions are given as $x_0 = (1,0)$ and $v_0 = (0,1)$. The available measurements consist of noisy position data

$$x_j = \big(\cos\tau_j + \epsilon_j^1,\ \sin\tau_j + \epsilon_j^2\big),$$

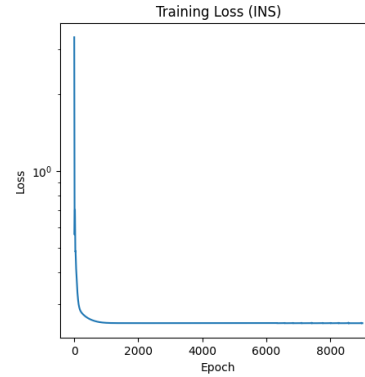
and noisy acceleration data

$$\ddot{x}_j = \big(-\cos t_j + \eta_j^1,\ -\sin t_j + \eta_j^2\big),$$

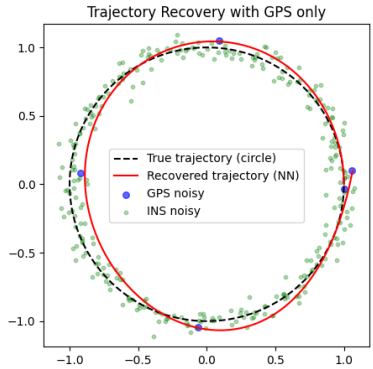
where ϵ_j^k and η_j^k are Gaussian noise terms with standard deviation equal to 5% of the signal amplitude. We used $N_T = 300$ INS acceleration measurements and $M_T = 5$



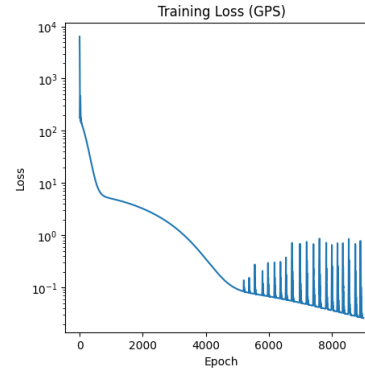
(a)



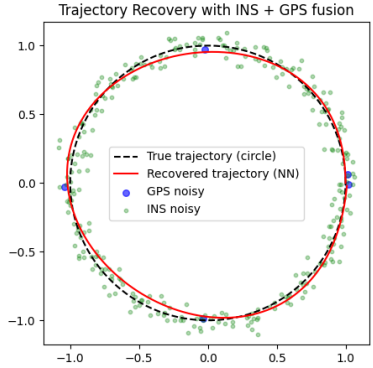
(b)



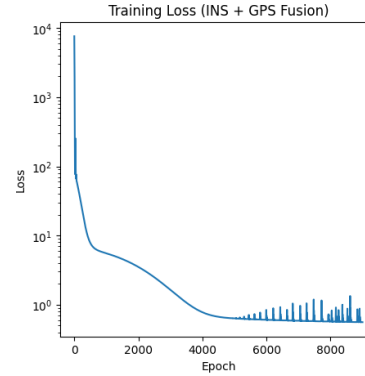
(c)



(d)



(e)



(f)

Figure 16.7: Trajectory recovery experiments using neural networks with INS and GPS data fusion: (a) Recovery using INS data only; (b) Loss curve for case (a); (c) Recovery with GPS data only; (d) Loss curve for case (c); (e) Recovery with both INS and GPS; (f) Loss curve for case (e).

GPS position measurements. The trajectory was modeled with a neural network having three hidden layers of 64 neurons each, with tanh activation and an L^2 regularization penalty of 10^{-5} per layer. Training was performed for 9,000 epochs using the Adam optimizer with a learning rate of 10^{-4}.

We present the outcomes in three cases:

(*i*) When using only the INS acceleration data, the method proves unstable due to measurement noise. The circular trajectory cannot be recovered, and instead the solution diverges quadratically over time, see Fig. 16.7(a). More training does not fix the problem as the loss already reached its saturation threshold, see Fig. 16.7(b).

(*ii*) When using only the GPS data, the neural network learns the trajectory from just 5 observation points. While it captures the general circular shape, the reconstruction remains lousy, see Fig. 16.7(c). Again, more training does not fix the problem as the loss function start to exhibit spikes, see Fig. 16.7(d).

(*iii*) When combining both INS and GPS data, the neural network successfully recovers the full circular trajectory, demonstrating the advantage of data fusion, see Figs. 16.7(e)–(f).

The entire code can be found at:

`https://tinyurl.com/2s46zr9k`

The integration of Deep Learning with GPS–INS sensor fusion and Kalman filtering, particularly for noise compensation, has emerged as a highly active area of recent research. Several contributions published in 2025 highlight different perspectives on this topic. Liu et al. [63] introduce a Deep Learning framework based on temporal convolutional networks to correct GNSS positioning errors, thereby enhancing GPS-based localization. Xue et al. [100] examine GPS/IMU fusion by comparing classical Kalman filtering methods with machine learning and deep learning algorithms. Wang [98] offers a systematic comparison between the Extended Kalman Filter (EKF) and various machine learning techniques for GPS-IMU fusion in navigation tasks.

This page intentionally left blank

Chapter 17

Miscellaneous Exercises

In this final part of the book, we present a collection of exercises that are not confined to the methods of a single chapter, as was the case with the end-of-chapter problems. Instead, these exercises are more challenging, draw on a wider range of techniques, and are intended to stimulate deeper reflection. Some may even serve as class projects or as starting points for further research.

Exercise 17.0.1 Let $t \neq -1$.

(a) Show that the function $x(s) = e^s$ satisfies the integral equation

$$\int_0^1 e^{ts}\, x(s)\, ds \;=\; \frac{e^{t+1}-1}{t+1}.$$

(b) Explain how one could use a neural network to approximate and recover the solution from part (a).

(c) Consider the modified equation

$$\int_0^1 e^{-s}\, x(s)\, ds = 1.$$

Can you determine the corresponding solution?

Exercise 17.0.2 We assume that measurements

$$(u_i, u_i') = \big(u(t_i), u'(t_i)\big), \qquad i = 0, \ldots, N,$$

are provided along a dynamical system trajectory $(u(t), u'(t))$ in phase space, with $t \in [0, T]$. Some of these measurements may contain noise, but we assume that a subset of N data points is exact (noiseless). In earlier problems we considered only the case when the boundary values

$$(u_0, u_0') = \big(u(t_0), u'(t_0)\big), \qquad (u_N, u_N') = \big(u(t_N), u'(t_N)\big),$$

were exact, which allowed us to hard-embed them into a neural network representation (as in the case of elastica curves) using cubic Hermite polynomials.

(a) Show that in the general case of N exact data points we can use the following hard embedding:

$$u_\theta(t) \;=\; \sum_{i=0}^{N} \big(u_i H_i(t) + u_i' K_i(t)\big) + \left(\prod_{j=0}^{N} (t-t_j)^2\right) \mathrm{nn}(\theta; t),$$

where the Hermite basis functions satisfy

$$H_i(t_j) = \delta_{ij}, \quad H_i'(t_j) = 0, \quad K_i(t_j) = 0, \quad K_i'(t_j) = \delta_{ij},$$

and are given explicitly by

$$H_i(t) = \Big(1 - 2(t-t_i)L_i'(t_i)\Big)L_i(t)^2, \qquad K_i(t) = (t-t_i)L_i(t)^2,$$

with

$$L_i(t) = \prod_{j \neq i} \frac{t-t_j}{t_i - t_j}$$

being the Lagrange interpolation polynomial.

(b) Consider the harmonic oscillator

$$u''(t) + u(t) = 0, \qquad u(0) = 0, \quad u'(0) = 1,$$

whose exact solution is $u(t) = \sin t$. Generate synthetic data $\{(u(t_i), u'(t_i))\}$ from this system on a uniform grid in $[0, \pi]$. Select $N = 2, 3, 4, 5$ exact data points and construct the corresponding hard embedding as in part (a). Train a physics-informed neural network (PINN) to approximate the solution $u(t)$ using these embeddings.

(c) Compare the results for different values of N. Discuss how the number of exact data points affects the accuracy, convergence, and stability of the PINN solution.

Exercise 17.0.3 Let

$$y \;=\; f(\theta; x_1, \ldots, x_n)$$

be a neural network with n-dimensional input $x \in \mathbb{R}^n$ and one-dimensional output $y \in \mathbb{R}$. Fix a point $x^0 = (x_1^0, \ldots, x_n^0)$. The linearization of f about x^0 is the affine function

$$L_{x^0}(x) \;=\; f(\theta; x^0) + (x - x^0)^\top \nabla_x f_\theta(x^0).$$

Now suppose that the linearized networks

$$L_{x^k}(x) \;=\; f(\theta; x^k) + (x - x^k)^\top \nabla_x f_\theta(x^k), \qquad k = 1, \ldots, N,$$

are known at N distinct points $x^1, \ldots, x^N \in \mathbb{R}^n$.

(a) Formulate the problem of finding the neural network $f_\theta(x)$ which best matches the given linear approximations $L_{x^k}(x)$.

(b) Show that this can be written as a minimization problem of the form

$$\min_\theta \sum_{k=1}^{N} \left(\big| f(\theta; x^k) - L_{x^k}(x^k) \big|^2 + \big\| \nabla_x f_\theta(x^k) - \nabla_x L_{x^k}(x^k) \big\|^2 \right).$$

(c) Implement a small example in which f_θ is taken to be a two-layer fully connected neural network. Generate synthetic data by choosing a known function $f(x)$ (for example, $f(x_1, x_2) = \sin(x_1 + x_2)$), compute its linearizations L_{x^k} at several points, and train your network to match them. Then compare the trained network's predictions with the original function.

Exercise 17.0.4 Consider the stationary temperature distribution in a bar whose ends $x = \pm 1$ are kept at temperature $y = 1$, and which transfers heat to the environment at temperature $y = 0$ proportionally to $y + \frac{1}{4}y^2$. The governing boundary value problem is

$$y''(x) \;=\; y(x) + \tfrac{1}{4}y(x)^2, \qquad y(-1) = y(1) = 1.$$

(a) Show that the function

$$f(y) = y + \tfrac{1}{4}y^2$$

is not Lipschitz on the interval $0 \le y \le 1$.

(b) Explain why it is nevertheless anticipated that the solution $y(x)$ exists and satisfies $0 \le y(x) \le 1$.

(c) Use a neural network (PINN approach) to approximate the solution $y(x)$. *Hint:* construct a trial solution that satisfies the boundary conditions and embed the NN output in it.

(d) Investigate the possibility of multiple solutions by changing the neural network architecture (e.g. number of layers, hidden units, activation functions). Discuss whether different architectures lead to different admissible solutions.

Exercise 17.0.5 Consider the boundary value problem

$$y''(t) \;=\; t\big(e^{y(t)/t} - 1\big), \qquad y(0) = -1, \qquad y(+\infty) = 0.$$

(a) Show that if the problem has a solution $y(t)$, then it must satisfy

$$-1 \;\le\; y(t) \;\le\; 0, \qquad \text{for all } t \ge 0.$$

(b) Use a neural network (PINN approach) to approximate the solution $y(t)$ subject to the conditions in part (a). *Hint:* construct a trial solution that embeds the boundary conditions and restricts the network output to lie between -1 and 0 (for example, by using a scaled sigmoid activation).

Exercise 17.0.6 Modify Abel's mechanical problem (see Section 16.1) by replacing the motion of a mass under gravity with that of a charged particle of charge q moving in an electric field. Assume the electric field is generated by a potential

$$U(y) = \frac{1}{y},$$

and that the particle moves from a point p_1 at height y_1 down to a point p_0 at height $y_0 > 0$, measured from a reference line along which an electric current circulates.

(a) Derive the new Abel-type integral equation that governs the descent time of the particle under the action of the electric potential.

(b) Solve this equation both numerically and using neural networks in a particular case (by specifying y_0, y_1 and q).

Exercise 17.0.7 Given data points (q_i, q_i') representing the trajectory in phase-space, one can use neural networks to learn both the Lagrangian trajectory $q(t)$ and a candidate Lagrangian $L(q, q')$ whose Euler–Lagrange equations generate a solution that best fits the observed data.

The learned Lagrangian $L(q, q')$ is not necessarily of the natural form (kinetic energy minus potential energy). To address this, consider introducing a neural network

$$F_\theta(q, q') = (\tilde{q}, \tilde{q}'),$$

which performs a coordinate transformation in phase space.

(a) Train the network F_θ such that the transformed data $(\tilde{q}_i, \tilde{q}_i')$ can then be used to learn a natural Lagrangian of the form

$$\tilde{L}(\tilde{q}, \tilde{q}') = \tfrac{1}{2}\tilde{q}'^2 - U(\tilde{q}),$$

so that the learning problem reduces to identifying the potential function $U(\tilde{q})$.

(b) Applying the inverse transformation F_θ^{-1}, pull back $\tilde{L}$ and recover a corresponding Lagrangian $L(q, q')$ in the original variables.

Exercise 17.0.8 Consider the stationary diffusion equation

$$\operatorname{div}\big(a(x)\,\nabla u(x)\big) = 0, \qquad x \in \Omega,$$

with boundary conditions

$$u|_{\partial\Omega} = \phi, \qquad \frac{\partial u}{\partial n}\Big|_{\partial\Omega} = \psi.$$

Explain how neural networks can be employed to reconstruct the coefficient $a(x)$ from the boundary data (ϕ, ψ) together with interior measurements $(x_j, u(x_j))$. In particular, discuss how to embed the PDE and boundary conditions into the loss function, and how to train the network so that the learned coefficient $a(x)$ best matches the observed data.

Exercise 17.0.9 Let $M \subset \mathbb{R}^3$ be a smooth surface. Pick N points $x_1, \dots, x_N \in M$, and let $T_{x_k}M$ denote the tangent planes to M at these points. Suppose we "erase" the surface M but retain the anchor points x_k together with their tangent planes. The problem is: *how can one reconstruct (approximately) the surface M from this discrete data?*

We suggest the following neural implicit function approach, by representing the surface implicitly by a neural network $f_\theta : \mathbb{R}^3 \to \mathbb{R}$, such that the surface is given as the zero level set

$$M_\theta \;=\; \{x \in \mathbb{R}^3 \;:\; f_\theta(x) = 0\}.$$

The function f_θ is often trained to approximate a signed distance function (SDF), where $f_\theta(x) > 0$ outside the surface and $f_\theta(x) < 0$ inside.

At each anchor point x_k we require the following constraints:

$$f_\theta(x_k) = 0, \qquad \nabla f_\theta(x_k) \parallel n_k = 0,$$

where n_k is the unit normal vector to the tangent plane $T_{x_k}M$. If f_θ is an SDF, we also want $\|\nabla f_\theta(x_k)\| = 1$.

These conditions can be enforced by the composite loss

$$\mathcal{L}(\theta) = \lambda_1 \mathcal{L}_{\text{point}} + \lambda_2 \mathcal{L}_{\text{normal}} + \lambda_3 \mathcal{L}_{\text{eik}},$$

with

$$\mathcal{L}_{\text{point}} = \frac{1}{N}\sum_{k=1}^{N} |f_\theta(x_k)|^2,$$

$$\mathcal{L}_{\text{normal}} = \frac{1}{N}\sum_{k=1}^{N} \|\nabla f_\theta(x_k) - n_k\|^2,$$

$$\mathcal{L}_{\text{eic}} = \frac{1}{N}\sum_{k=1}^{N} \big(\|\nabla f_\theta(x_k)\| - 1\big)^2.$$

For training we do the following:

1. Sample minibatches of the anchor points $\{x_k\}$.
2. Compute $f_\theta(x_k)$ and $\nabla f_\theta(x_k)$ using automatic differentiation.
3. Minimize $\mathcal{L}(\theta)$ with an optimizer such as Adam.

After training, the reconstructed surface is obtained as the zero level set

$$M_\theta = \{x \in \mathbb{R}^3 : f_\theta(x) = 0\},$$

which can be extracted numerically.

Exercise 17.0.10 (Localizing the strike on a circular gong). A homogeneous and isotropic circular copper plate of radius R and thickness h vibrates freely after being struck at an unknown point x^* inside the disk $D_R = \{x \in \mathbb{R}^2 : \|x\| < R\}$. Neglecting damping and coupling to air, the transverse displacement $u(t, x)$ satisfies the Kirchhoff–Love plate equation

$$\partial_t^2 u(t, x) + \Delta^2 u(t, x) \;=\; 0, \qquad (t, x) \in (0, \infty) \times D_R,$$

with free-edge boundary conditions on ∂D_R (zero bending moment and effective shear), and initial data

$$u(0, x) = 0, \qquad \partial_t u(0, x) = \delta(x - x^*).$$

Assume the recorded sound provides access to all plate *eigenfrequencies* $\{\nu_k\}_{k\geq 1}$ and to the modal amplitudes (up to a global factor) excited by the strike. Design a neural network that maps the vector of measured spectral features to x^*.

Exercise 17.0.11 (Learning the heat kernel on the sphere $\mathbb{S}^2$). Consider the heat equation on the unit sphere $\mathbb{S}^2$:

$$(\partial_t - \Delta_{\mathbb{S}^2})K(t;\varphi,\phi) = 0, \qquad t > 0,\ (\varphi,\phi) \in (0,\pi) \times [0,2\pi),$$

with initial condition $K(0;\varphi,\phi) = \delta_{(\varphi,\phi)=(0,0)}$ (a point source at the north pole) and periodicity $K(t;\varphi,\phi+2\pi) = K(t;\varphi,\phi)$. Here $\Delta_{\mathbb{S}^2}$ is the Laplace–Beltrami operator in spherical angles

$$\Delta_{\mathbb{S}^2} f = \frac{1}{\sin\varphi}\,\partial_\varphi\big(\sin\varphi\,\partial_\varphi f\big) + \frac{1}{\sin^2\varphi}\,\partial_\phi^2 f.$$

(a) Parametrize the sphere by colatitude $\varphi \in [0,\pi]$ and longitude $\phi \in [0,2\pi)$. Use the periodic embedding

$$K_\theta(t;\varphi,\phi) = \delta_\varepsilon(\varphi)\,\delta_\varepsilon(\phi) \;+\; t\,\mathrm{NN}_\theta\big(t,\cos\varphi,\sin\varphi,\cos\phi,\sin\phi\big),$$

where δ_ε is a narrow mollifier and the trigonometric features enforce periodicity in ϕ and regularity near the poles.

(b) Using automatic differentiation, implement the residual

$$\mathcal{R}(t,\varphi,\phi) := \partial_t K_\theta(t;\varphi,\phi) \;-\; \Delta_{\mathbb{S}^2} K_\theta(t;\varphi,\phi).$$

Train NN_θ by minimizing the PDE loss over interior collocation points (t_i,φ_j,ϕ_k) with $t_i > 0$:

$$\mathcal{L}_{\mathrm{PDE}}(\theta) = \frac{1}{N}\sum_{i,j,k}\big(\mathcal{R}(t_i,\varphi_j,\phi_k)\big)^2.$$

Augment the objective with:

$$\mathcal{L}_{\mathrm{IC}}(\theta) = \|K_\theta(0;\cdot,\cdot) - \delta_\varepsilon\|^2_{L^2(\mathbb{S}^2)}, \qquad \mathcal{L}_{\mathrm{cons}}(\theta) = \frac{1}{N_t}\sum_{i=1}^{N_t}\Big(\int_{\mathbb{S}^2} K_\theta(t_i;\varphi,\phi)\,d\sigma - 1\Big)^2,$$

and minimize the total loss $\mathcal{L}_{\mathrm{tot}} = \mathcal{L}_{\mathrm{PDE}} + \lambda_{\mathrm{IC}}\mathcal{L}_{\mathrm{IC}} + \lambda_{\mathrm{cons}}\mathcal{L}_{\mathrm{cons}}$.

(c) The exact heat kernel on $\mathbb{S}^2$ with source at the north pole depends only on the geodesic angle $\gamma = \varphi$ and admits the spectral expansion

$$K_{\mathrm{exact}}(t;\varphi) = \sum_{\ell=0}^{\infty} \frac{2\ell+1}{4\pi}\,e^{-\ell(\ell+1)t}\,P_\ell(\cos\varphi),$$

where P_ℓ are the Legendre polynomials. Truncate the sum at $\ell = \ell_{\max}$ (e.g. $\ell_{\max} \in [40,100]$ depending on t) and use it to compute $K_{\mathrm{exact}}(t;\varphi)$.

(d) Train your network on $t \in (0,T]$ (e.g. $T = 1$). Plot $K_\theta(t;\varphi,0)$ versus $K_{\mathrm{exact}}(t;\varphi)$ for $t \in \{0.05, 0.1, 0.2, 0.5, 1.0\}$ and $\varphi \in [0,\pi]$.

Exercise 17.0.12 (The Inverse Laplace Transform). Given a function $F(s)$ for $s > 0$, we shall learn a neural approximation $f_\theta(t)$ so that $\mathcal{L}\{f_\theta\}(s) \approx F(s)$ and hence $f_\theta \approx \mathcal{L}^{-1}\{F\}$ on $t \in [0,T]$ represents the inverse Laplace transform.

(a) We prepare test pairs and sampling: Select analytic pairs (f, F), such as $(e^{-at}, 1/(s+a))$, $(t^n e^{-at}, n!/(s+a)^{n+1})$, $(\sin(bt)e^{-at}, b/[(s+a)^2+b^2])$. Sample $t \in [0, T]$ on a grid $\{t_j\}$ and $s \in (0, S_{\max})$ on points $\{s_k\}$.

(b) Use a scalar neural network $h_\theta(t)$ (with smooth activations, e.g. Swish, tanh) and consider the hard embedding

$$f_\theta(t) = \mathrm{softplus}(h_\theta(t))\, e^{-\alpha t}, \qquad \alpha > 0.$$

(c) Define a discrete Laplace transform

$$\widehat{\mathcal{L}}\{f_\theta\}(s_k) = \sum_j f_\theta(t_j)\, e^{-s_k t_j}\, \Delta t,$$

and minimize the loss

$$\mathcal{L}_{\mathrm{Lap}}(\theta) = \frac{1}{K} \sum_{k=1}^{K} \left| \widehat{\mathcal{L}}\{f_\theta\}(s_k) - F(s_k) \right|^2.$$

(d) For synthetic cases with known f, add a small supervised term on a subset $\mathcal{J}$:

$$\mathcal{L}_{\mathrm{sup}}(\theta) = \frac{1}{|\mathcal{J}|} \sum_{j \in \mathcal{J}} |f_\theta(t_j) - f(t_j)|^2.$$

Train with $\mathcal{L} = \mathcal{L}_{\mathrm{Lap}} + \lambda_{\mathrm{sup}} \mathcal{L}_{\mathrm{sup}}$.

(e) For evaluation report L^2/L^∞ errors on f (for synthetic pairs). Check consistency by recomputing $\mathcal{L}\{f_\theta\}$ at unseen s and comparing to $F(s)$. Plot $f_\theta(t)$ versus ground truth and study robustness under noise added to F.

Exercise 17.0.13 (Rediscovering Kepler's Laws). We simulate planetary motion around the Sun and use neural networks to rediscover Kepler's laws.

The dynamics is described by Newton's law. Let $\mathbf{r}(t) = (x(t), y(t))$, $r = \|\mathbf{r}\|$, and $\mu = G(M+m) \approx GM$. The equations of motion are

$$\ddot{\mathbf{r}}(t) = -\mu \frac{\mathbf{r}(t)}{r(t)^3}, \qquad \ddot{x} = -\mu \frac{x}{(x^2+y^2)^{3/2}}, \quad \ddot{y} = -\mu \frac{y}{(x^2+y^2)^{3/2}},$$

with $\mu > 0$ constant.

(a) For data simulation integrate with a numerical scheme for N bound orbits by varying initial $(\mathbf{r}_0, \mathbf{v}_0)$ and keeping a fixed value for μ. Record $(x, y, \dot{x}, \dot{y})$ over multiple periods and split into training/validation/test sets.

(b) Train a network to fit conic parameters through the positions data. Verify that we obtain ellipses with eccentricity $e < 1$ and that the Sun is at a focus.

(c) Compute the areal velocity $\dot{A}(t) = \frac{1}{2}|x\dot{y} - y\dot{x}|$. Learn a conserved quantity $I_\theta(x, y, \dot{x}, \dot{y})$ by minimizing $\int (\frac{d}{dt} I_\theta)^2 dt$; show correlation with $|x\dot{y} - y\dot{x}|$ and quantify the constancy of $\dot{A}(t)$.

(d) Estimate period T and semimajor axis a for each orbit. Fit $\log T$ vs. $\log a$ with a network or linear head to recover $T \approx Ca^\alpha$; report the learned exponent $\alpha \approx 3/2$.

(e) Add a residual penalty

$$\mathcal{L}_{\text{phys}} = \frac{1}{K} \sum_{k} \left\| \ddot{\mathbf{r}}_k + \mu \frac{\mathbf{r}_k}{\|\mathbf{r}_k\|^3} \right\|^2$$

using predicted states to encourage adherence to the central-force law.

(f) Estimate μ from data; use a Hamiltonian Neural Network to learn H and verify energy conservation; add small perturbations and analyze deviations from Keplerian behavior.

Bibliography

[1] C. Aarset, M. Holler, and T. T. N. Nguyen. Learning-informed parameter identification in nonlinear time-dependent pdes. *Applied Mathematics & Optimization*, 88(3):76, 2023.

[2] N. H. Abel. Auflösung einer mechanischen aufgabe. *Journal für die reine und angewandte Mathematik*, 1:153–157, 1826.

[3] J. Adler and O. Öktem. Learned primal-dual reconstruction. *IEEE Transactions on Medical Imaging*, 37(6):1322–1332, 2018.

[4] M. R. Alhafiz, L. R. Zuhal, D. V. Dung, and P. S. Palar. An explainable deep learning for data-driven turbulence model feature discovery. *IEEE Access*, 2025.

[5] H. W. Alt and L. A. Caffarelli. *Existence and Regularity for a Minimum Problem with Free Boundary*. Annals of Mathematics Studies. Springer, 1981.

[6] I. V. Andrianov and V. V. Danishevskyy. Asymptotic analysis of perforated plates and membranes. part 2: Static and dynamic problems for large holes. *International Journal of Engineering Science*, 50(1):93–106, 2012.

[7] I. G. Avramidi. *Heat Kernel Method and its Applications*. Birkhauser, New York, 2015.

[8] H. T. Banks and K. Kunisch. *Estimation Techniques for Distributed Parameter Systems*, volume 1 of *Progress in Systems and Control Theory*. Birkhäuser, Boston, 1989.

[9] M. Barros, O. J. Garay, and D. A. Singer. Elasticae with constant slant in the complex projective plane and new examples of Willmore tori in five-spheres. *Tohoku Mathematical Journal*, 51(2):177–192, 1999.

[10] J. Baumeister. *Stable Solutions of Inverse Problems*. Vieweg Teubner Verlag, Wiesbaden, 1987.

[11] G. Borg. Eine umkehrung der Sturm–Liouvilleschen eigenwertaufgabe. *Acta Mathematica*, 78:1–96, 1946.

[12] W. L. Briggs. *A Multigrid Tutorial*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 1987.

[13] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I.a Sutskever, and D. Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 1877–1901, 2020.

[14] T. M. Buzug. *Computed Tomography: From Photon Statistics to Modern Cone-Beam CT*. Springer, Berlin, 2008.

[15] O. Calin. *Deep Learning Architectures - A Mathematical Approach*. Springer, Springer Series in the Data Sciences, New York, 2020.

[16] O. Calin, D. C. Chang, K. Furutani, and C. Iwasaki. *Heat Kernels for Elliptic and Sub-elliptic Operators*. Birkhauser, Applied and Numerical Harmonic Analysis, 2011.

[17] O. Calin and D.C. Chang. *Geometric Mechanics on Riemannian Manifolds*. Birkhauser, Applied and Numerical Harmonic Analysis, Boston, 2004.

[18] O. Calin, D.C. Chang, and P. Greiner. *Geometric Analysis on the Heisenberg Group and Its Generalizations*. AMS/IP, 2007.

[19] G. Carleo, I. Cirac, K. Cranmer, L. Daudet, M. Schuld, N. Tishby, L. Vogt-Maranto, and L. Zdeborova. Machine learning and the physical sciences. *Reviews of Modern Physics*, 91(4):045002, 2019.

[20] S. Cen, B. Jin, Q. Quan, and Z. Zhou. Numerical analysis of unsupervised learning approaches for parameter identification in pdes. *arXiv preprint arXiv:2508.15381*, August 2025.

[21] H. Chen, Y. Zhang, M. K. Kalra, F. Lin, Y. Chen, P. Liao, J. Zhou, and G. Wang. Low-dose CT with a residual encoder-decoder convolutional neural network. *IEEE Transactions on Medical Imaging*, 36(12):2524–2535, 2017.

[22] T. P. Chen and H. Chen. Approximations of continuous functionals by neural networks with application to dynamic systems. *IEEE Transactions on Neural Networks*, 4(6):910–917, 1993.

[23] T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, 6571–6583, 2018.

[24] El-H. Chouzenoux, C. Della V., and J.-C. Pesquet. Inversion of integral models: a neural network approach. *arXiv preprint*, 2021.

[25] D. Christodoulou. The Euler equations of compressible fluid flow. *Bulletin of the American Mathematical Society*, 44(4):581–602, 2007.

[26] P. Constantin, D. Córdoba, F. Gancedo, L. Rodriguez-Piazza, and R. M. Strain. On the Muskat problem: Global in time results in 2d and 3d. *American Journal of Mathematics*, 138(6):1455–1494, 2016.

[27] R. Courant and D. Hilbert. *Methods of Mathematical Physics, Vol. I*. Interscience Publishers, New York, 1953. Reprinted by Wiley/Wiley-VCH, e.g. 1989.

[28] J. Crank. *Free and Moving Boundary Problems*. Oxford University Press, Oxford, 1984.

[29] M. Cranmer, S. Greydanus, S. Hoyer, P. Battaglia, D. Spergel, and S. Ho. Lagrangian neural networks. *arXiv preprint arXiv:2003.04630*, 2020.

[30] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.

[31] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 4171–4186, Minneapolis, Minnesota, 2019. Association for Computational Linguistics.

[32] A. Dong, A. Starr, and Y. Zhao. Neural network-based parametric system identification: a review. *International Journal of Systems Science*, 54(13):2676–2688, 2023.

[33] G. Dziuk, E. Kuwert, and R. Schätzle. Evolution of elastic curves in $\mathbb{R}^n$: Existence and computation. *SIAM Journal on Mathematical Analysis*, 33(5):1228–1245, 2002.

[34] A. Friedman. *Partial Differential Equations of Parabolic Type*. Prentice-Hall, Englewood Cliffs, New Jersey, 1964. Dover reprint 2008.

[35] A. Friedman. *Variational Principles and Free Boundary Value Problems*. Wiley-Interscience, New York, 1982.

[36] W. Fulks and R. B. Guenther. A free boundary problem and an extension of Muskat's model. *Applicable Analysis*, 8:177–192, 1969.

[37] I. M. Gel'fand and B. M. Levitan. On the determination of a differential equation from its spectral function. *Izvestiya Akademii Nauk SSSR. Seriya Matematicheskaya*, 15:309–360, 1951.

[38] D. Gilbarg and N. S. Trudinger. *Elliptic Partial Differential Equations of Second Order*. Springer-Verlag, Berlin, classics in mathematics edition, 2001. Reprint of the 1998 edition.

[39] R. González-García, R. Rico-Martínez, and I. G. Kevrekidis. Identification of distributed parameter systems: A neural net based approach. *Computers and Chemical Engineering*, 22(SUPPL.1):S965–S968, 1998.

[40] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.

[41] R. Gorenflo and S. Vessella. *Abel Integral Equations: Analysis and Applications*, volume 1461 of *Lecture Notes in Mathematics*. Springer-Verlag, Berlin, 1991.

[42] S. Greydanus, M. Dzamba, and J. Yosinski. Hamiltonian neural networks. In *NeurIPS*, 2019.

[43] R. B. Guenther and J. W. Lee. *Partial Differential Equations of Mathematical Physics and Integral Equations*. Prentice Hall, Englewood Cliffs, NJ, 1988.

[44] J. He, Y. Yang, Y. Wang, D. Zeng, Z. Bian, J. Sun, and Z. Xu. Optimizing a parameterized plug-and-play ADMM for iterative low-dose CT reconstruction. *IEEE Transactions on Medical Imaging*, 38(2):371–382, 2019.

[45] M. Hénon and C. Heiles. The applicability of the third integral of motion: Some numerical experiments. *Astronomical Journal*, 69:73–79, 1964.

[46] G. T. Herman. *Fundamentals of Computerized Tomography: Image Reconstruction from Projections*. Springer, London, 2009.

[47] G. A. Holzapfel. *Nonlinear Solid Mechanics: A Continuum Approach for Engineering*. Wiley, 2000.

[48] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[49] A. Hosseinian et al. An efficient numerical scheme to solve generalized Abel's delay integral equations. *Journal of Computational and Applied Mathematics*, 441:115957, 2024.

[50] K. H. Jin, M. T. McCann, E. Froustey, and M. Unser. Deep convolutional neural network for inverse problems in imaging. *IEEE Transactions on Image Processing*, 26(9):4509–4522, 2017.

[51] V. Jurdjevic. Non-euclidean elastica. *American Journal of Mathematics*, 117(1):93–124, 1995.

[52] A. C. Kak and M. Slaney. *Principles of Computerized Tomographic Imaging*. Society for Industrial and Applied Mathematics, Philadelphia, 2001.

[53] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3:422–440, 2021.

[54] D. Kinderlehrer and G. Stampacchia. *An Introduction to Variational Inequalities and Their Applications*. SIAM, 1980.

[55] A. Kirsch. *An Introduction to the Mathematical Theory of Inverse Problems*, volume 120 of *Applied Mathematical Sciences*. Springer, New York, 1996.

[56] C. Kravaris and J. H. Seinfeld. Identification of parameters in distributed parameter systems by regularization. *SIAM Journal on Control and Optimization*, 23(2):217–241, 1985.

[57] I. E. Lagaris, A. Likas, and D. I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998.

[58] L. D. Landau and E. M. Lifshitz. *Fluid Mechanics*. Course of Theoretical Physics, Vol. 6. Pergamon Press, 1987.

[59] H. Li, J. Schwab, S. Antholzer, and M. Haltmeier. NETT: Solving inverse problems with deep neural networks. *Inverse Problems*, 36(6):065005, 2020.

[60] Z. Li and G. Hu. New approach of the solution of physical fields of fluid dynamics: Physics-informed long short-term memory network. *International Journal of Heat and Fluid Flow*, 2026.

[61] Z. Li, N. B. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. M. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[62] H. W. Lin, M. Tegmark, and D. Rolnick. Why does deep and cheap learning work so well? *Journal of Statistical Physics*, 168(6):1223–1247, 2017.

[63] X. Liu, Z. Tang, and J. Wei. Temporal convolutional network based on squeeze-and-excitation module and multi-scale feature fusion for gnss positioning error correction. *GPS Solutions*, 2025.

[64] L. Lu, P. Jin, G. Pang, Z. Mao, M. Cai, et al. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.

[65] M. Lutter, C. Ritter, and J. Peters. Deep Lagrangian Networks: Using Physics as Model Prior for Deep Learning. *arXiv preprint arXiv:1907.04490*, 2019.

[66] X. F. Ma et al. Asymmetric Abel inversion by neural network for plasma density reconstruction via a multilayer neural network. *Nuclear Instruments and Methods in Physics Research Section A*, 489(2–3):490–500, 2002.

[67] V. A. Marchenko. Some questions of the theory of one-dimensional linear differential operators of the second order. *Trudy Moskovskogo Matematicheskogo Obshchestva*, 1:327–420, 1952.

[68] J. E. Marsden and T. J. R. Hughes. *Mathematical Foundations of Elasticity*. Dover Publications, 1994.

[69] B. V. Matioc. The Muskat problem in two dimensions: Equivalence of formulations, well-posedness, and regularity results. *Analysis and PDE*, 12(2):281–318, 2018.

[70] A. Meirmanov. The Muskat problem for a viscoelastic filtration. *Interfaces and Free Boundaries*, 13(4):485–502, 2011.

[71] A. M. Meirmanov. *The Stefan Problem*. Walter de Gruyter, Berlin, 1992.

[72] S. G. Mikhlin. *Linear Partial Differential Equations*. North-Holland, Amsterdam, 1967. English translation of the Russian original *Lineinye uravneniya v chastnykh proizvodnykh*.

[73] D. A. Murio. *The Mollification Method and the Numerical Solution of Ill-Posed Problems*. John Wiley & Sons, New York, 1988.

[74] M. Muskat. A note on mixed boundary value problems in logarithmic potential theory. *Journal of Applied Physics*, 6(1):27–30, 1935.

[75] B. Nadler and D. J. Steigmann. Modeling the indentation, penetration and cavitation of elastic membranes. *Journal of the Mechanics and Physics of Solids*, 54(6):1271–1294, 2006.

[76] F. Natterer. *The Mathematics of Computerized Tomography*. Wiley and Teubner, Stuttgart, 1986.

[77] J. Prüss and G. Simonett. On the Muskat flow. *arXiv preprint*, 2016.

[78] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners.

[79] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[80] G. R. Richter. Numerical identification of a spatially varying diffusion coefficient. *Mathematics of Computation*, 36(154):567–589, 1981.

[81] C. J. G. Rojas, J. L. Boldrini, and M. L. Bittencourt. Parameter identification for a damage phase field model using a physics-informed neural network. *Theoretical and Applied Mechanics Letters*, 13(2):100450, 2023.

[82] L. I. Rubinstein. *The Stefan Problem*. American Mathematical Society, 1971.

[83] Alvaro Sanchez-Gonzalez, Victor Bapst, Kyle Cranmer, and Peter Battaglia. Hamiltonian graph networks with ode integrators. *arXiv preprint arXiv:1909.12790*, 2019.

[84] M. Sellier. An iterative method for the inverse elasto-static problem. *Journal of Fluids and Structures*, 27(8):1280–1288, 2011.

[85] D. A. Singer. Lectures on elastic curves and rods. *AIP Conference Proceedings*, 1002:3–32, 2008. Lecture/tutorial on Euler elastica and Kirchhoff rods.

[86] C. Sparling, A. Ruget, J. Leach, and D. Townsend. Arbitrary image reinflation: A deep learning technique for recovering 3d photoproduct distributions from a single 2d projection. *Review of Scientific Instruments*, 93(2):023303, 2022.

[87] J. Stefan. Über die theorie der eisbildung, insbesondere über die eisbildung im polarmeere. *Annalen der Physik und Chemie*, 42:269–286, 1889.

[88] A. M. Tartakovsky, C. O. Marrero, P. Perdikaris, G. D. Tartakovsky, and D. Barajas-Solano. Learning parameters and constitutive relationships with physics-informed deep neural networks. *arXiv preprint arXiv:1808.03398*, 2018.

[89] The Royal Swedish Academy of Sciences. The Nobel Prize in Chemistry 2024 – press release. https://www.nobelprize.org/prizes/chemistry/2024/press-release/, 2024. D. Hassabis, J. Jumper, and D. Baker "for protein structure prediction and computational protein design."

[90] The Royal Swedish Academy of Sciences. The Nobel Prize in Physics 2024 – press release. https://www.nobelprize.org/prizes/physics/2024/press-release/, 2024. J. Hopfield and G. Hinton "for foundational discoveries and inventions that enable machine learning with artificial neural networks."

[91] N. Thuerey, B. Holzschuh, P. Holl, G. Kohl, M. Lino, Q. Liu, P. Schnell, and F. Trost. *Physics-based Deep Learning*. WWW, 2021.

[92] A. N. Tikhonov and A. A. Samarskii. *Equations of Mathematical Physics*. Pergamon Press, Oxford, 1963. English translation; reprinted by Dover, 1990.

[93] P. Toth, D. J. Rezende, A. Jaegle, S. Racanière, A. Botev, and I. Higgins. Hamiltonian generative networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2019.

[94] H. Trac and U.-L. Pen. A primer on Eulerian computational fluid dynamics for astrophysics. *Publications of the Astronomical Society of the Pacific*, 115(811):303–321, 2003.

[95] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 5998–6008, 2017.

[96] A. Visintin. *Models of Phase Transitions*. Birkhäuser, Boston, 1996.

[97] V. S. Vladimirov. *Equations of Mathematical Physics*. Marcel Dekker, New York, 1971. English translation of the Russian original.

[98] Y. Wang. Comparison of the effectiveness of extended Kalman filter and machine learning algorithms in sensor fusion. *Proceedings of The 4th International Conference on Artificial Intelligence and Advanced Manufacturing*, 2024.

[99] G. M. Wing. *A Primer on Integral Equations of the First Kind: The Problem of Deconvolution and Unfolding*. Society for Industrial and Applied Mathematics, Philadelphia, 1992.

[100] J. Xue. Research on data fusion algorithm of gps and imu based on Kalman filter, machine learning, and deep learning algorithms. *Proc. SPIE 13561, Communication Technology, Image Engineering, and Electron Materials (CTIEEM 2024)*, 2025.

[101] Y. Yao, B. Hu, C. Wang, and J. Fan. Prediction and optimization of gas turbine secondary air system cooling efficiency based on deep learning. *Engineering Applications of Computational Fluid Mechanics*, 2025.

[102] J. C. Ye. *Geometry of Deep Learning: A Signal Processing Perspective*, vol. 37 of *Mathematics in Industry*. Springer Nature Singapore, 2022.

[103] F. Yi. Global classical solution of Muskat free boundary problem. *Journal of Mathematical Analysis and Applications*, 288(1):442–461, 2003.

[104] B. Zhu, J. Z. Liu, S. F. Cauley, B. R. Rosen, and M. S. Rosen. Image reconstruction by domain-transform manifold learning. *Nature*, 555:487–492, 2018.

Index

Abel's
    integral equation, 493
    transform, 497
accelerometer, 508
action, 464
action integral, 238
activation functions, 4
Adam, 20
Adam optimizer, 384, 402, 448, 452
adiabatic process, 364
algebraic constraint, 31
astroid, 476
attention, 480
attention block, 484
attention distribution, 471
attention layer, 475
attention map, 475
attention-weighted potential, 486
attention-weighted sum, 484
autonomous equation, 405
average entropy, 477

backpropagation, 10, 20
baseline network, 448, 454, 464
Bernoulli lemniscate, 476
BERT, 470
Bessel function, 473
bias vector, 5
biharmonic operator, 307
boundary conditions, 499
brachistochrone, 165
brachistocrone, 161

canonical transformations, 196
capacity, 5
catenary, 156
catenary curve, 233
Cauchy
    functional equation, 32
    problem, 289, 299, 310, 320, 434
Cauchy's inequality, 253
Cauchy–Green tensor, 276, 285
chain rule, 9, 10
charged particle, 458
circular motion, 261
classical Hamiltonian, 187
classical solution, 311, 319
classifying conics, 488
coarse solution, 90
coefficient matrix, 289
coefficient regression, 488
complex conjugate, 21
complex root, 19
computer tomography, 502
conservation law, 343, 347
conservation laws, 15, 207
conservation loss, 382
conservation of energy, 346
conservation of mass, 351
conservation of momentum, 352, 356
constant force, 454
constraint manifold, 487
Contraction Mapping Principle, 44
convective acceleration, 356
coordinates change, 139
cosine approximation, 399
curl vector, 358
curvature, 475
cycloid, 488

d'Alembert layer, 435
d'Alembert operator, 290, 323
d'Alembert's formula, 311, 435
Darcy's law, 269
data loss, 367
data-driven learning, 421

dataset, 4
decision layer, 488
degrees of freedom, 187, 468
density normalization, 499
depth, 5
Descartes' folium, 488
diffusive coupling, 482
Dirac delta, 290, 420, 502
Dirichlet
  boundary condition, 278, 291
  boundary conditions, 419, 497
  boundary value problem, 295
  condition, 290, 296, 319
  eigenvalue problem, 91
  energy functional, 293
  problem, 290, 431, 436
divergence residual, 357
domain sampling, 35
dynamical systems, 119

early stopping, 14
efficiency, 7
eiconal equation, 253, 262
eigenpair, 91
eigenvalues, 498
elastica, 175
electromagnetic field, 322
electrostatics, 295
ellipse, 475
elliptic equations, 290
energy
  density, 277
  functional, 276
  level, 448
  variation, 280
energy-drift regularization, 484
energy-variance regularization, 485
entropy, 471
equation discovery, 333, 338
equation of state, 352
equation residual, 357
equilibrium boundary condition, 272
equilibrium configuration, 278
Euclidean
  gradient, 446
  metric, 275
  plane, 276
  scalar product, 225
Euler's equation, 352, 356, 362
Euler's method, 19, 76
Euler–Bernoulli
  bending energy, 182
  functional, 175
Euler–Lagrange equation, 153, 156, 172, 289, 409, 410, 456
Euler–Lagrange equations, 232
evolution problem, 278
exploding gradients, 11
exponential decay, 147
exponential integral equation, 51
expressivity, 7
exterior Dirichlet problem, 298
external body forces, 356
external forces, 362

Fermat's principle, 169
finite-difference approximation, 77
flow, 126
fluid dynamics, 351
force field, 336
Fourier layer, 426
Fourier loss, 507
Fourier Slice Theorem, 506
Fourier transform, 506
Fredholm
  equation, 42
  integral, 42, 44, 90
free-boundary problem, 263
frequency estimation, 378
Frobenius norm, 277, 442
functional equation, 31
functional invariance, 221

Galilean transforms, 140
Gauss' divergence theorem, 355
Gaussian contributions, 425
Gaussian kernel, 51
Gaussian noise, 448, 466, 488, 509
Gaussian random variable, 443
general Lagrangian case, 410
generating function, 200, 467
Generating-Function Networks, 467

geodesic, 253
geodesic motion, 487
GPS data, 509
GPS receiver, 508
GPS–INS sensor, 507
GPT family, 470
Gram-Schmidt, 93
Green's formula, 281
Green's function, 291
ground-truth level, 448
group of diffeomorphisms, 223, 224

Hénon–Heiles Hamiltonian, 206
Hadamard product, 11
Hamilton–Jacobi equation, 237, 252, 261, 465, 467
Hamilton–Jacobi Neural Networks, 464
Hamiltonian
  function, 187
  neural networks, 5
  system, 187, 196
Hamiltonian Neural Network, 448
Hamiltonian Neural Networks, 445, 482
hard constraints, 15
hard embedding, 177, 207, 378, 380, 382, 384
harmonic gradient field, 215
harmonic oscillator, 154, 190, 205, 206, 208, 231, 347, 378, 448, 457
harmonicity conditions, 270
heat equation, 232, 306, 433
heat kernel, 383, 419, 424, 433
heat operator, 298
Heisenberg group, 458
Helmholtz decomposition, 215
Helmholtz equation, 395
Hermite
  interpolant, 179
  operator, 424
Hermite interpolant, 179
hidden layers, 4
hot spots, 470, 475
hybrid approach, 22
hybrid learning, 65
hybrid method, 76
hyperbola, 488
hyperbolic equations, 310
hyperbolic oscillator, 249
hypersurface, 487

ideal gas law, 363
ideal pendulum, 461
ill-posed, 43
incompressibility, 215
incompressibility condition, 364
incompressible flow, 356
information method, 479
informational observers, 490
initial-condition loss, 53
input layer, 4
integral curve, 122
integral curves, 106, 224, 354
integral equation, 42
integral formulation, 124
internal parameter learning, 373
internal trainable parameter, 378, 380
invariance symmetries, 221
inverse function method, 22
inverse problem, 371
inverted cycloid, 494
invertible function, 22
inverting integrals, 80
isochrone curve, 494

Jacobian, 109, 443
Jensen–Shannon divergence, 480

Kalman filter, 508
Kepler's problem, 148, 261
Keplerian orbits, 149
key, 474
Kirchhoff's current law, 393
Kirchhoff's formula, 311, 435
Klein–Gordon equation, 394
Kullback–Leibler divergence, 478

labels, 22
Lagrange
  multipliers, 121, 183
  polynomial, 132
Lagrangian, 191
  function, 153
  mechanics, 153

neural networks, 5
Lagrangian Neural Networks, 455
Laguerre polynomials, 104
Lambert function, 31
Laplace
equation, 291, 431
operator, 290
problem, 291
Laplacian operator, 362
lateral boundary, 299
learnability, 7
learnable parameters, 5
learning-rate schedule, 9
Legendre transform, 191, 238, 252
lemniscate, 476
length estimation, 379
Levi–Civita connection, 252
linear oscillator, 203, 258
Lipschitz condition, 53, 106, 122
Lorentz's equation, 193
loss function, 36
Lotka–Volterra system, 386
low-capacity approximation, 83

Markov process, 452
Markovian, 441
mass flux, 355
mass-spring system, 460
material derivative, 361
matrix invariants, 276
Maxwell equations, 323
mean-squared residual, 52
mechanical oscillator, 397
method of characteristics, 106
metric parameterization, 486
mixed problem, 298, 300, 319
Muskat problem, 268

natural Hamiltonian, 414, 447
natural Lagrangian, 410
Navier–Stokes equations, 362
Neural ODEs, 441, 480
neuron, 5
Newton's
cooling law, 65, 371
equation, 135, 336
second law, 80, 187, 362
Newtonian gravity, 295
NLP, 470
no-flux boundary conditions, 382
Noether's theorem, 224, 225, 227, 232
noise regularization, 509
nonholonomic constraint, 226, 458
nonlinear pendulum, 452
normalization loss, 423
normalization property, 429
numerical methods, 145

observer, 471
Occam's razor, 257
ODE, 52
one-phase Stefan problem, 263
optimal observer, 477
orthogonality prior, 324
oscillatory source, 84
output layer, 4

parabola, 476
particle trajectory, 124
penalty term, 49
pendulum, 349
permutation-invariant, 31
phase-space, 229, 455
phase-space flow, 446
Picard's iteration, 44
PINNs, 4, 499
Poincaré one-form, 238
Poisson equation, 290, 292
Poisson integral, 432
Poisson kernel, 51
Poisson's
equation, 295
formula, 290, 432, 435
solution formula, 311
Poisson-bracket relations, 455
Poisson-layer, 432
polynomial approximation, 398
polynomial equation, 19
potential energy, 80
potential energy density, 292
pressure forces, 362
pressure gradient force, 356

propagator layer, 306
pullback, 275
punctured membrane, 274

quadratic form, 289
quadratic potential, 261
quartic oscillator, 188
query, 474

radial reduction, 293
Radon transform, 502
real root, 19
reflection method, 320
refraction, 165
regularization, 7, 48
ReLU, 7
Residual ODEs, 441
Riemannian
    distance, 253
    manifold, 253, 276
    metric tensor, 275
rotated image, 503
rotational invariance, 227
Runge–Kutta method, 145, 192, 349, 400

scalar product, 6
Schrödinger equation, 104, 220, 304
Schrödinger kernel, 306
scores, 470
second-order differential operator, 289
second-order PINN, 336
several observers, 477
simple pendulum, 171, 191
single cusp, 488
sinogram, 505
Snell's law, 165, 170
soft constraint, 267, 323
soft constraints, 10, 15, 176, 207, 381
Softplus, 6, 384
Sommerfeld radiation condition, 318
space–time cylinder, 299
spring-mass system, 130, 147, 448
state equation, 363
stress tensor, 275
Sturm–Liouville problem, 497
sub-Riemannian geodesics, 458
submanifold, 276
Swish, 6
symmetric kernels, 48
symmetries, 15
symmetry groups, 226
symmetry loss, 423
symplectic gradient, 446
symplectic matrix, 484

tangent space, 234
tangent vector, 446
tanh, 6
tautochrone, 493
Taylor-Green vortex, 359
temperature distribution, 300, 433
temperature parameter, 473
temporal acceleration, 356
thermal diffusivity, 380
three-phase resolution, 87
Tikhonov regularization, 49
time-invariant kernels, 48
tomographic reconstruction, 502
total derivative, 361
total energy, 80
trial-solution, 131
trial-solution ansatz, 147
Tricomi equation, 417
Tricomi problem, 428
two-body problem, 147, 453
two-body system, 462
two-phase resolution, 83
two-phase Stefan problem, 286

UAT, 12
unbiased estimator, 8
uniform distribution, 471
upper half-plane, 262

vanishing gradients, 11
variational method, 292
variational problems, 169
vector field, 113, 122
vector potential, 215
velocity field, 331, 353, 354
velocity loss, 124
velocity-Hessian matrix, 457

vibrating plate equation, 307

wave equation, 319, 323, 434
wavefunction, 304
weight matrix, 5
width, 5

X-ray beam, 504